

Modern Data Mining — with — Python

A risk-managed approach to developing and deploying explainable and efficient algorithms using ModelOps

Dushyant Singh Sengar

Vikash Chandra



Modern Data Mining — with — Python

A risk-managed approach to developing and deploying
explainable and efficient algorithms using ModelOps

Dushyant Singh Sengar

Vikash Chandra



Modern Data Mining with Python

*A risk-managed approach to
developing and deploying
explainable and efficient
algorithms using ModelOps*

**Dushyant Singh Sengar
Vikash Chandra**



www.bpbonline.com

First Edition 2024

Copyright © BPB Publications, India

ISBN: 978-93-55519-146

All Rights Reserved. No part of this publication may be reproduced, distributed or transmitted in any form or by any means or stored in a database or retrieval system, without the prior written permission of the publisher with the exception to the program listings which may be entered, stored and executed in a computer system, but they can not be reproduced by the means of publication, photocopy, recording, or by any electronic and mechanical means.

LIMITS OF LIABILITY AND DISCLAIMER OF WARRANTY

The information contained in this book is true to correct and the best of author's and publisher's knowledge. The author has made every effort to ensure the accuracy of these publications, but publisher cannot be held responsible for any loss or damage arising from any information in this book.

All trademarks referred to in the book are acknowledged as properties of their respective owners but BPB Publications cannot guarantee the accuracy of this information.

To View Complete
BPB Publications Catalogue
Scan the QR Code:



www.bpbonline.com

Dedicated to

*My sources of joy: daughters **June** and **Marchie**
and*

*My pillars of strengths: wife **Amrita** and
parents **Usha** and **Narendra Singh Sengar***

- Dushyant Singh Sengar

Foreword

Over the last 20 years, my passion for Data Mining, Machine Learning, and Deep Learning has been a cornerstone of my career. My evolving interest in these fields has enabled me to drive forward the Banking, Payments, and Retail industries by emphasizing the crucial role of AI/ML in business advancement. As a Thought Leader and Coach for over 15 years, I have trained leaders in strategic AI/ML initiatives and fostered a culture of continuous learning.

In this data-centric era, the book, *Modern Data Mining with Python: A risk-managed approach to developing and deploying explainable and efficient algorithms using ModelOps* emerges as a vital resource. It is a comprehensive guide aimed at demystifying data mining complexities, especially focusing on explainability and traceability. Spanning 13 well-crafted chapters, the book is a beacon for understanding and implementing data mining techniques in the banking sector and beyond.

Educational Journey and Ethical Implications

The book starts from the basics of statistics and exploratory data analysis and then ventures into advanced deep learning techniques. It emphasizes ethical machine learning model development, tackling biases, ensuring algorithmic transparency, and adhering to responsible AI principles. This approach is not only about learning techniques but also about becoming a responsible decision-maker in the data-driven business world.

Methodology and Case Studies

Adopting a first-principles teaching method, the book starts with fundamental business questions and then applies data mining concepts, particularly in the regulated banking industry. It includes case studies reflecting best practices in model building and maintenance, applicable across various industries. This approach simplifies complex topics, making them accessible without requiring deep knowledge of algebra, probability, or calculus.

Welcome to a journey of mastering data mining with a conscience.

- Dr. Rajendra Prasad Gangavarapu

About the Authors

- **Dushyant Singh Sengar** is a passionate leader in AI and Risk management with experience building high-performing teams and leading organizations to become data-driven. His extensive 18 years of experience on both sides of the Atlantic spans various roles, including model development, risk assessment, and driving AI product development initiatives. He is a seasoned professional in the banking and consulting industry with experience modernizing retail, credit risk, and marketing platforms leveraging AI/ML techniques on modern-day cloud and MLOps infrastructure.

Dushyant Sengar also brings a wealth of experience in the model risk management (MRM) arena that enables him to provide a holistic and efficient road map for the AI-based product development lifecycle.

He holds an M.S. in data science from Northwestern University, Chicago, and a B.E. in Information technology from M.I.T.S. Gwalior, India. He has also been a freelance data science and analytics trainer during his India-based professional experience and is now a tenured speaker in the AI, Innovation, and Risk management-driven conference scene in the United States. Dushyant thrives on bringing cross-functional knowledge and teams together to ensure alignment between technology, risk, and business goals. He has a proven track record of mentoring students from

various backgrounds and nurturing their data science potential.

When he is not working, you can often find him reading, writing, or exploring new places and cultures. He is passionate about using technology for social good, driven by a mission to spread technology and AI knowledge among enthusiasts for positive change.

- **Vikash Chandra** is a data scientist and software developer having industry experience in executing and implementing projects in the area of predictive analytics and machine learning across multiple business domains. He has experience in handling and modifying large quantities of both structured and unstructured data leveraging SAS, R, Python, and other big data technologies.

He is an alumni of prestigious institutions like Jawaharlal Nehru University, and Shri Ram College of commerce, University of Delhi, India.

About the Reviewers

- ❖ **Dishant Banga** is currently working as a Senior Data Analyst at Bridgetree, USA. He received his master's degree in systems Engineering and Engineering Management with a specialization in Data Analytics/Data Science from the University of North Carolina, Charlotte, in 2018. His interests include developing statistical and machine learning models, artificial intelligence, Machine Learning, and Data Science applications to solve complex business problems. He has participated in various national and international competitions and is recognized for his work.
- ❖ **Vivek Chaudhary**, Founder of Dyota AI, pioneers advancements in artificial intelligence, propelling the company to the forefront of AI technology. With pivotal roles at startups like Vitra.ai, Neurodynamic.ai, and CodeVector. Vivek, an accomplished mentor, has guided 5000+ AI professionals through collaborations with Upgrad, Board Infinity, EC Council, Packt, AlmaBetter, and freelance projects. As a Thought Leader in Computer Vision at Global AI, he authored Amazon's bestselling "Data Investigation-EDA the Right Way." Vivek's 5+ years of expertise in EDA, Machine Learning, and Computer Vision shape the AI education landscape as a mentor for CODERED E-Learning.
- ❖ **Swagata Ashwani** is a seasoned data scientist with a rich background in analytics and big data. Currently serving as the Principal Data Scientist at Boomi, Swagata

plays a crucial role in harnessing the power of data to drive innovation and efficiency. In her role, she plays a crucial role in leading generative AI initiatives for the company. She is also Chapter Lead at SF Women in Data, where she fosters building a rich community for women to celebrate women in varied data roles.

Acknowledgement

Writing a book is a bit like trying to build an aircraft in the hope that it would take readers on a fulfilling journey. I started this endeavor with my professional experiences as the raw materials and a dream as the duct tape. However, undoubtedly, the unconditional support of loved ones, who sacrificed a lot of fun family times, also served as the bedrock of this illuminating experience.

First and foremost, I would like to thank my wife and daughters for being patient with me and continuously encouraging me to write the book. My parents have always taught me to chase the North Star and this life-long learning helped me sail through the twists and turns of book writing odyssey as well.

I am grateful to my co-author, Vikash, for helping me with his Python expertise, which adds immense value to the application focus of this book. I gratefully acknowledge Mr. Dishant Banga, Ms. Swagata Ashwini, and Mr. Vivek Chadhury for their technical scrutiny of this book.

I would also like to extend my heartfelt thanks to Dr. Rajendra Prasad Gangavarapu for his time, wisdom, and the genuine passion he brought to the foreword. His willingness to lend a voice to this project is an honor that I cherish deeply. My sincere gratitude also goes to the team at BPB Publication for being supportive and understanding enough to smoothen and expedite the book publishing process.

Last but not least, to the reader who is holding this book—a big, heartfelt thank you for joining me on this AI journey of

infinite possibilities. Your curiosity and willingness make all the late nights, early mornings, and weekends entirely worth it.

Preface

In an era where data is the new gold, the ability to mine this vast resource for insights and decisions is not just an advantage but a necessity. *Modern Data Mining with Python: A risk-managed approach to developing and deploying explainable and efficient algorithms using ModelOps* is a comprehensive guide tailored to unravel the complexities of data mining, particularly with an emphasis on black-box model explainability and traceability.

Through its 13 meticulously crafted chapters, this book describes responsible AI approaches to improving various AI/ML techniques adoption and business processes by demystifying best practices in model risk management and operations. Author Dushyant Sengar created this guide for budding data scientists and experienced business leaders looking to improve real-world AI/ML system outcomes in the banking sector and beyond.

This book establishes a solid foundation of data mining, starting with exploratory data analysis and inferential statistics and progressing through advanced techniques like XGBoost, and deep learning. Each chapter, with a focus on a specific data mining technique and complete with its applications and nuances, serves as a building block of a comprehensive data mining knowledge base. You will explore industry best practices for making fair and efficient decisions while learning technical approaches for responsible AI across model validation, explainability, bias

management, and AI-based product development using MLOps.

In a world where business decisions have far-reaching consequences on consumers' lives, the ethical deployment of data mining techniques is paramount. As of December 2023, there have been numerous public reports of model mismanagement and algorithmic discrimination, leading to unexplainable outcomes in highly regulated industries. This book addresses this need head-on, emphasizing the importance of developing machine learning models that are efficient, ethical, and explainable. It guides you through the intricacies of mitigating biases, ensuring algorithmic transparency, and upholding the principles of responsible AI. By doing so, it aims to foster an environment where the benefits of data-driven decision-making are equitably distributed.

The unique approach of this book lies in its method of teaching. It adopts a first-principles approach, starting with fundamental business questions. Subsequently, these questions are broken down into smaller manageable chunks and framed as data, model, and optimization problems. The case studies provided offer a window into the best practices and critical considerations in model building and maintenance specifically in the highly regulated banking industry, and further extend their relevance across other industries. The beauty of this approach is that it simplifies complex topics without requiring a deep background in algebra, probability, or calculus.

As you turn the pages of this book, you are not just learning data mining techniques; you are embarking on a journey to become a responsible and effective decision-maker in the data-driven world. This book is not just a learning tool; it is a commitment to ethical and fair data practices that will shape the future of business and technology. Welcome to

the journey of mastering data mining with a conscience. The chapter-wise details are listed below.

Chapter 1: Understanding Data Mining in a Nutshell - introduces data mining, explaining the data mining growth journey, biases, and risks while discussing how humans have been pushing the boundaries over the decades to train machines using approaches similar to various human learning models. This chapter provides a foundation for understanding the development of modern-day data mining models that touch various aspects of human life, from agriculture to space exploration. It discusses various risks and challenges associated with machines trying to imitate human learning and making decisions before wrapping up with potential solutions to these modern-day data mining challenges.

Chapter 2: Basic Statistics and Exploratory Data Analysis - discusses the art of exploring datasets to gain an understanding of their features' strengths and discrepancies to prove real-world hypotheses. Before diving into inferential statistics, the chapter teaches the use of non-graphical and graphical data exploration methods using various descriptive techniques. The chapter leverages Python for data exploration and inferential statistics and will continue to do so for all case study implementations throughout the book.

Chapter 3: Digging into Linear Regression - covers the depth of the regression technique with an explanation of fitting a line to the data to the advanced twists and turns of regularization, including the need and complexities of Lasso and Ridge techniques. This statistical deep dive will be supplemented by implementation details on MLflow, a model reproducibility and experimentation tracking tool to manage model-development related risks.

Chapter 4: Exploring Logistic Regression - covers logistic regression with the intricacies of its data requirements, the art of estimating probabilities, and the movements of loss functions. This will include a discussion on the challenges and assumptions while decoding the results and mastering the interpretation of logistic regression models. This chapter will also lay the foundation for model explainability and interpretability, which is a cornerstone of modern-day data mining.

Chapter 5: Decision Trees with Bagging and Boosting - uncovers decision trees – a versatile, non-parametric method that is foundational for powerful algorithms like Random Forest and Gradient Boosting. This chapter explains the ensemble modeling concepts of bagging and boosting by navigating through four case studies using MLFlow and SHAP libraries. These Python tools emphasize the importance of model reproducibility and interpretability in machine learning model development.

Chapter 6: Support Vector Machines and K-Nearest Neighbors - expands the discussion from chapters 4 and 5 on supervised methods by introducing unique concepts of support and look-alike neighbors. The chapter builds the conceptual foundations theoretically, following it up with a detailed case study that highlights the effectiveness of SVM and KNN in solving a specific business problem in the financial services industry.

Chapter 7: Putting Dimensionality Reduction into Action - explores a crucial data preparation step in a model development exercise. The dimensionality reduction techniques allow practitioners to distill complexity into simplicity and uncover the essence of the data. This chapter will discuss best practices for transforming high-dimensional data into lower-dimensional representations to gain a

deeper understanding, facilitate visualization, and enhance subsequent modeling and analysis tasks using a case study.

Chapter 8: Beginning with Unsupervised Models -

explains what unsupervised models are and how they differ from supervised ones. It covers three widely used methodologies, including DBSCAN, that fit different use cases well. This chapter aims to provide a holistic approach to developing real-world unsupervised solutions using a case study approach in the financial services industry.

Chapter 9: Structured Data Classification using Artificial Neural Networks -

provides a profound explanation of the Artificial Neural Network (ANN). It will also discuss the practical examples based on a real-life scenario and explain how to apply this modeling technique to solve a medium-complexity problem.

Chapter 10: Language Modeling with Recurrent Neural Networks -

describes the solid principles of Recurrent Neural Networks (RNN) using real-life examples. It will provide a comparison between popular practices behind the principles of text data handling, preparation and RNN modeling techniques suitable for language processing and understanding.

Chapter 11: Image Processing with Convolutional Neural Networks -

focuses on bridging the gap between raw visual information and actionable insights. Convolutional Neural Networks (CNNs), are revolutionizing industries by tackling image processing challenges such as extracting hierarchical features for recognizing patterns, shapes, and objects within images, regardless of their scale, orientation, or position. This chapter will discuss CNNs capabilities with their high-dimensional data requirements and architectural design to ultimately present a case study to extract meaningful business information from PDF documents.

Chapter 12: Understanding Model Risk Management for Data Mining Models

- explains the importance of understanding, identifying, and handling the risks associated with the use of data mining models during development and in a production environment, making inferences on previously unseen data. This chapter will shed light on various guidelines, regulations, and technology-driven approaches to risk mitigation. We will conclude with an industry-focused case study to make clear the role of a model developer in risk management efforts within an organization.

Chapter 13: Adopting ModelOps to Manage Model Risk

- brings together the knowledge from the previous 12 chapters on model development, operational efficiency, and risk management. It builds a foundation for developing a responsible AI solution by first describing the intricacies of the financial services regulatory landscape, specifically in the fair credit lending space. It then goes on to explore the significance of MRM as it relates to fair lending. Eventually, the readers will gain from a compelling case study: the development of a Fair Lending risk assessment web application leveraging modern ModelOps tools.

Code Bundle and Coloured Images

Please follow the link to download the **Code Bundle** and the **Coloured Images** of the book:

<https://rebrand.ly/9b9424>

The code bundle for the book is also hosted on GitHub at <https://github.com/bpbpublications/Modern-Data-Mining-with-Python>. In case there's an update to the code, it will be updated on the existing GitHub repository.

We have code bundles from our rich catalogue of books and videos available at <https://github.com/bpbpublications>. Check them out!

Errata

We take immense pride in our work at BPB Publications and follow best practices to ensure the accuracy of our content to provide with an indulging reading experience to our subscribers. Our readers are our mirrors, and we use their inputs to reflect and improve upon human errors, if any, that may have occurred during the publishing processes involved. To let us maintain the quality and help us reach out to any readers who might be having difficulties due to any unforeseen errors, please write to us at :

errata@bpbonline.com

Your support, suggestions and feedbacks are highly appreciated by the BPB Publications' Family.

Did you know that BPB offers eBook versions of every book published, with PDF and ePub files available? You can upgrade to the eBook version at www.bpbonline.com and as a print book customer, you are entitled to a discount on the eBook copy. Get in touch with us at :

business@bpbonline.com for more details.

At www.bpbonline.com, you can also read a collection of free technical articles, sign up for a range of free newsletters, and receive exclusive discounts and offers on BPB books and eBooks.

Piracy

If you come across any illegal copies of our works in any form on the internet, we would be grateful if you would provide us with the location address or website name. Please contact us at business@bpbonline.com with a link to the material.

If you are interested in becoming an author

If there is a topic that you have expertise in, and you are interested in either writing or contributing to a book, please visit www.bpbonline.com. We have worked with thousands of developers and tech professionals, just like you, to help them share their insights with the global tech community. You can make a general application, apply for a specific hot topic that we are recruiting an author for, or submit your own idea.

Reviews

Please leave a review. Once you have read and used this book, why not leave a review on the site that you purchased it from? Potential readers can then see and use your unbiased opinion to make purchase decisions. We at BPB can understand what you think about our products, and our authors can see your feedback on their book. Thank you!

For more information about BPB, please visit www.bpbonline.com.

Join our book's Discord space

Join the book's Discord Workspace for Latest updates, Offers, Tech happenings around the world, New Release and Sessions with the Authors:

<https://discord.bpbonline.com>



Table of Contents

1. Understanding Data Mining in a Nutshell

Introduction

Structure

Objectives

What defines modern data mining

The lifecycle: Data to insights consumption

Understanding pattern recognition

Significance of the human learning process

The human learning process and mental models

Data: The key ingredient for meaningful patterns and relationships

How machines leverage data to build models

Machine learning process

Two dominant strategies: Classification and regression

Biases and learning shortfalls

Measuring learning accuracy and balancing trade-offs

Can data size and sample impact learning

How do humans benefit from data and learning

Modern-day data mining challenges and possible remediation

Conclusion

Points to remember

2. Basic Statistics and Exploratory Data Analysis

Introduction

Structure

Objectives

Setting up Python 3.x

Data mining and statistics

Statistics: Foundation, key terms, needs, and types

Descriptive statistics

Graphical and non-graphical exploratory data analysis

Non-graphical and graphical representation of univariate data

Non-graphical representation of multivariate data

Graphical representation of multivariate data

Probability theory

Probability distribution

Inferential statistics

Hypothesis testing with commonly used statistical tests

Introduction to Time Series Data

Exploratory data analysis: HMDA case study

Conclusion

Points to remember

3. Digging into Linear Regression

Introduction

Structure

Objectives

Linear regression

Background

- Under the hood*
 - Challenges and assumptions including multi-collinearity*
 - Detailed EDA*
 - Dataset description*
 - Missing value treatment*
 - Outlier analysis*
 - Correlation*
 - Checking on the assumptions of linear regression*
 - Feature selection*
 - Regression execution and results*
 - Regression result interpretation*
- Optimization algorithm
 - Gradient descent*
- Regularization
 - Lasso regression*
 - Ridge regression*
 - Elastic-Net regression*
- MLflow introduction: Need and implementation
 - MLflow experiment tracking*
- Case study
- Conclusion
- Points to remember

4. Exploring Logistic Regression

- Introduction
- Structure
- Objectives
- Logistic regression
- Background

Under the hood

Data

Estimating probabilities

Loss function

Challenges and assumptions

Logistic regression result and interpretation

Model interpretability and explainability

Performance metrics

Model generalization

K-fold cross-validation

Ensemble learning

Model lifecycle processes

Model development process

Case study: Loan repayment likelihood prediction

Conclusion

Points to remember

5. Decision Trees with Bagging and Boosting

Introduction

Structure

Objectives

Decision trees

Background

Under the hood

Data

Model

Loss function

Challenges and assumptions

Decision tree result and interpretation

Ensembling: Bagging, boosting, and stacking

Random forest

Gradient boosting

Ensembling using the stacking method

Conclusion

Points to remember

6. Support Vector Machines and K-Nearest Neighbors

Introduction

Structure

Objectives

Classification algorithms with a twist

Background

Under the hood

Data

Model

Loss function: Achieving optimal algorithmic results

Challenges and assumptions

Case study: Predicting customer propensity to subscribe to a term deposit

Conclusion

Points to remember

7. Putting Dimensionality Reduction into Action

Introduction

Structure

Objectives

Dimensionality reduction

Background

Under the dimensionality reduction hood

Data

Model: Reducing dimensions and variance

Principal component analysis

Linear discriminant analysis

t-distributed Stochastic Neighbor Embedding

Loss: Measuring Variance Reduction

Challenges and assumptions

Case study: Predicting loan repayment propensity using
logistic regression, PCA, and LDA

PCA parameters and interpretation

LDA parameters and interpretation

Logistic regression

Conclusion

Further reading

Points to remember

8. Beginning with Unsupervised Models

Introduction

Structure

Objectives

Unsupervised learning

Background

Unsupervised learning techniques

Data

*Model: Building meaningful clusters and profiling
them*

K-means clustering

Density-based spatial clustering of applications with noise

Hierarchical clustering

Loss: Efficiently achieving the optimal number of clusters

Challenges and assumptions

Case study: Bank customer portfolio segmentation

Advanced unsupervised learning: A primer

Conclusion

Points to remember

9. Structured Data Classification using Artificial Neural Networks

Introduction

Structure

Objectives

Artificial neural network

Background

Under the hood of neural networks

Data

Model

Loss function: Achieving optimal results

Back-propagation and regularization

Challenges and assumptions

Case study: Explainable and Interpretable ANN Model

Interpretable and explainable AI using SHAP and PiML

Conclusion

Points to remember

10. Language Modeling with Recurrent Neural Networks

Introduction

Structure

Objectives

Language modeling

Background

Under the hood of language modeling

Data: From spoken languages to modeling datasets

Model: The language with context

Recurrent neural network

Long short term memory

Loss: Quest for the best model

Challenges and assumptions related to text data and model

Case study: Customer complaint classification explained with LIME

Rise of transformers: A primer on BERT and GPT

Conclusion

Further reading

Points to remember

11. Image Processing with Convolutional Neural Networks

Introduction

Structure

Objectives

Deep learning for computer vision tasks

Background

Under the hood of CNN models

Data

Model

Loss: How to achieve optimal results

Challenges and assumptions

The race for the best model and transfer learning: A primer

Case study: PDF document parser

Conclusion

Further reading

Points to remember

12. Understanding Model Risk Management for Data Mining Models

Introduction

Structure

Objectives

Data mining challenges and risks

Why do model risks occur

Introduction to Model Risk Management

Key regulatory frameworks

Pillars of Model Risk Management

Introduction to Model Operations

ModelOps: Product first vs. model first mindset

How ModelOps facilitates MRM

Case study: Regulatory requirement fulfillment using MRM and ModelOps

Conclusion

Points to remember

13. Adopting ModelOps to Manage Model Risk

Introduction

Structure

Objectives

Model risk management for fair banking

Background

Case study: Fair lending model lifecycle implementation

- concept to inference

Fair lending model lifecycle

Data

Model Operations tools primer

Architecting the model lifecycle using ModelOps

Fair Lending Risk Assessment: The application

Challenges and assumptions

Future of AI and its practitioners

Conclusion

Further reading

Points to remember

Index

CHAPTER 1

Understanding Data Mining in a Nutshell

Introduction

Human ingenuity, innovation, and the quest for knowledge have kept us moving from the ancient trading days of the Silk Route in 130 B.C. to the modern days of global, electronic, algorithmic trading and banking platforms. Notably, the growth trajectory of statistics, followed by data mining, and related algorithms has had a major role in the advancements of various financial services disciplines over the centuries. This potentially stems from the fact that financial services and data mining are both driven by the same human principles of observation, critical thinking, problem solving and community sharing. Today, these two disciplines have evolved to arrive at the promising juncture of the **financial technology (FinTech)** revolution, but not without multiplying the risks of the two components together.

Specifically, the recent rapid advancements in data mining technologies have come with challenges and risks that warrant the development of ever-new regulations for industries such as financial services. This ensures the

financial safety of globally connected trading markets and other customers of FinTech services in various local markets.

In this regard, this chapter will explore the various aspects of the data mining process, and trace its similarities with the human learning process, to track its growth journey, biases, and risks while discussing how humans have been pushing the boundaries to train machines using approaches similar to their learning model. This has led to the development of modern-day data mining models touching various aspects of human life ranging from agriculture to financial services to space exploration. We will discuss various risks and challenges associated with machines trying to imitate human learning and making decisions before wrapping them up with potential solutions to these modern-day data mining challenges.

Structure

This chapter will cover the following topics:

- What defines modern data mining
- The lifecycle: Data to insights consumption
- Understanding pattern recognition
 - Significance of the human learning process
 - The human learning process and mental models
 - Data: The key ingredient for meaningful patterns and relationships
- How machines leverage data to build models
 - Machine learning process
 - Two dominant strategies: classification and regression
 - Biases and learning shortfalls

- Measuring learning accuracy and balancing trade-offs
- Can data size and sample impact learning
- How do humans benefit from data and learning
- Modern-data data mining challenges, risks, and remediations frameworks

Objectives

By the end of this chapter, you will be able to describe how various types of learning needs drive the learning approaches supported by the available data. You will be able to identify the right learning technique in a given scenario and outline the numerous pitfalls associated with the modern-day data mining landscape.

What defines modern data mining

We are drowning in information but starved for knowledge

- John Naisbitt, 1982 Megatrends

Data mining originated in the 1970s, flourished in the 1980s, and continues strong today. This is thanks to many new data formats and storage systems that the world is getting exposed to. In simple terms, data mining can be defined as the extraction or mining of knowledge from large and varied amounts of data. This typically involves characterization, discrimination, association or correlation analysis, classification, prediction, clustering, outlier analysis, or evolution analysis. Myriad data mining tools perform data analysis and may uncover important data patterns, contributing greatly to business strategies, knowledge bases, and scientific and medical research. However, the growing gap between newer data and essential knowledge recently

calls for innovative data mining tools and techniques to turn the new-age data wastelands into orchards of knowledge.

Historically, many manual processes that relied on sophisticated database querying, statistical techniques, and human intervention were leveraged to detect patterns and relationships in cleaned and stored databases. One such industry was retail which transitioned from manual processes to sophisticated **customer relationship management (CRM)** systems that involved the use of advanced database querying, statistical techniques, and human expertise to extract valuable patterns and relationships from cleaned and stored databases. The main types of such relationships are classes (partitioned into predefined groups), clusters (partitioned into logically related groups), associations, behavior patterns, and trends. This transformation significantly enhanced the ability of retail businesses to understand and respond to customer behavior in a more data-driven and effective manner.

However, this had to change with the advent of big data which required the process of knowledge extraction and learning to be automated and more intelligent on its own. In recent years, we have also seen the rise of machine learning and **artificial intelligence (AI)** techniques that have redefined the data mining landscape by enabling computers to learn a bit as humans do. These ML algorithms can imitate human learning by learning from experiences and grow better in decision-making when exposed to more diverse datasets.

Modern data mining systems hence involve computers that are supplied with high-quality data, to discover novel data patterns driven by sophisticated machine learning algorithms for knowledge discovery on a fair and continuous basis.

The lifecycle: Data to insights consumption

Over the decades, different process frameworks have been attached to the field of data mining to derive the best return on investment of time and effort. These framework proposals have been driven primarily by the data quality, data size, business needs, and the level of algorithmic sophistication prevalent in the era. In today's world, the most consistent framework that is used across a range of organizations across the globe can be described in the following six steps:

1. **Defining the problem:** In a bank, this could mean improving the credit risk assessment process to minimize default rates and enhance lending decisions. This is the most critical step that needs to be done right.
2. **Data identification and pre-processing:** This would mean collecting historical data on loan applicants, including their financial information, credit history, employment status, and other relevant variables, and cleaning up all this information to resolve inconsistencies and anomalies.
3. **Defining the data mining requirement:** Such as characterization, discrimination, association or correlation analysis, classification, prediction, clustering, anomaly or outlier analysis to group loan applicants into *low risk* and *high risk*.
4. **Domain knowledge and measurement indicators for pattern evaluation:** This helps in developing and maintaining a healthy credit risk program over a period based on business and industry guidelines.
5. **Data visualization for easy consumption:** These visualizations help non-technical stakeholders understand the model's performance and interpret results for better adoption of algorithm results.

6. **Deployment and decision monitoring:** Lastly, the integration of the predictive model into the bank's loan approval system and automation of the credit assessment process for continuous monitoring of any potential deviations. This allows for regular updates of the model using new data to adapt to changing patterns.

These six steps correlate heavily with the five steps of the **Knowledge Discovery in Databases (KDD)** that was coined by *Gregory Piatetsky-Shapiro* in 1989. Another industry standard framework known as the **Cross-Industry Standard Process for Data Mining (CRISP-DM)** is perfectly consistent with the current six steps of the data mining process. It is an iterative process; many tasks backtrack to previous tasks and repeat certain actions to bring more clarity. These six major phases of the CRISP-DM process are idealized sequences of activities, but they relate well with the six-step industry-standard framework discussed above. [Figure 1.1](#) illustrates the CRISP-DM process steps (colored boxes) with the industry-standard steps (in text) mapped alongside it:

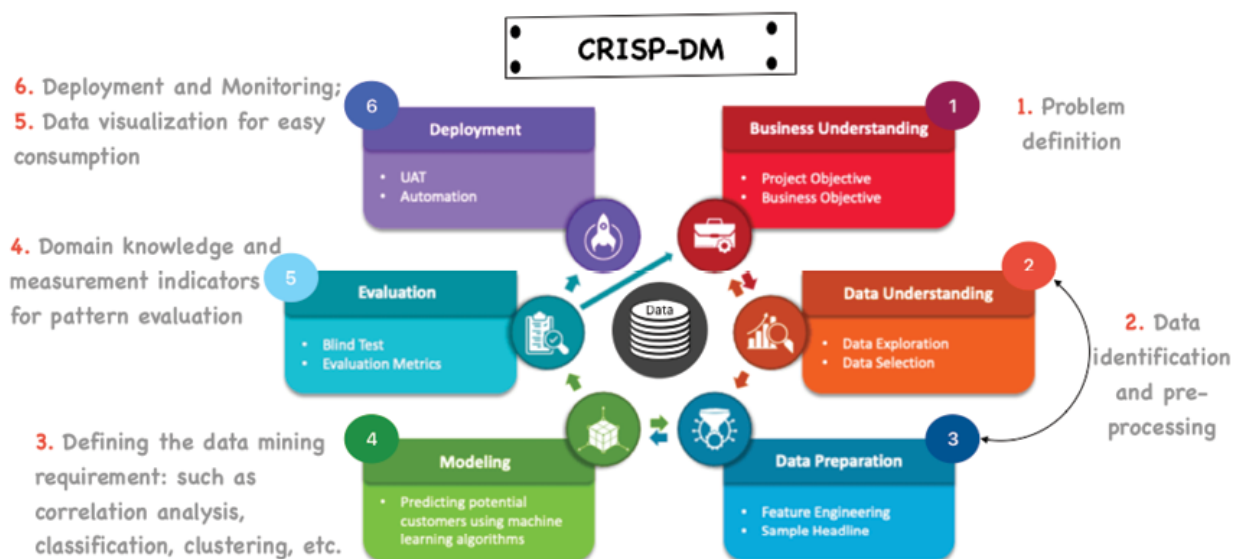


Figure 1.1: CRISP-DM mapped to six standard industry practices

In summary, the outcome of a data mining process is either an exploration exercise to understand the data patterns or build predictive models also known as mathematical equations based on classification or regression techniques. However, in today's world, it is not sufficient to explore the data and build a model, but it is equally important to derive a long-term return on investment by evaluating and deploying the model. Steps 5 and 6 need careful consideration to ward off any potential risks from continuous usage of the developed models. Hence model monitoring becomes a significant critical step in the process. Before moving ahead with data mining, let us understand more about models and how they learn from data.

Understanding pattern recognition

Let us start by understanding human mental models, which are made up of interrelated memories, conceptual knowledge, and causal beliefs that create an understanding of how some things work in the real world and form expectations about future events. For example, many individuals in the world have a mental model of a bank's operation and money-making process.

Primarily, banks take deposits from individuals and compensate depositors with certain interest rates and then lend that money to other borrowers at a higher interest rate and profit from the interest rate spread. The interest rates vary depending on many external factors and could be unique to individuals. Humans use this particular set of conceptual knowledge, expectations, and causal beliefs to form a pattern or model around the bank's profit-making. This is formed primarily by human historical personal experiences and might also be formed by the transfer of theoretical knowledge through literature and experimentation.

Similarly, anything in the physical world that can be described in terms of mathematical expression can be termed a mathematical representation of reality also known as a mathematical model or equation. This model is an outcome of a learning algorithm and comprises of underlying causal effects of reality, individual importance of these effects, and often any apriori a.k.a. previously known knowledge related to these effects. This model composition is based on the historical data that the algorithm has consumed. Learning theory has an analog in almost every problem that has ever been studied in statistical science, making the learning theory universally applicable. Humans have always found it fascinating to take the important general results from the framework of learning theory to a statistical formulation.

Significance of the human learning process

Humans learn using one of the five broad methods described below to form small-scale mental constructs also known as models. These are essentially apparent representations of external reality. These mental models are then used to anticipate events, reason, and form explanations. One such process is cognitive mapping through which a person acquires, stores, codes, and recalls information about the world. For example, if you work with your friend in the same office, reside in the same apartment complex, and give your friend a ride in your car to the office daily, both of you would have two different cognitive maps of the route. Both of you sketch maps of the same route differently because you concentrate more on the road while the friend enjoys the ride.

Abel et.al. (1998) conceptualized this cognitive map as a spatial mental model in acknowledging the theory of cognitive mapping. The origination of cognitive mapping could be traced back to spatial cognition studies, closely

associated with the work of psychologist *Edward C. Tolman*, who conducted groundbreaking research on spatial learning and cognitive maps in the early to mid-20th century.

Tolman's experiments, particularly those involving rats navigating mazes, contributed to the understanding of how animals (and by extension, humans) create mental representations or maps of their spatial environment. Tolman proposed that individuals, including animals, acquire knowledge about the spatial layout of their surroundings and form cognitive maps that help them navigate and make decisions.

In the process of cognitive mapping described above, individuals have different reasoning and predictive capacities when it comes to recalling or describing the route to a third person.

The process of cognitive mapping is analogous to the approach that machines take to learn from past data using various algorithms (also known as *learning techniques*) to draw cause-effect conclusions and build models.

Computational scientists use the cognitive map to represent cause-effect relationships of an event. In the above office-ride example, it is apparent that you driving the car leads you to process more visual data, and hence you have a better cognitive map of the route than your friend. Machines are no different in terms of their learning processes:

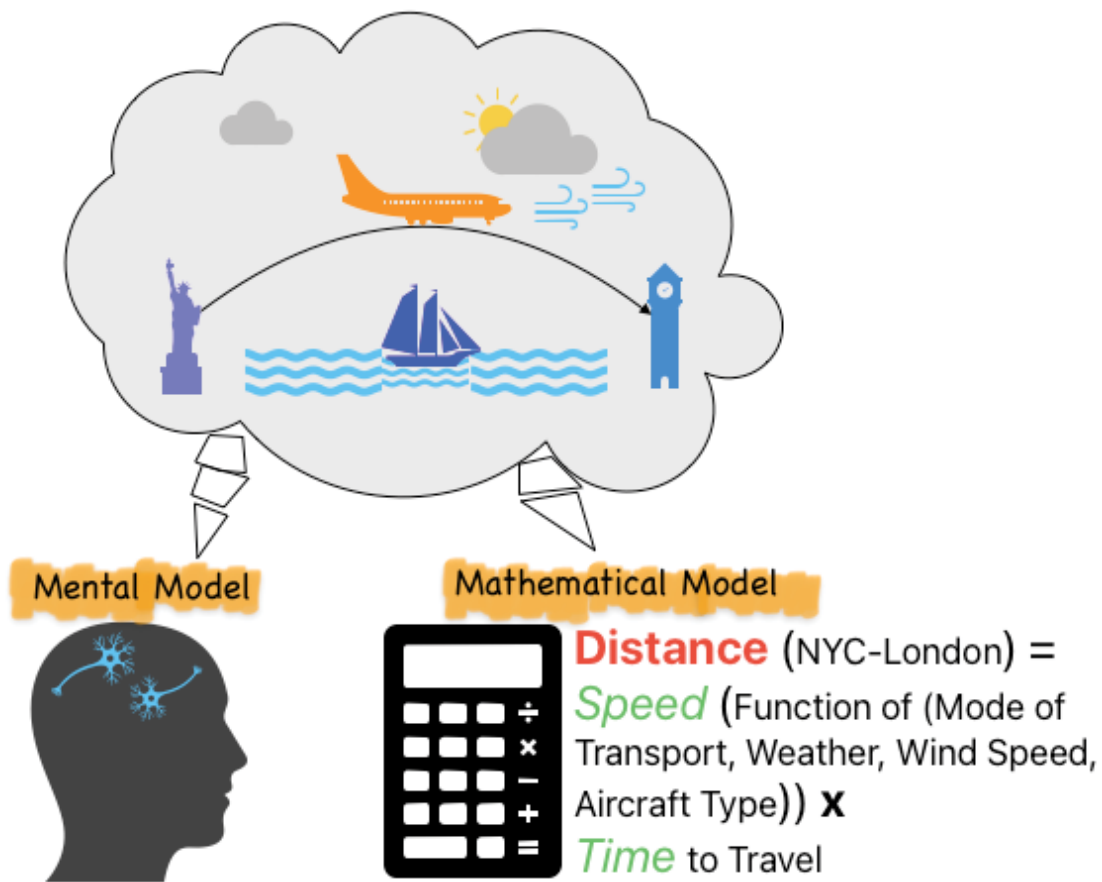


Figure 1.2: Human vs Mental Model Formulation

In [Figure 1.2](#), the human brain imagines and processes all the information about the distance between New York City and London by considering all the factors that might affect travel time and speed by leveraging neuron interactions. A mathematical model considers the same factors and codifies their interaction.

The human learning process and mental models

Before we understand how modern data mining techniques such as machine learning learn patterns from a large amount of data in an automated way, let us revisit the variety of ways that humans employ to learn. Some of the most prominent learning theories that have influenced our day-to-day lives are listed below:

- **Behaviorism:** This postulate learning as starting with a blank page and driven by the negative or positive response of the environment, it leads to change in human behavior.
- **Cognitivism:** It involves an active mental process that incorporates prior knowledge and assumptions to arrive at an understanding. Learning is defined as a new understanding and not a behavioral change.
- **Constructivism:** This, again is an active mental process except that it happens while interacting with the current environment. Humans rely on situational experiences to make subjective conclusions.
- **Humanistic:** This theory defines role models as the best teachers and people who need to be able to fulfill their ambitions through observations and accumulated experiences to display new learnings.
- **Experiential:** This sees learning as a four-step process that starts with concrete experiences leading to reflective observation followed by abstract conceptualism. The last step involves active experimentation, defined by testing further to gain new experiences.

In summary, human learning can be broadly classified in a two-dimensional framework of the need for a prior mental model and a prompt-driven approach. Humans can utilize a prior mental model and may or may not need a hint or direction from new situations to arrive at an understanding or a new model. Alternatively, they can start as a blank slate, meaning without prior assumptions of the new experience and form new mental models while needing a hint or not. A child learning a new language can be defined as having no prior assumption and no fixed direction category. On the other hand, an adult trying to learn to use chopsticks can be categorized as experiential wherein the person has concrete

prior assumptions about the use of chopsticks and new hints lead to the gain of new experiences.

Let us keep this two-dimensional human learning in mind for now. We will use this framework to describe the machine's various learning approaches ahead in the section *The machine learning process*.

Data: The key ingredient for meaningful patterns and relationships

Humans learn from experiences and similarly, machines are dependent on data for learning. Over the last few decades, businesses and academics have been trying to keep pace with new data formats and available storage options to support the decision-making process. In the last decade, the increase in volume, velocity, and variety of data has led to the development of advanced data mining techniques and algorithms that can effectively handle the complexity and heterogeneity of modern data sources. Hence, the need to process these increasingly larger and varied data sets led to the evolution of traditional data mining into modern data mining over the last decade.

For example, in the early 2000s, most data generated and consumed by businesses was structured and stored in relational databases. The growth of unstructured data, such as text, images, and videos, has driven the development of natural language processing, computer vision, and multimedia data mining techniques. Similarly, the rise of big data has led to the growth of scalable and distributed data mining methods, such as *Hadoop* and *Spark*, that can process large datasets found commonly in E-commerce businesses efficiently. These developments have enabled more powerful and effective data-driven decision-making and helped businesses such as Amazon to derive insights,

such as recommending products based on a customer's fast-changing browsing history.

The world has moved on from traditional relational data warehouses to more sophisticated storage mechanisms for efficient data mining. The evolution of traditional to modern data mining is about moving from tasks like clustering similar items or predicting values based on linear regressions to modern data-hungry techniques such as neural networks and complex ensemble methods. For instance, Google Photos not only groups photos based on locations but also recognizes and tags faces using neural networks, a hallmark of modern data mining. A survey of various data sources that support modern data mining initiatives is described below:

- **Cloud-based data storage:** Cloud-based data storage, such as *Amazon S3* and *Azure data lake storage gen2*, provides scalable and cost-effective solutions for storing and processing large amounts of structured and unstructured data. These services offer advanced data management and analysis tools, making them a popular choice for modern data types:
- **NoSQL databases:** These are designed to handle large amounts of unstructured and semi-structured data. They are highly scalable, and flexible and provide real-time access to data. Examples include MongoDB and Cassandra. In most **Content Management Systems (CMS)**, content items like articles, images, and user profiles may have varying attributes. MongoDB allows for dynamic schema design, making it easier to handle diverse content types without a predefined structure.
- **Columnar databases:** These are designed for high-performance data storage and analysis and are optimized for data warehousing and big data analytics needs. Apache Cassandra and Amazon Redshift are some popular names in this space. For instance,

Cassandra, a distributed and columnar database, is excellent for storing time-series data like stock prices, sensor readings, or IoT device data.

- **Graph databases:** Social media data analysis involves techniques such as network analysis to extract valuable insights. Graph databases store interconnected entities in nodes and edges format, allowing for complex relationships between data elements to be represented and analyzed. Some popular graph databases are Neo4j, Tiger Graph, and Amazon Neptune. Neo4j's graph querying (Cypher) capabilities make it efficient for traversing and analyzing complex relationships, providing insights into network structures like social media platforms.
- **Key-value databases:** Key-value databases, such as Amazon DynamoDB, are optimized for fast access to data. These have been traditionally referred to as **Online Transactions Processing (OLTP)** databases, which are designed for high-speed and low-latency data retrieval, making them well-suited for web applications and real-time data storage. For instance, Redis, another key-value store with an in-memory nature and fast key-based retrieval, is commonly used for caching frequently accessed data and managing session storage in web applications.
- **Object-relational databases:** Object-relational databases, such as PostgreSQL and MySQL, combine the capabilities of relational databases with the flexibility of object-oriented programming, making them well-suited for modern data types. For instance, PostgreSQL's support for complex queries and relational integrity makes it suitable for scenarios where data consistency and relational structures are critical.

While such databases or information repositories require sophisticated facilities to efficiently store, retrieve, and update large amounts of complex data, they also provide opportunities for efficient modern-day data mining. These data formats storages are adept in handling a multitude of data types such as structured, semi-structured, unstructured, time-related data (such as weather or stock exchange data), stream data (such as video surveillance and sensor data, where data flow in and out like streams), web click streams, and spatial data (such as maps).

How machines leverage data to build models

Now that we understand that humans apply a range of learning approaches and build mental models based on experiences, causal relationships, and theoretical knowledge, let us deep dive into the model-building process of a machine driven by the data types and storages discussed in the previous section. The premise is of course that humans have taught machines to imitate their learning behavior.

Machine learning algorithms, similar to humans, are learning to build a target function also known as a model that relates a set of inputs to an output. This could be as simple as deciding to approve or deny a loan application based on the attributes of the borrower. During this decision-making process, models try to assign different weights to an applicant's attribute, just like humans would do. Let the attributes or the inputs be described as $X_i, i=\{1,2,3....n\}$. These inputs can be mapped to experiences, causal relationships, and theoretical knowledge from human learning traits.

Output can be described as a Y and the target statistical function (f) is analogous to the human mental models.

The general form of a statistical model that uses various mathematical concepts, including calculus, linear algebra,

optimization, and probability theory to define its form is:

$$Y = f(\theta, x) + \varepsilon$$

ε represents the error term, which is a random variable that captures the residual variance not explained by the model. In other words, this error tells us that there are not enough inputs to sufficiently describe the outcome Y using x . In the case of a loan approval decision, it would mean low confidence in the final decision. Alternatively, for a house price prediction, it would mean a high error in actual vs. predicted price.

Let us revisit human decision-making using an example of a simple classification (**IF-THEN**) rules-based model. If the goal is to associate a set of inputs to determine if a borrower deserves a loan or not, a decision tree approach can be used. The flowchart of the tree structure approach is a model, where each node denotes an evaluation of a loan criterion, each tree branch represents an outcome of the criterion, and tree leaves represent the eventual class or the final decision of the model. Decision trees can easily be converted to classification rules for future prediction on new and unseen data.

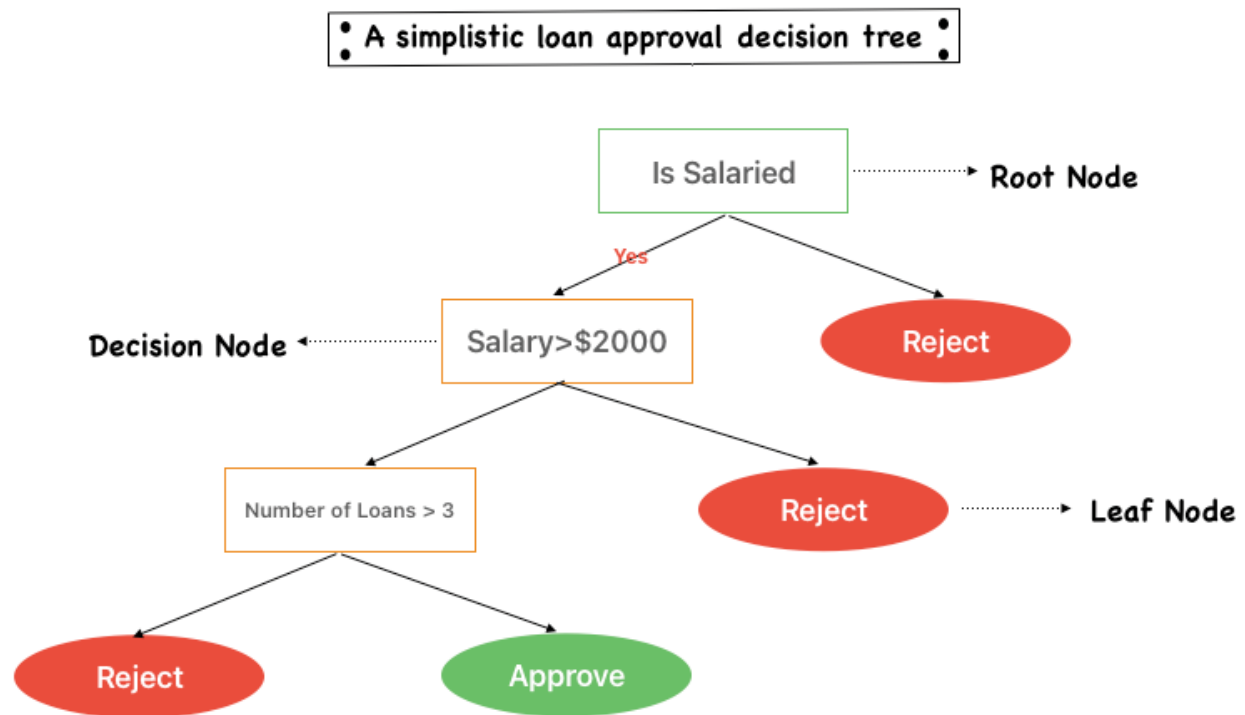


Figure 1.3: Loan approval decision tree

The decision tree algorithm visualized in [Figure 1.3](#) learns a pattern based on the dataset it has seen under certain constraints. Assuming this data pattern holds in the future, it can be utilized for making a decision on loan underwriting going forward. At times this learning will have gaps leading to decision errors. These learning gaps can occur due to limited or biased data and failure to estimate the proper form of function (f).

Machine learning process

Every learning algorithm aims to learn from available historical data to make accurate predictions on unseen data. Much like humans, they can do this by assuming a prior form of the target function (f) and then learning based on complete trial-and-error or some guided cues, which are essentially directions that serve as a signal for the model to begin learning from data in a specific way. Alternatively, they can make no assumptions about the prior form of the

function (f) and learn completely based on guided cues or trial-and-error methods.

In this regard, methods that assume a prior functional form and attempt to fit the learning from the data into that form are called parametric learning methods. In statistical terms, this would mean a parametric equation employs an independent variable called a **parameter** (often denoted by x) in which dependent variables Y are defined as continuous functions of the parameter. And understandably, the methods that do not assume prior functional form are called **non-parametric**. The biggest disadvantage of parametric methods is that the assumptions we make may not always be true. For instance, you may assume that the form of the function is linear when it is not. On the other hand, since non-parametric methods do not make any underlying assumptions with respect to the form, such methods are capable of estimating the unknown function (f) that could be of any form. These two methods can be together termed as *relationship-driven*, because regardless of their prior assumption status, they eventually form a relationship structure between the output Y and input X .

Another class of methods that are defined by a need for guided cues for learning are called supervised approaches and the ones that do not need cues are called unsupervised approaches. These two can be termed “output-driven” meaning driven by the availability of the dependent Y .

The majority of machine learning approaches can be classified based on their dependent variable requirements and prior functional form assumptions. For instance, the decision tree approach that classifies loan borrowers as risky or non-risky can be categorized as non-parametric but supervised. The decision tree does not assume any prior form of the function (f) but evolves based on the data it sees and needs to have a cue or a label differentiating a risky customer from a non-risky one.

In [Figure 1.4](#), trekking can be categorized as a *parametric-unsupervised* activity because one needs to follow a set of rules (functional form) to stay out of trouble, and remain on track, to arrive at a meaningful and worthwhile destination. On the other hand, swimming can be done in any style but the target destination has to be the shore, hence it can be categorized as a *nonparametric-supervised* activity.

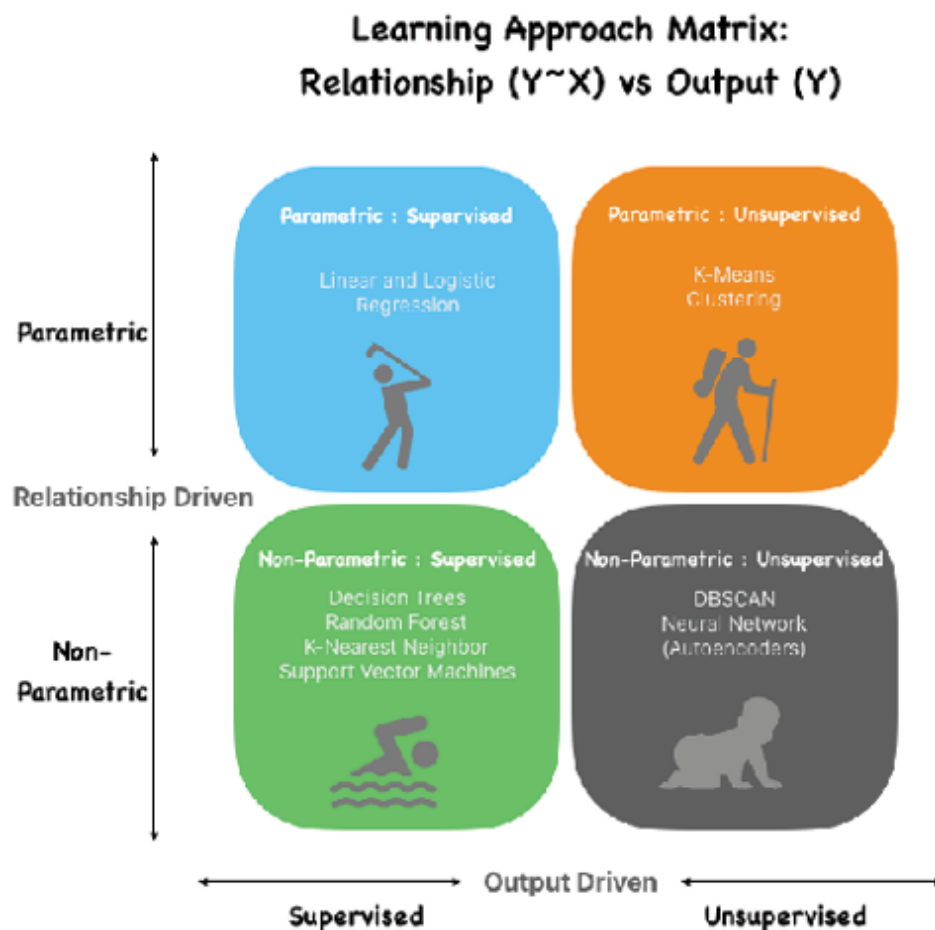


Figure 1.4: Activities based on their learning approaches

To summarize, a model making predictions based on unseen data with certain parameters or attributes after having learned from the same features in a training dataset is termed predictive modeling. It is called the problem of function approximation in mathematics, wherein the task is to approximate a mapping function (f) from input features

(X) to output variables (Y). The modeling algorithm's task is to determine the optimal mapping function given the time and resources at hand during training.

Two dominant strategies: Classification and regression

Generally, we can segregate most function approximation tasks into regression and classification. Both of these tasks are supervised machine-learning algorithms where a model is trained with correctly labeled output variable (y) data. However, the most significant difference between regression and classification is that while regression helps predict a continuous quantity, classification predicts discrete class labels. For instance, predicting

binary approval vs denial outcomes on a home loan application is considered a classification task, while predicting continuous values of home prices using characteristics of a house falls under the regression task. In simplest terms, regression means a relationship (between Y and X).

It is interesting to note that the features corresponding to the value and condition of the home can be used to perform both a classification and regression task and it is the task of the algorithm to learn the association between inputs and output.

Biases and learning shortfalls

Learning is never absolute and is accompanied by various shortfalls. Humans learn in certain scenarios and based on their attention to skills, they pick up nuances of the scenarios. Some details are typically missed leading to a shortfall or error in understanding. When this partial knowledge is applied in a similar scenario elsewhere, humans are generalizing based on their past learning. This generalization also comes with a level of error.

In data mining, the first-time learning error is called *Bias* and the generalization error is termed *Variance*.

Let us understand using an example of a kid who is trained to recognize a range of dress shirts that are long-sleeves, have a collar, and are made of cotton. When a child's knowledge is assessed using a long-sleeved T-shirt with a collar, they routinely perform poorly because they mistakenly think that T-shirts are dress shirts. The child's understanding of a dress shirt is not sophisticated enough to recognize all of its features. This is a gap in learning or error in recognizing the dress shirt appropriately. In statistical terms, it is known as bias.

On the other hand, consider the child is trained with a variety of dress shirts with additional characteristics like a full button front, French cuffs, and pointed collars. Though this learning can help correctly predict a dress shirt as not a t-shirt, it will falsely predict any brand-new dress shirt that does not have point collars or dress shirts that do not come with a French cuff. This learning has a high variance because it is more complex and does not generalize well to the new scenario that the kid is now exposed to.

In the machine learning perspective, bias arises from underfitting, or oversimplifying the model in which the algorithm does not detect the relationship between features and outcome. Variance is the result of overfitting or adhering too closely to the noise also known as every minute detail in the data, and hence fails to generalize on the new data.

Measuring learning accuracy and balancing trade-offs

One of the most important tasks in data mining while training a model is reducing the prediction error while maintaining high training accuracy. At times it is tempting to believe that prediction errors on live unseen data can be minimized by minimizing the training error and fitting the

model on the learning data as accurately as possible. But as seen in the dress shirt example, it is not advisable because it may make the selected model too complex and non-generalizable. Remember, how the kid with all the learnings about a dress shirt could not identify a new dress shirt with a slight variation of French hand-cuffs? This means that the kid had over-learned the features and hence the resulting learning error or bias is very small (because the fitted model has been optimized for the data set), but the test or prediction error or variance on a new dress shirt is large (a consequence of overfitting the learning dataset).

Bias and variance are each a component of test error in many supervised learning algorithms, particularly concerning regression and classification algorithms. It is important to note that bias is an inherent problem of the model that may still be observed with infinite training data. On the other hand, variance can be thought of as the degree to which predictions for a given data point vary between different versions of the trained model.

Overall prediction or test error can be defined by the below formula. The derivation of the formula is beyond the scope of this book, but for curious minds, the reference to the below equation can be found in Hastie, et.al. 2009:

$$\text{Expected Test error} = \text{Variance} + \text{Bias}^2 + \text{Noise}$$

Noise is simply a feature of all datasets used for learning and can never be avoided. The goal of any predictive modeling exercise is to avoid learning the noise.

While a data scientist seeks to minimize each error, it is only logical to conclude that a sweet spot for a model is the level of complexity at which the rate of change of bias concerning complexity is equivalent to the rate of change of variance.

Additionally, in a world with imperfect models (with variance) and finite training data (to cause bias), there is a tradeoff between minimizing the bias and the variance. Hence a

balance is sought and not a minimum of each of the two errors. It is critical to understand these two types of error for diagnosing model results using an accurate measure of prediction error and avoiding the mistake of developing an over or under-fitting model.

The eventual goal should be to explore the level of complexity that minimizes the test error. Unfortunately, there is no analytical way to find this sweet spot.

Can data size and sample impact learning

As indicated above, in an ideal world, bias can be reduced to zero with infinite training data. On the other end of the spectrum, a small sample may not be a true representation of the population leading to high bias. However, most of the machine learning algorithms rest on the fact that the small sample is enough to represent the true population. Some biases that can arise in small samples are as follows:

- **Availability bias:** Just because it is not available, does not mean it never happened.
- **Survivorship bias:** We normally like to study winners and not losers, and hence miss important causations.
- **Historical bias:** History repeats itself hence it is important to learn from it.
- **Selection bias:** Missing the wood for the trees because the sampling methodology is not truly randomized.
- **Confirmation bias:** social media is my small world because it shows me what I like to see in the world.
- **Outlier bias:** I like to hide inconvenient truths in averages.

Great care is needed when trying to learn from small datasets because they may not be sampled right in addition

to limited learning opportunities in the limited sample size.

How do humans benefit from data and learning

The traditional method of turning data into knowledge has relied on manual analysis and interpretation. It started with *Thomas Bayes's* paper that was published in 1763 and talked about prior probabilities about current probability. This was known as **Bayes theorem** and was followed in 1805 by *Adrien-Marie Legendre* and *Carl Friedrich Gauss* applying regression to determine the orbits of bodies around the Sun. Statistics has much in common with KDD techniques and can be described fundamentally as a statistical effort.

In the early days of KDD, statistics provided a means to quantify the varying patterns that were observed in population samples. For instance, identifying the probability that it will rain today given it rained yesterday. With the advent of many more statistical measures, subsequent growth of large data storage systems, and huge computer processing power, we have moved on from what was essentially descriptive statistics. This primarily involved analyzing and quantifying historical data using the statistical measures of central tendency, dispersion, and skewness. Quantification of human experiences, driven by inferential statistics helped us move ahead by enabling inferences from a sample about the larger population from which the sample was drawn.

We hypothesized and concluded that a quantified pattern is interesting if it validated a hypothesis that the user sought to confirm. It was easily understood by humans and was valid on new or test data with some degree of certainty, and potentially useful. For instance, a website owner can hypothesize that changing the color scheme of the website's **call-to-action (CTA)** buttons will result in increased user engagement. The hypothesis is that users will be more likely to click on the CTA buttons if the color is changed. This test

can be performed by collecting the data for the two groups of customers, one that experiences the original color scheme and the other experiencing the new color scheme. Testing and comparing the click data across two groups of customers can reveal if the new color scheme is successful. After all, only an interesting pattern can represent new knowledge.

Today, we can leverage these principles of statistics and talk about various forms of analytics such as the four mentioned in below [Figure 1.5](#):

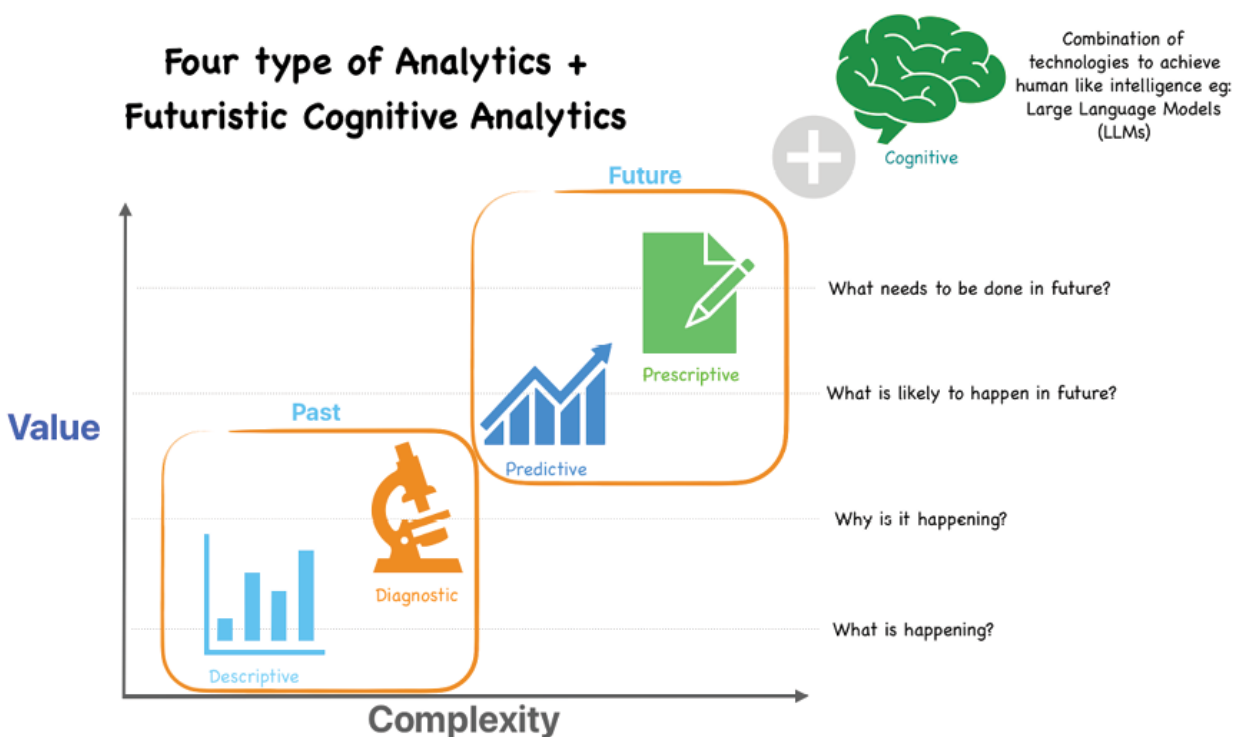


Figure 1.5: Types of analytics

Cognitive analytics is unfolding very recently with the emergence of Generative AI but is not commonplace in the industry. We will talk about each of the above analytical processes in more detail in subsequent chapters. Humans have now realized the benefits of the broader philosophy of KDD and understood that there is both an economic and scientific need to scale up human analysis capabilities to handle the large amount the data that is now generated

daily. Businesses use data to gain competitive advantage, increase efficiency, and provide more valuable services to customers. Today, numerous systems use KDD techniques to automatically produce useful information from large masses of raw data for businesses around the globe.

Modern-day data mining challenges and possible remediation

Modern data mining faces many challenges that can be categorized into two broad buckets: learning and usage challenges. Just like when humans are faced with new experiences, they require new mental models to comprehend them, machines are faced with the ever-growing challenge of large heterogeneous data sources that need advanced learning techniques. New learning techniques are becoming more complex driving explainability, scalability, privacy, and security concerns. It is fascinating to be living in an era where technology is begetting technology to solve the problems caused by technology. Let us try to summarize the two categories of problems:

- **Learning:** This set of challenges deals with new data sources, infrastructure, training, and validation of models. Businesses have long focused on these to solve problems plaguing them. Data scientists have played a huge role in combating this challenge.
- **Usage:** This is the most often overlooked set of challenges that bring the topic of model usage context, model operationalization, rapid changes in the model operating environment, performance monitoring, and the governance around the usage to prevent unintended consequences from model usage. Newer skill sets are needed to tackle this area which will assist data scientists in making the most of their

efforts. Having said that, data scientists should keep in mind the bigger picture around future model usage.

Throughout this book, while we will talk about learning, that is, describing various data mining models, a big focus will also be on building data-mining solutions that are easy to use, risk-managed, and deliver the intended outcomes throughout their life.

Model risk comes in various shapes and sizes and stems from immature model lifecycle handling. Hence most regulated industries such as banking employ **model risk management (MRM)** principles based on the Federal Reserve's S.R. 11-7 model risk management guidance.

The **National Institute of Standards and Technology (NIST)** specifies another framework (**NIST AI Risk Management Framework (AI RMF)**) for managing **artificial intelligence (AI)** risk to individuals, organizations, or society. These principles are deployed using the tools provided by **Model Operations (ModelOps)**. ModelOps is a principle-based approach to model usage that focuses primarily on the governance and life cycle management of a wide range of operationalized **AI** and decision models.

In this spirit, *Chapter 12, Understanding Model Risk Management for Data Mining Models*, of this book will talk about managing model-related risks by operationalizing data mining solutions also known as models and introduce the readers to the emerging field of ModelOps. Interestingly, it is easy to confuse ModelOps as a post-model development exercise only, while ignoring the fact that many model usage-related risks sneak in during model training. Hence, the underlying safety principles that need to be followed also include tracking model development environments, testing champion-challenger models, and versioning and storing models, to enable easy rollback and deployment.

Efforts to realize the **return on investment (ROI)** of data mining solutions should start at the data scientist level and go all the way up to executives struggling to rationalize data mining-related financial decisions.

Conclusion

Humans are clever and curious but are also biased. Bias creeps in because of numerous decision points and past experiences among other reasons. For millennia human efforts and enthusiasm for exploring the unknown have brought us where we are today but it has not been a straight path. Building physical maps, and quantifying the length of the day for navigation, everything requires careful data collection and studies to build experiences and form mental models. These models helped travelers repeat the effort with ease while codifying this generational wealth of knowledge, which again could be biased at times. These certain biased growth efforts have had risks involved but humans navigated these risks by developing checks and balances and eventually achieving their goals in the process.

The modern-day quest for knowledge is no different and comes laden with risks that can impact numerous decisions in the wrong way. As the complexity of knowledge discovery systems increases with growing data and processes, practitioners need to follow the science with a lot of art.

Every step along the KDD process is prone to unwarranted bugs raising their heads, which could lead to lost revenues and even lives. This book touches upon some of the industry best practices and frameworks that budding data scientists should follow to deliver impactful and fair data mining algorithms.

In the next chapter, we will learn about basic statistics and exploratory data analysis.

Points to remember

- Data is no more just oil; it has become solar. Every human action has the potential to keep generating data sustainably for the years to come. Humans have set off the chain reaction.
- Data needs to go through a **reliable, explainable, safe, interpretable, resilient, and transparent (RESIRT)** process to become trusted knowledge.
- Humans apply the gained knowledge through mental models and built algorithms that can mimic the same behavior of human learning to learn from data and form mathematical models.
- Classification and regression are still the two dominant forms of supervised learning data mining algorithms in the industry.
- Like humans, models also suffer from bias and make inaccurate decisions.
- Avoiding algorithmic bias and risks that can impact truth, fairness, and equity at various levels of society is one of the key challenges of current times.
- Risks are prevalent across the model lifecycle from conceptualization to maintenance to end-of-life.

Join our book's Discord space

Join the book's Discord Workspace for Latest updates, Offers, Tech happenings around the world, New Release and Sessions with the Authors:

<https://discord.bpbonline.com>



CHAPTER 2

Basic Statistics and Exploratory Data Analysis

Introduction

This chapter of the book will discuss the art of exploring datasets to build an understanding of the underlying features and discrepancies before making any assumptions. Before using inferential statistics, we will learn to use non-graphical and graphical data exploration methods using various descriptive methods. The chapter will start with a quick Python installation guide and use the most commonly used Python library for data exploration and inferential statistics.

Structure

The chapter will cover the following topics:

- Setting up Python 3.x
- Data mining and statistics
 - Statistics: Foundation, key terms, needs, and types
- Descriptive statistics
 - Graphical and non-graphical exploratory data analysis
 - Non-graphical and graphical representation of univariate data

- Non-graphical representation of multivariate data
- Graphical representation of multivariate data
- Probability theory
- Probability distributions
- Inferential statistics
 - Hypothesis testing with commonly used statistical tests
- Introduction to Time Series Data
- Exploratory data analysis: HMDA data case study

Objectives

By the end of this chapter, you will be able to describe the difference between descriptive and inferential data analysis by gaining knowledge of different statistical techniques applied under these two categories. Both are important and should go side by side for the practitioners to build a comprehensive understanding of their data. We will leverage the available datasets to run various hypothesis tests using basic Python programming. Eventually, readers would also appreciate that theory data analysis needs different tools than confirmatory data analysis. Future chapters will build on the knowledge gained in this chapter regarding Python statistical skills.

Setting up Python 3.x

Python distribution is available for Windows, Linux, and Mac operating systems. Alternatively, one can leverage the freely available and ready-to-use Google Colab environment for all practical purposes to perform any of the code snippets or exercises mentioned in this book.

Python can be downloaded and installed from the following link:

<https://www.python.org/downloads/>

Another useful link:

<https://realpython.com/installing-python/>

Google Colab can be found at the following link:

<https://colab.research.google.com>

Getting started is as easy as clicking the + Code button as shown in the following figure:

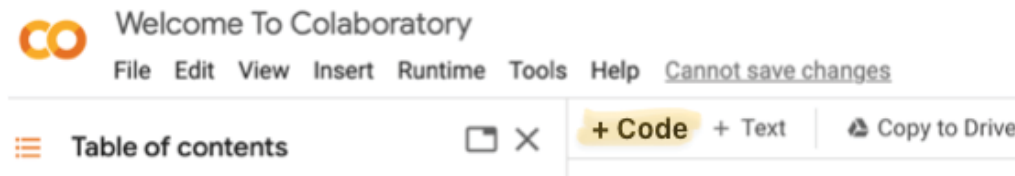


Figure 2.1: Google Colab welcome page

Data mining and statistics

Data mining is the nontrivial process of identifying valid, novel, potentially useful, and ultimately understandable patterns in data.
- Fayyad.

As data miners, our primary goal should be to define, design, and execute this nontrivial process described by Fayyad. In the quest for golden nuggets of information, data miners should dig through the variation and uncertainty in the available problem-specific data. The variation in natural occurrences causes uncertainties and patterns that are hard to establish without data. For instance, it is known that it invariably snows on Christmas in Chicago every year, but without data, it would be hard to prove this belief to someone in Dubai. This implies that often time the truth is out there, but we still have a limited view or are simply unaware due to a lack of data support. On the other hand, the uncertainty, even with data in hand, is because the outcome in question is not determined yet. For example, it snowed on Christmas last year, but we may not know whether it will snow this year. Hence, this first step to understanding the variation and uncertainty is a significant effort and success depends on how efficiently data is generated, captured, stored, and processed.

Our second goal should be determining the causes of this uncertainty and variation to determine the understandable patterns in the generated data. This process started in Europe, in

the late 17th century and saw the rise of statistics with English statisticians such as *John Graunt* and *William Petty* publishing a series of numbers, observations, and fancy calculations for quantitative studies of disease, population, and wealth. Graunt's goal was to chart the advance and decline of the plague while Petty wanted to quantify the monetary value of all those living in Ireland. Fast forward to the modern day, statistics has thrived as a heterogeneous but powerful set of tools and methods concerned with developing and studying methods for collecting, analyzing, interpreting, and presenting empirical data.

More specifically, statistics as a field of study facilitates empirical data quantification either graphically or by providing summary statistics to describe an entire population or even an individual sample. Alternatively, given the uncertainty or variation associated with experiments or population samples, statistics can also be used to make inferences rather than simply stating facts. These inference results are often stated in the form of probability.

Let us dig deeper into the two types of descriptive and inferential statistics and understand the role of probability in making inferences.

Statistics: Foundation, key terms, needs, and types

As we talk about statistics, it is important to remember that the early statisticians were experimenters and empiricists. They collected and analyzed data using various statistical methods to do two things: explore (by describing) and infer (by confirming). We will discuss both these methods in this chapter in detail but before that let us understand some key terms commonly used in statistical studies:

Key terms:

- **Population:** A population is the group of all available data points that are interests of the study. For instance, all the customers of ABC bank across branches. The obvious problem with taking the entire population for the study is its size, changing behavior, and unobservability. This brings us

to a small subset of the observable population known as the sample.

- **Sample:** A sample is an observable or manageable population subset. An unbiased sample should have all the characteristics of the population. For instance, the bank decided to survey 100 customers over the next week from various time slots and all branches to ensure a mix of representation. The properties of an ideal sample are:
 - It captures all the variations present in the population.
 - It is unbiased and presents an equal selection opportunity for the objects from the population.

In statistics, we typically work on a representative sample and generate a summary of samples like sample mean or proportion, known as sample statistics and denoted by Latin symbols. Numbers representative of the population mean are known as parameters and are denoted by the Greek alphabet.

- **Simple random sample:** A sample where every customer has an equal chance of getting selected. This implies that picking one customer from the population must be independent of picking another.

In Python, you can randomly select sample elements with functions such as `choice()`, `sample()`, and `choices()` from the random library. The `sample()` is used for random sampling without replacement, and `choices()` is for random sampling with replacement. Refer to the following code:

```
import random

CustomerID = [1, 2, 3, 4, 5]

#Let's take 3 samples without replacement
print(random.sample(CustomerID, 3))

#[1, 2, 4]

#Let's take 3 samples with replacement
```

```
print(random.choices(CustomerID, k=3))  
#[3, 4, 3]
```

- **Variable:** A variable is a feature characteristic of any customer of a population differing in quality or quantity from one customer to another. *Age* could be the variable of interest for the 100 surveyed customers.
- **Quantitative variable:** A variable differing in quantity and numeric in nature that can be measured, for example, the age of a person, the balance in the customer's savings bank account, and so on.
- **Discrete variable:** A discrete variable is one in which no value may be assumed between two given values which means there are discontinuities between values—for example, the number of children in a family and the number of dependents in the family cannot be 2.5 or 3.7, and so on.
- **Continuous variable:** A continuous variable is simply whose value can vary continuously between two given values, for example, the savings account balance, monthly credit card spending, and so on.
- **Qualitative variable:** A variable differing in quality is called a qualitative variable or attribute, which means they are non-numeric and cannot be measured, for example, the gender of the customer, or the rank of students. These can be of two types:
 - **Nominal:** Data that can be separated into discrete categories which do not overlap. For example: male and female.
 - **Ordinal:** Data that is placed into some order or scale for example rank of students

Descriptive statistics

Data exploration should be different from finding a needle in a haystack. You need the right tools and some understanding of where to look and how to dig deeper if needed. Descriptive

statistics or **Exploratory Data Analysis (EDA)** provides the tools and understanding to find the necessary information.

EDA methods are often called descriptive statistics because they describe or estimate the data sample under study. These methods are useful for:

- **Univariate analysis:** Checking individual variables for anomalies, variation, shape, and so on.
- **Multivariate analysis:** Investigating relationships (magnitude and strength) between two or more variables.

Since the data is available in tabular format, going typically into thousands and millions of rows and columns, humans devised EDA as a lens to highlight the most meaningful aspects of data by using either graphical or non-graphical methods. EDA is defined by cross-classification of these two methods with the number of variables (univariate or multivariate) under the study.

In summary, descriptive statistics is the term given to data analysis that helps describe, visualize, or summarize the data. This means that any given dataset can be described using three high-level characteristics:

- **Measures of central tendency:** It is a single value that identifies the central position of the data.
- **Measures of dispersion:** The measure of dispersion refers to the variability within the data where variability measures how close or far the data lie from the central value.
- **Distribution of the data:** Modality, shape, and outliers.

This is very similar to how an astronaut would describe Earth or other planets from space. Just like the Earth can be described in terms of its shape, center of gravity, and surface variability, a collection of data points is no different and can be summed up using the three basic characteristics outlined above. Let us learn more about these in the context of data in the following section.

Graphical and non-graphical exploratory data analysis

Descriptive statistics are affected by the data's type and sample of measurement. For instance, categorical variables such as nominal and ordinal will differ in the analysis from interval and ratio type data, often termed continuous type data. We also need to recognize that each time we repeat an experiment with a new sample of a given population the underlying descriptive statistics would change due to the random selection.

It is always a best practice to start with the univariate study of variables in the data before trying to perform the multivariate study. It is only intuitive to think about individuality before thinking about relationships in the real world. We will start with a discussion on univariate non-graphical and graphical EDA methods before moving on to the multivariate graphical and non-graphical methods.

Non-graphical and graphical representation of univariate data

The central tendency and variability of the data are the two aspects of non-graphical descriptive statistics used for continuous data. They have no meaning for nominal categorical data but in some situations, ordinal data can be treated as quantitative or continuous for EDA purposes.

These sample statistics for continuous data can generally be estimates of the corresponding population parameters. The reason is that they are closer to reality but still, there is a certain level of uncertainty, hence these are just estimates.

Non-graphical representation of univariate variables: In this section, we will discuss the methods to summarize continuous data.

- **Measures of central tendency:** This refers to a statistic that best characterizes the group as a whole. It is generally referred to as the average. Refer to the following figure for the three types of averages:

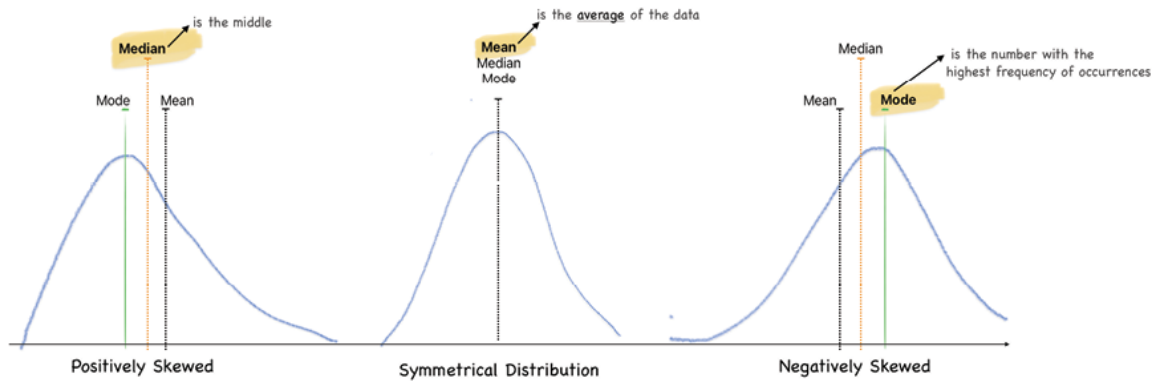


Figure 2.2: Three measures of central tendency

Let us calculate these sample statistics using Python in-built functions:

```
sample = [1,2,3,4,4,5,6]
import statistics
statistics.mean(sample)
#3.5714285714285716
statistics.median(sample)
# 4
statistics.mode(sample)
# 4
```

- **Measures of variability:** This refers to the spread or dispersion among a set of data points. It helps to answer the question: what is the magnitude of departure from the average value? If the average credit card spend of a bank's customers is \$100, then how far from the average is the individual spend value of various customers?

```
sample = [1,2,3,4,4,5,6]
statistics.variance(sample)
#2.952
statistics.stdev(sample)
```

#1.718

Figure 2.3 describes other key statistics related to a continuous variable's distribution:

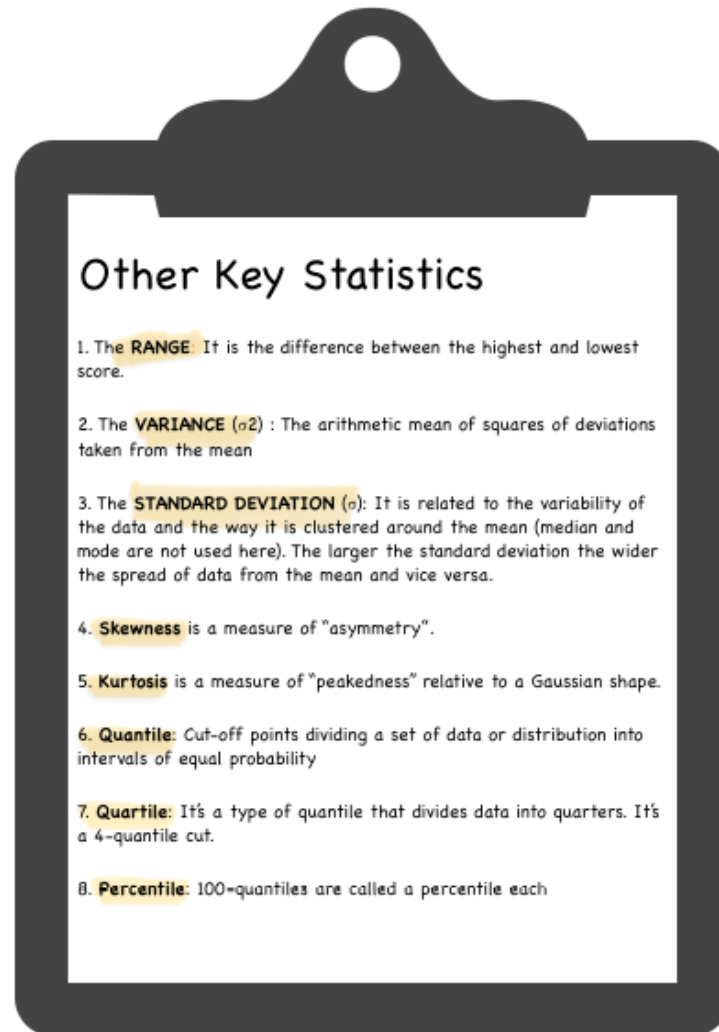


Figure 2.3: Other key statistics

Another important concept built around sample statistics is the **central limit theorem (CLT)**. It states that regardless of population distribution, the sample mean's distribution approximates a normal distribution as the sample size gets bigger. The average of the sample means and **standard distribution (SD)** gets closer to the population mean and SD as the sample size grows. Sample sizes equal to or greater than 30 are often considered sufficient for the CLT

criteria to hold, indicating that the distribution of the sample means is distributed fairly normally.

In other words, if 30 or more bank customers are sampled randomly daily for a month, the average daily account balances would be closer to the average daily balance of all the bank customers. Thanks to the central limit theorem, the customer survey team is confident in using sample averages to estimate the population mean, especially when they sample 30 customers or more. They can thus ensure that the bank process remains on target without assessing every single bank customer!

Hence, CLT, together with the law of large numbers is very useful in accurately predicting population characteristics.

Graphical representation of univariate variables: *Pictures are worth a thousand words – Einstein.* In the case of EDA, it can be worth a thousand numbers. A histogram, which outlines the frequency distribution of categorical or continuous variables, can tell much more about the data distribution than any summary statistics such as mean, median, mode, or standard deviation. Several graphical techniques exist for summarizing the data that force us to see what we never expected to know from numbers. These graphs can work alone or in conjunction with the preceding described statistics.

Let us take an example where we overlay summary statistics over a graphical display to understand the central tendency and spread of the continuous variable all at once.

For EDA purposes in this chapter, we will use **Home Mortgage Disclosure Act (HMDA)** data on the Consumer Financial Protection Bureau website for download. It is the most comprehensive source of mortgage lending activities disclosed by financial institutions(FI) in the U.S. mortgage market. This data sheds light on FI lending patterns that could be potentially discriminatory. Link: <https://www.consumerfinance.gov/data-research/hmda/>

We will start by calculating the five-number summary for an HMDA data variable `interest_rate`. This provides a quantification of

the following statistics corresponding to the quantitative variable `interest_rate`:

- Smallest number
- Q1 = middle number between smallest and median
- Q2 = median of the data
- Q3 = middle number between the median and the largest number
- The largest number of dataset

Essentially, based on the above definitions, the five numbers provide a summary of the data distribution. Python script to calculate these numbers for the variable `interest_rate` at which the mortgage loan was offered is shown below:

```
Q1 = HMDA_IL['interest_rate'].quantile(0.25)
print(Q1) # 2.75
Q3 = HMDA_IL['interest_rate'].quantile(0.75)
print(Q3) # 3.5
Q2= HMDA_IL['interest_rate'].quantile(0.50)
print(Q2) # 3.125
```

Additionally, `q3` and `q1` can be used to calculate the **Interquartile Range (IQR)** which is the difference between `q3` and `q1`. Please note that IQR is resistant to outliers because it focuses on the central portion of the data distribution, unlike the mean, which accounts for all the numbers. Hence, if we are looking to compare descriptive statistic that is not heavily influenced by extreme values and provides a summary of the central spread of the data, **IQR** is a suitable choice.

Since $IQR = q3 - q1$, the following steps need to be performed to calculate the **IQR**:

1. Define quartiles.
 - a. To compute the median, a dataset is sorted in ascending order and the middle value in the sorted dataset is the

median. The median divides a sorted dataset into two halves.

- b. The first quartile (Q_1) is the middle value in the first half of a dataset (dataset sorted in ascending order)
- c. The third quartile (Q_3) is the middle value in the second half of a dataset (dataset sorted in ascending order)

2. Calculate **IQR**.

#Python script for calculating **IQR** from Q_3 and Q_1

$IQR = Q_3 - Q_1$

`print(IQR) # 0.75`

Let us visualize the five-number summary on a box and whisker plot for another variable 'loan amount' from the HMDA data.

[Figure 2.4](#) showcases the box and whisker plot and provides a graphical display of the data based on the five-number summary:

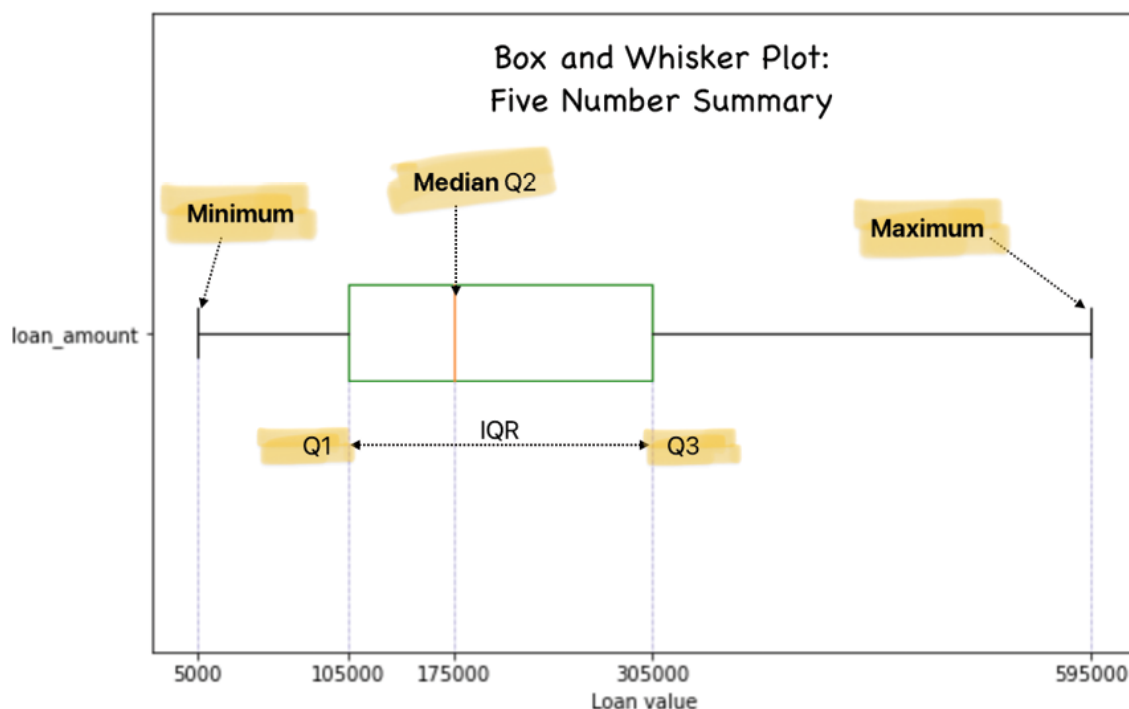


Figure 2.4: Box and Whisker Plot: Five-number summary

The following Python example illustrates the graphical and statistical outcomes as it relates to the five-number summary:

```
#Let's import the visualization library
import matplotlib.pyplot as plt
import seaborn as sns
fig = plt.figure(1, figsize=(9, 6))
ax = fig.add_subplot(111)
ax.boxplot(HMDA_IL_clean['loan_amount'], vert=False,
manage_ticks=True, boxprops=dict(color='green'))
ax.set_xlabel('Loan value')
ax.set_yticks([1])
ax.set_yticklabels(['loan_amount'])
quantiles = np.quantile(HMDA_IL_clean['loan_amount'],
np.array([0.00, 0.25, 0.50, 0.75, 1.00]))
ax.vlines(quantiles, [0] * quantiles.size, [1] *
quantiles.size,
color='b', ls=':', lw=0.5, zorder=0)
ax.set_ylim(0.5, 1.5)
ax.set_xticks(quantiles)
plt.show()
```

The box plot is also useful in analyzing outliers. [Figure 2.5](#) showcases the loan amount variable with (left chart) and without the outliers (right chart):

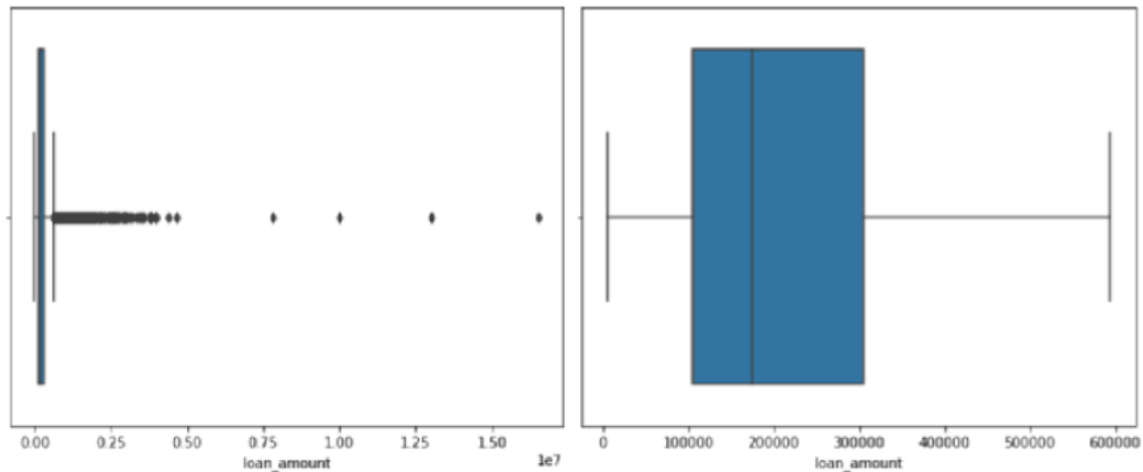


Figure 2.5: Outlier analysis using Box-plot

Other well-known graphs are histograms, bar graphs, line graphs, and Q-Q plots. We will talk about the usage of these in more detail in upcoming chapters.

Non-graphical representation of multivariate data

This section outlines two methods to represent multivariate data in a non-graphical way. We will continue to use the HMDA data.

Cross-tabulation of categorical variables: This is demonstrated using the below code which leverages two categorical variables `derived_race` and `action_taken`.

Python code:

```
Approval_vs_Race =
pd.crosstab(index=HMDA_IL_clean["derived_race"],
columns=HMDA_IL_clean["action_taken"])
```

Approval_vs_Race

Following is the cross-tabulation output is generated using the above code:

action_taken	Application Withdrawn	Approved	Approved but not accepted	Denied	File Incomplete	Purchased Loan
derived_race						
American Native	8	69	4	44	4	0
Asian	74	426	36	265	37	0
Black	65	358	33	359	34	0
Islander	0	2	0	4	1	0
Joint	24	87	4	28	6	0
Race Not Available	208	1053	102	575	91	398
White	520	3484	237	1727	264	0
minority races	0	2	0	3	0	0

Figure 2.6: Loan Approval vs Race cross tab in the HMDA data

Summary of categorical and continuous variables: This is generated using the following Python code.

Python code:

```
HMDA_IL_clean.describe(include=
['object']).drop('count').T
```

The summary table below demonstrates a summary of key categorical variables in the HMDA dataset:

	unique	top	freq
derived_race	8	White	6232
derived_sex	4	Male	3515
action_taken	6	Approved	5481

Figure 2.7: Summary by key categorical variables in the HMDA data

It is common to produce some standard univariate nongraphical statistics for the quantitative variables separately for each level of the categorical variable, then compare the statistics across levels.

Covariance, correlation, and causation between continuous variables:

Covariance is a statistical term that indicates the extent to which two random variables move together and in which direction (positive or negative). It ranges between $-\infty$ to $+\infty$ and is measured using the following formula:

$$cov(x, y) = \frac{1}{n-1} \sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})$$

- x_i represents the values of the X-variable
- y_i represents the values of the Y-variable
- $\bar{x}$ represents the mean (average) of the X-variable
- $\bar{y}$ represents the mean (average) of the Y-variable
- n represents the number of data points

Figure 2.8: Covariance calculation for two variables

The covariance matrix is used when the variables are on similar scales.

Correlation on the other hand is used to indicate some form of association between two quantitative variables. The degree of association, also known as a measure of linear association, is denoted by r , and it is also called Pearson's correlation coefficient after its originator's name. The coefficient of correlation is used to identify whether there is a positive, negative, or no association between two variables.

It is measured using the following formula: $cov(x, y)$ divided by standard deviations of x and y . This denominator scales the correlation to be measured between -1 and +1:

$$r_{xy} = \frac{cov(x, y)}{\sqrt{var(x)}\sqrt{var(y)}}$$

Let us look at the correlation between the three continuous variables (`loan_amount`, `interest_rate`, `income`) in the HMDA data:

```
correlations = HMDA_IL_clean.corr()
fig = plt.figure(figsize=(12, 10))
sns.heatmap(correlations, annot=True, cmap='GnBu_r',
center=1)
plt.show()
```

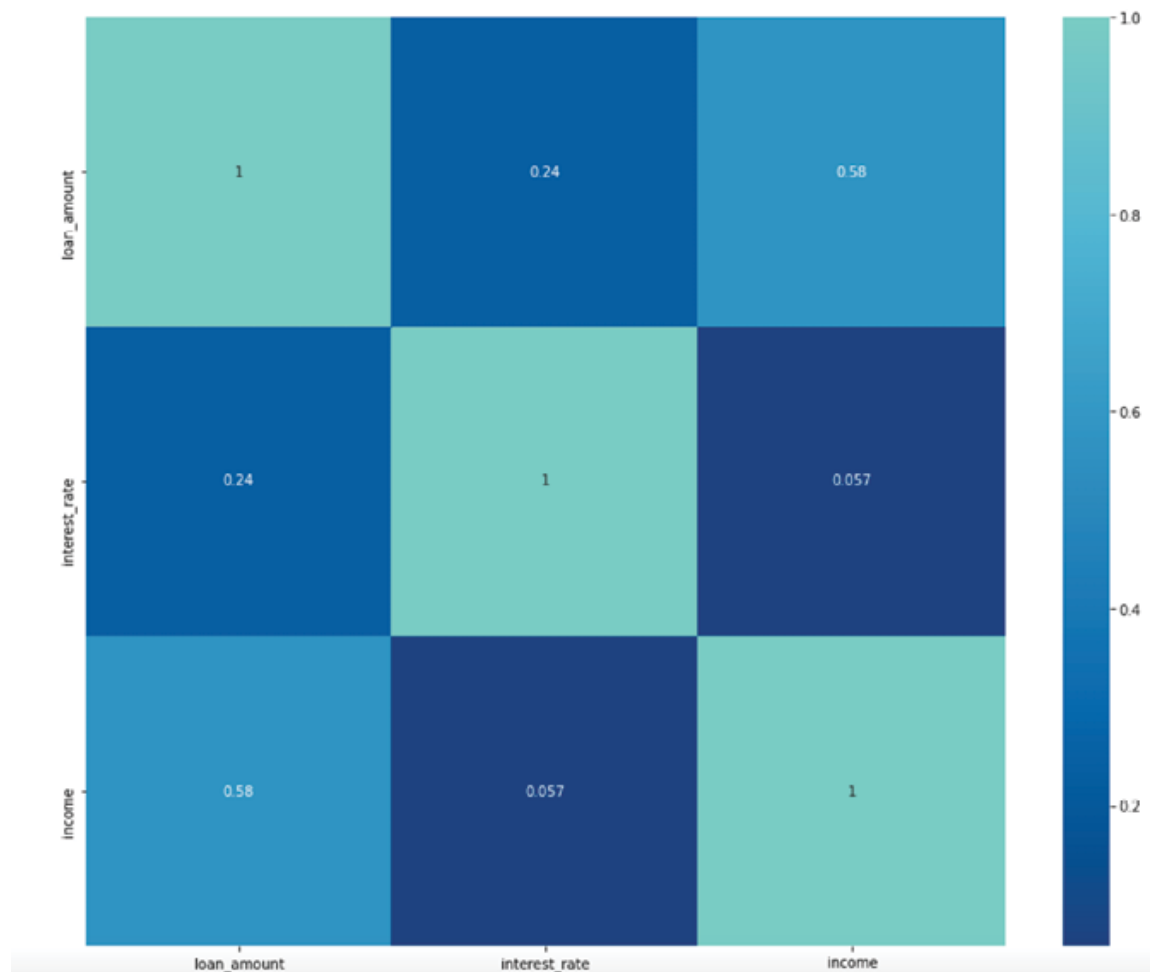


Figure 2.9: Correlation matrix of the three key HMDA variables

Please note, that a high correlation does not always imply causation, but causation always implies correlation. For instance, it is easy to say that water park ticket sales and ice cream sales are correlated. Both are associated to a great degree but does one cause another? Both of these sales are driven up by hot temperatures and do not necessarily cause each other directly. On the other hand, in our HMDA example above, we can say that income and `loan_amount` are correlated (0.58) and very likely high income directly causes the `loan_amount` to go up as well.

Graphical representation of multivariate data

The following are some commonly used multivariate graphs demonstrated using HMDA data:

- Scatter plots - For visualizing the movement of two variables.

Analyze interest rate and Loan amount pattern

```
sns.scatterplot(HMDA_IL['interest_rate'],HMDA_IL['loan_amount'])
```

```
plt.show()
```

Output for the preceding code is shown in the following figure:

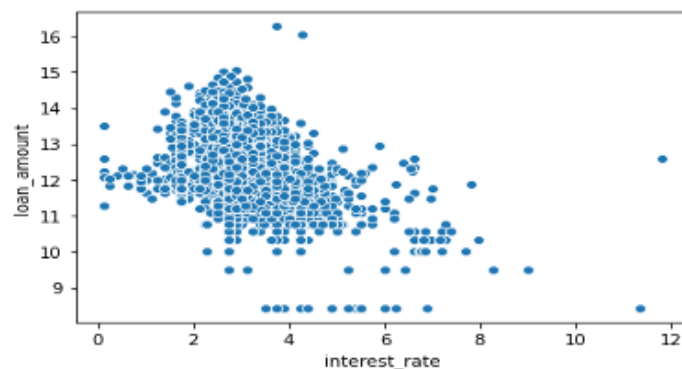


Figure 2.10: Interest rate and Loan amount relationship spread

- Adjacent Box plots - Comparing loan_amount by race

let's see loan distribution for different Ethnicity

```
plt.figure(figsize=(18,10))
```

```
sns.boxplot(x='derived_race',y='loan_amount',data=HMDA_IL_clean)
```

This Python code generates the following output:

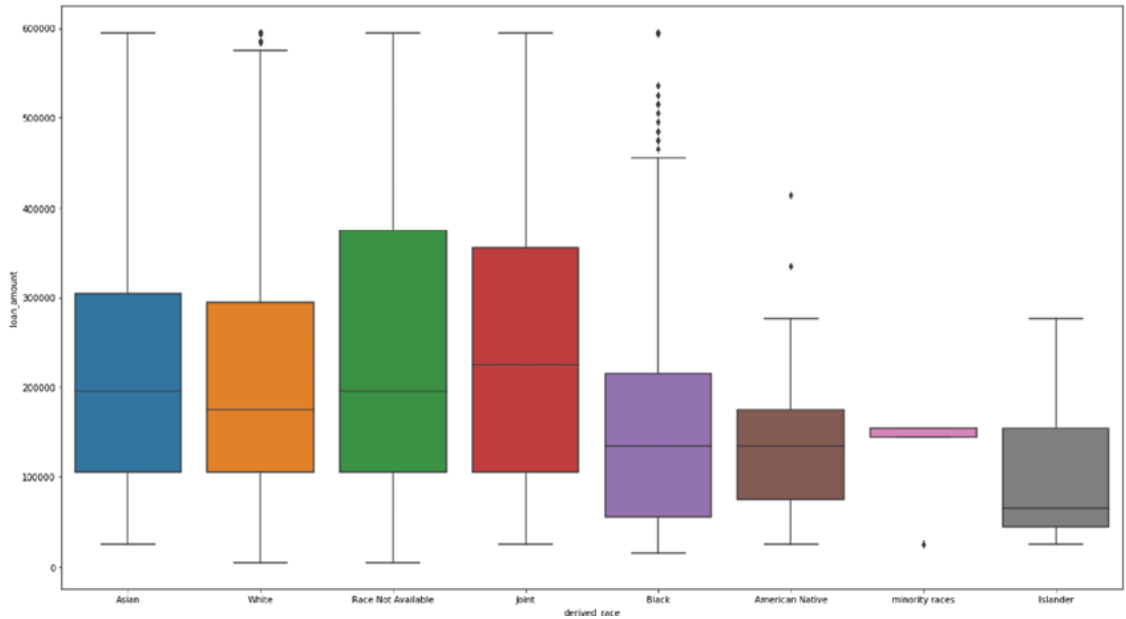


Figure 2.11: Box-plot by Race

The preceding figure shows the grey box plot for the `Islander` race has the least average `loan_amount` disbursed.

Probability theory

The properties of any sample data used for EDA are deeply linked to the corresponding properties of its variables. The properties such as the expected value, variance, correlations, and graphics (histograms, box plots, and so on) represent the uncertainty in the data when the experiment is repeated over and over. For instance, if a repeated sample of loans is drawn from the entire loan portfolio that is the population, the mean unpaid balance amount, calculated for each sample drawn, will hover around a value. We know this from the central limit theorem discussed earlier but here is also this uncertainty associated with random trials which prevents us from making any deterministic inferences about the future unpaid balance amount.

Fortunately, probability theory, which focuses on the analysis of random phenomena, allows statisticians to assess the certainty levels associated with the outcomes of an experiment. This will involve calculating probability, which looks at what might happen based on a large amount of collected data from an experiment

that is repeated over and over. There are two approaches to calculating this probability associated with random trials: Frequentist and Bayesian.

Frequentist or objective: This is different from the classical probability of an event, which can be stated before the occurrence of an event. For instance, consider an experiment of tossing a coin 10 times to assess the number of times the head (event) will occur. Everyone knows that in tossing a coin, the classical probability of a head showing up is 0.5. However, suppose one waits to observe the relative frequency of a head, occurring in the repeated coin-tossing experiment and only uses data from the coin-tossing experiment when evaluating the outcomes. In that case, the probability is called relative probability. For instance, the probability of seeing a head when a coin is tossed 10 times is simply the actual number of heads that occur divided by 10. This could be 7/10, which is different from the classical probability of 0.5 associated with seeing a head when tossing a fair coin:

$$\frac{\text{Favorable outcomes}}{\text{Total possible outcomes}}$$

To summarize, what is expected to happen does not necessarily always happen, hence we wait for the experiment to finish and calculate the relative probability as:

Let us understand the preceding section using an example of 5000 bank customers from branch X, who sign up for different loan types, where a customer can hold only one loan type. This is similar to the mutually exclusive coin toss outcomes where only one head or tail occurs at a time. The bank's staff has built the following table with the loan portfolio information:

Loan type	Number of customers	Relative frequency	Probability
Mortgages	1000	1,000/5,000	0.20
Auto	2500	2,500/5,000	0.50
Personal	1500	1,500/5,000	0.30

	5,000		1.00
--	-------	--	------

Table 2.1: *Calculating relative frequency*

Three key items to note in the preceding table are:

- **Relative frequency of loan customers:** This is 2500/5000 for an auto loan and also means that the probability of selecting a customer who signed up for an auto loan, relative to other customers, is 0.5. For mortgages, it is 0.2, and for personal, it is 0.3.
- **Probability distribution:** A set of probabilities for each loan type in the right-most column. The probabilities are distributed over the range of possible values of the categorical random variable credit card type.
- The sum of all probabilities for all possible values must equal 1 since the total number of customers is equal to the total number of loans across all three loan types.

In general, the relative frequency of an event tends to get closer to the classical probability of the event as we perform more trials. This means the experimental study of finding out the classical probability of auto loan customers can be performed by extending the preceding tabular calculation to other bank branches. The common relative probability value of the auto loan appearing will be the classical probability of customers opting for auto loans in general. This approach helps statisticians to calculate the probability of all possible events after repeated trials, thereby inferring the most probable outcome in the process. The following [Figure 2.12](#) shows the experiment outcome with relative frequencies on the Y-axis:

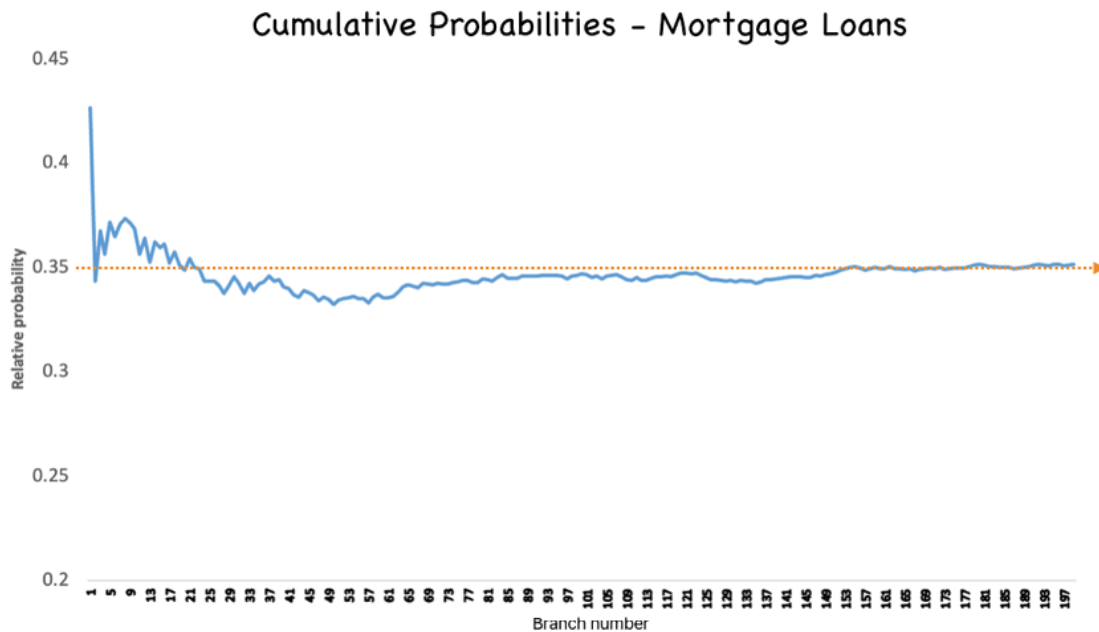


Figure 2.12: Auto loan probabilities across branches

In probability theory, this phenomenon is also known as the law of large numbers. It states that if an experiment is repeated a large number of times, then the average result of repeated experiments is close to the expected value.

Probability distribution

We already saw that the probability distribution summarizes the frequency of values taken by a random variable. Plotting the histogram or a frequency curve for a variable showcases how the data is distributed over its range alongside the information on the shape and spread of the data. [Figure 2.12](#) showcases the probability distribution of three continuous variables loan amount, interest rate, and income in the HMDA data:

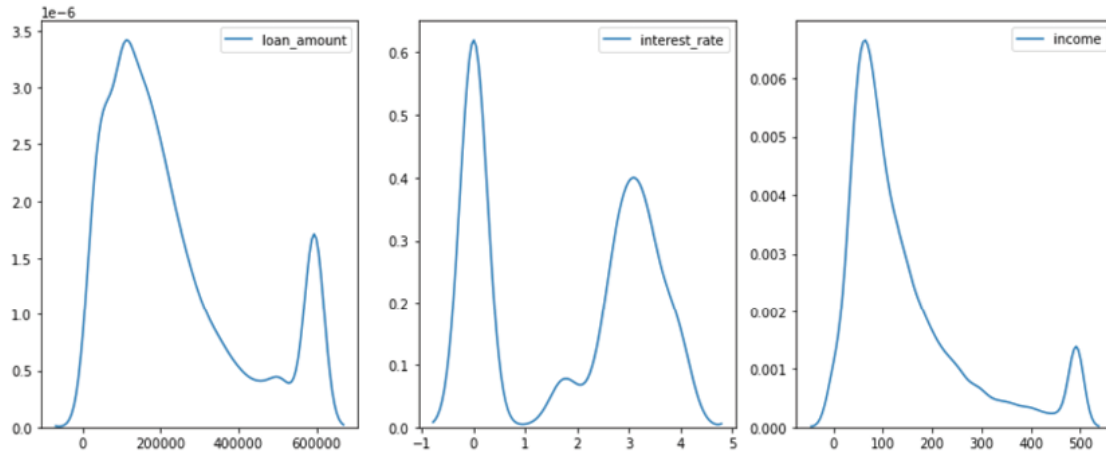


Figure 2.13: Probability distribution by HMDA variables

The probability distribution is of two types: discrete and continuous based on the type of random variable under consideration.

Discrete probability distribution: On a bank branch location visit, you figure out there are 4 windows ($W1$, $W2$, $W3$, and $W4$) to connect with the teller. Hence, the probability for you to be served on any one of the four windows is $\frac{1}{4}$. Since you are equally likely to be served on any of the windows, the probability distribution of this event X would take the value of $\frac{1}{4}$ for each of the windows. Since a **Probability Mass Function (PMF)** mathematically describes a probability distribution for a discrete variable, you can display a PMF with an equation or graph. The following graph describes the PMF for the preceding equally likely event of you getting served at one of the four windows:

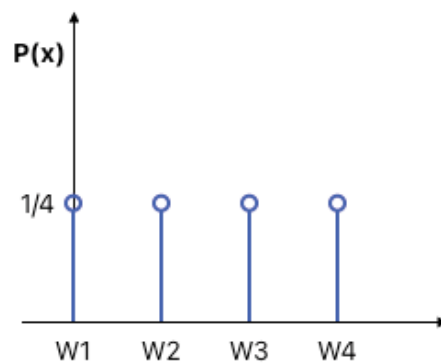


Figure 2.14: Probability mass function for four equally likely events

Let us discuss some other significant discrete probability distribution functions:

Binomial Distribution: This is a widely used probability distribution of a discrete random variable that emerges from the n Bernoulli trials. There are only two outcomes for each of these trials that are designated as success or failure. The probability of success is uniform throughout the n trials and the trials are independent of each other. The probability mass function gives the probability of obtaining exactly k successes in n trials. For instance, in service organizations like customer service centers, binomial distribution can help us get an idea of the proportion of customers who are satisfied with the service quality. Please refer to the following Python code for a customer service example for 'k' successful payment outcomes tested with six customers:

```
import scipy.stats as stats

#Based on past data, We know that 70% of customers are
satisfied with the services.

p=.7

#If a sample of 6 customers is selected at random

n = 6

# Calculate the probability mass function (PMF) for
each k

binomial = stats.binom.pmf(k,n,p)

#Plot the PMF

plt.plot(k,binomial,'o-')

plt.title('Binomial: n=%i , p=%.2f' % (n,p),
fontSize=15)

plt.xlabel('Number of Payment')

plt.ylabel('Probability of Payment')

plt.show()
```

Output:

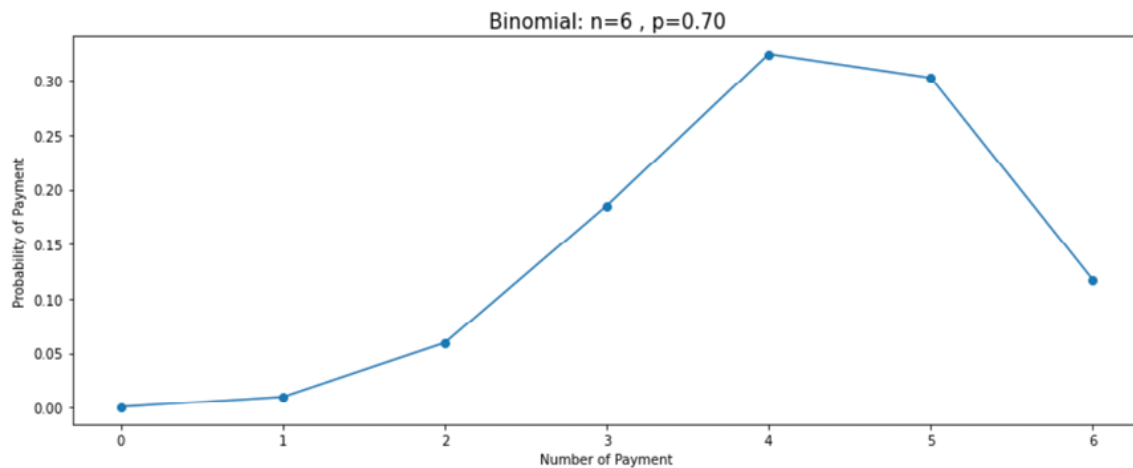


Figure 2.15: Binomial distribution for an event with a success rate of 0.7

Bernoulli distribution: An experiment that is conducted just once and has exactly two possible outcomes, success or failure, is described by a Bernoulli distribution. It is a special case of binomial distribution that has multiple trials conducted. For instance, to check if a credit card transaction is fraudulent (1) or not(0), please refer to the following code:

```
#Bernoulli distribution
```

```
from scipy.stats import bernoulli
```

```
# Bernoulli distribution with parameter p = 0.6, This  
parameter #represents the probability of success (in  
this case, the value 1)
```

```
bd = bernoulli(0.6)
```

```
# Outcome of the experiment will only have value of 0,  
1
```

```
X = [0, 1]
```

```
plt.figure(figsize=(7,7))
```

```
plt.xlim(-1, 2)
```

```
plt.bar(X, bd.pmf(X), color='orange')
```

```
plt.title('Bernoulli Distribution (p=0.6)',  
fontsize='15')  
  
plt.xlabel('Values of Random Variable X (0, 1)',  
fontsize='15')  
  
plt.ylabel('Probability', fontsize='15')  
  
plt.show()
```

The code generates the following output:

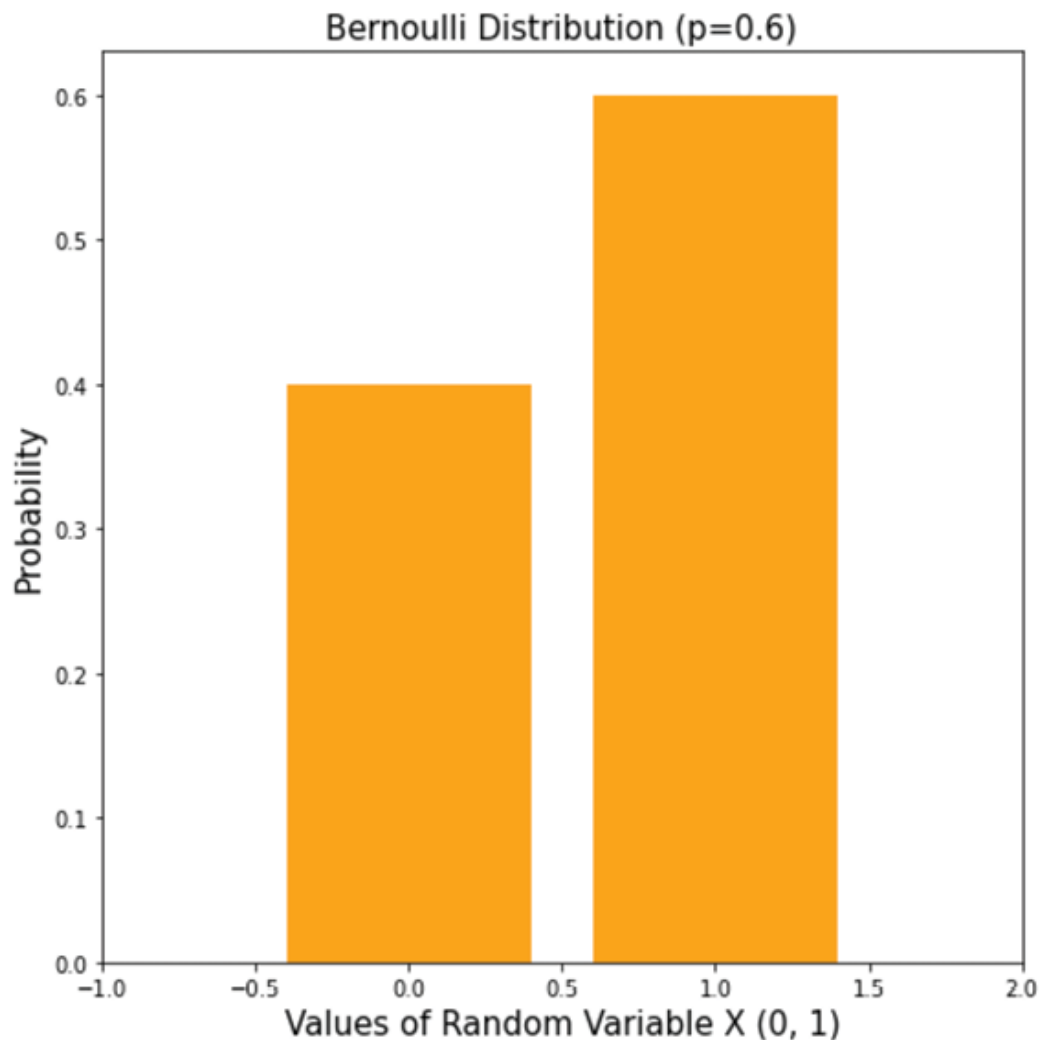


Figure 2.16: *Bernoulli distribution*

Poisson distribution: This is used to determine the probability of events that occur in a fixed interval of time. It is used to describe the distribution of rare and independent events happening at a

constant rate within a time interval. For instance, the number of calls received at a bank's customer service center in a day follows a Poisson distribution. If it is known that on average 3 customers call every minute at a bank during busy working hours, Poisson distribution can be used to answer the following questions:

What is the probability that more than three customers will call in a given minute?

Refer to the following code:

```
#Poisson Distribution parameters
rate = 3
n=np.arange(0,10)
#calculate the distribution and store the distribution
of probabilities in an array
poisson = stats.poisson.pmf(n,rate)
#probability of  $x \geq 4$ , i.e, probability of at least 4
calls per minute given an average of 3 calls per minute
1 - (poisson[0]+poisson[1]+poisson[2]+poisson[3])
#alternate- Probability that more than 3 calls will
arrive in the next 1 minute
1 - stats.poisson.cdf(k=3,mu=3)
plt.plot(n,poisson,'o-')
plt.title('Poisson:  $\lambda = %i$  ' % rate)
plt.xlabel('Number of calls in a minute')
plt.ylabel('Probability of number of calls in a
minute')
plt.show()
```

The code generates the following output:

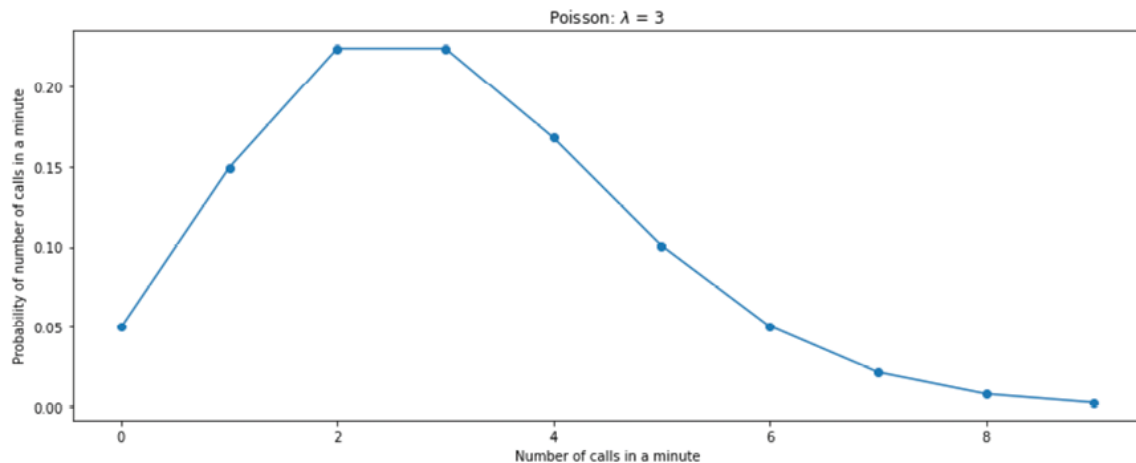


Figure 2.17: Poisson distribution for 'n' number of calls in a minute

To summarize the three distributions, the Bernoulli distribution represents the success or failure of a single Bernoulli trial while the Binomial distribution represents the number of successes and failures in n independent Bernoulli trials. The Poisson distribution is the limiting case of the binomial distribution where the number of trials, n , is infinite and the probability p of success in each trial is small, such that np remains moderate or finite. These concepts have many applications in real-life scenarios where there is a need for estimation of the probability of success with certain conditions.

Continuous probability distribution: As the name indicates, it describes the probabilities of continuous random variables, which can have infinite possible values between any two values. For instance, the number of checking accounts that are opened in a month at a bank branch. This event X can take possible values of any non-negative integer value, like 0,1,2,3,...,n. This corresponds to a continuous probability function also known as the probability density function that is used to find the chances that the value of a variable will occur within a range that the user specifies. This is achieved by integrating the PDF over that interval.

The mathematical difference between a discrete distribution and a continuous distribution is:

- The discrete distribution is defined by the probability mass function (pmf) which is the probability at a particular point.

- The continuous distribution is defined by the probability density function (pdf) which is the integration over its range. Refer to the following figure:

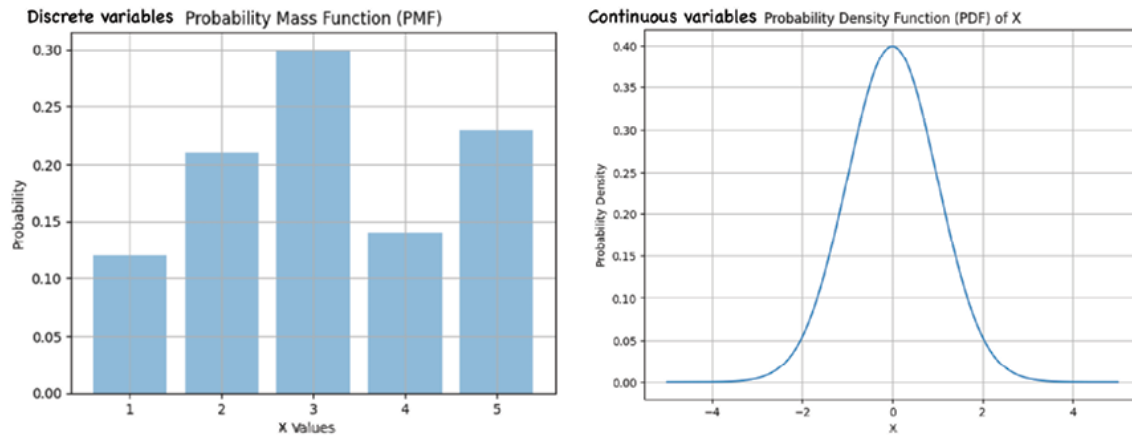


Figure 2.18: PMF vs PDF

Let us look at some of the different types of continuous probability distributions:

- **Normal distribution:** This is the most commonly used continuous distribution type and is also known as **Gaussian distribution**. The shape of this distribution is symmetrical around the mean value, indicating that the mean is the most commonly occurring value in the range and less frequent are found towards the tails. In other words, whenever things of the same kind are measured, a fairly large number of such measurements will tend to cluster around the middle value which is a measure of central tendency. Let us understand from the following graph:

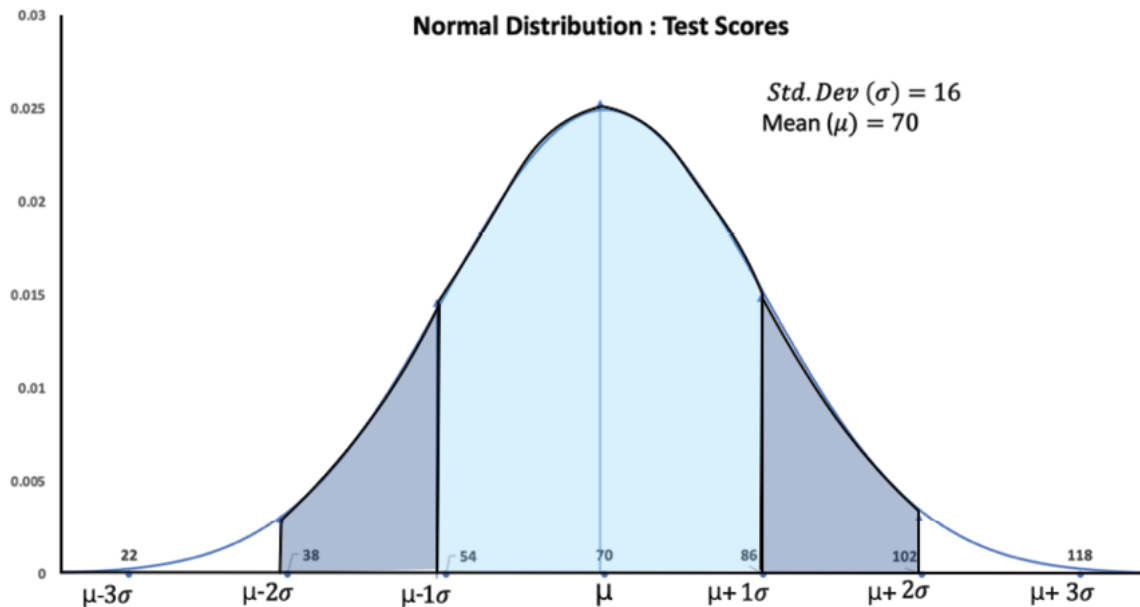


Figure 2.19: Normal distribution

The empirical rule approximates the variation of data in a bell-shaped distribution:

- Above normal distribution is generated using a mean of 70 and std. dev value of 16
- Approximately 68% of the data in a bell-shaped distribution is within 1 std of the mean or $\mu \pm 1\sigma$
- Approximately 95% of the data in a bell-shaped distribution lies within two standard deviations of the mean, or $\mu \pm 2\sigma$
- Approximately 99.7% of the data in a bell-shaped distribution lies within three standard deviations of the mean or $\mu \pm 3\sigma$
- In a perfect world, normal distribution with two parameters, mean and standard deviation, will hold the following properties:
 - A standard normal distribution has to mean of zero and a standard deviation as 1
 - Same Mean, mode, and median = μ
 - Symmetrical around the mean. Skewness = 0
 - Kurtosis = 3 (excess kurtosis is zero)
 - Asymptotic to the x-axis. If the tails are extended, they will run parallel to the horizontal axis without actually

touching it

- The distribution is symmetric about the mean

Let us check this with an experiment. In a random experiment performed repeatedly, if we sample the mean values from multiple samples and plot the mean values, it will look like a normal distribution graph. Please refer to the following code:

```
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
binom_dist = np.random.binomial(1, .5, 1000)
list_of_means = []
for i in range(0, 100000):
    list_of_means.append(np.random.choice(binom_dist,
    100, replace=True).mean())
fig, ax = plt.subplots()
ax = sns.distplot(list_of_means)
```

Output:

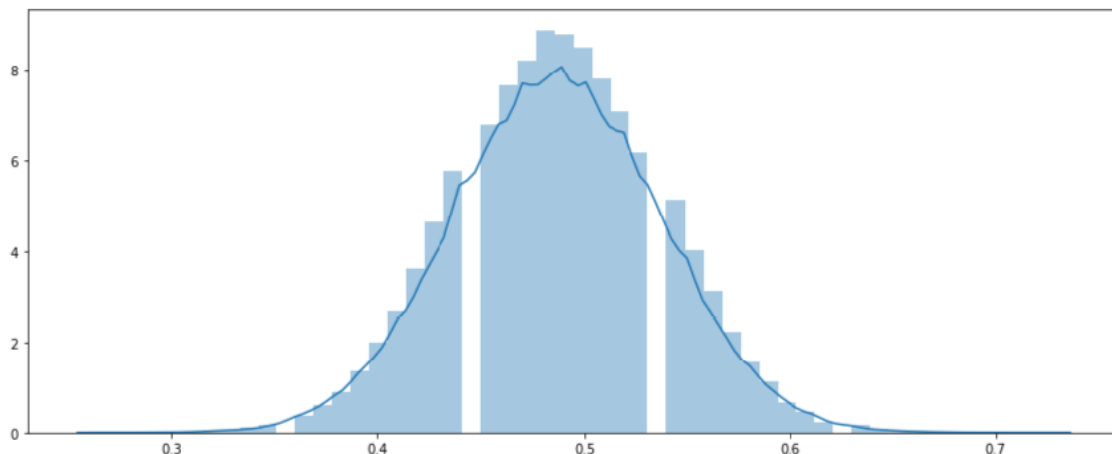


Figure 2.20: Normal distribution

Let us explore the mean, standard deviation, kurtosis, and skewness of loan amount information from HMDA data. Refer to

the following code:

```
from scipy import stats

l = ['loan_amount']

for i in l:
    sns.distplot( HMDA_IL_clean[i])
    plt.show()
    print('Mean:',round(HMDA_IL[i].mean(),0))
    print('Median:',HMDA_IL[i].median())
    print('Standard
Deviation:',round(HMDA_IL[i].std(),0))
    print('Variance:',round(HMDA_IL[i].var(),0))
    print('Skewness:',round(HMDA_IL[i].skew(),0))
    print('Kurtosis:', round(HMDA_IL[i].kurt(),0))
```

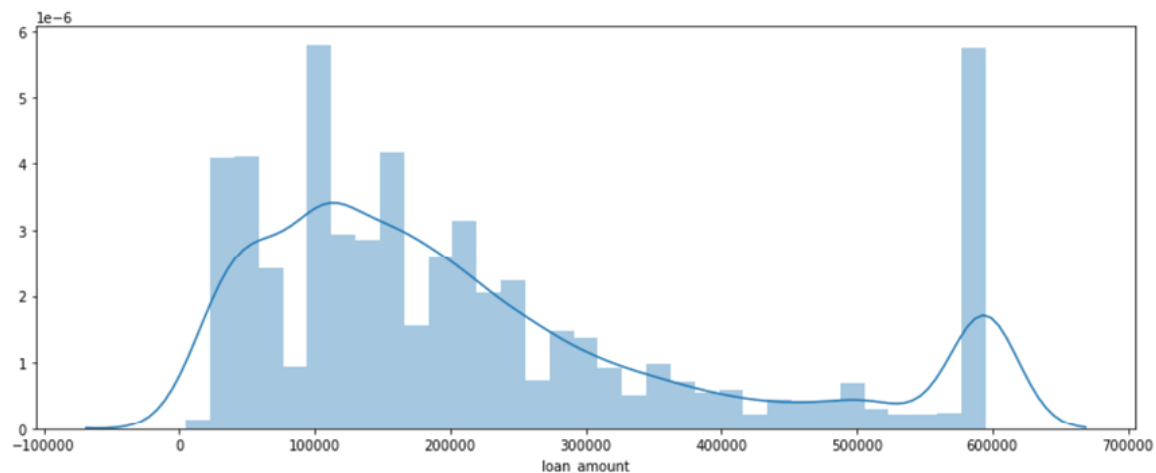


Figure 2.21: HMDA Loan amount distribution

Output:

Mean: 273373.0

Median: 175000.0

Standard Deviation: 402637.0

Variance: 162116570580.0

Skewness: 13.0

Kurtosis: 398.0

Other types of continuous distributions are lognormal and exponential. The following grid shows these distributions with the code to generate them:

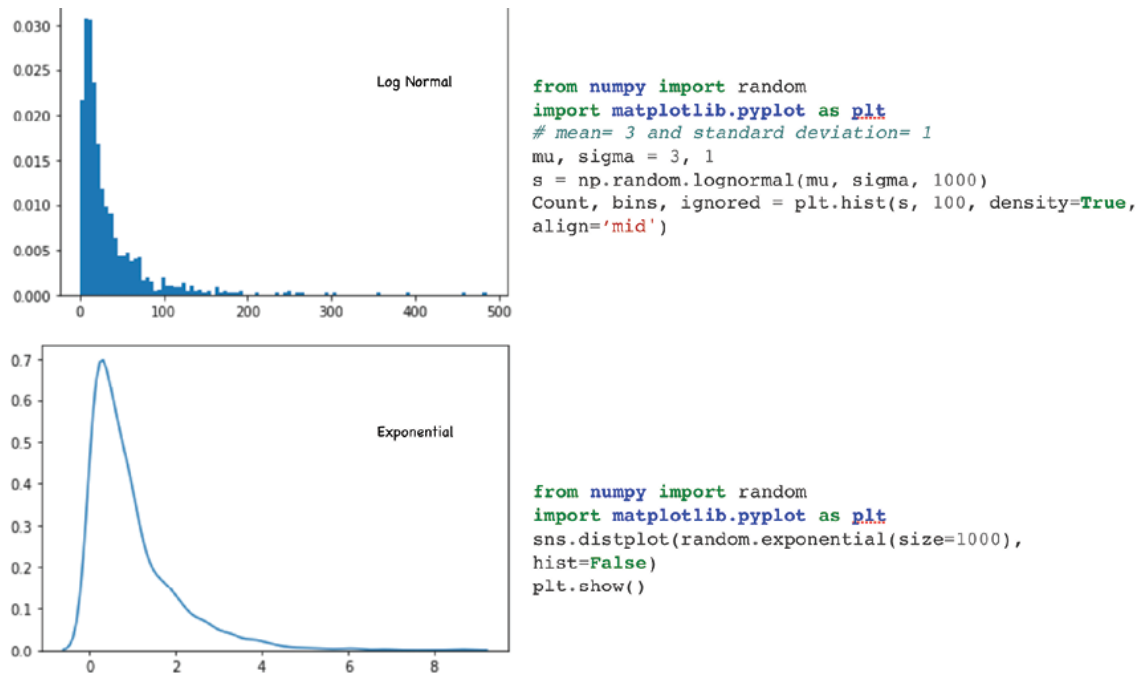


Figure 2.22: Other Continuous distributions

This concludes our discussion on probability calculation using the frequentist approach which is suitable for cases when data is abundant or prior information is scarce.

Subjective or Bayesian: Updating your beliefs in light of new evidence is the Bayesian approach to calculating probabilities associated with random events. With Bayesian statistics, probability expresses a degree of belief in an event by using prior knowledge as well as current experiment data to predict outcomes. It is called subjective probability because sometimes the prior probabilities could be personal beliefs.

For instance, based on a cricketer's past record, his probability of hitting a century is 0.6. In the past, it has always rained when he

scored a century. Would his probability of scoring a century in the next game change if it is known that it will not rain tomorrow? Let us look at the following decision tree to understand the various possible outcomes:

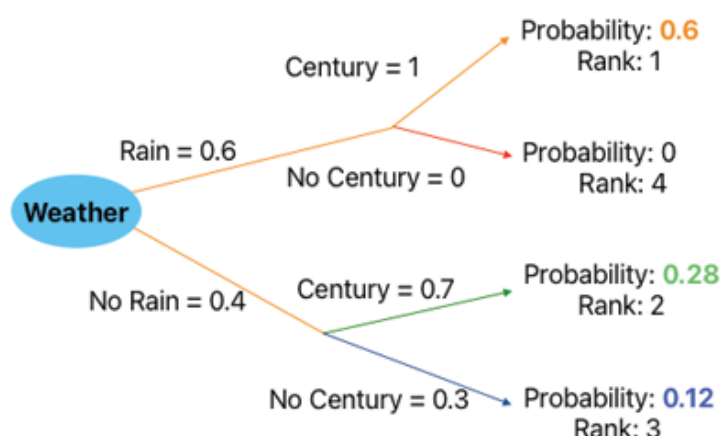


Figure 2.23: Conditional probability example

In [Figure 2.23](#), for each weather condition (rain vs no rain), the observed probability of a century is 1 and 0.7 respectively. The probability of rain on a match day is 0.6. Hence, the most likely outcome is rain and a century with a probability of 0.6 (0.6×1), and his probability of scoring a century in the next game changes to 0.28, if it does not rain.

Both the frequentist and Bayesian methods have their own strengths and weaknesses and have been debated for centuries. The frequentist approach is generally the most commonly practiced but Bayesian techniques have advantages in special cases when one is dealing with small sample sizes and can incorporate prior beliefs about the parameters.

Inferential statistics

Descriptive statistics is about finding a needle in the haystack with the right tools and understanding. This requires exploration, however, once the potential needle is found, it is equally important to determine if the piece of metal found is a needle. In other words, you need to do certain confirmatory tests to establish that the piece of metal found is actually a needle. In this scenario, you would compare with existing needle designs to be

able to conclude the search exercise with a certain level of confidence. Once you are confirmed, you can use this newly found needle's shape and size knowledge to make predictions in the future when you encounter similar objects.

In short, inferential statistics allows us to make inferences or generalizations about a population based on a sample of data with a certain level of confidence while descriptive statistics only allows us to explore the sample data at hand.

Interesting ways in which inferential statistics can be used are:

- Draw inferences about the population by using population samples
- Make hypotheses, collect and test evidence to make predictions
- Attach probability or likelihood with events or occurrences
- Risk and uncertainty management by quantifying uncertainty

Inferential statistics takes descriptive statistics forward by adding the concept of probability, thereby removing the sense of uncertainty and randomness in the historical data. It does this by working on a random sample of the population which is unbiased and representative of the population.

In this chapter, we will discuss hypothesis testing, which is an important inferential statistics method. In the absence of a holistic understanding of population parameters, it's only fair to start with a hypothesis about the population parameters.

For example, if a marketing department of a bank wants to run a promotion for existing credit card users to increase their spending, they cannot run this experiment on the entire portfolio of the customer. It may need to be more cost-effective or worse it may drive away customers. This experiment is typically conducted by randomly selecting a much smaller set of customers and analyzing the promotion treatment effect on this customer group. The goal is to understand the impact of promotion on larger population based on the impact it would have on the randomly selected representative sample. The impact on

the spending of sampled customers could be measured in terms of the change in average spending before and after the promotion was executed and followed up by estimating if that change is significant.

The purpose of the hypothesis test is to decide between two explanations:

- There does not appear to be a treatment effect of an event like a promotion. The difference between the sample and the population is probably due to sampling error.
- There is a treatment effect that is the difference between the sample and the population is too large to be explained by any sampling errors.

Some important terms to keep in mind while performing a hypothesis testing study:

- **Hypotheses:** We make two hypotheses about the population of interest.
 - **Null:** It states that the groups under study are the same. It is denoted by H_0 . This can mean the promotion has no effect on the group's spending before and after the promotion. H_0 : Average treatment group spending = \$500.
 - **Alternate:** It states that the groups under study are different. It is denoted by H_1 . Promotion has a positive effect on increasing the group's spending. H_1 : Average group spending is not equal to \$500.
- **Critical region:** The critical region consists of test statistics outcomes that lead to the rejection of the null hypothesis.
- **Confidence interval:** It is also known as the acceptance region that leads to acceptance of the null hypothesis.
- **One-tailed and two-tailed hypothesis testing:**
 - **One-tailed:** The critical region of the data, that would result in the rejection of the null hypothesis, is one-sided. For instance, this could mean H_0 : the average group

spending $\geq$ \$500 and H1: the average group spending $<$ \$500.

- **Two-tailed:** The critical region of the data, that would result in the rejection of the null hypothesis, is two-sided. This means that the average group spending is either greater or lesser than a range of values. For instance, H0: the average group spending = \$500 and H1: the average group spending not equal to \$500.

- **Type 1 and type 2 errors:**

- **Type 1 error:** Rejecting the null when it is true
- **Type 2 error:** Not rejecting null when it is false

The errors are made because of wrong judgment in deciding a cut-off value for making the decision about the null hypothesis.

The cut-off value is called the level of significance, Alpha (α). The alpha level also determines the risk level of making a type I error and is essentially a probability value between 0 and 1. Typical values chosen for α are .1, .05, or .01. So, for example, if $\alpha = .05$, there is a 5% chance that, when the null hypothesis is true, we will erroneously reject it.

- **Other causes of type I and type II errors:**

- By random chance, we may select a sample that is not representative of the population.
- Assumptions in our null hypothesis may be flawed.

- **P-value:** In the simplest sense, it is the probability that the null hypothesis is true. The lower the p-value, the greater the chances of the null hypothesis being rejected. For instance, If the p-value is .25, then there is a 25% chance of null being true implying there is a 25% chance that there is no change in average group spending of \$500. If the p-value is 0.05, then there is a 5% probability that there is no increase or decrease in the average group spending due to the new promotional campaign.

- We typically reject or fail to reject the null hypothesis based on an established α threshold for the experiment. If the obtained P-value is lower than the α , then the null is rejected. Lower values of $\alpha(0.01)$ imply that the experimenter set a high rejection window of 99% and plans to reject the null hypothesis when p-values lower than 1% are obtained. Alternatively, a p-value $< \alpha(0.01)$ indicates strong evidence against the null hypothesis, as there is less than a 1% probability the null is correct.
- **Sampling:** Extracting random observations from a bigger dataset (also known as population) in such a way that it is representative of the population. It can be done in multiple ways and the primary benefit is the reduction of computational cost and time. In data mining, a sample obtained using an appropriate sampling strategy can provide accurate results with a much lesser number of observations. Some key sampling methodologies are as follows:
 - **Simple random:** Simplest and most common method. The sample can be selected with or without replacement but each record has an equal probability of being selected
 - **Systematic sampling:** Sampling is done in a systematic manner like choosing every 10th record. It is used when there are presumably no patterns in the data
 - **Cluster sampling:** Data is sub-grouped as clusters and a random entire cluster is chosen as a sample. This is widely used in market research
 - **Stratified:** A sample point is selected from each subgroup in the population. Stratified sampling is done to accommodate the representation of every stratum or group into a sample.

The central limit states that for large-sized random samples of (usually $n > 30$) taken from a single population with mean μ and standard deviation σ , the sampling distribution of mean follows a normal distribution with mean μ and standard deviation $\sigma/\sqrt{n}$.

Refer to [Figure 2.23](#) to gauge the importance of sample size:

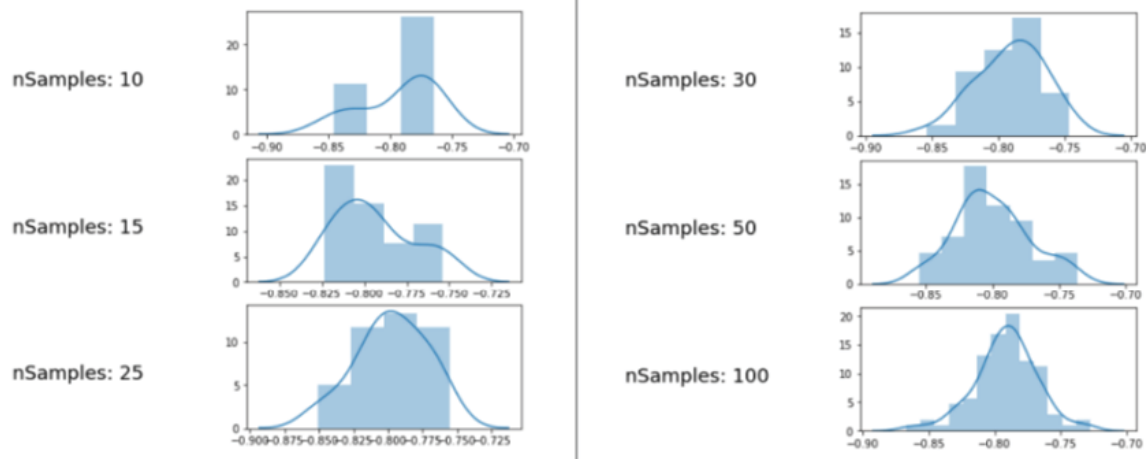


Figure 2.24: Sample size and normality

Degrees of freedom (df): Degrees of freedom are the number of independent data points that go into calculating the population estimate. For instance, if $X_1 + X_2 + X_3 = 30$, the number of independent data points that one needs to satisfy the equation is only two. Basically, you only have a choice with any two numbers and thereafter the third number is not free to be picked. For instance, once X_1 and X_2 are picked, X_3 is automatically known.

df is relevant here because it affects the critical value of various statistical tests by changing the shape of the probability distribution of the test statistics. As the **df** increases, the probability distribution becomes increasingly normal.

Now based on the understanding of the above key terms, let us dive into the four steps of performing a hypothesis test:

1. The first step is to state the two hypotheses so that only one can be right. The null hypothesis (H_0) and the alternate hypothesis (H_1) are stated. H_0 always states that the treatment has no effect. The population means that after the treatment is the same as before. Accepting the alternate hypothesis means that the treatment had an effect.
2. Secondly, formulate an analysis plan, which outlines how the data will be sampled and evaluated. Next, aim to define the critical region where the samples will have a p-value less than the threshold value alpha (α).

3. As a third step, decide on the right statistical test based on the type of variables and problem definition (one-tailed vs two-tailed and so on). All statistical tests generate a p-value indicating the level of test significance.
4. The fourth and final step is to analyze the results either by rejecting the null hypothesis or stating that the null hypothesis is plausible, given the data. If your predetermined level of significance is 0.01, this indicates that you are very conservative and trying to minimize the risk of incorrectly rejecting the null hypothesis (Type 1 error).

If the p-value is low, the Null must go

Please note, that the decision of the test procedure is one of the following:

- Reject H_0 implies the null hypothesis is false
- Fail to reject H_0 implies the null hypothesis is true

Note that the term here fails **to reject H_0** , the null hypothesis is assumed to be true unless proven to be otherwise. The null hypothesis gets the benefit of the doubt.

The alternative hypothesis has the burden of proof, that is it should have enough evidence from the data to say that the null hypothesis is false. Hence, it is preferred to say fail to reject H_0 , instead of accepting H_0 .

Hypothesis testing with commonly used statistical tests

Based on the understanding of the hypotheses testing needs, concepts, and procedural steps, let us try to understand the range of available tests. The primary criteria for picking one test among the many available are the variable type, its distribution, and the dataset structure. The validity of the statistical test relies on the sample size, which needs to be large enough to represent the population accurately.

All statistical tests can be broadly described as parametric or non-parametric based on the data normality assumption. This means that parametric tests are applicable to the data that follows a bell-

curve distribution, where the values are evenly spread out around the mean. Non-parametric tests do not make any assumption around the data distribution assumption. [Figure 2.25](#) describes the set of commonly used tests based on either testing the association between variables or comparison of mean/median:

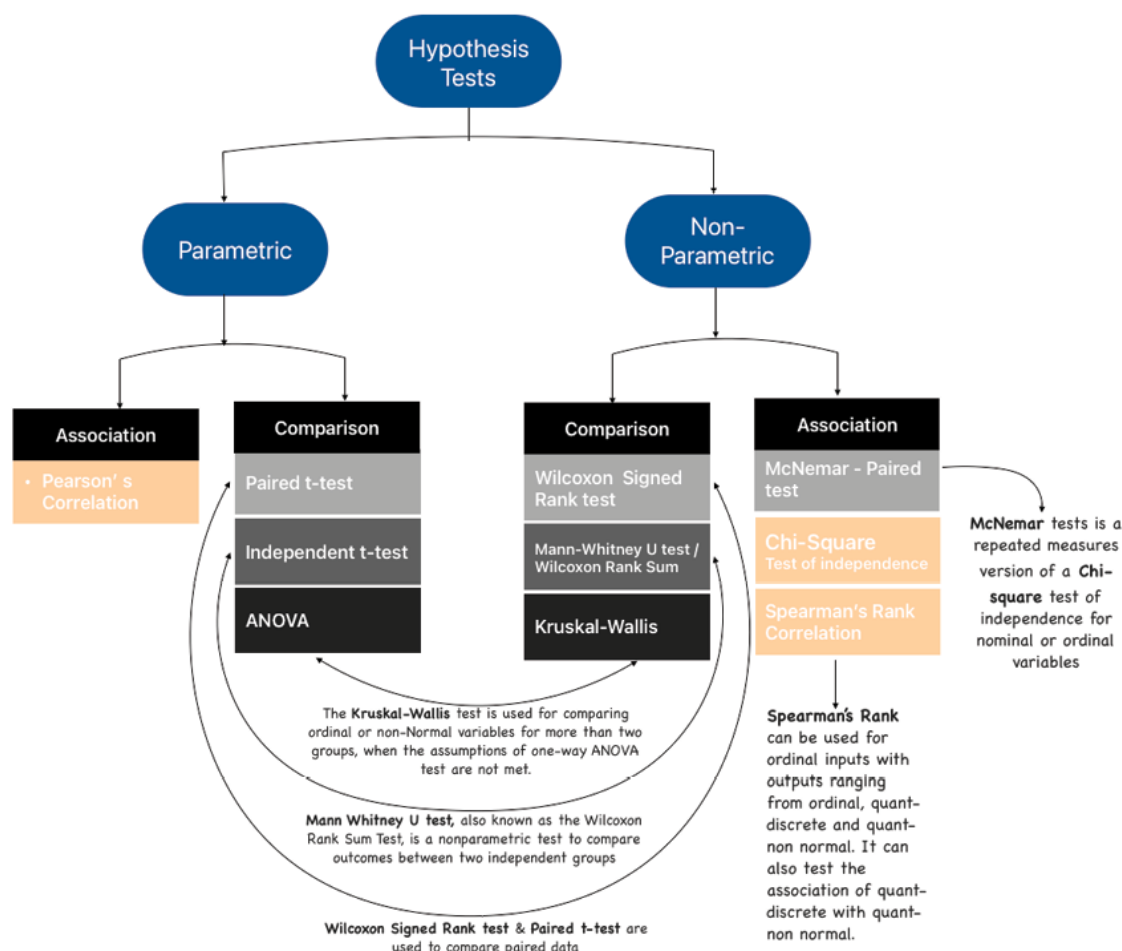


Figure 2.25: Hypothesis tests flow-chart

Some key points to remember:

- Parametric tests have more statistical power than non-parametric ones, meaning parametric tests are more likely to detect the difference if it exists.
- You can get away with the normality assumption of parametric tests if your sample is large enough ($n > 30$). Credit goes to the central limit theorem.

- While non-parametric tests do not make an assumption on data distribution, they might provide inaccurate results if the groups do not have the same variability in data.
- Non-parametric tests can work with data outliers and ordinal data. Non-parametric is a better choice if you decide to work with the median since it can assess the median and not just the mean.

Let us look at three statistical tests, across categories, in more detail. We will start with a statistical test of normality before digging deeper into the three tests:

- Shapiro-Wilk Test: Test for normality, this test helps in establishing if the particular sample has been generated from a Gaussian or normal distribution(Figure 2.19). This is particularly useful if we are trying to determine which among the parametric or non-parametric tests is applicable to a scenario.

The hypothesis to test whether the population is normally distributed is as follows:

H0: The sample has been generated from a normal distribution.

H1: The sample is not generated from a normal distribution.

Failing to reject H0, implies that the data is normally distributed.

The Shapiro-Wilk test is used as `scipy.stats.shapiro` (sample) to test the normality. The following is to test the normality of the variable `interest_rate` in the HMDA data.

Python code:

```
from scipy.stats import shapiro

stat, p_value =
shapiro(HMDA_IL_clean['interest_rate'])

# print the corresponding p-value
print('P-Value:', p_value)
```


Output:

P-Value: 0.0

- **t-test:** This is a parametric test that is conducted to compare means when population parameters (mean (μ) and standard deviation (σ)) are not known. There are 3 different ways this can be conducted.
 - **Paired t-test:** This test evaluates if there is a significant difference between the two values of the same variable before and after treatment. For instance, consumer spending before and after the promotion

The Python code to conduct a paired t-test for two populations means `scipy.stats.ttest_rel(Sample_1, Sample_2)`.

Python code:

```
#spend data before and after the campaign
```

```
spend_before_campaign=  
np.array([150,180,200,350,416,425,440])
```

```
spend_after_campaign=  
np.array([280,215,111,220,390,175,300])
```

```
# level of significance = 0.05
```

```
stats.ttest_rel(spend_after_campaign,  
spend_before_campaign)
```

Output:

```
Ttest_relResult(statistic=-1.4180124279929727,  
pvalue=0.20597132980745858)
```

Please note, since the p-value > 0.05 , we fail to reject the null hypothesis meaning the pre and post-spending do not change as an impact of the promotion. Please note that sometimes, we may actually be willing to fail to reject the null hypothesis. This means we were looking to prove that pre and post-spending do not change. We need to be careful with how we set up our hypothesis and experiment and interpret the output accordingly.

- **Independent t-test:** This is also referred to as a 2-sample or student t-test which is used to determine if there is a significant difference between the means of two population groups. For instance, you have two investment portfolios for long and short-term periods and want to check which of the two portfolios is giving you better returns or if the two returns are similar.

The first check for this parametric test is to ensure that the samples have equal variances. To test for equality of variances, we can use Levene's test.

Python code:

```
stats.levene(df0['interest_rate'],df1['interest_rate'])
```

Output:

```
LeveneResult(statistic=5.228508248346723,  
pvalue=0.02225549186313236)
```

At a 1% significance level (p-value>0.01), suggests that the variances are equal for the two interest rates.

Since both portfolios have equal variance, we can perform the equal mean test. Let us start by framing the hypothesis as follows:

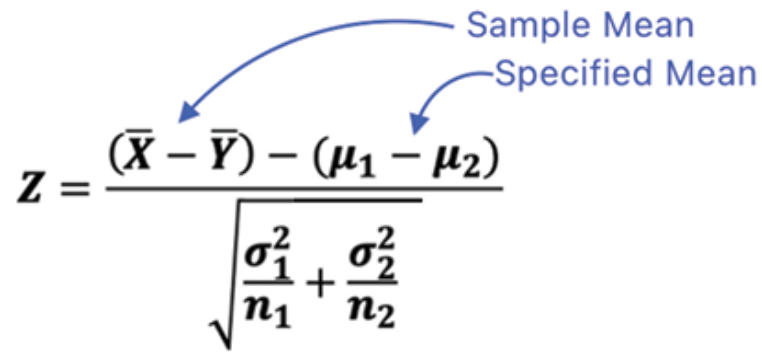
- H_0 : The two population means are equal (that is, $\mu_1 = \mu_2$)
- H_1 : The two population means are not equal μ_0 (that is, $\mu_1 \neq \mu_2$)

Please note, if you intend to check if one portfolio is better than another, then we do a 1-sample test and the hypothesis can be constructed as:

$H_0 : \mu_1 \geq \mu_2$

$H_1 : \mu_1 < \mu_2$

To use a 2-sample test, we will use Z-test statistics given by:



The image shows the Z-test formula with two blue arrows pointing to specific parts of the equation. One arrow points from the text 'Sample Mean' to the term $(\bar{X} - \bar{Y})$ in the numerator. The other arrow points from the text 'Specified Mean' to the term $(\mu_1 - \mu_2)$ in the numerator.

$$Z = \frac{(\bar{X} - \bar{Y}) - (\mu_1 - \mu_2)}{\sqrt{\frac{\sigma_1^2}{n_1} + \frac{\sigma_2^2}{n_2}}}$$

Figure 2.26: Z-test formula

A z-test is conducted if the population mean and the standard deviation are known. The Python code to conduct a Z test for two population means is

```
statsmodels.stats.weightstats.ztest(Sample_1,
Sample_2, value, alternative)
```

In case the population mean and the standard deviation is unknown, we use a t-test, the formula for which is shown in [Figure 2.27](#):

$$t = \frac{\bar{X} - \mu}{s/\sqrt{n}}$$

Figure 2.27: t-test formula

The Python code to conduct a 2-sample t-test for two populations (Interest rates offered to males vs females) means:

```
scipy.stats.ttest_ind(Sample_1, Sample_2)
```

Python code:

```
#2 sample t-test

from scipy.stats import ttest_ind

df0=HMDA_IL_clean[HMDA_IL_clean['derived_sex']=='Male']
```

```
df1=HMDA_IL_clean[HMDA_IL_clean['derived_sex']=='Female']
```

```
_,pval=ttest_ind(df0['interest_rate'],df1['interest_rate'])
```

Output:

```
p-value = 0.4319306829179371
```

- **One sample t-test:** This test compares the mean of a given population group with a known static number. For instance, the average loan amount offered to loyal customers is equal to \$175,000.

The Python code to conduct a 1-sample t-test for the population (loan amount) mean is different from 175,000 is.

Python code:

```
t_statistic, p_value =  
ttest_1samp(HMDA_IL_clean.loan_amount, 175000)  
print(t_statistic,p_value)
```

Output:

```
t_statistic = 30.52219927725251
```

```
p-value = 3.268745354241754e-196
```

- **Analysis of Variance (ANOVA) test:**

We will discuss one-way ANOVA which is used to test the differences in the mean of three or more groups. More specifically, it helps to assess if different levels of a predictor variable have a measurable effect on a dependent variable.

Since this is a parametric test, it requires that the dependent variable is normally distributed within each of the levels of the predictor variable and that the within-level variability is similar across all the levels.

Like all other methods, let us understand the null and alternate hypotheses related to this test.

H0: $\mu_1 = \mu_2 = \mu_3 = \dots = \mu_n$

H1: at least one group's mean is different from the rest.

Refer to the following figure:

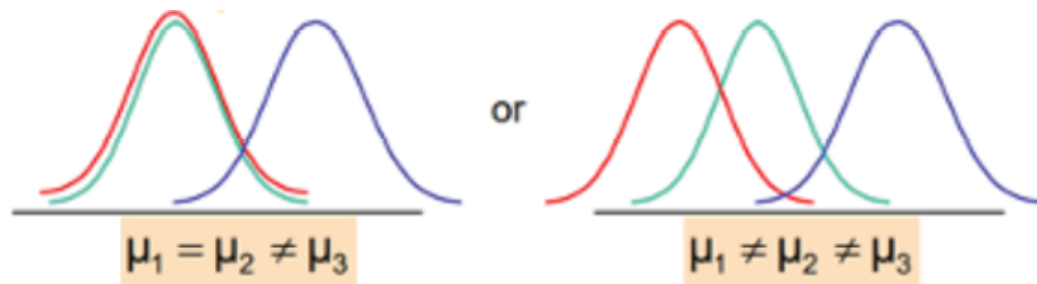


Figure 2.28: ANOVA - Difference in means

The limitation of ANOVA is that it does not tell us which of the many means we are comparing is different. But, fortunately, we can run a *tukey* test to identify which means are different from each other.

ANOVA performs the test of equality of more than two population means by analyzing the variance. ANOVA decomposes the total variation into two sources of variation: Between-group variance and Within-group variance. These two sources of variance subsequently explain the changes caused by these components in the dependent variable. The variation in the sum of squares of the dependent variable is caused only by variability due to the predictor levels, and the variability due to the random error.

*Sum of Squares (Total) $\equiv$ Sum of Squares (Predictor levels
i.e. Between-group variance)*

*+ Sum of Squares (Internal error i.e.
Within-group variance)*

$$= \sum n_j (\bar{X}_j - \bar{X})^2 + \sum (X_{ij} - \bar{X}_j)^2$$

Let us explore a situation that is best suited for ANOVA.

Tom is a facility manager at a regional bank branch that has 4 teller windows (A,B,C,D) in the branch. Tom wants to study whether all the windows have equal efficiency based on the total processing time to resolve customer queries.

Is it possible to test his claim?

The trivial solution is to conduct multiple t-tests. However, performing multiple t-tests has an effect on the type I error. As the number of t-tests increases the probability of at least one type I error increases. However, it is possible to test Tom's claim by using a one-way analysis of variance (one-way ANOVA) where the probability of type I error does not change.

Mean processing times at the 4 windows:

Window #	N	Mean (in minutes)	Overall Mean (in minutes)
A	10	10.65	9.30
B	10	10.43	
C	10	6.77	
D	10	9.37	

Table 2.2: Mean processing times by teller windows

Python code and output:

#Appending all the Window level data in one dataset

```
mean_time_df = pd.DataFrame()
```

```
df1 = pd.DataFrame({'Window': 'A', 'Mean_time':  
[10.65, 10.5, 10.7, 12.8, 15.5, 7.5, 7.8, 11.5,  
10.65, 8.9]})
```

```
df2 = pd.DataFrame({'Window': 'B', 'Mean_time':  
[10.43, 10.6, 10.7, 12.75, 13.5, 8.5, 6.8, 11.5,  
10.65, 8.9]})
```

```
df3 = pd.DataFrame({'Window': 'C', 'Mean_time':  
[6.7, 5.3, 4.7, 10.2, 8.5, 6.51, 4.7, 8.5, 4.6,  
8]})
```

```
df4 = pd.DataFrame({'Window': 'D', 'Mean_time':  
[8.7, 9.5, 8.7, 10.2, 8.7, 7.5, 12.7, 8.5, 9.5,  
9.7]})
```

```

mean_time_df = mean_time_df.append(df1)
mean_time_df = mean_time_df.append(df2)
mean_time_df = mean_time_df.append(df3)
mean_time_df = mean_time_df.append(df4)
#Executing ANOVA using statsmodels library
import statsmodels.api as sm
from statsmodels.formula.api import ols

mod = ols('Mean_time ~ Window', data =
mean_time_df).fit()
aov_table = sm.stats.anova_lm(mod, typ=2)
print(aov_table)

```

Output:

	sum_sq	df	F	PR(>F)
Window	95.06786	3.0	8.28829	0.000254
Residual	137.64170	36.0	NaN	NaN

The overall F-statistic in the ANOVA table above is a way to quantify the ratio of the between-group variation with the within-group variation. The larger the F-statistic, the greater the variation between group means relative to the variation within the groups indicating there is a difference between the group means. We can see in this example that the p-value that corresponds to an F-statistic of 8.2882 is .00254.

Since this value is less than $\alpha = .05$, we can reject the ANOVA null hypothesis and conclude that the four windows have similar processing times.

- **Chi-squared (χ^2) test:** This is the only non-parametric test we are going to discuss in this chapter. It is used to test the

association between two categorical variables. The chi-square tests are used to test:

Let us study the independence of two categorical variables:

A branch operations manager wants to study whether the happiness quotient of customers visiting the bank is related to the time spent at the branch during the visit. He collects data from 500 customers to find if is there enough evidence to claim that they are related.

H0: Satisfaction is related to the time spent at the branch

H1: Satisfaction is not related to the time spent at the branch

Observed frequencies:

	Visit time < 20 Min	Visit time 20 - 30 Min	Visit time > 30 Min
Satisfied	25	50	75
Unsatisfied	100	150	100

Table 2.3: Frequency table by customer satisfaction

We choose the Chi-squared test of independence in this case because time spent and satisfaction are both categorical variables and we are talking about a randomly selected group of 500 customers from the total customer base.

The Chi-squared (χ^2) test statistic for the chi-square test of independence is as follows:

$$\chi^2 = \sum_{i=1}^k \frac{o_i^2}{e_i} - N$$

The diagram shows the equation $\chi^2 = \sum_{i=1}^k \frac{o_i^2}{e_i} - N$. Three blue arrows point from text labels to parts of the equation: 'Observed Frequency' points to o_i , 'No of Observations' points to N , and 'Expected Frequency' points to e_i .

Figure 2.29: Chi-square test of independence equation

O is the observed frequency and E is the expected frequency.

*Expected Frequencies (Row Total * Column Total / N) are as shown below:*

	Visit time < 20 Min	Visit time 20 - 30 Min	Visit time > 30 Min
Satisfied	37.5	60	52.5
Unsatisfied	87.5	140	122.5

Table 2.4: Frequency table of expected frequencies

Based on the above data and the formula, $\chi^2 = 22.10$

Python code:

```
Visit_Time_array = np.array([[25, 50, 75],[100,
150, 100]])

chi_sq_Stat, p_value, deg_freedom, exp_freq =
stats.chi2_contingency(Visit_Time_array,correction=
False)

print('Chi-square statistic %3.5f P value %1.6f
Degrees of freedom %d' %(chi_sq_Stat,
p_value,deg_freedom))
```

Output:

```
Chi-square statistic 22.10884 P value 0.000016
Degrees of freedom 2
```

The goodness of fit is similar to the test of independence in terms of the formula but the goal is to determine if the sample is likely to be from a specific distribution or not. We compare the sample distribution (observed values) with the specific population distribution (expected values).

Chi-square study with population variance: Let us understand this further using an example where we compare the population variance with the hypothesized variance.

A branch operations manager wants to arrange a space for a waiting area for the customers. The manager claims that 45% of adult males, 30% of females, 15% of senior citizens,

and the remaining others visited the branch in a week. However, it was observed that in a sample of 110 people, 35 were adult males, 20 were Females, 30 were Others and 25 were senior citizens. Can we test the manager's claim at a 95% confidence level?

Let us follow the four-step process detailed earlier to conduct the hypothesis test. Here, in step 1, the hypothesis will be defined as:

- H0: The manager's claim is correct
- H1: The manager's claim is false

In step 2, let us examine the available data.

	Manager Claimed Frequency	The frequency expected from 110 customers (ei)	The frequency observed from 110 customers (Oi)
Male	45%	0.45 x 110 = 49.5 = ~50	35
Female	30%	0.30 x 110 = 33	20
Senior Citizens	15%	0.15 x 110 = 16.5 = ~17	25
Others	10%	0.10 x 110 = 11	30

Table 2.5: Frequency table of customer visits

Step 3 is about conducting the non-parametric test chi-Squared since it does not require the normality assumption, and this is about the categorical variable customer type. Chi-squared is estimated using the below equation:

$$\chi^2 = \sum_{i=1}^k \frac{(O-E)_i^2}{E_i}$$

The diagram shows the equation $\chi^2 = \sum_{i=1}^k \frac{(O-E)_i^2}{E_i}$. A blue arrow points from the text 'Observed Frequency' to the 'O' in the numerator. Another blue arrow points from the text 'Expected Frequency' to the 'E' in the denominator.

Figure 2.30: Chi-Squared equation

$$= [(35-50)^2/50 + (20-33)^2/33 + (25-17)^2/17 + (30-11)^2/11]$$

Python code:

```
import scipy.stats as stats

import scipy

observed_values      = scipy.array([35, 20, 30, 25])
n                    = observed_values.sum()

expected_values      = scipy.array([n*0.45, n*.30,
n*0.1, n*0.15])

chi_square_stat, p_value =
stats.chisquare(observed_values,
f_exp=expected_values)

print('At 5 %s level of significance, the chi-
square stat is %1.7f' %('%', chi_square_stat))

print('At 5 %s level of significance, the p-value
is %1.7f' %('%', p_value))
```

Output:

```
At 5 % level of significance, the chi-square stat
is 46.5656566
```

```
    At 5 % level of significance, the p-value is
0.0000000
```

Step 4 is about interpreting the results. Based on the p-value of the chi-squared statistics, we can determine if the null should be rejected or not. Since the confidence interval is 95%, the p-value should be less than 5% for the null to be rejected. We reject null in this case since $p=0.00$.

In this section, we have limited the discussion of tests to a few of the most commonly used ones based on the key selection criteria such as the number of samples and variable types. We urge readers to experiment with other similar tests (as highlighted in [Figure 2.25](#)) and compare results. All the detailed Python experiments on HMDA data from this chapter are available in the GitHub repository.

Introduction to Time Series Data

A time series is defined as a set of quantitative observations arranged in chronological order. We generally assume that time is a discrete variable. Time series observations may be collected for all the points in time. For instance, a region's rainfall is measured at the yearly level. Companies measure their profits and losses every quarter while sales volumes are measured monthly or weekly. Stock market movements are followed daily. All of the preceding examples are time series data. Even minute-by-minute movement of stock value can be modeled using time series. The two most important characteristics of time series data are that all observations must be collected at the same interval and continuous.

A time series is made up of four structural components. These are:

- **Trend:** The general direction in which the time series is moving.
- **Seasonality:** Repeating signal cycles within the time series.
- **Level:** The average value of the time series.
- **Noise:** Random components within the series.

The following figure demonstrates a time series of the daily closing value of an NYSE stock. The figure also showcases other components of the time series:

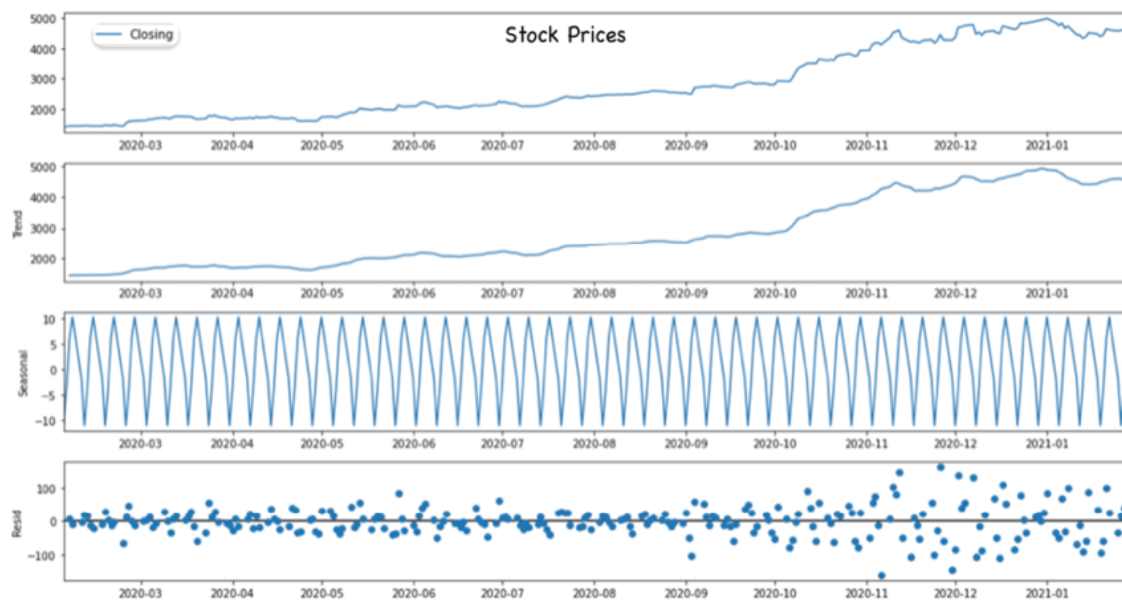


Figure 2.31: Time series component decompositions

Additionally, the **Augmented Dickey-Fuller (ADF)** test is performed to test the stationarity of the time series. Stationary time series has a constant mean and variance throughout the entire length of the time period under study. The null hypothesis of this test evaluates whether there is a unit root present in the time series. $H_0: \alpha=1$. One unit root means that there is one level of differencing needed to make the series stationary:

```
from statsmodels.tsa.stattools import adfuller
observations= df.values
test_result = adfuller(observations)
print('ADF Statistic: %f' % test_result [0])
print('p-value: %f' % test_result [1])
print('Critical Values:')
for key, value in test_result[4].items(): print('\t%s:
%.3f' % (key, value))
```

Output:

ADF Statistic: -0.007528997627654177,

p-value: 0.9579049966158176,

Critical Values:

'1%': -3.455952927706342,

'5%': -2.8728086526320302,

'10%': -2.572774990685656

Based on the preceding output the H_0 cannot be rejected, hence the series is considered non-stationary with at least one unit root present.

Exploratory data analysis: HMDA case study

This section will summarize the descriptive and inferential analysis work that was performed on the HMDA data throughout the chapter. After cleaning the data for outliers across the three key variables loan amount, interest rate, and income, we performed the following steps:

- **Box plot with and without outliers (loan amount).** This can be done with other continuous variables.
- **5 number theory:** use the loan amount for the outlier, min-max, etc. to understand the spread of the variable
- **Loan amount histogram with a normal line:** SD, variance, mean, median, kurtosis, and skewness to understand the distribution using various summary statistics.
- **Bivariate study:** Race by Loan amount (Box plot) and Interest Rate by Loan Amount (Scatter plot)
- **Inferential statistics:** We visualized some data patterns using the descriptive statistics work performed in previous steps, and then confirmed some observations using hypothesis testing at designated significance levels.
- Binomial, Bernoulli, Poisson code, and graph for various use cases

- T-test with 1 and 2 Sample tests for evaluating differences in interest rates between genders and ANOVA to examine processing times at teller windows.

While there is no set pattern for data understanding, it is always a good practice to start with exploration and end with confirmation before moving on to more advanced data mining techniques. Building an understanding of your analysis dataset and exploring underlying patterns is both an art and a science. The balance is learned from exposing oneself to varying datasets.

Conclusion

Statistics as a field of study is foundational to data mining and helps us make sense of data by applying various descriptive and inferential statistics methods. Descriptive statistics allow us to summarize and describe the characteristics of a dataset either graphical or non-graphically using measures such as mean, median, mode, standard deviation, variance, and range. These measures provide insights into the central tendency, variability, and distribution of the data.

On the other hand, inferential statistics allows us to make inferences or predictions about a population based on a sample of data. This is done by using probability theory, hypothesis testing, and confidence intervals. Probability theory is used to calculate the likelihood of an event occurring, while hypothesis testing is used to determine if there is a significant difference between two or more groups in a dataset. Confidence intervals, on the other hand, give us an estimate of the range of values within which a population parameter is likely to fall.

Hence, understanding descriptive statistics and inferential statistics methods is essential for making informed decisions based on data. By using these methods, we can analyze and interpret data, make predictions, and draw conclusions about a population based on a sample. Probability theory is a key component of these methods and provides a foundation for understanding the uncertainty and variability inherent in data analysis.

Points to remember

- Statistics involves collecting, analyzing, interpreting, presenting, and organizing data to make informed decisions.
- Explore descriptive statistics, which involves summarizing and describing the main features of a dataset. This includes graphical and non-graphical EDA techniques.
- Grasp the basics of probability theory, which provides a foundation for statistical inference. Understand key probability distributions, including discrete and continuous distributions with their examples.
- Both frequentist and Bayesian probability estimation approaches are widely used in statistics, and the choice between them often depends on the nature of the problem, the available data, and the preferences of the analyst.
- Learn about inferential statistics, which involves making inferences about a population based on a sample. Key concepts include hypothesis testing leveraging various statistical tests
- Gain familiarity with the unique challenges and characteristics of time series data, which refers to observations recorded over time.
- Keep developing a strong statistical foundation for data exploration and inferencing.

CHAPTER 3

Digging into Linear Regression

Introduction

Linear regression is the most popular and widely used technique in the data mining world. The methodology is simple enough to implement, explain, and interpret and has few assumptions. However, for best results, the practitioner must follow a straight line (pun intended) to avoid pitfalls.

Regression is referred to as an advanced statistical technique that helps quantify the relationship between two variables, which have a cause-effect relationship. Linear regression comes from a family of statistical techniques known as Regression analysis which was considered *trivial* by its inventor and the legendary mathematician *Carl Friedrich Gauss*. The statistics historian *Stephen M. Stigler* calls regression the *automobile* of statistical analysis. In his words - ... *despite its limitations, occasional accidents, and incidental pollution, it and its numerous variations, extensions, and related conveyances carry the bulk of statistical analyses, and are known and valued by nearly all.*

Structure

The chapter will cover the following topics:

- Linear regression
 - Background
 - Under the hood
 - Challenges and assumptions including multi-collinearity
 - Detailed EDA
 - Feature selection
 - Regression execution and results
 - Regression result interpretation
- Optimization algorithm
 - Gradient descent
- Regularization
 - Lasso regression
 - Ridge regression
 - Elastic-Net regression
- MLflow introduction: Need and implementation
 - MLflow experiment tracking
- Case study : Predicting credit card spending

Objectives

This chapter discusses one of the most popular data mining techniques known as Linear regression. While this is the most often talked about and a simple methodology for making predictions using data, it can easily deliver inaccurate results if the numerous assumptions are not

addressed before starting the modeling process. We will talk about those and various metrics to track the model's performance. The parameter estimation using different regularization and loss optimization strategies will also be discussed. In the spirit of this book's model risk mitigation theme, we will explain some key concepts using a case study on credit card spending data using MLflow. This Python library helps with model risk management by incorporating aspects of reproducibility and scalability in machine learning model training.

Linear regression

Linear regression is like finding the best-fitting path through the dots of existing reality, using this path next to predict what comes is the future. Like in real life, finding this path that best summarizes the past journey with minimum error is going to take some trial and error. Let us explore how that can be done mathematically.

Background

Regression analysis is a tool to investigate the relationship between two variables hypothesized as a cause-effect relationship. This differs from correlation analysis, a technique used to quantify only the associations between two continuous variables. Please note that correlation only means an association, and there is not necessarily a cause-effect relationship between the variables of interest.

For example, there could be an observed correlation between the number of bank branches in a region and the average income of the residents in that region, but this should only be treated as an association between the two variables. It does not mean that having more bank branches causes an increase in average income or vice versa. Other factors, such as population density, business opportunities,

or historical trends, could be influencing both the number of bank branches and the average income independently. A regression analysis with these loose associations is not advised and should only be performed when there is a strong directional relationship between the two variables. A good relationship example is a high credit score causing low interest rates loans. Please note:

Regression quantifies hypothesized relationships

In a simple linear regression analysis, the two variables are referred to as Y or dependent or the outcome variable, and X , the independent or explanatory variables. As highlighted above, X has to explain Y to be called an explanatory variable. Regression analysis, as we know it today, is primarily the work of *R.A. Fisher*, who combined the work of his predecessors *Gauss* and *Pearson* to develop a fully realized theory of the properties of least squares estimation. This technique helps identify a line of best fit, also referred to by *Pearson* as the *regression line*. Hence, this method of least squares and regression became somewhat synonymous.

Since this important development, there have been numerous variations of regression, including logistic regression, nonparametric regression, Bayesian regression, and regression incorporating regularization. [Chapter 4, Exploring Logistic Regression](#), will cover the details of logistic regression but it is important to understand regularization in the context of linear regression before moving to other variations.

Under the hood

A simple linear regression helps quantify the relationship between two variables, one of which explains the change in another continuous dependent variable. However, how does the quantification of this relationship between the

continuous dependent (Y) and independent (X) variables happen for linear regression and, in general, any **Machine Learning (ML)** algorithm?

A guiding theme of this book is that all ML outcomes (based on a $Y \sim X$ relationship) are obtained by different combinations of the following:

- **Data:** A set of data points that define the dependent (Y or $f(x)$) and independent (X) variables
- **Model:** A mathematical model or an assumed relationship form that is an outcome of many experiments. Outcome Y as a function of X can be written as:

$$Y \sim f(X)$$

- **Loss function:** A loss function is used to measure the quality of a particular experiment. The loss value quantifies the discrepancy between the true label y and the predicted label $f(x)$ or $\hat{y}$. A small (close to zero) loss value indicates a low discrepancy between the predicted label and the true label of a data point.

The basic principle of any ML method can then be formulated as learning (finding) an experiment outcome that incurs a minimum learning loss (error) L from a given set of experiments for any set of data points. This is termed as a model that is closest to reality based on learnings from a set of data points.

Each of these three components involves a certain design consideration for data, model, and loss. In this chapter, we will detail the design considerations for the linear regression model.

Let us start by talking about the data considerations for linear regression. If the goal of a bank is to quantify the factors that drive credit card spending of its customers, it

should start by hypothesizing and collecting data for such a study. For instance, some potential drivers of spending could be income, employment status, and more.

From a model perspective, the relationship between spend and income is that of a cause and effect wherein it is reasonable to assume that higher income will drive higher spending and vice versa. However, given the data for a particular bank, what does this relationship look like? Let us plot some data points on a scatter plot, as shown in [Figure 3.1](#):

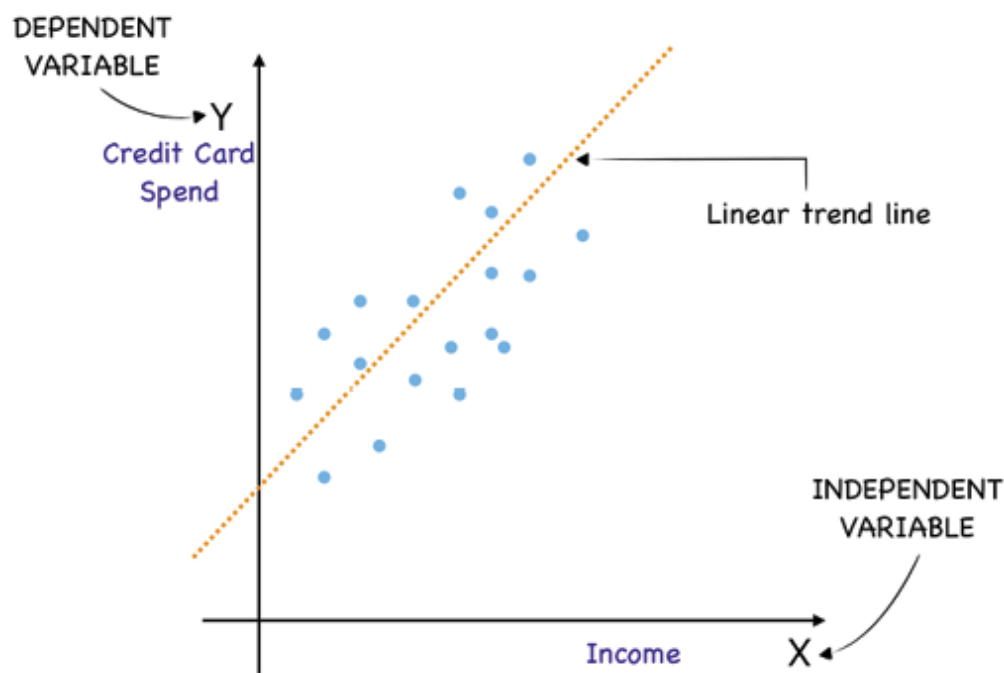


Figure 3.1: Spending vs Income Scatter plot with linear trend line

As we can see there is some sort of linear relationship between the two variables. Our goal is to find the best-fitted line that explains the relationship and can be used to predict future spending based on the customer's income.

This line is represented using the following equation which is:

$$y_i = \beta_0 + \beta_1 \cdot x_i + e \text{ (e is the random error component)}$$

- Where y is the dependent variable (example: credit card spending)
- x_i is the independent variable (example: customer income)
- β_0 is the intercept
- β_1 is the slope of the line

The next step in the process is to identify the best-fitted line. This requires finding the β_0 and β_1 by defining and minimizing a loss function to obtain a line of best fit. In Linear Regression, our loss function will be **Mean Squared Error (MSE)**, which can be seen in [Figure 3.2](#) as follows:

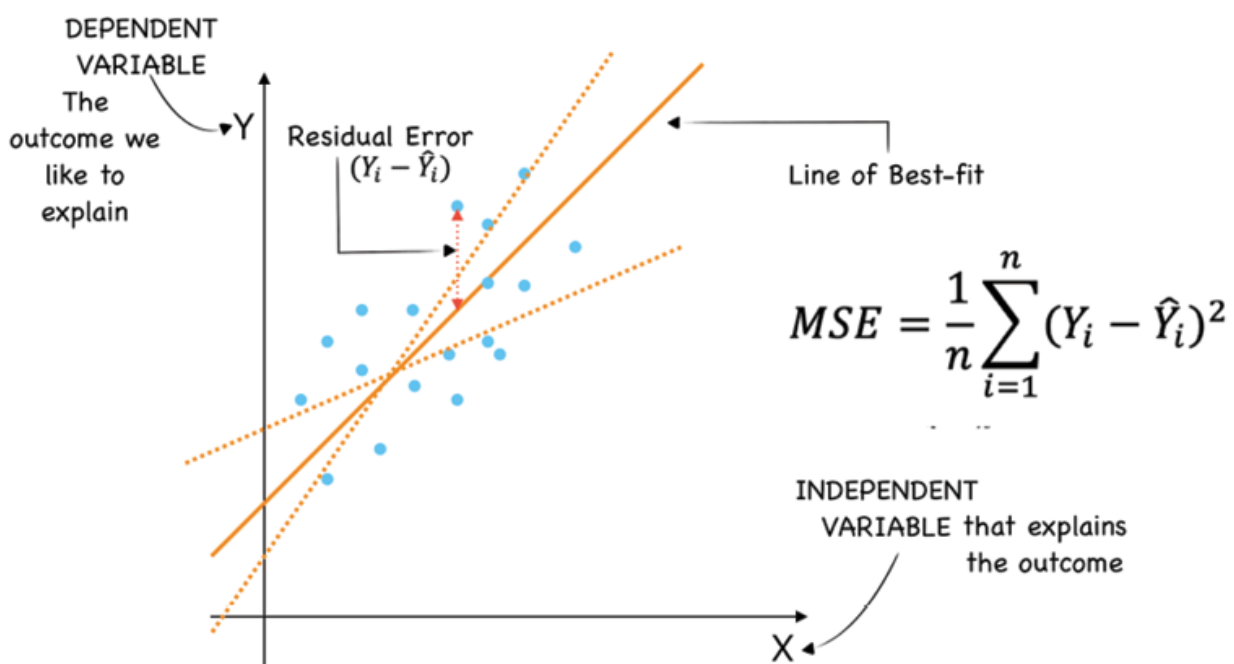


Figure 3.2: Best fit line based on the MSE

We can observe in [Figure 3.2](#), that the chosen line of best-fit has a residual error corresponding to each data point (some large and others small). This error is the difference between the actual data point and the predicted point on the line. The endeavor of identifying the best-fit line would require us to

choose a line that minimizes the sum of all the squared residual errors corresponding to each data point. Squaring is necessary to remove any negative signs and to give more weight to larger differences. This sum of squared errors divided by the number of data points (customer incomes) is known as the MSE and must be calculated for every line we try to fit. [Table 3.1](#) shows a hypothesized MSE calculation example based on [Figure 3.2](#).

S.No.	X	Y	$\hat{Y}_1$ (Dotted Line)	$\hat{Y}_1$ (Best Fit)	Residual-1 (Y- $\hat{Y}_1$)	Residual-2 (Y- $\hat{Y}_2$)
1	9	23.5	23	24	0.5	-0.5
2	8	21	20	23	1	-2
3	3	8.5	8	9	0.5	-0.5
4	10	26	24	25	2	1
5	9	23.5	23	23.2	0.5	0.3
6	4	11	11.5	11.1	-0.5	-0.1
7	9	23.5	24	24.5	-0.5	-1
8	6	16	17	16.1	-1	-0.1
9	7	18.5	18.3	18.2	0.2	0.3
10	8	21	20.3	20.8	0.7	0.2
	MSE				0.778	0.674

Table 3.1: Best fit line identification based on the MSE calculation

However, technically, there are infinite lines that can pass through these data points on the scatter plot in [Figure 3.2](#); how do we find the best-fit line?

A naïve approach could be trying all permutations and combinations of β_0 (intercept) and β_1 (slope) and tracking

MSE, but that is time-consuming and is not a computationally sound method. Also, this problem can grow multifold when we solve a multiple linear regression problem with more than one independent variable (x_1, x_2, x_3 , and so on), leading to multiple betas ($\beta_1, \beta_2, \beta_3$, and so on).

Note: The choice for the loss function should take into account the computational complexity of searching for the best model fit with minimum loss.

There are two common approaches to minimizing the loss function and finding the best-fit line in the case of linear regression. Let us discuss the first approach, **Ordinary Least Square (OLS)**.

The goal in OLS is to **minimize the cost function**: $J(\beta_0, \beta_1) = (y - \hat{y})^2$ or $(y - (\beta_0 + \beta_1 * x_1))^2$

Referencing some calculus will tell us that minima for the above function will happen when the derivative of the above cost function is equal to 0.

So let us perform the following three steps to obtain values of β_0 and β_1 that give the minimum value of $(y - (\beta_0 + \beta_1 * x_1))^2$:

1. Calculate the partial derivative of the above equation w.r.t β_0 and β_1 .
2. Set the partial derivatives to 0 to get minima. For instance, partial derivative w.r.t. β_1 set to zero:

$$\sum_{i=1}^m x_{1i} * (y - (\beta_0 + \beta_1 * x_{1i})) = 0$$

Solving for β_1 and β_0 , we get:

$$\beta_0 = \bar{Y} - \beta_1 \bar{X}$$

$$\beta_1 = \frac{\sum (X_i - \bar{X})(Y_i - \bar{Y})}{\sum (X_i - \bar{X})^2}$$

Please note that the optimal value of β_1 is the ratio of the covariance between y and x_1 to the variance of $\mathbf{x_1}$. $\bar{X}$ and $\bar{Y}$ are the means of two variables X and Y .

The second approach for finding the minima of the cost function J is using gradient descent. This approach updates the parameters β_0 and β_1 iteratively in the direction that reduces the cost function. We will learn more about it in the section ahead.

Challenges and assumptions including multi-collinearity

The linear regression equation is about fitting a straight line for a set of data points in an n -dimensional space. This is in the case of multiple linear regression with more than two independent variables that potentially explain the dependent Y .

However, some key data-related assumptions and challenges need to be examined as we perform the line-fitting exercise. Most real-life datasets will not pass all the examinations. In that case, we will have to work on them by transforming the data, performing feature engineering, tweaking the model, or even choosing a different algorithm to model the data. Please be mindful of the **DML (data, model, and loss function)** framework during a model development exercise. This framework will help you stay focused as you try to make the right modeling choices.

Let us explore the key assumptions and challenges which mostly are around the residuals (Observed value – Expected value). For instance, if we are trying to predict the credit card spending of bank customers using historical data, the six assumptions that must be tested on the residuals are as listed below:

- **Residuals' normality:** They should follow a normal distribution so that the estimators or the betas derived from the OLS are not biased. Residuals following a normal distribution imply that the majority are concentrated around a single mean value and the sum of the residuals is zero when an intercept is included. This implies the prediction error in credit card spending is fixed around a number and not random.
- **Residuals' variability:** They should be homogenous with constant variance. This implies that the errors or the residuals are within a certain range of the single mean value and do not increase or decrease with a change in the values of the independent variable. The constant residual variance is also known as **homoscedasticity** and implies the spending prediction error does not change with the values of the input variable.
- **Residual outliers:** The presence of any outlier in the residual implies that the model is not able to predict the dependent for certain values of independent variables accurately. The model output should be consistent.
- **Residuals' independence:** No autocorrelation between residuals implies that we cannot use one error to predict another. Ideally, all the prediction power should be in the non-error ($\beta_0 + \beta_1 \cdot x_1$) part of the regression equation, and errors should be completely

random. It simply means that you should not be able to see patterns in the residual plots.

- **Multicollinearity (MC):** The presence of a high level of MC between the independent variables makes it impossible to estimate the effect of each variable over the dependent variable with precision. In other words, MC occurs when an independent variable can be explained as a linear combination of other independent variables and this in turn implies less new information available for the model to explain the dependent. There should be no (low) MC among the independent variables.
- **Linearity:** The functional form of regression is correctly specified that is there exists a linear relationship between the independent variables and the dependent variable Y. In other words, changes in the independent variables will have the same marginal effect (constant slope) on the dependent regardless of their value.

For now, let us keep these concepts in mind until we deal with them in the upcoming sections of this chapter. While working on the credit card spending prediction case study, we will explore these concepts again.

Detailed EDA

Next, we will understand the various linear regression model development concepts using a case study on the credit card spending dataset. A global bank would like to understand the factors driving credit card spending to determine credit limits for its customers. In order to solve this problem, the bank conducted a survey of 5000 customers and collected data. Let us start by exploring the dataset to assess any data anomalies, patterns, and the presence of linear relationships between Y and X variables.

Dataset description

A high-level dataset view and summary can be obtained by `#Change default display`. These will help you to see all records as well as columns in the data set.

```
#Importing dataset as "data"
```

```
data=pd.read_excel("./Data Set.xlsx")
```

```
#Using head(), you can see first 5 rows
```

```
data.head()
```

```
pd.set_option('display.max_rows', 500)
```

```
pd.set_option('display.max_columns', 500)
```

```
pd.set_option('display.width', 1000)
```

```
#By directly calling the data set by name will show  
you the first and last 5 rows together
```

Data

Output:

	custid	region	townsize	gender	age	agecat	birthmonth	ed	edcat	jobcat	union	employ	empcat
0	3964-QJWTRG-NPN	1	2.0	1	20	2	September	15	3	1	1	0	1
1	0648-AIPJSP-UVM	5	5.0	0	22	2	May	17	4	2	0	0	1
2	5195-TLUDJE-HVO	3	4.0	1	67	6	June	14	2	2	0	16	5
3	4459-VLPQUH-3OL	4	3.0	0	23	2	May	16	3	2	0	0	1
4	8158-SMTQFB-CNO	2	2.0	0	26	3	July	16	3	2	0	1	1
...	...	...	...	...	...	...	...	...	...	...	...	...	...
4995	3675-GZFGOT-QJN	2	2.0	0	68	6	January	10	1	1	0	24	5
4996	4699-LEPCCE-3UD	3	3.0	0	51	5	May	14	2	1	0	6	3

Figure 3.3: Truncated view of the dataset

#First we will check the total number of records in data

```
print("Number of Records:", data.shape[0])
```

#Total number of variables in data

```
print("Number of Variable:", data.shape[1])
```

```
print(data.info())
```

Output:

```
Number of Records: 5000
Number of Variable: 130
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 5000 entries, 0 to 4999
Columns: 130 entries, custid to response_03
dtypes: float64(31), int64(97), object(2)
memory usage: 5.0+ MB
None
```

Figure 3.4: Dataset summary

We then perform other data cleaning steps such as dropping duplicates using:

```
data.drop_duplicates(inplace=True)
```

We can check datatypes of imported data columns using:

```
data.dtypes
```

Convert any categorical variable that is wrongly imported as a continuous using:

```
data[categorical_var]=data[categorical_var].astype(
object)
```

Please note that `categorical_var` is the list of all categorical variables.

Missing value treatment

There is no single good methodology for dealing with missing values in data. The treatment would differ by different solutions for data imputation depending on the kind of problem — Time series Analysis, ML, Regression, and more. There are two broad-level missing value treatments available as seen in [Figure 3.5](#):

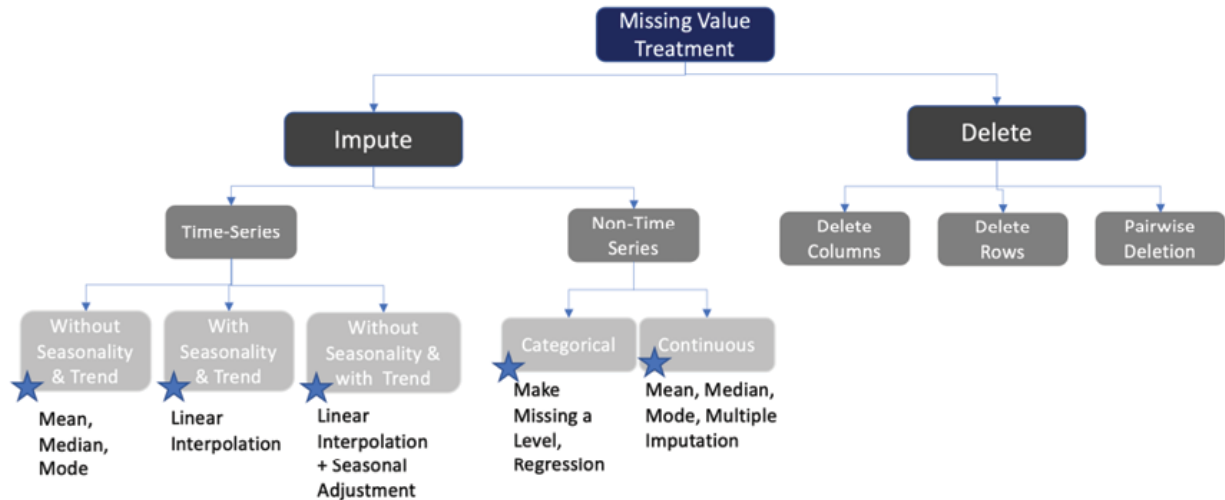


Figure 3.5: Missing value treatment decision tree

For this study, let us start by identifying the NULL values within each variable.

#checking missing values in percentage terms

```
(data.isnull().sum()/len(data)*100).sort_values(ascending=False)
```

Output:

lnwiremon	73.12
lnwireten	73.12
lnequipmon	65.92
lnequipten	65.92
lntollten	52.44
lntollmon	52.44
lncardten	28.44
lncardmon	28.38
lnlongten	0.06
longten	0.06

Figure 3.6: Sample missing value percentages

The total number of variables after dropping variables with >50% missing and the `custid` variable:


```
print ('Number of numerical variables    :-  
{}}'.format(len(numerical_var)))  
  
print ('Number of categorical variables :-  
{}}'.format(len(categorical_var)))
```

Output:

Number of numerical variables:- 39

Number of categorical variables:- 84

Outlier analysis

Statistically, outliers come from a different distribution than the rest of the samples in a variable. They present statistically significant abnormalities than most data points in the training dataset. They can be either legit or simply data entry errors. For instance, customer income variables can have regular household incomes along with a CEO's income in millions of dollars. This one value is not a data entry error and should be treated accordingly.

For the credit card spending dataset, let us explore the dependent variable `total_spent` for outliers using a box plot.

```
fig, ax = plt.subplots(figsize=(15,5))  
sns.boxplot(data['total_spent'],ax=ax)  
plt.grid()  
plt.show()
```

The output for the code is shown in [Figure 3.7](#):

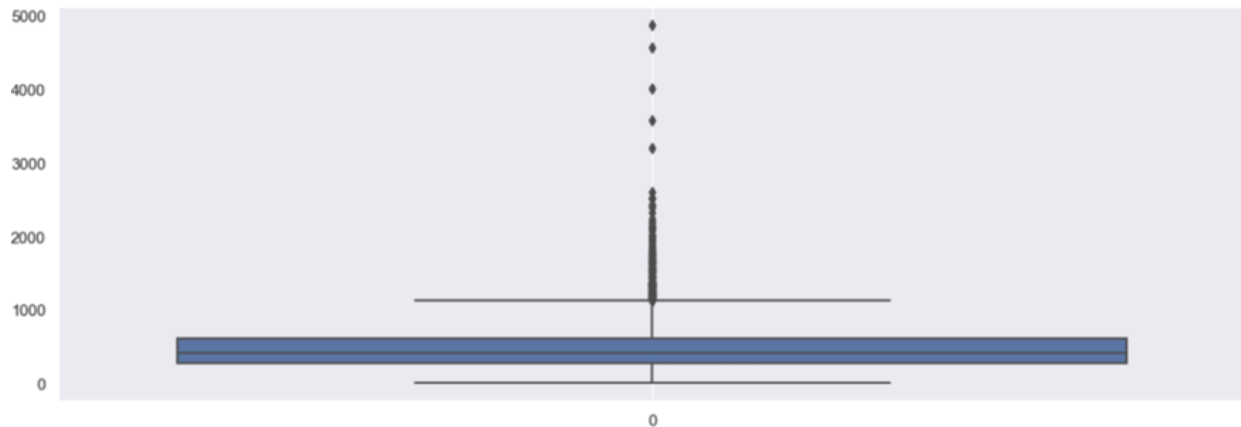


Figure 3.7: Box-plot distribution of the dependent variable 'total_spent'

[Figure 3.7](#) shows that the dependent variable has quite a few outliers. Also, the box plot shows that the spread of `total_spent` seems to be skewed with a long tail (majority values are less than 1000 with a long tail extending until 5000). We can try fixing both the outlier and skewness using a log transformation.

```
#Log transform to remove skewness and re-plotting  
box plot
```

```
data['total_spent_ln']=np.log(data['total_spent'])
```

```
fig, ax = plt.subplots(figsize=(15,5))
```

```
sns.boxplot(data['total_spent_ln'],ax=ax)
```

```
plt.grid()
```

```
plt.show()
```

Output:

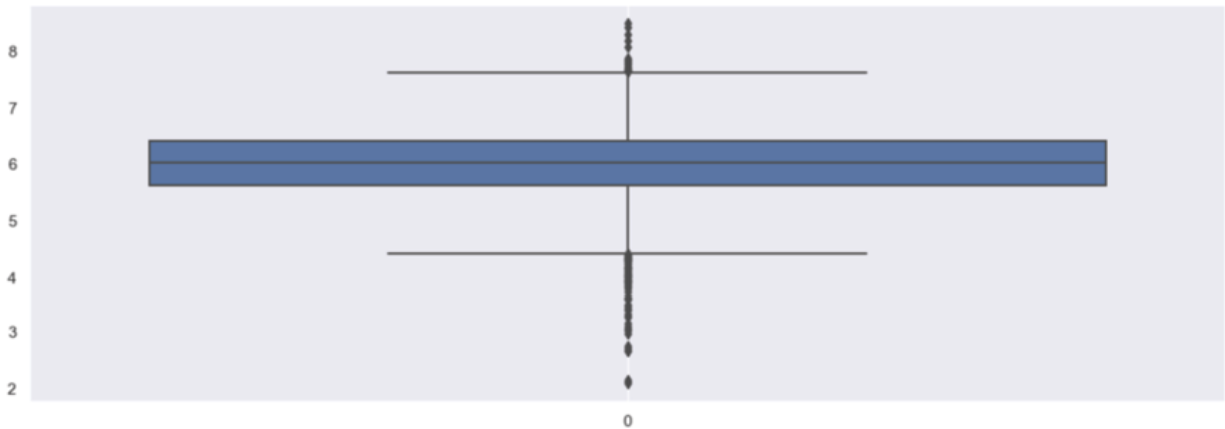


Figure 3.8: Outlier treated dependent variable 'total_spent_ln'

Figure 3.8 looks more normally distributed with fewer outliers. Figure 3.9 shows the case of the dependent variable total_spent_ln.

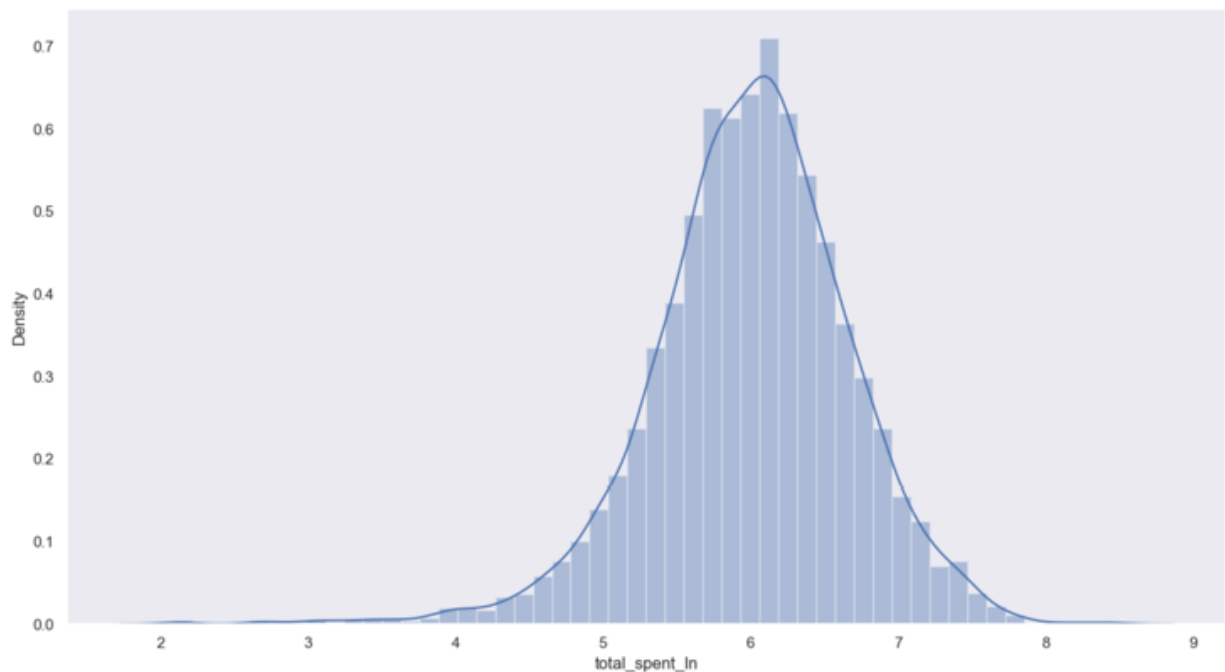


Figure 3.9: Uniformly distributed dependent variable 'total_spent_ln'

Similarly, we can check outliers in other continuous variables using the below code:

```
#check numeric variable distribution
```

```
data[numerical_var].describe(percentiles=[.1, .25, .75, .9, .95, .99]).T.round(2)
```

Output:

	count	mean	std	min	10%	25%	50%	75%	90%	95%	99%	max
age	4994.0	47.04	17.76	18.00	23.00	31.00	47.00	62.00	72.00	76.00	79.00	79.00
ed	4994.0	14.54	3.28	8.00	10.00	12.00	14.00	17.00	19.00	20.00	21.00	23.00
income	4994.0	54.76	55.38	9.00	16.00	24.00	38.00	67.00	109.00	147.00	272.07	1073.00
lninc	4994.0	3.70	0.75	2.20	2.77	3.18	3.64	4.20	4.69	4.99	5.61	6.98
debtinc	4994.0	9.95	6.40	0.10	2.80	5.12	8.80	13.60	18.60	22.20	29.20	43.10
creddebt	4994.0	1.86	3.42	0.00	0.18	0.39	0.93	2.06	4.30	6.38	14.28	109.07
lncreddebt	4994.0	-0.13	1.27	-6.60	-1.74	-0.95	-0.08	0.72	1.46	1.85	2.66	4.69
othdebt	4994.0	3.68	5.40	0.02	0.46	0.98	2.10	4.31	8.06	11.83	24.08	141.46

Figure 3.10: Percentile distribution of the continuous independent variables

We can see in [Figure 3.10](#) that `income`, `creddebt`, `othdebt` have outliers since their 99th percentile while is vastly different from the max value. Outliers in continuous variables can be fixed by capping them at the 99th percentile value. This can be done using following code:

```
data[numerical_var]=data[numerical_var].apply(lambda x: x.clip(lower = x.dropna().quantile(0.05), upper = x.quantile(0.99)))
```

Correlation

We studied covariance and correlation in [Chapter 2, Basic Statistics and Exploratory Data Analysis](#), but their relevance with respect to linear regression is important to understand. Correlation denotes a degree of association between two continuous variables and is denoted by r . We also assume that this association is linear and that one variable increases or decreases by a fixed amount for a unit increase or decrease in the other.

However, an association does not always imply a cause-effect relationship. For instance, credit card spending can

increase in summer with higher temperatures, but it does not imply that higher temperatures drive more consumer spending. Instead, we need to explore the data for various summer activities that have a causal effect on credit card spending. This causal relationship is then quantified using regression analysis.

Let us explore what associations exist in data to gain a preliminary understanding of various drivers of card spending.

Code:

```
#Covariance matrix with a single line of code  
data[numerical_var].cov().round(2)
```

Output:

	age	ed	income	lninc	debtinc
age	312.39	-5.33	173.17	2.10	4.24
ed	-5.33	10.16	27.74	0.48	0.24
income	173.17	27.74	2156.38	30.20	3.78
lninc	2.10	0.48	30.20	0.51	0.07
debtinc	4.24	0.24	3.78	0.07	38.36

Figure 3.11: Sample covariance matrix

```
#Correlation matrix  
corr_mat =  
data[numerical_var].corr(method='pearson').round(2)  
corr_mat
```

Output:

	age	ed	income	lninc	debtinc
age	1.00	-0.09	0.21	0.17	0.04
ed	-0.09	1.00	0.19	0.21	0.01
income	0.21	0.19	1.00	0.91	0.01
lninc	0.17	0.21	0.91	1.00	0.02
debtinc	0.04	0.01	0.01	0.02	1.00

Figure 3.12: Sample correlation matrix

The detailed matrix with all the continuous variables can be found in the Jupyter Notebook, but please note that this matrix is used as a preliminary check to gauge the presence of any association between various variables. For instance, the above table shows that age and income move together to some extent, as indicated by a correlation value of 0.21. A similar correlation check of all the independent variables with the dependent variable `total_spent_ln` can provide a preliminary view of the strength of their respective associations with the dependent variable. This prior knowledge comes in handy during the final model development stage.

Checking on the assumptions of linear regression

Let us test the two assumptions of linear regression on the credit card spending data. Checking the linearity of the independent variable `income` with the dependent `total_spent_ln`.

```
# Seaborn scatter plot with regression line. you
# can use scatter plot by using df.plot as well
sns.set(rc={'figure.figsize':(15,8)})
```

```
sns.lmplot(x='income', y='total_spent_ln',  
data=data)
```

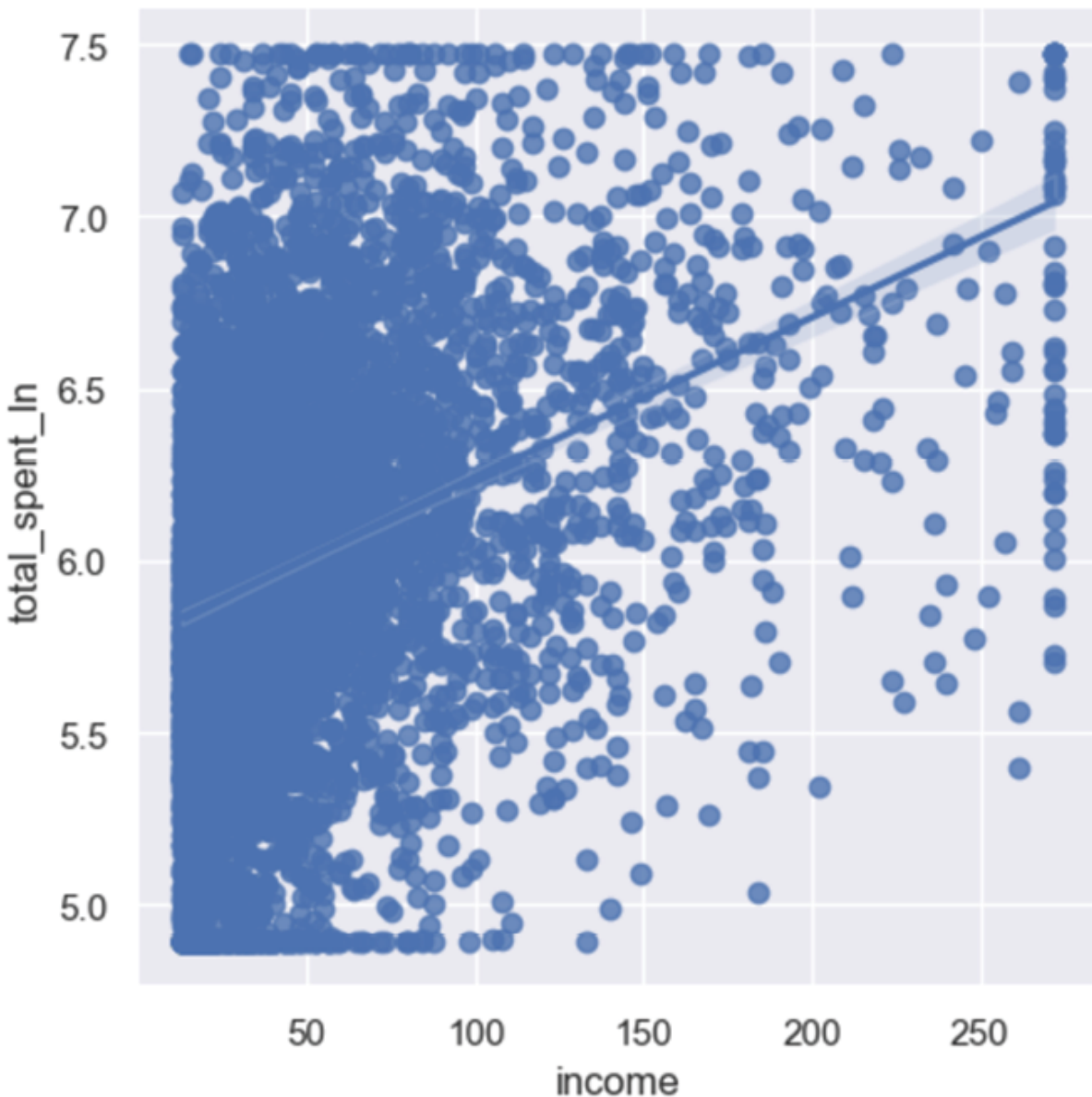


Figure 3.13: *Income vs Total_Spent_ln scatter plot*

As shown in [Figure 3.13](#), there appears to be a slight linear trend between income and card spending. Similar visualizations can be generated for other independent-dependent linearity checks as a visual strategy to evaluate linearity.

Also, [Figure 3.12](#) shows that there is a high correlation between the two independent variables `income` and `lninc`. This is expected since `lninc` is derived from the `income` but it suggests that these two independent variables should not be together in the same model. This causes multicollinearity issues in the model and makes the model unstable. This happens because the two independent variables being correlated can have an evolving relationship among themselves when they should ideally be explaining their relationship with the dependent variable independently. They also essentially explain the same information in the dependent variable and hence the impact due to either is not represented accurately.

In the upcoming section on LR results and interpretation, we will explore other residual-related LR assumptions.

Feature selection

While there are many automated feature selection techniques, we can use the results of the correlation matrix to ascertain which independent variables have a linear association with the `total_spent_ln`. A higher value of the Pearson correlation coefficient' r' indicates a high degree of association between the two variables, but a lower' r' may indicate a non-linear association. This can be further tested using Spearman's rank correlation test. If evidence of a non-linear relationship is found, the independent variable can be transformed to fit the linearity assumption, or, worst case, removed from the initial variable list for modeling purposes.

Note: These hypothesis generations, manual explorations, and understanding of the underlying relationships of the datasets help us in building an intuitive set of independent variables.

Other automated feature selection methods that are commonly used are:

```
#Feature selection method you can use
from sklearn.feature_selection import RFE
out=data['total_spent_ln']
ind =data[feature_columns]
lm = LinearRegression()
#Alternative of capturing the important variables
RFE_features=ind.columns[rfe.get_support()]
features1 = ind[RFE_features]
RFE_features
```

Output:

```
Index(['carbought_0', 'carbought_1',
'carcatvalue_1', 'carcatvalue_2', 'carcatvalue_3',
'carown_0', 'carown_1', 'cars_8', 'cartype_0',
'cartype_1'], dtype='object')
```

We will use this particular RFE feature selection technique to trim down our independent variable list to a set of ten statistically significant key variables. We will then encourage the reader to evaluate and build a hypothesis around these ten independent variables before trying them in the model equation.

Regression execution and results

Executing a basic regression model using Python can be done using the below code. We used some of the most intuitive variables as the independent variables here.

```
# Import Linear Regression machine learning library
```

```
import statsmodels.formula.api as smf

model_base = smf.ols('total_spent_ln ~ age + lninc + carvalue', data = data).fit()

model_base.summary()
```

The following figure explains the linear regression summary output obtained using the `statsmodel` library:

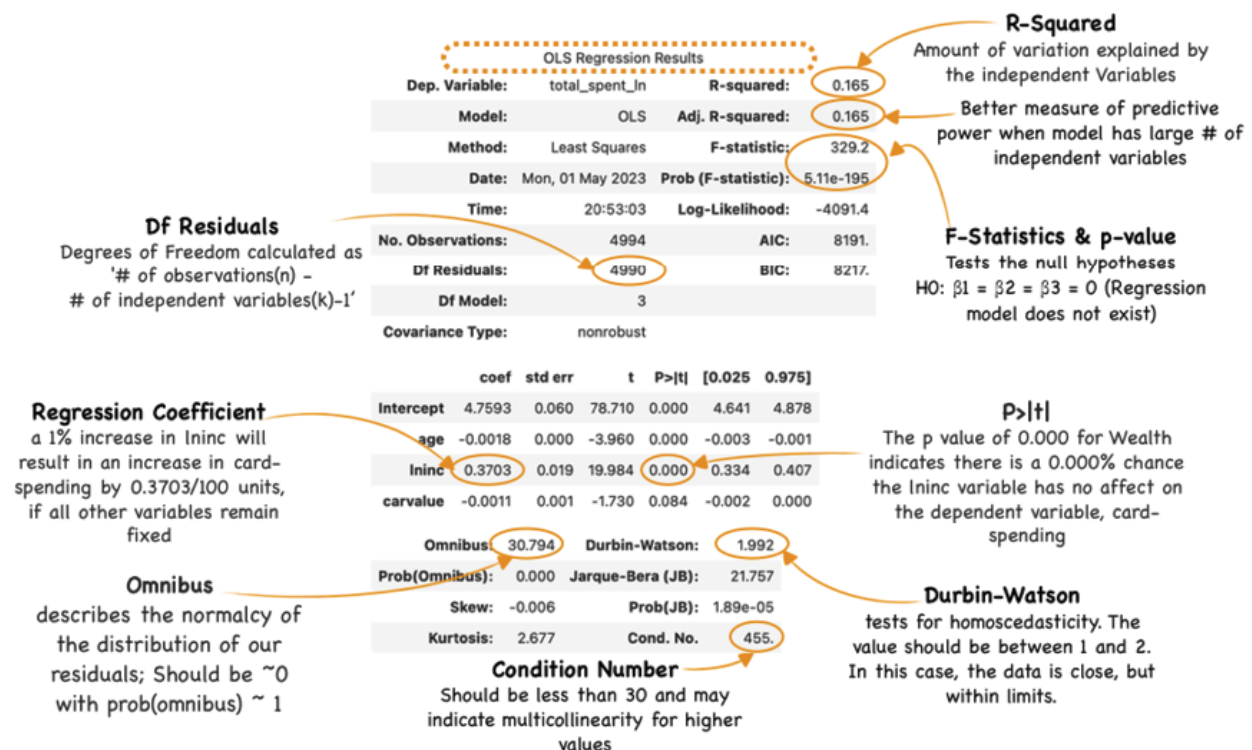


Figure 3.14: Linear regression model execution outcome

Alternatively, we can build an LR model using Sci-kit-learn as well. The code block for the same is shown below:

```
# Import Linear Regression machine learning library
from sklearn.linear_model import LinearRegression

# invoke the LinearRegression function and find the
best-fit model on training data

regression_model = LinearRegression()
```

```
regression_model.fit(X_train, y_train)
# Model score - R2 or coeff of determinant -  $R^2=1-\frac{RSS}{TSS}$ 
print("R2 Score: ", regression_model.score(X_train,
y_train))
```

Output:

R2 Score: 0.4005498381776107

The training dataset R-squared needs to be compared with the test dataset R-squared to assess overfitting:

```
# compare train and test performance to understand
over/underfitting
```

```
#Train performance
```

```
ypred_train=regression_model.predict(X_train)
print("RMSE Train: ",
np.sqrt(mean_squared_error(y_train, ypred_train)))
```

```
#Test performance
```

```
ypred_test=regression_model.predict(X_test)
print("RMSE Test: ",
np.sqrt(mean_squared_error(y_test, ypred_test)))
```

```
#If our test error>train error, it means our model
is Overfitting. we need more data observations,
variables, or a different model
```

Output:

RMSE Train: 0.4646856105859637

RMSE Test: 0.5077579370497193

In general, if the test R-squared is consistently higher than the train R-squared, it could be an indicator that something

unusual is happening with the model or the data, and it would be important to investigate further to understand the underlying reasons for this pattern.

Regression result interpretation

Out of the many statistics that we observed in [Figure 3.14](#), we pick two metrics, R-squared and RMSE as a measure of the Linear regressor's performance. Let us understand in detail what they indicate about the model's performance and why we decided to choose them.

In our case study example, the model's R-squared is 40.05%. The following discussion explains this number. R-squared, also known as the coefficient of determination, is a measure of a linear regressor's predictive power compared to an average model. It is a statistical term that determines how well the best-fit line approximates the actual data compared to a line which is the average of y values, parallel to the x-axis.

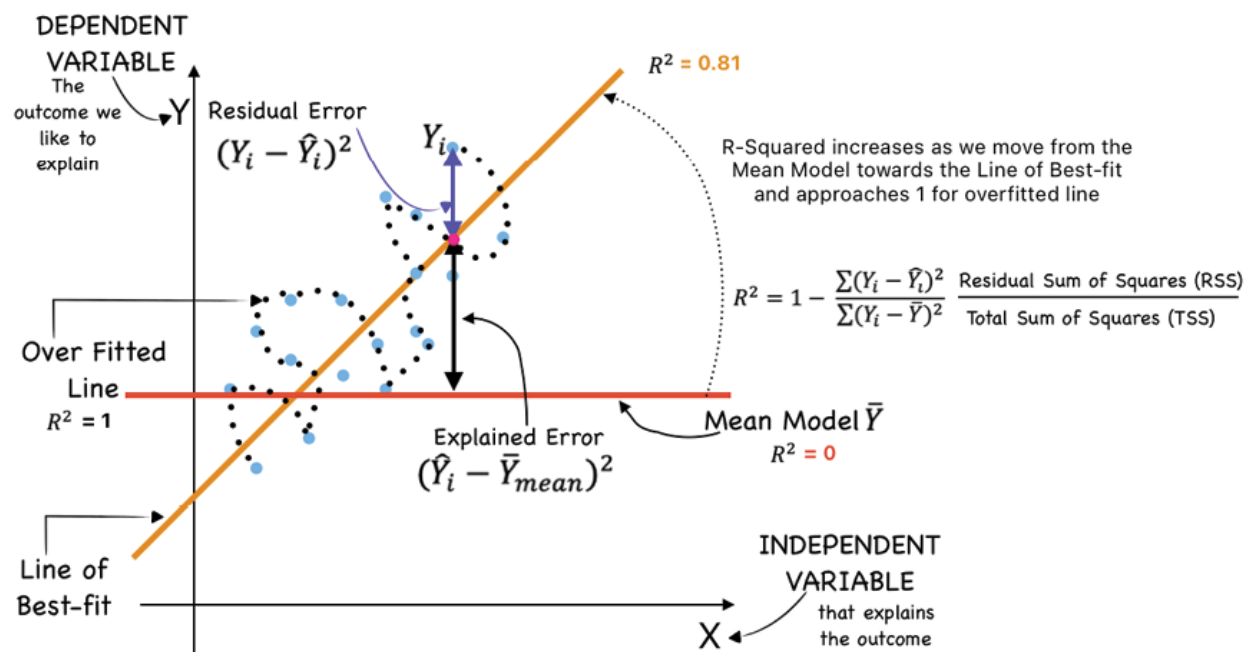


Figure 3.15: Linear regression R-Squared explained

Assume the two lines in [Figure 3.15](#) are drawn by two students. When a lazy student was asked to draw a line that best fit the blue dots on the chart, he drew a line that was parallel to the x-axis with an intercept at an average value of Y . A more diligent student draws the amber line at a slope that captures the trend of the blue dots. This trend capturing intuitively explains how the Y changes every time there is a change in X , in other words, X explains the variation in Y . Mathematically this is captured by R^2 , where the numerator is the sum of **residuals squared (RSS)** and the denominator is the sum of the distance between the actual data and the mean, all **squared (TSS)**. This simply means unexplained leftover error as a proportion of the total error and it is then subtracted from one to output the explained part of the total error. It will always take a value between 0 and 1 since it is a percentage.

R^2 will be zero when the best-fit line is equal to the average or the mean line parallel to the x-axis, suggesting that none of the actual dependent variable values could be predicted by the model. On the other hand, it will be one, when the sum of residual errors is zero indicating there is no difference between the actual and predicted Y values. This means that the line of best fit has captured all the variations in the data. This amounts to overfitting the training data and is, therefore, not ideal. Hence a higher value (~ 1) of R^2 is not necessarily good.

Interestingly and intuitively, the predictive power should increase with an increase in the number of independent variables, hence caution must be exercised in adding more independent variables to increase the predictive power R^2 . We should understand that in the quest for greater predictive power, we tend to over-fit our model ([Figure 3.15](#)).

This is where a modified version of R-squared (that is adjusted-R-squared) helps. Its value decreases if the addition

of another independent variable did not improve the predictive power of the model as expected. This indicates if the adjusted R-Squared decreases with the addition of an extra independent variable, the new additional independent variables are not valuable to the model. Instead, the model just started over-fitting the training data set. The formula for Adjusted- R^2 is as shown in [Figure 3.16](#):

$$\begin{aligned}\text{Adjusted } R^2 &= 1 - \frac{\text{RSS} / (N-1-p)}{\text{TSS} / (N-1)} \\ &= 1 - \frac{(N-1)}{(N-1-p)} \frac{\text{RSS}}{\text{TSS}} \\ &= 1 - \frac{(N-1)}{(N-1-p)} (1 - R^2)\end{aligned}$$

Figure 3.16: Adjusted R-squared formula derivation

We can observe that as the model acquires more variables, p increases and the factor $(N-1)/(N-1-p)$ increases which has the effect of reducing R^2 .

Let us now jump to understand another metric Root Mean Squared Error (RMSE) that was the output as the model results. Our model results indicate an RMSE of 0.4646 on the training dataset and 0.5077 on the test dataset. This metric is simply the square root of the MSE which is the average squared difference between the predicted and actual values. Since this measure squares the error, it places more weight on larger errors. It is also very intuitive since it is in the same unit as the dependent variable. In our case, the credit card spend is in USD, and the RMSE is also in USD leading to a more intuitive understanding of the model's accuracy.

Sometimes, another metric **Mean Absolute Percentage Error (MAPE)**, is used to measure the accuracy of a predictive model. It measures the average absolute percentage difference between the predicted value and the actual value. It is useful in cases where calculating

percentage changes is more intuitive than absolute values. For instance, the prediction of stock prices. A MAPE value of 0.2 suggests that, on average, the predictions are off by 20%.

The [Figure 3.17](#) visualizes various metrics used for measuring LR model prediction error:

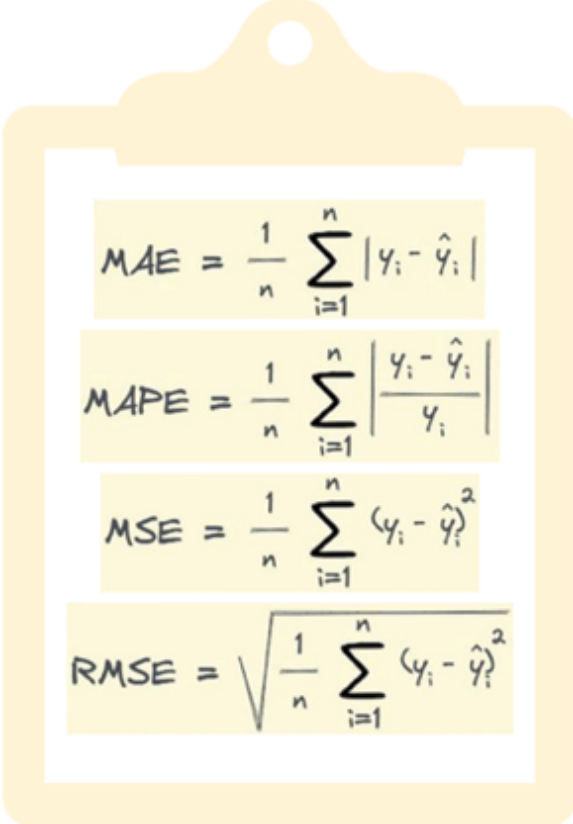

$$\begin{aligned} \text{MAE} &= \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i| \\ \text{MAPE} &= \frac{1}{n} \sum_{i=1}^n \left| \frac{y_i - \hat{y}_i}{y_i} \right| \\ \text{MSE} &= \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2 \\ \text{RMSE} &= \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2} \end{aligned}$$

Figure 3.17: Key performance metrics for linear regression models

Another general problem faced by practitioners is interpreting the coefficients and how it impact the dependent variable. We touched upon it a little in [Figure 3.14](#) for the scenario when independent is log-transformed. Some other non-typical scenarios are:

- The dependent variable is log-transformed: The percent increase (or decrease) in the dependent for every one-unit increase in the independent variable can

be obtained by taking the exponent of the coefficient and subtracting one from this number, before multiplying it by 100. Example: If the coefficient is 0.3408, then the calculation would be $(\exp(0.3408) - 1) * 100 = 40.6$. This indicates that the dependent variable card spending increases by about 40.6% for every one-unit increase in the independent variable income.

- Both the dependent and independent variable(s) are log-transformed: The coefficient indicates the percent increase in the dependent variable for every 1% increase in the independent variable. Example: if the coefficient is 0.3408. For every 1% increase in the independent variable, our dependent variable increases by about 0.34%.
- The most common scenario is when none of the dependent or independent are transformed. In that case, a 1-unit increase in independent will result in an increase in dependent by units, if all other variables remain fixed.

Optimization algorithm

Optimization, as the name suggests, is the process of finding the optimal model parameter estimate values that yield the minimum value of a cost function. It involves finding a *min* or *max* of an Objective Function or the Cost Function, $f(x)$.

Hence the optimization problem could be written as Find X for which $f(x)$ is minimum/maximum. This can also be written as $\text{argmin}(f(x))$: argument where the function $f(x)$ is minimum (or $\text{argmax}(f(x))$ conversely).

Optimization algorithms in turn help to automate the process of finding these optimal or near-optimal parameters for complex problems involving model training, parameter tuning, and hyperparameter optimization.

OLS is the most common optimization algorithm for slope and intercept estimation in the case of linear regression. However, this is not scalable and computationally expensive for multiple linear regression with non-linear relationships. This brings us to a discussion on another very innovative optimization algorithm called **gradient descent (GD)**.

Gradient descent

An alternate optimization strategy to OLS is GD. It is a very general methodology with wide application across a range of ML algorithms. It is much more efficient compared to OLS in finding the minima for the same cost function because it can use a learning rate parameter to find the minima on much larger and more complex datasets. If we plot β_1 and β_0 against MSE, it will acquire a bowl shape. This is as shown in [Figure 3.18](#) below:

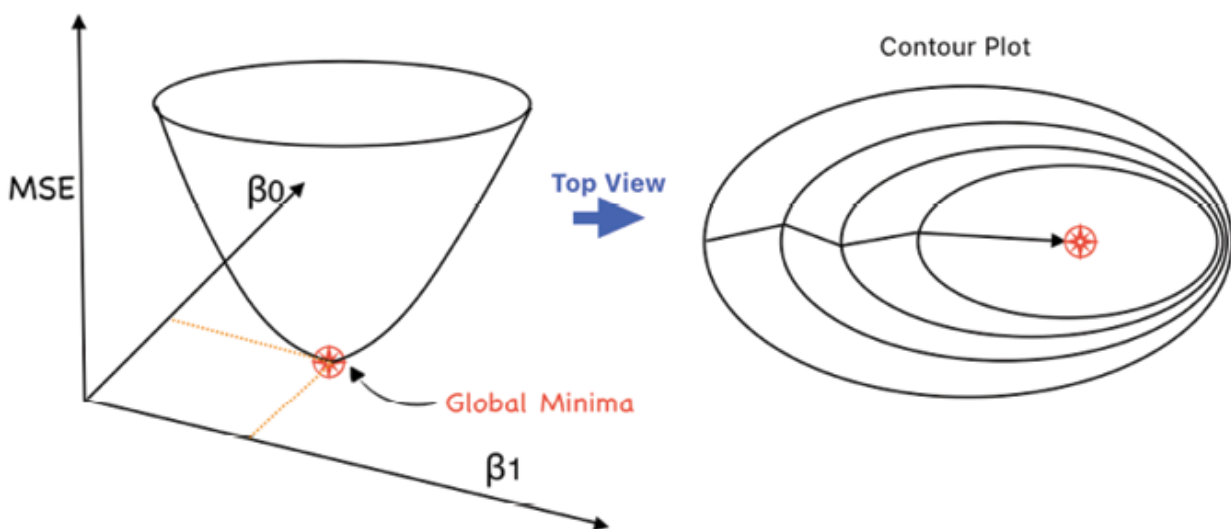


Figure 3.18: Gradient Descent Contour Plot

[Figure 3.18](#) shows that we will get the least MSE for some combination of β_1 and β_0 , leading us to the best-fit line. Below are the steps in a GD optimization strategy:

1. The algorithm starts with some value of β_1 and β_0 (usually $\beta_1 = 0$ and $\beta_0 = 0$) and an MSE (cost) is calculated at point $\beta_1 = 0$ and $\beta_0 = 0$. Let us say the MSE (cost) at $\beta_1 = 0$ and $\beta_0 = 0$ is 1000.
2. From here, the value of β_1 is reduced by an amount equal to the learning rate ($L \times \text{Partial Derivative of Cost function w.r.t. } \beta_1$ that is D_{β_1}). Similarly, β_0 is reduced by $L \times D_{\beta_0}$. This will result in a decrease in MSE (cost).
3. We will continue doing the same until our loss function is a small value or close to 0.

We take the partial derivative (interpreted as the slope of the graph of the function or, more precisely, as the slope of the tangent line at a point) of the cost function to know if our function is moving towards a minimum or not and secondly if we have a big enough slope we need to move more quickly and vice versa if the slope is small. This is achieved by the learning rate L , which basically tells how big or small steps we should take to reach the local minimum. See the below [Figure 3.19](#):



Figure 3.19: Optimization with different learning rates

Above mentioned optimization is known as Batch Gradient Descent as we are taking the error of the complete training batch or the data set per iteration. For instance, if there are 1 million samples then the gradient descent algorithm should sum errors of 1 million samples for every error

calculation iteration. This makes it less efficient for large datasets. We also have the following variants of Gradient Descent that we will explore further in [Chapter 9, Structured Data Classification using ANN](#):

- **Stochastic Gradient Descent (SGD):** We use one training sample at each iteration instead of using the whole dataset to sum all for every step. We need to randomly shuffle the training examples before using SGD, which is efficient for large datasets.
- **Mini Batch Gradient Descent:** It which is midway between Batch and Stochastic, divides the complete data set into mini-batches (n), instead of 1, and then applies weight updates after each batch.

Regularization

In [Chapter 1, Understanding Data Mining in a Nutshell](#), we discussed how biases and learning shortfalls can lead to overfitting and underfitting. Bias arises from underfitting, or oversimplifying the model in which the algorithm does not detect the relationship between features and outcome. Variance is the result of overfitting or adhering too closely to the noise also known as every minute detail in the data and hence fails to generalize on the new data.

Regularization is a technique in machine learning that aims to achieve more generalized models while minimizing the adjusted loss function, thereby preventing over or underfitting of the models. It does so by adding a penalty term to the model's objective function, discouraging the use of overly complex models that might fit the training data too closely.

In the context of linear regression, Lasso (L1) and Ridge (L2) regression are a couple of techniques to reduce the model complexity and prevent over-fitting by penalizing the

coefficients of the independent variables. This goal essentially is to prevent the model from learning from noise.

Lasso regression

Least Absolute Shrinkage Selector Operator (Lasso) regression, also known as L1 regularization, is another type of linear regression that adds a penalty term to the original cost function of the linear regression to shrink or eliminate some of the parameter estimates (betas) of the model. This leads to the removal of less significant explanatory variables thereby preventing inflated R-squared. The betas are penalized by the use of a term that is the absolute value of the betas multiplied by a scalar value called the regularization parameter. Refer to [Figure 3.20](#) for the updated cost function:

$$\sum(\hat{Y}_i - \bar{Y}_i)^2 + \lambda \sum|\beta_i|$$

Figure 3.20: Lasso Loss Function

The goal of Lasso is to minimize this cost function by adjusting the betas of the model. Interestingly, by eliminating insignificant variables, the lasso regression helps with feature selection as well. It tends to shrink the coefficient of less important features to zero, and in the process eliminate them from the model. This makes it ideal for cases where the number of features is too large and greater than the number of records.

In the case of Lasso regression, the second term of the cost function, will draw the vector to zero. The minima in the case of lasso will appear somewhere near the diamond shape of the second term and the ellipse of the first term. Refer to [Figure 3.21](#):

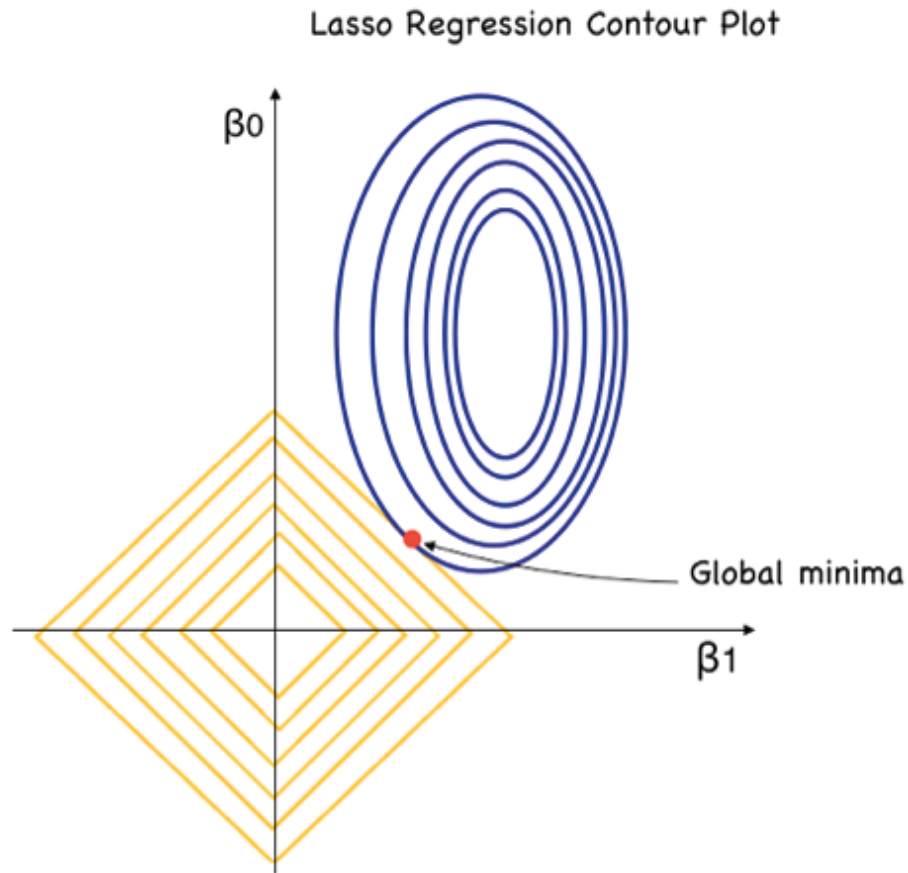


Figure 3.21: *Lasso contour plot*

Ridge regression

The main idea behind ridge regression is to find a best-fit line that generalizes well by introducing a regularization term that penalizes large coefficients. The line equation has a small amount of bias or error introduced in the cost function, but we get a significant drop in the variance or overfitting in return for this error introduction. Ridge regression is also particularly useful in scenarios where there is a risk of multicollinearity in the feature variables. It mitigates the effects of multicollinearity by shrinking the coefficients towards zero. Please note that Ridge, unlike Lasso, does not eliminate the coefficients completely.

Let us now see how penalty term impacts the cost function:

$$\sum (\hat{Y}_i - \bar{Y}_i)^2 + \lambda \sum \beta_i^2$$

Figure 3.22: Ridge regression loss function

As seen in [Figure 3.18](#) of gradient descent, the minima are inside the circles or at the bottom of the bowl, but with the improvised cost function shown in [Figure 3.22](#) above, the new term will *draw* parameters towards zero (intersection of β_0 and β_1 axis). In effect, the cost function is now trying to minimize the sum of two terms instead of one MSE term in [Figure 3.22](#).

The MSE (first term) pulls the minima toward the center of the blue circles shown in [Figure 3.23](#), and the second factor with the lambda factor pulls the vector to zero. Intuitively, a larger lambda will make the second factor more powerful. This will land the minima somewhere in between on the edges of blue and amber circles. Refer to [Figure 3.23](#):

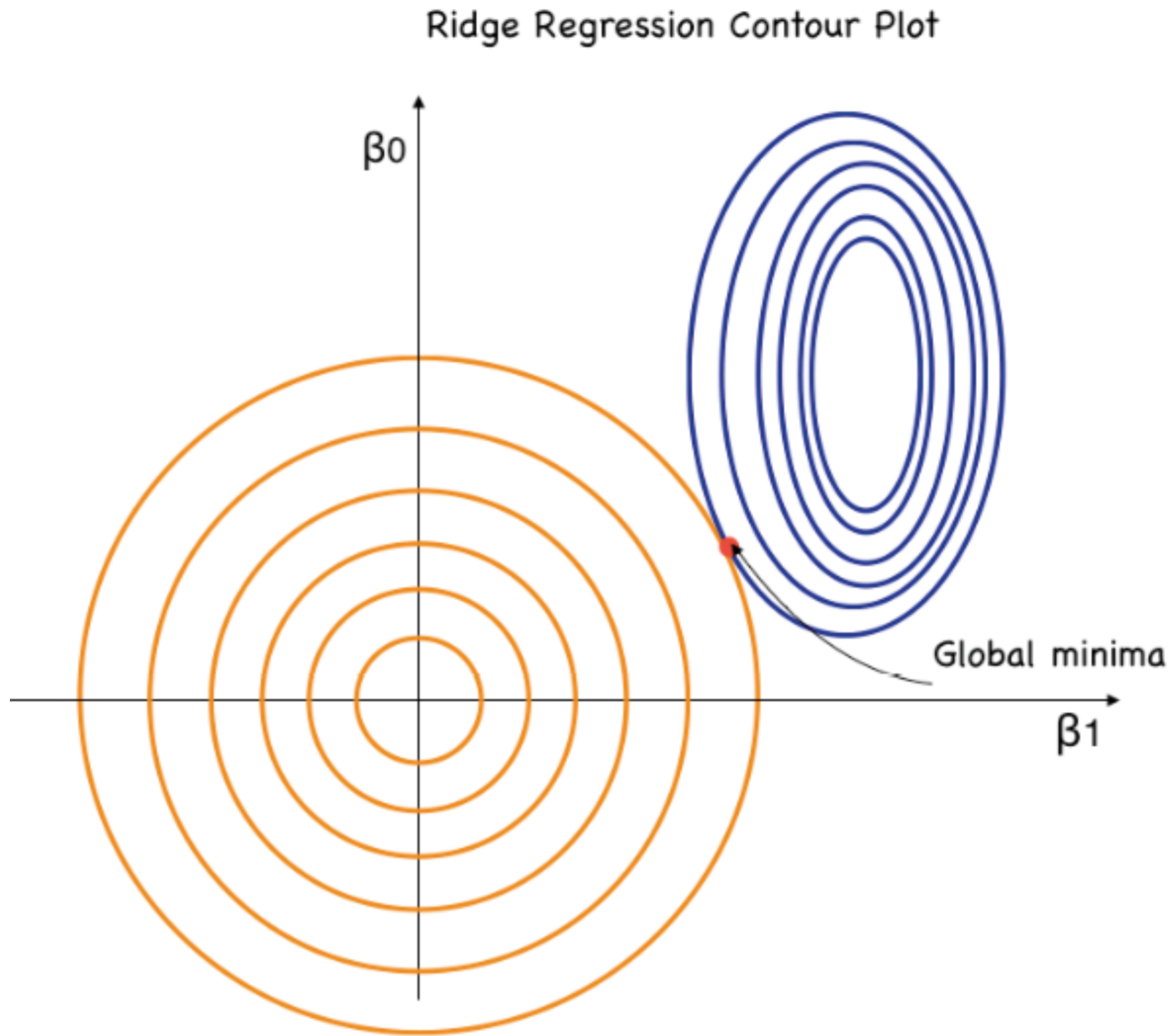


Figure 3.23: Ridge contour plot

Elastic-Net regression

In essence, Elastic-Net takes benefits of both the Lasso and Ridge and tries to minimize the improvised cost function shown in [Figure 3.24](#):

$$\sum (\hat{Y}_i - \bar{Y}_i)^2 + \lambda_1 \sum \beta_i^2 + \lambda_2 \sum |\beta|$$

Figure 3.24: Elastic-Net regression loss function

Please note that l_1 and l_2 values do not have to be equal. In Ridge regression, we know which variables are useful and try to constrain them whereas in Lasso, we know which variables from the long list of features are useless and we try to eliminate them. Elastic-Net is typically used when we do not know a lot about the variables. Please refer to [Figure 3.25](#) for the elastic-net contour plot showcasing the minima:

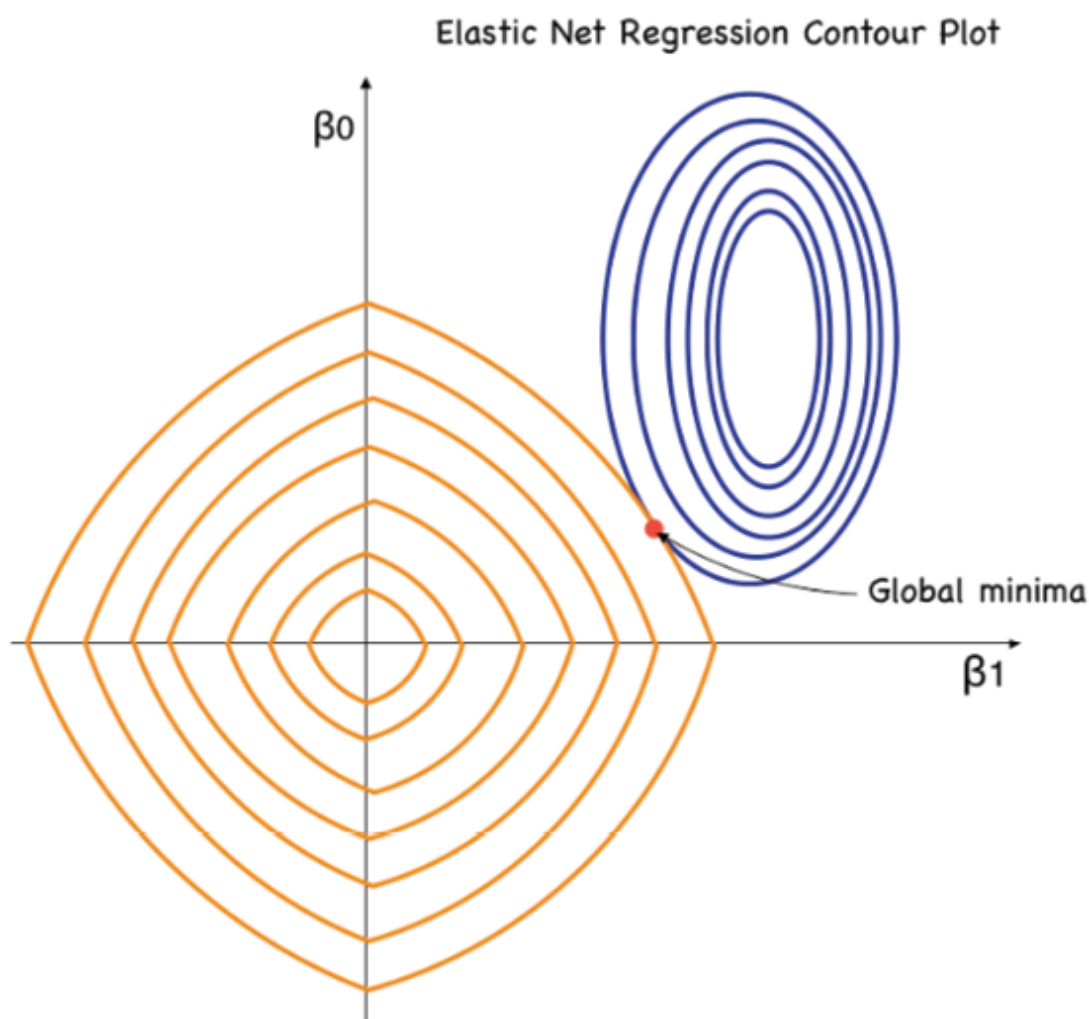


Figure 3.25: Elastic-Net regression loss function

This concludes our conceptual study of the three widely used regularization techniques, but we will understand these better in the upcoming case study on credit card spending prediction.

MLflow introduction: Need and implementation

In *Chapter 1, Understanding Data Mining in a Nutshell* we talked about modern-day data mining and machine learning challenges and categorized one set of challenges into learning or training-related ones. This includes scalability and version control as you run through multiple iterations during training and validation. Another set of issues in learning is about reproducibility wherein it becomes difficult to generate the same results with the same parameters. This could be due to data and code versions changing across iterations. The finalized model object versions then need to be placed in inventory for easy tracking and productionization. These challenges need to be overcome to avoid a downstream risk due to the usage of numerous machine learning models.

It is highly recommended and essential for data scientists to understand and follow best practices to avoid such model usage risks. Modern-day open-source platforms such as MLflow are excellent for managing machine learning processes and offer four great features for avoiding model risk. These are:

- Experimentation tracking
- Package and re-use model objects
- Model Inventory
- The project component for reproducibility

MLflow experiment tracking

In this chapter, we are going to build various versions and iterations of a linear regression model and focus on experimentations that are defined in MLflow as basic units of MLflow organization. All MLflow runs are part of an experiment, that can be analyzed, and the results of

different runs can be compared. This helps in easily retrieving metadata artifacts for analysis using downstream tools.

MLflow experimentation tracking is an API and user interface component that records data about machine learning experiments and lets us query it. MLflow tracking supports Python, as well as various APIs like REST, Java API, and R API. We can use this to log several aspects of our run such as code version, parameters, and artifacts (pickle files, PNG outputs, parquet files and more). We can also record start and end times and other key-value model metrics such as loss convergence tracking.

We will use these features of MLflow throughout the credit card spending case study and utilize various experiment-tracking features of MLflow.

Case study

Predicting credit card spending to understand the underlying drivers using MLflow experiment tracking

In this section, we will augment the Scikit-learn code with the MLflow library which will help us keep tabs on our experiments. Experiments in MLflow essentially allow you to group your various models and individual iterations with any relevant metrics.

Below is the model training function with `mlflow.log_metric` function to track model training with R-squared:

```
#Define model training function to run on Scaled data and to log it in MLFlow
```

```
def train(model, X_train_std, y_train):  
    model = model.fit(X_train_std, y_train)
```

```
train_rsquare = model.score(X_train_std,  
y_train)
```

```
mlflow.log_metric("train_rsquare",  
train_rsquare)
```

```
print(f"Train R Square: {train_rsquare:.2%}")
```

Model evaluation function code with `mlflow.log_metric` function to track model error RMSE on test data:

```
#Define model metrics function to run on Scaled  
data and to log it in MLFlow
```

```
def evaluate(model, X_test_std, y_test):
```

```
    preds = model.predict(X_test_std)
```

```
    RMSE = np.sqrt(mean_squared_error(y_test,  
preds))
```

```
    mlflow.log_metric("RMSE", RMSE)
```

```
    print(f"RMSE: {RMSE:.3}")
```

A linear regression model can be executed using the above two functions (train & evaluate) with the following code:

```
#Run Linear Regression on Scaled data
```

```
model = LinearRegression()
```

```
with mlflow.start_run():
```

```
    train(model, X_train_std, y_train)
```

```
    evaluate(model, X_test_std, y_test)
```

```
    mlflow.sklearn.log_model(model,  
"Linear_reg_model")
```

```
    print("Model run: ",  
mlflow.active_run().info.run_uuid)
```

```
mlflow.end_run()
```

In the preceding code, we use

`mlflow.set_experiment("LinearReg_experiment")` command to set up our experiment run `LinearReg_experiment`. This will guide MLflow to create an experiment name and put all the runs under this name. The code with `mlflow.start_run()`: will take our train and evaluate functions under the context of one MLflow run. The code `mlflow.sklearn.log_model(model, "Linear_reg_model")` will log our model. The last line of the code, `mlflow.active_run().info.run_uuid` gets the current model run id that the model and metrics are being logged to and prints it out. This makes it easy if we want to retrieve the run directly from the notebook itself instead of going to the UI to see the details.

Output:

Train R Square: 40.05%

RMSE: 7.09e+09

Model run: 3ece93e28c1149b9afb1dc8d23f5f850

Next, let us execute another model on this dataset using ridge regression and track its metrics using MLflow. Please note that we will scale the training dataset using the max-min normalization before model training. This is done to ensure all features are on the same scale approximately and each feature is equally important. Feature scaling is extremely important to those models that leverage optimization algorithms such as gradient descent, which weigh input variables (for example: regression). This also allows for a direct comparison of model coefficients or weights.

In this case, Max-Min normalization will re-scale the features with a distribution value between 0 and 1. Another important scaling technique is standardization which ensures the re-

scaled values have a mean of 0 and a standard deviation of 1. This is more robust to outliers in the data. Since we have already done outlier treatment by capping the values at the 99th percentile, we have chosen to use max-min normalization.

Code for max-min normalization:

```
from sklearn.preprocessing import
MinMaxScaler,StandardScaler

# Scale all the columns of our data. This will
produce a numpy array

std = StandardScaler()

X_train_std = std.fit_transform(X_train)

X_train_std = pd.DataFrame(X_train_std,
columns=X_train.columns)

X_test_std = std.transform(X_test)

X_test_std = pd.DataFrame(X_test_std,
columns=X_test.columns)
```

Ridge regression code:

```
# Run Ridge Regression on Scaled data

from sklearn.linear_model import Ridge

model = Ridge(alpha=.3)

with mlflow.start_run():

    train(model, X_train_std, y_train)
    evaluate(model, X_test_std, y_test)

    mlflow.sklearn.log_model(model,
    "Ridge_reg_model")
```

```
    print("Model run: ",
mlflow.active_run().info.run_uuid)
mlflow.end_run()
```

Output:

Train R Square: 40.05%

RMSE: 0.508

Model run: 686a173b1cba41a2aefc1f544141be13

Let us try the Lasso regression next with cross-validation. We will continue to explain this concept of cross-validation in [Chapter 4, Exploring Logistic Regression](#) in more detail but for now understand that cross-validation is a re-sampling technique that helps in reducing overfitting by repeated sampling of a train and test datasets. This helps in better model generalization.

Lasso regression code:

```
# select the best alpha with LassoCV on Scaled data
```

```
from sklearn.linear_model import LassoCV
```

```
model = LassoCV(n_alphas=100, random_state=1)
```

```
with mlflow.start_run():
```

```
    train(model, X_train_std, y_train)
```

```
    evaluate(model, X_test_std, y_test)
```

```
    mlflow.sklearn.log_model(model,
"Lasso_CV_reg_model")
```

```
    print("Model run: ",
mlflow.active_run().info.run_uuid)
```

```
mlflow.end_run()
```

Output:

Train R Square: 35.64%

RMSE: 0.501

Model run: 04e581e77d654455ab076858fb717d04

Below code outputs the MSE for the LassoCV on
train and test datasets

```
pred_lasso= lassoregcv.predict(X_test_std)
```

```
train_pred= lassoregcv.predict(X_train_std)
```

```
print('MSE for  
Test:',metrics.mean_squared_error(y_test,pred_lasso  
)
```

```
print('MSE for  
Train:',metrics.mean_squared_error(y_train,train_pr  
ed))
```

Output:

MSE for Test: 0.25118458901268875

MSE for Train: 0.23182963946155025

All of the above model executions and others can be tracked
on the MLflow UI as shown in [Figure 3.26](#):

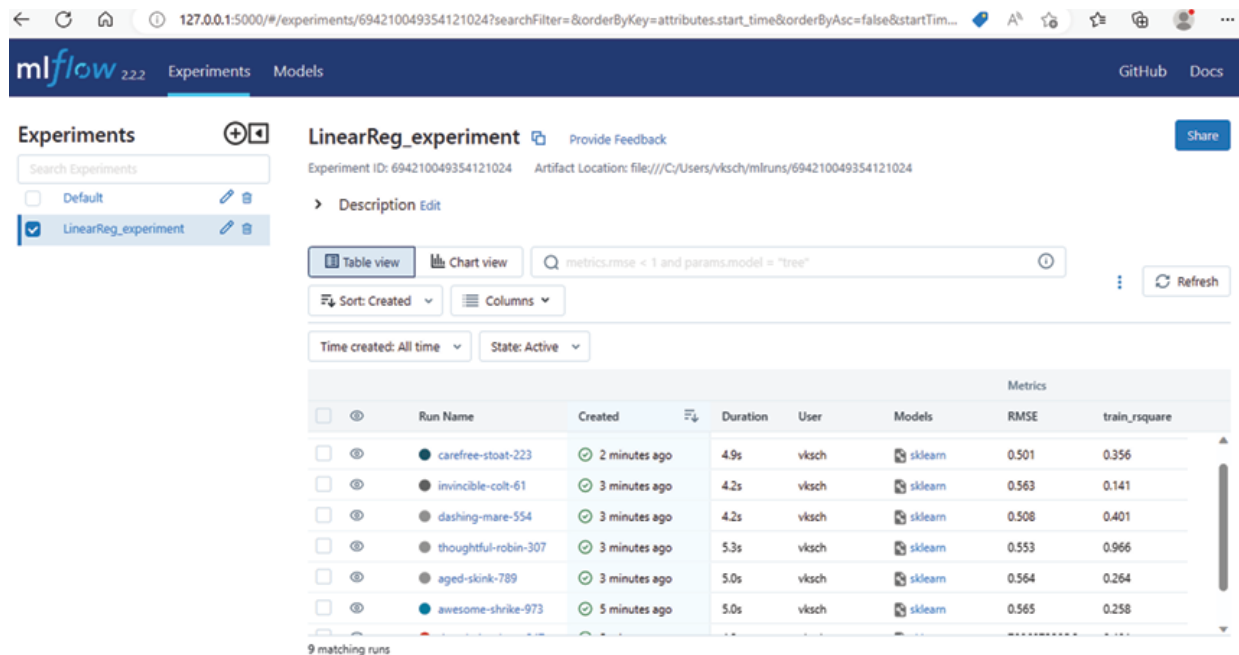


Figure 3.26: MLflow User interface

We will continue to explore this interface in the following chapters, but for now, we will end here with various model results as part of the experiment tracking.

Residual analysis

We have now seen experimentation tracking with MLflow for building the LR models, but it is time to validate if these models are conceptually sound. This means the model does not violate any of the previously stated four assumptions of linear regression related to the residuals and LR has been used in the right context. For this evaluation, let us pick the Lasso model, which eliminates the insignificant independent variables and hence is relatively less overfitted on the training dataset.

Normality: The residuals should be normally distributed.

Code:

```
#Check Normality and Residuals
```

```
residuals=y_test - pred_lasso
```



```
sns.distplot(residuals)
```

Output:

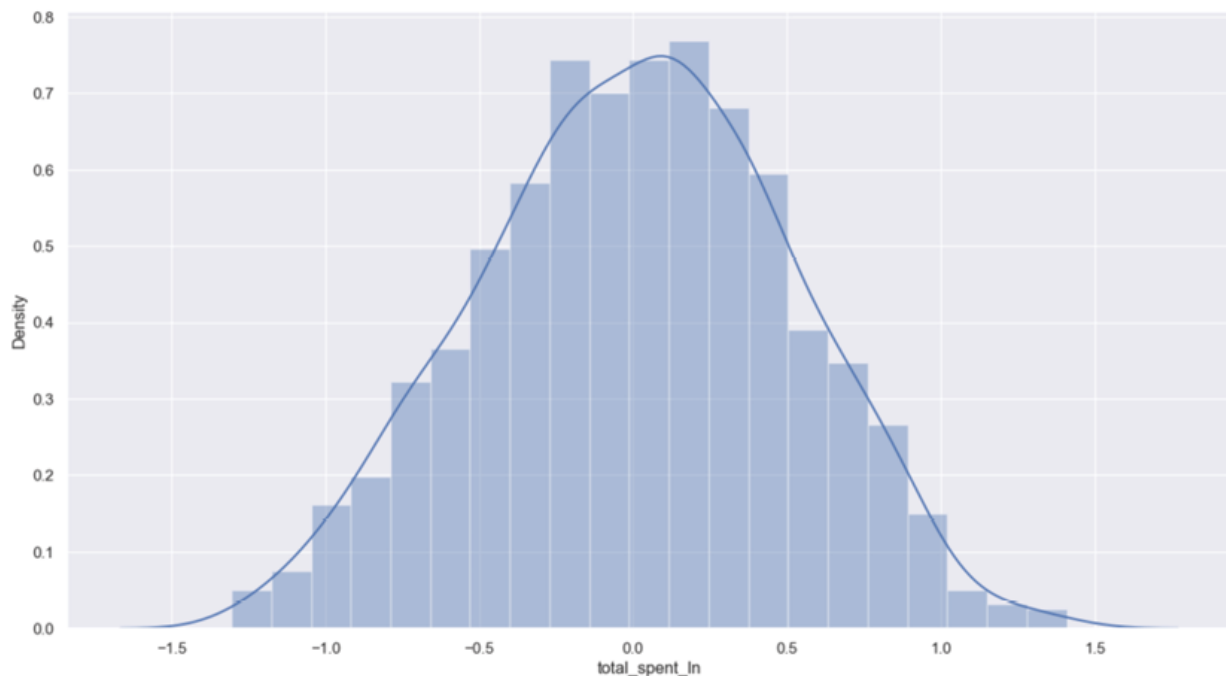


Figure 3.27: Residual normality test

Variability, independence, and outliers: For these three tests, we can draw scatter plots and look for any patterns. The absence of any patterns indicates a successful test.

Code for Residual vs Predicted values:

```
fig, ax = plt.subplots(figsize=(15,8))
```

```
_ = ax.scatter(y_pred, residuals)
```

Output:

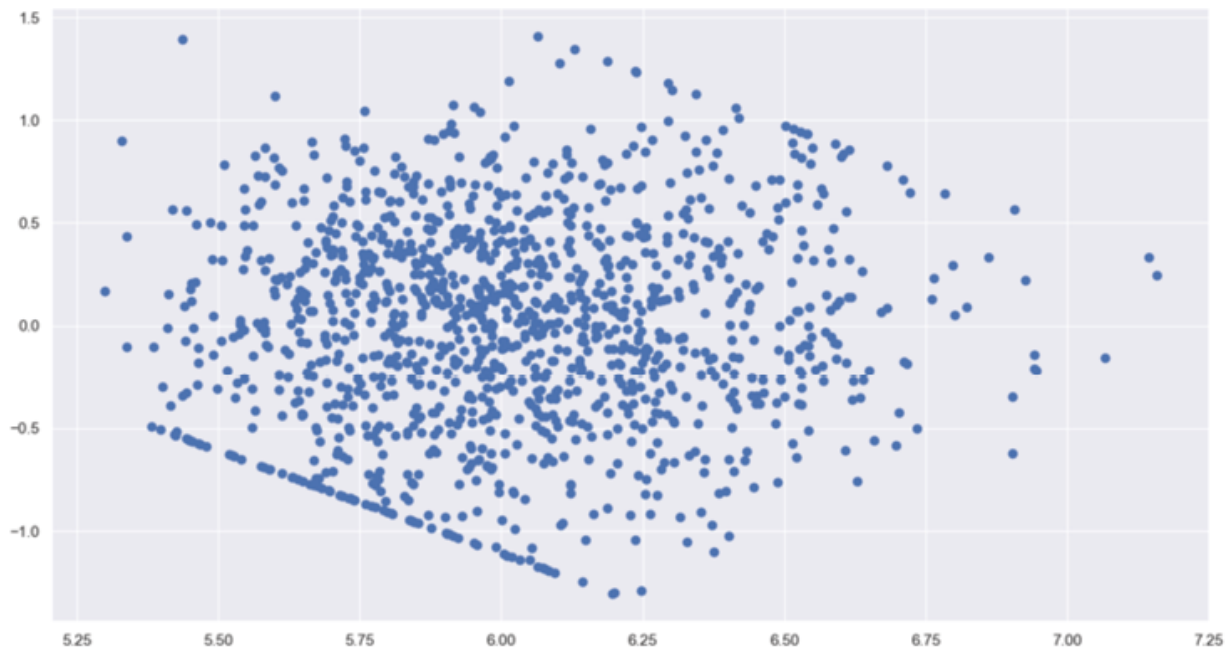


Figure 3.28: *Residual vs Predicted scatter plot*

[Figure 3.28](#) shows some pattern with some residual values being higher around 6.25, but we also observe that the majority of values are less than 6.5 and there is no clear pattern until this point. However clearly, we do not see errors that systematically get larger in one direction by a significant amount. Let us also check the residuals for any outlier values to mark any extreme prediction errors along with the mean of residuals.

Code for Residual scatter plot:

```
# Expected mean of residuals is zero  
np.mean(residuals)  
sns.scatterplot(residuals)
```

Output:

```
0.005107645583910471
```

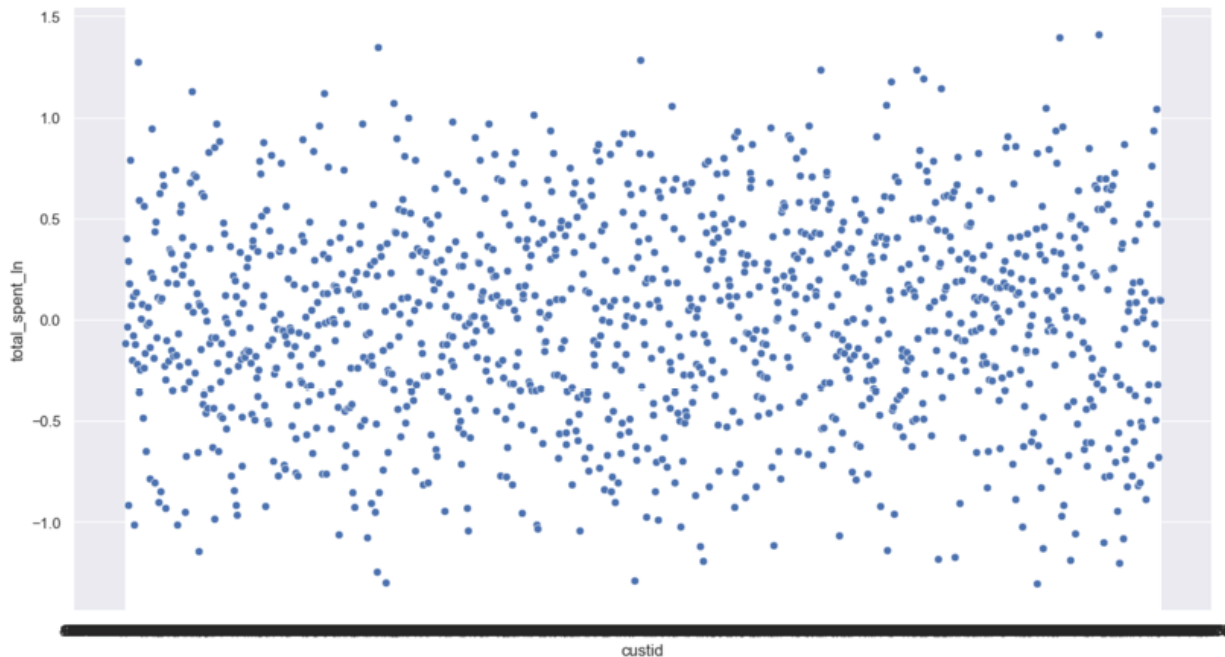


Figure 3.29: *Residual scatter plot*

Looking at [Figure 3.29](#), we can conclude that for this non-time series data, the residuals are randomly and symmetrically distributed around zero under all conditions distributed by rows (`custid`). This indicates that the residuals are independent of each other.

Conclusion

In conclusion, linear regression is a powerful technique for predictive modeling technique which is widely used in various fields, including economics, finance, social sciences, healthcare, and more.

The basic theory behind linear regression is to find the best-fit line or plane that minimizes the sum of squared errors between the predicted values and the actual values. This can be achieved using the method of least squares, which involves finding the coefficients of the regression equation that minimize the sum of squared residuals. Another optimization strategy, GD, involves finding the best values for the coefficients of the regression equation by iteratively

updating the coefficients based on the direction of the gradient of the cost function. Stochastic GD and mini-batch GD are variants of gradient descent that use a subset of the training data at each iteration to speed up the optimization process.

However, linear regression can suffer from overfitting, which occurs when the model fits the training data too closely and fails to generalize well to new data. Regularization is a technique used to address this issue by adding a penalty term to the cost function, which encourages the model to have smaller coefficients and reduces overfitting. The two most common regularization techniques are Lasso (L1) regularization (Lasso) and Ridge (L2) regularization. Sometimes Elastic-Net which is also used for this purpose brings in the best of L1 and L2 techniques.

The chapter concludes with a case study that utilizes a powerful Python library known as MLflow to address the model development challenges of reproducibility and scalability. In the next chapter, we will learn about logistic regression.

Points to remember

- Linear regression belongs to a family of statistical analyses known as Regression analysis. Other regression techniques that are commonly used are logistic, polynomial, lasso, ridge, decision tree, and more. We will discuss the logistic regression, used for classification problems, in greater detail in the next chapter.
- Linear regression has four primary assumptions related to the input data, that can cause issues on the model fit side if they are violated. Residual testing helps identify the lack of adherence to assumptions.

- Multicollinearity impacts the parameter estimation since it limits the information available for the model to explain the variance.
- R-squared (R^2) is the ratio of the explained variation to the total variation and is always less than 1. It increases with an increase in the number of independent variables in the model.
- Adjusted R-squared is another variation of R-squared which is not always increasing. It decreases with the addition of less significant variables, thus providing more reliable and accurate metrics. It does this by compensating and penalizing for the inclusion of a bad variable.
- OLS is not very efficient in estimating the parameters for large datasets with multiple variables. Gradient Descent is a great alternative as an optimization technique.
- Overfitting in linear regression is common and regularization techniques such as Lasso and Ridge help in the reduction of the overfitting. Lasso can also be used as a variable reduction technique since it makes the coefficients of less useful variables zero.
- Most importantly, it is critical to keep track of your experiments for the scalability and reproducibility of your hard work. Follow model operation principles to reduce risk in model development by utilizing available open-source tools for model experimentation tracking and artifact storage.

Join our book's Discord space

Join the book's Discord Workspace for Latest updates, Offers, Tech happenings around the world, New Release and Sessions with the Authors:

<https://discord.bpbonline.com>



CHAPTER 4

Exploring Logistic Regression

Introduction

Pedro Domingos, a computer scientist and statistician, once said about machine learning models, *the best way to avoid failure is to turn it into success*. This quote emphasizes the importance of learning from failure and mistakes and is very relatable when developing machine learning models. It highlights the iterative nature of the modeling process, where failure can provide valuable insights and lead to better models in the process.

In this chapter, we will explore the reason behind *Domingos'* quote by discussing the anatomy of logistic regression, from its assumptions to its applications. Being a classification algorithm, it works like an astrologer telling you whether your favorite team wins the World Cup finals or not. We will also discuss getting better at such predictions over multiple iterations of the modeling process. The goal is to show readers how to wield algorithmic powers to make data-driven decisions with confidence and precision.

So, grab your Python Jupyter notebooks, import your libraries, and get ready to join us on a thrilling ride through the world of logistic regression.

Structure

This chapter includes the following topics:

- Logistic regression
- Background
- Under the hood
 - Data
 - Estimating probabilities
 - Loss function
- Challenges and assumptions
- Logistic regression results and interpretation
- Model interpretability and explainability
- Performance metrics
- Model generalization
 - K-fold cross-validation
 - Ensemble learning
- Model lifecycle processes
- Model development process
- Case study: Loan repayment likelihood prediction

Objectives

This chapter will expand upon another popular regression technique, logistic regression. The goal of this chapter is to understand when data scientists would prefer logistic regression to ordinary linear regression and calculate parameter estimates to quantify the dependency of **Dependent Variables (DV)** on the **Independent Variable (IV)**. We will talk about the DV form and learn about various metrics to track model performance. The optimization strategy for parameter estimation will also be discussed. We will expand on the concept of cross-validation, introduced briefly in the previous [Chapter 3, Digging into Linear Regression](#), to draw focus on the importance of model generalization. Additionally, readers will be exposed to model

development with a flavor of model risk management which is the cornerstone of modern-day data mining.

This chapter will end with a high-level summary of typical model development steps from a regression modeling perspective. These steps serve as a foundation for the rest of the data mining topics discussed in the book ahead.

Logistic regression

How often do you expect that your questions are answered in binary that is yes or no? Often times you are faced with decisions that have two possible outcomes and you have multiple inputs driving these outcomes. At times, these decision boundaries are not black or white but there are shades of grey meaning the decision presents itself as a range of possibilities between the two potential outcomes or mathematically saying a *range of probabilities*.

The probability of an event happening, or an outcome lies between zero and one. Since logistic regression, which derives its name from the underlying logistic function also known as the sigmoid function, outputs values between zero and one, it makes it suitable for such probability prediction. This however leads us to a few other questions such as:

What is the functional form of a logistic regression equation? What is the logistic function or curve? Why have statisticians preferred logistic regression to ordinary linear regression when the DV is binary? How are probabilities, odds, logistic, and logit related? What is a loss function for logistic regression? What is the maximum likelihood estimate? How is the weight in logistic regression for a variable related to the logit of its DV?

We will answer the above questions and more throughout this chapter.

Background

From [Chapter 3, Digging into Linear Regression](#), we understand that regression analysis is a type of supervised predictive

modeling technique that is used to find the relationship between a DV and either one IV or a series of IVs. Logistic regression, unlike linear regression, is a classification model that works on the regression principle described so far but works for binary and linear classification problems. This means that its DV is a category (most commonly binary), such as *black* or *white*, *loan approved* or *loan denied*, *true* or *false*, and so on. It achieves very good performance with linearly separable classes. It is widely used in the industry as a base model and more often than not is employed as a final model of choice for classification problems due to its simplicity and explainability.

The primary difference between linear regression and logistic regression is that logistic regression's range is bounded between 0 and 1. As discussed above, this is achieved by a [logistic function](#), also known as the sigmoid function (P), defined below to model a binary output variable:

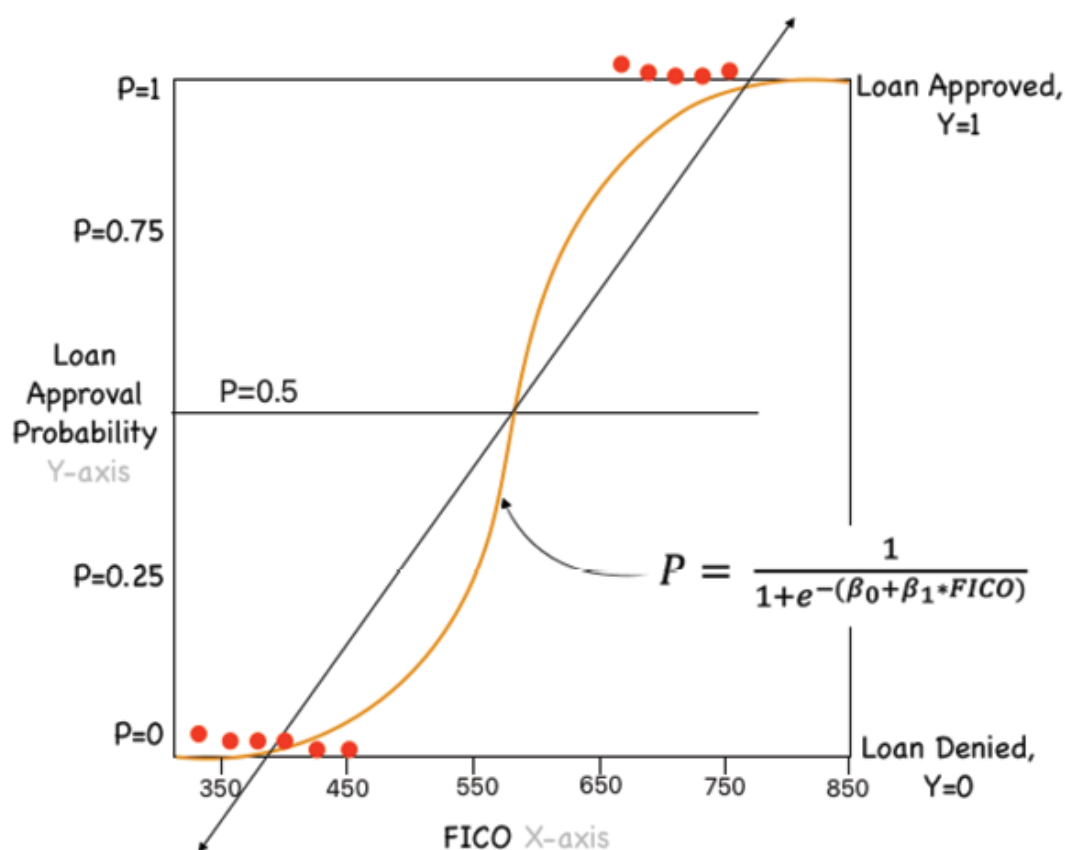


Figure 4.1: Regression of FICO on loan approval probability

Some key observations from [Figure 4.1](#) are as follows:

- The logistic function maps y as a sigmoid function of x .
- The raw data points (red dots) do not fall on the regression line. They all fall on zero or one.
- The logit function returns only values between 0 and 1 for the dependent variable, irrespective of the values of the independent variable.
- A probability threshold for the predicted class ($Y=1$) helps us eventually in the classification of a data point to zero or one. For instance, all data points with a threshold $>60\%$ for the predicted class one, can be classified as one, and data points with a threshold $<60\%$ as zeros.

These observations lead us to question how a new logistic regression equation better serves the above situation rather than **Ordinary Linear Regression (OLR)**. Some limitations of OLR in serving the situation described in [Figure 4.1](#) are as follows:

- If we use linear regression for binary outcomes, the predicted values will become greater than one and less than zero, when we move far enough on the X-axis. Such values are out of bounds for a zero-one outcome.
- With a binary DV, the homoscedasticity (constant variance) assumption of linear regression is violated. The variance ($P*(1-P)$) approaches zero when P approaches one or zero. It reaches the maximum value of 0.25 when 50 percent of the people belong to either class (1s or 0s).
- The significance testing of the coefficients is dependent upon the assumption that prediction errors (ϵ) are normally distributed. It is hard to justify this assumption because Y only takes the values zero and one.

Under the hood

In this section, we will further investigate how logistic regression overcomes the limitations of OLR in serving a classification problem with binary DV. We will discuss the holy trinity of data,

model, and loss function that defines any data mining solution. Let us start by understanding the input data.

Data

The dependent variable must be dichotomous or multinomial (more than two levels). These discrete levels can be either nominal or ordinal.

However, the independent variables can fall into any of the following categories:

- **Continuous:** Such as salaries in dollars. In technical terms, continuous data is categorized as either interval data, where the intervals between each value are equally split, or ratio data, where the intervals are equally split and there is a true or meaningful zero as in the case of salary. In other words, if someone earns zero dollars, it truly means nothing.
- **Discrete, ordinal:** Data that can be placed into some kind of order on a scale. For example, a customer satisfaction survey.
- **Discrete, nominal:** Data that fits into named groups that do not represent any kind of order or scale. For example, regions may fit into the categories North-east, Mid-west, or West, but there is no hierarchy to these categories.

In this chapter, we will perform a case study on the Lending Club loan approval dataset. This dataset can be found at the below link:

https://www.openintro.org/data/index.php?data=loans_full_schema

This data set represents thousands of loans made through Lending Club, an online peer lending platform. This is a data frame with 10,000 records.

The details of the dataset can be seen in [Figure 4.2](#). **SweetViz** is a very useful library for exploratory data analysis as we can observe that with one line of code, we can generate univariate and multivariate summaries like correlation charts for the entire dataset.

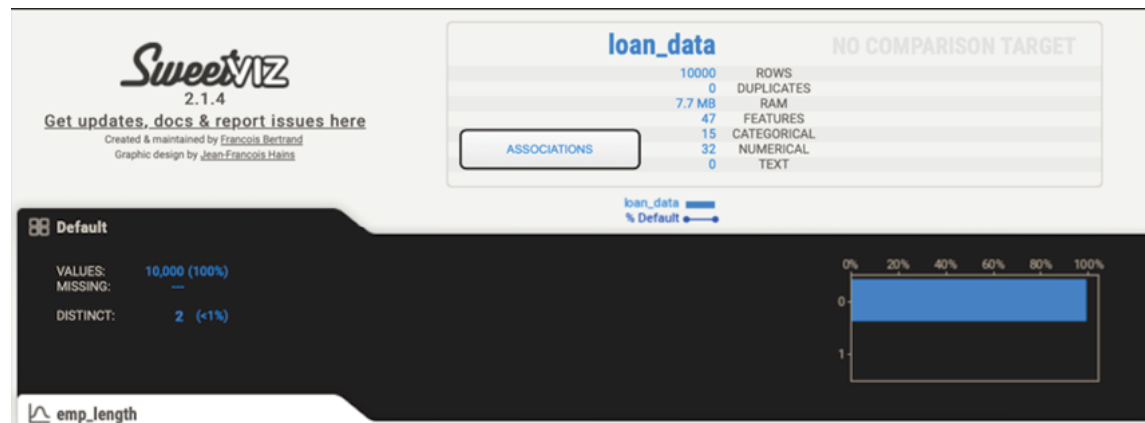


Figure 4.2: Case study: loan approval data profile using Sweet Viz

The previous visualization [Figure 4.2](#) can be generated by a simple Python code as shown below:

```
Report_fullData =
sweetviz.analyze([loan_data,"loan_data"],target_feat='Default')
```

#publish the report in the form of an HTML file

```
Report_fullData.show_html('Report_fulldata.html')
```

The target variable **Default** has been derived from the raw variable **loan_status** and has two levels with the number of '1's (0.73%) negligible compared to '0's (99.27%). Please explore the full HTML report by running the above code yourself. This provides more detailed insights into all the variables (univariate) as well as their correlations.

Estimating probabilities

In Logistic regression, the dependent variable is a logit, which is

the natural log of the odds, that is, $\log\left(\frac{P}{1-P}\right)$. Since log odds are

linearly related to X , the relation between X and P is nonlinear and has the form of the S-shaped curve you saw in [Figure 4.1](#) and the function form (logistic regression equation) can be seen below:

$$\log\left(\frac{P}{1-P}\right) = \beta_0 + \beta_1 x_1$$

But why logit or log of odds? A simple response to this question is that the right-hand side of the above equation generates continuous values that range from $-\infty$ to $+\infty$, and hence this cannot be used to generate probability values for dichotomous outcomes, one and zero. Probability always lies between one and zero. So intuitively, it makes sense to have a functional form on the **left-hand side (LHS)** that can generate values beyond zero and one.

But even with odds, that is another form of probability

representation and defined as $\frac{P}{1-P}$, we can only generate LHS

values between zero and $+\infty$. Ultimately, it makes sense to take the natural log of the odds to be the LHS of logistic regression. Log odd values range from $-\infty$ to $+\infty$. Also, it is important to note that conversion to log odds results in symmetry around zero. Refer to the following figure:

Step 1	Step 2	Step 3
Probability	Odds	Log Odds
0.100	0.111	-2.197
0.200	0.250	-1.386
0.300	0.429	-0.847
0.400	0.667	-0.405
0.500	1.000	0.000
0.600	1.500	0.405
0.700	2.333	0.847
0.800	4.000	1.386
0.900	9.000	2.197

Figure 4.3: Probability vs Odds vs Log Odds

Alternatively, we can also understand this using the example of loan approval probabilities. Let us say that the probability of loan approval is .90. Then the odds of loan approval would be $0.9/(1-0.9) = 9$ and the odds of loan denial would be $0.1/(1-0.1)=0.11$. This asymmetry is unappealing because the odds of loan approval should be the opposite of the odds of loan denial. We can take care of this asymmetry through the natural logarithm, $\ln$. The natural log of 9 is 2.197 ($\ln(.9/.1)=2.197$). The natural log of 1/9 is -2.197 ($\ln(.1/.9)=-2.197$), so the log odds of loan approval are exactly opposite to the log odds of loan denial and range from $-\infty$ to $+\infty$. These two properties of log odds make it perfectly suitable for placement on the LHS while having a probability component to it.

The logistic regression equation, shown above, can be repurposed to calculate the probability P . This is illustrated in the following [Figure 4.4](#):

$$\ln \left(\frac{P}{1-P} \right) = \beta_0 + \beta_1 * x_1$$

$$\frac{P}{1-P} = e^{(\beta_0 + \beta_1 * x_1)}$$

$$P = \frac{1}{1 + e^{-(\beta_0 + \beta_1 * x_1)}}$$

Figure 4.4: Logit to probability derivation

As seen above, the logistic regression model shows an S-shaped non-linear relationship between the input x and the output P .

Loss function

A loss function is a measure of fit between a mathematical model of data and the actual data. We choose the parameters of our model to maximize the goodness-of-fit of the model to the data. With ordinary least squares (the only loss function we have used thus far), we minimize the residual sum of squares $\sum (Y_i - \hat{Y}_i)^2$.

With some models, like the logistic curve, there is no mathematical solution that will produce least squares estimates of the parameters. For many of these models, the loss function chosen is called maximum likelihood. A likelihood is a conditional probability (for example, $P(Y|X)$, the probability of Y given X). We can pick the parameters of the model (b_1 and b_2) of the logistic curve) at random or by trial-and-error and then compute the likelihood of the outcome given those parameters. The loss function also known as log loss or binary cross entropy is shown below:

$$\log \text{loss} = -\frac{1}{N} \sum_{i=1}^N y_i \cdot \log(p(y_i)) + (1 - y_i) \cdot \log(p(1 - y_i))$$

Few important points to note with regards to the log loss function:

- If represents the actual class (assume as 1), then $p(y_i)$ is the probability of that class.
- Depending on the actual class of the record, only one of the first or the second terms of the above loss function will be non-zero. For instance, if the actual class is zero, then the second term will be non-zero.
- The predicted probability is typically corresponding to the higher labeled class, for example, 1 and not zero. This means the raw probability will need to be converted to actual probability for the other class labeled zero.
- The likelihood function in product notation is shown below.

$$L(p) = \prod_{i=1}^N p^{y_i} (1 - p)^{1-y_i}$$

- Minimizing the log loss function is equal to maximizing the log-likelihood shown below. This is obtained by taking the log of likelihood function shown above.

$$\log (L(p)) = \prod_{i=1}^N y_i \cdot \log(p(y_i)) + (1 - y_i) \cdot \log(p(1 - y_i))$$

The following figure shows the log-likelihood of the data for a set of predicted probabilities:

Step 1 → Step 2 → Step 3				
Row ID	Actual Class	Predicted Class (1) Probabilities	Actual Class Probabilities	Log (Actual Class Probabilities)
1	1	0.91	0.91	-0.041
2	1	0.90	0.90	-0.046
3	0	0.76	0.24	-0.620
4	1	0.62	0.62	-0.208
5	0	0.58	0.42	-0.377
6	1	0.45	0.45	-0.347
7	1	0.34	0.34	-0.469
8	0	0.13	0.87	-0.060
9	1	0.20	0.20	-0.699
10	0	0.66	0.34	-0.469
Step 4		Log-Likelihood		-3.334

Figure 4.5: Logistic regression sample outcome

The optimization algorithm will choose parameters or the estimates, that result in the maximum log-likelihood computed. These estimates are called maximum likelihood estimates and the techniques actually employed to find the maximum likelihood estimates fall under the general label numerical analysis. This implies that the higher the value of the log-likelihood, the better a model fits a dataset. Also, the actual log-likelihood value for a given model is mostly meaningless, but it is useful for comparing two or more models.

This brings us to **Gradient Descent (GD)**. As discussed in [Chapter 3, Digging into Linear Regression](#), GD is a numerical

method used by a computer to calculate the minimum of a loss function by tweaking the parameters b_1 and b_2 . Since the loss function is related to the maximum likelihood function, the gradient descent is finding a maximum likelihood estimator of a parameter (the regression coefficients). As discussed earlier, with GD, first the computer will pick up some initial estimates of the parameters. Then it will compute the likelihood of the data given these parameter estimates. Then it will improve the parameter estimates slightly and recalculate the likelihood of the data. The GD algorithm will do this forever until it converges, which is usually when the parameter estimates do not change much (usually a change .01 or .001 is small enough to tell the computer to stop).

In summary, **Maximum Likelihood Estimation (MLE)** sets up the optimization problem and gradient descent is a method for finding a specific solution to the optimization problem.

Challenges and assumptions

While logistic regression is a regression technique, it does not share the assumptions of linear regression and it is primarily due to the form of its DV which is logit.

Some key challenges and assumptions related to logistic regression are:

- The DV is dichotomous in a binary logistic regression, and more than two class in a multinomial logistic regression.
- For a binary regression, the factor level one of the DV should represent the desired outcome.
- No multicollinearity between the IVs. The independent variables should be independent of each other. *Spearman's rank correlation coefficient* or *the Pearson correlation coefficient* can be used to assess multicollinearity.
- The independent variables are linearly related to the log odds.
- Logistic regression requires quite large sample sizes. Larger sample sizes enable more reliable results for your

analysis.

- In logistic regression, the observations should not come from repeated measurements or matched data making the observations dependent.
- Lastly, logistic regression works well for cases where the dataset is linearly separable. A dataset is said to be linearly separable if it is possible to draw a straight line that can separate the two classes of data from each other, and this works really well when the DV takes only two values.

Logistic regression result and interpretation

If you are used to doing logistic regression in R or SAS, what comes next will be familiar. Once we have trained the logistic regression model with the `statsmodels` library, the summary method will easily produce a table with statistical measures including p-values and confidence intervals.

Please note that the logit equation defined above ([Figure 4.5](#)) is implemented using the `Logit` class from the `statsmodels` library. The `Logit` class is specifically designed for logistic regression. It takes the dependent variable (y) and the independent variables (x) as arguments.

The code to train and visualize a logistic regression model using `statsmodel` is:

```
import statsmodels.api as sm
#Start with preprocessed and transformed data
y= y_train
X= X_tr_transformed_final
#Instantiate a Logit model and fit the model on Y and X
logit_model=sm.Logit(y,X)
result=logit_model.fit()
print(result.summary())
```

Output:

Logit Regression Results						
Dep. Variable:	Default	No. Observations:	7000			
Model:	Logit	Df Residuals:	6989			
Method:	MLE	Df Model:	10			
Date:	Fri, 19 May 2023	Pseudo R-squ.:	0.4088			
Time:	10:23:15	Log-Likelihood:	-178.44			
converged:	True	LL-Null:	-301.83			
Covariance Type:	nonrobust	LLR p-value:	2.591e-47			
	coef	std err	z	P> z	[0.025	0.975]
grade	0.1652	0.656	0.252	0.801	-1.120	1.450
annual_income	3.602e-06	3.59e-06	1.003	0.316	-3.44e-06	1.06e-05
debt_to_income	-0.0299	0.018	-1.650	0.099	-0.065	0.006
num_active_debit_accounts	-0.0972	0.160	-0.609	0.543	-0.410	0.216
num_cc_carrying_balance	-0.0862	0.111	-0.780	0.435	-0.303	0.130
loan_amount	0.0008	0.000	6.561	0.000	0.001	0.001
term	-0.1194	0.022	-5.547	0.000	-0.162	-0.077
interest_rate	0.1531	0.066	2.325	0.020	0.024	0.282
balance	-0.0005	0.000	-3.953	0.000	-0.001	-0.000
paid_principal	-0.0070	0.001	-7.525	0.000	-0.009	-0.005
paid_interest	-0.0037	0.001	-3.918	0.000	-0.006	-0.002

Figure 4.6: Logistic regression output

The first column shows the value for the parameter estimates or the coefficient. In the preceding model, `loan_amount` = 0.0008, `debt_to_income` = -0.0299. Let us pick `loan_amount` and see how it impacts the chances of loan approval. Increasing the `loan_amount` by 1 unit (\$1) will result in a 0.0008 increase in $\text{logit}(p)$ or $\text{log}(p/1-p)$. Now, if $\text{log}(p/1-p)$ increases by 0.0008, that means that $p/(1-p)$ will increase by $\exp(0.0008) = 1.001$. This is a 1% increase in the odds of loan approval (assuming that the other variables remain fixed).

The fourth column, with the heading $P>|z|$, shows the p-values. A p-value is a probability measure, and p-values above .05 are frequently considered, *not statistically significant*. We can observe that `grade` is considered statistically insignificant even with a high coefficient value. On the other hand, `loan_amount` is considered statistically significant. Some statistical packages like R and SAS have built-in methods to select the features to include in the model based on which predictors have low (significant) p-values, but unfortunately, this is not available in `statsmodels`.

Python provides us with two popular libraries for performing Logistic regression. `Statsmodels` offers modeling from the

perspective of statistics while Scikit-learn offers some of the same models from the perspective of machine learning. Scikit-learn's focus is on optimizing predictive accuracy rather than statistical inference. General guidance is that you should use Scikit-learn for logistic regression unless you need the statistical results provided by statsmodels. Some key comparison points between the two libraries are mentioned in [Figure 4.7](#):

Modeling Concepts		
	 statsmodels	 scikit-learn
Hyperparameter Tuning	The user needs to write a code for model tuning	Efficient model tuning using GridSearchCV
Regularization	No regularization by default	Defaults to L2; Regularization can be turned off using <code>penalty='none'</code>
Intercept	Included by using the <code>add_constant</code> method	Included by default
Model Evaluation	p-values, confidence intervals, and other statistical measures are obtained using the <code>summary</code> method	prediction accuracy is obtained using the <code>score</code> method
Scenario	For statistical inferences	For accurate predictions

Figure 4.7: Statsmodel vs Scikit-Learn

Model interpretability and explainability

The discussion around the two Python libraries in the previous section highlights another critical decision point of interpretability vs. accuracy within the realm of model development. Often developers chase accuracy and sacrifice model interpretability because high accuracy is typically considered a hallmark of superiority. We touched upon the interpretation of linear regression coefficients and other model outputs in [Chapter 3, Digging into Linear Regression](#), and also discussed the same for logistic regression in the preceding section to highlight the importance of learning to interpret basic model statistics before jumping to the accuracy metrics. For logistic regression, the model statistics interpretation was slightly less straightforward due to the involvement of log transformation, but as we move beyond regression techniques, the interpretability of model features or predictors in the context of model objective and input data will continue to become harder. This reduced interpretability

increases model usage risk which needs to be managed by data scientists as the first line of defense.

However, the dilemma between choosing to build a carefully crafted `statsmodel`, which does a good job of providing statistical information for a complete interpretation of model outcomes, and a highly automated scikit-learn approach that sacrifices interpretability is a widespread industry problem.

Another important aspect of model development is explainability. It refers to the human ability to explain specific decisions made by the machine learning model. The rise of explainability is attributed to most ML models being complex and black boxes with no explanation for their actions leading to less transparency. For instance, if a loan is denied, the loan approval system should be able to explain the reason behind the loan denial. While we may be able to interpret the relative feature importance and other model statistics, it is equally essential for us to understand how this combination of important features was used to make a decision on loan denial. Explainability goes a step further than interpretability.

To summarize, the aforementioned dilemma does not imply that accuracy be sacrificed but suggests a future of data mining that balances trust (interpretability and explainability) with accuracy. We will continue to focus on this topic of model interpretability and explainability in all upcoming chapters since these are the foundations of model risk management in the realm of modern data mining.

Performance metrics

Any classification problem is characterized by the level of accurate prediction of the category of a given record based on its corresponding attributes or features. This is also termed model performance and the choice of the most appropriate model performance evaluation metric depends on the aspects of model performance the user would like to optimize. In other words, if we choose the wrong metric to evaluate the classification model, we are likely to choose a poor model.

We discussed some common performance metrics such as MAPE, RMSE, and so on, for linear regression, which has a continuous DV. In the case of logistic regression with a binary DV, the following evaluation metrics apply:

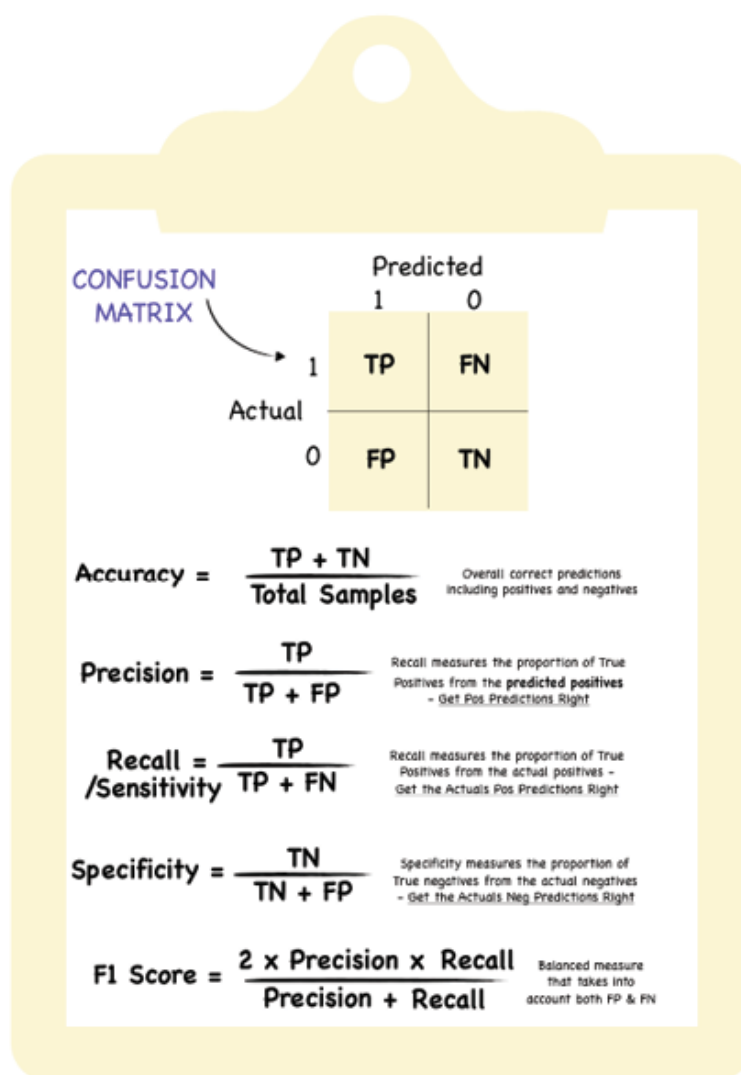


Figure 4.8: Logistic regression evaluation metric

As seen in [Figure 4.8](#), the confusion matrix is a performance measurement summary, that is extremely useful for measuring some of the key performance metrics such as accuracy, recall, precision, and most importantly AUC-ROC curves. All of these metrics also apply to classification problems other than logistic regression. We will discuss these other classification algorithms in

Chapter 5, Decision Tree Algorithm with Bagging and Boosting, and beyond.

Let us understand these metrics based on a dummy sample of 10 records we used in the following figure.

Figure 4.9 expands on this sample and illustrates the predicted class and a label for each row:

Row ID	Actual Class	Step 1	Step 2	Step 3	Step 4
		Predicted Class (1) Probabilities	Actual Class Probabilities	Predicted Class	Label (Threshold=0.4)
1	1	0.91	0.91	1	TP
2	1	0.90	0.90	1	TP
3	0	0.76	0.24	1	FP
4	1	0.62	0.62	1	TP
5	0	0.58	0.42	0	TN
6	1	0.45	0.45	1	TP
7	1	0.34	0.34	0	FN
8	0	0.13	0.87	0	TN
9	1	0.20	0.20	0	FN
10	0	0.66	0.34	1	FP

Figure 4.9: Logistic regression sample outcome expanded

The confusion matrix, and all the metrics calculated from it, based on *Figure 4.9* is shown in the following *Figure 4.10*. It is a table with 4 different combinations of predicted and actual values.

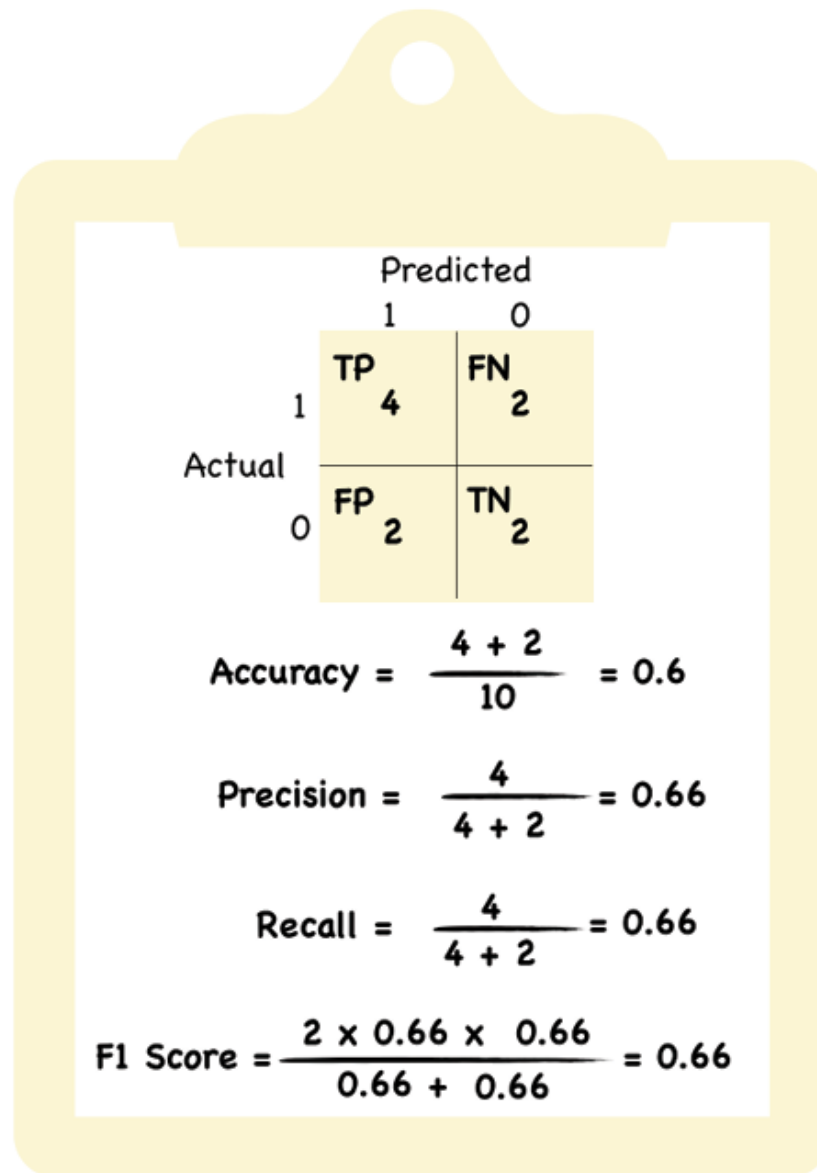


Figure 4.10: Confusion Matrix for binary classification problems

[Figure 4.10](#) illustrates the calculation and summarizes all the classification metrics at a probability threshold of 0.4.

It is important to note in [Figure 4.10](#) that the values of each of these metrics will change as we lower or up the threshold. For instance, if the threshold is set to 0.8 (implying we want only high-confidence cases), the True Positives will reduce from 4 to 2 and the corresponding actual ones (Row ID 4 and 6) will be marked as zeros, indicating 2 additional false negatives cases.

Essentially, by varying the classification threshold from 0 to 1, and calculating the True Positive rate (TPR/Sensitivity) and FPR (1-Specificity) for each of these thresholds, a ROC curve and a corresponding AUC value can be calculated. The diagonal line represents the performance of a random classifier that makes random guesses about the class label of each sample. The dark shaded area under the ROC curve is referred to as AUC in [Figure 4.11](#):

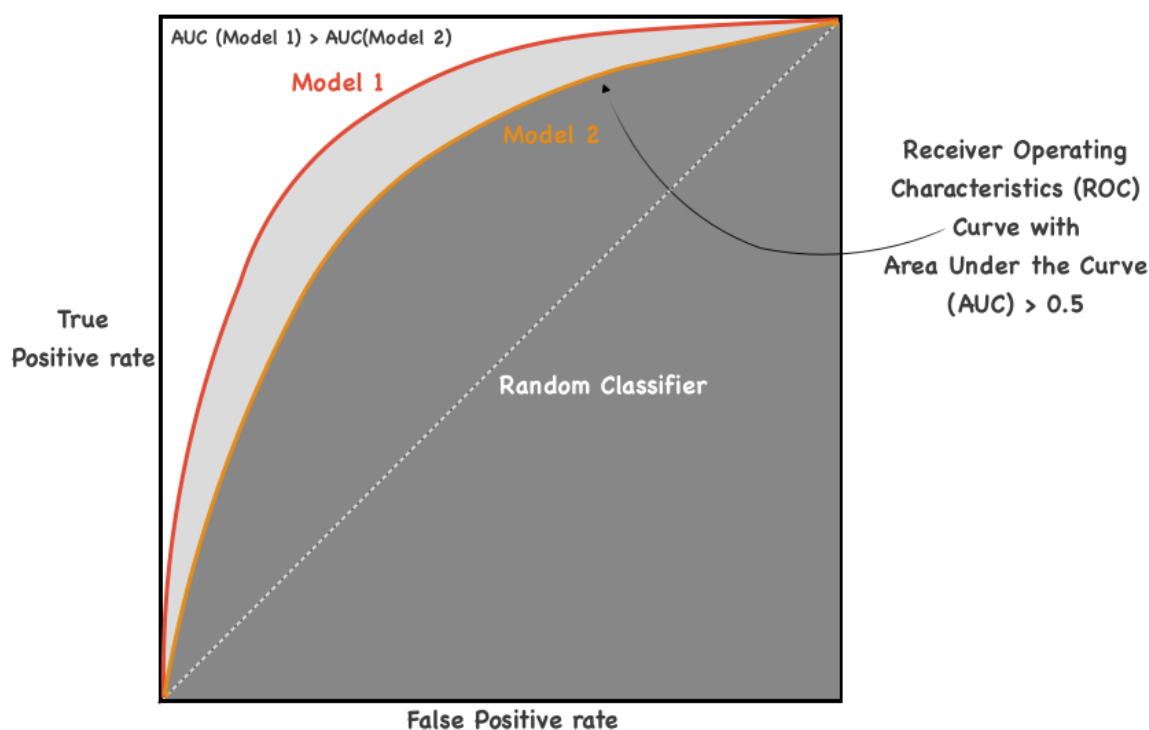


Figure 4.11: ROC curve

AUC measures how well a binary classifier can distinguish between positive and negative classes across different thresholds. This is indicated by the fact that at every threshold value, the model has a better True Positives rate than the False Positives rate.

For the most part, accuracy serves as a simple and intuitive metric to evaluate a model's performance when the classes in a dataset are balanced—meaning if there is roughly an equal number of samples in each class. But it is highly inappropriate for imbalanced because high accuracy can also be achieved by a low-performance model that predicts the majority class (zeros) really

well. In case, when there are a low number of ones in the dataset, the goal is to be able to make the model learn the patterns in the minority class(ones) with high precision. Hence, precision is a much better metric for an imbalanced dataset (minority class<1% of records).

With the knowledge gained so far, let us evaluate the metrics for the logistic regression model on the loan approval dataset. Please remember our loan approval dataset has an approval rate is (Total Approvals/Total Applications)<1%.

#Compare between train & test prediction

```
y_pred_train = result.predict(X_tr_transformed_final)
prediction_train = list(map(round, y_pred_train))
print(classification_report(y_train, prediction_train))
print(classification_report(y_test, prediction))
```

Output:

Training		precision	recall	f1-score	support
	0	0.99	1.00	1.00	6949
	1	1.00	0.24	0.38	51
	accuracy			0.99	7000
	macro avg	1.00	0.62	0.69	7000
	weighted avg	0.99	0.99	0.99	7000

Test		precision	recall	f1-score	support
	0	1.00	1.00	1.00	2978
	1	0.79	0.50	0.61	22
	accuracy			1.00	3000
	macro avg	0.89	0.75	0.80	3000
	weighted avg	0.99	1.00	0.99	3000

Figure 4.12: Performance metrics for the loan approval on the training and test dataset

Please note that in [Figure 4.12](#), the base logistic model trained using the statsmodels library shows a high accuracy score. However since the approval rate is so low, it is better to take note of the f1-score, which provides a balanced view of precision and recall. The overall f1-score suffers due to a low recall, meaning a large number of the actual ones have not been predicted as ones by the model.

Additionally, the trained model is underfitting since the test performance metrics are better. In the case study section ahead, we will try a few other methods to see if the base model performance improves.

Model generalization

Humans learn from available information and can generalize on unseen information on the fly. For instance, they can easily identify a cat as a cat even if they have never seen a specific cat breed before. However, it is a challenge for data mining models. Hence a common practice in model development is to test a trained model on an unseen dataset before passing it off as a final solution. This is done by splitting the entire model dataset into training and test sets. It is generally 80% training and 20% test but can be higher for tests depending on the absolute size of the overall dataset. Regardless, the goal is to train the model on as much data as possible, which is typically 80% of the available data, and then test its performance on the remaining 20% sample. If the model performs poorly on the unseen test data set, we term this as a model overfitting the training set. But this leaves some key questions related to data splitting, which are:

- How do we choose the size of the split effectively?
- Do we need a different strategy in case of smaller datasets when we want to train on as much data as possible?
- Will smaller test dataset-based results be trustworthy?

Cross-validation (CV) can be a useful approach to mitigate the risk of overfitting and can make the difference between a good and bad model. We touched upon this topic very briefly in [Chapter 3, Digging into Linear Regression](#) but will discuss it here

in detail. An important point to note here is that train-test splitting, or cross-validation is applicable across all model development exercises and not specific to classification alone.

K-fold cross-validation

K-fold CV is the most commonly used cross-validation technique other than the simple hold-out method of 80-20 split. Hold-out is great for large datasets when the training could be expensive if performed multiple times. However, this approach of testing the model only once can be problematic if the test dataset is not similar to the training dataset due to improper sampling.

K-fold cross-validation is a technique that minimizes the disadvantages of the hold-out method by splitting the dataset k times into k *folds* or groups. Here k usually is set to 5 or 10. At every split, the model will be trained on $k-1$ folds of the data, while the one remaining fold will be used as a test fold. This process is repeated k times until all folds have been used for validation on the test fold. The k different validation metrics are then averaged at the end to get the final score. The following [Figure 4.13](#) illustrates the process.

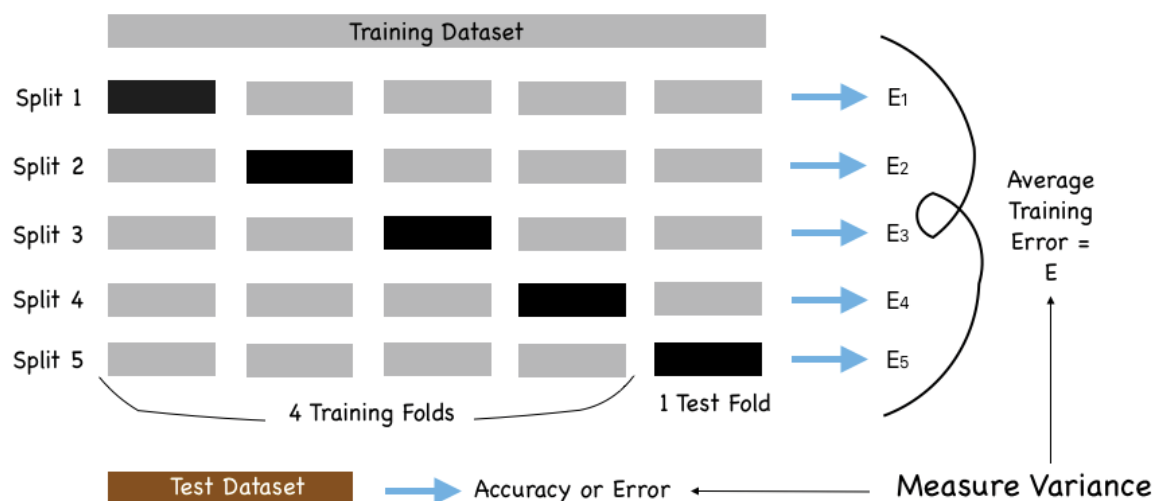


Figure 4.13: K-fold CV process with $K=5$

Another variation of k-fold CV is **Leave-one-out CV (LOOCV)** where $k=n$ (number of observations). While almost all the data is available for training since only one observation is held out for

testing, it requires building n models which are computationally very expensive. It is a general rule to work with a 5 or 10-fold CV over LOOCV. The biggest advantage of a CV is when dealing with a smaller dataset which demands training on the entire dataset. This method is also advantageous because it averages the values of a single performance metric to provide an accurate picture of model performance on a dataset.

In cases where model training is very computationally expensive like in deep neural network models, sometimes CV is not advised, especially with large datasets. Instead, it is better to split the dataset into three parts:

- **Training:** A part of the dataset to train on
- **Validation:** A part of the dataset to validate while training
- **Testing:** A part of the dataset for final validation of the model

Ensemble learning

Unlike the CV concept where multiple homogenous model training performances are averaged to deliver a final model, ensemble learning refers to an approach where multiple final models, across different modeling techniques, are merged to enhance the accuracy and reduce generalization error of the ultimate predictions. There is multiple ensemble learning algorithms that we will evaluate in [Chapter 5, Decision Tree Algorithm With Bagging and Boosting](#), but is important to highlight the concept of ensemble learning in model generalization here. Some key and powerful ensemble techniques are:

- **Max voting:** Predictions from the majority of the models are used as the final prediction and mostly used for classification problems.
- **Averaging:** Applicable for linear regression problems where the output from different models is averaged to derive the final value. For instance, home prices.
- **Weighted averaging:** As an extension of the averaging method, it assigns different weights to various models,

defining the importance of each model for prediction.

Apart from regularization and proper sampling, CV and ensemble learning techniques play a very important role in model generalization.

Model lifecycle processes

Data scientists have tools to build and tune models but that is just the start of the value realization process. They can play a huge role in model value realization by making the model training process transparent and efficient. This way a high-quality and trustworthy model is deployed to production faster for day-to-day business use. The following [Figure 4.14](#) illustrates the model lifecycle from concept to deployment and the post-production monitoring to prevent performance degradation:

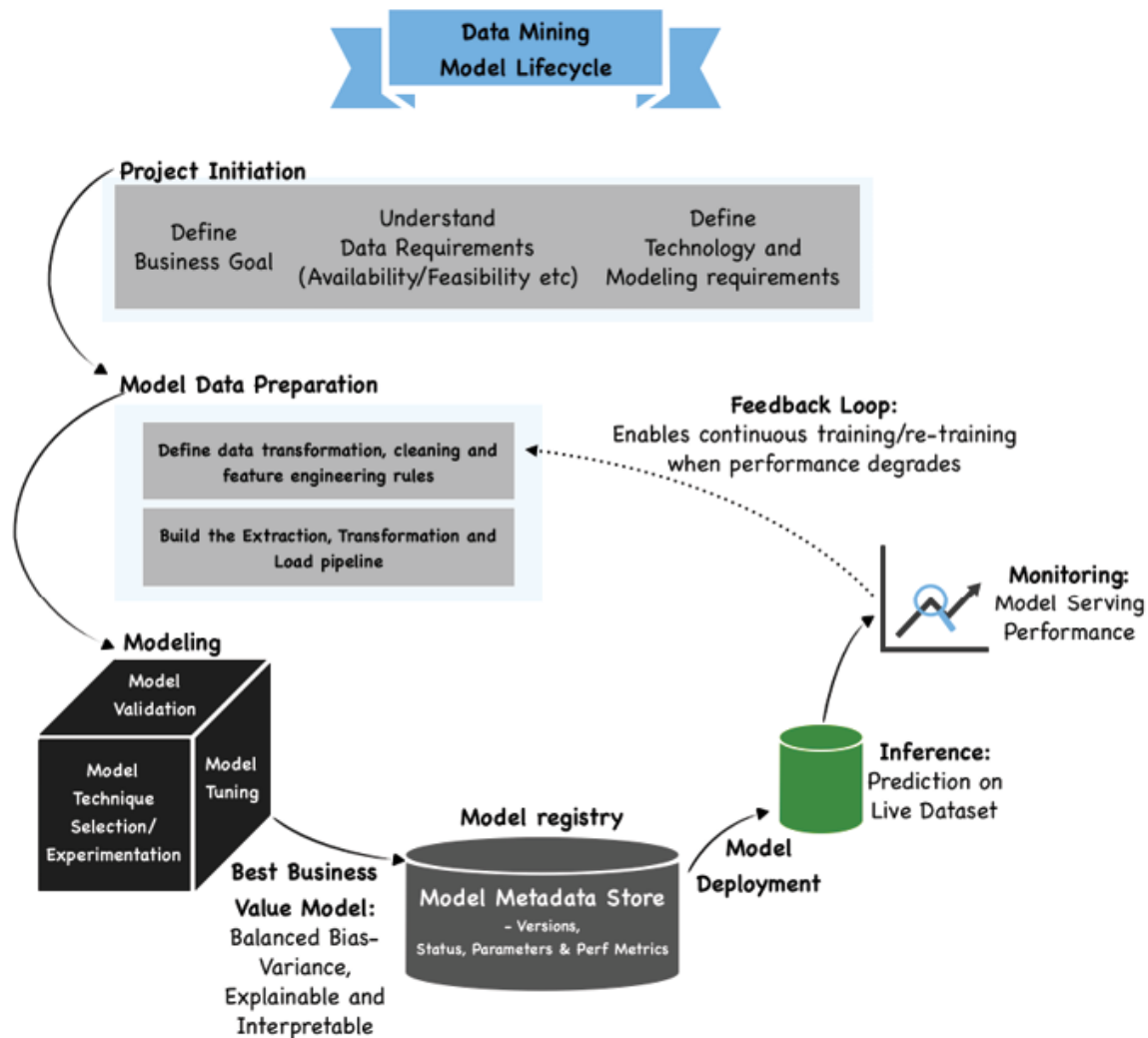


Figure 4.14: The Model Lifecycle

This business value realization needs a set of practices for collaboration and communication between data scientists and model operations professionals (also known as ML Engineers) across the model lifecycle. These practices improve the model quality, simplify the model management process, and automate the deployment of machine learning and deep learning models in large-scale production environments. This way it is easier to align models with business needs, as well as regulatory requirements. It is important for model developers to note that in modern-day data mining, they have a role to play across the model lifecycle. The framework that drives business value across the model lifecycle is called **ModelOps** and developers are key to the framework's overall success. The readers will be introduced to

various ModelOps concepts throughout the book before getting a conceptual and practical deep dive, in [Chapter 12, Understanding Model Risk Management for Data Mining Models](#), and [Chapter 13, Adopting ModelOps to Manage Model Risk](#).

Model development process

As we conclude this chapter on logistic regression and together with a background on linear regression, we are fully equipped to summarize a specific block of the model lifecycle, the model development process, or *modeling*. This knowledge is foundational for building any machine-learning model. [Figure 4.15](#) outlines the typical steps of a model development exercise:

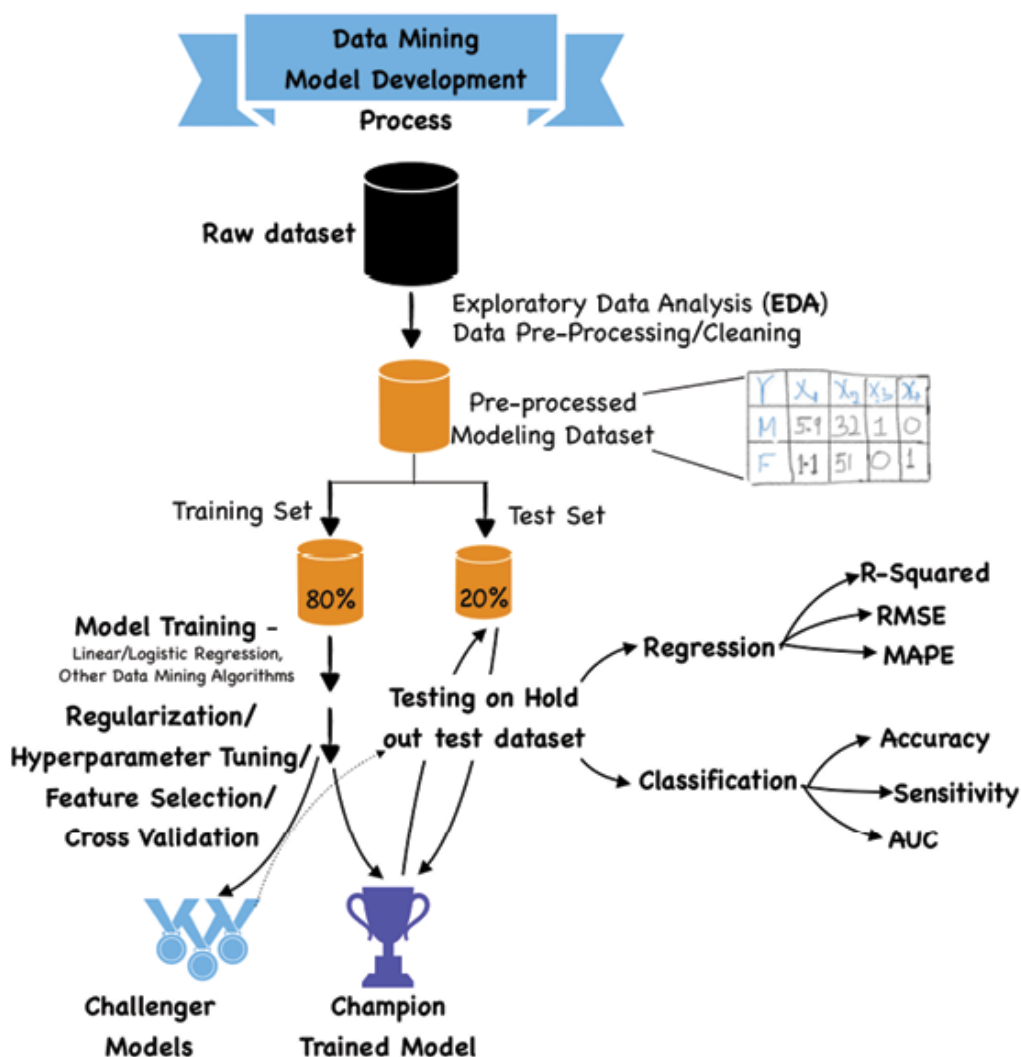


Figure 4.15: *The model development process*

Key points to note with respect to model development are as follows:

- Model development is similar to experimentation, where we build multiple models followed by challenger models to pick the best one.
- We need to keep track of our experimentation exercise to pick the best model.
- Feature selection/cross-validation are meant to reduce the training error but at the same time, we need to perform regularization and hyperparameter tuning to prevent overfitting and thereby reduce variance.
- Final testing on a hold-out test dataset helps with further evaluating model variance.
- Pick the right model performance metrics as per the business goal, data, and algorithm of choice.

Case study: Loan repayment likelihood prediction

We have already looked at the Lending Club loan data and trained a base model, refer to [Figure 4.12](#), on this dataset. In this section, we will try to improve the shortcomings of the base model trained using the `statsmodel` library. However, before we start, let us run the following code:

```
#check target variable distribution  
loan_data.Default.value_counts(normalize=True)
```

Output:

```
0 0.9927
```

```
1 0.0073
```

Name: Default, dtype: float64

The event rate (proportion of 1s) or records with loans approved is 0.73%, indicating that the dataset is imbalanced. This is the primary reason why our base model performed so poorly. But let

us use `scikit-learn` with a few methods such as cross-validation and hyperparameter tuning using grid search to ensure we have covered all bases in terms of data and algorithmic possibilities to improve model performance.

A few things to note before starting model training are as follows:

- Check the train and test data split.

We split the data in a 70-30 ratio to provide some additional test data in a low event rate data set. It is also perfectly right to argue for an 80-20 split and say the developer might need more data for training than testing. There is no one right approach and this decision depends on available modeling data, developers experience, and testing approach, and so on.

```
X_tr_transformed_final=X_tr_transformed[to_keep]
print(X_tr_transformed_final.shape)
X_tst_transformed_final=X_tst_transformed[to_keep]
print(X_tst_transformed_final.shape)
```

Output:

```
(7000, 11)
```

```
(3000, 11)
```

There are a total of 11 shortlisted features in the list `to_keep` with a 70:30 train-test split. The 11 features are the same as reported in [Figure 4.5](#) with the base `statsmodels` model.

Another important thing to note is that feature selection has been performed on the overall dataset using a combination of correlation plots and **Mutual Information (MI)**. The MI score falls in the range of 0 to ∞ , and the higher the value, the closer the connection between this feature and the target.

- Let us start with the basic `scikit-learn` model, the detailed code of which is available in the GitHub file:

```

LR = LogisticRegression(class_weight='balanced')
LR.fit(X_tr_transformed_final, y_train)
#Predict on Train data
y_pred_train = LR.predict(X_tr_transformed_final)
print(classification_report(y_train, y_pred_train))

```

Output:

		precision	recall	f1-score
support				
	0	1.00	0.87	0.93
6949				
	1	0.04	0.78	0.08
51				
accuracy				0.87
7000				
macro avg		0.52	0.83	0.51
7000				
weighted avg		0.99	0.87	0.93
7000				

We can observe that the f1-score has deteriorated further to 0.08 with scikit-learn on training data. Let us try predicting on the 3000-sized test dataset and compare the metrics:

```

#Predict on Test data
y_pred_test = LR.predict(X_tst_transformed_final)
print(classification_report(y_test, y_pred_test))

```

Output:

		precision	recall	f1-score
support				

2978	0	1.00	0.87	0.93
22	1	0.05	0.86	0.09
3000	accuracy			0.87
3000	macro avg	0.52	0.87	0.51
3000	weighted avg	0.99	0.87	0.92

It can be observed that the train and test metrics are roughly the same, but the f1-score and precision are very low.

The following is the code for ROC curve:

```
plt.figure(figsize = (12,8))
plt.plot(fpr_lr, tpr_lr, linewidth=1,
label='Logistic Regression (area = %0.2f)' %
logit_roc_auc)
plt.plot([0, 1], [0, 1], 'r--')
plt.xlim([0.0, 1.0])
plt.ylim([0.0, 1.05])
plt.title("ROC Curves")
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.legend(loc = 'lower right')
plt.show()
```

Output:

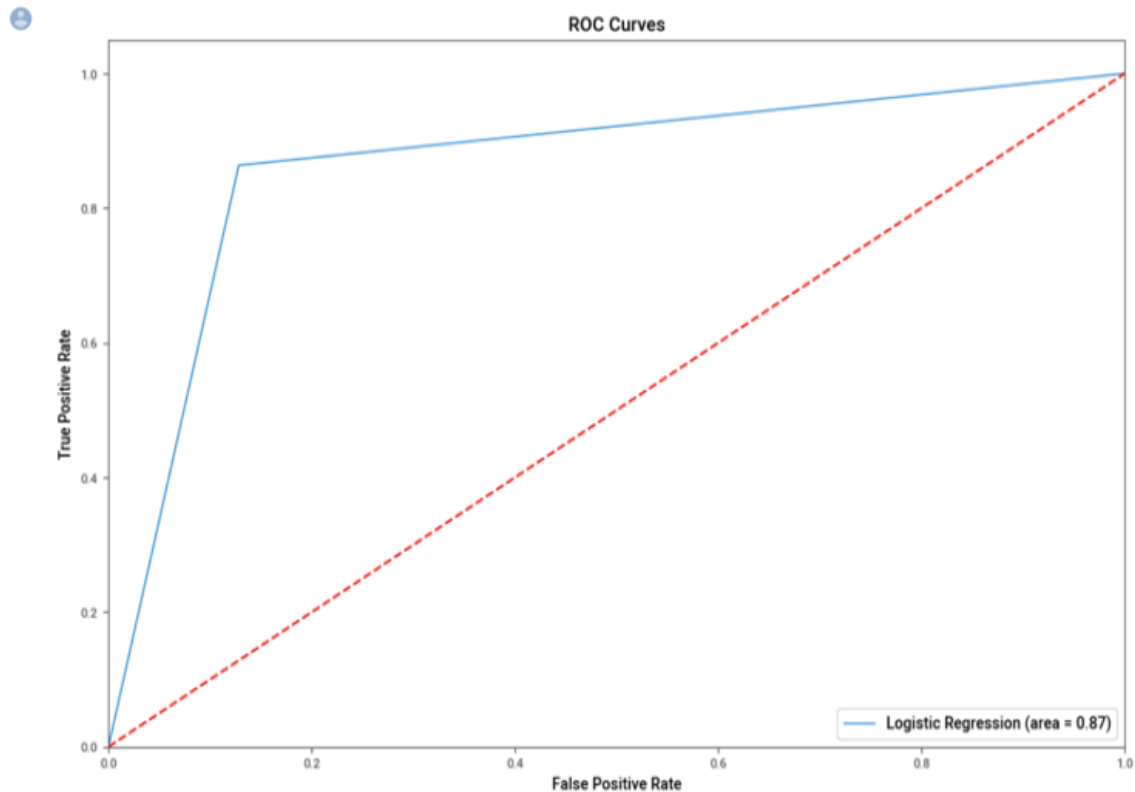


Figure 4.16: The ROC curve for scikit-learn model

- Let us try K-fold cross-validation with K=10.

Code:

```
from sklearn import model_selection
from sklearn.model_selection import cross_val_score
kfold = model_selection.KFold(n_splits=10,
random_state=45,shuffle=True)

model_CV =
LogisticRegression(class_weight='balanced')

scoring = 'accuracy'

results = model_selection.cross_val_score(model_CV,
X_tr_transformed_final, y_train, cv=kfold,
scoring=scoring)

print("10-fold cross validation average accuracy:
%.3f" % (results.mean()))
```

Output:

10-fold cross-validation average accuracy: 0.845

We have a model accuracy of 0.845 with CV which is lower than the `scikit-learn` model accuracy of 0.87. This is because the CV has averaged the accuracies across different folds.

- Lastly, we will perform CV with grid-search hyperparameter tuning.

This will require us to first define a grid of parameters that will be used in various permutations and combinations for training various models. The algorithm will output the best model out of all the iterations of the model training. Grid search is the right tool for efficient model training.

Code:

```
# Set up parameters
penalty = ['l1','l2']
C = [0.005,0.001,0.05,0.01,0.5,0.1]
random_state=[seed+10]

lr_para =
dict(penalty=penalty,C=C,random_state=random_state)

# Grid Search CV

log_reg_cv =
GridSearchCV(LogisticRegression(class_weight='balanced'), lr_para, n_jobs=num_core-1)

log_reg_cv.fit(X_tr_transformed_final, y_train)

print("Tuned hyperparameters as: (best parameters)
\n", log_reg_cv.best_params_)
```

Output:

Tuned hyperparameters as: (best parameters)

```
{'C': 0.01, 'penalty': 'l2', 'random_state': 2033}
```

Next, we can build a model using these parameters.

Code:

```
log_reg_best = LogisticRegression(penalty='l2',
C=0.01, random_state=2033,class_weight='balanced')

log_reg_best.fit(X_tr_transformed_final, y_train)

y_predLR_r =
log_reg_best.predict(X_tst_transformed_final)

print(classification_report(log_reg_best.predict(X_
tst_transformed_final), y_test))
```

Output:

		precision	recall	f1-score
support				
	0	0.87	1.00	0.93
2597				
	1	0.86	0.05	0.09
403				
accuracy				0.87
3000				
macro avg		0.87	0.52	0.51
3000				
weighted avg		0.87	0.87	0.82
3000				

We can clearly see that given the current feature set and available data sample, the f1-score has not improved even after model tuning (**GridsearchCV**) and cross-validation.

In this scenario, we have the following options:

- The cost matrix builds a matrix that assigns a cost to each cell in the confusion matrix. This basically means understanding from the business which specific metric to

focus on. We have used the f1-score, but for them, precision alone could be very important.

- We can use under-sampling techniques (Tomek, Edited Nearest Neighbour (ENN)) which remove the samples of the majority class in order to have an overall more balanced dataset, however, we run the risk of losing information from the majority class.
- We can also try other methods, such as **Synthetic Minority Over-sampling Technique (SMOTE)** to handle this situation which will increase the minority class samples, 1's sample in this case. SMOTE does this by creating synthetic samples of the minority class to make the minority class equal to the majority class.
- A combination of oversampling and undersampling increases the separation between the majority and minority classes. For instance, SMOTE is applied to create synthetic data points of minority class samples, and subsequent ENN application removes the data points on the border or boundary to increase the separation of the two classes.
- We can also play with the different combinations of variables and their transformation.

Having said this, logistic regression is not the right technique for solving classification problems on imbalanced datasets. There are more sophisticated techniques available in the literature that are better able to capture the patterns in the minority class even with lesser samples.

Conclusion

To conclude, logistic regression is a supervised predictive modeling technique that is widely used in various fields, including marketing, finance, social sciences, healthcare, etc. for modeling binary outcomes. By analyzing the relationship between input variables and a binary outcome, logistic regression can predict the probability of an event occurring based on observed data.

One of the key strengths of logistic regression is its simplicity and interpretability. Unlike more complex machine learning algorithms, logistic regression can be easily understood and explained, making it a popular choice for applications where stakeholders need to understand how predictions are being made. This makes it a default technique for most data scientists as they build a base model for solving a business problem.

Despite its simplicity, logistic regression can be highly effective in practice, particularly when used in conjunction with techniques such as feature selection and regularization. By carefully selecting the most relevant input variables and controlling for irrelevant or highly correlated features, logistic regression models can achieve high levels of accuracy and predictive power. This also becomes the drawback of logistic regression since it requires a lot of work to get to a great-performing model.

Another important consideration in model development is the need to be transparent about the assumptions and limitations of the model. By providing stakeholders with a clear understanding of how the model works and what factors are driving its predictions, logistic regression can be a powerful tool for decision-making in a wide range of contexts.

While this chapter concludes the exploration of simple yet powerful regression-based prediction techniques, we have decision tree-based predictions to be excited about in the next chapter. These two are algorithmically different but are highly interpretable supervised learning approaches.

Points to remember

- Logistic regression is a classification technique suitable for linearly separable classes.
- Logistic regression gives a continuous value of $P(Y=1)$, log odds, for a given input X , which is later converted to $Y=0$ or $Y=1$ based on a threshold value. For this reason, it is a regression first and a classifier next.
- Classification performance in general is measured by building a confusion matrix for a given threshold value.

- Various performance metrics that are calculated using this confusion matrix are accuracy, precision, recall, and F1-score.
- The choice of the performance metrics for model evaluation is dependent on the business problem and dataset imbalance, assessed by the event rate of the dataset.
- The ROC curve is built by plotting sensitivity and 1-specificity for different threshold values.
- A good model generalizes well. Regularization and cross-validation are two important techniques for improving model generalization while model training.
- A great model is explainable and interpretable. This is non-negotiable from a model risk management perspective.

Join our book's Discord space

Join the book's Discord Workspace for Latest updates, Offers, Tech happenings around the world, New Release and Sessions with the Authors:

<https://discord.bpbonline.com>



CHAPTER 5

Decision Trees with Bagging and Boosting

Introduction

All models are wrong, but some are useful,
- **George Box, British Statistician**

A lot goes into bringing a model closer to reality and making it useful. So far, we have seen that an individual model with a defined loss function on a certain dataset can deliver an optimal result. Oftentimes, we are not satisfied with the outcomes and wish for an alternative technique working on a different feature set that can deliver better results. We would ideally want this exercise to be done over many iterations with different permutations and combinations of data, model, and loss functions to arrive at the best results. We would also not want to be constrained by the model form, such as linearity between independent and dependent variables, and rather have a technique that assumes no prior form while fitting independent variables to a dependent variable. This is because most relationships between variables in the real world are not linear.

In this chapter, we will talk about the usual *wrongs*, that is, high bias, high variance, or both, of a model and explore innovative ways to make the model *useful*. We will still be solving the same problems of regression and classification but in a more intuitive and unconstrained way. So, let us dive right in.

Structure

In this chapter, we will learn the following topics:

- Decision trees
 - Background
 - Under the hood
 - Data
 - Model
 - Loss function
 - Challenges and assumptions
 - Decision tree results and interpretation
- Ensembling: Bagging, boosting, and stacking
 - Random forest
 - Gradient boosting
 - Ensembling using the stacking method

Objectives

This chapter will discuss, decision trees, a non-parametric method that does not depend upon probability distribution assumptions or assumes any prior form of the model equation. This algorithm is also the basic building block of the other popular tree-based and supervised machine learning algorithms such as Random Forest and Gradient

Boosting. These other sophisticated tree-based methods solve for high variance and high bias using the underlying principles of bagging and boosting and hence are also referred to as ensemble methods. We will discuss all these variants of a decision tree and understand the different parameters that are critical for building these tree-based classifiers and regressors. In the spirit of this book's model risk mitigation theme, we will explain some key concepts using four case studies on the MLFlow and SHAP libraries. This Python library helps with model risk management by incorporating aspects of reproducibility and interpretability in machine learning model training.

Decision trees

To imitate human decision-making, humans-built decision trees, a flowchart-like methodology. We all use decision trees in our day-to-day lives and love the intuitiveness of the process. We will use a simple example to illustrate the concept here: predicting the current month's balance on your credit card.

Background

A decision tree is a supervised machine learning algorithm that uses a set of rules to make decisions, similar to how humans make decisions. For instance, to answer the credit card balance question, we would need to ask a set of questions related to the card's credit limit, the holder's income, marital status, and so on. We will then start forming an initial reasonable range for this card balance which might be accurate to a certain extent. Gradually, we can reduce the prediction range by asking additional questions on education levels, gender, and so on until we are confident enough to make a single prediction. If we are provided a hint or some *supervision* in terms of the exact card balance

number, we can calculate our error in prediction and ask better questions in the second attempt to reduce the error.

This is how we humans make rule-based decisions or judgments all the time. In the case of models, this is called prediction. We usually say that the model predicts the class of the never-seen-before input, but, behind the scenes, the decision tree has to decide which class to assign. It is important to note that decision trees are used for predicting both card balance (continuous DV) and loan approval (binary DV).

Since decision trees can perform both classification and regression tasks, sometimes they are called **CART** algorithms: Classification and regression tree.

Under the hood

With some background on the topic, it is time to explore the inner workings of the decision tree algorithm. To better understand the various decision tradeoffs between relevant values like model accuracy, non-discrimination, etc., data mining users and practitioners need access to three things: the training data, the model algorithm, and the objective function of the algorithm which will be optimized. These components serve well to describe any model-based decision-making. So, let us start with understanding the data needs of decision trees.

Data

We now know that **decision trees (DT)** can closely imitate human- decision making processes and can do more in terms of handling high-dimensional data with good accuracy. In our credit card balance example mentioned above, the dataset has a large number of both categorical and continuous independent variables, which brings some level of complexity to the decision-making by the DT algorithm. This

makes the process different from the human-decision making process, which largely works with a limited number of questions in a yes/no fashion.

For instance, if we phrase our question such that the answer can take multiple values, the number of probable outcomes can become large. So, humans like to ask pointed questions that limit the number of responses. The DT has no such constraint and can handle responses in the form of continuous variables or categorical values with more than three levels.

In our credit card balance example, while gender takes two levels, education can take multiple levels and so does credit-limit. Additionally, the dependent variable, card balance, is a continuous value that can take any value depending on the education, gender, card limit, and income levels of the cardholder. We can import and display the credit card balance dataset using the below Python code:

```
from ISLP import load_data  
  
card_balance = load_data('Credit')  
  
card_balance.head(3)
```

Figure 5.1 illustrates a snapshot of the dataset:

	Income	Limit	Rating	Cards	Age	Education	Gender	Student	Married	Ethnicity	Balance
34	31.367	1829	162	4	30	10	0	0	1	2	0
241	29.705	3351	262	5	71	14	1	0	1	1	148
23	20.103	2631	213	3	61	10	0	0	1	0	0

Figure 5.1: Card balance dataset snapshot

Model

Everything should be made as simple as possible, but not simpler.

- **Albert Einstein**

This theory resonates with the intuition behind DT in which we continue to break down a complex problem into its simpler version. Most DT algorithms, such as *Iterative Dichotomiser* ID3, C4.5, or **Classification and Regression Tree (CART)** are variations of a core algorithm that follows a top-down, greedy search and constructs various possible decision trees in a hypothesis space. This hypothesis space is finite and relative to the available features. This was first documented in Quinlan 1986 and 1993.

As a supervised machine learning model, a DT learns to map input data (credit limit, customer income, and so on.) to output (card balance) in the training phase of model building. This is done by selecting the best feature option available at the moment at a particular node and not worrying whether the current best option will bring the overall optimal result. The DT takes the form shown in Figure 5.2, by evaluating the best feature to select to make the most accurate estimates possible at that time. The algorithm never reverses the earlier decision even if the choice is wrong. This is termed a greedy approach and is employed in a top-down fashion.

It is important to note that a DT uses a condition tuple (feature, value, comparison) to perform binary splits, that is, split the available data into only two parts at any node.

This process continues until either of the two conditions are met, 1) all features have been evaluated or 2) in the case of a classification problem, the training examples in the leaf node belong to just one class, that is node entropy is zero. Alternatively, in the case of a DT regression, the variance from the actual value should be close to zero. We will learn more about entropy and variance in the next section about the model loss function.

Meanwhile, it is important to note that the tree has only learned to map a set of features to the target variable with no knowledge of anything about the target *card balance* itself. In other words, the tree is trained to answer the card balance estimate based on the *best option* feature set it has seen during training. If we provide the decision tree with another feature, outside the best set, it will have no response because it has been trained for one specific task.

In [Figure 5.2](#), the DT makes a *Loan Approved* decision based on salary range using a greedy approach. But you can also qualify for a loan with a *FICO score* > 620 regardless of the salary range. So, how do we know the right order of the questions that need to be asked in a multiple-question scenario?

Secondly, how do we know what questions to ask in the first place OR which questions lead us to the closest prediction to begin with?

In simplest terms, DT is a set of questions leading to a prediction. Others can also represent DT as a set of if-then-else rules to improve the interpretability and readability of the decision.

Let us understand the classification example in the following [Figure 5.2](#) better:

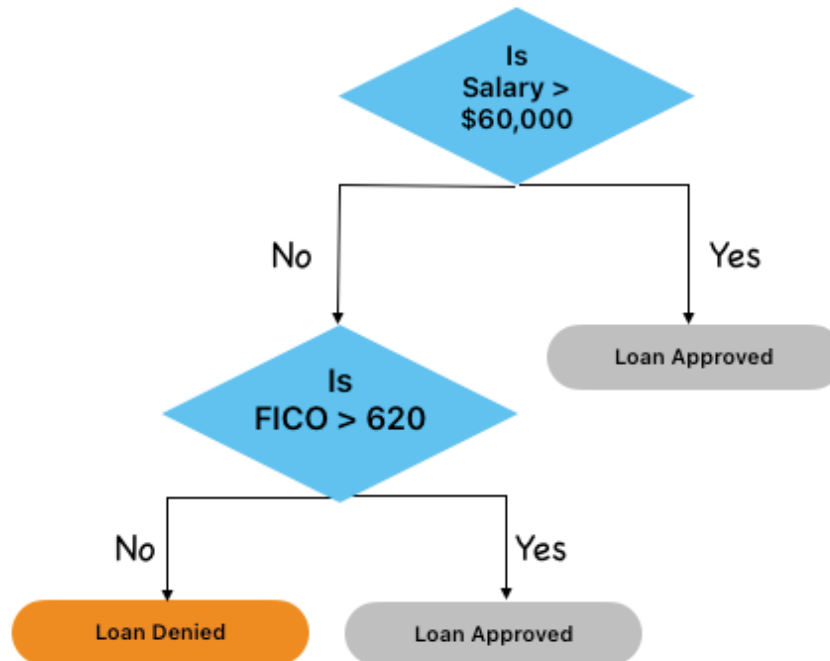


Figure 5.2: Decision tree greedy approach

As seen in [Figure 5.2](#), every DT starts with the root node and grows top-down until a meaningful leaf node, estimating the target value, is reached. Each node in the tree, including the root node, specifies a statistical test of the feature or the question being asked, and each node descending from that node corresponds to one of the possible responses to the question. The most popular statistical selection measures are *Information Gain* and *Gini Gain*.

Let us try to understand this from the perspective of a classification problem. The goal of the statistical test is to determine how well the feature classifies the training data into *pure* nodes. Purity refers to the presence of any one class in abundance in a particular node and is measured by entropy. The following [Figure 5.3](#) explains the entropy calculation with an example of a node with nine Yes and four No:

Yes	9
NO	4
Entropy	0.84

$$\text{Entropy or Info } (D) = - \sum_{i=1}^k p_i \log_2 p_i$$

$$\begin{aligned} \text{Entropy} &= -((0.69 * \log_2(0.69)) + (0.31 * \log_2(0.31))) \\ &\approx -((0.69 * (-0.44)) + (0.31 * (-1.73))) \\ &\approx -(-0.3036 + (-0.5363)) \\ &\approx -(-0.8399) \approx 0.8399 \end{aligned}$$

Figure 5.3: Entropy calculation for a node with two classes Yes/No

Entropy is 1 when a sample has an equal number of positives and negatives and helps in determining a statistical property called **Information Gain (IG)**. Information gain computes the difference between entropies before the split and the weighted average entropy after the split of the dataset based on given attribute values. The following [Figure 5.4](#) shows the Gain calculation:

$$\Delta \text{Gain} (A) = \text{Info} (D) - \text{Info}_A(D)$$

$$\begin{aligned} \text{Information Gain} &= 0.8399 - 0.7837 \\ &= 0.0562 \end{aligned}$$

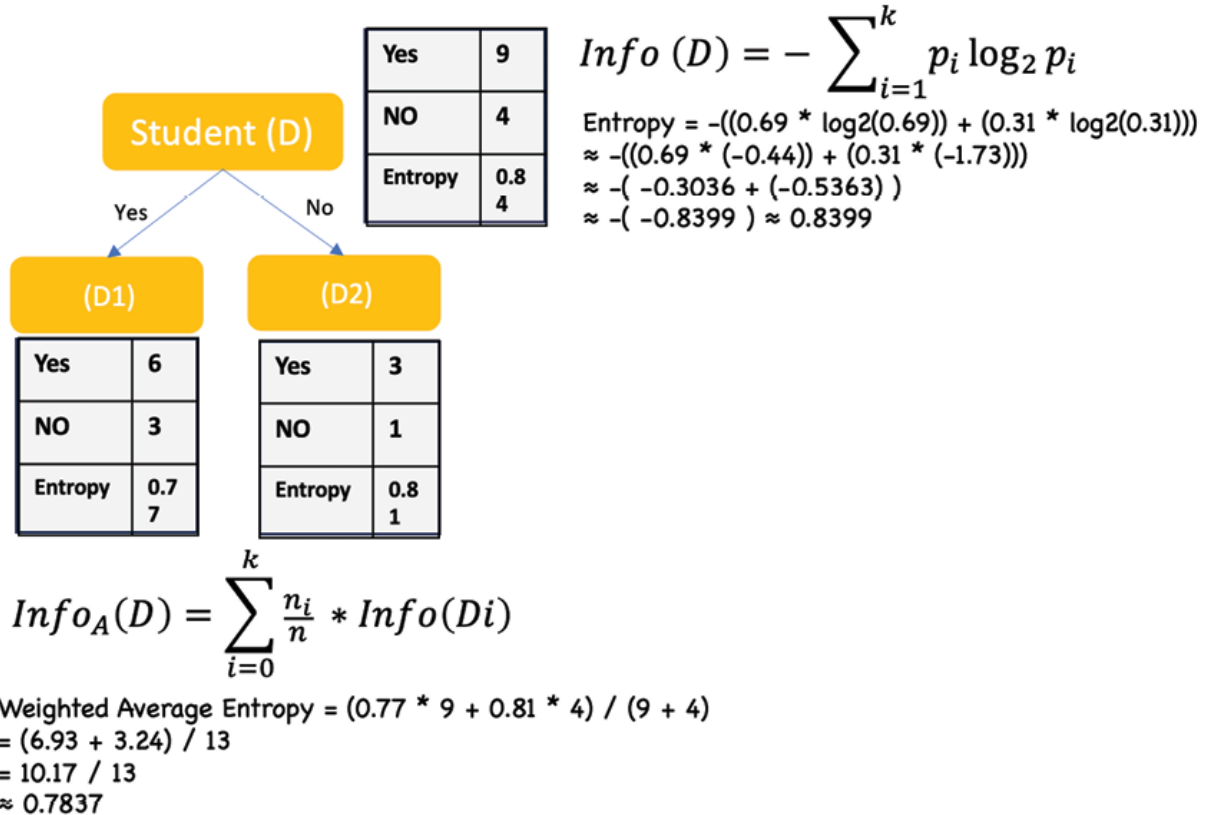


Figure 5.4: Information Gain calculation

The higher the information gain, the better the feature. This helps answer the question of which feature to choose at the root node to split. This process is repeated at every new subsequent node that is created.

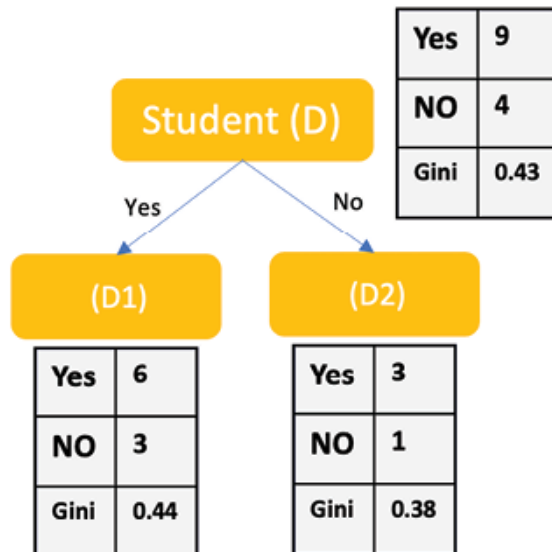
ID3 decision tree algorithm uses information gain. it characterizes the impurity of a random sample of data at a node.

Another decision tree algorithm **Classification and Regression Tree (CART)** uses the Gini method to create split points. The attribute with the minimum Gini impurity index is chosen as the splitting attribute. See [Figure 5.5](#) for the Gini gain and impurity calculation:

$$\Delta Gini(A) = Gini(D) - Gini_A(D)$$

$$\begin{aligned} \text{Gini Gain} &= 0.4278 - 0.4220 \\ &= 0.0058 \end{aligned}$$

An attribute with the maximum Gini Gain is selected for splitting the node.



$$Gini(D) = 1 - \sum_{i=1}^k P_i^2$$

$$\begin{aligned} \text{Gini impurity} &= 1 - ((0.69)^2 + (0.31)^2) \\ &= 1 - (0.4761 + 0.0961) \\ &= 1 - 0.5722 = 0.4278 \end{aligned}$$

An attribute with the smallest Gini Impurity is selected for splitting the node.

$$Gini_A(D) = \frac{n_1}{n} Gini(D1) + \frac{n_2}{n} Gini(D2)$$

$$\begin{aligned} \text{Weighted Average Gini Impurity} &= (0.44 * 9 + 0.38 * 4) / (9 + 4) \\ &= (3.96 + 1.52) / 13 = 5.48 / 13 \\ &\approx 0.422 \end{aligned}$$

Figure 5.5: Gini calculation

The attribute with the maximum Gini gain is chosen as the splitting attribute.

In practice, both Gini impurity and Gini gain can be used as splitting criteria in decision tree algorithms. Gini gain tends to be the more commonly used metric because it focuses on the improvement achieved by a split. By selecting the split that maximizes the Gini gain, decision trees aim to find the splits that result in the greatest reduction in impurity. Ultimately, the choice between using Gini impurity or Gini gain as the splitting criterion depends on the specific decision tree algorithm or library being used and the preferences of the practitioner.

Further, as you can notice, Gini gain uses the Gini index and Information gain uses entropy. The underlying Gini index and entropy have two main differences:

- Gini index values fall between the interval $[0, 0.5]$ whereas the interval of the Entropy is $[0, 1]$
- Entropy is computationally more complex than the Gini index since it uses logarithms

Since the algorithm uses the Gini index to find nodes with maximum purity (all data points belong to one class), it would simply assign the appropriate class in such pure leaf nodes.

In other cases where leaf nodes have a mix of different classes after the split is complete, the algorithm assigns a class to the leaf nodes by max voting. This means the algorithm assigns the most common class among all data points in that node.

Loss function

Since the greedy algorithm chooses the best feature at the node and there is no global strategy, the information gain and the Gini gain are the loss functions that need to be maximized at every node split.

We have already discussed loss in entropy or gain in information for classification problems, so let us take a regression example to understand loss a little differently and more comprehensively. We know from [Chapter 3, Digging Into Linear Regression](#), that the most obvious choice of a regression loss function that measures the error between this actual and the predicted is the quadratic loss, or, the **mean squared error (MSE)**.

The process looks like as follows:

1. The algorithm identifies a set of attributes and splitting conditions tuples (feature, value, comparison)

2. The algorithm splits the node based on the condition tuples and calculates loss (MSE) for each of the two nodes created. Since there are two branches, we define their loss terms individually as L_1 and L_2 . The overall loss for the whole tree is then the sum of these two loss terms: $L = L_1 + L_2$. Individual loss L_1 is defined as shown in [Figure 5.6](#):

$$L_1 = \frac{1}{|D_1|} \sum_{i=1}^n (y_i - \hat{y}_{D_1})^2$$

Figure 5.6: Decision tree Loss calculation

These losses L_1 and L_2 are averaged per data point in the node which makes the loss formula in [Figure 5.6](#) similar to the variance of the numbers in a node.

3. The algorithm chooses the tuple with the lowest loss or variance as the node splitting condition and creates two leaf nodes. This means that loss reduction can be termed as variance reduction as well.
4. Steps 1 through 3 are repeated for each leaf node, which is being treated as the root of a new tree.

So, the above splitting algorithm minimizes the loss or reduces the variance at every node. The overall loss after every split can be calculated by loss before (L_0) and after (L) the split, as shown in [Figure 5.7](#):

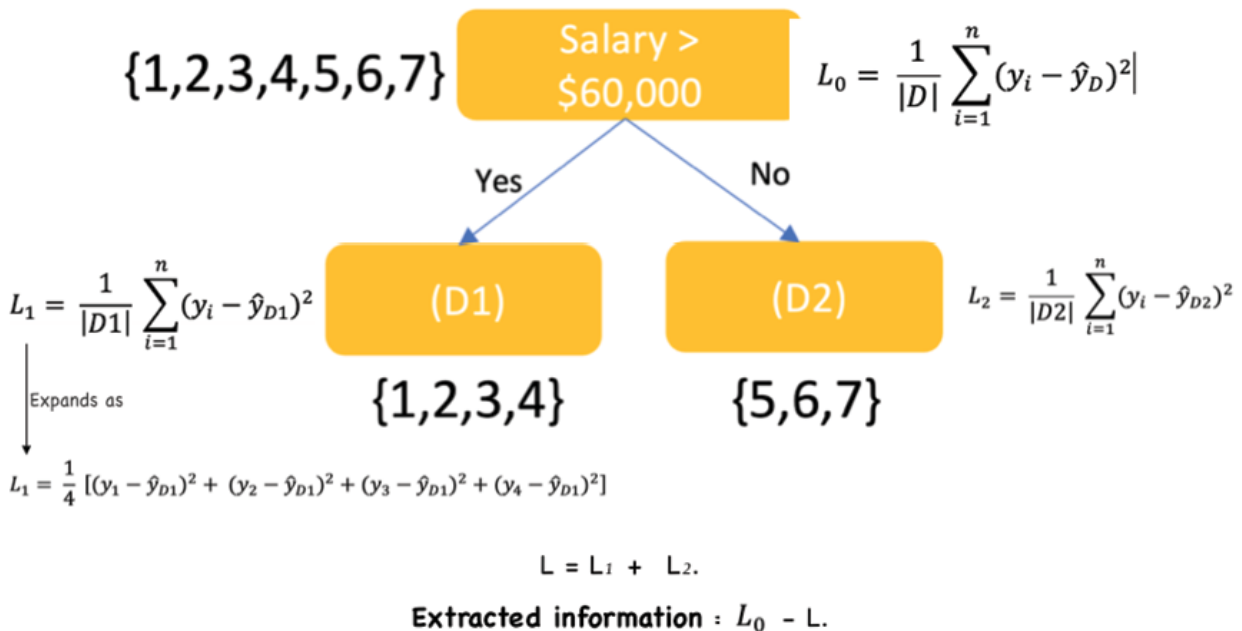


Figure 5.7: Node split with loss calculated at each level

Note: There is no Gradient Descent (GD) applied for the minimization of this loss, since the loss function is just evaluating a split based on the proportion of data points belonging to each class before and after the split.

The goal is to minimize loss at every split, and it is a basic mathematical exercise of finding the minimum loss among many losses.

Challenges and assumptions

Decision trees are easy to understand and do not assume any prior forms for the relationship between the features and the target variable, but they come with their own assumptions and challenges. This algorithm is best suited for problems with the following characteristics:

- DT is a good choice for problems when interpretability is important, and performance takes the back seat.

However, simpler trees (shallow) have high variance, and deeper trees are overfitting on the training data. The depth of the tree is characterized by the total number of edges from the root node to the leaf node in the longest path.

- DT can easily generate biased results with imbalanced or noisy datasets. Extreme care is required to have a good understanding of the data beforehand in such cases
- The upside is no missing value or major data pre-processing such as normalization is required before training. Each feature is evaluated individually by the algorithm
- DTs are very sensitive to the underlying training data since the splitting criteria, or the voting mechanism can get impacted quickly with the introduction of any new data

In the upcoming sections, we will discuss how to address some of the issues related to the DTs and have the best of both worlds of interpretability and performance.

Decision tree result and interpretation

Case study 1: Predicting credit card spending using DT regression

In other words, the goal is to determine what factors influence the credit card balance of any given individual.

Let us start by analyzing the available card balance dataset. A simple code below provides us with the following output.

Python code:

```
#Read Input data
```

```
credit_df = pd.read_csv("./Credit.csv",
index_col=0)

credit_df.head()
```

Output:

	Income	Limit	Rating	Cards	Age	Education	Gender	Student	Married	Ethnicity	Balance
1	14.891	3606	283	2	34	11	Male	No	Yes	Caucasian	333
2	106.025	6645	483	3	82	15	Female	Yes	Yes	Asian	903
3	104.593	7075	514	4	71	11	Male	No	No	Asian	580
4	148.924	9504	681	3	36	11	Female	No	No	Asian	964
5	55.882	4897	357	2	68	16	Male	No	Yes	Caucasian	331

Figure 5.8: Card balance dataset

Post some data treatment, like eliminating records with zero balances (to remove probable inactive accounts) and making the dependent variable **Balance** normally distributed, the data distribution of all the features looks as in the following figure.

Python code:

```
active_credit_df.describe().T
```

Output:

	count	mean	std	min	25%	50%	75%	max
Income	310.0	49.978810	37.881628	10.354	23.15025	37.141	63.74025	186.634
Limit	310.0	5485.467742	2052.451743	1160.000	3976.25000	5147.000	6453.25000	13913.000
Rating	310.0	405.051613	137.967389	126.000	304.00000	380.000	469.00000	982.000
Cards	310.0	2.996774	1.426740	1.000	2.00000	3.000	4.00000	9.000
Age	310.0	55.606452	17.341794	23.000	42.00000	55.500	69.00000	98.000
Education	310.0	13.425806	3.208904	5.000	11.00000	14.000	16.00000	20.000
Gender	310.0	0.532258	0.499765	0.000	0.00000	1.000	1.00000	1.000
Student	310.0	0.125806	0.332167	0.000	0.00000	0.000	0.00000	1.000
Married	310.0	0.619355	0.486331	0.000	0.00000	1.000	1.00000	1.000
Ethnicity	310.0	1.258065	0.834830	0.000	0.25000	2.000	2.00000	2.000
Balance	310.0	670.987097	413.904019	5.000	338.00000	637.500	960.75000	1999.000

Figure 5.9: Card balance dataset variable distribution

A redacted pair plot using the `sns` library showcases the relationship of `Balance` with all other independent variables (last row) using scatter plots along with `Balance` distribution. Please see the following figure:

Python code:

```
# Pairplot using sns
sns.pairplot(active_credit_df , diag_kind = 'kde')
```

Output:

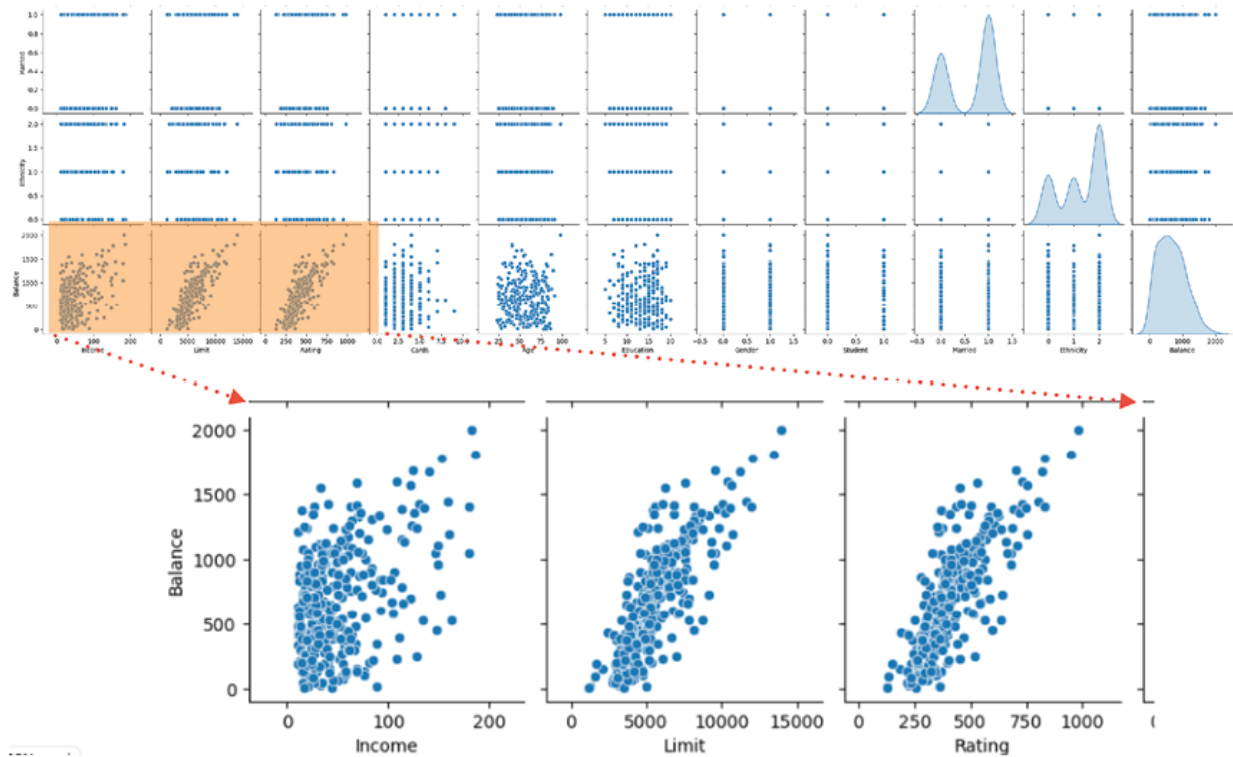


Figure 5.10: Card balance dataset pairplot

We can see in above [Figure 5.10](#) (please refer to the shaded portion of the pairplot output that is zoomed-in for better readability) that **Balance** is linearly correlated with **Limit**, **Rating**, and to some extent with **Income** in the scatter plots but is not the case with other variables such as **Age** or **Education** (*unshaded last row*). Let us evaluate how these variables influence the **Balance** while being part of a DT model.

To track various experiments, we will create a model training and evaluation function using MLflow:

Python code:

```
#define a model training function to use in MLflow
def train(dt_model, X_train, y_train):
    model = dt_model.fit(X_train, y_train)
    train_rsquare = dt_model.score(X_train,
y_train)
```

```

        mlflow.log_metric("train_rsquare",
train_rsquare)

        print(f"Train R Square: {train_rsquare:.2%}")
def evaluate(dt_model, X_test, y_test):
    preds = dt_model.predict(X_test)

    RMSE = np.sqrt(mean_squared_error(y_test,
preds))

    mlflow.log_metric("RMSE_Test", RMSE)

    print(f"RMSE Test: {RMSE:.3}")
experiment_name = "Basic_DT_Regressor"
run_name="Active_Balance"
mlflow.get_tracking_uri()
#Train base model

dt_model = DecisionTreeRegressor()

```

Please note that the primary objective of the above `DecisionTreeRegressor()` is to predict continuous numerical value (Card balance) as opposed to discrete classes. This allows you to specify the criterion used for making splits. Common criteria include `mse` (mean squared error) and `mae` (mean absolute error), which are used to evaluate the quality of a split.

In the case of discrete classes, model training would happen using `model = DecisionTreeClassifier()` and the default hyperparameters may include options like the Gini impurity criterion for splitting. Let us move on to create `MLFlow` experiments.

Add the above model to ML flow experiment. Refer to Chapter 3 for detail

```

#set experiment name
mlflow.set_experiment(experiment_name)
with mlflow.start_run():
    train(dt_model, X_train, y_train)
    evaluate(dt_model, X_test, y_test)
    mlflow.sklearn.log_model(dt_model,
"DT_reg_model")
    test_rsquare = dt_model.score(X_test, y_test)
    mlflow.log_metric("test_rsquare", test_rsquare)
    print(f"Test R Square: {test_rsquare:.2%}")
    #predict your model on train/test data
    pred_dt_train = dt_model.predict(X_train)
    RMSE_Train =
np.sqrt(mean_squared_error(y_train, pred_dt_train))
    mlflow.log_metric("RMSE_Train", RMSE_Train)
    print(f"RMSE Train: {RMSE_Train:.3}")
    print("Model run: ",
mlflow.active_run().info.run_uuid)
    mlflow.set_tag("tag1", "Base Decision Tree")
mlflow.end_run()

print('Run - %s is logged to Experiment - %s' %
(run_name, experiment_name))

```

Output:

```

2023/05/25 22:20:03 INFO mlflow.tracking.fluent:
Experiment with the name 'Basic_DT_Regressor' does

```

not exist. Creating a new experiment.

Train R Square: 100.00%

RMSE Test: 0.449

Test R Square: 81.42%

RMSE Train: 0.0

Model run: 928889f9d8f842bfbff9cc251a5fcf72

Run - Active_Balance is logged to Experiment -
Basic_DT_Regressor

The following code shows the important features of the
preceding DT model.

Python code:

```
print (pd.DataFrame(dt_model.feature_importances_,  
columns = ["Imp"], index = X_train.columns))
```

Output:

	Imp
Income	0.115042
Limit	0.715842
Rating	0.068312
Cards	0.002007
Age	0.004524
Education	0.017571
Gender	0.004301
Student	0.065922
Married	0.001172

Ethnicity 0.005307

In the first iteration of the model, we can clearly see that **Card Limit** is the most important feature, followed by **Income** and **Rating**. This is as hypothesized earlier from the scatterplots in the `pairplot`.

We can drop some unimportant features such as **Married**, **Ethnicity**, **Gender**, **Cards** and **Age** while tuning the DT hyperparameters such as `max_depth`, `max_features`, `min_sample_leaf`, `min_samples_split` using `GridSearchCV`.

Python code:

```
#do hyperparamter tuning once
from sklearn.model_selection import GridSearchCV
param_grid = {
    'max_depth': [2,3,4,5,6],
    'max_features': [2, 3],
    'min_samples_leaf': [3, 4],
    'min_samples_split': [5,10]
}
DT = DecisionTreeRegressor(criterion =
    'absolute_error',random_state = 45)
grid_search = GridSearchCV(estimator = DT,
    param_grid = param_grid,
                                cv = 3, n_jobs = 1,
    verbose = 0, return_train_score=True)
# Fit the grid search to the data
grid_search.fit(X_train, y_train);
```

```
grid_search.best_params_
```

Output:

```
{'max_depth': 6, 'max_features': 3,  
'min_samples_leaf': 3, 'min_samples_split': 10}
```

The **Hypertuned** model with the above hyperparameter values has the following output:

Code:

```
#Hypertuned model
```

```
dt_model = DecisionTreeRegressor(criterion =  
'absolute_error',max_depth=6,max_features=3,min_sam  
ples_leaf=3,min_samples_split=10,random_state=45)
```

```
run_name="Active_Balance Reduced variable  
Hypertuned"
```

```
mlflow.set_experiment(experiment_name)
```

```
with mlflow.start_run():
```

```
    train(dt_model, X_train, y_train)
```

```
    evaluate(dt_model, X_test, y_test)
```

```
    mlflow.sklearn.log_model(dt_model,  
"DT_reduced_Hypertuned_reg_model")
```

```
    test_rsquare = dt_model.score(X_test, y_test)
```

```
    mlflow.log_metric("test_rsquare", test_rsquare)
```

```
    print(f"Test R Square: {test_rsquare:.2%}")
```

```
    #predict your model on train/test data
```

```
    pred_dt_train = dt_model.predict(X_train)
```

```
    RMSE_Train =  
np.sqrt(mean_squared_error(y_train, pred_dt_train))
```

```

mlflow.log_metric("RMSE_Train", RMSE_Train)
print(f"RMSE Train: {RMSE_Train:.3}")
print("Model run: ",
mlflow.active_run().info.run_uuid)
mlflow.set_tag("tag1", "DTree Reduced
Hypertuned model")
mlflow.end_run()
print('Run - %s is logged to Experiment - %s' %
(run_name, experiment_name))
print (pd.DataFrame(dt_model.feature_importances_,
columns = ["Imp"], index = X_train.columns))

```

Output:

Train R Square: 87.86%

RMSE Test: 0.438

Test R Square: 82.30%

RMSE Train: 0.342

Model run: 5491000d85094503bcad4abf09b85597

Run - Active_Balance Reduced variable Hypertuned is
logged to Experiment - Basic_DT_Regressor

	Imp
Income	0.155231
Limit	0.599868
Rating	0.133513
Education	0.059644
Student	0.051744

With this case study, we can conclude that it is always a good exercise to build a hypothesis around important features using the initial EDA and later validate this hypothesis using the model by iteratively eliminating unimportant features from the model. It is important to couple this exercise with model hyperparameter tuning to ensure there is a bias-variance balance. This simple technique will take you to the right model in most cases.

However, in some cases, we will not be satisfied with an R Square of 87.86% or will need an accuracy higher than 90% in case of classification problems. In such cases, we need to explore more complex algorithms such as Ensembles.

Ensembling: Bagging, boosting, and stacking

Decision trees are simple, but the concept is powerful. This concept can be extended to solve some of the common challenges of bias-variance trade-offs related to DTs. This section is going to discuss the popular bagging, boosting, and stacking techniques that have helped data scientists solve these machine-learning challenges related to bias and variance. These learning techniques consist of combining multiple machine learning models and are famously known as Ensemble learning techniques.

The reason behind combining multiple models is the poor performance of most individual models. These low-accuracy models are referred to as weak learners but with research, it has been established that these individual weak learners can generate superior results when they come together.

Since an overfit model has a high variance and low bias (error), and an underfit model has a high bias and low variance, ensemble learning tries to balance this bias-variance trade-off by reducing either the bias or the variance. In essence, various ensemble learning techniques

will aim to reduce the bias in an underfit model and reduce variance in an overfit model.

Bagging for variance reduction: *Bagging*, an acronym for bootstrap aggregation, works by bootstrapping the training data and then building multiple classification or regression DT in parallel on these bootstrapped data samples.

Bootstrapping is a sampling technique in which we create multiple random samples, with replacement, from a single training data set. This seems to give the effect of increasing the training data and can be a useful technique when the training dataset is small.

For a regression problem, the final prediction is the average of all the DT models created in parallel whereas for classification, max voting, or the class that receives the majority of the votes, from different models, is returned by the ensemble model. It is worth noticing that ensembling is working on a set of homogenous models, that is DTs.

Bagging brings the *wisdom of the crowd* together and does not rely on one model for prediction. [Figure 5.11](#) illustrates the three-step bagging process:

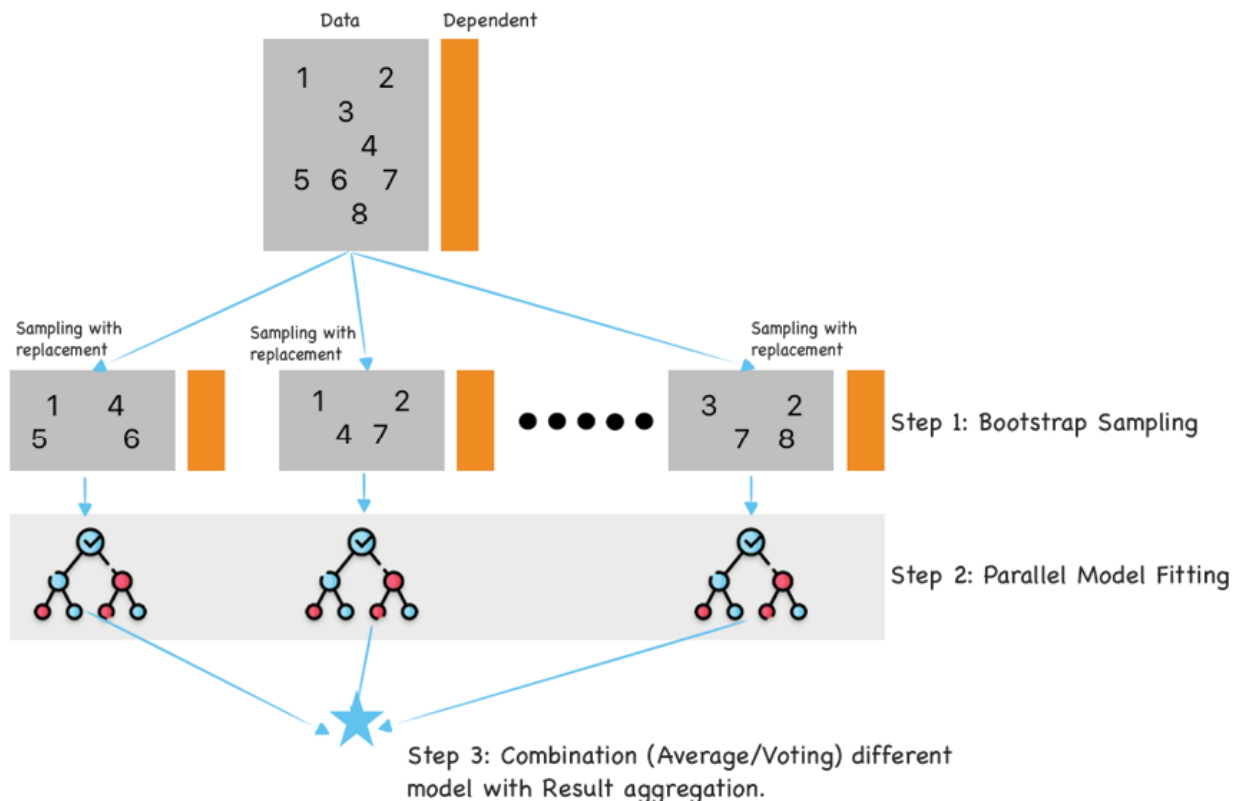


Figure 5.11: The bagging processes

Boosting for bias reduction: *Boosting is a sequential method that produces an ensemble model that is in general less biased than the series of weak learners that built it.* Conceptually the training of a model at a given step depends on the models fitted at the previous steps and the focus of the current step model is the high error observations from the previous steps. This sequential error reduction focus of the algorithm delivers a strong learner with a lower bias at the end of the process. With this technique, caution must be exercised in training models since the effort to reduce bias can lead to overfitting training data.

Figure 5.12 shows the sequential boosting process where the existing trees in the model do not change as a new tree is trained. The newly trained tree fits the residuals from the previous model.

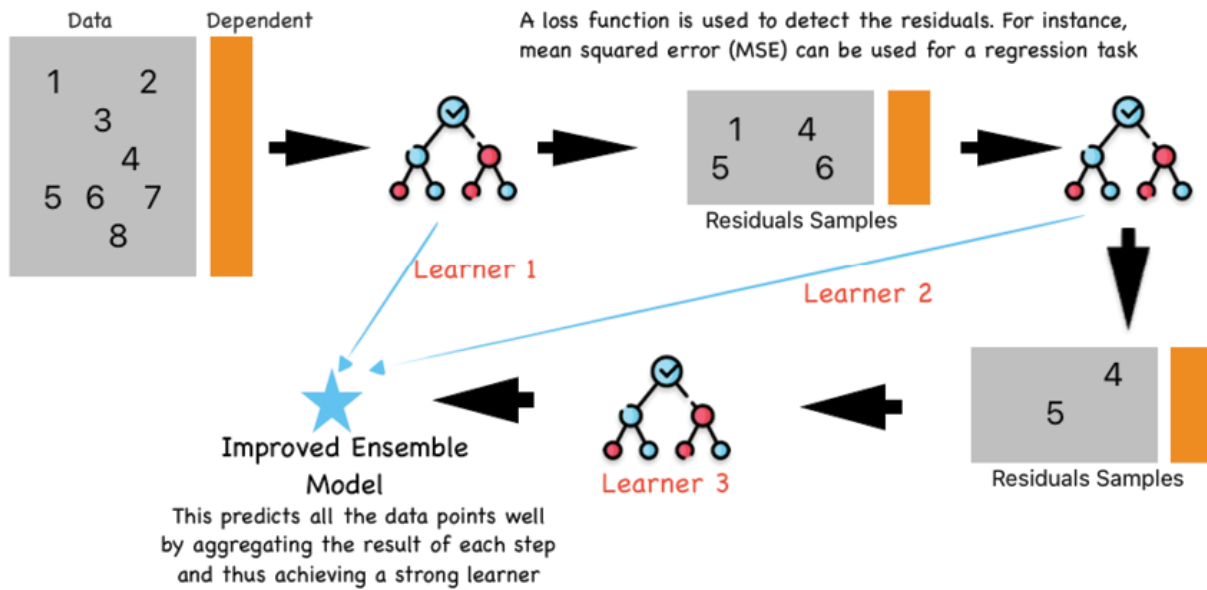


Figure 5.12: The boosting process

Stacking for accuracy improvement: Stacking is designed to ensemble heterogeneous base models together to obtain a meta-model with superior overall performance. The meta-model is trained on the following:

- An out-of-sample data that none of the base models have seen during the training
- This meta-model training happens on the base model predictions as the input variables and the expected output as the target variable.
- Since the goal of the meta-model is to simply combine the predictions of base models, it is wise to not use complex algorithms for training. Regression algorithms such as linear and logistic can serve the purpose well.

The expectation from this exercise is to reduce bias but care should be taken to not worsen the variance situation. The sole goal of the exercise is to build a meta-model that will perform better than any single base model.

It is recommended that if this higher performance cannot be achieved, we should revert to using the best base model.

Figure 5.13 demonstrates the stacking process:

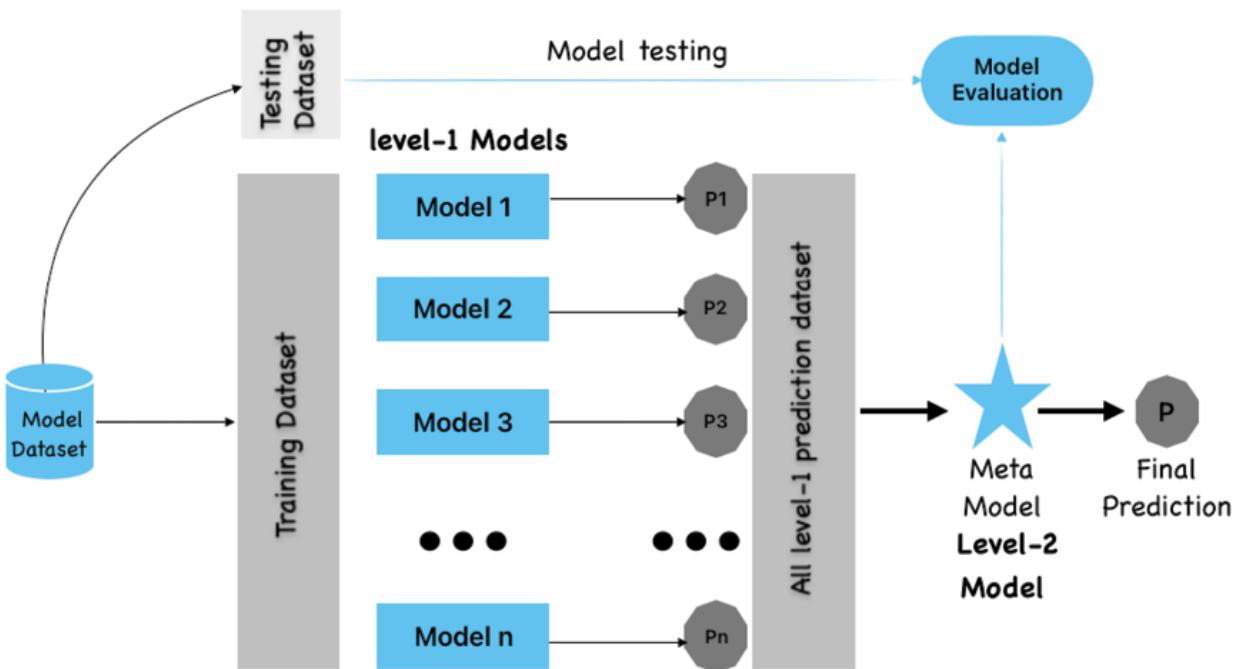


Figure 5.13: The stacking process

Random forest

*Bagging is a useful technique and a very popular bagging algorithm widely used in the industry is known as **random forest (RF)**.* This is based on an ensemble or forest of homogenous decision tree algorithms. We already know that bagging helps in reducing the variance of deep trees and combining multiple such trees can reduce their variance as well when their bias is already low due to the bigger depth of these trees.

Another important point to note about a random forest approach is feature sampling. We all know that bootstrapping helps in a random sampling of rows, but each of these samples also has randomly selected features. This leads to a minimum correlation between various DTs developed as part of the RF process. Additionally, since the

algorithm is not dependent on just one set of features for prediction, RF is largely immune to missing data and outliers.

In summary, a large group of uncorrelated decision trees can produce more accurate and stable results than any individual DTs. Now let us try to understand the RF algorithm execution using an example. The following steps are involved:

1. Choose a bootstrap observation sample and feature sample from the training dataset. The sample size can be similar to the training dataset size but will have only some randomly chosen records. For instance:

Training_set = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10], Sample_1 = [9, 4, 6, 6, 4, 2, 3, 2, 5, 9]

2. Train a decision tree on `Sample_1` and leave out an **out-of-bag (OOB)** sample of records that do not appear in `Sample_1`. `Sample_OOB_1=[1, 7, 8, 10]` is used as a validation set for the decision tree built using `Sample_1`. The validation error on this `Sample_OOB_1` set is referred to as the OOB error
3. Repeat steps 1 and 2 for multiple bootstrapped samples.
4. Aggregate the results from multiple DTs; For instance: In case of a classification problem, out of n trees, if $n-5$ trees predicted the observation label as 1 and five trees predicted the label as 0, and if $n-5 > 5$, then the aggregated predicted label is 1.
5. Test the model using a test set and get the accuracy score.

Figure 5.14 shows the steps of the random forest training as previously described:

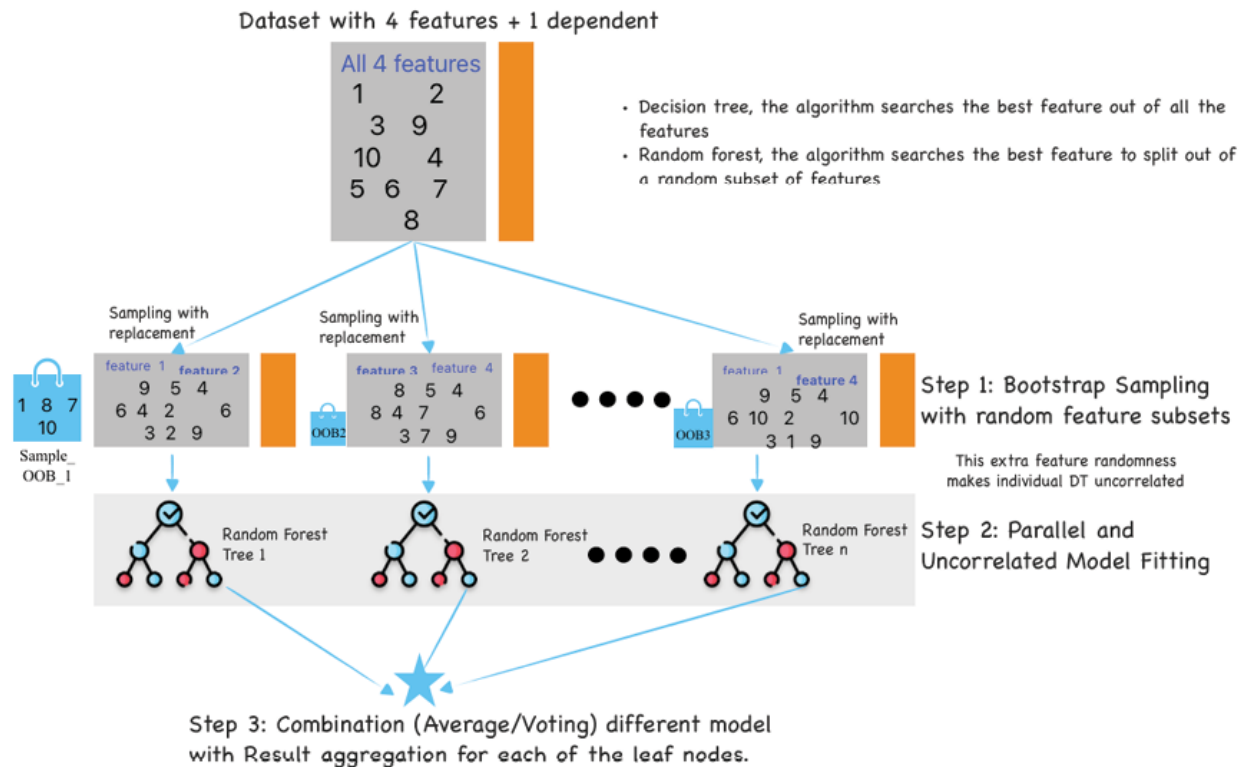


Figure 5.14: The Random Forest processes

Some key model hyperparameters to tune are listed as follows:

- **n_estimators:** The number of decision trees in the forest that the algorithm will build. The total trees can be equal to the number of observations in your training set, but this would mean slower training time.
- **max_depth:** The maximum depth of the tree. This can be obtained using good judgment but a grid search on a single decision tree is an excellent way to identify this depth. Thereafter this value can be used for an RF classifier for the same dataset.
- **bootstrap:** It is used to perform bootstrap sampling to get uncorrelated trees. This is set **True** by default.
- **oob_score:** It is used to perform an **Out-of-bag (OOB)** evaluation of each of the trained decision trees. This slows down the training process, but a high **oob_score**,

similar to cross-validation, is an indicator of less variance.

From a model interpretability perspective, it is important to note that RF produces a feature importance score for each feature used during the training. This is the relative importance of features used and the importance scores sum up to one. A higher score denotes higher feature importance. Additionally, a low feature importance only means that the feature wasn't used for training the specific RF model. If a feature continues to come up as less important with different train-test data splits, we can be sure that the feature has zero importance.

We will understand all of these concepts in *Case study 2* in the upcoming section.

Gradient boosting

Boosting focuses on reducing bias for the base models or weak learners with a low variance but high bias. This could be a shallow tree and can benefit from any approach that sequentially combines the predictions of multiple weaker models to create a robust, more accurate model. While there are multiple boosting techniques with their respective pros and cons, in this chapter we will be discussing gradient boosting and its improvised version of extreme **gradient boosting (Xgboost)**.

The Gradient boosting algorithm follows the following steps:

1. The algorithm tries to reduce the error from the predecessor decision tree by creating a new tree to fit the residuals/error of the predecessor tree. The residual values are the target variable for the new tree.
2. Then the algorithm computes the value of the optimal step size or **learning rate (LR)** and calculates the new prediction for all rows using the formula:

$$\text{new-prediction} = \text{old-prediction} + \text{LR1} * \text{Residual1}$$

3. With the new prediction, the algorithm calculates the new residuals ($\text{Res2} = \text{Original target} - \text{new-prediction}$) and fits a new tree on these residuals.
4. Steps 1-3 are repeated until there is convergence and the residual is sufficiently minimized.

The final result is the outcome of the above process that translates as below:

$$\text{Final-prediction} = \text{First-prediction} + (\text{LR1} * \text{Res1}) + (\text{LR2} * \text{Res2}) + \dots + (\text{LRn} * \text{Resn})$$

Figure 5.12 shows the gradient boosting process.

XGBoost builds on this idea of Gradient Boosting by using decision trees as the base models and a gradient descent algorithm to improve the model's predictive performance.

An important feature of the XGBoost algorithm is its ability to handle missing values and imbalanced data. However, it is prone to overfitting since the goal is to reduce training error. Cross-validation can help in keeping a check on the variance going wild. Additionally, XGBoost is fast and scalable, making it a good choice for large-scale and high-dimensional data.

A summary of different boosting methods is given as follows:

- **Light Gradient Boosting Machine (LightGBM)** is a gradient boosting framework that uses tree-based algorithms and follows the principle of leaf-wise growth, as opposed to depth-wise growth. In the leaf-wise growth strategy of LightGBM, the algorithm selects the leaf node that will result in the largest reduction in the loss function (typically measured by gradient descent) and splits it to create two child nodes. This process continues recursively, selecting the leaf with the highest gain and splitting it, until a

stopping criterion is met (for example, maximum tree depth or minimum number of data points in a leaf).

- Like XGBoost, LightGBM is an ensemble learning algorithm that combines the predictions of multiple weak models to create a robust and more accurate model. The main difference between the two is that LightGBM uses a leaf-wise tree growth algorithm, which tends to converge faster than the depth-wise algorithm used by XGBoost. This approach makes LightGBM faster and more efficient, especially on large-scale data.
- *CatBoost is an open-source gradient-boosting library designed to work with categorical data.* One of the main advantages of CatBoost is that it can automatically handle categorical variables without any preprocessing, which is a common problem in machine learning. This algorithm is suitable for working with datasets with multiple categorical features.
- Adaptive boosting (often called **adaboost**), we try to define our ensemble model as a weighted sum of L weak learners. To make predictions with the AdaBoost model, each weak learner's prediction is combined using a weighted voting system, where the weights are determined by the accuracy of the weak learner.

Additionally, a summary table with the pros and cons of different boosting techniques is shown in the following [Table 5.1](#):

Boosting method	Pros	Cons
Adaboost	• Simple and intuitive algorithm • Can be less prone to overfitting	• Sensitive to noisy data and outliers • Sequential training makes it harder to parallelize

	• Works well with weak learners • Good for binary classification tasks	• May require tuning of hyperparameters for optimal results
CatBoost	• Handles categorical variables automatically • Built-in handling of missing values • Robust to outliers and noisy data • Fast and efficient training	• Relatively slower training speed • Requires more memory for training • Longer model training time • May require tuning of hyperparameters for optimal results
LightGBM	• Fast and efficient training • Low memory usage • Handles large-scale datasets • Good for handling high-dimensional features	• Prone to overfitting if not properly regularized • Requires tuning of hyperparameters for optimal results • Not as effective with small datasets
XGBoost	• Strong performance across various tasks • Effective handling of missing values and outliers • Provides feature importance scores • Supports parallel and distributed computing	• Requires careful tuning of hyperparameters • Can be memory-intensive, especially for large datasets • Longer training time compared to some other methods

Table 5.1: Comparison of different Boosting methods

It is time to use the knowledge gained so far to train some models. We will start with the following case study.

Case study 2: Predicting customers that will subscribe to a term deposit based on a recent bank marketing campaign using Logistic Regression, RF, and XGBoost

The modeling data relates to the direct marketing campaigns of a Portuguese banking institution. The classification goal is to predict if the client will subscribe (yes/no) to a term deposit (variable `y`) based on a marketing call. Here the `Target` column, has the client subscribed to a term deposit? (binary: *yes, no*), is our dependent variable. Rest all variables are independent.

In this exercise, we are going to build three different models using the Logistic Regression, RF, and XGBoost techniques. This is done to contrast the performance of these three entirely different techniques and record this experiment using the `MLflow` library. We have seen in the previous chapters that `GridSearch` is an effective method for hyperparameter tuning during model development. However, sometimes with large datasets, this method can slow down the model development process.

For this reason, we plan to evaluate the `hyperopt` library, which uses a form of Bayesian optimization for parameter tuning and import some of its features such as `STATUS_OK`, `Trials`, `fmin`, `hp`, `tpe` for finding the best parameters for the three algorithms in consideration. The `hyperopt` advantage is that it allows the algorithms to search more efficiently by providing more information about where the function is defined, and where we think the best values are.

Like always, we will also use `MLflow` to log the three model outcomes in an experiment *Ensemble classification*. So let us start by understanding the data.

Data profiling using the library `ydata_profiling`. This will generate a report that not only summarizes the different continuous and categorical variables but also has a smart `Alerts` section that outlines the summary statistics with meaningful commentary. [Figure 5.10](#) illustrates the output excerpt generated by using the following code:

Python code:

```
#!pip install pandas-profiling
from ydata_profiling import ProfileReport

profile = ProfileReport(Bank_df, title="Profiling
Report")

profile
```

Output:

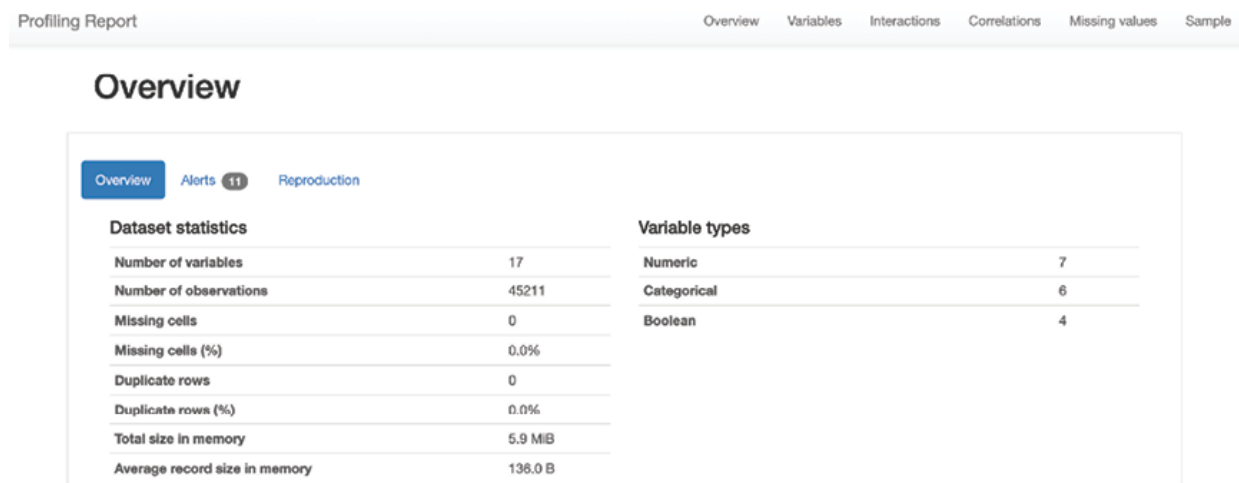


Figure 5.15: The overview and alerts screenshot

The correlation plot generated as part of the report by the above code is seen as follows:

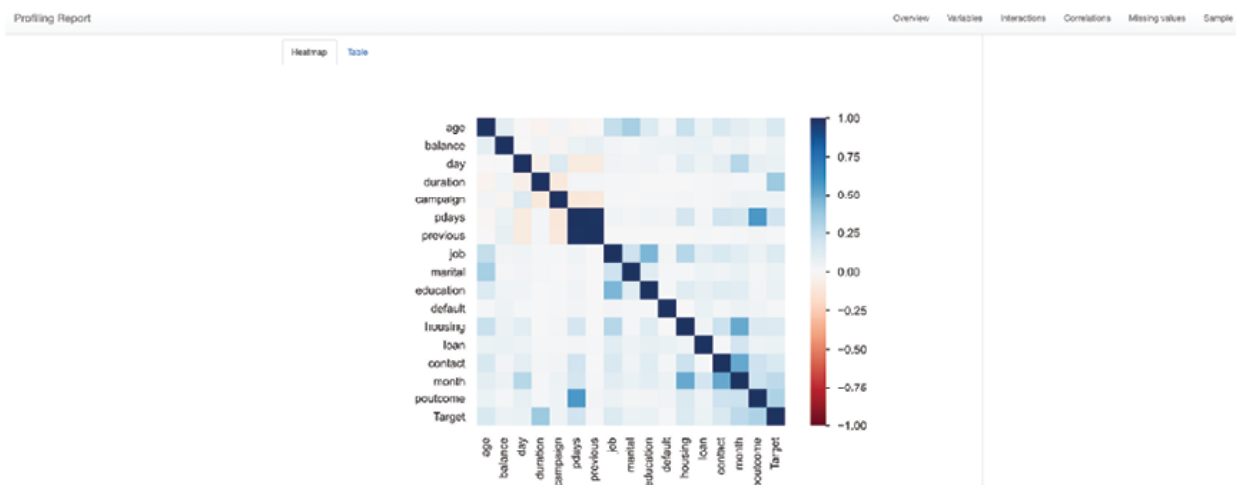


Figure 5.16: The data profiling correlation plot

The `sample` variable summary generated as part of the report is seen as follows:

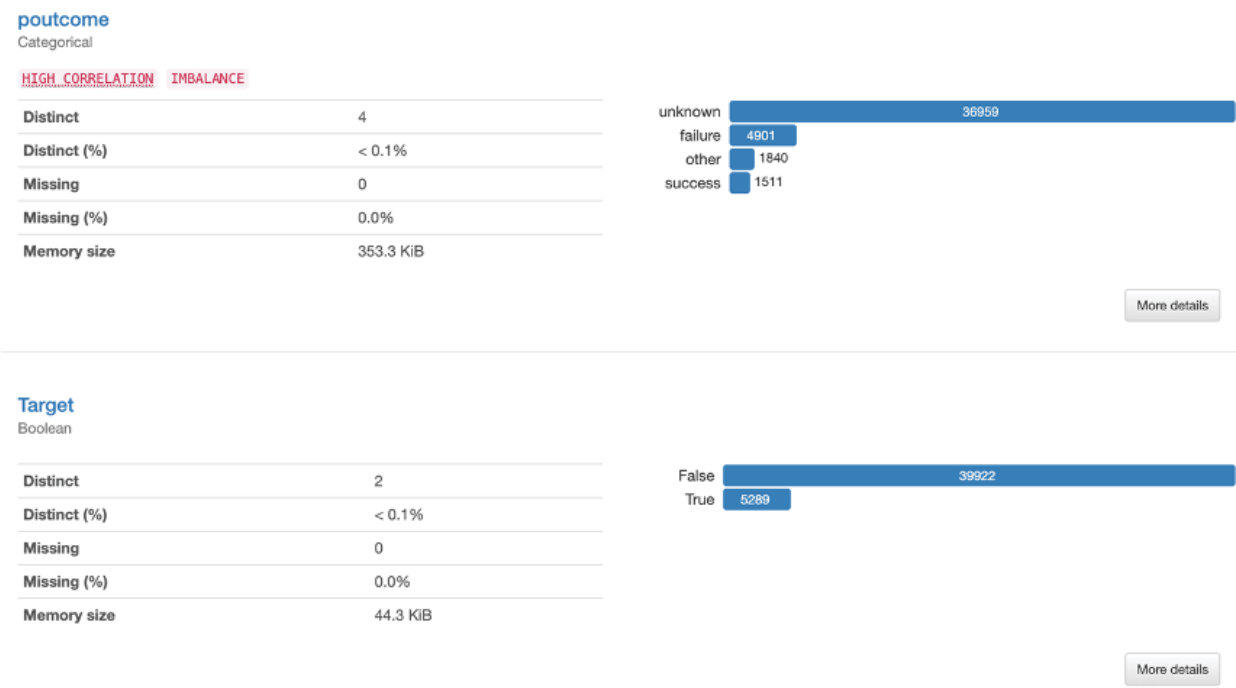


Figure 5.17: The data profiling day-month variable overview

We can see in [Figure 5.17](#) that the Target variable has an event rate of 11.7% (5289/45211). This is a fairly balanced dataset and any accuracy, roughly above 90% means that the model is not only able to predict the majority class 0 well but predict a good number of actual “1”s as “1” as well.

The sampled first 10 rows were generated as part of the report are shown below:

Sample

First rows		Last rows														
age	job	marital	education	default	balance	housing	loan	contact	day	month	duration	campaign	pdays	previous	poutcome	Target
58	management	married	tertiary	no	2143	yes	no	unknown	5	may	261	1	-1	0	unknown	no
44	technician	single	secondary	no	29	yes	no	unknown	5	may	151	1	-1	0	unknown	no
33	entrepreneur	married	secondary	no	2	yes	yes	unknown	5	may	76	1	-1	0	unknown	no
47	blue-collar	married	unknown	no	1506	yes	no	unknown	5	may	92	1	-1	0	unknown	no
33	unknown	single	unknown	no	1	no	no	unknown	5	may	198	1	-1	0	unknown	no
35	management	married	tertiary	no	231	yes	no	unknown	5	may	139	1	-1	0	unknown	no
28	management	single	tertiary	no	447	yes	yes	unknown	5	may	217	1	-1	0	unknown	no
42	entrepreneur	divorced	tertiary	yes	2	yes	no	unknown	5	may	380	1	-1	0	unknown	no
58	retired	married	primary	no	121	yes	no	unknown	5	may	50	1	-1	0	unknown	no
43	technician	single	secondary	no	593	yes	no	unknown	5	may	55	1	-1	0	unknown	no

Figure 5.18: The data profiling day-month-target variable overview

Now, as we understand the data well, let us dive right into training our first model of logistic regression. We will first split the data into training and test datasets.

The code is as follows:

```
y = Bank_df['Target']  
  
df_train, df_test =  
train_test_split(Bank_df, test_size=0.3, random_state  
= 12, stratify = y)  
  
df_train.shape, df_test.shape
```

Output:

```
((31647, 17), (13564, 17))
```

We can compare the two datasets using the pandas profiling library.

```
#let's compare the train test data as well  
from pandas_profiling import ProfileReport  
  
train_report = ProfileReport(df_train,  
title="Train")
```

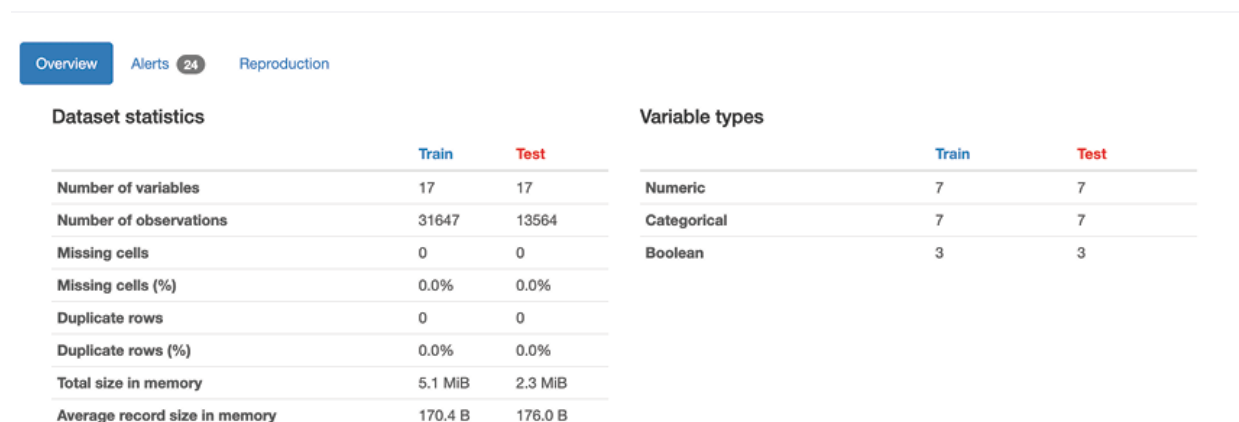
```
test_report = ProfileReport(df_test, title="Test")
comparison_report =
train_report.compare(test_report)

#to export the report

#comparison_report.to_file("comparison.html")
```

Output:

Overview



The screenshot shows a web-based report titled 'Overview'. It has three tabs: 'Overview' (selected), 'Alerts' (with 24 alerts), and 'Reproduction'. The report is divided into two main sections: 'Dataset statistics' and 'Variable types'. Each section has a table comparing 'Train' and 'Test' datasets.

Dataset statistics			Variable types		
	Train	Test		Train	Test
Number of variables	17	17	Numeric	7	7
Number of observations	31647	13564	Categorical	7	7
Missing cells	0	0	Boolean	3	3
Missing cells (%)	0.0%	0.0%			
Duplicate rows	0	0			
Duplicate rows (%)	0.0%	0.0%			
Total size in memory	5.1 MiB	2.3 MiB			
Average record size in memory	170.4 B	176.0 B			

Figure 5.19: The data profiling train-test dataset comparison

Next, we will transform our categorical variables in the train and test datasets separately using the WOE encoder. This allows the training and test datasets to have different distributions and is the right way to prepare the dataset for modeling.

Code:

```
categorical_cols = ["job", "marital",
"education", "default", "housing", "loan",
"contact", "month", "poutcome"]

from category_encoders import woe

encoder = woe.WOEEncoder(cols=categorical_cols)
```

#Always remember to fit on train data, not test data

```
encoder.fit(X_train[categorical_cols],y_train)
```

```
woe_df =
```

```
encoder.transform(X_train[categorical_cols])
```

```
X_tr_transformed =
```

```
pd.concat([woe_df,X_train[numerical_cols]],axis=1)
```

The transformed and splitted train-test dataset dimensions look like :

```
((31647, 17), (13564, 17))
```

With the dataset ready, we will create an **MLflow** experiment tracking instance as follows:

```
#Log in MLflow
```

```
experiment_name = "Ensemble_Classification"
```

```
run_name="Active_Subscriber"
```

```
mlflow.get_tracking_uri()
```

This step is followed by an actual model execution by invoking the MLflow experiment name and the **autolog** function to keep track of the experiments. This looks like:

```
#set experiment name
```

```
mlflow.set_experiment(experiment_name)
```

```
mlflow.sklearn.autolog()
```

```
with mlflow.start_run():
```

```
    LR = LogisticRegression()
```

```
    LR.fit(X_train_sampled, y_train_sampled)
```

```
    predictions = LR.predict(X_tst_transformed)
```

```

#log model features

mlflow.sklearn.log_model(LR,
"logistic_reg_model")

test_Accuracy = accuracy_score(y_test,
predictions)

mlflow.log_metric("test_Accuracy",
test_Accuracy)

print(f"Test Accuracy: {test_Accuracy:.2%}")

print("Model run: ",
mlflow.active_run().info.run_uuid)

mlflow.set_tag("tag1", "Logistic Regression")
mlflow.end_run()

print('Run - %s is logged to Experiment - %s' %
(run_name, experiment_name))

```

Output:

Test Accuracy: 84.49%

Model run: e6edd0758e5d4c69a445e91184d5a503

Run - Active_Subscriber is logged to Experiment - Ensemble_Classification

With a base logistic regression model in place, we will build and evaluate the performance of a Random Forest model. However, we need to find the best RF model that is a specific combination of hyperparameters such as `n_estimator`, `max_depth`, `max_features`, and so on. As discussed earlier, we will find the best hyperparameter combination using the `hyperopt` library. We need to describe the following to use the library:

- The objective function to be minimized:

```
def objective(space):
```

```

        model = RandomForestClassifier( criterion =
space['criterion'],

                                     max_depth =
space['max_depth'],

max_features = space['max_features'],

n_estimators = space['n_estimators']

)

```

- The search space defines the ranges and the functional forms of all the hyperparameters:

```

#Hypertune Random Forest parameter space

from hyperopt import
hp,fmin,tpe,STATUS_OK,Trials

from hyperopt.pyll import scope

space = {'criterion' : hp.choice('criterion',
['entropy','gini']),

        'max_depth':
scope.int(hp.quniform("max_depth", 2, 18, 1)),

        'max_features' :
hp.choice('max_features',
['auto','sqrt','log2',None]),

        'n_estimators':
hp.choice('n_estimators',
[10,50,350,550,750,1200])

}

```

- The database in which to store all the point evaluations of the algorithm search; The following code creates the default `Trials()` database instance, and the minimum of the objective function is obtained using the `tpe.suggest` algorithm.

```
#Instantiate the Trials() database to store the
function outputs and minimize the objective
function using fmin()
```

```
trials = Trials()

best = fmin(fn = objective,
            space=space,
            algo= tpe.suggest,
            max_evals=80,
            trials=trials
            )
```

Best

Output:

```
80/80 [2:26:51<00:00, 110.14s/trial, best loss:
-0.9020937776467118]
```

```
{'criterion': 0, 'max_depth': 18.0,
 'max_features': 1, 'n_estimators': 4}
```

We can observe that the best loss is obtained using the hyperparameter combination shown in the `fmin()` output above. We are going to use these hyperparameters to build an RF model and log the results in the `MLflow` experiment. The code for the same is shown as follows:

```
#Hypertuned RF model
```

```

mlflow.set_experiment(experiment_name)
mlflow.sklearn.autolog()
with mlflow.start_run():
    RF_tuned =
RandomForestClassifier(criterion='entropy',max_dept
h=
18,max_features='sqrt',n_estimators=750,random_stat
e = 222)

    RF_tuned.fit(X_train_sampled, y_train_sampled)
    predictions =
RF_tuned.predict(X_tst_transformed)
    y_pred = np.where(predictions>0.5,1,0)
    mlflow.sklearn.log_model(RF_tuned, "Random
Forest Hypertuned")

    test_Accuracy = accuracy_score(y_test,
predictions)

    mlflow.log_metric("test_Accuracy",
test_Accuracy)

    print(f"Test Accuracy: {test_Accuracy:.2%}")

    f1 = f1_score(y_test, y_pred)

    print(f"Test F1 Score: {f1:.2%}")


mlflow.log_metric(key="f1_experiment_score",value=f
1)

    print("Model run: ",
mlflow.active_run().info.run_uuid)

```



```
mlflow.set_tag("tag1", "Hypertuned Random
Forest")
mlflow.end_run()
print('Run - %s is logged to Experiment - %s' %
(run_name, experiment_name))
```

Output:

Test Accuracy: 90.14%

Test F1 Score: 56.45%

Model run: c1d21ec2b7d14fd3b17e97f1a61a6973

Run - Active_Subscriber is logged to Experiment -
Ensemble_Classification

The RF model has better accuracy (90.14%) on test data
compared to the logistic regression (84.49%) model.

The RF model object also stores the feature importance, of
each variable used in the model, which can be visualized
using the below code in [Figure 5.20](#):

```
importances = RF_tuned.feature_importances_  
feature_names = X_train_sampled.columns  
std = np.std([tree.feature_importances_ for tree in  
RF_tuned.estimators_], axis=0)  
forest_importances = pd.Series(importances,  
index=feature_names)  
fig, ax = plt.subplots()  
forest_importances.plot.bar(yerr=std, ax=ax)  
ax.set_title("Feature importances using MDI")  
ax.set_ylabel("Mean decrease in impurity")
```

```
fig.tight_layout()
```

Output:

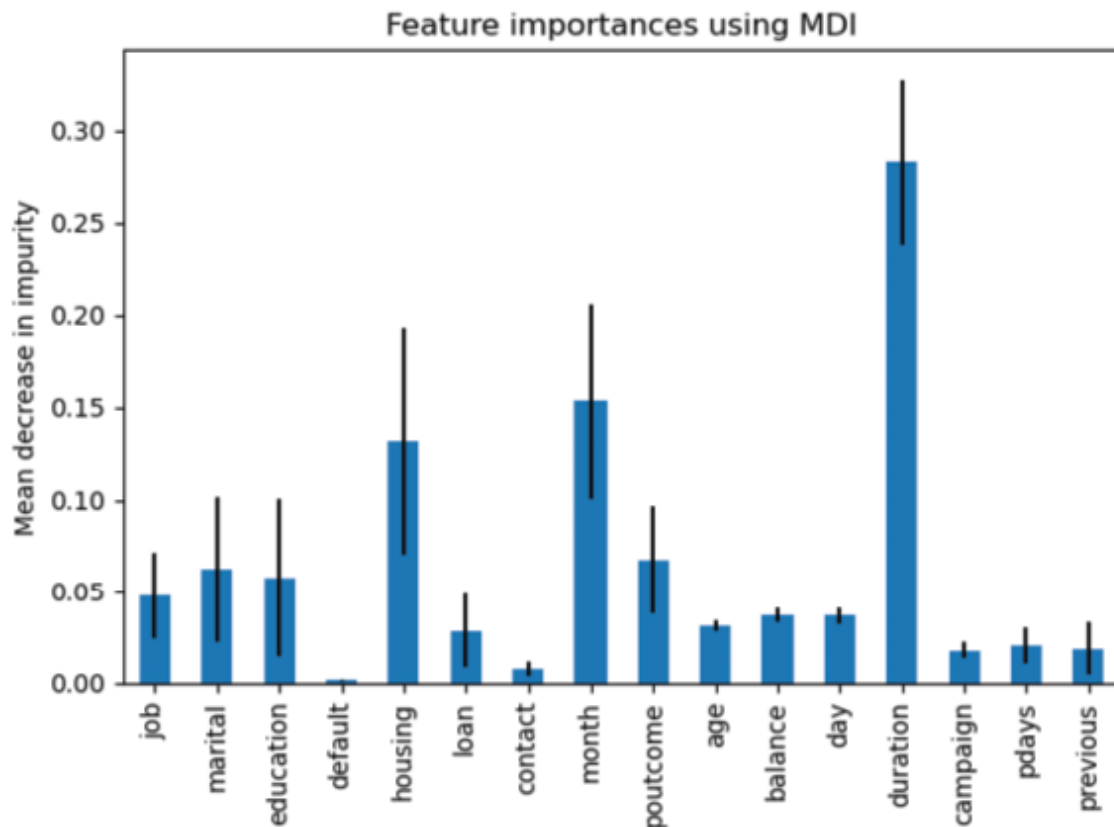


Figure 5.20: Random Forest feature importance

We have displayed the importance of the impurity-based feature, which is the sum of the impurity decrease of all trees when the feature is selected to split a node.

The following code showcases a base decision tree model:

```
#you can use any model as base estimator, default  
is decision tree
```

```
mlflow.set_experiment(experiment_name)
```

```
mlflow.sklearn.autolog()
```

```
with mlflow.start_run():
```

```

    bagtree = BaggingClassifier(base_estimator =
DecisionTreeClassifier() , random_state = 222)

    bagtree.fit(X_train_sampled, y_train_sampled)

    predictions =
bagtree.predict(X_tst_transformed)

    y_pred = np.where(predictions>0.5,1,0)

    mlflow.sklearn.log_model(bagtree, "Bagging")

    test_Accuracy = accuracy_score(y_test,
predictions)

    mlflow.log_metric("test_Accuracy",
test_Accuracy)

    print(f"Test Accuracy: {test_Accuracy:.2%}")

    f1 = f1_score(y_test, y_pred)

    print(f"Test F1 Score: {f1:.2%}")


mlflow.log_metric(key="f1_experiment_score",value=f
1)

    print("Model run: ",
mlflow.active_run().info.run_uuid)

    mlflow.set_tag("tag1", "Bagging")

mlflow.end_run()

print('Run - %s is logged to Experiment - %s' %
(run_name, experiment_name))

```

Output:

Test Accuracy: 89.45%

Test F1 Score: 50.84%

Model run: 1a2f548922e242f0a22c97d1a7448d29

Run - Active_Subscriber is logged to Experiment - Ensemble_Classification

We can again observe that DT model accuracy on the test data is higher than the logistic regression but slightly lower than the RF. We understand that the predictive performance of DT is lower than the ensemble methods, but the ensemble models sacrifice the interpretability of a single decision tree to achieve this performance.

The last model in this case study is the Xgboost model. We will follow the same approach of first identifying the best hyperparameters using the `hyperopt` library, which is then used to train the final model in the `MLflow` experiment setup. The code for the final `xgboost` model (with the best hyperparameters) is as shown:

```
mlflow.set_experiment(experiment_name)

mlflow.xgboost.autolog()

with mlflow.start_run():

    XG_final =
    XGBClassifier(objective='binary:logistic',n_estimators=39,colsample_bytree=0.52, gamma=7.6,
    max_depth=18, min_child_weight= 9.0, reg_alpha=
    66.0, reg_lambda= 0.33,seed = 222)

    XG_final.fit(X_train_sampled, y_train_sampled)

    predictions =
    XG_final.predict(X_tst_transformed)

    y_pred = np.where(predictions>0.5,1,0)

    mlflow.sklearn.log_model(XG_final, "XG Boost
    Hypertuned")
```

```

    test_Accuracy = accuracy_score(y_test,
predictions)

    mlflow.log_metric("test_Accuracy",
test_Accuracy)

    print(f"Test Accuracy: {test_Accuracy:.2%}")

    f1 = f1_score(y_test, y_pred)

    print(f"Test F1 Score: {f1:.2%}")

mlflow.log_metric(key="f1_experiment_score",value=f
1)

    print("Model run: ",
mlflow.active_run().info.run_uuid)

    mlflow.set_tag("tag1", "Hypertuned XG Boost")
mlflow.end_run()

print('Run - %s is logged to Experiment - %s' %
(run_name, experiment_name))

```

Output:

Test Accuracy: 90.25%

Test F1 Score: 56.84%

Model run: 4706fa15fe0e4100a76297b2aab5b31a

Run - Active_Subscriber is logged to Experiment - Ensemble_Classification

Xgboost has slightly higher test accuracy(90.25%) compared to the RF (90.14%)model. The feature importance of the xgboost model is shown in [Figure 5.21](#):

Code:

```
import xgboost

xgboost.plot_importance(XG_final, importance_type='gain', max_num_features=15)

plt.title("xgboost.plot_importance(model)")

plt.show()
```

Output:

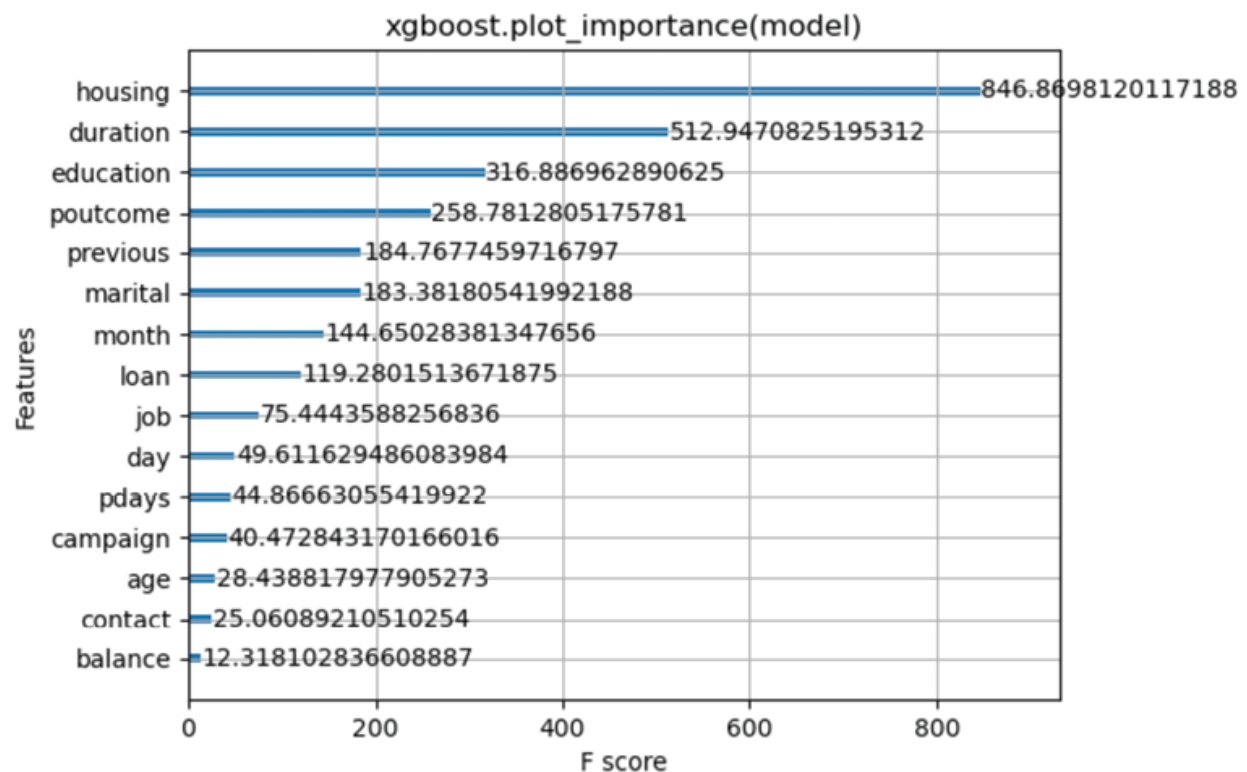


Figure 5.21: The xgboost feature importance

A comparative table of different model experiments using the three techniques is shown as follows. `MLflow` allows us to download the experiments along with their metric in a CSV format. We have reduced the number of columns to compare just the model metrics [Figure 5.22](#):

Start Time	Duration	Run ID	Name	accuracy_score_X_tst_transformed	f1_experiment_score	test_Accuracy	training_accuracy_score	training_f1_score	training_log_loss	training_precision_score	training_recall_score	training_roc_auc	training_score	tag1
11.4s	12.4s	4706fa15fe0e	enchanted-3	0.9024624	0.568352365	0.9024624								Hypertuned XG Boost
11.7s	13.7s	d4e07bb7537	zealous-squ	0.908286641	0.568654646	0.908286641								Base XG Boost
11.8s	18.2s	1a2f548922a7	luxuriant-gru	0.894500147	0.508416352	0.894500147	0.995992127	0.995992098	0.037872347	0.996006802	0.995992127	0.999937368	0.99599213	Bagging
12.4min	2.4min	c1d21ec2b7d	sedate-flea-6	0.901356532	0.564453125	0.901356532	0.991143317	0.991143303	0.067705302	0.991146488	0.991143317	0.999711171	0.99114332	Hypertuned Random Forest
13.3s	33.3s	73eba73540d	salty-colt-813	0.900766735		0.900766735	1	1	0.037277684	1	1	1	1	1 Base Random Forest
12.8s	12.8s	e6edd0758e5	languid-mare	0.844883515		0.844883515	0.845356951	0.84534883	0.376775158	0.845429505	0.845356951	0.918461262	0.84535695	Logistic Regression
14.9s	14.9s	caa04e11704	marvelous-sr	0.844883515		0.844883515	0.845356951	0.84534883	0.376775158	0.845429505	0.845356951	0.918461262	0.84535695	Logistic Regression

Figure 5.22: MLflow Model comparison CSV output

Please note that the detailed model execution steps for the above case study are available in the Jupyter Notebook titled [Chapter 5, Decision Trees with Bagging and Boosting](#).

Case study 3: Interpreting Blackbox models like XGBoost using Shapash

Model interpretation starts with the understanding of the variable importance. This can be calculated in many ways and very intuitively by measuring the incremental improvement in model performance with the addition of a feature in the model. This can be followed by summarizing this incremental improvement for the entire model. We can use this to identify those features that are deemed to have little or no importance and remove them from the model. Variable importance is also an approach for model explainability, whereby we attempt to understand how the model makes predictions at the atomic (observation value) level by first assessing which features are deemed important during model training.

This is the main intuition behind **Shapely Additive Explanations (SHAP)**. In this methodology, the feature importance is estimated by calculating how well the model performs with and without a specific feature for every combination of remaining features. It is important to note that SHAP calculates the local feature importance for every observation which is different from the method used in scikit-learn which computes the global feature importance. The following table summarizes the key differences between SHAP and Scikit-learn approaches:

Dimension	SHAP	Scikit-learn
-----------	------	--------------

Purpose	SHAP values are designed to provide an explanation for the output of a specific model for a specific instance (or prediction).	Scikit-learn provides various methods for calculating global feature importance, such as Gini impurity for decision trees, coefficients for linear models, and permutation importance.
Interpretability	SHAP values are based on cooperative game theory and aim to fairly distribute the contribution of each feature to the model prediction across all features.	Global feature importance measures assess the overall importance of each feature across the entire dataset rather than focusing on a specific instance.
Usage	SHAP are particularly useful for understanding why a specific prediction was made for a particular input. Hence, these are also called post-hoc explainability methods	They are commonly used for model selection, understanding which features are generally more influential in predicting the target variable across the entire dataset.

Table 5.2: Comparison of SHAP vs Scikit-learn

Given the information [Table 5.2](#), SHAP local feature importance becomes relevant in certain cases like loan applications where each data point is a person, and fairness and equity need to be ensured through explainability. You can also get the global measure of feature importance by aggregating the local feature importance for each data point in SHAP.

There are several important variables in Xgboost, and they can be computed in several different ways. Gain is the default type if you construct the model using `scikit-learn`. The `gain` type shows the average gain across all splits where the feature was used.

Alternatively, when you access the Xgboost model object and get the importance with the `get_score` method, the default importance is `weight type`. The `weight` shows the number of times the feature is used to split data. The

importance of this type of feature can be biased towards numerical and high cardinality features. You can check the type of importance with `xgb.importance_type`, which can also be `cover`, `total_gain`, and `total_cover` types of importance.

SHAP is a recent method that aims to resolve this feature bias issue caused by more potential split points by using a game-theoretic approach. SHAP helps to understand and explain how each feature drives the final prediction. In recent years, SHAP has rapidly become popular in explaining black-box models and conducting feature selection, in part due to its perceived robustness and resolution of bias concerns.

Let us understand the two most important visuals that SHAP provides.

Firstly, the Summary or Beeswarm plot shows the first indications of the relationship between the value of a feature and the impact on the prediction. But to see the exact form of the relationship, we then have to look at the SHAP dependence plots. For the Xgboost model described above, the summary plot is shown as follows:

Code:

```
shap.summary_plot(shap_values, X_train_sampled)
```

Output:

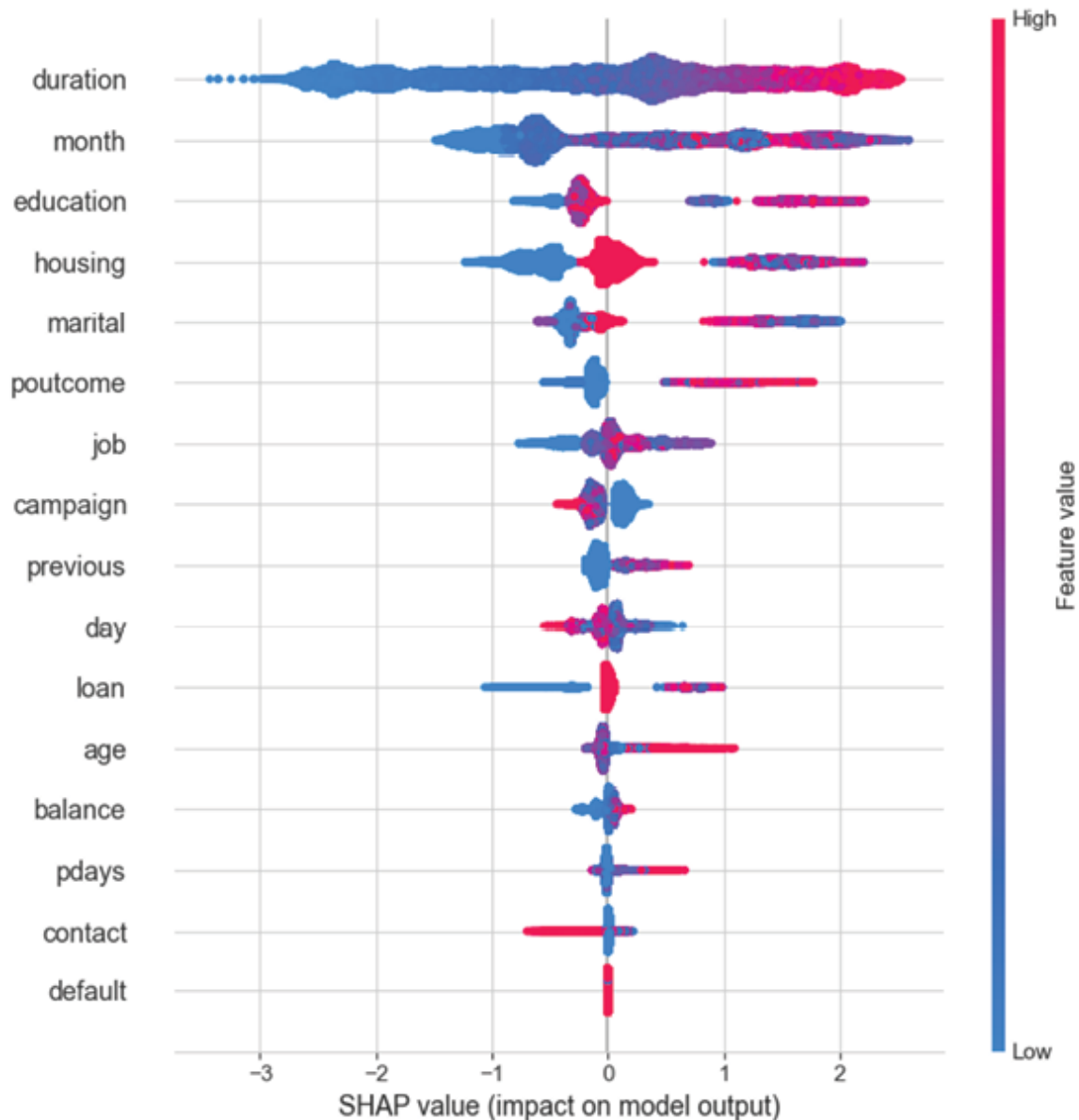


Figure 5.23: SHAP summary plot

The preceding plot is made up of numerous dots (per record) ordered by features from top to bottom. Each dot has the following two characteristics:

- For every row, the color range shows if the feature ranks high or low in value
- The horizontal dispersion from left to right shows a higher or lower prediction impact.

For instance, in [Figure 5.23](#), the feature call *duration* has some really small values (indicated by blue color) towards the left end, which negatively impacts the prediction for those records.

This demands the user to dig deeper into a **Partial Dependence Plot (PDP)** to understand how a single feature impacts prediction. SHAP dependent contribution plots provide a similar insight as PDP's, but they add a lot more detail. The below figure describes the distribution of effects for a single feature.

Code:

```
# create a SHAP dependence plot to show the effect
of a single feature across the whole dataset

sns.set_style("whitegrid")

shap.dependence_plot("duration", shap_values,
X_train_sampled)
```

Output:

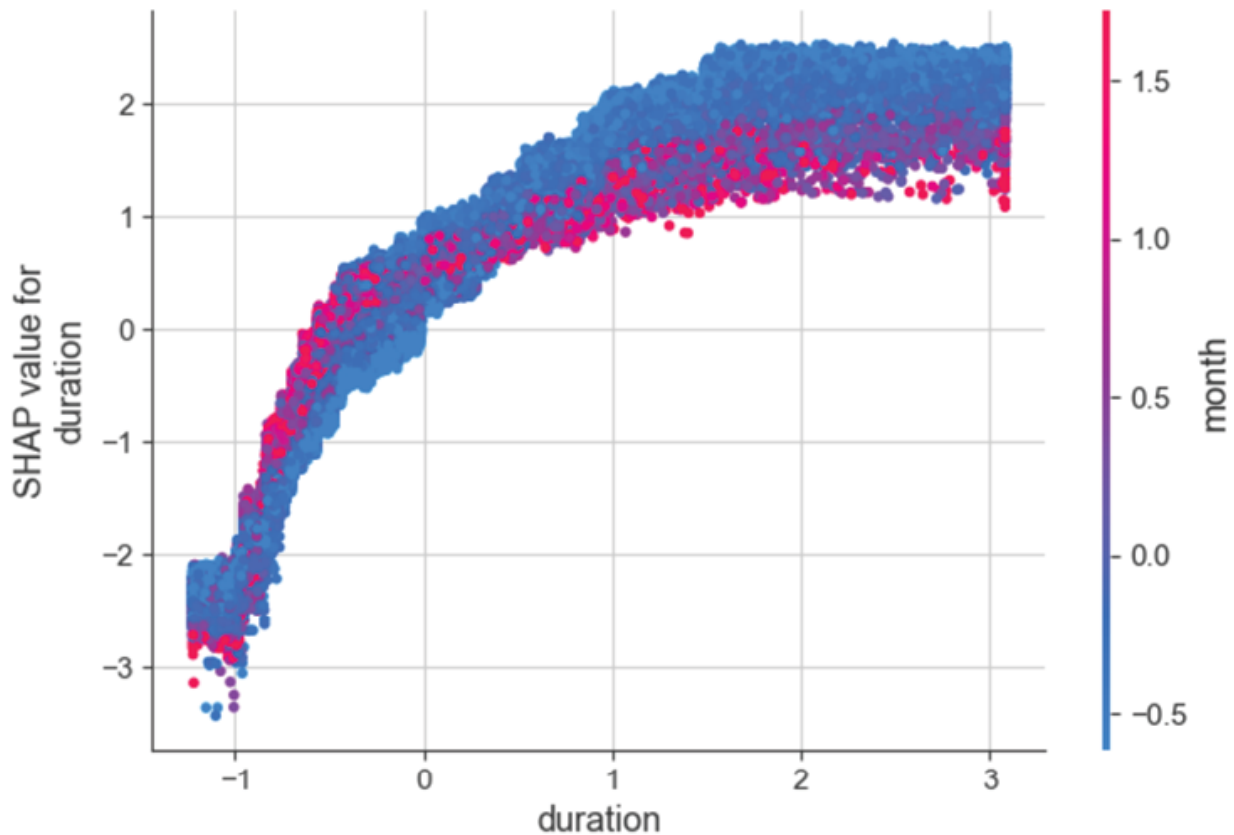


Figure 5.24: SHAP PDP plot

Key points to note with respect to the above plot are:

- Each plotted point on the chart is the prediction from the model.
- The x-axis is the value of the feature *duration* which is transformed.
- The y-axis is the SHAP value of the feature demonstrating the impact of the **duration** value on the output of the model for the particular record.
- The color of the points on the chart corresponds to a second feature (month) that may have an interaction effect with the feature we are plotting which is **duration**. This second feature is chosen automatically by default.

For the example above, the earlier months in the year (blue dots) with higher call durations are more likely for clients to

subscribe to a term deposit than calls made in the later part of the year. This suggests an interaction effect between the month of the year and call durations meaning customers are more likely to buy term deposits in earlier months of the year if bank agents talk longer with them.

SHAP has a fast implementation for tree-based models, especially in the case of using an xgboost model, where SHAP has some optimizations. This fast computation makes it possible to compute the many Shapley values needed for the global model interpretations. The global interpretation methods include feature importance, feature dependence, interactions, clustering, and summary plots. With SHAP, global interpretations are consistent with the local explanations, since the Shapley values are the *atomic unit* of the global interpretations.

The authors implemented SHAP in the [shap](#) Python package. This implementation works for tree-based models in the [scikit-learn](#) machine learning library for Python. The shap package was also used for the examples in this chapter.

Ensembling using the stacking method

In the predictive modeling realm, the quest for improved accuracy and robustness is an ongoing pursuit. To overcome the limitations of individual models, data scientists have turned to ensemble modeling, which combines the predictions of multiple models to achieve superior performance. The key idea is that by aggregating the predictions of diverse models, that capture different aspects of the underlying patterns in the data, the weaknesses of individual models can be mitigated, leading to improved overall performance. In this section, we will delve into one such powerful ensemble technique *stacking*. By understanding the principles and benefits of stacking, you will be equipped with a valuable tool to enhance your predictive modeling endeavors.

At its core, the concept of stacking involves blending the predictions of several diverse base models into a meta-model, which learns to make the final predictions based on the collective knowledge of the individual models. This meta-model can often outperform any single base model, as it leverages the strengths and compensates for the weaknesses of each constituent model.

The stacking process typically involves three key steps:

1. **Base model training:** A set of heterogeneous base models is trained on the available training data. These models can be of various types, such as decision trees, support vector machines, random forests, or neural networks. The diversity in the base models is essential to capture different aspects of the data and make complementary predictions.
2. **Base model predictions:** The trained base models are then used to generate predictions for the validation or test data. These predictions serve as the input features for the next step.
3. **Meta-model training:** A meta-model is trained using the predictions from the base models as input features, along with the corresponding ground truth labels. This meta-model learns to combine the predictions from the base models, often using techniques like logistic regression, gradient boosting, or neural networks. The objective is to find the optimal way to weigh and blend the predictions to make the final predictions.

There are four key benefits of stacking described as follows:

- **Improved predictive performance:** Stacking allows us to tap into the collective intelligence of multiple models, resulting in enhanced predictive accuracy and robustness. By combining the strengths of various modeling techniques, stacking can mitigate the

limitations of individual models and provide more reliable predictions.

- **Handling complex relationships:** Stacking excels at capturing complex relationships in the data. Since it combines predictions from different models, each model can capture a unique aspect of the underlying patterns and dependencies, leading to a more comprehensive understanding of the data.
- **Flexible architecture:** Stacking offers flexibility in model selection and customization. Practitioners can experiment with different combinations of base models and meta-models to find the optimal configuration for a given problem domain. This adaptability makes stacking suitable for a wide range of predictive modeling tasks.
- **Adaptability to new data:** Stacking can readily adapt to new data by retraining the base models and updating the meta-model. This allows for continuous learning and adaptation, ensuring that the ensemble remains effective as the data evolves. Please note that stacking is particularly useful with base models when these individual base results are very different.

As part of the case study series, let us use the three models (Logistic regression, random forest, and Xgboost) built so far for building a new stacked model.

Case study 4: An ensemble stacked model using previously built logistic, RF, and XGBoost models

For this purpose, we are going to leverage the models built in *case study 2* and stack them using an ensemble-learning meta-classifier. The meta-classifier can either be trained on the predicted class labels or probabilities from the ensemble.

The following code leverages the `mlxtend` Python library to import `StackingClassifier`:

Python code:

```
#stacking models

from mlxtend.classifier import StackingClassifier

rf = RandomForestClassifier()

sclf = StackingClassifier(classifiers=[LR, RF,
XG_final],use_probabilities=True,

                                average_probabilities=False,
meta_classifier = rf)

for clf, label in zip([LR, RF, XG_final, sclf],
['Logistic Regression', 'Random Forest', 'XG
Boost', 'StackingClassifier']):

    scores = model_selection.cross_val_score(clf,
X_train_sampled, y_train_sampled, cv=10)

    print("Accuracy: %0.2f (+/- %0.2f) [%s]" %
(scores.mean(), scores.std(), label))
```

Output:

Accuracy: 0.85 (+/- 0.01) [Logistic Regression]

Accuracy: 0.94 (+/- 0.09) [Random Forest]

Accuracy: 0.92 (+/- 0.11) [XG Boost]

Accuracy: 0.94 (+/- 0.08) [StackingClassifier]

The preceding code demonstrates the accuracy of each of the classifiers along with the stacked classifier.

It is important to note that with the stacking classifier, it is important to make sure that the base estimators are well-fitted to give accurate predictions. The goal is to avoid ensembling a set of bad predictors, which might make things worse.

In the above example, we see that the Stacking classifier is as good as the Random Forest, suggesting that the XG boost and Logistics Regression did not predict any cases that are substantially different from the Random Forest prediction leading to a complete overlap of their prediction results. This also means that Stacking exercise is not always needed or beneficial.

Conclusion

In this chapter, we discussed decision trees, where we have witnessed the power of these versatile models in predictive analysis. Decision trees provide a straightforward and intuitive approach to solving complex problems by breaking them down into simpler, more manageable parts. We explored the fundamental principles behind decision tree construction, including top-down, greedy search, and binary splits with Gini and Information gain concepts.

While decision trees offer a solid foundation for predictive modeling, we recognize that there is always room for improvement. Therefore, we turned our attention to more advanced ensemble techniques, namely RF and XGBoost, which elevate the performance of decision trees to new heights.

Random Forest, as an ensemble of decision trees, leverages the collective wisdom of multiple trees to achieve enhanced accuracy and robustness. XGBoost, on the other hand, introduces a gradient-boosting framework that iteratively improves upon the weaknesses of weak learners, in this case, decision trees.

As we continue to explore these ensemble techniques, we also discuss model explainability. Understanding the importance of features in the decision-making process is vital for gaining insights and building trust in the model. Moreover, we have explored the concept of SHapley Additive

exPlanations, which quantifies the impact of each feature on the model's output, thereby enhancing the transparency and interpretability of the model.

As we moved forward in our exploration, we dived into the world of stacking, an ensemble technique that combines the strengths of diverse modeling techniques. Through stacking, we further enhanced our predictive modeling capabilities, embracing both accuracy and interpretability to unravel complex data domains and unlock valuable insights.

In conclusion, decision trees serve as the foundation of our ensemble modeling journey. By harnessing the power of Random Forest, XGBoost, and Stacking we can unlock the true potential of ensemble techniques and achieve superior predictive performance. Moreover, the assessment of feature importance and the application of SHAP values contribute to the overall model explainability, enabling us to gain insights into the inner workings of these complex modeling algorithms. Remember, the goal will be to balance model complexity with model explainability as we move on to the next set of chapters.

The next chapter will focus on a set of innovative supervised classification algorithms known as **support vector machines (SVM)** and **K-nearest neighbors (k-NN)**. While SVM focuses on finding a hyperplane that separates classes, k-NN relies on the distance-based similarity of data points in the feature space. The next chapter six, Support Vector Machines and k-NN, will wrap up our discussion on supervised machine learning algorithms.

Points to remember

- Decision trees are prone to overfitting, so it's important to apply techniques like pruning, setting a maximum depth, or using regularization parameters to control the complexity of the tree.

- Decision trees can handle both numerical and categorical features.
- It is beneficial to visualize the decision tree to understand the splits and interpret the importance of the features.
- Random forest is an ensemble method that combines multiple decision trees to make predictions.
- Random forest reduces the risk of overfitting by averaging the predictions from multiple trees.
- It's important to tune parameters like the number of trees, maximum depth, and the number of features considered at each split.
- Random forests can handle large datasets and high-dimensional feature spaces effectively.
- XGBoost (Extreme Gradient Boosting) is a boosting algorithm known for its performance and scalability.
- It uses a gradient-boosting framework and sequentially builds a sequence of decision trees to correct the errors made by the previous trees.
- Parameter tuning is critical in XGBoost, including the learning rate, number of boosting rounds, maximum depth, and regularization parameters.
- Feature engineering plays a significant role in XGBoost. Creating relevant and informative features can improve the model's performance.
- Stacking involves training multiple models and combining their predictions using a meta-model.
- It's essential to select diverse base models that capture different aspects of the data.
- Careful cross-validation is necessary to prevent overfitting during model stacking.

- Feature engineering and model selection for base models are important to ensure optimal performance.
- The choice of the meta-model, such as linear regression, logistic regression, or another model, should be based on the problem at hand.
- Regularly monitor and analyze the importance of features and model performance to identify areas for improvement and potential issues.
- Interpretation and business insights: Translating statistical output into actionable recommendations is more important than building a highly accurate model.

Join our book's Discord space

Join the book's Discord Workspace for Latest updates, Offers, Tech happenings around the world, New Release and Sessions with the Authors:

[**https://discord.bpbonline.com**](https://discord.bpbonline.com)



CHAPTER 6

Support Vector Machines and K-Nearest Neighbors

Introduction

Sometimes, the most powerful solutions are found by embracing the support of those around us.

In the arena of data classification and predictions, we have so far looked at regression and tree-based approaches. However, as the data complexity grows, we need more innovation in the algorithms to address non-linear relationships. Trees do this by splitting data points, but in this chapter, we will unlock the concepts of support proximity in the realm of machine learning to understand complex relationships. **Support Vector Machines (SVM)** and **k-nearest Neighbors (KNN)** are couple of such unique and robust methodologies, that illuminate patterns and unravel the mysteries of data. So get ready to harness the power of decision boundaries and the similarity of neighbors, as we journey through the captivating landscape of SVM and KNN, where lines or hyper-planes divide, but neighbors come together, shaping the world of intelligent predictions. This knowledge equips you with diverse skills to tackle even more demanding challenges in machine learning!

Structure

In this chapter, we will learn the following topics:

- Classification algorithms with a twist
 - Background
 - Under the hood of SVM and KNN
 - Data
 - Model
 - Loss function: Achieving optimal algorithmic results
- Challenges and assumptions
- Case study: Predicting customer propensity to subscribe to a term deposit

Objectives

The objective of this chapter is to provide a comprehensive understanding of two essential machine learning techniques: SVM and KNN. We will delve into the inner workings of these models, exploring how they function and how they can be applied to solve real-world problems. Specifically, we will explore how SVM finds optimal decision boundaries, maximizing the separation between classes, and how KNN relies on the proximity of neighbors for predictions. By understanding the conceptual foundations, we will understand how these models make intelligent decisions. We will present a detailed case study that highlights the effectiveness of SVM and KNN in solving a specific problem. Through this case study, we will witness these models in action, from data preprocessing to model evaluation, and gain a clear understanding of their capabilities and limitations.

Classification algorithms with a twist

In supervised learning, the classification of objects is not always as easy as sorting apples and oranges. For instance, nowadays it is common for banks to use classification to identify fraudulent customer activities based on numerous fraud indicator variables. Since fraudsters are getting smarter, algorithms that banks use have to evolve and identify challenging patterns quickly as well. We now know that regression helps us find class probabilities and tree-based methods are great in defining rules and separating the classes out. But why can classification not simply mean cutting across a dataset such that we find the best plane to separate different groups of data? It could be like having a magic marker that draws a line right where it needs to be, creating a clear boundary between things like apples and oranges in a basket.

There can potentially be another approach where the algorithm examines the data points around each other, making predictions based on what their nearby neighbors are doing. It is like asking the people next door for advice and going with the most popular advice.

In my opinion, these are very valid approaches and have been widely practiced in the industry for decades. SVM is the super separator, and KNN is like asking your best buddies for help in making decisions. Each method has its own special way of solving different types of classification problems, and they are like different tools in your problem-solving toolbox!

Background

It is time to discuss SVM and KNN in slightly more detail. As we understand, the primary objective of the algorithm is to identify a hyperplane in an n -dimensional space, which translates to identifying a line in a 2-D space to differentiate between two classes. We will try to understand the algorithm

using a 3-D hyperplane example to help readers get the best of complexity and explainability.

Additionally, we now know that the KNN algorithm's objective is to determine the class of an unclassified point by counting the majority class vote from its k -nearest neighbor training points. For example, an activity whose attributes closely match those of the most common fraudulent activity in the vicinity would be classified as a fraudulent activity. k is obviously an important parameter to look out for.

A key point to note with respect to both algorithms is that neither SVM nor KNN makes explicit model specifications about the data distribution, such as the normality of the data.

Additionally, KNN is a non-parametric algorithm because it avoids a priori assumptions about the shape of the class boundary and can thus adapt more closely to nonlinear boundaries as the amount of training data increases. On the other hand, SVM are basic support vector machines and are linear classifiers that are parametric. However, RBF kernel SVMs are considered non-parametric learning algorithms since the number of parameters grows with the size of the training set and are not constrained by a set number of parameters. We will learn more about each of these techniques in the next section.

Under the hood

In this section, we will further investigate data needs, algorithm operations, and the quest for the best algorithmic outcome. We know these three by the data-model-loss trinity.

Data

The Portuguese banking institution dataset contains 45,211 records with 17 attributes. In this dataset, each record

represents a person who has subscribed to a term deposit from the bank. Each person is classified as having *yes* or *no* as per the set of corresponding customer attributes. This cleaner dataset is sourced from the UC Irvine repository for learning purposes and is available in a readable CSV file. The selected attributes are shown in [Figure 6.1](#):

	age	job	marital	education	default	balance	housing	loan	contact	day	month	duration	campaign	pdays	previous	outcome	Target
18249	57	blue-collar	divorced	secondary	no	644	no	yes	cellular	31	jul	263	4	-1	0	unknown	no
28096	32	admin.	single	secondary	no	197	no	no	cellular	28	jan	205	3	-1	0	unknown	no
4223	46	blue-collar	married	secondary	no	679	yes	no	unknown	19	may	723	1	-1	0	unknown	no
14882	29	blue-collar	married	secondary	no	2870	yes	no	cellular	16	jul	988	1	-1	0	unknown	yes
3355	55	blue-collar	married	primary	no	158	yes	no	unknown	15	may	176	18	-1	0	unknown	no

Figure 6.1: Portuguese banking dataset snapshot

Please note that, based on the correlation analysis performed in [Figure 6.9](#) (case study section), the variable **duration** (duration of the last call) is highly correlated with the **Target** (dependent variable). For instance, if **duration=0** then **Target=0**, because there was no last call and hence the customer has no term deposit. We will avoid using this variable to study the effect of other variables better.

The link to the original dataset can be found as follows:

<https://archive.ics.uci.edu/dataset/222/bank+marketing>

While there are no strict limits on data size for KNN and SVM, the practical considerations include the number of data points, memory availability, and the nature of the data (dimensionality and sparsity). SVM only needs a small subset of training points (the support vectors) to define the classification rule, making it often more memory efficient and less computationally demanding when inferring the class of a new observation. In contrast, kNN typically requires higher computation and memory resources because it needs to use all input variables and training samples for each new observation to be classified.

Standardization of all features to bring them on the same scale is a best practice in most modeling exercises. [Table 6.1](#) compares SVM and KNN data requirements:

Dimension	SVM	KNN
Data size	Efficient working with low to moderate-size datasets.	Slow prediction on a large dataset
Memory usage	Linear SVMs have less memory needs compared to non-linear ones	High, since it has to store the entire dataset in memory for comparison
Dimensionality	Works well with sparse and high-dimensional datasets	Struggles in high dimensional space since a lot of neighbors are at a similar distance

Table 6.1: SVM vs KNN data needs

Model

In this section, we will discuss the algorithm execution in a stepwise manner. Let us start with SVM.

We know the goal of SVM is to find an optimal hyperplane, in a n-dimensional space that can separate the classes with the largest possible gap between them. It is like drawing a line on paper to keep two groups apart or drawing a plane in n-dimensions, as shown in [Figure 6.2](#):

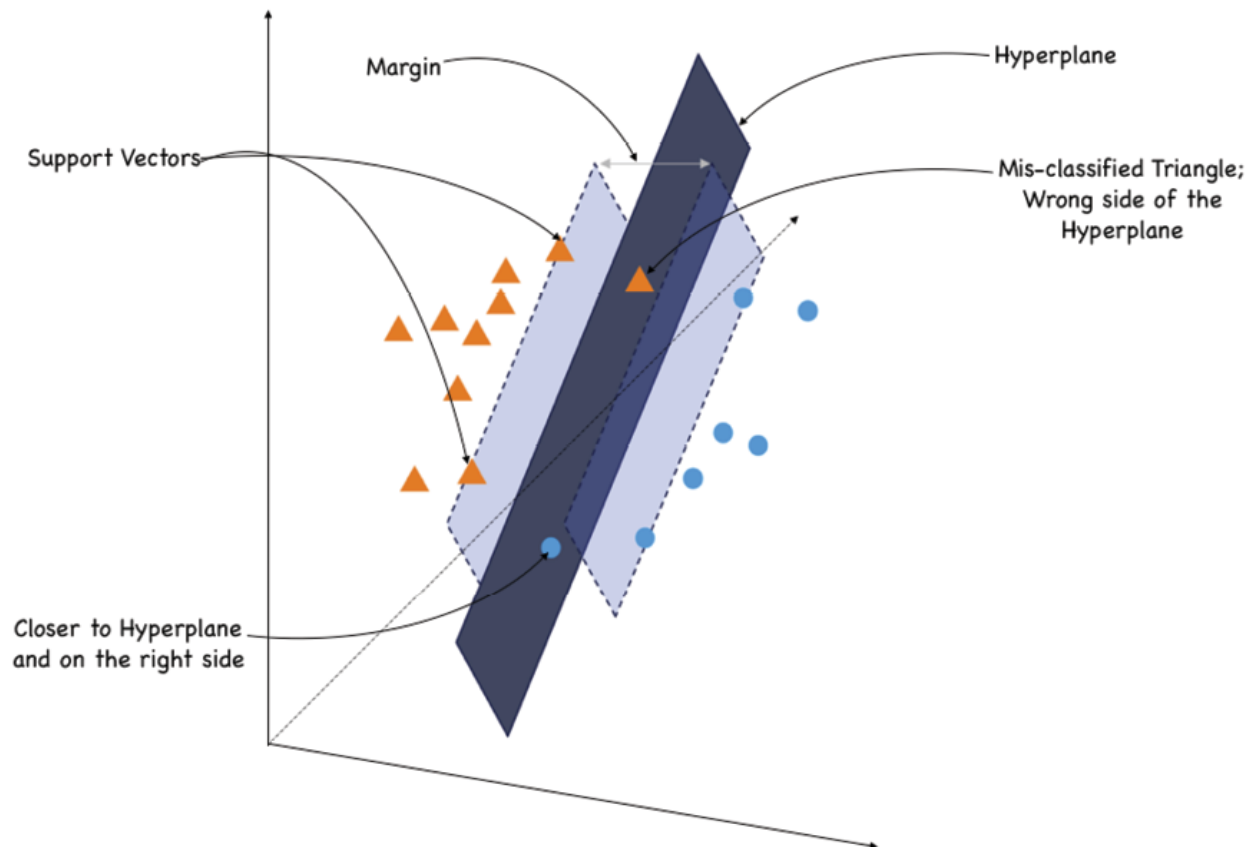


Figure 6.2 (a): SVM Hyperplane in a 3-D space

As seen in the above figure, the dark blue separating hyperplane is a decision boundary that separates the data points of one class from another. For a two-dimensional space, the decision boundary is a straight line, while in a three-dimensional space, it becomes a plane. The data points closest to the separating line are called *support vectors*. They are crucial because they define the position of the plane and the gap between the groups.

Another important term to note in this algorithm is *margin*. The algorithm maximizes this margin between the support vectors and the separating line or hyperplane. This should be as wide as possible, so there's more room or margin for new data to be accurately classified. But, since most data sets in the real world are not linearly separable, and no matter how narrow the margin is, any separating line will result in

misclassifications. We say that the margin is violated by a sample if it is on the wrong side of the separating hyperplane ([Figure 6.2\(a\)](#), amber triangle) or is on the correct side, but is within the margin just like the blue circle.

In this context, the margin can be of two types:

- A hard margin SVM, where the goal is to find a hyperplane that perfectly separates the two classes without allowing any data points to be misclassified. This implies that there should be no data points between the margin boundaries.
- Soft margin SVM introduces flexibility by allowing for some misclassification. This is particularly useful when the data is not perfectly separable or when there are outliers that might disrupt the hard margin approach.

Please note that the trade-off here is allowing a few data points to be misclassified as long as the margin is wide enough for the major chunk of the data. In SVM, C is a hyperparameter to tune the overall fitting behavior of an algorithm that balances the trade-off between margin width and misclassification. The margin and C have an inverse relationship. Please see below [Figure 6.2 \(b\)](#) for a relationship between C and margin:

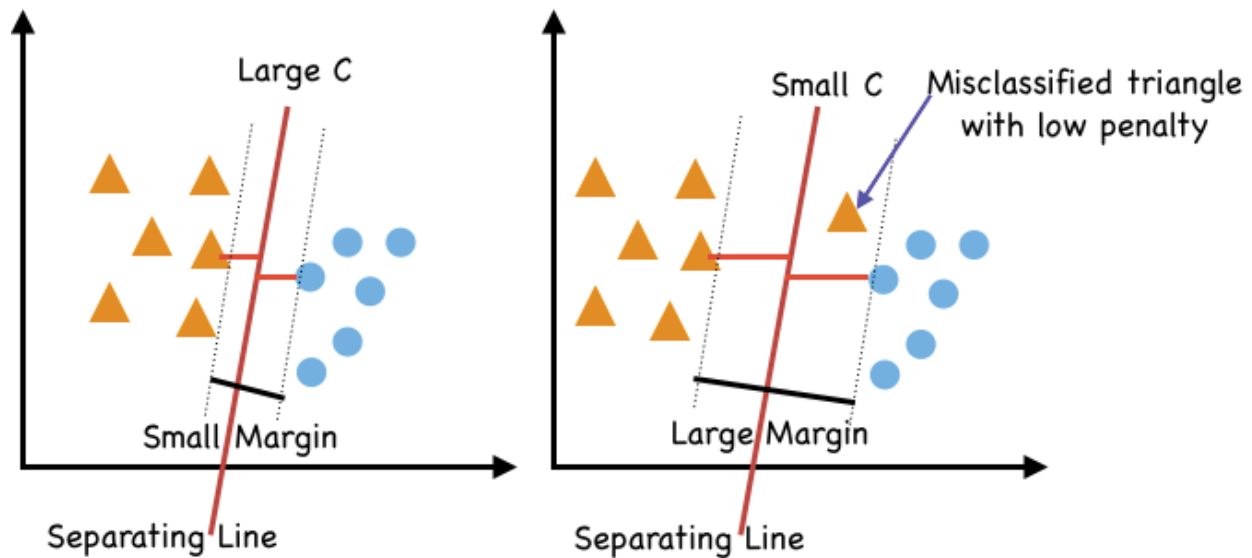


Figure 6.2 (b): C and Margin relationship in a 2-D space

In the above [Figure 6.2\(b\)](#), when C is large, the margin width is the least and the separating line is placed in such a way as to minimize the sum of the misclassification or violation penalties. When C is smaller, the misclassified points have lower impact, and the line is placed with a focus to maximize the margin further. This often results in a better-generalized model at the cost of some misclassification. When C is very small, classification penalties become insignificant, and the margin grows to encompass all points. Does this mean large values of C translate into underfitting the data and small values with large margins result in an overfitting model? This is true and hence choosing C becomes a cross-validation exercise. We will use the Hyperopt library for the parameter tuning in the case study section to identify an optimal C value.

Once the best-separating hyperplane is found, SVM can use it to predict the category of a new observation based on features. For instance, it checks which side of the line or the hyperplane the new customers fall on, and that's the predicted category.

Lastly, if the data is not easily separated by a straight line, SVM can use a trick called *kernel functions* to transform the data into a higher-dimensional space where it becomes separable. For example, if the inputs are two-dimensional, the kernel function will map them into a 3-D space. In this space, the data will be linearly separable. This way, it can handle more complex patterns. There are two such powerful kernels used in SVM, which are called *Polynomial* and **Radial Basis Function (RBF) Kernel**.

Refer to [Figure 6.3](#) for individual kernel equations and key parameters:

Polynomial Kernel	Radial Basis Function Kernel
$K(x_1, x_2) = (x_1^T x_2 + c)^d$	$K(x_1, x_2) = e^{(-\gamma \ x_1 - x_2\ ^2)}$
Dot product of feature vectors represented as row or column matrices	Large 'σ' means smaller 'γ' and translates to larger width of region of similarity. This means that points that are farther away can be considered to be similar when 'γ' is small.
Matrix multiplication is done on transposed row matrix of the first vector and column matrix of the other	A smaller 'γ' value results in a smoother decision boundary that may be more generalizable to new data and larger value leads to tight-knit new datapoints.
The parameter 'c' can be used to control the trade-off between the fit of the training data and the size of the margin. A large c value will give a low training error but may result in overfitting.	The RBF kernel computes the similarity (Euclidean distance) of two datapoints and multiplies the 'γ', before taking the exponential.
A high degree 'd' may overfit the data, while a low degree d may under fit the data.	RBF Kernel is popular because of its similarity to K-NN Algorithm.
The result of this transformation is a set of new features such as x_1^2, x_2^2 and $x_1 * x_2$ that capture the non-linear relationships between the input data.	The RBF Kernel SVM is implemented in the Scikit-learn library and has two hyperparameters associated with it, 'C' for SVM regularization and 'γ' for the RBF Kernel 'width of similarity'.

Figure 6.3: SVM kernel functions: Poly and RBF

As seen in [Figure 6.3](#), a polynomial kernel or function is defined as the dot product of the data point vectors (x_1 and x_2) in the original space added to a constant C and overall raised to a power of a degree parameter to map the data.

There are a number of parameters that can be tuned to improve *polynomial* performance, including its degree and coefficient.

On the other hand, the RBF kernel finds the dot products and squares of all the features in the dataset and then performs the classification using the basic idea of Linear SVM. The RBF kernel uses the radial basis function shown and described in [Figure 6.3](#) for data projection into higher space. In the context of RBF, the choice of the gamma parameter is crucial for achieving the right balance between underfitting and overfitting. Just like the c parameter, tuning the gamma parameter involves using techniques like grid search or cross-validation to find the optimal value that maximizes the model's performance on a validation set.

Please note that while the RBF kernel's mathematical form involves the Euclidean distance, the kernel function only implicitly maps the data into a higher-dimensional space where the separation using a hyperplane is achieved. This indirect mapping, using the kernel function calculated distances, allows the SVM to calculate distances in the transformed space, without explicitly performing the transformation. Thereafter, the optimization problem finds the Lagrange multipliers (weights) associated with the support vectors to define the decision boundary in the high-dimensional space. These determined optimal weights are then used to make predictions for new, unseen data points.

The essence of the SVM RBF kernel's transformation lies in the fact that it cleverly leverages the kernel function to implicitly project data into a space where linear separation is achieved. This allows the SVM to handle non-linear relationships between features without explicitly performing the high-dimensional transformation, making the process computationally efficient.

Overall, SVMs are classifiers in the real sense as in they try to put a wedge between the two classes and behave like Linear regression OLS to some extent in linear SVM cases. Just like OLS tries to find the line of best fit by minimizing the distance from the data points, SVM tries to find a line or plane of best separation by maximizing the distance. The end objectives are different, but you get the picture in your head.

With the SVM knowledge, let us move on to explore the KNN's inner workings.

K-Nearest Neighbor: It is a common saying that you are the company you keep. KNN follows the same principle and relies on the fact that similar data points share the same labels. It is a very popular and easy-to-understand, supervised, and nonparametric ML algorithm. It is also known as a lazy learner because it does not learn and builds a discriminative function during training. It loads the entire training data when it needs to make some predictions.

KNN can be used for both classification and regression tasks but is primarily used for classification problems. While it is not the best technique for large datasets due to its lazy approach, it is highly interpretable and easy to use. [Figure 6.4](#) describes the two-step classification process:



Figure 6.4: The two-step KNN process

It is crucial to realize the impact of K in the training process. To select the value of K that fits our data, we run the KNN algorithm multiple times with different K values. We'll use accuracy as the metric for evaluating model performance for different values of K . If the value of accuracy changes proportionally to the change in K , then it is a good candidate for the K value. It implies that as K changes, the accuracy should show a consistent and reasonable trend without sharp, unexpected drops, spikes, or performance plateaus. This can be achieved by visualizing the decision boundaries or accuracy curves for different K . Alternatively, we can use the cross-validation method to find out the optimal value of K . Starting with the minimum value of K that is $K=1$, run cross-validation and measure the accuracy until the results are consistent. We will explore this approach in the case study. Please note that K is dependent on the number of features and sample size per group.

We notice it is a simple algorithm but there is a downside to this simplicity wherein it is hard to make predictions for rare events (fraud) because it has no clue about rare events in an otherwise regular customer activity environment. In other words, everything looks the same. Talking about similarity, it is based on the distance between two data points. Several distance metrics determine correlation and similarity in KNN. Even though there are plenty of distance functions to choose from, we should carefully evaluate the function that best fits the nature of our data, some notable metrics are as follows:

Distance metric	Description
Euclidean	Used for quantitative data and calculates the straight-line distance between two points. It works well when the features have similar scales.
Manhattan	It is the distance between two points measured along axes at right angles and suitable for datasets with features on different scales.

Distance metric	Description
Minkowski	Used in real-valued vector spaces and is a generalized distance metric that includes both Euclidean and Manhattan distances as special cases.
Jaccard index	Also used with categorical data and often used in applications when dealing with binarized data, it calculates the size of the intersection of sets divided by the size of their union.
Hamming	Typically used with data transmitted over computer networks. And also used with categorical variables where it counts the number of positions at which two strings of equal length differ.

Table 6.2: KNN distance metric

To summarize, it is crucial to experiment, evaluate, and possibly perform hyperparameter tuning to find the optimal configuration for a specific problem scenario to address the bias-variance trade-off issue.

The choice of K and the distance metric in KNN helps balance the bias-variance play but they depend on the characteristics of the data, the business problem at hand, and the desired balance between bias and variance. For instance, small values of K, implying less number of similar neighbors, capture local patterns and distinguish between clusters well. Also, with lower K, the model tends to have a lower bias because it can capture intricate and fine-grained patterns in the training data better. But this can lead to overfitting. The reverse is true for higher values of K.

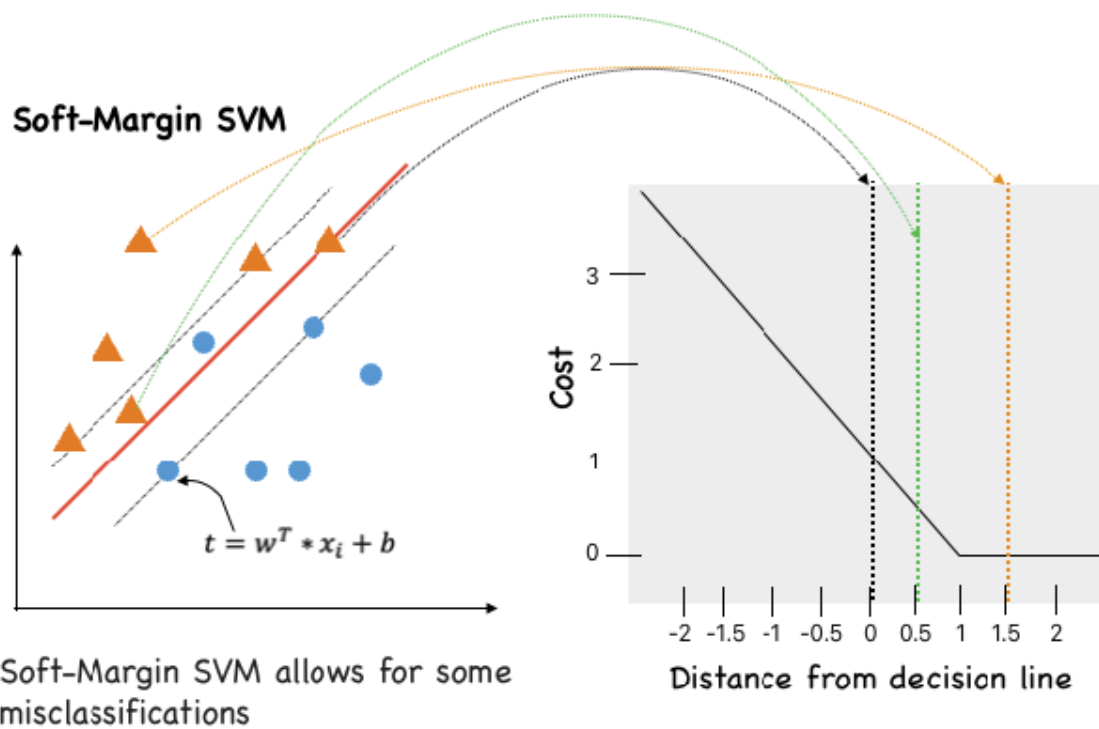
Secondly, if data has outliers, use distance metrics that are relatively less sensitive to outliers (for example, Manhattan) if you do not want to treat the outliers. Let us switch gears to understand the loss function related to the two techniques.

Loss function: Achieving optimal algorithmic results

Support vector machines, using a linear function, output a real-valued number, that can be thought of as the *distance* or *margin* from the data point to the classification boundary

or hyperplane. The sign of this number determines the predicted class. Positive values indicate one class and negative values indicate the other class. Additionally, it incurs a penalty on the data point if it is classified incorrectly, that is it falls on the wrong side of the hyperplane, or if the distance from the decision boundary line is not large enough. To measure this, SVM employs hinge loss as a specific type of cost function that incorporates a margin or distance from the hyperplane into the cost calculation. The hinge loss increases linearly and is defined according to the following formula:

For an intended output of $t = \pm 1$ and a classifier score y , the hinge loss of the prediction y is defined as $L(y) = \max(0, 1 - t \cdot y)$



We incur a loss when the distance from the boundary is less than 1. At 0 distance (the data point is on the boundary), the loss is 1.

Observations that end up on the wrong side of the hyperplane, such as ●, incur a loss > 1 , that increases further linearly.

Figure 6.5: Hinge loss formula and graph

It can be noted in [Figure 6.5](#), that the further an observation (amber triangle) lies from the line, the more confident SVM is in the classification.

For example, if for an observation (amber triangle) the SVM produces an output of 1.5, and the data point is beyond the margin, the loss is equal to 0. As is evident, the hinge loss is responsible for penalizing misclassified points. In addition to the hinge loss in the cost function, another regularization term is represented as the squared Euclidean distance of the weight vector w . Since w determines the orientation of the hyperplane, this regularization term helps control the complexity of the model and prevents overfitting by ensuring the correct classification of data points within their respective margins. [Figure 6.6](#) shows the cost function:

Optimize this first term: Vector orthogonal to the data separating hyperplane

Minimize Hinge Loss term

Distance from hyperplane

$$\min \frac{1}{2} \sum_{i=1}^n w_i^2 + C \sum_{j=1}^m \max(0, 1 - t_j \cdot y_j) \quad \text{Where } t = w^T * x_i + b$$

x_i is the feature vector of the i -th data point.

" b " is the bias term (also called the intercept) that shifts the hyperplane or line.

y_j is the true label of the i -th data point.

The minimum distance between an observation x_i and the hyperplane can be measured along a line that is orthogonal to the plane and goes through projection of x_i on the orthogonal line w .

The vector projection of x_i on w can be taken by taking the dot product between the w^T and x_i divided by the norm of w .

C is a regularization parameter that balances between achieving a larger margin and minimizing the classification error. Smaller C indicates stronger regularization, meaning the model will attempt to maximize the margin and be more tolerant towards misclassifications.

Figure 6.6: SVM cost function

The goal of training an SVM is to find the weight vector " w " and bias term " b " that minimize this cost function while satisfying the constraints. This optimization process aims to

find the optimal hyperplane that achieves the best trade-off between margin maximization and misclassification error.

On the other hand, KNN is a fairly simple classification algorithm that only needs to know the number of categories and an optimal value of K. As you can imagine, since its performance can be affected by the choice of K and the distance metric, careful parameter tuning is necessary for optimal results.

Challenges and assumptions

Let us summarize some key points we have discussed so far with regard to SVM and KNN algorithms.

The following are the top challenges of working with SVM:

- **Interpretability:** SVM's decision boundary can be complex, especially in higher dimensional problems, making it challenging to interpret why specific predictions are made.
- **Kernel selection:** Appropriate kernel function (linear, polynomial, radial basis function, etc.) selection is crucial to avoid poor performance or overfitting.
- **Parameter tuning:** Finding the right values for the regularization parameter (C) and kernel parameters can be tricky and might require a few iterations. Cross-validation is the preferred technique in the industry.
- **Memory and computational complexity:** SVM can be memory-intensive and computationally expensive, especially when using non-linear kernels on large datasets
- **Sensitivity to outliers:** SVM can be sensitive to noisy data points, as they might significantly impact the placement of the separating hyperplane.

The following are the top challenges of working with KNN:

- **Memory and computational complexity:** Large datasets cost immensely in terms of distance calculation between the new point and each existing point, which decreases the performance of the algorithm.
- **Curse of dimensionality:** High dimensional data is another form of large dataset and faces the same challenge of distance calculation and computational complexity. Additionally, the concept of distance-based similarity becomes less meaningful in high dimensions, making KNN less effective.
- **Parameter tuning:** Selecting the right value for K is critical since a smaller K can lead to noisy predictions, while a larger K might oversimplify the model.
- **Sensitive to noise and outliers:** KNN is highly sensitive to the noise present in the dataset and requires manual imputation of the missing values along with the removal of outliers.

Now that we understand some of the primary challenges of working with these two algorithms, let us list the assumptions these algorithms make.

The following are the assumptions of SVM:

- **Linear separability:** Linear SVM assumes that the data is linearly separable. Hence, it will not perform well in non-linear situations without using the right kernel function.
- **Class imbalance:** SVM works best when the classes are balanced (similar number of data points in each class).
- **Feature scaling:** It is recommended to scale features to ensure that no single feature dominates the others since SVM is sensitive to the scale of features.

- **Continuous data:** SVM is designed for continuous data, and applying it to categorical data will require data transformations.
- **Low-dimensional data:** While SVM can work with high-dimensional data, lower dimensions are more suitable as they reduce the chances of overfitting.

The following are the assumptions of KNN:

- **Similar means same:** KNN assumes that data points within the same class are similar and can be used to infer the class of new data points. The similarity is based on the closeness of the data points.
- **Feature scaling:** Methods such as standardization and normalization need to be performed on the dataset before feeding it to the KNN algorithm to generate reliable predictions.
- **Class imbalanced:** KNN can be sensitive to imbalanced classes, where it would understandably favor the majority class.

The discussion so far gives us sufficient understanding to work with some data to perform a classification exercise.

Case study: Predicting customer propensity to subscribe to a term deposit

As part of the case study, we are going to use Portuguese banking institution data. This is shown in [Figure 6.1](#) and has been used previously in other chapters to perform a classification task to predict if the client will subscribe to a term deposit. We are going to re-use the dataset to perform classification using SVM and KNN in this chapter.

We know that the dataset has 45,211 records with 17 attributes and can be considered large for SVM and KNN purposes. In other words, they can work with much smaller

datasets and still produce reliable results. Let us start by randomly selecting 10,000 records and performing a quick EDA using the `ProfileReport` function.

EDA using the `ProfileReport` function:

```
from ydata_profiling import ProfileReport

profile = ProfileReport(Bank_df, title="Profiling Report")
```

`profile`

Output:

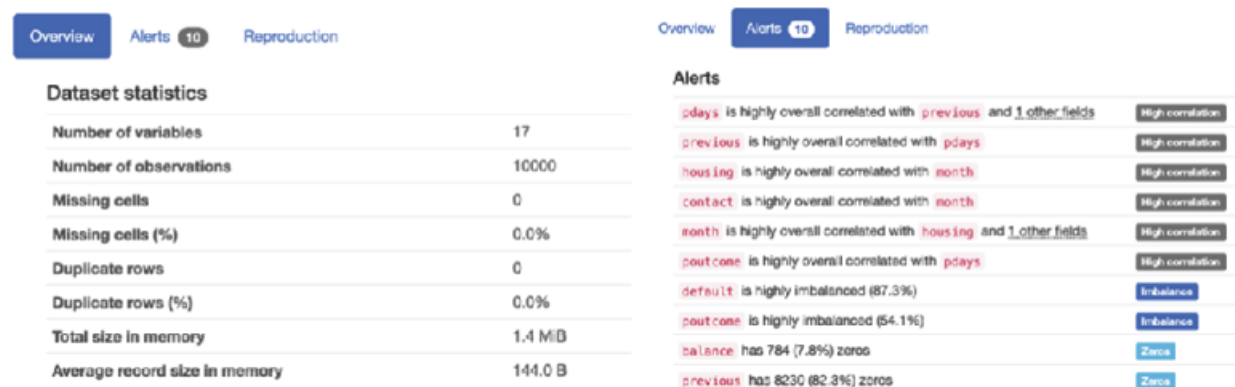


Figure 6.7: Data profile: Overview and summary tabs

The `Alert` section provides a great summary of issues in the dataset in one place. For instance, `poutcome` is highly overall correlated with `pdays` implying multi-collinearity in the dataset and only one of these two variables should be used in the model training. Secondly, if the variable `previous` has 82.3% zeros, we should consider omitting it from the modeling exercise since it most likely does not explain any variation in the `Target` variable.

The `Target` variable profile looks like in [Figure 6.8](#) and is generated using the same code shown as follows:



Figure 6.8: Data profile: Target variable

Target is defined as *client subscribes to a term deposit*(binary: True, False). The 'True' event rate is 1139/10,000, which is 11.39%.

The correlation heat map generated using the **ProfileReport** function is also shown:

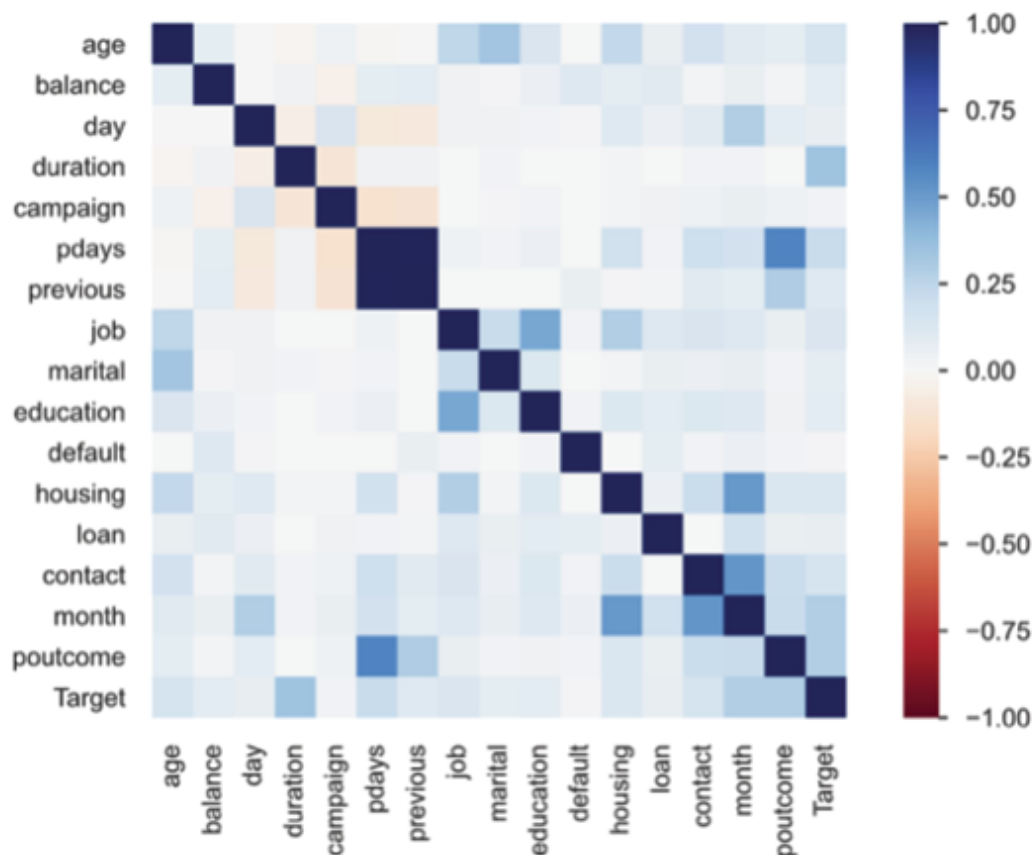


Figure 6.9: Data profile: Correlation heatmap¹

A visual inspection of the heatmap in [Figure 6.9](#) shows that **Target** (rightmost column) is mostly correlated with **month**, **poutcome**, and **duration**. With basic EDA in place, we gain an understanding of the data quality such as missing values and outliers across both numerical and categorical variables. We clip the outliers at the 99th percentile using the following code:

```
Bank_df[numerical_cols]=Bank_df[numerical_cols].apply(lambda x: x.clip(lower = x.dropna().quantile(0.01), upper = x.quantile(0.99)))
```

After data treatment (more detailed steps are available in the GitHub code) is complete, we move on to split the dataset in train and test using the following code:

```
# Lets first separate train and test sets
y = Bank_df['Target']

df_train, df_test =
train_test_split(Bank_df, test_size=0.3, random_state
= 12, stratify = y)

df_train.shape, df_test.shape
```

Output:

```
((7000, 17), (3000, 17))
```

An important thing to note when working with categorical data in ML is to convert these variables into a numerical format that algorithms can understand. Two commonly used techniques for encoding categorical variables are one-hot and label encoding. While One-Hot Encoding creates dummy variables, label encoding simply assigns a numerical value to a category in alphabetical order. We chose label encoding because we do not want to create additional dimensions with

One-Hot encoding. The following code performs label encoding:

```
categorical_cols = ["job","marital",  
"education","default","housing","loan",  
"contact","month","poutcome"]  
  
from sklearn.preprocessing import LabelEncoder  
  
le = LabelEncoder()  
  
#Always remember to fit on train data, not test  
data  
  
le.fit(X_train[categorical_cols].values.flatten())  
  
le_df =  
X_train[categorical_cols].apply(le.fit_transform)  
  
X_tr_transformed =  
pd.concat([le_df,X_train[numerical_cols]],axis=1)  
  
del le_df  
  
le_df =  
X_test[categorical_cols].apply(le.fit_transform)  
  
X_tst_transformed =  
pd.concat([le_df,X_test[numerical_cols]],axis=1)
```

As a next step, it is important to scale the continuous features since both SVM and KNN techniques are sensitive to the scale of the features. The following code performs the Standard scaling on train data.

```
#scale the numeric columns  
  
from sklearn.preprocessing import MinMaxScaler,  
StandardScaler  
  
#numerical_cols=  
["age","balance","day","campaign","pdays","previous
```

```

"]

#create and fit scaler

scaler = StandardScaler()

X_tr_transformed[numerical_cols]=
scaler.fit_transform(X_tr_transformed[numerical_cols])

#scale selected data

X_tr_transformed.head()

```

Output:

	job	marital	education	default	housing	loan	contact	month	poutcome	age	balance	day	duration	campaign	pdays	previous
8572	1	0	0	0	1	0	0	6	1	0.579548	-0.510373	-1.542669	4.433574	-0.259866	-0.409297	-0.351652
32388	9	1	2	0	1	0	0	0	1	-0.479343	1.435399	0.026977	2.870348	-0.259866	-0.409297	-0.351652
8699	9	2	2	0	1	0	0	6	1	-0.960656	-0.425490	-1.542669	-0.997647	5.018233	-0.409297	-0.351652
27037	4	1	2	0	1	0	0	9	1	0.194497	-0.531267	0.630687	-0.949959	-0.636873	0.769324	0.338055
32219	1	0	1	0	1	0	0	0	1	0.772073	-0.767197	0.026977	-0.745962	-0.636873	-0.409297	-0.351652

Figure 6.10: Standardized and label encoded train data

Next, we perform scaling on test data separately:

```

# Scaling numeric values of test data

X_tst_transformed[numerical_cols] =
scaler.transform(X_tst_transformed[numerical_cols])

X_tst_transformed.head()

```

Output:

	job	marital	education	default	housing	loan	contact	month	poutcome	age	balance	day	duration	campaign	pdays	previous
27269	9	1	2	0	0	1	0	9	1	0.387022	-0.793750	0.630687	3.217407	-0.259866	-0.409297	-0.351652
16590	7	1	1	0	1	0	0	5	1	-1.153182	-0.551726	0.992914	-0.796299	0.871156	-0.409297	-0.351652
22673	9	2	2	0	1	0	0	1	1	-0.575605	-0.445514	1.113656	-0.830740	0.871156	-0.409297	-0.351652
1379	5	1	0	0	0	0	0	8	1	1.830963	-0.385008	-0.938959	0.634329	-0.636873	-0.409297	-0.351652
16992	7	1	1	1	1	0	0	5	1	-0.768131	-0.704079	1.113656	-0.481031	-0.636873	-0.409297	-0.351652

Figure 6.11: Standardized & label encoded Test data

Please note, that it is important to perform scaling on train and test data separately to prevent data leakage. When we standardize the data using mean and standard deviation values calculated from the entire dataset (*train+test set*), we introduce information from the test set into the training process. This can severely impact the model's ability to perform well on new, unseen data. Data leakage can mislead you into believing that your model is performing better than it actually would on new data.

With this modeling (train and test) dataset ready, let us move on to training some models. We will begin by setting up an **MLFlow** experiment to log our experiments using the following code and then train a base SVM model:

```
#Log in MLFlow

experiment_name = "SVM_KNN_RUN"
run_name="Product_Subscriber"
mlflow.set_tracking_uri("http://127.0.0.1:5000/")
mlflow.get_tracking_uri()


#set experiment name
mlflow.set_experiment(experiment_name)
mlflow.sklearn.autolog()
with mlflow.start_run():
    model_svm = SVC(kernel='linear',
                    C=1.0,
                    degree=1,
                    random_state=42)
```

```

    model_svm.fit(X_tr_transformed, y_train)

    predictions =
model_svm.predict(X_tst_transformed)

    mlflow.sklearn.log_model(model_svm,
"SVM_base_model")

    test_Accuracy = accuracy_score(y_test,
predictions)

    mlflow.log_metric("test_Accuracy",
test_Accuracy)

    print(f"Test Accuracy: {test_Accuracy:.2%}")

    print("Model run: ",
mlflow.active_run().info.run_uuid)

    mlflow.set_tag("tag1", "Base SVM Model")

mlflow.end_run()

print('Run - %s is logged to Experiment - %s' %
(run_name, experiment_name))

```

Output:

Test Accuracy: 88.60%

Model run: 97841c159d3240c6995e2bb963a1aeb3

Run - Product_Subscriber is logged to Experiment - SVM_KNN_RUN

This is a decent accuracy given the 11.39% event rate (# of yes in `Target`) in our train dataset. It is accurately able to predict 6202 labels (yes and *no* both) out of the 7,000 labels we have in the training dataset. We used a `c` value of 1 with a `linear` kernel to train this model. Can we do better? Let us perform some hyperparameter tuning using the `Hyperopt`

library. The following code sets up the tuning space for different hyperparameters:

```
from hyperopt import hp,fmin,tpe,STATUS_OK,Trials
from hyperopt.pyll import scope

space = {
    'C': hp.choice('C',
np.arange(0.005,1.0,0.01)),
    'kernel': hp.choice('kernel',['linear',
'poly', 'rbf']),
    'degree':hp.choice('degree',[2,3,4]),
    'probability':hp.choice('probability',[True])
}
```

Few points to note about above piece of code:

- `hp.choice('C', np.arange(0.005, 1.0, 0.01))` specifies that the search space for `C` is a choice among values in the range `[0.005, 0.015, 0.025, ..., 0.995]`
- `hp.choice('kernel', ['linear', 'poly', 'rbf'])` specifies that the search space for `kernel` is a choice among the listed kernel types.
- `hp.choice('degree', [2, 3, 4])` specifies that the search space for `degree` is a choice among polynomial degrees 2, 3, and 4.
- The hyperparameter `probability` controls whether the SVM should enable probability estimates for classification. When set to `True`, the `predict_proba` method becomes available.

Then, we define the objective function that returns a value that indicates the model's performance using the following code:

```
def objective(space):
    model = SVC(**space)
    model.fit(X_tr_transformed,y_train)
    y_pred = model.predict(X_tst_transformed)
    accuracy = accuracy_score(y_test, y_pred)
    return {'loss' : -accuracy , 'status' :
STATUS_OK}
```

We return the test accuracy score using the objective function. Next, Hyperopt iteratively samples the hyperparameters from the defined search space based on the chosen algorithm. It then evaluates the objective function using these hyperparameters and collects the results using the following code:

```
from sklearn.model_selection import cross_val_score
trials = Trials()
best = fmin(fn = objective,
            space=space,
            algo= tpe.suggest,
            max_evals=80,
            trials=trials
            )
```

Best

The preceding code runs until a predefined number of iterations or until a termination condition is met. As it progresses, it focuses more on the promising areas of the hyperparameter space. After the search process is complete, Hyperopt returns the set of hyperparameters that yield the

best performance according to the objective function. The following output is the **Best** result of the preceding code:

Output:

```
{'C': 58, 'degree': 2, 'kernel': 1, 'probability': 0}
```

We will use these **Best** hyperparameters to train our best SVM model. The code is shown as follows:

```
#Hypertuned model
```

```
mlflow.set_experiment(experiment_name)
```

```
mlflow.sklearn.autolog()
```

```
with mlflow.start_run():
```

```
    svm_tuned = SVC(kernel='poly',
```

```
                    C=58,degree=2,
```

```
                    random_state=42)
```

```
    svm_tuned.fit(X_tr_transformed, y_train)
```

```
    predictions =
```

```
svm_tuned.predict(X_tst_transformed)
```

```
    y_pred = np.where(predictions>0.5,1,0)
```

```
    mlflow.sklearn.log_model(svm_tuned, "SVM  
Hypertuned")
```

```
    test_Accuracy = accuracy_score(y_test,  
predictions)
```

```
    mlflow.log_metric("test_Accuracy",  
test_Accuracy)
```

```
    print(f"Test Accuracy: {test_Accuracy:.2%}")
```

```
    f1 = f1_score(y_test, y_pred)
```

```

    print(f"Test F1 Score: {f1:.2%}")

mlflow.log_metric(key="f1_experiment_score",value=f1)

    print("Model run: ",
mlflow.active_run().info.run_uuid)

    mlflow.set_tag("tag1", "Hypertuned SVM")
mlflow.end_run()

print('Run - %s is logged to Experiment - %s' %
(run_name, experiment_name))

```

Output:

Test Accuracy: 89.77%

Test F1 Score: 35.37%

Model run: 99cf02f45d704e72a75f0d83515311bb

Run - Product_Subscriber is logged to Experiment - SVM_KNN_RUN

We can observe that our test accuracy has improved from 88.6% of the base model to 89.77% of this hyperparameter-tuned model. The AUC on the test dataset can be obtained using the following code:

```

#you can select threshold based on roc curve
RF_roc_auc = roc_auc_score(y_test, y_pred_test_SVM)
fpr, tpr, thresholds = roc_curve(y_test,
y_pred_test_SVM)
plt.figure(figsize = (12,8))
plt.plot(fpr, tpr, linewidth=1, label='Random
Forest (area = %0.2f)' % RF_roc_auc)

```

```
plt.plot([0, 1], [0, 1], 'r--')
plt.xlim([0.0, 1.0])
plt.ylim([0.0, 1.05])
plt.title("SVM ROC Curves")
plt.xlabel('False Positive Rate (1 - Specificity)')
plt.ylabel('True Positive Rate (Sensitivity)')
plt.legend(loc = 'lower right')
plt.show()
```

Output:

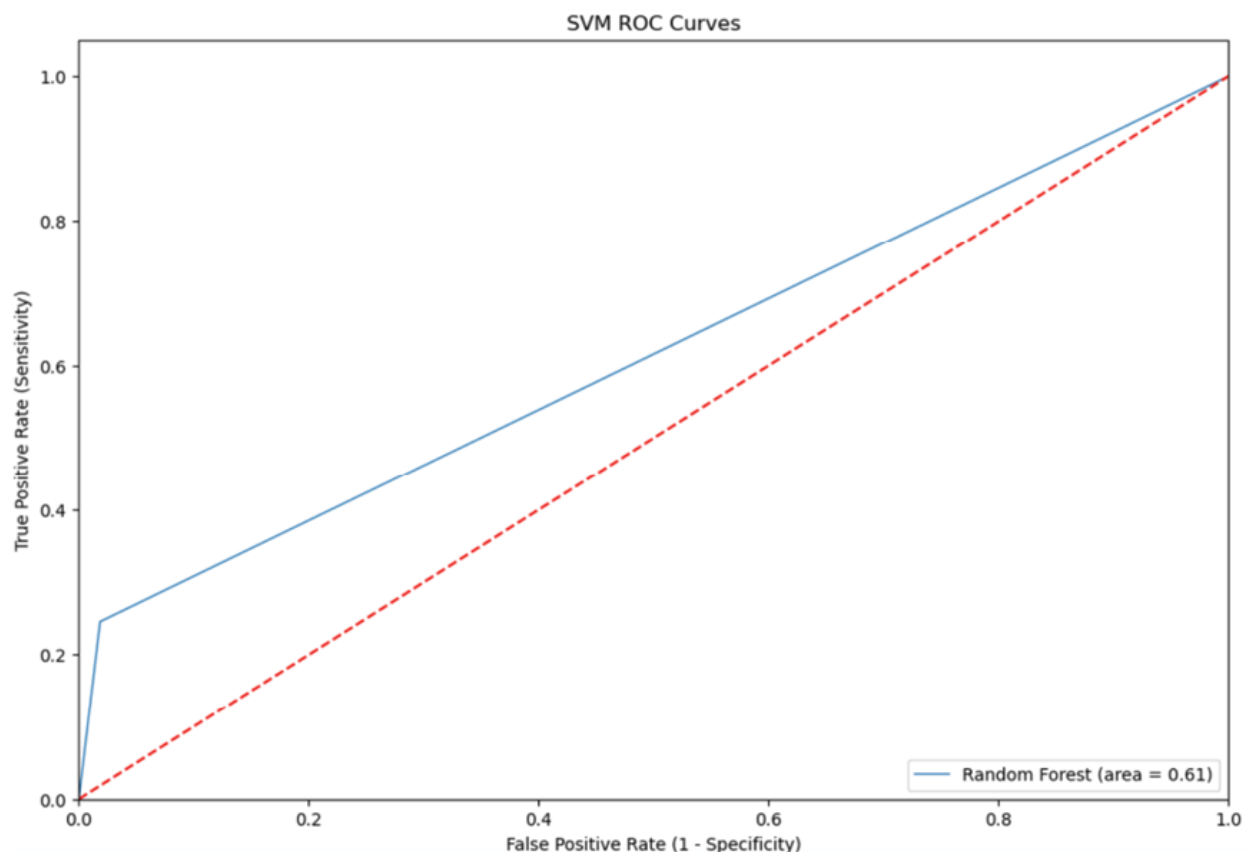


Figure 6.12: SVM: Hyperparameter tuned ROC curve

This was a high-level attempt to train an SVM model, but there are additional strategies to improve this performance further by trying other feature scaling methods, kernel types, adjusting regularization parameter c , and limiting the features to only the most important ones. Understanding the key features, out of the total 17, contributing to the model's decision can help refine the model and improve its performance.

Now, let us compare SVM's performance with KNN. It is important to note that when the KNN algorithm gets the training data, it does not learn and make a model, it just stores the data. Instead of finding any discriminative function with the help of the training data, it follows instance-based learning. It then uses the training data when it actually needs to make some predictions on the unseen datasets. As a result, KNN does not immediately learn a model but rather delays the learning, thereby being referred to as *lazy learner*.

We will start by identifying the right value of k , that is the number of neighbors, before settling on the final model. This can be done by trial-and-error, trying a range of k values with the algorithm and tracking the model accuracy. The following code performs this procedure:

```
from sklearn.neighbors import KNeighborsClassifier
# Train different values of k for KNN and finding
accuracy for them
knn = KNeighborsClassifier(n_neighbors=1)
accuracy_rate=[]
for i in range(1,20):
    knn=KNeighborsClassifier(n_neighbors=i)
```

```
score=cross_val_score(knn,X_tr_transformed,y_train,
cv=10)

    accuracy_rate.append(score.mean())

#print(accuracy_rate)

# plotting accuracy of KNN for every value of K.
Accuracy is highest when K=5

plt.figure(figsize=(20,10))

plt.plot(range(1,20),accuracy_rate,color='green',li
nestyle='dashed',marker='o',markerfacecolor='yellow
',markersize=10)

plt.title('Accuracy rate vs k value')

plt.xlabel('k')

plt.ylabel('Accuracy rate')
```

Output:

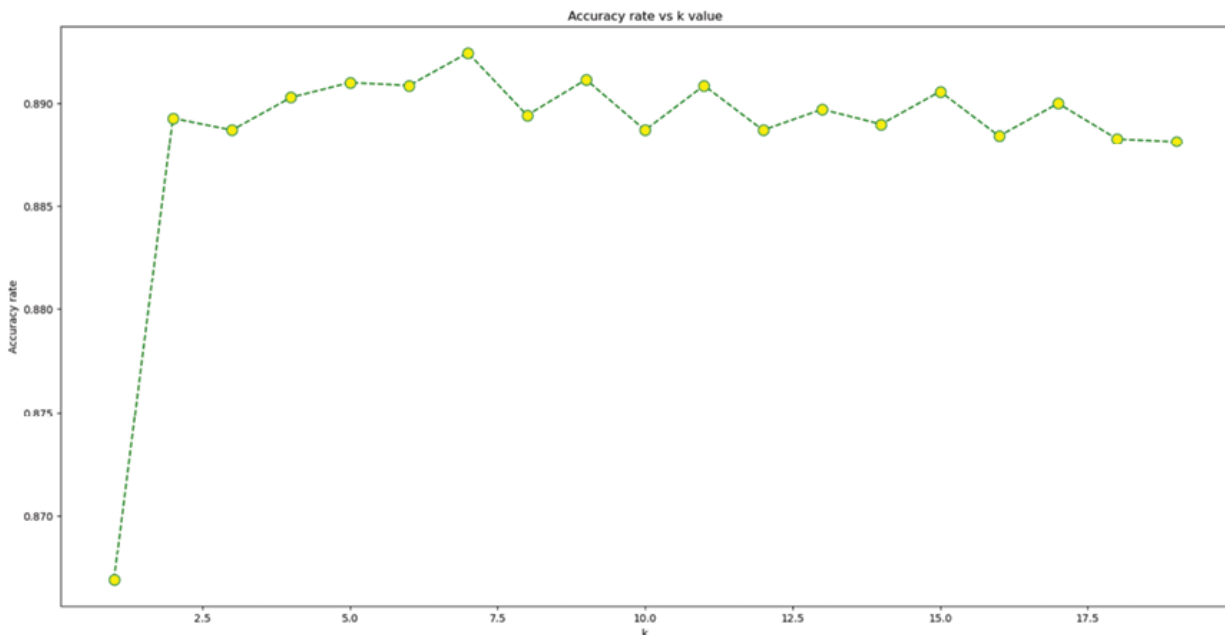


Figure 6.13: KNN - Accuracy Plot by K value

From above [Figure 6.13](#), it is apparent that $k=5$ gives the best accuracy. We will log this in our MLFlow experiment using the following code:

```
mlflow.set_experiment(experiment_name)
mlflow.xgboost.autolog()
with mlflow.start_run():
    knn = KNeighborsClassifier(n_neighbors=7)
    knn.fit(X_tr_transformed, y_train)
    predictions = knn.predict(X_tst_transformed)
    y_pred = np.where(predictions>0.5,1,0)
    mlflow.sklearn.log_model(knn, "KNN")
    test_Accuracy = accuracy_score(y_test,
    predictions)
    mlflow.log_metric("test_Accuracy",
    test_Accuracy)
    print(f"Test Accuracy: {test_Accuracy:.2%}")
    f1 = f1_score(y_test, y_pred)
    print(f"Test F1 Score: {f1:.2%}")

mlflow.log_metric(key="f1_experiment_score",value=f
1)

    print("Model run: ",
mlflow.active_run().info.run_uuid)
    mlflow.set_tag("tag1", "KNN")
mlflow.end_run()
```

```
print('Run - %s is logged to Experiment - %s' %  
(run_name, experiment_name))
```

Output:

Test Accuracy: 89.07%

Test F1 Score: 27.11%

Model run: f156bc3f47d944ad870cf4bd0a1ff026

Run - Product_Subscriber is logged to Experiment -
SVM_KNN_RUN

KNN accuracy, 89.07%, lands slightly short of the SVM accuracy (89.77%) we observed earlier. The AUC is also understandably lower than that of SVM. The following code outputs the ROC curve with the AUC value:

```
#you can select threshold based on roc curve
```

```
KNN_roc_auc = roc_auc_score(y_test, predictions)
```

```
fpr, tpr, thresholds = roc_curve(y_test,  
predictions)
```

```
plt.figure(figsize = (12,8))
```

```
plt.plot(fpr, tpr, linewidth=1, label='KNN (area =  
%0.2f)' % KNN_roc_auc)
```

```
plt.plot([0, 1], [0, 1], 'r--')
```

```
plt.xlim([0.0, 1.0])
```

```
plt.ylim([0.0, 1.05])
```

```
plt.title("ROC Curves")
```

```
plt.xlabel('False Positive Rate (1 - Specificity)')
```

```
plt.ylabel('True Positive Rate (Sensitivity)')
```

```
plt.legend(loc = 'lower right')
```

```
plt.show()
```

Output:

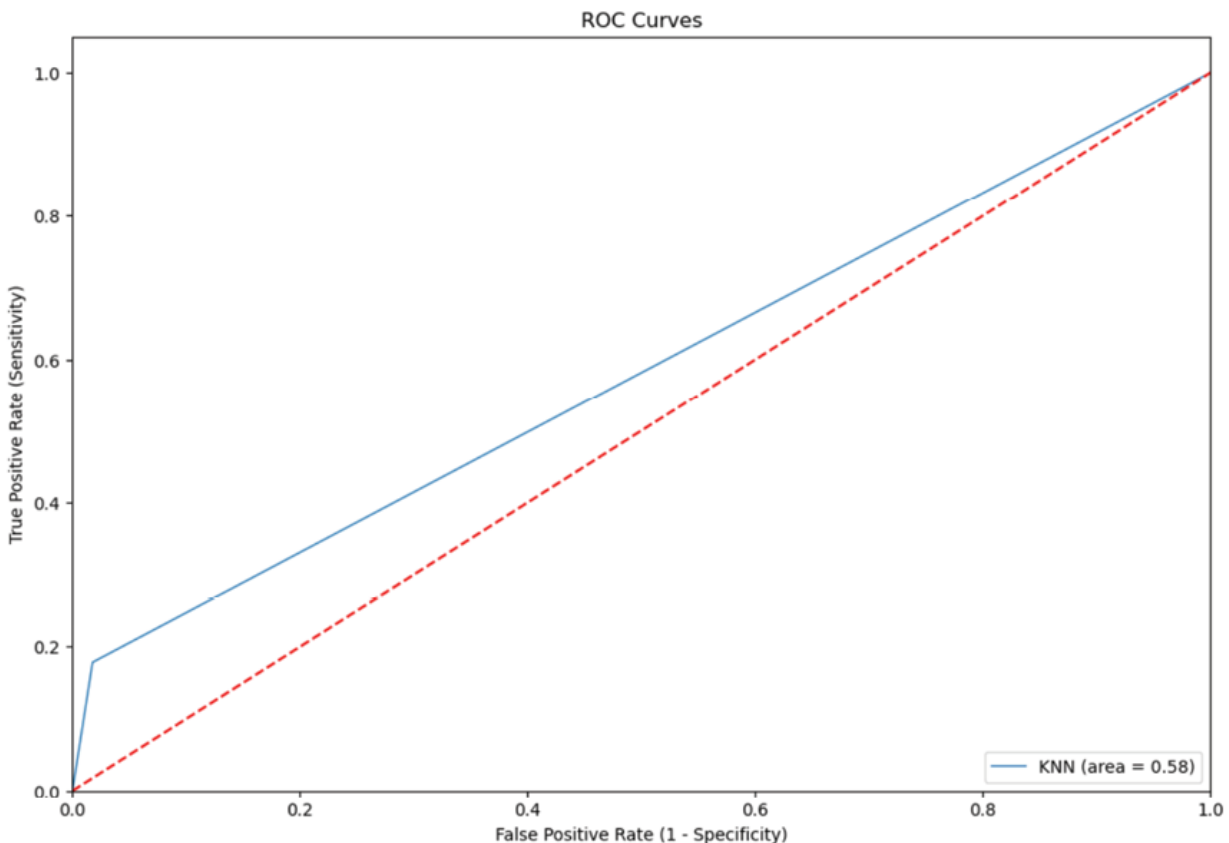


Figure 6.14: KNN - ROC curve

It is no surprise that SVM performs better than KNN with its strong mathematical backing. But we still have an opportunity with KNN to try better feature engineering and dimensionality reduction to improve results.

Going back to the results of the logistic regression, random forest, and xgboost on the same dataset in [Chapter 5: Decision Trees with Bagging and Boosting](#), it is worth mentioning that the RF model performed better on test data (accuracy: 90.14%) compared to the logistic regression (accuracy: 84.49%) model but xgboost outshined both in test accuracy with a value of 90.25%. The following table

compares the results of five classification techniques on the Portuguese banking data:

Classification technique	Test accuracy
Logistic regression	84.49%
Random forest	90.14%
Xgboost	90.25%
SVM	89.77%
KNN	89.07%

Table 6.4: Performance Comparison by Test Accuracy

This chapter concludes our study of the various traditional classification algorithms and it is interesting to note how a simple algorithm like KNN can achieve powerful predictions. These are comparable to their sophisticated counterparts while being highly interpretable. But in some use cases, a 1% accuracy gain could be a huge deal over explainability, leading us to choose Xgboost over KNN. The good news is that with functionalities like SHAP (discussed in Chapter 5, Decision Tree Algorithm With Bagging And Boosting), we can have the dual advantage of accuracy and explainability while working with Xgboost.

Conclusion

In the real-world use cases of machine learning, simple solutions will always triumph over sophisticated ones. This is especially true in regulated industries where interpretability and explainability are big factors in ML adoption.

The tale of SVM resonates with the quest for balance, finding the fine line between fitting the data just right and guarding against overfitting. However, since interpretability can be an issue, it may lose out if higher accuracy is not the only motivation.

KNN, on the other hand, whispered to us simplicity yet effectiveness. This taught us that sometimes, the best way to understand something is to listen to the echoes of those around it and still produce remarkable results.

The case study illuminated these concepts further, showing us a real-world application and concluding with a comparative study of various classification techniques. SVM's knack for classifying responders vs not and KNN's role in recognizing types of responders from their attributes demonstrated how these algorithms play a vital role in solving practical problems. The case study in this chapter demonstrates that every algorithm has a unique place in ML and their selection should be fully dependent on the specific use case need.

Our journey through SVM and KNN has not only expanded our knowledge but has also provided us with tools to tackle classification challenges comprehensively. So, as we step into the broader landscape of machine learning that includes unsupervised methods, let us carry the wisdom of Support Vector Machines and K-nearest neighbors with us, knowing that in classification, they stand as steadfast guides to navigate patterns and predictions. But before we go any further with learning additional algorithms, in the next chapter, we need to learn to solve issues caused by high-dimensional data that could also be sparse. In most situations, this high-dimensional data is not informational and needs to be reduced to useful features.

Points to remember

- The SVM cost function has an L2 Regularization feature, which provides it with good generalization capabilities, hence preventing over-fitting.
- Use cross-validation to select the optimal value of k for the k -NN algorithm, which helps improve its

performance and prevent overfitting or underfitting.

- The right value for K will depend on the data set and the specific use case. For instance, a lower value of K will make the model more sensitive to outliers, while a higher value will make the model more resistant to them.
 - Low dimensional space requirement for KNN and SVM is critical in improving the performance of these algorithms.
 - Perform feature selection to select the most relevant features for the model or perform feature transformation to transform the data into a more suitable form for these algorithms.
 - Avoid data leakage while performing model data preparation for accurately assessing the model performance on unseen data.
 - Key overall considerations for classification model training:
 - Preprocess data appropriately (scaling, handling missing values, outliers).
 - Tune hyperparameters carefully to prevent overfitting.
 - Consider the interpretability and computational complexity of the algorithm.
 - Handle class imbalance and outliers when necessary.
 - Validate and evaluate models using appropriate metrics (accuracy, precision, recall, F1-score, AUC-ROC, and so on).
 - Keep track of your experiments for efficiency and explainability
-

¹ *Figure 6.9: The difference in colors might not show as the book is printed in black and white. Please refer to the colored image bundle link provided at the beginning of this book.*

Join our book's Discord space

Join the book's Discord Workspace for Latest updates, Offers, Tech happenings around the world, New Release and Sessions with the Authors:

[**https://discord.bpbonline.com**](https://discord.bpbonline.com)



CHAPTER 7

Putting Dimensionality Reduction into Action

Introduction

Simplicity is the ultimate sophistication.

- Leonardo da Vinci

In the ever-expanding world of data and information, complexity often hinders our ability to extract meaningful insights and make sense of the underlying patterns. The abundance of features and variables in high-dimensional datasets poses challenges for data analysis and modeling. Fortunately, dimensionality reduction techniques come to our aid, allowing us to distill complexity into simplicity and uncover the essence of the data. In this chapter, we explore the realms of dimensionality reduction, where we explore the powerful techniques of **Principal Component Analysis (PCA)**, **t-distributed Stochastic Neighbor Embedding (t-SNE)**, and **Linear Discriminant Analysis (LDA)**. These techniques enable us to transform high-dimensional data into lower-dimensional representations while preserving crucial information and structures. At the heart of dimensionality reduction lies the idea of simplification. Just as Leonardo da Vinci believed that simplicity holds the essence of sophistication, we aim to distill the intricate complexity of high-dimensional data into compact representations that retain the critical aspects. By reducing the dimensionality, we can gain a

deeper understanding, facilitate visualization, and enhance subsequent analysis tasks.

So, let us explore simplification in the data mining realm, where we harness the techniques of PCA, t-SNE, and LDA to unravel the essence of our data and bring sophistication to the forefront.

Structure

In this chapter, we will learn the following topics:

- Dimensionality reduction
- Background
- Under the dimensionality reduction hood
 - Data
 - Model: Reducing dimensions and variance
 - Principal Component Analysis
 - Linear Discriminant Analysis
 - t-distributed Stochastic Neighbor Embedding
 - Loss: Measuring variance reduction
- Challenges and assumptions
- Case study: Predicting loan repayment propensity using logistic regression, PCA, and LDA
- PCA parameters and interpretation
- LDA parameters and interpretation
- Logistic regression

Objectives

The objective of this chapter is to explore the powerful techniques of dimensionality reduction, namely **Principal Component Analysis (PCA)**, **t-distributed Stochastic Neighbor Embedding (t-SNE)**, and **Linear Discriminant Analysis (LDA)**. The aim is to provide a comprehensive

understanding of these techniques, their inner workings, and their practical applications through a real-world case study.

The detailed case study will provide a step-by-step walkthrough, showcasing the application of PCA, t-SNE, and LDA on a real-world dataset. By examining the inner workings of these techniques within the context of a practical scenario, readers will gain valuable insights into the mathematical foundations of these techniques. This will help unravel the intricacies of transforming high-dimensional data into lower-dimensional representations while preserving important patterns and structures.

Furthermore, we will discuss the evaluation of dimensionality reduction results, strengths, and limitations of each technique, while interpreting and visualizing the reduced-dimensional representations generated by PCA, t-SNE, and LDA.

By the end of this chapter, readers will be equipped with the knowledge and practical insights necessary to effectively evaluate and validate the dimensionality reduction results to ensure the suitability and effectiveness of the applied techniques.

Dimensionality reduction

The greatest ideas are the simplest.

- William Golding

In the vast landscape of data mining and machine learning, the concept of dimensionality reduction empowers us to extract valuable insights from large and complex datasets. As we navigate through the intricate world of data and data mining, the ability to reduce the dimensionality of our data becomes very important. In simplest terms, dimensionality reduction means, the process of transformation of data from high dimensional space to low dimensional space while maintaining most of the meaningful insights from the original data. In this chapter, we will explore the transformative power of dimensionality reduction using three prominent techniques: PCA, t-SNE, and LDA.

Background

In the realm of data mining and machine learning, we often encounter high-dimensional datasets characterized by an abundance of features. While this wealth of information holds the potential for deep insights, it also poses significant challenges. The curse of dimensionality looms large, leading to increased computational complexity, overfitting, and difficulties in visualization and interpretation. This is where the practice of dimensionality reduction comes to our help. Dimensionality reduction techniques aim to simplify complex datasets by transforming them into lower-dimensional representations while preserving essential information and inherent structures.

By condensing the data into a more manageable space, we gain a clearer understanding of the underlying patterns and relationships, making subsequent analysis and modeling tasks more efficient and effective.

Please note that it is not a modeling technique for making predictions but within the realm of data mining and machine learning, dimensionality reduction plays a crucial role in several aspects:

- **Feature extraction or combination:** With an abundance of features, it becomes essential to identify the most relevant and informative ones. Dimensionality reduction helps us extract the most salient latent features by weighted additive combinations of all the columns in the original dataset, thereby improving model performance.
- **Clustering and visualization:** Visualizing high-dimensional data directly is a formidable task. Dimensionality reduction techniques enable us to project data onto lower-dimensional spaces, facilitating visualization and enabling the identification of clusters, patterns, and outliers.
- **Computational efficiency:** High dimensionality often leads to increased computational complexity, making certain algorithms infeasible or time-consuming. By reducing the dimensionality, we can streamline computations and accelerate model training and inference.

- **Overfitting prevention:** High-dimensional data carries the risk of overfitting, where models become too specialized and struggle to generalize to unseen data. Dimensionality reduction mitigates overfitting by capturing the most informative aspects of the data and removing noise and redundancy.

To ensure the above goals are achieved optimally, the choice of dimensionality reduction method does matter. This choice, in turn, depends on the nature of the data (continuous vs. categorical) and the user's understanding of the data in the context of the business domain. In this chapter, we will look under the hood of PCA, t-SNE, and LDA by digging into their mathematical foundations, practical applications, and the benefits they bring to data mining and machine learning. We will take up a case study on loan repayment data and build a basic logistic regression model followed by a reduced-dimension logistic model to compare performances.

By mastering the art of dimensionality reduction, we uncover the simplicity behind the complexity and unveil only the greatest ideas that lie within. So, let us witness the extraordinary simplicity that awaits us in the realm of data mining and machine learning by unraveling the mysteries of dimensionality reduction using PCA, t-SNE, and LDA.

Under the dimensionality reduction hood

In this section, we will further investigate the differences between the three techniques. We will do this by discussing the holy trinity of data, model, and loss function that defines any data mining solution. Let us start by understanding the input data first.

Data

The loan repayment dataset as shown in [Figure 7.1](#), presents an interesting scenario where both PCA and LDA can be applied to gain insights and simplify the analysis and hence we choose this dataset again to showcase the workings of these techniques. This

is a classification problem and the goal of the exercise using this dataset is to predict if a customer will repay the loan (Yes/No). Following are a couple of specific reasons for this dataset choice:

- The loan repayment dataset consists of 42 features related to borrowers' demographics, financial information, loan details, and repayment history. With numerous variables, PCA can effectively identify the principal components that explain the most significant variations by retaining the essential information needed for analysis and prediction.
- On the other hand, LDA can be a valuable technique because it is specifically designed for supervised classification tasks and can identify the discriminative features that differentiate loan recipients based on their loan repayment behavior.

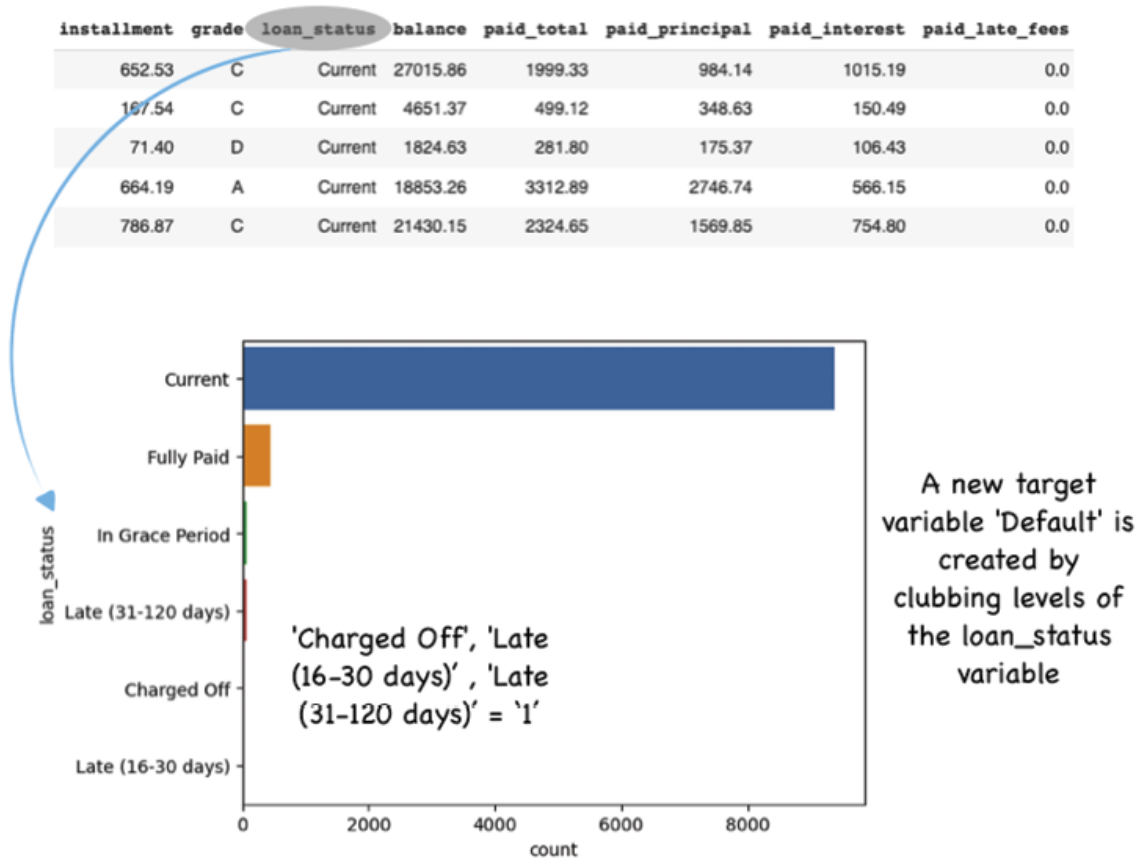


Figure 7.1: Loan prepayment dataset with target variable loan_status

In summary, the loan repayment dataset offers a suitable context for both PCA and LDA. PCA can help us identify the key factors driving loan repayment behavior by reducing the dimensionality of the dataset and capturing the most significant variations. On the other hand, LDA can focus on the discriminative features that differentiate borrowers based on their loan repayment outcomes, enabling better risk assessment and prediction. Applying these dimensionality reduction techniques to the loan repayment dataset can simplify the analysis, reveal important patterns, and enhance decision-making in the context of loan repayment and risk management.

Model: Reducing dimensions and variance

Intuitively, this approach is extremely useful when working with data sets having a large number of features. We very commonly come across datasets that are sparse or have correlated features. Applications such as image processing and language models always have to deal with thousands if not tens of thousands of features. Scikit-Learn, the most popular library for dimensionality reduction consists of three main modules for dimensionality reduction algorithms:

- Discriminant analysis
 - Linear discriminant analysis
- Decomposition algorithms
 - Principal component analysis
 - Kernel principal component analysis
 - Non-negative matrix factorization
 - Singular value decomposition
- Manifold learning algorithms
 - t-Distributed stochastic neighbor embedding
 - Spectral embedding
 - Locally linear embedding

For compressing neural network inputs, frameworks like Pytorch, Pytorch Lightning, Keras, and TensorFlow help build autoencoders.

- Deep learning algorithms
 - Autoencoders can reduce dimensions and learn features and representations.

While there are different motivations and mathematical approaches to reducing dimensions in a dataset, we will focus on LDA and PCA which both help in dimensionality reduction by focusing primarily on projecting the features in higher dimension space to a lower dimension one. LDA, being a supervised algorithm, focuses on maximizing the separability among known data categories by creating a new linear axis and projecting the data points on that axis, but PCA transforms the data to a new coordinate system such that the greatest variance by some projections lies on the first coordinate.

We will also conceptually understand the t-SNE algorithm which is about modeling (and not projecting) high-dimensional data into lower-dimensional space by non-linear mapping.

Principal component analysis

Let us start by exploring PCA which transforms the existing variables into a new set of linearly combined variables by approximating a line in which the projection of the points ([Figure 7.2](#)) is the most spread out. This ensures the maximum variation present in the dataset is retained, squeezed, or compressed into the first few sets of new variables (also known as **Principal Components**), and that these PCs are uncorrelated. Since PCA is an unsupervised algorithm, it summarizes the feature set without relying on the output.

Let us start by understanding the following five key steps that lead to the development of these PCs with a 2-variable scenario:

1. **Data standardization:** To ensure fair comparison and equal influence of variables, it is essential to standardize the data by either the standard scaler or the min-max scaling technique. Additionally, convert categorical

variables into numerical ones since PCA only works on numerical variables. This step ensures that variables with larger scales do not dominate the analysis.

2. **Covariance matrix computation:** Next, the covariance matrix of the standardized data is computed. The covariance matrix reveals the relationships and interdependencies between the variables. Each element of the covariance matrix represents the covariance between two variables.
3. **Eigendecomposition of the covariance matrix to identify the PC:** The eigendecomposition of the covariance matrix is performed to obtain its eigenvalues and eigenvectors. The eigenvalues represent the amount of variance captured by each PC or the eigenvector. These vectors simply imply the directions of the axes where there is the most variance or information. See [Figure 7.2](#) as follows:

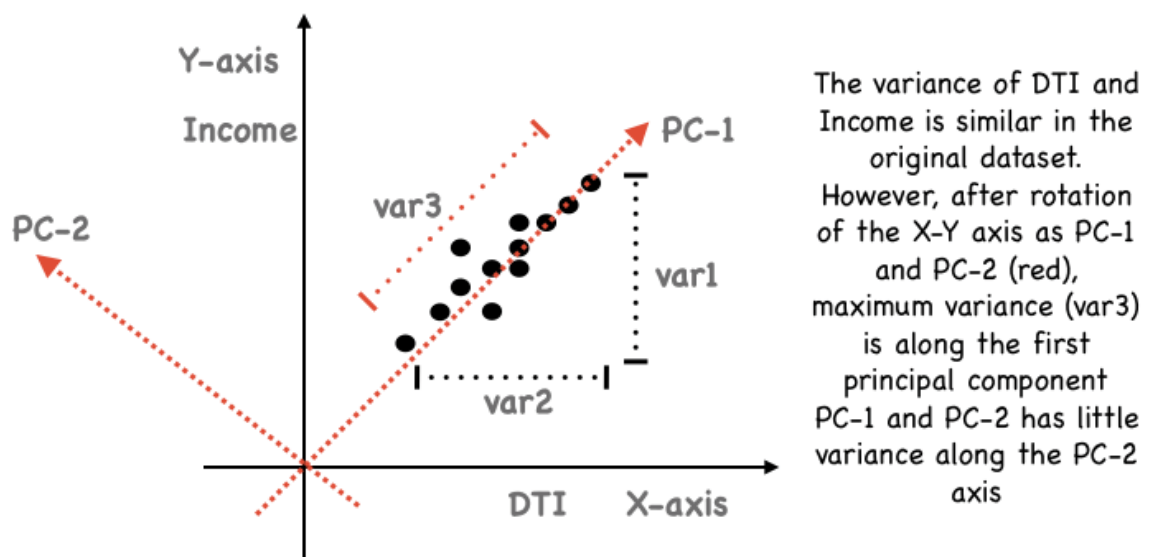


Figure 7.2: Axis transformation with a 2-feature example

A mathematical explanation of the above transformation is provided as follows. As the first step, in the above example with two variables (DTI and Income) a 2×2 covariance matrix is obtained. It looks like the below matrix A:

$$\begin{bmatrix} 510 & 360 \\ 360 & 430 \end{bmatrix}$$

The diagonal blue numbers 510 and 430 are the variance of the individual variables DTI and Income. The off-diagonal number 360 is the covariance of DTI to Income and vice versa.

Given the above square matrix A , v a vector, and λ a scalar that satisfies $Av = \lambda v$, the λ is called the eigenvalue associated with eigenvector v of A . Also, since we have a 2 x 2 matrix we are going to have 2 eigenvalues. They are obtained by solving the equation given in the expression as follows:

$$|A - \lambda I| = 0$$

The left-hand side of the above equation shows the 2 x 2 matrix A minus λ times the Identity matrix. Solving for the determinant of the resulting matrix lends a polynomial of order 2 (this is equal to the dimension of the square matrix A which is 2 as well).

This polynomial is set equal to zero to obtain the desired eigenvalues. In general, there will be 2 solutions for λ and so there are 2 eigenvalues, not necessarily all unique. The polynomial equation is shown as follows:

$$|A - \lambda I| = 0$$

In the preceding equation, $\dim.A = 2$ and solving for the above polynomial (quadratic in this case) provides:

$$\lambda^2 = 1 - \dim.A \text{ and } \lambda_2 = 1 + \dim.A$$

Thereafter, the eigenvectors are solved using the following equation:

$$\left\{ \begin{pmatrix} 510 & 360 \\ 360 & 430 \end{pmatrix} - \lambda \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix} \right\} \begin{pmatrix} e_1 \\ e_2 \end{pmatrix} = \begin{pmatrix} 0 \\ 0 \end{pmatrix}$$

4. **Selection of principal components:** The principal components are ranked based on their corresponding eigenvalues. The principal component with the highest eigenvalue captures the most variance in the data, followed by the subsequent components in descending order. The user determines the number of principal components to retain, usually based on the desired amount of variance to be preserved. [Figure 7.3](#) shows that PC-1 with var3 ([Figure 7.2](#)) explains 95% ($1.9/2$) of the variance:

Feature	Variance	Feature	Variance
DTI	1	PC-1	1.9
Income	1	PC-2	0.1
TOTAL	2	TOTAL	2

Figure 7.3: Retained Principal component table with proportion retained

5. **Projection onto principal components:** The dataset from the original axes is projected onto the selected principal components, transforming it into reduced-dimensional space. This projection is achieved by taking the dot product of the standardized data with the eigenvectors corresponding to the chosen principal components.

A few important points to note with regard to Principal components are:

- They are constructed as linear combinations of the initial variables in the dataset. This means that 10 features in the dataset will give us 10 PCs, but PCA tries to front-load the variance by putting the maximum possible variance information in the first component.
- The second principal component is calculated with the condition that it is uncorrelated or orthogonal with the first principal component and that it accounts for the second-highest variance.
- Variance and information are related because the larger the variance carried by an eigenvector, the larger the dispersion of the data points along it implying the more information it has.

- Eigenvectors and eigenvalues always come in pairs, which means that in an n -dimensional data set, there are n variables, therefore there are n eigenvectors with n corresponding eigenvalues.

Figure 7.4 shows the cumulative variance distribution by PCs:

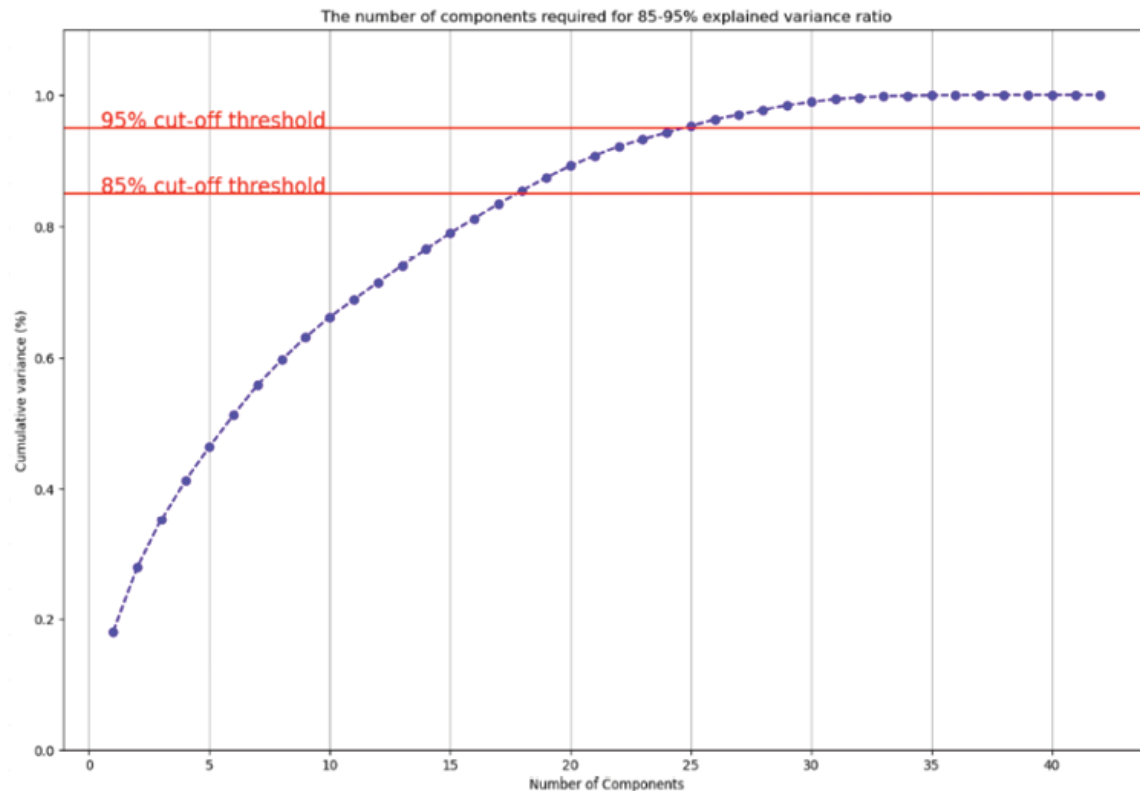


Figure 7.4: Scree plot with 25 PCs explaining 95% variance

Linear discriminant analysis

On the other hand, LDA does not work on finding the principal component but rather looks at what type of features give more discrimination to separate the categorical data by finding a line that maximizes the category separation. This implies that it is a supervised learning algorithm used for classification tasks.

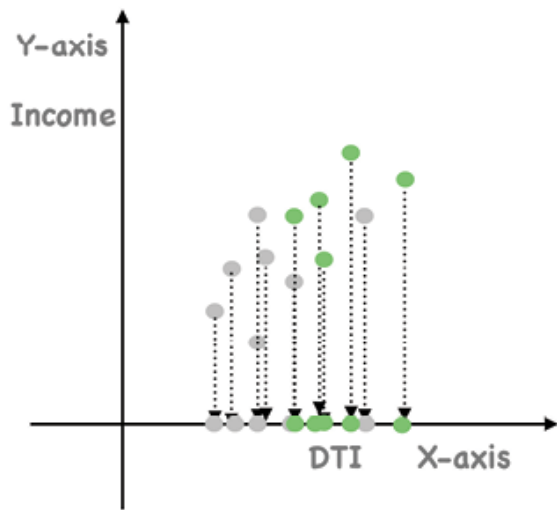
For instance, by applying LDA, we can uncover the features that contribute the most to distinguishing between borrowers who successfully repay their loans and those who default. This information can be highly valuable for predicting loan repayment behavior for new borrowers. LDA reduces the dimensionality of

the dataset while maximizing the separability between different loan repayment classes, enabling more accurate classification or risk assessment. LDA focuses primarily on projecting the features in higher dimension space to lower dimensions. You can achieve this in three steps:

1. Firstly, the separability between classes, which is the distance between the mean of different classes, is calculated. This is called the between-class variance.
2. Secondly, the distance between the mean and sample of each class is calculated. It is also called the within-class variance.
3. Finally, the lower-dimensional space (in some cases a line), which maximizes the between-class variance and minimizes the within-class variance, is defined. Fisher proposed a solution to maximize a function $J(w)$, that represents the difference between the means, normalized by a measure of the within-class variability.

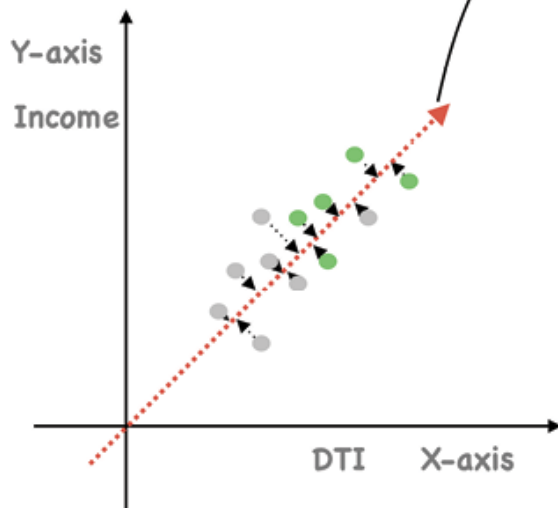
Figure 7.5 shows the comparison of non-LDA and LDA scenarios with a detailed three-step LDA process in scenario 2:

Scenario 1: There is a lot of overlap and no clear categorization of the two classes



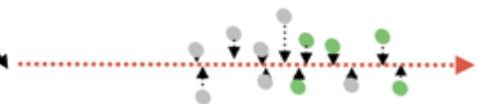
Projection on x-axis (just one variable):

Scenario 2: LDA - There is less overlap and better category separation

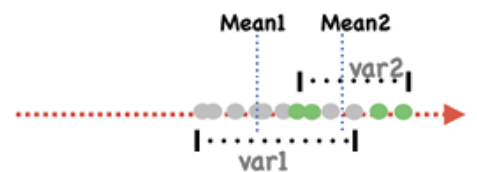


Projection using X & Y axis (both variables)

Step 1: Data projected on the new axis



Step 2: Calculate class parameters



Step 3: Achieve max class separation

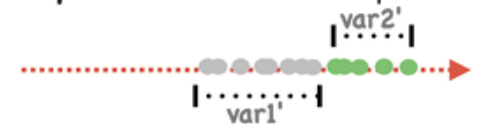


Figure 7.5: The 3-step LDA process with change from 2D to 1D and the objective function

The detailed three-step process of LDA scenario 2 is described as follows:

1. Original data points are projected onto a new axis (dotted red) by leveraging both the features.
2. Mean and variance of each class of data points is calculated and the objective function $\{(Mean1 - Mean2)^2 \div (var1^2 - var2)^2\}$ is applied.
3. Objective function is maximized by maximizing the between class variance (numerator) and minimizing the within-class variance (denominator) until max separation between the two classes is achieved.

A few points to be noted concerning [Figure 7.5](#):

- Data points projected on the x-axis (scenario 1), with information of just one variable, are overlapping.
- A new axis (dotted red- scenario 2) is plotted in the 2D graph such that it maximizes the distance between the means of the two classes and minimizes the variation within each class. This is done to increase the separation between the data points of the two classes.
- All the data points of the classes are plotted on this new dotted red axis and are shown in the 1D graph in scenario 2-step 1.
- Scenario 2-step 3 shows better data point separation compared to scenario 1.

t-distributed Stochastic Neighbor Embedding

Lastly, let us understand the **t-distributed Stochastic Neighbor Embedding (t-SNE)** dimensionality reduction technique which is an unsupervised non-linear method for data exploration and visualizing high-dimensional data. It gives you a sense of how data is arranged in higher dimensions when the linear capabilities of PCA fail.

t-SNE works in the following three steps:

1. The algorithm starts by creating a probability distribution that represents similarities between neighbors. The similarity measure is the conditional probability that points

A would pick point B as its neighbor if neighbors were picked in proportion to their probability density. The similarity of points is calculated under a Gaussian (normal) distribution centered at A. This means we generate a Gaussian distribution with a mean at A, and place our Euclidean distances on the X-axis. See [Figure 7.6](#) as follows:

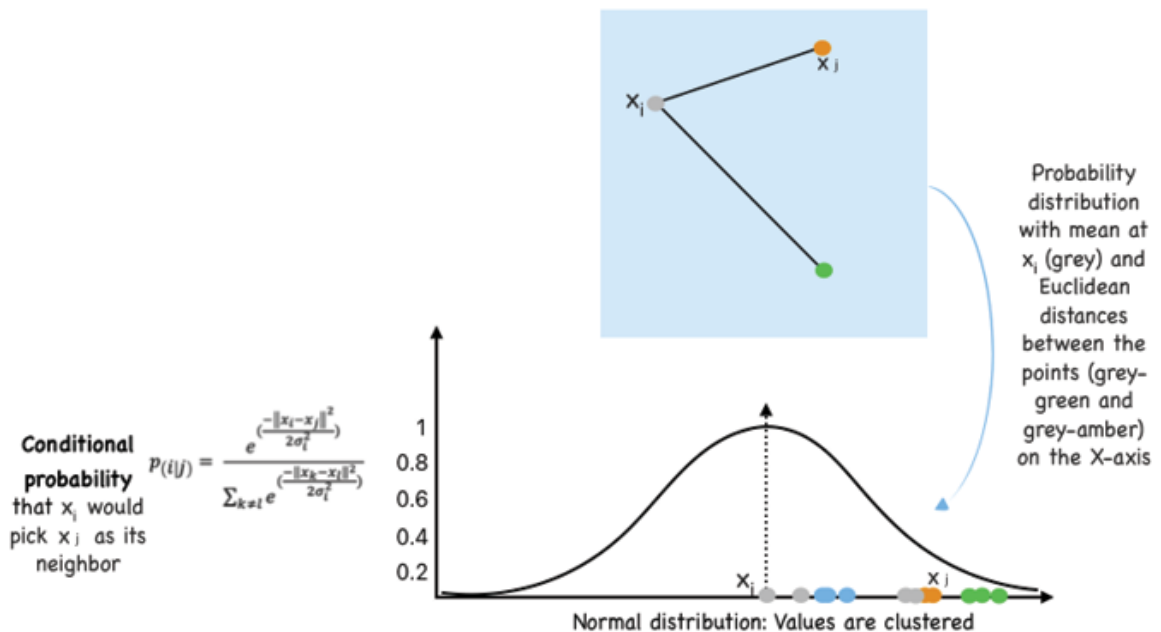


Figure 7.6: Normal distribution of euclidean distances

It is important to note that sigma-sq in the conditional probability formula represents the variance of the normal distribution and is affected by the number of points surrounding the mean or in this case the neighbors. This is where we define perplexity, which is a target number of neighbors for the central point. A higher perplexity indicates a higher variance value.

2. The algorithm then tries to minimize the difference between these conditional probabilities (or similarities) both in higher-dimensional and lower-dimensional space for a perfect representation of data points in lower-dimensional space. Please note that a lower-dimensional space has the same number of points as in the original space but has a Student t-distribution as against the normal distribution. This makes the point getting more evenly distributed and

prevents crowding of the long-distance points along the “short tail” of the normal distribution. A t-distribution is steep and has a “long tail” so points will not get packed at a single point. The conditional probability is shown below in [Figure 7.7](#):

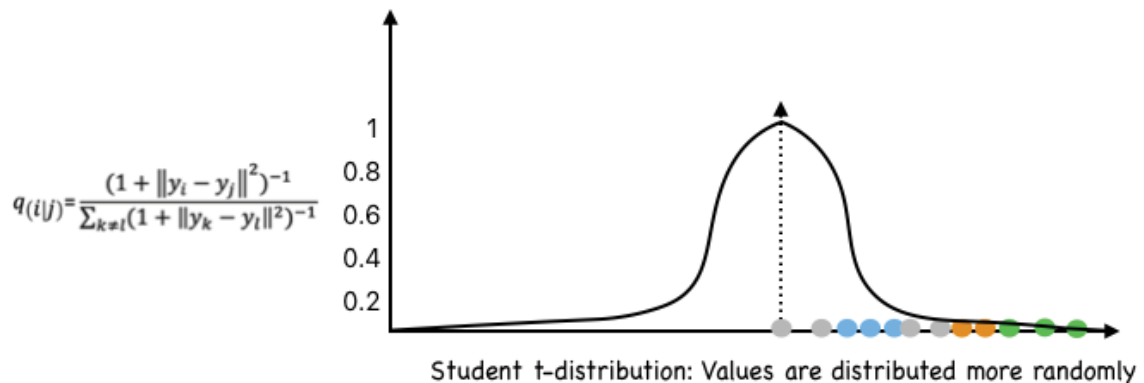


Figure 7.7: Conditional Probability with t-distribution

3. To measure the minimization of the sum of the difference of conditional probability t-SNE minimizes the sum of Kullback-Leibler divergence of overall data points using a gradient descent method. This is the loss optimization method that leads to the optimal distribution of points between the conditional probabilities $p(A|B)$ and $q(A|B)$. One can visualize the gradient optimization process as repulsion and attraction between points to cluster meaningfully along the axis.

Please note that if we take two points and try to calculate conditional probability between them then the values of $p(A|B)$ and $p(B|A)$ will be different. The reason for that is because they are coming from two different distributions.

Loss: Measuring Variance Reduction

Principal Component Analysis: The goal of the objective function is to find a vector such that the variance of projections of points in dataset X onto this one-dimensional vector subspace is maximized. The variance of points projected on such vector v1

is $\text{var}(Xv_1)$ or $v_1^T S v_1$, where S is the sample covariance matrix given $S = \frac{1}{n-1} X^T X$

The first optimization problem may therefore be written as:

$$\max v_1^T X^T X v_1 \text{ s.t. } v_1^T v_1 = 1$$

Figure 7.8: PCA loss function

The constraint of each principal component(v_i) is to be a unit vector. This can be solved by the Lagrange Multipliers technique which provides a strategy for finding the maximum/minimum of a function subject to constraints. $L(v_1^T, \lambda)$ is converted to the Lagrange form $Sv = \lambda v$. This implies that the stationary point (where the partial derivative is zero) of the Lagrangian must be an eigenvector v of the matrix S and one that also maximizes the PCA objective must be the associated eigenvalue λ (Lagrange multiplier) is the largest.

Linear Discriminant Analysis: The goal of the LDA objective function is to maximize the between-group variance and minimize the within-group variance. As shown in [Figure 7.9](#), the larger the numerator greater the separation between groups. While the smaller the denominator less variance within the groups.

$$\frac{(\text{Mean1} - \text{Mean2})^2}{(\text{var1}^2 + \text{var2}^2)}$$

Figure 7.9: LDA loss function

The overall goal is to maximize the above term in [Figure 7.9](#) to find a project matrix W . This is done by differentiating the term with respect to W and setting the derivatives equal to 0, in order to find the maximum. The eigenvector corresponding to the largest eigenvalue is the ideal hyperplane matrix W to project the data onto, to obtain maximum S_B and minimum S_W . The differentiated equation with eigenvalues is shown below:

$$S_B W = I S_W W$$

t-distributed Stochastic Neighbor Embedding: To optimize the distribution t-SNE uses Kullback-Leibler divergence between the conditional probabilities $p(A|B)$ and $q(A|B)$. This is used to measure the similarity between two probability distributions:

$$D_{KL} = \sum_i P(i) \log \frac{P(i)}{Q(i)}$$

Figure 7.10: t-SNE loss function

If $p = q$, then $\log(1) = 0$, so the cost function becomes zero as well.

Challenges and assumptions

All three dimensionality reduction techniques discussed above have different challenges and assumptions. These are listed as follows:

Key challenges and assumptions related to PCA are:

- Since PCs are linear combinations of different original variables, they are not interpretable as anything in the real world
- It is highly affected by outliers and works best with continuous data only

The LDA process has the following key points to be noted:

- LDA does not work well for imbalanced datasets
- It is not applicable to non-linear problems
- Independent variables are normal within each class
- LDA also assumes that the covariance matrices of the different classes are equal

For t-SNE, the following points need to be kept in mind:

- It is a great tool to visualize and understand high-dimensional datasets such as image processing but there

are better techniques for dimensionality reduction techniques such as PCA and LDA for ML problems such as credit risk modeling and other similar problems.

- Since it is not deterministic, it could produce a different result with every iteration.

Case study: Predicting loan repayment propensity using logistic regression, PCA, and LDA

Before we begin building models, let us understand the key functions and parameters concerning PCA and LDA

PCA parameters and interpretation

The `sklearn` library makes it very easy to execute PCA with just three lines of code. We will discuss PCA implementation with a detailed case study but before that, let us understand the `pca.explained_variance_ratio_` attribute of the PCA object we create.

The above attribute outputs the following:

```
[1.80110879e+01  9.78242428e+00  7.38067064e+00
 6.04597774e+00
 5.16263060e+00  4.84090609e+00  4.60317420e+00
 3.78591356e+00
 3.47169437e+00  3.06275128e+00  2.66727780e+00
 2.62747392e+00
 2.56102320e+00  2.46894912e+00  2.45651921e+00
 2.23976607e+00
 2.20175990e+00  2.07644203e+00  1.95523906e+00
 1.79780017e+00
 1.52578601e+00  1.42888114e+00  1.06242448e+00
 1.04234536e+00
 1.00588718e+00  9.77727265e-01  7.53990588e-01
 7.15560190e-01]
```


6.86317740e-01 5.15273208e-01 4.46335722e-01
2.33669530e-01

2.01724456e-01 7.75358030e-02 5.23929841e-02
3.63831499e-02

2.76907765e-02 8.40107091e-03 1.32582630e-03
8.66421704e-04

1.05353249e-31 1.05353249e-31]

The ratio of variance is simply eigenvalue / total eigenvalues for each of the PCs and intuitively adds up to 100. These can also be represented as a cumulative sum in a bar chart format ([Figure 7.4](#)). Please note that explained variance can also be used to compare different PCA models. For example, if we compare two models that both reduce a dataset from 100 dimensions to 2, but one explains 80% of the variance and the other explains 95% of the variance with 2 PCs, we would say that the latter model is better at representing the data.

LDA parameters and interpretation

LDA in Python is implemented using the `LinearDiscriminantAnalysis` class of `sklearn.discriminant_analysis` library. It is commonly used as a preprocessing step for classification tasks. An important part of LDA analysis using `sklearn` is specifying the `LDA(n_components=1)` parameter. This refers to the number of linear discriminants that we want to retrieve. Subsequently, we execute the fit and transform methods to retrieve the linear discriminants to be used for modeling purposes.

It is important to remember that `n_components` is equal to `n-1` (`n`=number of classes) and cannot be more than no of classes, since for instance there cannot be more than two hyperplanes separating three classes. The total variance of 1 gets distributed between the number of components wherein the first component explains the majority variation.

Logistic regression

The goal of this case study is to analyze a dataset consisting of Home Loan Information to comprehend the factors that influence the repayment of a loan and to predict the likelihood of Loan Default of a given individual. For this purpose, we will build a base logistic regression model with all variables followed by another logistic regression model that has dimensions reduced using a PCA exercise. We will finally apply the LDA dimensionality reduction techniques with a third logistics regression model and compare the results of the three models.

Please refer to the Under the Hood-Data section for details of the data used for this analysis. The event rate of this data after combining various `loan_status` under one target variable `Default` is as shown as follows:

Code:

```
#check distribution
```

```
loan_data.Default.value_counts(normalize=True)
```

Output:

```
0 0.9927
```

```
1 0.0073
```

```
Name: Default, dtype: float64
```

Clearly, this is a highly imbalanced dataset with less than 1% event rate or instances of default. There are six categorical features, which need to be converted to numerical with the following code:

```
#let's work on the categorical variable now
```

```
categorical_cols =
```

```
['state','homeownership','verified_income','loan_purpose','grade']
```

```
#Convert categorical to numeric
```

```
from sklearn.preprocessing import LabelEncoder
```

```
le = LabelEncoder()
```

```
le.fit(loan_data[categorical_cols].values.flatten())
```

```
le_df =
```

```
loan_data[categorical_cols].apply(le.fit_transform)
```

The data pre-processing also involves checking the distribution of numerical variables and fixing the outliers at the 99th - %ile and replacing any `na` with zeros. We also studied the presence of correlation among features to gain a pre-understanding of the impact of dimensionality reduction on such datasets. Intuitively, less correlation among the feature set will not have a significant impact on the PCA or LDA exercises. More details are available in the Python code.

Let us start building the three different logistic models we discussed. Base Logistic regression with all 42 variables.

Code:

```
# First separate train and test sets for base logistic regression
```

```
y = df_data['Default']
```

```
df_train, df_test =
```

```
train_test_split(df_data, test_size=0.3, random_state = 12, stratify = y)
```

```
X_train = df_train.drop(["Default"], axis=1)
```

```
X_test = df_test.drop(["Default"], axis=1)
```

```
y_train = df_train["Default"]
```

```
y_test = df_test["Default"]
```

```
LR = LogisticRegression(class_weight='balanced')
```

```
LR.fit(X_train, y_train)
```

```
# and create a function for getting all classification metrics
```

```
def get_all_measurements(y_test, y_pred, model_name):
```

```

    column_names = ['Accuracy', 'Precision', 'Recall',
                    'F1-Score', 'AUC']

    acc = accuracy_score(y_test, y_pred)
    cm = confusion_matrix(y_test, y_pred)
    tn, fp, fn, tp = cm.ravel()
    precision = tp / (tp + fp)
    fpr, tpr, thresholds = roc_curve(y_test, y_pred)
    Auc = auc(fpr,tpr)
    recall = tp / (tp + fn)
    f1 = (2 * precision * recall) / (precision +
    recall)

    values = np.array([[acc, precision, recall, f1,
    Auc]])

    summary = pd.DataFrame(values, index=[model_name],
    columns=column_names)

    return summary

```

The preceding code make us ready for result generation. Let us start by printing train data statistics.

Code:

```

#Predict on Train data
y_pred_train = LR.predict(X_train)
print(classification_report(y_train, y_pred_train))

LR_summary_train = get_all_measurements(y_train,
y_pred_train, 'Logistic Regression')

LR_summary_train

```

Output:

Training	Accuracy	Precision	Recall	F1-Score	AUC
Logistic Regression	0.846429	0.041219	0.901961	0.078835	0.873991

Figure 7.11: Training data performance statistics

The same model prediction on test data yields the following results:

Test	Accuracy	Precision	Recall	F1-Score	AUC
Logistic Regression	0.850333	0.042827	0.909091	0.0818	0.879495

Figure 7.12: Test data performance statistics

Since this is an imbalanced dataset, the best metric to evaluate performance would be AUC or F1-Score. We can observe that this model has a very low precision score leading to an overall low F1-Score across both test and training datasets. This simply means that the model is not able to predict the events (1) with a high degree of accuracy.

Let us see how the PCA intervention on this dataset impacts the performance of this model.

We start by transforming or scaling the feature set using the `StandardScaler` class.

Code:

```
#To apply PCA we must have a scaled data.

from sklearn.preprocessing import
PowerTransformer,StandardScaler

pt = StandardScaler()

loan_data_processed =
pd.DataFrame(pt.fit_transform(loan_data_transformed),
columns = loan_data_transformed.columns)

loan_data_processed.head()
```

Output:

	state	homeownership	verified_income	loan_purpose	grade	emp_length	annual_income	• • •	debt_to_income
0	0.509904	-0.980199	1.495421	1.865941	0.488565	-0.825666	0.246495		-0.086111
1	-0.780666	1.181449	-1.150406	-0.328509	0.488565	1.146709	-0.778705		-1.308467
2	1.664625	1.181449	0.172508	2.304831	1.358363	-0.825666	-0.778705	• • •	0.209818
3	0.985378	1.181449	-1.150406	-0.328509	-1.251030	-1.389202	-0.983745		-0.825933
4	-1.256139	1.181449	1.495421	-0.767399	0.488565	1.146709	-0.881225		3.678974

Figure 7.13: Scaled sample feature set

Next, we decompose the covariance matrix using `pca.fit_transform()` function.

Code:

```
from sklearn.decomposition import PCA, KernelPCA
pca = PCA()
loan_data_pca = pca.fit_transform(loan_data_processed)
var_ratio = (pca.explained_variance_ratio_)*100
cum_var_ratio = np.cumsum(var_ratio)
print('Cumulative sum is:\n',cum_var_ratio)
```

Output:

The cumulative sum is:

```
[ 18.01108785  27.79351213  35.17418277  41.22016051  46.3827911
 51.22369719  55.8268714   59.61278496  63.08447933  66.14723061
 68.81450842  71.44198234  74.00300554  76.47195466  78.92847387
 81.16823994  83.36999984  85.44644187  87.40168093  89.1994811
 90.72526711  92.15414824  93.21657273  94.25891809  95.26480527
 96.24253253  96.99652312  97.71208331  98.39840105  98.91367426
 99.36000998  99.59367951  99.79540397  99.87293977  99.92533275
 99.9617159   99.98940668  99.99780775  99.99913358 100.
100.         100. ]
```

Let us plot this cumulative variation data by PCs on a line plot with PCs or the feature count on the x-axis. This is also known as a scree plot and helps in determining the number of features/Principal components required to explain a certain amount of variation. The following figure shows that 25 features explain 95% of the variance:

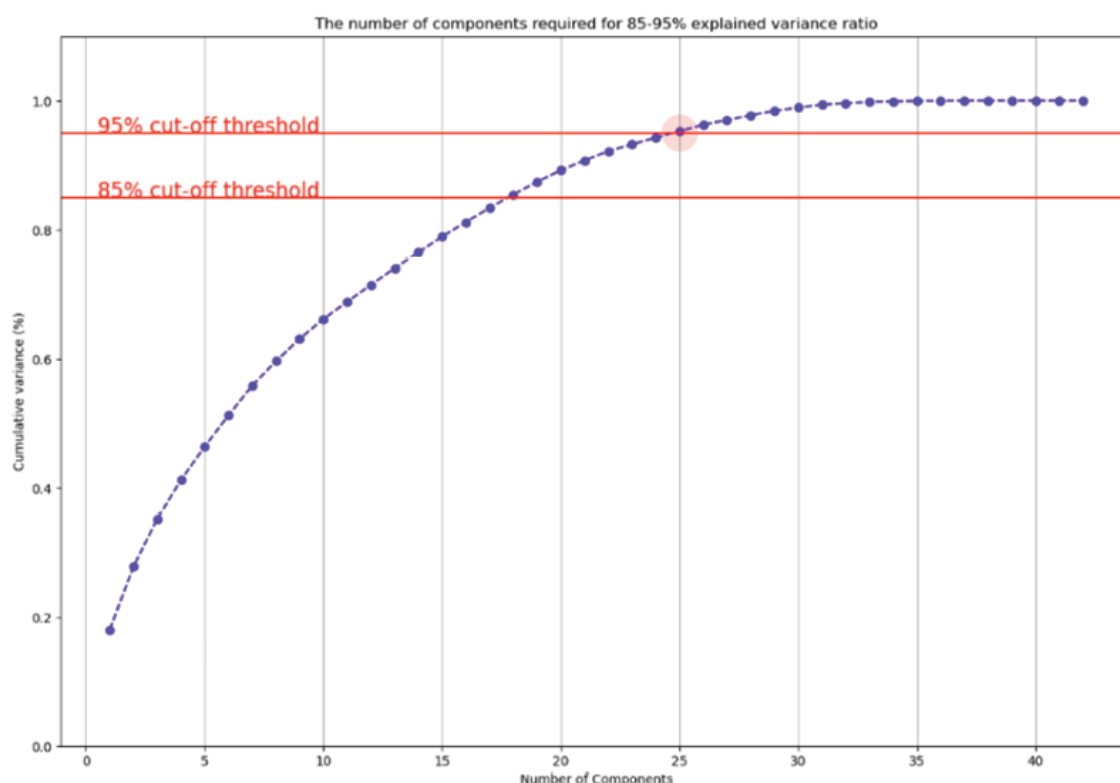


Figure 7.14: Scree plot with *StandardScaler* class

We can also transform the data using the `MinMaxScaler` class and see if that makes any difference to the number of components needed to explain the same amount of variation. The following figure shows the updated scree plot:

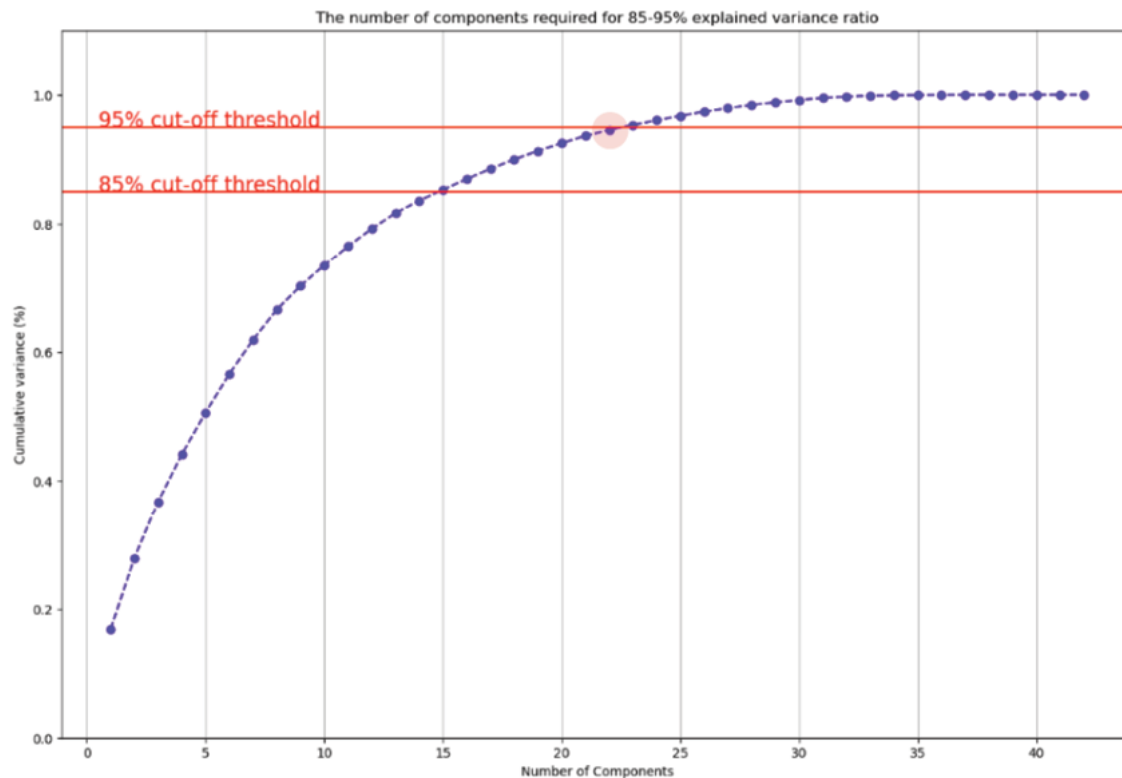


Figure 7.15: Scree plot with *MinMaxScaler* class

We can observe that `MinMaxScaler` explains the 95% variation with 23 variables as against the 25 variables using the `StandardScaler`. We will use this dataset to train the logistic model using PCA output.

Code:

```
pca = PCA(n_components= 0.95, random_state = 45)
df_pca = pca.fit_transform(loan_data_Scaled)
df_pca = pd.DataFrame(df_pca, columns= ['pc1', 'pc2',
'pc3', 'pc4', 'pc5', 'pc6', 'pc7', 'pc8', 'pc9', 'pc10',
'pc11', 'pc12',
'pc13',
'pc14', 'pc15', 'pc16', 'pc17', 'pc18', 'pc19',
'pc20', 'pc21', 'pc22', 'pc23'])
df_pca.head()
```


The preceding code sets the `n_components` at 0.95 on the `MinMaxScaled` dataset `loan_data_Scaled`.

Output:

	pc1	pc2	pc3	pc4	pc5	pc6	pc7	pc8	pc9	pc10
0	0.707084	-0.682472	-0.301294	0.419834	0.083308	0.148051	-0.351363	0.251378	0.065022	-0.063906
1	-0.577071	0.488306	0.631146	0.003473	-0.860847	0.329739	0.751965	-0.228296	-0.066213	-0.022441
2	-0.708961	0.140468	0.419231	0.635077	-0.043541	0.130947	-0.467036	-0.053654	0.425613	-0.249389
3	-0.895454	-0.333111	0.171328	-0.640076	0.293863	-0.173650	0.395818	0.408402	0.451298	0.225009
4	0.346165	0.116364	0.980576	-0.086226	0.289437	0.590221	-0.125587	-0.681600	-0.343371	-0.137375

Figure 7.16: Sample PCA transformed dataset

A quick correlation test of PCs on the `df_pca` dataset yields no output in terms of correlated features. Please see the following code:

```
cor = df_pca.corr()
cor = cor.abs().unstack() # absolute value of corr
coef
cor = cor.sort_values(ascending=False)
cor = cor[(cor >= 0.5) & (cor < 1)]
cor = pd.DataFrame(cor).reset_index()
cor.columns = ['feature1', 'feature2', 'corr']
cor
```

Output:

feature1	feature2	corr
----------	----------	------

This is because all PCs are supposed to be orthogonal. Executing the logistic model training on `df_pca` dataset yields following training statistics:

Code:

```
LR = LogisticRegression(class_weight='balanced')
```

```

LR.fit(X_train, y_train)
#Predict on Train data
y_pred_train = LR.predict(X_train)
print(classification_report(y_train, y_pred_train))
#Predict on Test data
y_pred_test = LR.predict(X_test)
print(classification_report(y_test, y_pred_test))

```

Output:

Training		precision	recall	f1-score	support
	0	1.00	0.74	0.85	6949
	1	0.02	0.80	0.04	51
	accuracy			0.74	7000
	macro avg	0.51	0.77	0.45	7000
	weighted avg	0.99	0.74	0.85	7000

Test		precision	recall	f1-score	support
	0	1.00	0.73	0.85	2978
	1	0.02	0.73	0.04	22
	accuracy			0.73	3000
	macro avg	0.51	0.73	0.44	3000
	weighted avg	0.99	0.73	0.84	3000

Figure 7.17: Train-test performance statistics comparison

We can observe that all metrics (F1-score, precision, and so on) have dropped in performance with a reduction of 5% variance overall.

The ROC curve for the above model can be obtained using the following code:

Code:

```
logit_roc_auc = roc_auc_score(y_test, y_pred_test)
fpr_lr, tpr_lr, thresholds = roc_curve(y_test,
y_pred_test)
plt.figure(figsize = (12,8))
plt.plot(fpr_lr, tpr_lr, linewidth=1, label='Logistic
Regression (area = %0.2f)' % logit_roc_auc)
plt.plot([0, 1], [0, 1], 'r--')
plt.xlim([0.0, 1.0])
plt.ylim([0.0, 1.05])
plt.title("ROC Curves")
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.legend(loc = 'lower right')
plt.show()
```

Output:

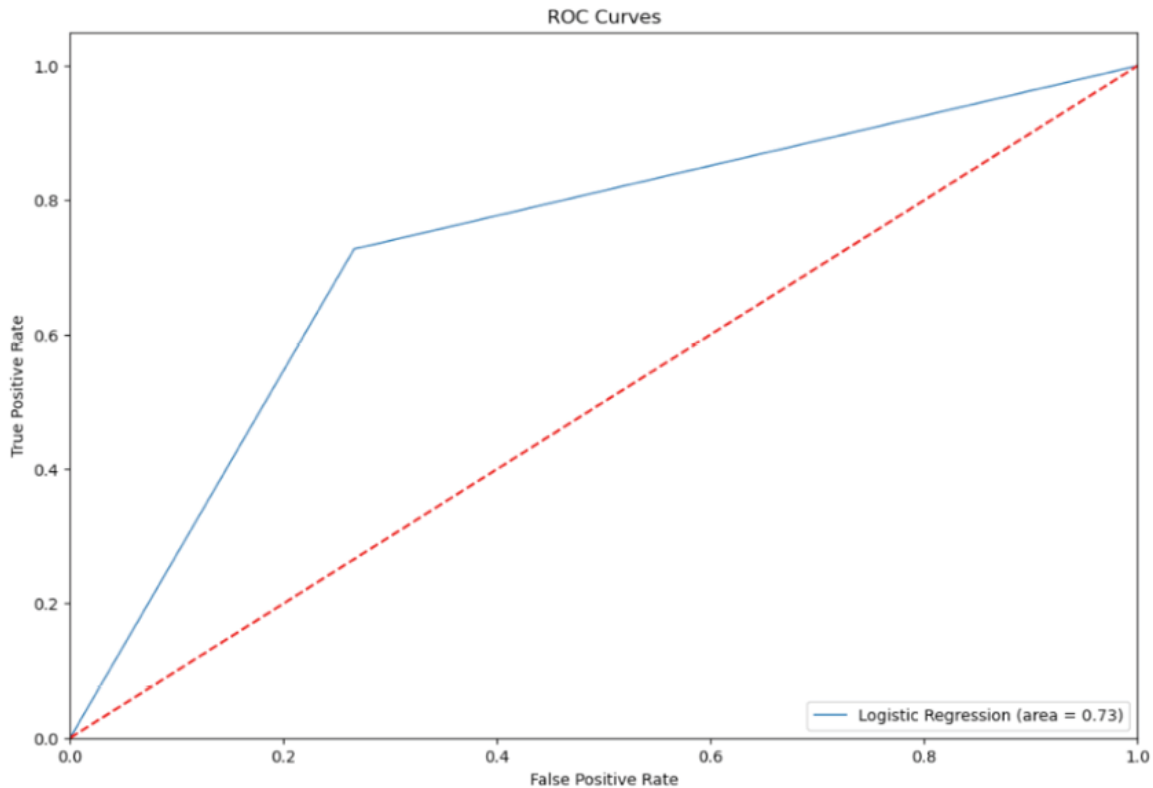


Figure 7.18: ROC curve for PCA treated logistic model

The **area under the curve (AUC)** is 0.73 which is much lower than the base logistic model AUC of 0.873991.

Lastly, let us finish this case study by building a logistic model with a 1-component LDA-treated dataset. Here, one component refers to the number of linear dimensions (discriminants) that we want to retrieve. We will use the same `MinMaxScaled` dataset for this exercise as well.

Code for LDA fit and transform:

```
sklearn_lda = LDA(n_components=1)
X_lda_sklearn =
sklearn_lda.fit(loan_data_Scaled,loan_data['Default'])
.transform(loan_data_Scaled)
X_lda_sklearn= -X_lda_sklearn
Print(X_lda_sklearn)
```

Output:

```
[[-0.23475288]
 [-0.46345305]
 [-1.50024756]
 ...
 [ 1.27491471]
 [-0.33795206]
 [ 0.14208209]]
```

Next, let us create a final dataset with just this component and the `Default` target variable for model training.

Code:

```
principalDf = pd.DataFrame(data = X_lda_sklern,
                           columns = ['PC1'])
# Adding labels
finalDf = pd.concat([principalDf,
                    loan_data['Default']], axis = 1)
finalDf.head()
```

Output:

	PC1	Default
0	-0.234753	0
1	-0.463453	0
2	-1.500248	0
3	0.258609	0
4	-0.204121	0

A one dimension projection of the dataset after LDA transformation looks like in the [Figure 7.19](#). The code for generating the same is as follows:

```
fig = plt.figure()
```

```

ax = fig.add_subplot(1, 1, 1)
ax.set_title('LDA')
ax.plot(X_lda_sklearn[0:20], np.zeros(20),
        linestyle='None', marker='o', color='blue',
        label='NSCLC')

ax.plot(X_lda_sklearn[20:40], np.zeros(20),
        linestyle='None', marker='o', color='red',
        label='SCLC')

fig.show()

```

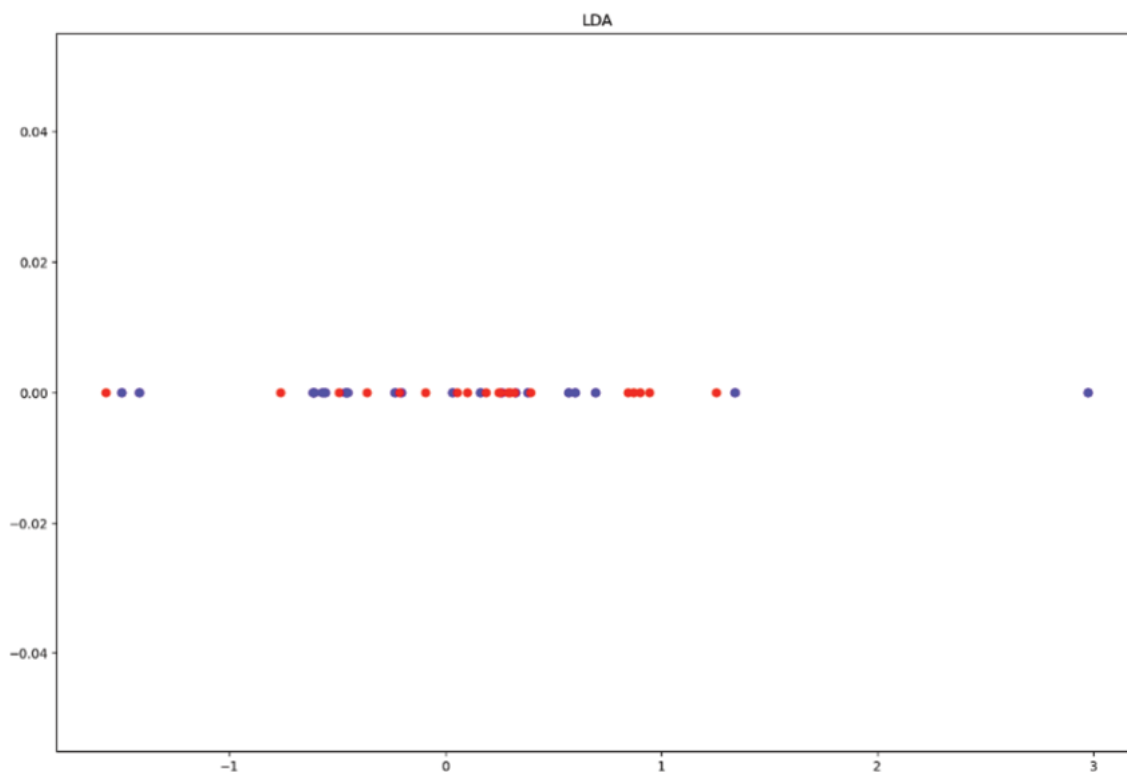


Figure 7.19: LDA projection with component = 1

Finally, let us execute the logistic model code with LDA transformed dataset and check the performance metrics.

Code:

```

LR = LogisticRegression(class_weight='balanced')
LR.fit(X_train, y_train)

```

```
#Predict on Train data
y_pred_train = LR.predict(X_train)
print(classification_report(y_train, y_pred_train))
#Predict on Test data
y_pred_test = LR.predict(X_test)
print(classification_report(y_test, y_pred_test))
```

Output:

Training		precision	recall	f1-score	support
	0	1.00	0.86	0.92	6949
	1	0.04	0.75	0.07	51
	accuracy			0.86	7000
	macro avg	0.52	0.80	0.50	7000
	weighted avg	0.99	0.86	0.92	7000

Test		precision	recall	f1-score	support
	0	1.00	0.86	0.92	2978
	1	0.04	0.86	0.08	22
	accuracy			0.86	3000
	macro avg	0.52	0.86	0.50	3000
	weighted avg	0.99	0.86	0.92	3000

Figure 7.20: Train-test performance statistics comparison

Interestingly, the logistic regression on the LDA-transformed dataset provides accuracy results comparable to the base logistic regression. This implies LDA-based model obtained similar performance with reduced model complexity in the case of this dataset.

We can conclude from this study that PCA and LDA have different goals and assumptions, and hence they produce different results depending on the data and the problem. LDA typically performs better than PCA in scenarios where the goal is supervised classification. PCA, being an unsupervised algorithm, is more

suitable for exploratory data analysis, where we want to discover the main patterns and sources of variation in the data. Since PCA does not use any class information, it may not capture the features that are relevant for discrimination and hence we saw a lower AUC with a PCA model in this case study. LDA is also a better bet in imbalanced dataset cases because it emphasizes the features that discriminate between classes while reducing the dimensional complexity of the dataset.

Conclusion

To conclude, data dimensionality reduction using PCA, LDA, and t-SNE has unlocked a treasure trove of insights within our high-dimensional datasets, enabling us to simplify the analysis, enhance visualization, and improve predictive model stability.

PCA, with its emphasis on capturing maximum variance, elegantly simplifies the representation of complex datasets, providing us with a clearer understanding of the underlying patterns in the feature set. Moving beyond PCA, and by incorporating class labels, LDA harnessed the discriminative power of the data, maximizing the separation between classes in the reduced-dimensional space. This has proven immensely useful in classification tasks, as LDA's focus on enhancing class separability has enhanced our ability to make accurate predictions and uncover valuable insights specific to different classes. This is all while reducing the dimensions.

Lastly, the exploration of t-SNE has opened new avenues in our quest for effective visualization and understanding of complex non-linear data structures. By capturing the local structure of the data and preserving pairwise similarities, t-SNE has enabled us to create compelling visualizations that reveal clusters, patterns, and outliers.

As we wrap up this chapter, we are reminded of the power these dimensionality reduction techniques hold in transforming our future data analysis and machine learning endeavors. We have witnessed how these three techniques complement one another where PCA excels in capturing global patterns, LDA thrives in

enhancing class separability, and t-SNE reveals local structures with unparalleled visual richness in non-linear problems. We will see an implementation of t-SNE in Chapter 10 on image processing with CNN.

As we venture forth in our data exploration and analysis journey, the understanding of dimensionality reduction using PCA, LDA, and t-SNE will undoubtedly continue to be a valuable asset. By harnessing the power of these techniques, we can unravel hidden patterns, gain deeper insights, and make more informed decisions in the ever-expanding landscape of data science and machine learning.

In the next chapter, readers will be exposed to another set of unsupervised learning techniques known as clustering that work on records in a dataset, unlike PCA, LDA, and t-SNE which work on the variables. These unsupervised learning techniques do not rely on labeled training data, which can be costly and time-consuming to obtain in many real-world scenarios. Please note that unsupervised techniques, such as clustering and dimensionality reduction, can be applied directly to unlabeled data, making them more accessible in some cases.

Further reading

- Dimensionality reduction methods:

<https://journals.plos.org/ploscompbiol/article/figure?id=10.1371/journal.pcbi.1006907.t001>

- Python/R libraries for various dimensionality reduction methods

<https://journals.plos.org/ploscompbiol/article/figure?id=10.1371/journal.pcbi.1006907.t002>

Points to remember

- PCA is an unsupervised technique and LDA is a supervised classification technique suitable for linearly separable classes.

- The choice of technique depends on the specific objectives and characteristics of the dataset at hand.
- Careful consideration of the data, the nature of the problem, and the intended use of the reduced-dimensional representation will guide us in selecting the most suitable technique.
- With uniformly distributed independent features, LDA almost always performs better than PCA.
- If the features are highly skewed, then PCA is a better choice since LDA can be biased toward the majority class.
- The transformed components in both PCA and LDA have little or no interpretability in explaining the outcome.

Join our book's Discord space

Join the book's Discord Workspace for Latest updates, Offers, Tech happenings around the world, New Release and Sessions with the Authors:

<https://discord.bpbonline.com>



CHAPTER 8

Beginning with Unsupervised Models

Introduction

The real voyage of discovery consists of not in seeking new landscapes, but in having new eyes.

– Marcel Proust

The art of unsupervised learning lies in finding harmony within the data, where seemingly chaotic patterns emerge as meaningful clusters. In this chapter, we discuss three different eye lenses or prominent unsupervised learning techniques - K-means clustering, Hierarchical clustering, and **Density-Based Spatial Clustering of Applications with Noise (DBSCAN)**. These powerful algorithms enable us to uncover hidden patterns, untangle intricate relationships, and think beyond the boundaries of pre-defined labels while creating labels and clustering data points within these labels.

Unsupervised learning sets itself apart from its supervised counterpart by its unique discovery approach without the guidance of target labels. The goal is to unearth structures and groups within the data, uncovering insights that might remain hidden under the veil of mystery. K-means,

Hierarchical clustering, and DBSCAN each possess their own unique strengths and attributes, bringing diversity to the world of unsupervised learning.

Chapter 7, Putting Dimensionality Reduction Into Action, PCA learnings were about finding patterns in the features and combining them in an unsupervised way. In this chapter, we will undertake a thrilling quest of exploration, peeking into the world of unsupervised learning from a data records perspective. The unsupervised algorithms of K-means, Hierarchical clustering, and DBSCAN empower us to see beyond the obvious, think beyond the known, and unravel the hidden patterns within our data. As we embrace the spirit of discovery in data mining, we will gain invaluable insights into the foundations and inner workings of these unsupervised learning techniques. So, let us embark on this captivating journey of unsupervised learning, where we discover the unseen and think the unthinkable.

Structure

In this chapter, we will learn the following topics:

- Unsupervised learning
- Background
- Unsupervised learning techniques
 - Data
 - Model: Building meaningful clusters and profiling them
 - K-means clustering
 - Density-based spatial clustering of applications with noise
 - Hierarchical clustering

- Loss: Efficiently achieving the optimal number of clusters
- Challenges and assumptions
- Case study: Bank customer portfolio segmentation
- Advanced unsupervised learning: A primer

Objectives

The objective of this chapter is to explore the popular unsupervised learning techniques such as K-means, Hierarchical clustering, and DBSCAN. The aim is to provide a comprehensive understanding of these techniques, their inner workings, and their practical applications through a real-world case study.

We will talk about the mathematical foundations of these techniques and will emphasize the practicality of these techniques through various case studies. This will be through a step-by-step walkthrough of a real-world dataset-based analysis aimed at exploring the various factors to consider while selecting between the three unsupervised learning techniques. By understanding the strengths and limitations of each technique, readers will be empowered to make informed decisions based on their specific requirements and data characteristics.

Furthermore, we will briefly discuss the evolution of unsupervised learning techniques in the neural network realm. These specific techniques such as **Autoencoders** and their variations, **Self-Organizing Maps (SOMs)** have proven to be powerful tools for discovering complex patterns and representations in various types of data including image, speech, and text.

By the end of this chapter, readers will have a comprehensive understanding of the underlying workings,

practical applications, and evaluation techniques of the three popular unsupervised techniques.

Unsupervised learning

The genesis of unsupervised learning can be traced back to the mid-20th century, with seminal works by early pioneers in machine learning and statistics. During this era, researchers began exploring the idea of allowing algorithms to learn from data without explicit target labels. The motivation behind unsupervised learning stemmed from the recognition that not all datasets are amenable to supervised techniques and that discovering inherent structures within data can be invaluable. Hence most unsupervised learning techniques focus on identifying certain traits of the underlying distribution. A related idea is to partition the data into clusters of points that are in some sense *similar* to each other.

Background

As discussed in [Chapter 1, Understanding Data Mining In A Nutshell](#), unsupervised techniques have been around for decades and have served as a secret lens for analysts to understand unknown data patterns. This said the applications and advancements of unsupervised learning techniques have garnered significant attention in recent years. Various studies have explored the potential of these techniques in exciting fields such as computer vision, natural language processing, bioinformatics, and anomaly detection. We have moved on from using these techniques for customer segmentation, surveys, and market research. Researchers have continuously strived to enhance the performance of these algorithms, tackle challenges related to scalability, and adapt them to deal with different types of data.

Additionally, the combination of unsupervised learning techniques with other machine learning paradigms, such as semi-supervised and transfer learning, has sparked considerable interest. The integration of deep learning architectures, such as autoencoders and **Generative Adversarial Networks (GANs)**, with unsupervised learning, has led to breakthroughs in generative modeling.

As we turn pages in this chapter, we will learn about the time-tested and most commonly used techniques of K-means, DBSCAN, and Hierarchical clustering, unraveling their strengths and weaknesses. By understanding these unsupervised learning techniques and examining their applications across domains, we open doors to new possibilities and embrace the art of discovering the abstract structures that lie within our data.

This is particularly useful when you are provided a dataset to analyze with no prior knowledge of the underlying patterns. Some questions that unsupervised learning answers by identifying the underlying distribution of features in the data and forming clusters are:

- The natural groupings or patterns in the data: Unsupervised learning techniques like clustering can identify natural groupings or clusters of data points that share similar characteristics. By examining these clusters, we can gain insights into the inherent structure of the data and discover patterns that may not be evident at first glance.
- Can we segment customers or users? Clustering can be applied to customer or user data to segment them into distinct groups based on their behavior, preferences, or characteristics. This segmentation can help businesses personalize marketing strategies, improve customer experience, and identify high-value segments.

- Can we identify subpopulations? Clustering can help identify subpopulations within a larger dataset. For instance, in Financial Services, unsupervised learning can be used to find distinct subgroups of customers with different risk profiles or customer service responses.
- Are there anomalies or outliers in the data? Clustering can also help detect outliers or anomalies in the dataset. These are data points that deviate significantly from the majority of the data and may indicate errors, rare events, or potential problems in the data.
- How can we impute missing values? By understanding the underlying distribution of features, unsupervised learning techniques can help impute missing values in the dataset. Clustering methods can be used to estimate missing values based on the characteristics of other data points in the same cluster.
- Can we simplify image or text data? Unsupervised learning techniques, such as autoencoders and word embeddings, can be used to simplify and represent complex image or text data in a lower-dimensional space, enabling more efficient processing and analysis. We will discuss these advanced concepts in [Chapter 10, Language Modeling with Recurrent Neural Networks](#).
- How can the data be preprocessed or reduced? Unsupervised learning methods like dimensionality reduction (for example, PCA, t-SNE) can be used to preprocess the data and reduce its dimensionality while preserving essential information. We have seen the benefits of these techniques in [Chapter 7, Putting Dimensionality Reduction Into Action](#), already.

Please note that as we talk about unsupervised techniques, you would notice the use of terms such as clustering,

segmentation, and groupings interchangeably. These are the outcomes of unsupervised learning techniques. With this background, let us explore the algorithmic side of the three key unsupervised algorithms such as K-means, hierarchical clustering, and DBSCAN. This theoretical understanding will form the base for a case study.

Unsupervised learning techniques

In this section, we will further investigate the differences between the three techniques. We will do this by discussing the holy trinity of data, model, and loss function that defines any data mining solution. Let us start by understanding the input data first.

Data

The German credit dataset contains 1000 records with 20 categorical attributes prepared by Prof. Hofmann. In this dataset, each record represents a person who has taken credit from the bank. Each person is classified as having good or bad credit risks according to the set of attributes. This cleaner and leaner dataset is sourced from Kaggle for learning purposes and is available in a readable CSV file with 1000 records and 9 attributes. The selected attributes are shown in [Figure 8.1](#):

	Age	Sex	Job	Housing	Saving accounts	Checking account	Credit amount	Duration	Purpose
0	67	male	2	own	NaN	little	1169	6	radio/TV
1	22	female	2	own	little	moderate	5951	48	radio/TV
2	49	male	1	own	little	NaN	2096	12	education
3	45	male	2	free	little	little	7882	42	furniture/equipment
4	53	male	2	free	little	little	4870	24	car

Figure 8.1: German dataset snapshot

The link to the original dataset can be found as follows:

<http://archive.ics.uci.edu/dataset/144/statlog+german+credit+data>

For clustering purposes, we need to do a couple of things to transform the data:

- Categorical features transformed into numerical
- Standardization of all features to bring them on the same scale

Another interesting statistical check on the dataset that helps in assessing the clustering tendency of the dataset is called the *Hopkins test*. This is done by measuring the probability that a given data set is generated by a uniform data distribution. In other words, it tests the spatial randomness of the data.

The null and the alternative hypotheses for this statistic are defined as follows:

- **H₀**: the data set D is uniformly distributed (i.e., no meaningful clusters)
- **H_a**: the data set D is not uniformly distributed (i.e., contains meaningful clusters)

A value of $H > 0.75$ indicates a clustering tendency at the 90% confidence level.

The following Python code calculates the Hopkins statistics for this dataset after the above-mentioned data preprocessing steps are complete.

Python code:

```
# Calculating Hopkins score to know whether the data is good for clustering or not.
```

```
def hopkins(X):
```

```
    d = X.shape[1]
```

```

n = len(X)
m = int(0.1 * n)
nbrs =
NearestNeighbors(n_neighbors=1).fit(X.values)
rand_X = sample(range(0, n, 1), m)

ujd = []
wjd = []
for j in range(0, m):
    u_dist, _ =
nbrs.kneighbors(uniform(np.amin(X,axis=0),np.amax(X
,axis=0),d).reshape(1, -1), 2,
return_distance=True)
    ujd.append(u_dist[0][1])
    w_dist, _ =
nbrs.kneighbors(X.iloc[rand_X[j]].values.reshape(1,
-1), 2, return_distance=True)
    wjd.append(w_dist[0][1])

HS = sum(ujd) / (sum(ujd) + sum(wjd))
if isnan(HS):
    print(ujd, wjd)
    HS = 0
return HS
hopkins(data_scaled)

```

Output:

0.7450470272078146

We can observe that this dataset is not the best suitable for clustering given a close to 0.75 Hopkin score result.

However, we will proceed with clustering and see what the algorithms showcase. At the same time, the important point to keep in mind here is that sometimes clustering algorithms impose a grouping on the random uniformly distributed data set even if there are no meaningful clusters present in it. So, the ultimate goal should be to see if there are non-overlapping dense clusters in the data or not. Please note that the Hopkins score is not a very standard practice in the industry since practitioners believe it is better to run the clustering algorithms directly, visualize, and infer from the clustering results instead of ending the clustering exercise based on pre-clustering statistics such as the Hopkin score.

So, after finding out that a dataset is clusterable, the next step is to determine the number of optimal clusters in the data because some algorithms need that information as input. We will learn that in the next section on building unsupervised models.

Model: Building meaningful clusters and profiling them

Unsupervised learning happens when an algorithm learns by observing patterns. There are no labels for the algorithm to associate a feature with and learn. For example, a toddler may learn by observing that there are a set of creatures with whiskers that bark and other similar creatures with whiskers just meow. As they grow in observation skills, they realize more differences and learn to classify these creatures in two different buckets in their head. Eventually, they learn from their parents that the one bucket with barking creatures is labeled dogs, and the other bucket with meowing creatures

is called cats. Once the labels are well established in the child's brain, the subsequent process of identifying an animal as a cat or a dog is supervised. The unsupervised modeling is similar wherein once the clusters are formed, the practitioner simply profiles the clusters and tries to assign a *easy-to-remember* label for each cluster. Cluster profiling helps with setting the boundaries of each cluster. For instance, the age range within a specific cluster is the age profile of that cluster.

Unsupervised learning is classified into two categories of algorithms:

- **Clustering:** The goal is to discover the underlying groups in the data such as identifying similar customers based on online activity behavior
- **Association:** This algorithm aims to find rules that associate an event to another event in the dataset. A common example is market basket analysis where the sale of item A is highly correlated with the sale of item B.

In this chapter, we will focus primarily on the clustering algorithms. Let us start with the most commonly used K-mean algorithm.

K-means clustering

It is one of the most widely used unsupervised learning techniques, characterized by its simplicity and efficiency. Introduced in 1957 by Stuart Lloyd, K-means aims to partition data points into K distinct clusters, where K represents the pre-defined number of clusters. The value of K is obtained through analysis and business intuition. Once K is assigned, the algorithm iteratively assigns each data point to the cluster whose centroid is closest, updating the centroids until convergence. The final result is a set of clusters, each characterized by its centroid, which serves as

the representative point for the cluster. K-means has been successfully applied in diverse fields, such as image segmentation, customer segmentation, and anomaly detection.

Let us understand the algorithm in slightly more detail using a simplified version of the German dataset. As a bank manager, you would like to better understand your customers to reduce the risk of default. You have some data that contains their age and credit amount, but it is difficult to segment your customers manually given the sheer size of the data. Let us see how we can cluster the customer using the two variables age and credit amount. Remember that it is an iterative process of assigning each data point to the group centroid with the objective of minimizing the sum of distances between the data points and the cluster centroid.

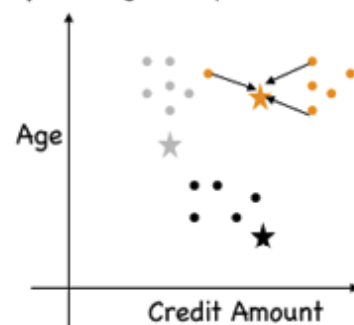
Figure 8.2 illustrates the five-step K-means process:

Step 1: Choose value of "K" – Number of clusters"

Step 2: Initialize K=3 centroids denoted by ★ ★ ★



Step 3: Assign data points to the nearest cluster centroid



Step 4: Re-initialize the centroids



Step 5: Repeat 3 & 4, If needed

Figure 8.2: K-means four-step process

Five steps in [Figure 8.2](#) are detailed below for further clarity:

1. Choose K, which is the number of clusters. In the illustrated example, the value of K is chosen as three.
2. We start by selecting K random data points and defining them as centroids for each cluster. Initially, these centroids are assumed to be the center of a

hypothetical cluster and are represented by the three stars in step two of [Figure 8.2](#).

3. Data points are assigned to the nearest (based on Euclidean distance) cluster centroid. Black-colored points belong to the black centroid being closest to it. This logic is applied to other centroids as well.
4. Next, the centroids are re-initialized by calculating the average of all data points of that cluster. This leads data points to be assigned to the nearest cluster centroid again.
5. Repeat step 3 and 4 until convergence happens or there is no change in the centroid position.

Remember K-means is affected by outliers since that can alter the position of the centroid. We need to be cautious when analyzing K-means results if outliers are included in the input dataset. Another drawback of K-means is that it performs a hard assignment of data points to a cluster implying close to 100% certainty. However, in some cases, it might be more appropriate to express uncertainty and model the possibility of a data point belonging to multiple clusters. Additionally, K-means assumes that clusters are spherical and equally sized. In real-world scenarios, clusters may have different shapes, sizes, and orientations. This assumption can limit the effectiveness of k-means when dealing with non-spherical or elongated clusters.

This takes us to our next algorithm **Gaussian Mixture Model (GMM)**. GMM is a probabilistic model used for clustering and density estimation. It assumes that all the data points come from finite distributions, each representing a distinct cluster. Eventually, the goal of the algorithm is to identify the parameters of the Gaussian distribution such as the mean and variance to identify the underlying clusters and the corresponding probabilities of the data points. In this process, GMM performs soft-clustering, unlike K-means, by

assigning each data point with a probability of belonging to all possible clusters, also known as Gaussian distributions. Interestingly, while K-means uses just the mean to update the centroid, GMM takes into account both the mean and variance. Some other key differences between K-means and GMM are:

Dimension	K-means	GMM
Cluster size	Assumes clusters to be of roughly equal size	Since it uses weighted Gaussian distributions, it can handle clusters of different sizes
Cluster shape	Assumes clusters to be spherical, and isotropic. It may struggle with elongated or irregularly shaped clusters.	Can model clusters with different shapes and sizes, as it allows for covariance between variables.
Use case	It is best suited for large-scale, well-separated clusters, where interpretability and simplicity are more important.	It is useful when clusters have different shapes, sizes, or orientations, and when uncertainty in cluster assignment is important. This means when the data has multiple peaks in the distribution or there is no apriori knowledge about the number of clusters.

Table 8.1: Key differences between K-means and GMM

We will end our discussion on the probability density-based GMM at this point in this book, but we will explore another interesting, conceptually similar, and density-based algorithm called DBSCAN in more detail.

Density-based spatial clustering of applications with noise

It was proposed by *Martin Ester et al.* in 1996 and presents another innovative approach to unsupervised clustering.

Density-based spatial clustering of applications with noise (DBSCAN) identifies clusters based on the density of data points, as opposed to relying on pre-defined cluster

counts (K) and centers in K-means. On the other hand, the DBSCAN algorithm categorizes data points as core, border, or noise based on their proximity to densely populated regions. It needs just two parameters, min points and epsilon (distance measure) to define these three points. The DBSCAN shines in its ability to detect clusters of varying shapes and sizes, making it particularly effective for datasets with irregular and non-linear structures. DBSCAN has found applications in various fields, including spatial data analysis, outlier detection, and image processing.

The three-step DBSCAN process can be visualized using the [Figure 8.3](#):

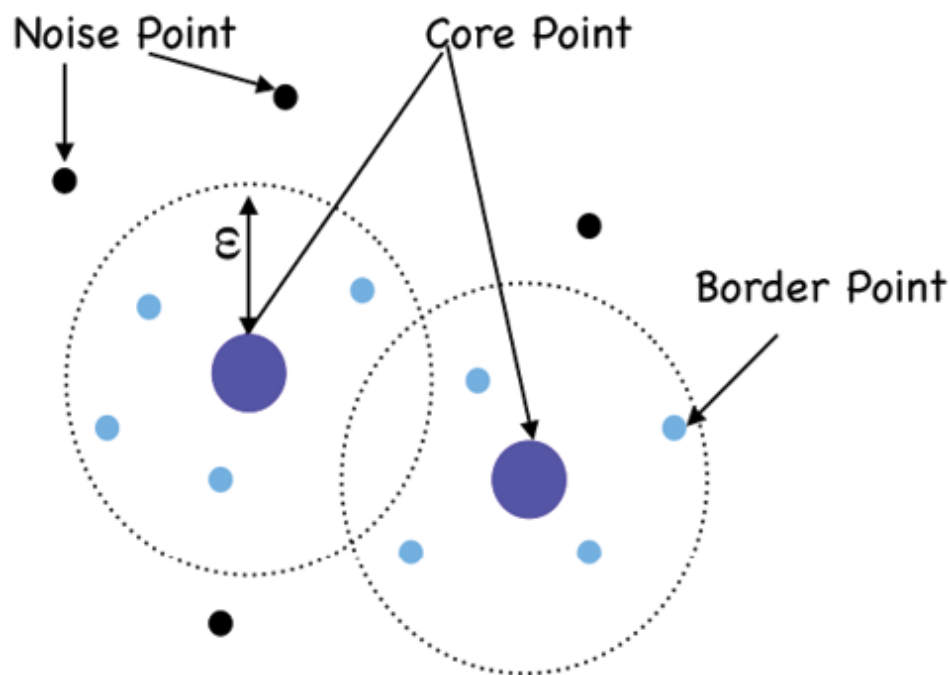


Figure 8.3: DBSCAN Four-Step Process

1. Starting with step 1, the algorithm arbitrarily picks up a core point in the dataset. See the big blue dot in [Figure 8.3](#).
2. The algorithm defines a set of points to be a cluster if there are at least `minPoint` points with a radius of ϵ . In

Figure 8.3 we see four `minPoint` small dots near the big dot that are within the radius ϵ .

3. The algorithm expands the clusters by applying the step 2 logic to each point until all points have been visited.

Please note the following for above three steps:

- **minPoints**(n) = 4 (Blue points); The minimum number of points clustered together for a region to be considered dense
- **Epsilon** (ϵ): A distance measure or the radius of the circle that will be used to locate the points that will be considered cluster

Additionally, while core and border points form a cluster based on the above two parameters, some points are neither core nor border and end up getting marked as 'noise' by the algorithm; For instance, these points will never have 4 points at a distance ϵ from them in the dataset

It is also worth noticing that, unlike K-means, the DBSCAN algorithm does not necessitate every point to be part of at least one cluster. If a point is never close enough to a certain minimum number of points, it is marked noise implying it does not fall in a dense region. Very intuitive right? Let us discuss the last algorithm in focus in this chapter.

Hierarchical clustering

It dates back to the 1960s and takes a different approach by organizing data points in a tree-like structure, known as a dendrogram. The algorithm recursively merges similar data points into clusters, forming a hierarchy of nested clusters. Agglomerative and divisive are two main types of hierarchical clustering algorithms, each with its strengths and weaknesses. Divisive clustering, which is a top-down approach, may appear computationally less expensive and

efficient since it starts with an initial single cluster and recursively divides it. In this process, it only calculates the distance between sub-clusters but is more sensitive to the choice of the initial clusters. However, since it generally produces big clusters that are hard to interpret and the algorithm has no `skit-learn` implementation, we will focus our discussion on agglomerative clustering in this chapter.

[Figure 8.4](#) shows the recursive process for agglomerative clustering:

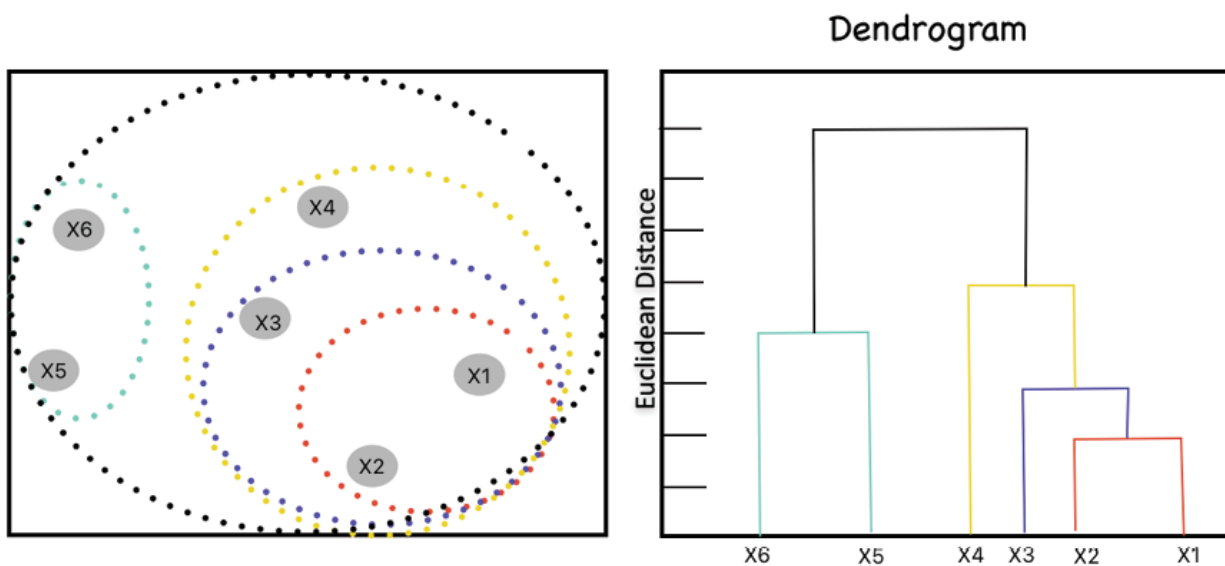


Figure 8.4: Hierarchical clustering four-step process with dendrogram formation

The six points in the left square box in [Figure 8.4](#) are represented as the dendrogram on the right box in four steps as shown:

1. Each of the six data points is considered a single-point cluster - a total of 6 clusters to begin with
2. The two closest data points are taken and combined into one cluster - combine X1 and X2 -> red circle cluster

3. Again, take the two closest clusters and make them one cluster - combine the red cluster from Step 2 with data point X3 to form the blue cluster
4. Repeat step 3 until there is only one cluster left -> outermost black circle

An important property of a dendrogram is that the height of the tree branch represents the distance between clusters. The distance is simply the Euclidean distance between the two points getting clustered together. For instance, the highlighted red branch has a distance of 2 between the two clusters. Notably, both agglomerative and divisive hierarchical clustering need the desired number of clusters specified upfront for the algorithm to terminate.

In summary, Hierarchical agglomerative clustering provides a powerful visualization of the data's hierarchical relationships, enabling us to explore both individual clusters and the broader connections between them. We will understand agglomerative hierarchical clustering using a case study later in this chapter. This methodology has been extensively used in biology, social sciences, and marketing research and has been widely used in the industry when data is structured enough.

To close the discussion on various clustering methodologies, it is important to note the following:

- K-means, agglomerative clustering, and DBSCAN represent three distinct clustering methods with varying approaches and use cases. K-means is a centroid-based algorithm, agglomerative clustering is a hierarchical, bottom-up approach, and DBSCAN, on the other hand, is a density-based algorithm
- Each method caters to specific data characteristics and analytical goals, providing flexibility for a wide range of clustering scenarios. For instance, DBSCAN is effective

in detecting clusters of arbitrary shapes and handling noise, making it suitable for datasets with irregular structures such as text or unstructured data

Loss: Efficiently achieving the optimal number of clusters

Unsupervised learning methods such as K-means, DBSCAN, and hierarchical clustering do not have global loss functions that can be directly minimized because these methods do not have explicit target labels to guide the learning process. Instead, they often involve iterative or hierarchical processes that aim to optimize specific criteria or objectives related to the structure and compactness of the resulting clusters.

In K-means clustering, the objective is to minimize the sum of squared distances between data points and their corresponding cluster centroids. The loss function, often referred to as the *within-cluster sum of squares* or *inertia*, quantifies the compactness of the clusters. A good model has low inertia and a low number of clusters (K). However, this is a tradeoff because as K increases, inertia decreases and vice versa. The elbow method is used to find the optimal value of K for a dataset. This involves plotting the inertia and K and finding the point where the decrease in inertia begins to slow. [Figure 8.5](#) shows K=3 as the *elbow* of the graph, though not very clearly:

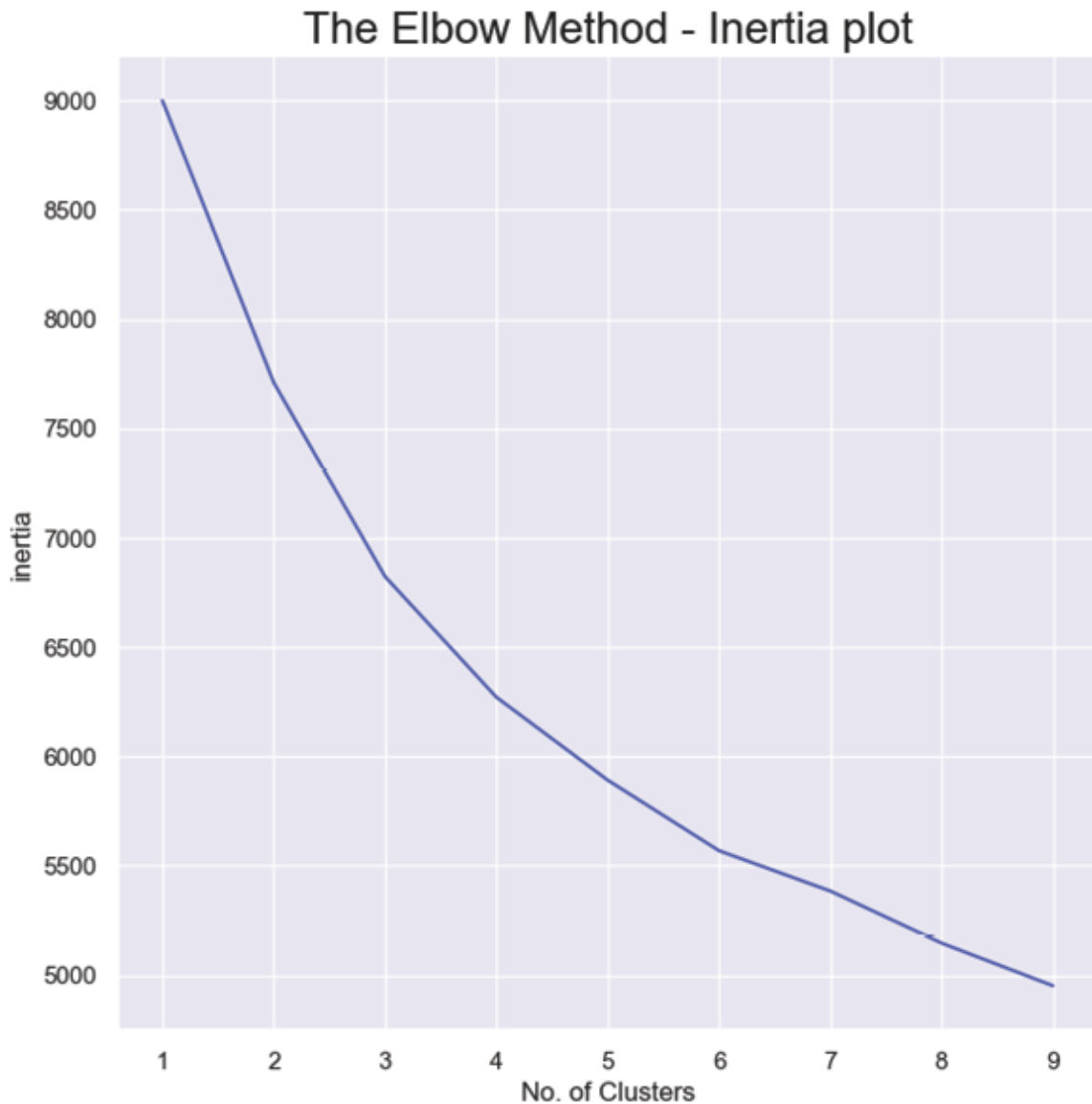


Figure 8.5: *Inertia plot: elbow method*

Following is the code to generate the previous elbow curve:

```
#Elbow Method - Inertia plot
inertia = []
#looping the inertia calculation for each k
for k in range(1, 10):
    #Assign KMeans as cluster_model
```

```

    cluster_model = KMeans(n_clusters = k,
random_state = 24)

    #Fit cluster_model to X
    cluster_model.fit(data_scaled)

    #Get the inertia value
    inertia_value = cluster_model.inertia_

    #Append the inertia_value to inertia list
    inertia.append(inertia_value)

##Inertia plot
plt.plot(range(1, 10), inertia)
plt.title('The Elbow Method - Inertia plot',
fontsize = 20)
plt.xlabel('No. of Clusters')
plt.ylabel('inertia')
plt.show()

```

The optimization process iteratively updates the cluster centroids to minimize the inertia until convergence. We can observe in [Figure 8.5](#) that raising the number of clusters beyond 3 leads to a reduction in inertia at a slower pace. It can be seen that the drop from inertia 9000 to 7000 happens within 3 clusters but the drop from 7000 to 5000 takes 6 (3-9) clusters suggesting there is a slight kink at '3' clusters. Hence we take $k=3$ using this graph. One may argue for two clusters as well with the same logic, but for now, we will experiment with $k=3$.

Another method to determine the optimal number of clusters is Silhouette Score. This measures the appropriateness of a data point's assignment to a cluster that is cluster cohesion

and its distance from the neighboring clusters. It ranges from -1 to 1, where a score closer to 1 indicates that the data point is well-clustered, and a score closer to -1 indicates that the data point is misclassified and should belong to a neighboring cluster. A score of 0 indicates that the data point is on the boundary between two clusters.

The key point to note is that the score is calculated for each data point in a dataset and then averaged to get the mean silhouette score for the entire dataset. The following code outputs a set of silhouette scores for a bunch of K values:

Code:

```
# compute Silhouette score
from sklearn.metrics import silhouette_score
sil=[]
cl=[2,3,4,5,6,7,8,9,10,11,12]
for k in cl:
    mod=KMeans(k)
    mod.fit(data_scaled)
    score=silhouette_score(data_scaled,mod.labels_)
    print(score)
    sil.append(score)
```

Output:

0.17822990851354975

0.15307287155863025

0.14704204238290128

0.13281843929670942

0.1269931577194742

0.14147382304220238

0.12974689483370208

0.13191661018205086

0.12934838014010022

0.14253763851130147

0.13731999704439057

The higher the mean silhouette score for a dataset, the better the clustering. For this dataset, we can observe that both the inertia plot and silhouette scores do not indicate a clear indication of non-overlapping clusters. The highest silhouette score of 0.178 is obtained for having two clusters and inertia plot also does not show a clear kink in the graph to determine a fixed number of non-overlapping clusters. The Hopkins test statistics for this dataset is also 0.74 which is high but also doesn't suggest clear cluster tendency (~ 1).

Intuitively in such scenarios, the best path forward is to start small with a business-aligned number of clusters and iterate based on the cluster profiles that are obtained. We will understand this part a little better in the upcoming case study.

Let us now look at DBSCAN which does not have a traditional loss function in the same sense as K-means. Instead, it defines clusters based on the density of data points. The key parameters in DBSCAN are the minimum number of points (**minPts**) required to form a dense region and the maximum distance (**eps**) that defines the neighborhood of a data point. The algorithm aims to identify dense regions with a sufficient number of points and separate outliers as noise. DBSCAN optimizes the clustering process by grouping data points into clusters that meet the density and distance criteria.

Lastly, Hierarchical clustering also does not involve a specific loss function like K-means or DBSCAN. Instead, it creates a dendrogram that represents the hierarchical relationships between clusters. The loss optimization in hierarchical clustering occurs during the agglomerative or divisive process, where data points or clusters are merged or split to create the hierarchical tree. The algorithm aims to find the optimal way to form clusters that minimize the overall dissimilarity or distance between data points while preserving the hierarchical structure.

Evaluating the quality of the clustering results can be challenging, and various validation metrics, such as silhouette scores, or manual cluster profiling and validation techniques, are used to assess the quality of the clustering and guide the optimization process. Please note that the unsupervised methods do not result in trained models as the supervised algorithms. Unsupervised methods are also rarely used for future prediction in the way supervised methods are used. They are impactful as data pattern analysis tools that guide or support business intuitions.

Challenges and assumptions

All three unsupervised techniques discussed above have different challenges and assumptions, these are listed as follows:

Key challenges and assumptions related to K-means are:

- **Sensitive to initialization:** K-means is sensitive to the initial choice of centroids, which can result in different clustering outcomes.
- **Fixed number of clusters:** It requires pre-defining the number of clusters (K) in advance, which may not always be known beforehand.

- **Sensitive to outliers:** Outliers can significantly affect the cluster centroids and lead to suboptimal results.
- **Cluster shape:** K-means assumes that the clusters are spherical and have similar sizes. This means that it works best when dealing with well-separated and roughly equally-sized clusters.

The DBSCAN process has the following key points to be noted:

- **Scalability:** DBSCAN's computational complexity can be high for large datasets, especially in high-dimensional spaces.
- **Difficulties with varying density:** It may not perform well when dealing with clusters of varying densities.
- **Outlier-friendly:** This algorithm works well with outliers
- **Dense clusters:** DBSCAN assumes that clusters are dense regions in the data space, separated by regions of lower density. Thus, it can handle clusters of arbitrary shape and size.
- **Cluster integrity:** It also assumes that there are noise points in the data. Points which do not belong to any cluster and are considered outliers.

For hierarchical clustering, the following points need to be kept in mind:

- **High computational complexity:** Hierarchical clustering can be computationally expensive, especially for large datasets.
- **Memory usage:** It may require significant memory resources to store the dendrogram, especially for large datasets.

- **Outlier Sensitive:** This algorithm is sensitive to noise and outliers.
- **Unique clusters:** It creates non-overlapping clusters, where each data point belongs to only one cluster. This may not be suitable for data with inherent overlapping patterns.
- **Distance dependent:** The algorithm also assumes the availability of a distance metric to measure the dissimilarity between data points. The choice of distance metric can impact the clustering results.

Case study: Bank customer portfolio segmentation

As part of the case study, we are going to use German bank data to segment bank customers into different groups and study the group characteristics.

Let us start exploring the K-means algorithm. As discussed earlier, we will be using a K value of 3 that was derived by observing the outputs of the Inertia plot and silhouette scores. The K-means code is shown below:

```
k = 3

kmeans = KMeans(n_clusters=k,
random_state=0).fit(data_scaled)

data['Cluster'] = kmeans.labels_

data['Cluster'] =
data['Cluster'].astype('category')

data
```

We add a new column `cluster` in the dataset which serves as the label of each record based on the identified cluster. The updated dataset is shown in [Figure 8.6](#):

	Age	Sex	Job	Housing	Saving accounts	Checking account	Credit amount	Duration	Purpose	Cluster
0	67	1	2	1	4	0	1169	6	5	1
1	22	0	2	1	0	1	5951	48	5	0
2	49	1	1	1	0	3	2096	12	3	1
3	45	1	2	0	0	0	7882	42	4	0
4	53	1	2	0	0	0	4870	24	1	0
...	...	...	...	...	...	...	...	...	...	...
995	31	0	1	1	0	3	1736	12	4	2
996	40	1	3	1	0	0	3857	30	1	0
997	38	1	2	1	0	3	804	12	5	1
998	23	1	2	0	0	0	1845	45	5	0
999	27	1	2	1	1	1	4576	45	1	0

1000 rows x 10 columns

Figure 8.6: Clustered dataset with 'Cluster' label

The clusters can be seen in two-dimension by plotting any two of the features, for instance, **Age** and **Credit Amount** along with the new label **cluster**. The code for the same is shown as follows:

```
#visualize the cluster result
sns.scatterplot(x="Age", y="Credit
amount",data=data, hue="Cluster",palette=
'rainbow')
```

Output:



Figure 8.7: Visualize 'Cluster' results

A simple `describe()` function can help us understand the profile of each of the created clusters. The cluster profile helps in studying the range of each feature value within each cluster. The following code outputs the cluster profiles:

```
data.groupby('Cluster').describe()
```

Output:

Cluster	Credit amount								Duration							
	count	mean	std	min	25%	50%	75%	max	count	mean	std	min	25%	50%	75%	max
0	204.0000	7392.2500	3277.4527	1845.0000	4917.2500	6917.5000	9080.5000	18424.0000	204.0000	37.3382	1.9539	6.0000	30.0000	36.0000	48.0000	72.0000
1	519.0000	2222.9595	1276.5322	276.0000	1298.0000	1938.0000	2882.0000	7228.0000	519.0000	16.5299	7.5750	4.0000	12.0000	15.0000	24.0000	45.0000
2	277.0000	2200.4440	1453.1358	250.0000	1216.0000	1778.0000	2969.0000	8471.0000	277.0000	16.9928	7.9112	4.0000	12.0000	15.0000	24.0000	48.0000

Figure 8.8: Cluster profile

It can be observed from [Figure 8.8](#) that Cluster 2 has 277 records (28 % population) with the least average credit amount and comparable credit duration in months with Cluster 1. Cluster 1, on the other hand, is the biggest group with 52 % population with a slightly higher average credit amount, but slightly lower credit duration. Looking at these average numbers by features and the visual spread of the data points in [Figure 8.7](#), we can conclude that clusters 1 and 2 are overlapping and no real purpose is served by keeping them separate.

Only Cluster 0 (blue point in [Figure 8.7](#)) which has 20 % of the population has a distinctly high credit amount and credit duration. As seen in [Figure 8.7](#), this can be driven by a lot of outliers in the data. This is the first pass at the K-means clustering but as a reader, you should re-run the code with two changes:

- Since clusters 1 and 2 are not really that different, re-run the code with K=2
- Removing outliers is not a solution here since they are actual customers even though they form a less dense group. However, feel free to experiment with K-means after removing outliers in some of the key features such as credit amount, credit duration and so on.

Now having observed the strengths and limitations of K-means, on this dataset, let us execute DBSCAN and compare the clustering outcomes with K-means.

Code:


```

from sklearn.cluster import DBSCAN

dbscan = DBSCAN(eps=2,
min_samples=5).fit(data_scaled)

labels = dbscan.labels_

data['dbscan']=labels

data.head()

# play with the eps and min_samples. The default
for eps is 0.5, it's like the radius distance

```

Output:

	Age	Sex	Job	Housing	Saving accounts	Checking account	Credit amount	Duration	Purpose	Cluster	dbscan
0	67	1	2	1	4	0	1169	6	5	1	1
1	22	0	2	1	0	1	5951	48	5	0	0
2	49	1	1	1	0	3	2096	12	3	1	1
3	45	1	2	0	0	0	7882	42	4	0	1
4	53	1	2	0	0	0	4870	24	1	0	1

Figure 8.9: Clustered dataset with 'Cluster' and DBSCAN labels

Let us plot the DBSCAN clusters with the **Age** and **Credit Amount** features:



Figure 8.10: Visualize 'DBSCAN' results

The purple points are marked -1 and treated as outliers(noise) by the DBSCAN algorithm. Simple profiling of the 4 DBSCAN clusters is shown:

dbscan	Credit amount								Duration							
	count	mean	std	min	25%	50%	75%	max	count	mean	std	min	25%	50%	75%	max
-1	108.0000	7393.5185	4441.4391	1175.0000	3723.2500	7302.5000	11052.7500	18424.0000	108.0000	32.1481	15.3214	6.0000	18.0000	30.0000	48.0000	72.0000
0	265.0000	2328.2943	1768.6013	250.0000	1207.0000	1795.0000	3017.0000	10974.0000	265.0000	17.7585	9.0278	4.0000	12.0000	15.0000	24.0000	48.0000
1	619.0000	2879.4330	2032.2819	276.0000	1392.0000	2315.0000	3651.5000	11054.0000	619.0000	20.0145	10.9470	4.0000	12.0000	18.0000	24.0000	60.0000
2	8.0000	9173.8750	1937.9122	6527.0000	8313.7500	9166.0000	10214.5000	12389.0000	8.0000	42.0000	11.1098	24.0000	36.0000	42.0000	48.0000	60.0000

Figure 8.11: Profile 'DBSCAN' results

Cluster '-1' as noise stands out in the average credit amount and duration features. DBSCAN also creates a rather useless Cluster '2' with just 8 data points and is probably driven by our choice of `minpts` and `epsilon` parameters. Clusters 0 and 1 look distinct in [Figure 8.11](#) but visually appear overlapping in [Figure 8.10](#). It is recommended to play around with `minPts` and `epsilon` parameters. A good next iteration would be a lower value of `minPts` (`min_samples`).

Lastly, let us evaluate the hierarchical clustering using the simple code shown:

```
# Using Hierarchical Clustering

from scipy.cluster.hierarchy import linkage ,
dendrogram, fcluster,cophenet

mergings = linkage(data_scaled,
method='ward',metric='euclidean')

dendrogram(mergings,truncate_mode='lastp',p=100)

plt.show()
```

Output:

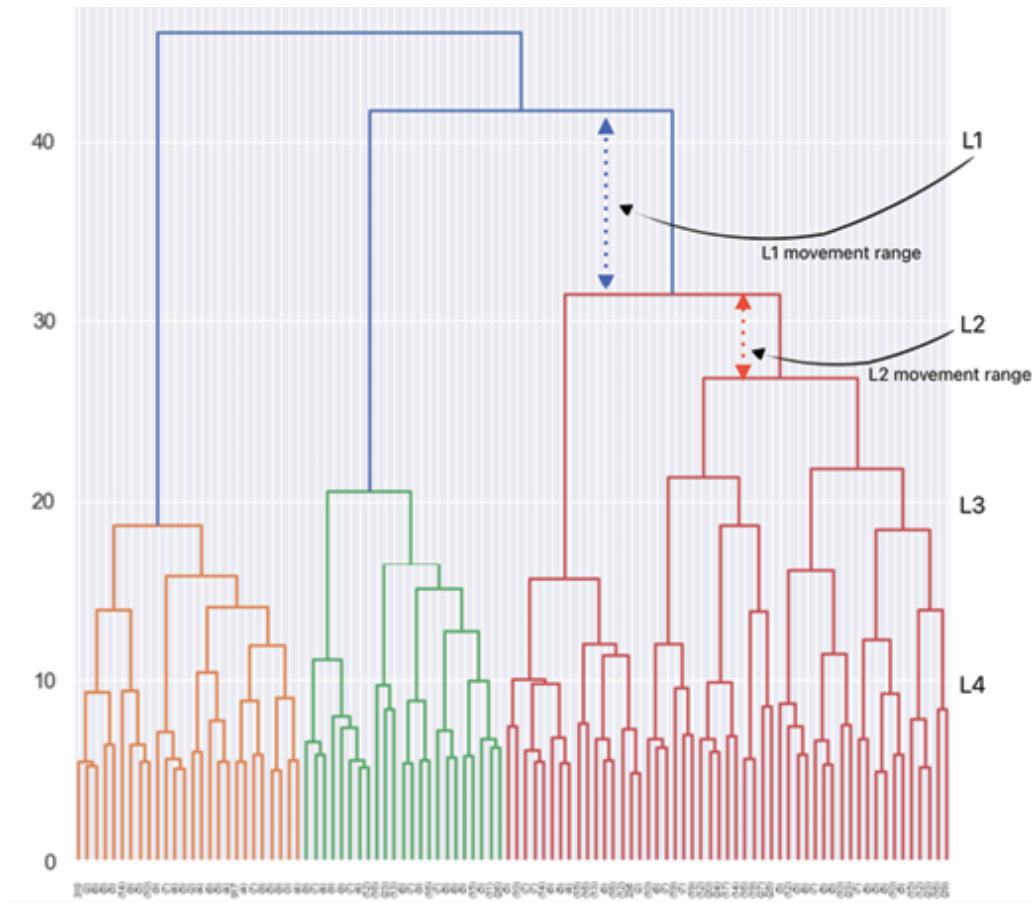


Figure 8.12: Hierarchical clustering dendrogram

In the preceding dendrogram, we can see that the horizontal white lines intersect several vertical lines at different distances (10,20,30,40). Using the below code we can identify the line *L1*(40) that can move the maximum distance without intersecting the merge points. The number of intersections (3) of this line *L1* is the optimal number of clusters. Other lines *L2*(30) and *L3*(20) can move less vertically compared to *L1*. The following Python code identifies the number of intersections for various lines:

```
for i in [20,30,40,50,51,59,60,70,76,80]:
    clusters = fcluster(mergings, i,
criterion='distance')
```

```
print('The number of cluster for the distance  
of', i, ' is ', len(np.unique(clusters)))
```

Output:

The number of clusters for a distance of 20 is 8

The number of clusters for a distance of 30 is 4

The number of clusters for a distance of 40 is 3

The number of clusters for a distance of 50 is 1

The number of clusters for a distance of 51 is 1

The number of clusters for a distance of 59 is 1

The number of clusters for a distance of 60 is 1

The number of clusters for a distance of 70 is 1

The number of clusters for a distance of 76 is 1

The number of clusters for a distance of 80 is 1

The optimal number of clusters as per this method is three for the line at a distance of 40. The key point to remember here is that this method is sensitive to outliers and not suitable for data with inherent overlapping patterns. These results could be misleading without visually checking the output clustering results.

In summary, this dataset has overlapping patterns with outliers, and hence DBSCAN is the safest method to analyze the patterns of this dataset.

Advanced unsupervised learning: A primer

Advanced learning implies complex learning with many layers. It will not involve finding Euclidean distances within a set of data points but instead involve finding complex patterns and features using more advanced mathematics

and then summarizing them. Thereafter, another algorithm will take note of these summaries and then learn to reconstruct a closer-to-reality outcome. It is worth noting that these algorithms go beyond K-means and so on, by not just summarizing but making use of the same learned pattern for future work.

For instance, over the years our brains are trained to identify features of various living objects, and these features are stored in our memory as notes. If we are ever asked to draw a dog, we will refer to our memory notes and draw it out. The more dog breeds we have seen better our notes are and even better is the reconstruction. If you note, the label dog is not important here because one can ask a friend what living object you saw in the park this morning that was tied to a leash, and he or she can recreate the image. This is unsupervised learning needing no labels but just pattern-mining instinct. How do computers accomplish such complex activities akin to human brain skills?

In the following three chapters, we will learn about neural networks which operate similarly to the human brain. We know that humans have many learning styles including an unsupervised one. We will dig deeper into neural networks and their different kinds later but meanwhile, let us list the several advanced neural network-based unsupervised learning techniques that have emerged in recent years. Each technique is designed to tackle complex data representations and discover hidden patterns without the need for labeled data. Some of these techniques include:

- **Autoencoders:** These are neural networks that learn to compress the input layer data based on attributes such as correlations between the input feature vector. This discovery happens during data training and subsequently, the network aims to reconstruct the input data at the output layer, by decompressing the information in the intermediate hidden layers. In

essence, this compression forces the model to maintain only the variations in the data required to reconstruct the input without holding on to redundancies within the input. Many different variants of the general autoencoder architecture exist with the goal of ensuring that the compressed representation represents only meaningful and generalizable attributes of the original data input. Some of them are listed as follows:

- **Variational Autoencoders (VAEs):** VAEs are a type of autoencoder that incorporates probabilistic modeling, which enables building an encoder that outputs a probability distribution for each latent attribute rather than a single value to describe each latent state attribute. VAEs learn a latent space distribution, allowing for smoother and more continuous interpolation between data points during the generative process for any sample of the latent state distribution. For instance, we obtain a probability distribution of the object wearing glasses as against a hard yes or no.
- **Temporal Autoencoders:** Temporal autoencoders are variants of traditional autoencoders that are designed to process sequential data, such as time series or natural language data. They aim to capture temporal dependencies and patterns in the data, allowing for more robust representations of sequential information.
- **Denoising Autoencoders:** Denoising autoencoders are trained to reconstruct clean data from noisy versions of the input. By introducing noise during training, denoising autoencoders learn to extract meaningful features and robust representations that are less affected by noise. This approach of slightly corrupting the input data and still maintaining the

uncorrupted data as our target output helps develop a generalizable model.

- **Generative Adversarial Networks (GANs):** GANs consist of two neural networks, a generator, and a discriminator, that engage in a game-like setting. The generator gets trained and attempts to create synthetic data points, while the discriminator tries to distinguish between real and fake data working as a classifier. This competition results in the generation of high-quality synthetic data that closely resembles the original data distribution. GANs can serve well as domain-specific data augmentation solutions alongside other generative solutions, such as image-to-image translation.
- **Self-organizing Maps (SOMs):** SOMs are a type of neural network used for unsupervised learning and dimensionality reduction. SOMs organize data points in a 2D or 3D grid based on similarity, preserving the topological relationships between data points. They are particularly useful for visualizing high-dimensional data in a lower-dimensional space.
- **Deep Belief Networks (DBNs):** DBNs are multi-layered neural networks that incorporate a probabilistic generative model. They are composed of multiple layers of hidden units, and each layer forms a restricted Boltzmann machine(RBM). DBNs are constructed from layers of RBMs, and it is necessary to train each RBM layer before training them together. The greedy algorithm teaches one RBM at a time until all RBMs are trained. These were created by *Geoffrey Hinton* in 2006 and are not as widely used these days.

These advanced neural network-based unsupervised learning techniques have proven to be powerful tools for discovering complex patterns and representations in various

types of data. They have found applications in diverse domains, including image and speech processing, natural language understanding, generative modeling, and anomaly detection.

Let us take an example of the use of autoencoders in anomaly detection and understand the process a little deeper. Anomaly detection can be thought of as both a supervised and unsupervised problem. The former would require annotations on data that consist of two classes, *normal* and *abnormal* (or *anomaly*), and the algorithm would learn to discriminate between those two classes. However, in most real-life datasets anomalies are rare making it tougher for the supervised algorithms to learn to discriminate between the normal and anomalous patterns. However, when this situation is structured as an unsupervised problem, the algorithm would aim at detecting anomalies by modeling the majority behavior and considering it as *normal*. Then it detects the *anomaly* or fraudulent behavior by searching for examples that do not fit well with the normal behavior.

Above unsupervised learning can be implemented using an autoencoder that is trained on a large dataset of legitimate transactions, and learns to reconstruct the input transaction data. It is important to note that an autoencoder is a type of neural network with the same number of neurons in the output layer as the input layer. The purpose is to reconstruct its own inputs instead of predicting the target value from the given inputs. Once this network is trained, the autoencoder aims to minimize the input reconstruction error, effectively learning the normal patterns inherent in legitimate transactions.

During deployment, the autoencoder is used to reconstruct new transactions, and instances with a high reconstruction error are flagged as potential anomalies. This approach compares the embeddings of the normal samples against the

new transaction samples and based on a threshold determines fraudulent behavior. The financial institutions would typically fine-tune hyperparameters and the neural network architecture through experimentation and cross-validation, optimizing the model's ability to detect anomalous transactions while minimizing false positives.

The unsupervised nature of autoencoders allows for continuous learning and adaptation to evolving fraud patterns, providing a robust solution for detecting new and sophisticated attack vectors without the need for labeled fraudulent examples. As the field of unsupervised learning continues to evolve, these techniques play a crucial role in advancing our understanding of data and extracting valuable insights without the need for labeled examples.

Technical details on the autoencoders have been skipped in this chapter, but readers will be able to learn more about the encoder-decoder architecture of this special neural network in [Chapter 10, Language Modeling with RNN](#).

Conclusion

In conclusion, this chapter explored the three powerful clustering techniques: K-means, DBSCAN, and hierarchical clustering in detail. Through a Python case study, we witnessed the practical application of these algorithms and gained insights into their strengths and limitations.

K-means, known for its simplicity and efficiency, showcased its ability to form distinct clusters by iteratively updating cluster centroids. DBSCAN, on the other hand, demonstrated its versatility in handling more complex and irregularly shaped data structures. Its automatic determination of the number of clusters proved particularly advantageous in scenarios where the data's underlying structure was less clear.

Hierarchical clustering revealed the power of hierarchy representation, providing a dendrogram that visualizes the hierarchical relationships between clusters. By either agglomerative or divisive methods, we witnessed how hierarchical clustering created a tree-like structure that helped us understand the data's hierarchical organization.

The Python case study showed us the implementation steps of these techniques in practical data analysis scenarios. From data preprocessing and visualization to the application of clustering algorithms, we explored the step-by-step process of unsupervised learning with Python's rich ecosystem of libraries.

As we conclude this chapter, we must appreciate that each technique comes with its own assumptions and strengths, and understanding when to use them is essential in achieving meaningful unsupervised results. This is true for advanced neural-network-based unsupervised techniques as well, which will be touched upon in more detail in the subsequent chapters. Remember, in the realm of clustering, discovering patterns and structures is only the beginning of an exciting adventure that awaits us in the world of data exploration and understanding. The consumption of these unsupervised technique results needs a lot of domain context for the most impact and a safe landing.

However, this discussion on supervised and unsupervised techniques does not end here. In the chapters so far, we talked about modeling techniques that needed a lot of data exploration, cleaning, and feature engineering before commencing the model development exercise. The upcoming three chapters on neural networks and their variations such as **recurrent neural network (RNN)** and **convolutional neural network (CNN)** are capable of self-identifying numerous connections in the input data with minimal prior human intervention. These are remarkable algorithms that can self-learn based on prediction error,

which is ultimately optimized during the learning process. It is time to change gears.

Points to remember

- The unsupervised technique clustering techniques work well in low-dimensional space where it is easy to decide on some level of similarity between points.
- High-dimensional space leads to lower similarity between points leading to reduced cluster tendency. For instance, it is easier to find facial similarity if we match just hair color, but as we include more features such as eyes, nose, and face dimensions, the similarity quotient will reduce.
- It is advisable to perform exploratory data analysis and preprocessing to ensure that the unsupervised learning methodology assumptions align with the characteristics of the data and the objectives of the analysis. One should have an intuition of the clusters that might come out.
- K-means excels with well-separated spherical clusters, DBSCAN thrives in handling irregular shapes and varying densities, and hierarchical clustering beautifully unveils the hierarchical relationships within the data.
- Advanced methods such as neural networks have impacted the field of unsupervised learning and allow us to explore, learn, and generate even unstructured data in unsupervised ways.

Join our book's Discord space

Join the book's Discord Workspace for Latest updates, Offers, Tech happenings around the world, New Release and Sessions with the Authors:

[**https://discord.bpbonline.com**](https://discord.bpbonline.com)



CHAPTER 9

Structured Data Classification using Artificial Neural Networks

Introduction

As our world becomes data-rich and nuanced, the human mind seeks deeper insights, deciphering data puzzles that elude simpler models.

Artificial Neural Networks (ANNs) offer a unique and powerful approach to problem-solving that complements and even augments other machine-learning techniques in several ways. ANNs are infamous for being black-box but can handle high complexity and high-dimensional data with an obvious requirement for tremendous computational resources. However, they are a marvel of human ingenuity inspired by the intricate web of our brain's neurons, which have ushered us into an era where machines mimic our cognitive processes.

As we embrace the power of this technology, we find ourselves facing a crucial question: How can we ensure transparency and interpretability in the realm of AI that will also make the usage of this technology responsible?

Structure

In this chapter, we will learn the following topics:

- Artificial neural networks
- Background
- Under the hood of neural networks
 - Data
 - Model
 - Loss function: **Achieving optimal results**
 - Backpropagation and regularization
- Challenges and assumptions
- Case study: Explainable and Interpretable ANN Model
 - Interpretable and explainable AI using SHAP and PiML

Objectives

The objective of this chapter is to expose readers to the ANN ways of solving the classification tasks that we are accustomed to solving using institutionalized methods. In the spirit of daring to explore and not debilitate one's creativity and purpose, we will create new neural pathways through new stimuli (experiences) and learn ANN.

We will create these new experiences by exploring the theoretical workings and uncovering the mechanics behind ANNs, demystifying layers, nodes, weights, and activations that together create the foundation of deep learning. However, we will not stop there, we will explore a captivating case study where ANNs are employed in a transparent and explainable manner, bridging the gap between AI's power and human comprehension and responsibility towards society.

As we journey through the theory and practical application of ANNs, we will emphasize the importance of not just achieving accuracy but also understanding how and why our AI models make certain decisions. So, let us navigate our way into the world of neural networks, where theory meets real-world application,

and where the quest for explainable and transparent AI shapes the future of technology.

Artificial neural network

ANN speaks the language of complex relationships, echoing the human desire to grasp the nuances of data just like an artist captures subtleties in a masterpiece. Each of these subtleties eventually comes together on the canvas to paint the rich tapestry of data.

This is reflected in the ANN's pursuit of mirroring human cognition. ANNs structure their architecture to loosely replicate the interconnected structure of neurons in the human brain like a large piece of fabric. Just as our brain cells communicate through synapses, ANNs use interconnected nodes to process and transmit information. When data is fed into an ANN, it travels through layers of nodes, each layer extracting and transforming features.

These ANNs shine when it comes to complex problems such as image classification, natural language processing, and speech recognition tasks that involve large amounts of data and do not have pre-defined features such as FICO score, age, and so on.

Background

So far in this book, we have talked about data mining techniques that help us cluster similar data points together, identify outliers, or classify data into different categories based on its attributes. We learned about some advanced techniques that involved bagging and boosting to perform data mining tasks accurately and more importantly precisely. These advanced algorithms fall in the category of machine learning and take us in the direction of achieving artificial intelligence.

However, as data sources such as images and large texts appeared on the horizon, humans realized the importance of digging deeper into the preliminary work done in the fields of neurophysics and mathematics. Throughout the 1940s, *Warren McCulloch*, *Walter Pitts*, and *Donald Hebb* did pioneer work to

explain the interconnection and working of neurons in the human brain and explained the fact that neural pathways are strengthened each time they are used, a concept fundamentally essential to how humans learn. Hebb argued that if two nerves fire at the same time, the connection between them is enhanced. This leads to the fundamental idea behind the nature of neural networks which is if it works in human brains, it should work in silicon-based computers. However, the formidable challenge remains in terms of imitating all the complexity and paraphernalia of the human brain, and hence the future of neural networks lies in the development of processing power and hardware.

Today we know the machine learning realm and this specialized field of deep learning that picked up speed in the year 2006. This was the year when Hinton published *Reducing the Dimensionality of Data with Neural Networks* which claimed that neural networks called *Deep Belief Networks* can be efficiently trained by using a strategy called greedy layer-wise pre-training. This initiated the third wave of neural networks that made also the use of the term deep learning popular. Hinton had already made his mark in the AI community in the year 1986 when he published *Learning representations by back-propagating errors* with David Rumelhart and Ronald Williams. The paper described a new learning procedure called *backpropagation for training multi-layer neural networks*.

Due to the heterogeneity of deep learning approaches a comprehensive discussion is very challenging, but the most notable algorithm in deep learning is the ANN algorithm which can solve a variety of problems with structured and unstructured data sources.

Under the hood of neural networks

In this chapter, we will investigate the components and workings of ANN while working with structured data. This will be followed by a discussion on other popular neural networks such as **convolutional neural networks (CNN)** and **recurrent neural networks (RNN)** in the upcoming next two chapters. While ANNs are versatile and applicable to various tasks, CNNs excel in spatial

data processing such as images and videos, and RNNs shine in sequential data modeling such as natural language processing, making each architecture uniquely suited to different machine learning challenges. ANNs have interconnected nodes organized in layers, while CNNs follow a similar multi-layer interconnection concept but leverage convolutional layers to capture spatial hierarchies and recognize patterns. On the other hand, RNNs are designed for sequential data, possessing memory to process information in a time-dependent manner.

With that, it is time to jump right into the anatomy of an ANN, which is built in a way to solve complex problems by breaking down the problem into smaller pieces. What do we mean by smaller pieces?

Similar to neurons in the brain, perceptrons (artificial neurons) produce an output once a certain stimulus threshold is met. Each of these perceptrons is associated with different weights and biases that help solve a smaller problem. ANN then composes simple but non-linear modules that each perceptron solves as a smaller problem at one level (starting with the raw input) into a representation at a higher, slightly more abstract level. With the composition of many such solutions, very complex data patterns can be learned. [Figure 9.1](#) shows the working of a perceptron:

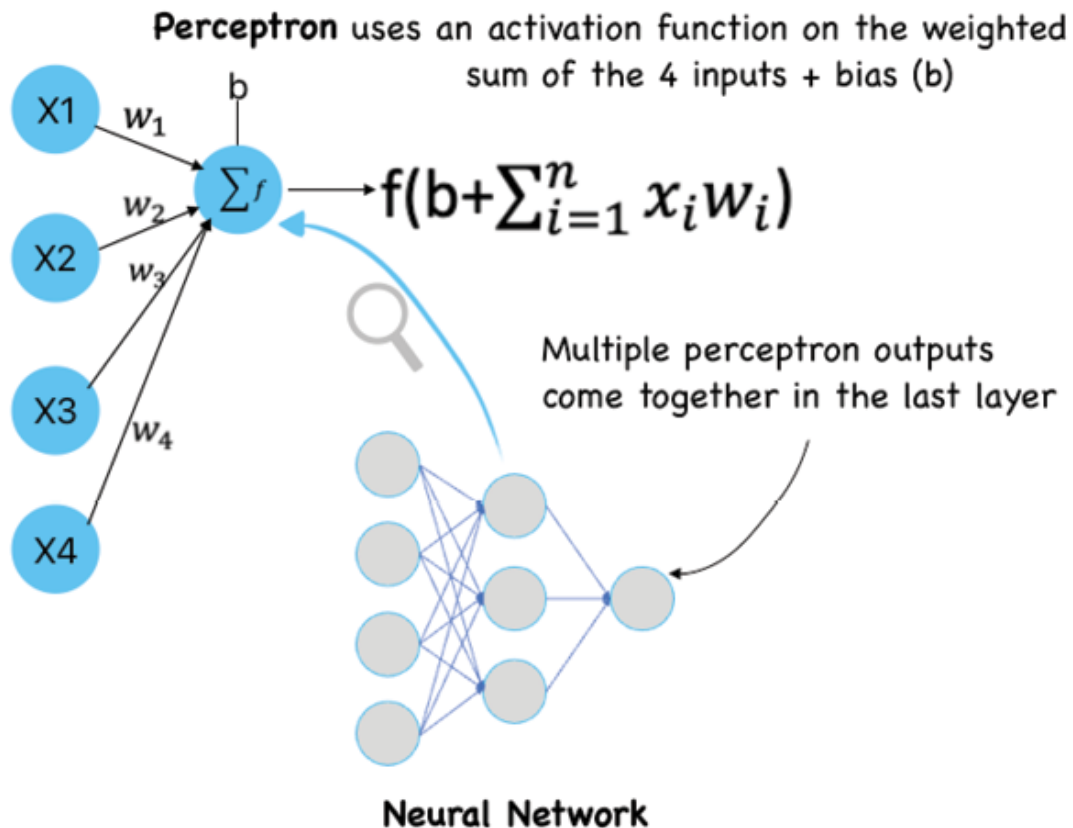


Figure 9.1: Anatomy of an ANN

The perceptron's output is determined by whether the weighted sum, $\sum$, is less than or greater than some *threshold value*. Just like the weights, the threshold is a real number which is a parameter of the neuron.

Data

The Portuguese bank marketing dataset contains 45,211 records with 17 attributes. We have used this dataset in *Chapters 5, Decision Tree Algorithm with Bagging and Boosting* and [Chapter 6, Support Vector Machines and KNN](#), for classification tasks and will continue to use it in this chapter as well to demonstrate the algorithmic performance across the range of supervised techniques such as trees, SVM, KNN, and ANN. In this dataset, each record represents a bank customer who has subscribed(Yes/No) to a term deposit at the bank. We will use the entire dataset with some transformations for NN modeling but will

sample records to demonstrate some post hoc analysis. The sample observations are shown in [Figure 9.2](#):

	age	job	marital	education	default	balance	housing	loan	contact	day	month	duration	campaign	pdays	previous	outcome	Target
0	58.0	management	married	tertiary	no	2143.0	yes	no	unknown	5.0	may	261.0	1.0	-1.0	0.0	unknown	no
1	44.0	technician	single	secondary	no	29.0	yes	no	unknown	5.0	may	151.0	1.0	-1.0	0.0	unknown	no
2	33.0	entrepreneur	married	secondary	no	2.0	yes	yes	unknown	5.0	may	76.0	1.0	-1.0	0.0	unknown	no
3	47.0	blue-collar	married	unknown	no	1506.0	yes	no	unknown	5.0	may	92.0	1.0	-1.0	0.0	unknown	no
4	33.0	unknown	single	unknown	no	1.0	no	no	unknown	5.0	may	198.0	1.0	-1.0	0.0	unknown	no
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
45206	51.0	technician	married	tertiary	no	825.0	no	no	cellular	17.0	nov	977.0	3.0	-1.0	0.0	unknown	yes
45207	71.0	retiree	divorced	primary	no	1729.0	no	no	cellular	17.0	nov	456.0	2.0	-1.0	0.0	unknown	yes
45208	72.0	retiree	married	secondary	no	5715.0	no	no	cellular	17.0	nov	1127.0	5.0	104.0	0.0	success	yes
45209	57.0	blue-collar	married	secondary	no	668.0	no	no	telephone	17.0	nov	508.0	4.0	-1.0	0.0	unknown	no
45210	37.0	entrepreneur	married	secondary	no	2971.0	no	no	cellular	17.0	nov	361.0	2.0	188.0	11.0	other	no

Figure 9.2: Portuguese banking dataset snapshot

The link to the original dataset can be found as follows:

<http://archive.ics.uci.edu/dataset/222/bank+marketing>

For modeling purposes, we need to do a couple of things to transform the data:

- Categorical features transformed with one-hot encoding
- Standardization of all numerical features to bring them on the same scale

Model

We now know that perceptrons take several binary inputs, $x_1, x_2, \dots, x_n$, and produce a single binary output. However, the perceptron alone cannot completely model complex human decision-making, but it seems plausible that a complex network of perceptrons could make quite subtle non-linear decisions. These complex neural networks where the output from one layer is used as input to the next layer are called **feedforward neural networks (FFNN)**. FFNNs are ANNs in which nodes do not form loops implying the information always moves forward and is never fed back. Some ANNs with feedback loops are called **recurrent neural networks (RNNs)**. The idea in these models is to have neurons that fire for some limited duration of time, before becoming quiescent. We will study these architectures in [Chapter 10, Language Modelling with RNN](#), but here, let us explore the

multi-layer FFNN, traditionally referred to as **multi-layer perceptrons (MLP)**. The following figure illustrates an FFNN:

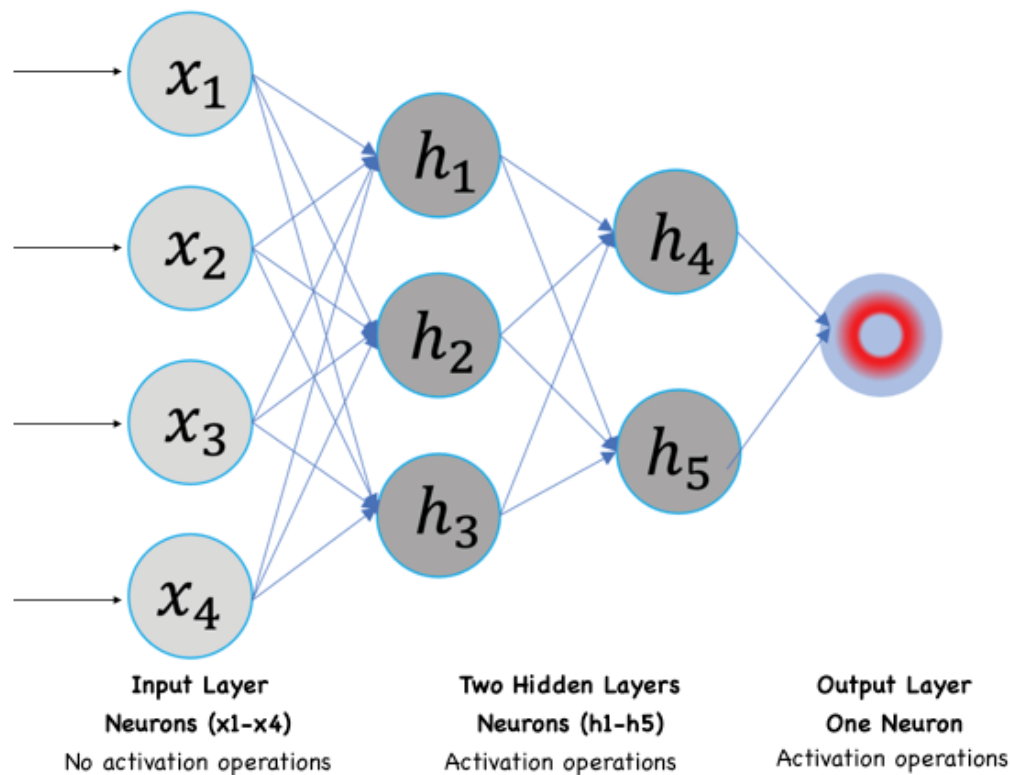


Figure 9.3: Feedforward neural network

As seen in [Figure 9.3](#), during data flow, when input layer nodes receive data, it goes without any operations. The second or hidden layer takes values from the first layer does weight multiplication, bias addition, and activation operations, and then subsequently passes this value to the next layer. The same process repeats for all subsequent hidden layers before finally an output value is obtained from the last output layer. Please note that no links exist in the network that could be used to send information back from the output node.

Now, before moving forward let us learn about a few key aspects of the network components in detail:

- **Hidden layer:** A feedforward network with multiple hidden layers can compute complex functions and learn hierarchal representations. As data passes through successive hidden layers, the first hidden layers capture simple features, while

subsequent layers combine these features to form more complex ones. This allows the network to capture intricate and abstract relationships in a layered manner.

For instance, many real-world problems involve non-linear relationships. Hidden layers introduce non-linear activation functions (for example, ReLU, sigmoid) that enable the network to approximate complex functions such as classifying between a dog and a cat. This happens when the network combines the activations of multiple hidden neurons, that recognize specific features such as color patterns or bodily features, to feed into an output neuron. This leads to the formation of complex decision boundaries involving the intricate relationships between color, features, and body type of the animal for classification purposes. This capacity to model non-linearities makes DNNs powerful tools for various tasks.

Essentially, why use hidden layers can be answered by stating that hidden layers help get rid of the feature engineering requirement of the traditional ML algorithms. A single hidden network can learn various decision boundaries. For instance, if there are two inputs X_1 and X_2 , these can be combined to form the X_1, X_2, X_1^2, X_2^2 , etc. Essentially, an NN with a single hidden layer with nonlinear activation functions is considered to be a **Universal Function Approximator (UFA)** implying it is capable of learning any function but with conditions of network width, appropriate weights, and so on.

Secondly, the choice of hidden layer neurons is a very active research area in deep learning. The type of hidden layer differentiates between the types of neural networks like CNNs, RNNs, and so on. Thirdly, the number of hidden layers, which is termed the depth of the neural network, is a matter of NN design. One question you might ask is exactly how many layers in a network make it deep enough to capture all the intricacies of the data? There is no right answer to this question. In general, deeper networks can learn more complex functions.

- **Activation functions:** These are the filters or the decision-makers that determine the amount of information that passes to the next layer of the network. They serve different functions by introducing non-linearity to neural networks or squashing the values in a smaller range. For instance, a sigmoid activation function squashes values between a range of 0 to 1. There are many activation functions used in the deep learning space.

Figure 9.4 describes a few commonly used activation functions:

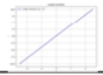
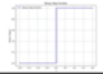
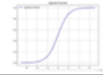
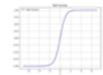
Name	Description	Function	Range	Plot
Identity/ Sigmoid	It performs a straightforward linear transformation of the input data without introducing any non-linearity.	x	$(-\infty, \infty)$	
Binary Step	The binary step function is a simple activation function that outputs 0 if the input is less than or equal to 0 and 1 if the input is greater than 0.	$\begin{cases} 0 & \text{if } x < 0 \\ 1 & \text{if } x \geq 0 \end{cases}$	$\{0, 1\}$	
Sigmoid/ Logistic	This function squashes input values between 0 and 1, making it useful for tasks like yes or no or binary classification. It has been around for a while, but it can sometimes cause problems in deeper networks.	$\sigma(x) = \frac{1}{1 + e^{-x}}$ ^[1]	$(0, 1)$	
Softmax	Softmax is used for multi-classification in logistic regression model (multivariate) whereas Sigmoid is used for binary classification in logistic regression model. The sum of probabilities across classes add up to one.	$\text{Softmax}(z)_i = \frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}}$	$(0, 1)$	
Tanh	It is a good choice when the model needs to handle both positive and negative inputs and when a degree of zero-centeredness is needed. It is similar to sigmoid except that it ranges from -1 to +1.	$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$	$(-1, 1)$	
ReLU	ReLU is a popular choice these days. It lets positive numbers pass through as they are and sets negative numbers to zero. ReLU is great for speeding up training.	$\begin{cases} 0 & \text{if } x \leq 0 \\ x & \text{if } x > 0 \end{cases}$ $= \max\{0, x\} = x \mathbf{1}_{x > 0}$	$[0, \infty)$	
Leaky ReLU	It is a variant of the ReLU that allows a small, non-zero gradient when the input is negative. This helps prevent some neurons from "dying out" and improves the flow of information.	$\begin{cases} 0.01x & \text{for } x < 0 \\ x & \text{for } x \geq 0 \end{cases}$	$(-\infty, \infty)$	

Figure 9.4: Commonly used activation functions

Choosing the right activation function depends on the network's architecture, the data you're working with, and the business objective you want to accomplish. For instance, the **Rectified Linear Unit (ReLU)** activation function is commonly used in hidden layers of neural networks since it helps introduce non-linearity into the model, allowing it to learn complex patterns and features from datasets such as images. The activation function replaces all negative values in the neuron's output with zero, effectively introducing

sparsity and aiding in the model's ability to learn hierarchical representations. On the other hand, the Softmax activation function is commonly used in the output layer of a neural network for multi-class classification problems like sentiment analysis. In sentiment analysis, the goal is to classify a piece of text (such as a review or tweet) into predefined sentiment classes like positive, negative, or neutral. Softmax is well-suited for this task as it normalizes the output scores across all classes, converting them into probabilities. Lastly, Sigmoid is used to perform binary classification tasks similar to logistic regression and is used in the output layer as well.

- **Output layer:** This is the layer that delivers the predictions. The activation function to be used in this layer varies by the problem statement. Please refer to [Figure 9.5](#) for a detailed description of the design of this layer along with some common use cases associated with the activation functions in the output layer.

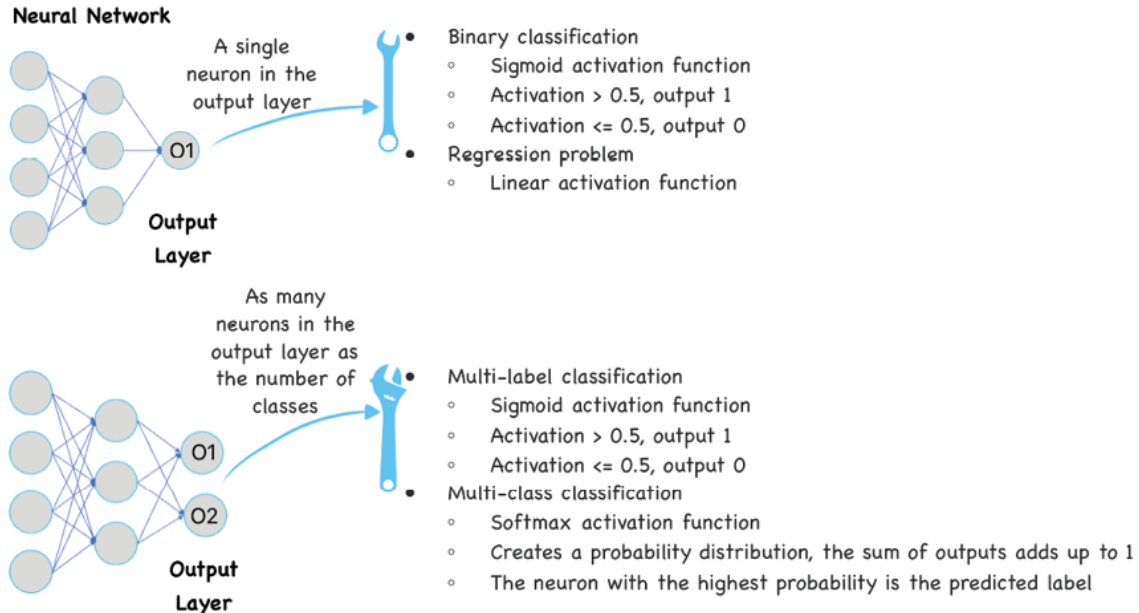


Figure 9.5: Commonly used activation functions with use cases

With the knowledge of different network components, it is important to ask yourself - How does the FFNN learn? A short response is that the training samples are passed through the

network, either as a batch or one at a time, and the output obtained from the network is then compared with the actual output to calculate an error. This error depends on the type of dependent variable, binary vs. continuous, which is used to change the weights of the neurons across multiple layers such that the overall error decreases gradually. This is done using the backpropagation algorithm. This process is repeated by passing multiple batches of data through the network and updating the weights so that the error is reduced and convergence happens. The amount by which the weights are changed is determined by a parameter called learning rate. The details of backpropagation will be covered in the next section which covers loss definition and measurements.

Loss function: Achieving optimal results

We now know that FFNN learns by reducing the error or the loss for each observation by propagating the error back in the network and adjusting the weights associated with every neuron. This requires a cost function that tracks the error when adjusting weights and biases. In this process, an optimal network output $y(x)$ is obtained for all training inputs x in the dataset. [Figure 9.6](#) shows the cost function for both binary and continuous dependent variables:

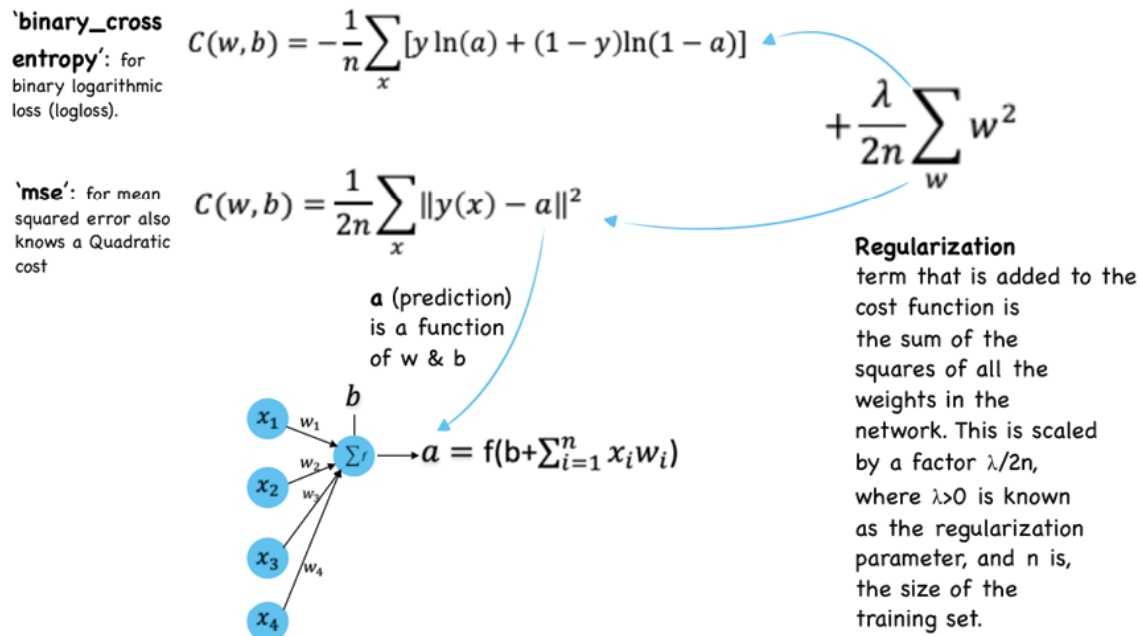


Figure 9.6: Cost functions

It is important to note that the loss function computes the error for a single training observation while the cost function is the average of the loss functions of the entire training set. That is why in [Figure 9.6](#), $C(w, b)$ equation, we see the little x at the bottom of the sigma notation, indicating that the individual training losses are summed over all the x .

[Figure 9.6](#) also shows the regularization term, which is the same for the quadratic and the cross entropy loss equations. The effect of this term is to make the network learn smaller weights, all other things being equal. Alternatively, regularization can be viewed as a way of trade-off between finding small weights and minimizing the original cost function. Lambda (λ) drives this trade-off between these two elements by minimizing the original cost function when λ is small, and driving smaller weights when λ is large.

Dropout is another method of regularizing the networks that involves reducing reliance on specific neurons during training and thereby preventing the network from becoming too specialized on the training data and performing poorly on new, unseen data. In an FFNN with dropout, at each training iteration, a random subset of neurons in a layer is *dropped out* or deactivated. This means

their outputs are set to zero, and they do not contribute to the forward pass of that iteration. The probability of dropping out of a neuron is a hyperparameter and typically ranges from 0.2 to 0.5. A good way to think about dropout is as an ensemble of multiple networks, each with a different subset of active neurons. This ensemble effect helps in improving generalization by reducing co-dependencies between neurons and ensuring that no single neuron becomes a critical piece of the model.

During testing or inference, the dropout is turned off, and all neurons are active. However, the weights are scaled down by the dropout probability to maintain the expected values from training. This is crucial to ensure that the network's behavior during testing matches its behavior during training.

With knowledge of the cost functions and regularization terms' impact on its final value, We will move on to understand other ways that impact the network weights leading to the minimization of the cost function.

Another critical concept in FFNN training is **Batch Normalization**. After the original input to the network is normalized (mean=0 and standard deviation=1) and sent through the first layer, with time the output across subsequent layers no longer remains on the same scale. This is because as the data passes through multiple layers of NN, various activation functions lead to an *internal covariate shift* in the data. This means during the training of neural networks, there are changes in the distribution of network activations (outputs of hidden layers). These shifts in the distributions of inputs to subsequent hidden layers impact the learning dynamics and can slow down the training process, as each layer must continuously adapt to the changing distribution of inputs.

Batch normalization is a method for mitigating the internal covariate shift by normalizing the interlayer outputs of a neural network. As a result, the next layer receives a normalized output distribution from the preceding layer, allowing it to train effectively without worrying about the constant change in the input distribution. Since normalization guarantees that no activation value is too high or too low, and since it enables each

layer to learn independently from the others, this strategy leads to quicker learning rates.

In addition to its role in stabilizing training, batch normalization has a positive side effect of providing regularization, contributing to improved generalization performance in neural networks. It is common practice to apply batch normalization before a layer's activation function, and it is commonly used in tandem with other regularization methods like a dropout.

Back-propagation and regularization

In the early days, the perceptron learning algorithm could not be extended to multi-layer FFNNs for solving complex non-linear problems. This led to AI disappointment and skepticism during the 1970s and 80s. The breakthrough of backpropagation in 1986, led to the development of more powerful neural network architectures, including multi-layer perceptrons, which showed promising results in various non-linear applications such as image recognition, speech processing, and natural language. This was possible because multi-layered FFNN could be trained with accuracy by adjusting network weights and biases of all neurons in each layer until the actual vs. predicted error is reduced. This self-learning approach led to the renewed success of neural networks and their ability to handle intricate patterns and non-linear relationships. [Figure 9.7](#) shows the two-step learning process using the backpropagation approach:

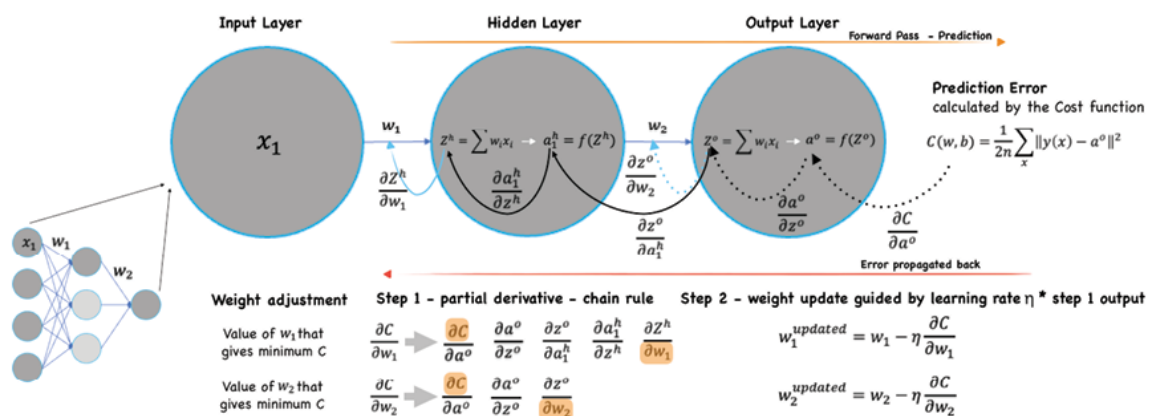


Figure 9.7: Backpropagation in feed-forward neural network

Backpropagation is a **gradient descent (GD)** based technique with the goal of reducing the cost function while learning a neural network for a given training data. The algorithm does this by calculating the gradient ($\partial C/\partial w$ and $\partial C/\partial b$) of the cost function, with respect to the weights and biases, iteratively and then updating the weights and biases in the opposite direction.

As shown in [Figure 9.7](#), for the highlighted three-neuron sequence starting with x_1 , back-propagation uses the chain rule of differential calculus to calculate the partial derivatives of the prediction error value with respect to the two weight (and) values. The figure demonstrates how the chain rule will help us identify how much each of the two weights contributes to the overall error and the direction to update each weight to reduce the error. These partial derivatives are also called gradients and together with the learning rate are used to update the respective weights. This process works well for the most part except in certain cases when the weight update is not perfect leading to ineffective learning. This is detailed in the next section under the header vanishing and exploding gradients.

In summary, for every weight in the network, the weight update follows the following steps:

1. Calculate the gradient for every observation error
2. Average these gradients over all the training observations (in the case of batch GD)
3. Adjust the weights as shown in Step 2 of [Figure 9.7](#).

These steps are performed by optimizers, that play a crucial role in training neural networks by adjusting the model parameters during the optimization process to minimize the loss function.

ADAM (Adaptive Moment Estimation) is one such optimizer designed to enhance the efficiency and convergence speed of gradient-based optimization algorithms. They contribute significantly to the efficiency and effectiveness of training neural networks by adapting learning rates, incorporating momentum, addressing biases, and providing robustness to various challenges encountered during the optimization process. There are other optimizers such as **RMSprop (Root Mean Square**

Propagation), Adagrad (Adaptive Gradient Algorithm), SGD, and so on as well. Notably, the choice of optimizer can have a profound impact on the convergence speed and the ability of the neural network to generalize well to new, unseen data.

In the case of stochastic **gradient descent (SGD)**, the average is over training samples x in a mini-batch instead of all the observations like in the batch GD. The SGD backpropagation approach updates the weight once for the mini-batch and hence helps us move closer (walking along the gradient) to the local minima faster.

Please note in [Figure 9.7](#), that for each iteration we perform forward propagation to compute the outputs and backward propagation to attribute the errors; one such complete forward-backward pass is known as an epoch. It is standard practice to report evaluation metrics such as loss and accuracy after each epoch so that we can track the evolution of our neural network as it trains.

Challenges and assumptions

This section summarizes the obvious based on the knowledge we have gained so far. FFNNs are pathbreaking because they can do self-learning to optimize network weights and raise the performance bar every time, but they do come with their own challenges.

Challenges of DNNs as unique and suit their solution paradigm:

- **Interpretability:** This one is on top because understanding the decisions and reasoning of complex deep networks can be difficult, hindering their interpretability. We will try to address this challenge in the upcoming case study.
- **Vanishing and exploding gradients:** In very deep networks, gradients can become too small (vanishing gradients) or too large (exploding gradients) during backpropagation, making training difficult. In the vanishing gradient problem, as gradients are backpropagated through multiple layers during training, the chain rule of calculus results in the multiplication of many small gradient values,

causing the overall gradient to diminish exponentially, leading to negligible updates for early layers. This phenomenon is particularly pronounced when using activation functions, such as the sigmoid or tanh functions, whose derivatives approach zero for certain input ranges. On the other hand, the exploding gradient problem occurs when gradients grow exponentially during backpropagation, often caused by weight parameters that are initialized too large or due to the choice of activation functions with unbounded output ranges. Both issues hinder the effective update of weights in deep layers, impeding the convergence of the training process and making it challenging to learn meaningful representations in deep neural networks. Techniques such as careful weight initialization, batch normalization, and the use of activation functions with well-behaved gradients, like ReLU, aim to mitigate these problems and facilitate the training of deep neural networks.

- **Overfitting:** Deep networks with a large number of parameters can overfit the training data if not properly regularized, leading to poor generalization of new data. Batch normalization and careful dropout rate selection can reduce overfitting.
- **Computational intensity:** Training deep networks requires significant computational resources and time, especially for very deep architectures that require large amounts of labeled data for effective training.
- **Architecture and tuning:** Choosing appropriate hyperparameters, such as learning rate and network architecture, can be challenging for beginners and impact the network's performance.

Assumptions of **Deep Neural Networks (DNNs)** can be grouped into three categories:

- **Data:** DNNs assume that relevant features for a task exist in the data and can be learned through multiple layers of abstraction apparent in the network's hidden layers. This assumption also drives another assumption of the

availability of a substantial amount of data to effectively capture complex patterns.

- **Model scalability:** DNNs assume that network performance can improve with deeper and wider architectures while formulating complex functions, given sufficient computational are available
- **Learning:** DNNs assume that gradients can be effectively propagated through all layers during backpropagation, while employing regularization techniques, like L2, dropout, or weight decay, to ensure effective and efficient learning.

Case study: Explainable and Interpretable ANN Model

As part of the case study, we are going to use the data related to the direct phone marketing campaigns of a Portuguese banking institution to train an ANN model. We will be using the Keras and SHAP Python libraries for model training and interpretability.

Thereafter, we will rebuild this model using the low-code panels of the PiML python library to interpret the model build during development. This is important because NN models are inherently considered black-box models and this lack of interpretability of many machine learning models including NN makes it difficult to understand and trust the model-based decision making. More details on the PiML library can be found at:

https://selfexplainml.github.io/PiML-Toolbox/_build/html/guides/introduction.html. Python packages such as InterpretML and PiML make it easy to design inherently interpretable machine-learning models.

As it relates to this case study, the classification goal is to predict if the customer will subscribe to a term deposit and we have already seen the performance of different supervised learning techniques on this dataset in [Chapter 5, Decision Tree Algorithm With Bagging and Boosting](#) and [Chapter 6, Support Vector Machines And KNN](#).

Please note that these libraries offer a wide range of interpretability tools and it may not be possible to cover

everything as part of this case study. The goal is to introduce readers to the latest in the world of ML and DNN models and stresses the importance of interpretability and explainability of otherwise black-box models. This becomes important in domains where model-driven decision-making can have significant consequences such as unfair lending or unfair banking overall.

Data for this exercise is tabular with 45,211 records and 17 features. While NNs are capable of handling more complex and larger datasets, we will start small in this chapter to learn the basics of NN architecture, hyperparameters, and interpretability first.

The data preprocessing steps requirements will not be discussed here again as they remain mostly similar to other modeling techniques from [Chapter 5, Decision Tree Algorithm With Bagging and Boosting](#) and [Chapter 6, Support Vector Machines And KNN](#). Outlier and missing value treatment, data augmentation and transformation(one-hot encoding and feature scaling) are all standard ways to feed the best data to the network for efficient training. With that let us write the first code to build a basic FFNN:

```
#import libraries

from keras.models import Sequential
from keras.layers import Dense, Activation, Dropout
from keras.optimizers import SGD
from keras.callbacks import History

#build the sequential FFNN architecture
model = Sequential()
model.add(Dense(10,
input_dim=X_train_preprocessed.shape[1],
activation='relu'))
model.add(Dense(1, activation='sigmoid'))
```

```
model.compile(loss='binary_crossentropy',  
optimizer='adam', metrics=['accuracy'])
```

The above code produces no output until we run the `model.fit()` statement next, but before doing that let us understand these four lines of architecture building code. The first line above instantiates a sequential model that is appropriate for the network architecture with a linear stack of layers.

The `model.add()` adds the first layer with 10 neurons each with a ReLU activation function, and the second layer has a single neuron with a sigmoid activation function. The model is compiled with a binary cross-entropy loss function to train binary classification tasks and the Adam optimizer, and accuracy is used as the evaluation metric.

Please note, as discussed earlier, the optimizer is a search technique, which is used to update weights in the model during backpropagation. The three widely used optimizers are listed as follows:

- **SGD: Stochastic Gradient Descent (SGD)**, with support for momentum. We have learned about this in the previous section.
- **Adam: Adaptive Moment Estimation (Adam)** uses adaptive learning rates and is a very commonly used technique.
- **RMSprop**: Adaptive learning rate optimization method proposed by *Geoff Hinton*.

As a next step, after setting up the `MLflow` experiment, we run the below code to start training the NN model that was designed and compiled previously:

```
# Start the MLflow run
```

```
with mlflow.start_run() as run:
```

```
    # Fit the Keras model separately using the  
    preprocessed training data
```

```
    model.fit(X_train_preprocessed, y_train_encoded)
```

```

# Evaluate the model on the train data
loss, accuracy =
model.evaluate(X_train_preprocessed, y_train_encoded)
print(f'Train Loss: {loss:.4f}')
print(f'Train Accuracy: {accuracy:.4f}')

# Evaluate the model on the testing data
loss, accuracy =
model.evaluate(X_test_preprocessed, y_test_encoded)
print(f'Test Loss: {loss:.4f}')
print(f'Test Accuracy: {accuracy:.4f}')

```

The `model.fit()` statement trains the model and produces the following output:

Train Loss: 0.3021

Train Accuracy: 0.8922

283/283 [=====] - 1s 2ms/step
- loss: 0.3128 - accuracy: 0.8891

Test Loss: 0.3128

Test Accuracy: 0.8891

It is important to note that we have removed the feature `duration` from the modeling dataset for training this NN. This is because we noticed in [Chapter 5, Decision Tree Algorithm with Bagging and Boosting](#), that this feature is the most correlated with the `Target` since in the scenario when the duration of the last call with the customer is non-zero, it is highly likely that the customer subscribed to the term deposit. This leads to the feature `duration` turning out to be the most important and suppressing contribution of other features.

Since NN is very powerful, we want to evaluate the impact due to other features while aiming for similar accuracy levels. We can already see that a single-layer NN with 10 neurons gives us a test accuracy of 88.91% compared to the 89.25% test accuracy of the

Xgboost model from [Chapter 5, Decision Tree Algorithm With Bagging and Boosting](#), (`duration` was included). Very comparable indeed with one less feature.

Let us build a slightly more complex FFNN. The code for the same is as follows:

```
# create deep layer model
model_1 = Sequential()
model_1.add(Dense(30, activation='relu',
input_dim=X_train_preprocessed.shape[1]))
model_1.add(Dropout(0.10))
model_1.add(Dense(20,
activation='relu',kernel_initializer='uniform'))
model_1.add(Dense(12,
activation='relu',kernel_initializer='uniform'))
model_1.add(Dropout(0.10))
model_1.add(Dense(6, activation='relu'))
model_1.add(Dense(1, activation='sigmoid'))
#Creating an Stochastic Gradient Descent
sgd = SGD(learning_rate = 0.01, momentum = 0.9)
model_1.compile(optimizer='adam',
                loss='binary_crossentropy',
                metrics=['accuracy'])
```

We added three hidden layers with 20,12, and 6 neurons starting from the first input layer with 30 neurons. The last layer with the sigmoid neuron helps with the logistic regression type output and hence helps with the binary classification. Please note we also added some regularization in the form of dropout meaning that during each training iteration, 10% of the neurons in the layer will be randomly set to zero.

It is critical to understand that the dropout mask is applied to both the forward and backward passes to ensure consistency and the gradients are only backpropagated through the active neurons to update their weights. The following one-line code shows the summary of the model architecture:

```
model_1.summary()
```

Output:

```
Model: "sequential_1"

```

Layer (type)	Output Shape	Param #
dense_2 (Dense)	(None, 30)	1230
dropout (Dropout)	(None, 30)	0
dense_3 (Dense)	(None, 20)	620
dense_4 (Dense)	(None, 12)	252
dropout_1 (Dropout)	(None, 12)	0
dense_5 (Dense)	(None, 6)	78
dense_6 (Dense)	(None, 1)	7

```

Total params: 2187 (8.54 KB)
Trainable params: 2187 (8.54 KB)
Non-trainable params: 0 (0.00 Byte)

```

Figure 9.8: ANN model summary output

The code for executing the model defined in [Figure 9.8](#) using a MLflow experiment is as follows:

```
# Fit the pipeline to the training data
# Start the MLflow run
with mlflow.start_run() as run:
    model_1.fit(X_train_preprocessed, y_train_encoded,
epochs=30, batch_size=32, validation_split=0.25,
callbacks=[history_callback], verbose=2)
    # Evaluate the model on the training data
```

```

    loss, accuracy =
model_1.evaluate(X_train_preprocessed, y_train_encoded)
    print(f'Train Loss: {loss:.4f}')
    print(f'Train Accuracy: {accuracy:.4f}')
    # Evaluate the model on the testing data
    loss, accuracy =
model_1.evaluate(X_test_preprocessed, y_test_encoded)
    print(f'Test Loss: {loss:.4f}')
    print(f'Test Accuracy: {accuracy:.4f}')
    # Get the epoch-wise training metrics from the
History callback
    train_metrics = history_callback.history
# Log the epoch-wise training metrics to MLflow
    for metric_name, values in train_metrics.items():
        for epoch, value in enumerate(values, 1):
            mlflow.log_metric(f'train_{metric_name}',
value, step=epoch)

```

An important consideration in the above code is the use of `history_callback.history` statement. This is a Keras object and attribute that contains information about the training process in a dictionary format, including various metrics such as loss and accuracy, recorded at the end of each training epoch. We log this information as the `MLflow` metric for traceability purposes. Another hyperparameter is the `Batch Size` which is the number of training examples in one forward/backward pass. The higher the batch size, the more memory space is needed.

After training for 30 epochs the model produces the following output:

Train Loss: 0.2736

Train Accuracy: 0.8998

283/283 [=====] - 1s 3ms/step
- loss: 0.2958 - accuracy: 0.8914

Test Loss: 0.2958

Test Accuracy: 0.8914

This is definitely closer to the Xgboost test accuracy results of 89.25%. Additionally, let us look at following [Figure 9.9](#) which shows the loss and accuracy training curves by epoch for the training and validation data. The code for the same is as follows:

```
plt.plot(history_callback.epoch,
history_callback.history['val_loss'], 'r',
         history_callback.epoch,
history_callback.history['loss'], 'g')
plt.title('model loss')
plt.ylabel('loss')
plt.xlabel('epoch')
plt.legend(['train', 'test'], loc='upper left')
plt.show()

plt.plot(history_callback.epoch,
history_callback.history['val_accuracy'], 'r',
         history_callback.epoch,
history_callback.history['accuracy'], 'g')
plt.title('Model Accuracy')
plt.ylabel('Accuracy')
plt.xlabel('Epoch')
plt.legend(['train', 'test'], loc='upper left')
plt.show()
```

Output:

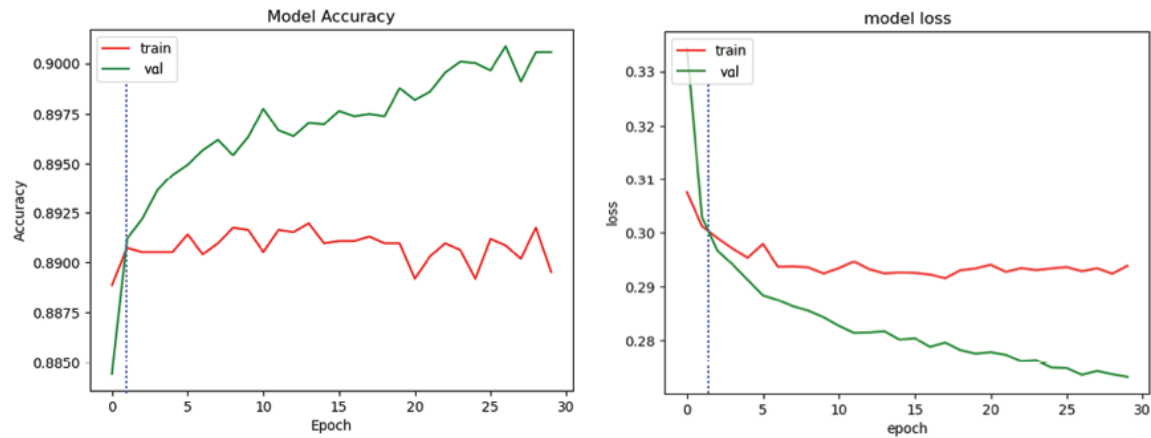


Figure 9.9: Loss and accuracy by epochs

It can be seen that the difference between the train and test accuracies starts to widen around the 3rd epoch. This demonstrates overfitting and we should stop training the model after the 3rd epoch. At this point, we have two options to prevent the model from overfitting. Firstly, we can re-run the above code by updating the epoch hyperparameter value to 3 and get the following results:

Train Loss: 0.2734

Train Accuracy: 0.9002

283/283 [=====] - 1s 2ms/step
- loss: 0.2965 - accuracy: 0.8903

Test Loss: 0.2965

Test Accuracy: 0.8903

Alternatively, instead of hardcoding the epoch value based on a previous visual loss and accuracy plotting, we can use `EarlyStopping` as a callback during the training process and update the `model.fit()` as shown below.

```
early_stopping = EarlyStopping(monitor='val_loss',
                                patience=5)
```

```
model.fit(X_train, y_train, validation_split=0.2,
          callbacks=[early_stopping], epochs=30)
```


The monitor parameter specifies the validation metric to monitor (for example, `val_loss` for validation loss), and patience is set to 5, indicating that the training will stop if validation loss does not improve for 5 consecutive epochs. The maximum possible number of epochs is set as 30. In this case, the training would typically stop by the 8th epoch instead of going onto the 30th epoch.

Anyway, we are seeing fantastic results with just 3 epochs of training and not using the `duration` feature while getting test results better than Xgboost.

As the next step, we run the following code to get a sense of metrics other than accuracy.

```
#Predict on Train data
y_pred_train = model_1.predict(X_train_preprocessed)
y_pred_train = (y_pred_train> 0.5)
print(classification_report(y_train_encoded,
y_pred_train))

Train_summary_train =
get_all_measurements(y_train_encoded, y_pred_train,
'ANN Model')

Train_summary_train
```

Output:

	Accuracy	Precision	Recall	F1-Score	AUC
ANN Model	0.900188	0.676048	0.268938	0.384799	0.626000

Figure 9.10: Training classification report

In [Figure 9.10](#), we can see that while the accuracy is high, AUC and Precision are not necessarily the best. This implies our model has overall high accuracy in determining 0s and 1s together, but predicting actual 1s as 1s happens only in 67.6% of cases. Is this an issue? That will depend on the business' appetite for sending emails to unlikely customers. More often than not businesses

would care about high accuracy in a marketing model rather than precision. The latter is a useful metric in a fraud scenario when predicting actual frauds (1s) with a high precision rate is a priority. At this juncture when we are talking about business objectives, let us shift gears to understand why it is important for business people to interpret model outcomes and explain how the model made a certain decision that aligns with the objective.

Interpretable and explainable AI using SHAP and PiML

We now have a high accuracy model but do we trust the decisions it is making in terms of fairness in targeting customers from protected classes? What are some of the important features in the decision-making overall (globally) vs. individual customer targeting(locally)?

We have previously discussed the SHAP Python library and we are going to explore some SHAP visualizations here again. Force plots, like the one shown below in [Figure 9.11\(a\)](#), allow us to see how the presence of different features in the model impacts the prediction for a specific observation. This is perfect for being able to explain to someone exactly how the model arrived at the prediction it did for a specific observation. The following code generates the force plot for two observations with index 0 and 3:

```
explainer = shap.KernelExplainer(model_1.predict,
X_test_sampled)

shap_values = explainer.shap_values(X_test_sampled)

shap.force_plot(explainer.expected_value,
shap_values[0][0], X_test_sampled.iloc[0])

shap.force_plot(explainer.expected_value,
shap_values[0][0], X_test_sampled.iloc[3])
```

Output:



Figure 9.11 (a): SHAP force plot-index 0

In the plot above, the bold **0.09** is the model's score for the 1st observation, where the final score is equal to the sum of the base value and the SHAP values of all features. The higher scores drive the model to predict the Target variable as '1'. The important features for this observation are shown in red and blue, with red representing features with a positive impact on the score, and blue representing a negative impact. Higher impact features are located closer to the dividing boundary between red and blue, and the size of that impact is represented by the size of the bar. The force plot for the record with `index=3` is shown below:



Figure 9.11 (b): SHAP force plot-index 3

When [Figures 9.11 \(a\)](#) and [9.11 \(b\)](#) are compared, the interaction between the SHAP and feature values of the two chosen observations is apparent. This comparison can help us determine that the feature `marital_married` has an opposite impact when the value is '1' vs. '0' in the two figures. Also, each observation is different as we can see that for [Figure 9.11 \(b\)](#), the feature `month_jul` has the highest negative impact and has the lowest positive impact in [Figure 9.11 \(a\)](#). This is one of the most impactful ways to showcase local behavior at the individual observation level and is the level of detail expected from an explainable AI solution as it makes predictions.

Another important plot in the SHAP library is the summary plot which explains the global behavior of the features. This involves feature importance ranking, feature impact by observation, original feature value, and feature correlation with the target. The following code generates one amazing transparent view of the model's inner workings. The SHAP values are only generated for the sampled test dataset since it is faster to run the code for a smaller set. These 10 SHAP values are plotted on the summary plot thereafter:

```
#Explain many predictions
```

```
shap.summary_plot(shap_values[0], X_test_sampled)
```

Output:

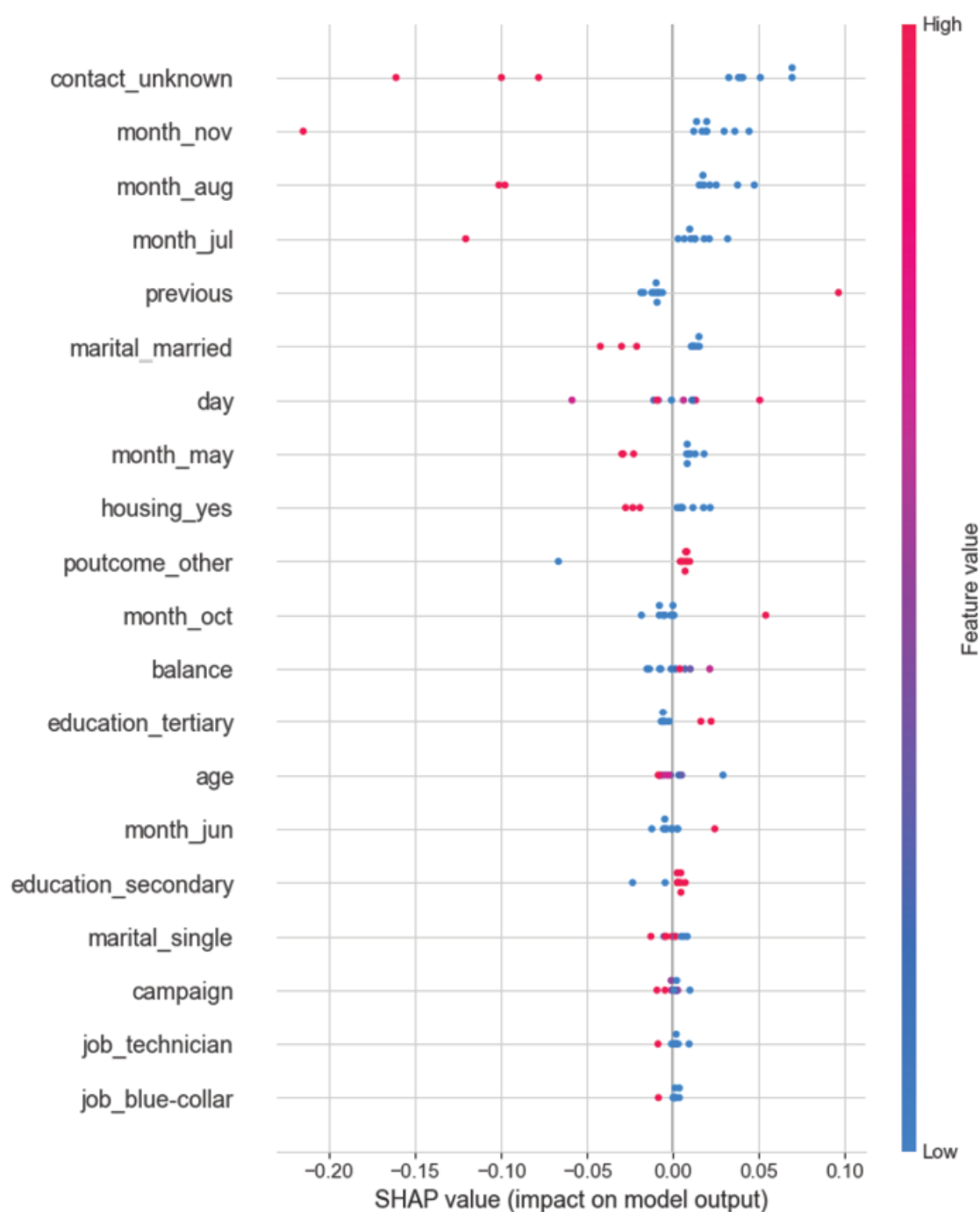


Figure 9.12: SHAP Summary plot

It is important to note that all 10 observations have been plotted for each feature on the summary plot above. The `contact_unknown` is

the most important feature and negatively impacts the target in the case of three observations (the three red dots to the left of the line). This implies a negative correlation and like in other regular scenarios, these SHAP values only indicate correlation and causation.

Lastly, let us look at the **partial dependence plot (PDP)** that shows the marginal effect of one or two features on the Target prediction across the whole dataset (or chosen sample - 10 records). The below code generates the PDP for the feature 'age':

```
# create a SHAP dependence plot to show the effect of a  
single feature across the whole dataset
```

```
sns.set_style("whitegrid")
```

```
shap.dependence_plot("age", shap_values[0],  
X_test_sampled)
```

Output:

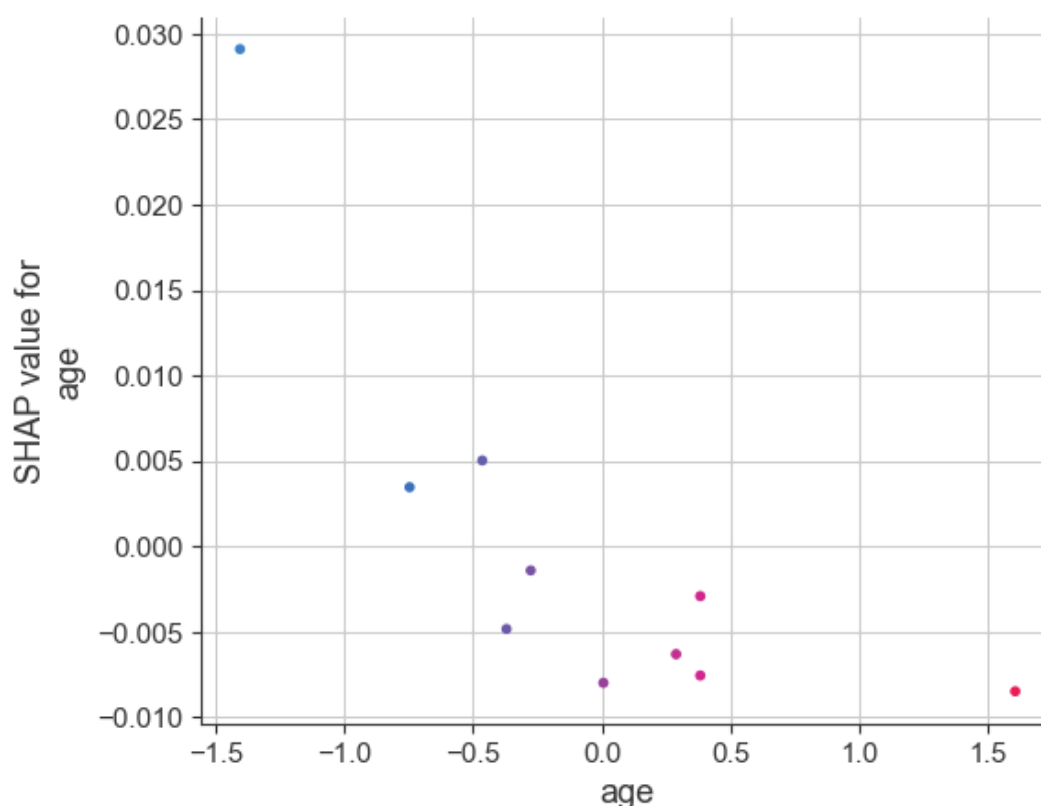


Figure 9.13: SHAP partial dependence plot

The preceding figure shows a negative relationship between age and the target variable because a higher value of `age` is related to the most negative SHAP values.

In addition to explainability, a key requirement in assessing model fairness is understanding if the model is exhibiting any biased behavior towards a protected group based on age, race, or gender. We may want to investigate pockets of data (sensitive attribute groups) for patterns and differences in feature contributions. If certain attributes consistently lead to higher or lower contributions, it could indicate potential bias. However, SHAP values are not enough on their own and we would have to utilize fairness metrics to quantify and assess fairness. These metrics, such as **adverse impact ratio (AIR)**, compare the predictions across sensitive attribute groups (males vs. females) and indicate if there's a significant difference in outcomes. AIR is defined as the ratio of the protected class approval rate and the control group approval rate.

With this thought, we conclude our study of the SHAP values with respect to this case study but consider this an important tool for all your AI exploration in the future. SHAP is a post hoc explainability library, meaning it explains the behavior and decisions of a machine learning model after it has been trained. Hence, model behavior can only be changed through data augmentation, retraining, etc. Regardless, this approach answers all the questions such as how, why, and what around the black-box AI models, and hence builds trust in the solution.

Before we end this case study, let us explore some features of another library called PiML that go beyond the post hoc SHAP capabilities. PiML helps build inherently interpretable models as it allows for testing for model prediction reliability, robustness (sensitivity to noisy data), resilience (data distribution drift), and weak spots during training itself. This is a pathbreaking tool, primarily because it has been built by industry practitioners, who work at the forefront of the explainable AI challenge in one of the large US banks. They understand the landscape and regulations that demand such answers around model conceptual soundness from the AI models.

There are multiple ways to implement this work in our AI solutions, but we will use the low code method to showcase some exciting features for now. We will encourage readers to explore the library online as they build confidence with basic ANN modeling steps.

PiML can help with data preprocessing, EDA, and model training before you can use other functions to explain model decision-making and diagnose model reliability, robustness, and resiliency with just one line of code. The following code set helps set up the experiment and eventually train the model in the Jupyter or Colab notebook:

```
from piml import Experiment
exp = Experiment()
exp.data_loader()
exp.data_summary()
exp.data_prepare()
exp.feature_select()
exp.eda()
exp.model_train()
```

[Figure 9.14](#) shows the output of the just the `model_train()` function for brevity purposes:

Choose Model

☐ GLM	o
☐ GAM	o
☐ Tree	o
☐ FIGS	o
☐ EBM	o
☐ XGB1	o
☐ XGB2	o
☐ GAMI-Net	o
☒ ReLU-DNN	o

Customize Model

Model name:

Layer_sizes:

Max_epochs:

Learning_rate:

Batch_size:

L1_regularization:

Dropout_rate:

Early_stop_epochs:

Random_state:

Rank Metric:

Leaderboard

	Model	test_ACC	test_AUC	test_F1	train_ACC	train_AUC	train_F1	Time
0	ReLU-DNN	0.8882	0.7368	0.1918	0.8909	0.7549	0.2029	4.1

Figure 9.14: PiML model training user interface

This statement allows us to choose from a range of modeling techniques and set hyperparameters for them individually. We chose the ReLU DNN and set some basic hyperparameters to get similar results as the ANN model we trained using Keras. This is followed by the model diagnostics run using the following code:

```
exp.model_diagnose()
```

The output looks like the as follows:

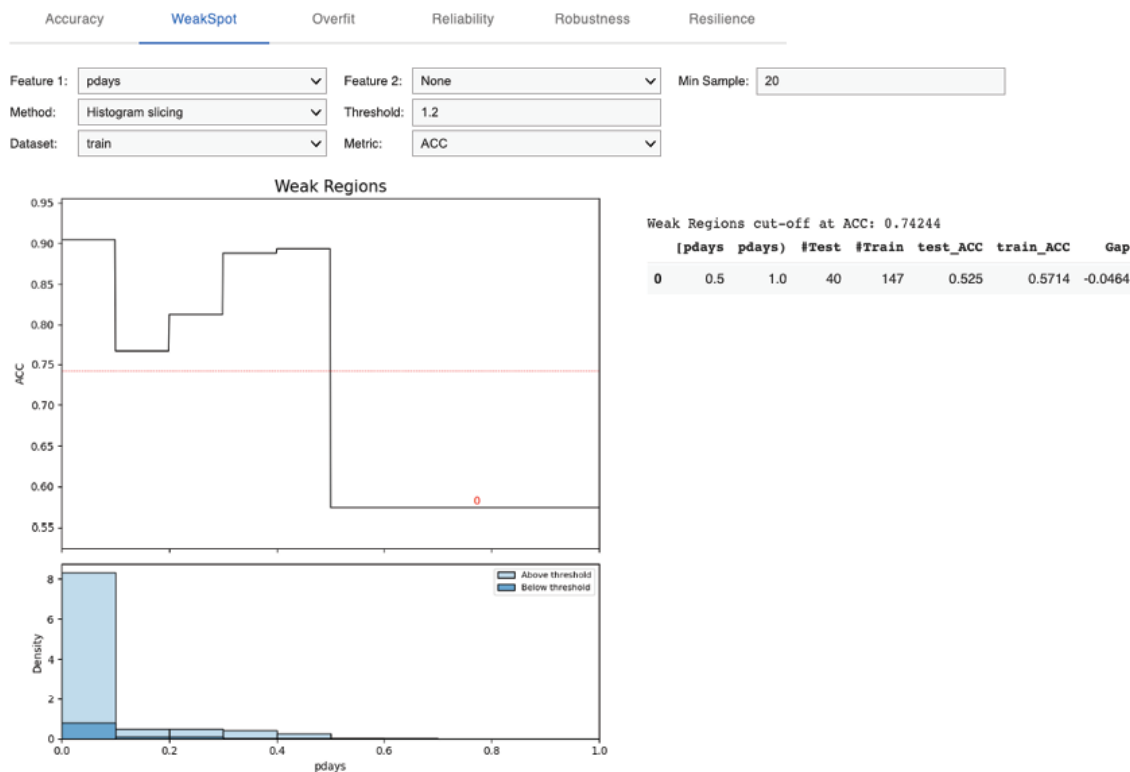


Figure 9.15: PiML model diagnostics features

An important diagnostics to look at in [Figure 9.15](#) is WeakSpot which identifies areas or regions where a model is prone to underperform or make incorrect predictions. These weaknesses can arise due to factors like inadequate or biased data, overfitting, the use of unsuitable algorithms, or insufficient model complexity. We can see in Figure 9.15 that the model underperforms with a training accuracy of 0.5714 when $0.5 < \text{'pdays'} < 1$.

The PiML library has a lot to offer in terms of model explainability, and diagnostics in a low-code environment. We conclude this case study here, but detailed Python code is available in the GitHub repository.

Conclusion

Feed Forward NNs have uncovered the fascinating synergy between mathematical principles and the replication of human cognition. Through this chapter, we have demystified the intricate

layers of neurons, their connections, and the research that gives birth to AI's self-learning prowess.

The phone marketing campaign case study serves as a testament to the ANNs' immense potential to decipher patterns in tabular data. This may not be the best use of the transformative power of this technology but such applications are evidence of the tangible difference ANNs have over other traditional methods that need extensive feature engineering.

However, amidst the complexity lies the necessity for clarity. We've highlighted the significance of explainable AI, using both post hoc techniques like SHAP, and inherent interpretability techniques such as PiML. Just as understanding the "why" behind a child's question fosters trust and learning, comprehending ANNs' decisions is paramount for their responsible deployment. This chapter underscores that the quest for AI's power should be harmonized with the pursuit of transparency and accountability.

In a world where technology drives progress, let us be the custodians of not just innovation but understanding. As we close this chapter, let us remember that unlocking the full potential of ANNs does not just lie in their complexity, but in their ability to be understood and harnessed for the betterment of our world. In the next chapter, readers will learn to use a variation of neural networks, that is recurrent neural networks, on unstructured text data.

Points to remember

- The terms **Artificial Neural Network (ANN)**, **Deep Neural Network(DNN)**, and **Feed Forward Neural Network(FFNN)** are used interchangeably but all indicate a model architecture with multiple layers, that organize neurons into input, hidden, and output layers.
- These layers decipher complex non-linear relationships with the help of activation functions deployed at every neuron node.
- Choose an appropriate loss function based on the problem type: **Mean Squared Error (MSE)** for regression and

Cross-Entropy for classification.

- Gradient Descent variants (SGD, Adam, RMSprop) update the network weights to minimize the loss function during backpropagation. Loss minimization is governed by the learning rate hyperparameter that affects the convergence speed
- Backpropagation computes the partial derivatives $\partial C_x / \partial w$ and $\partial C_x / \partial b$ for a single training example. We then achieve $\partial C / \partial w$ and $\partial C / \partial b$ by averaging over all the training examples.
- Hyperparameters like the number of layers, neurons in each layer, learning rate, number of training epochs, and batch size affect model performance.
- Use early stopping criteria to avoid model overfitting by detecting the growing differences between train vs. validation metrics
- Use K-fold cross-validation along with other NN-specific regularization techniques such as dropout and batch normalization for a more comprehensive assessment of how well the model generalizes to different subsets of the data. Its specific advantage is that it ensures that each data point is used for both training and validation exactly once.
- **Explainable AI (XAI)** is an important research area that has been guiding the development and trust around AI techniques such neural network
- Use techniques like SHAP or LIME to interpret the model's predictions. Transparency is critical for black-box models that enable faster adoption
- The SHAP values do not identify causality, which is still better identified by experimental design or similar approaches.
- SHAP helps in post hoc analysis such as model performance explainability during the inference phase but tools such as PiML help with model performance diagnostics during the model build phase.

- Revisit or monitor these black-box models to fine-tune them as new data becomes available or business requirements change.

Join our book's Discord space

Join the book's Discord Workspace for Latest updates, Offers, Tech happenings around the world, New Release and Sessions with the Authors:

<https://discord.bpbonline.com>



CHAPTER 10

Language Modeling with Recurrent Neural Networks

Introduction

Language Modeling is one of the five senses that AI is developing to unlock the potential for machines to truly understand, reason, and converse like humans.

Language has led humans through the complex maze of cultures, thoughts, and ideas. It has served as the medium through which cultural norms, values, and traditions are passed from one generation to the next by communication of complex ideas, emotions, and intentions with one another. Language modeling takes this communication of ideas from amongst humans to between humans and machines. This need makes understanding and modeling language a fundamental pursuit in the realm of artificial intelligence and **natural language processing (NLP)**.

This chapter leads us on a journey into the heart of language modeling, exploring the ingenuity of **recurrent neural networks (RNNs)**, the memory-enhanced prowess of **long short-term memory networks (LSTMs)**, and the

transformative power of cutting-edge transformer-based text modeling techniques.

Structure

In this chapter, we will learn the following topics:

- Language modeling
- Background
- Under the hood of language modeling
 - Data: From spoken languages to modeling datasets
 - Model: The language with context
 - Recurrent neural networks
 - Long-short term memory
 - Loss: Quest for the best model
- Challenges and assumptions related to text data and model
- Case study: Customer complaint classification explained with LIME
- Rise of transformers: A primer on BERT and GPT

Objectives

This chapter's objective is to revisit deep learning mechanisms and their applications in language modeling using **recurrent neural networks (RNNs)** and their variants. The chapter will introduce the key concepts of RNNs, such as the recurrent structure and the use of hidden states, and how they work to process and analyze sequential data such as text. It will also cover the other advanced RNN architecture, that is, **long short-term memory (LSTM)**, and discuss their application to different types of language

modeling tasks, such as text generation and classification. This will include training and optimization of RNNs, including the use of loss functions and regularization techniques, and evaluating the performance of RNN models on language datasets.

Furthermore, we will dive into the revolutionary era of transformers, which have not only shattered benchmarks but also reshaped the landscape of NLP through their attention mechanisms and scalability. By the end of this chapter, the readers will have a comprehensive understanding of deep learning and its applications in language modeling using RNNs and related techniques while working on a case study, with the tools and techniques used to improve the performance and accuracy of these models.

Language modeling

Language modeling is a foundational concept in NLP that has evolved significantly, with deep learning models like RNNs, LSTMs, and transformers leading to breakthroughs in various NLP applications. The primary goal of language modeling is to capture the structure and patterns of human language, allowing machines to understand and generate text that is coherent and contextually relevant. This enables a wide range of language-related tasks and applications, including machine translation, text generation, sentiment analysis, named entity recognition, question answering, text summarization, and chatbots, among various others. Language modeling is an active area of research, with continual advancements in architecture, training techniques, and applications. Researchers are constantly working to improve model efficiency, interpretability, and generalization.

Background

Human language is a complex, structured communication system that comprises syntax (grammar) for organizing words into sentences, semantics for conveying meaning, and phonology for sound patterns. For instance, let us consider the sentence *Man made money, money made man mad*. From a simple syntactic structure with a subject-verb-object pattern, *Man* is the subject, *made* is the verb, and *money* is the object. The second part, *Money Made Man Mad*, maintains this structure. The sentence also carries a metaphorical meaning(semantics). The first part suggests that humans created the concept of money, while the second part implies that the pursuit or influence of money can negatively affect human behavior, leading to madness. Lastly, the sentence exhibits phonological elements such as rhyme between *made* and *mad*.

A deep learning architecture such as RNN and LSTM is considered well suited to understand such structures because they are designed to handle human language sequential patterns described in the above example effectively. LSTMs, in particular, address the long-term dependencies in human sentences well. For instance, *Dog jumped the fence, which was two meters high*. LSTMs can figure out that it is the fence, which is two meters high, and not the dog. They can do this using memory cells that can store and update information over longer sequences, making them capable of capturing long-term dependencies in text. This capability is especially essential when understanding the meaning of a word depends on the context of the surrounding words. RNNs and LSTMs excel at capturing this contextual information at varying levels, enabling them to understand the semantics of sentences and documents.

The following sections will unravel the inner workings of an RNN and LSTM while deciphering human language to deliver NLP tasks.

Under the hood of language modeling

In this section, we will deep dive into the data requirements and transformations that enable the NLP tasks to deliver the intended output. This will be followed by unraveling the architectures that Excel at capturing sequential dependencies found in text data and are foundational in many NLP applications such as text classification, summarization, prediction, and so on. Let us dive in.

Data: From spoken languages to modeling datasets

Data requirements for NLP tasks are unique. We know that structured data modeling is affected by the correlation between input data features for a dataset. Similarly, the text data analysis needs a better understanding or representation of the semantical and syntactical information between the words in a sentence. Additionally, deep learning models only work on numbers, not sequences of texts.

For this reason, the process of feature extraction from long strings of text happens in two steps. Firstly the text pre-processing steps like raw text cleaning, tokenization, stemming, and lemmatization help convert sentences into an array of unique words. Next, the words are converted into *word embeddings*, which are numerical representations of the unique words that capture both syntactical and semantical information related to these words. This is done by mapping words to vectors in a high-dimensional space where similar words are close to each other. The vector representation of words or tokens is then fed to the machine-learning model for further processing.

This implies we need a way to extract meaningful numerical feature vectors, that is word embeddings. Interestingly, there is no one best method to create word embeddings, and the choice of word embedding generation method would depend

on the specific NLP task. Words can be simply converted either into one-hot vectors or more advanced word embeddings like Word2Vec, GloVe, SpaCy, fastText, or embeddings from transformer-based models such as **Bidirectional Encoder Representations from Transformers (BERT)**.

There are other feature extraction methods, such as **Bag of Words (BoW)**, TFIDF, and `CountVectorizer`, but they lack syntactical or semantic information since they store just the word frequency. On the other hand, word embeddings reduce input data dimensionality by treating sparsity, helping predict the words around a specific word, and capturing the interword semantics. [Figure 10.1\(a\)](#) shows a BoW illustration for the two phrases *Man made money* and *Money made man mad*.

	Man	Made	Money	Mad
Phrase 1	1	1	1	0
Phrase 2	1	1	1	1

Figure 10.1 (a): Illustrative BoW representation

The embedding matrix in [Figure 10.1 \(a\)](#) has four columns equivalent to the total vocabulary word count. Each phrase is portrayed as a blend of words either present or absent.

For instance, in a dataset with a vocabulary size of 100, an input sentence of length 5 transforms into a vector of size 100. The bits corresponding to the 5 words in the sentence are activated, while the remaining 95 bits remain inactive. Noticeably, this process creates a sparse matrix which is not efficient.

Contrary to this, Word2Vec adopts a distinct approach by considering each word individually instead of representing entire sentences or phrases as entities. A finite-dimensional

embedding space is chosen, with each row signifying a word in the vocabulary. Please note that Word2Vec has two neural network-based variants: **Continuous Bag of Words (CBOW)** and Skip-gram. The fundamental concept underlying Word2Vec is that a word's representation can be constructed by considering the words in its proximity, within a defined window size. Following this definition, it is intuitive to employ two vectors for characterizing a word: the center word vector (V) and the context word vector (U). This is because a single word may serve both as the center word and as the context word for another central word. Skip-gram model predicts a context word given a center word while the **Continuous Bag of Words (CBOW)** model predicts a center word given a bag of context words.

Throughout the Word2Vec training process, each word acquires a value (or weight) for each dimension, constituting its vector representation. These weights are influenced by the context and context window of each word but typically ignore that the same string of letters may have different senses (dining table vs. table of contents). Therefore Word2Vec is termed a static word embedding because it collapses different contexts into a single vector. On the contrary, BERT will generate two different vectors for the word table being used in two different contexts. One vector will be similar to words like dining, desk, and so on. The other vector would be similar to vectors like books, memorandums, and so on.

As a result, phrases like *The dining table is long* and *Book had no table of contents* lead to the association of the word *table* with *dining* and *book* in two different ways in the embedding space. [Figure 10.1 \(b\)](#) shows the illustrative BERT embedding for the word 'table' in two different contexts:

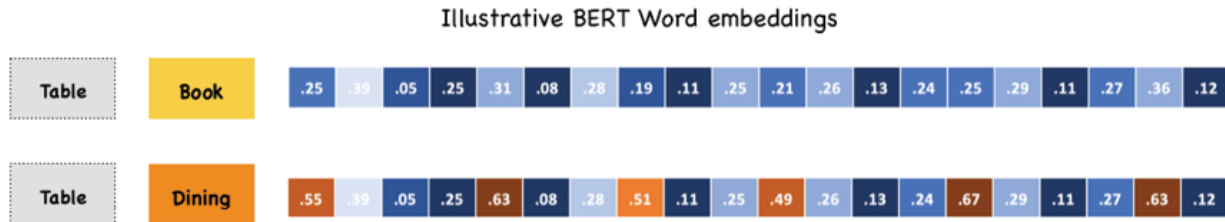


Figure 10.1 (b): Illustrative Word Embeddings for the word “table” in two contexts

Please note that the size of these Word2vec vectors depends on the vocabulary size among other factors such as the nature of the language being modeled, the task at hand, and the available computational resources. A larger vocabulary may require a higher-dimensional embedding space to capture diverse semantic relationships among words. However, this does not necessarily mean that the embedding dimension should be directly proportional to the vocabulary size. Ultimately, the best approach is to experiment with different dimensionalities and evaluate the performance of our model. Most common dimensionalities are 50 or 300 but higher or lower dimensionalities can also be experimented with. Additional information on Word2Vec and BERT is available in the *Further Reading* section at the end of this chapter.

With this let us understand the **Consumer Financial Protection Bureau (CFPB)** consumer complaint database which is an open-source dataset containing 4,086,377 records with 18 attributes. This dataset is updated daily and each record represents a complaint received by a company/bank. The consumer’s narrative description of their complaint is published if the consumer opts to share it publicly and after the Bureau has taken steps to remove the customer’s personal information. We will be filtering out a smaller sample for case study purposes since RNN and LSTM can take a long time to train on such a large dataset. Some selected attributes and sample records are shown in [Figure 10.2](#):

Complaint ID	Date received	Product	Sub-product	Issue	Sub-issue	Consumer complaint narrative	Company public response	Company	State	ZIP code
2567810	2017-07-07	Vehicle loan or lease	Loan	Getting a loan or lease	Confusing or misleading advertising or marketing	NaN	Company has responded to the consumer and the ...	SYNCHRONY FINANCIAL	TX	79930
3668797	2020-05-26	Credit reporting, credit repair services, or o...	Credit reporting	Improper use of your report	Reporting company used your report improperly	NaN	Company has responded to the consumer and the ...	TRANSUNION INTERMEDIATE HOLDINGS, INC.	TN	37391
3362527	2019-09-04	Credit reporting, credit repair services, or o...	Credit reporting	Unable to get your credit report or credit score	Problem getting your free annual credit report	XX/XX/2019 XXXX, XXXX, Experian all refuse to ...	Company has responded to the consumer and the ...	Experian Information Solutions Inc.	CA	90804
5767711	2022-07-14	Credit reporting, credit repair services, or o...	Credit reporting	Improper use of your report	Credit inquiries on your report that you don't...	NaN	Company has responded to the consumer and the ...	Experian Information Solutions Inc.	FL	32839

Figure 10.2: CFPB consumer complaint dataset

The fields of interest for this analysis are the *consumer complaint narrative* and the **Product**. The CSV file can be downloaded here:

<https://www.consumerfinance.gov/data-research/consumer-complaints/#download-the-data>.

For modeling purposes, we need to do a couple of things to transform the data:

- Text preprocessing including stop words and punctuation removal
- Text tokens transformed into word embeddings

We will talk about these in detail in the case study section.

Model: The language with context

With a fair understanding of the data and its processing, let us understand the deep learning model architectures that

will help us make sense of the input word vectors. In this chapter, we will be focusing on the RNN and its variant LSTM. However, we will also be briefly discussing another popular architecture called Transformers. These have transformed the language modeling landscape for good by enabling large language models called **Chat Generative Pre-Trained Transformer (ChatGPT)**. Other famous transformer models are BERT, RoBERTa, ALBERT, DistilBERT, and BART.

Recurrent neural network

Models need memory to understand the context of a specific word in a sequence. In traditional NNs, we feed all inputs at the same time because there is no time dependency between inputs, but if we want to predict the next word in a sentence, we need to know the words that came before it. RNNs solve this problem by repeatedly performing the same task for every element of a sequence, with the output being dependent on the previous computations (hidden state or memory) and current input value. This hidden state is updated every time a new element is encountered by the model.

Let us understand this using an example. If we try to spell the word *Bank*, our brain starts with the first letter *B* and no context. However, we at least need the first letter to be able to predict the second letter *a* with some confidence.

Similarly, the third letter *n* is dependent on the fact that the first letter was *B* and the current input (think of it as a clue) letter *a*. Essentially a context needs to be retained for the third prediction by the brain and this also means our brain possesses a dynamic hidden state to remember a series of letters. Imagine if we had learned or memorized the spelling earlier, the third letter in the sequence would have occurred to us easily, and the probability of uttering *n* as the third letter would have been high. This implies a better-hidden state (memory) leads to better future prediction and

continuous interaction between the previous hidden state (a memory vector) and the current input drives the eventual sequence of letters that is *Bank*.

Let us translate this understanding into an RNN architecture as shown in [Figure 10.3](#):

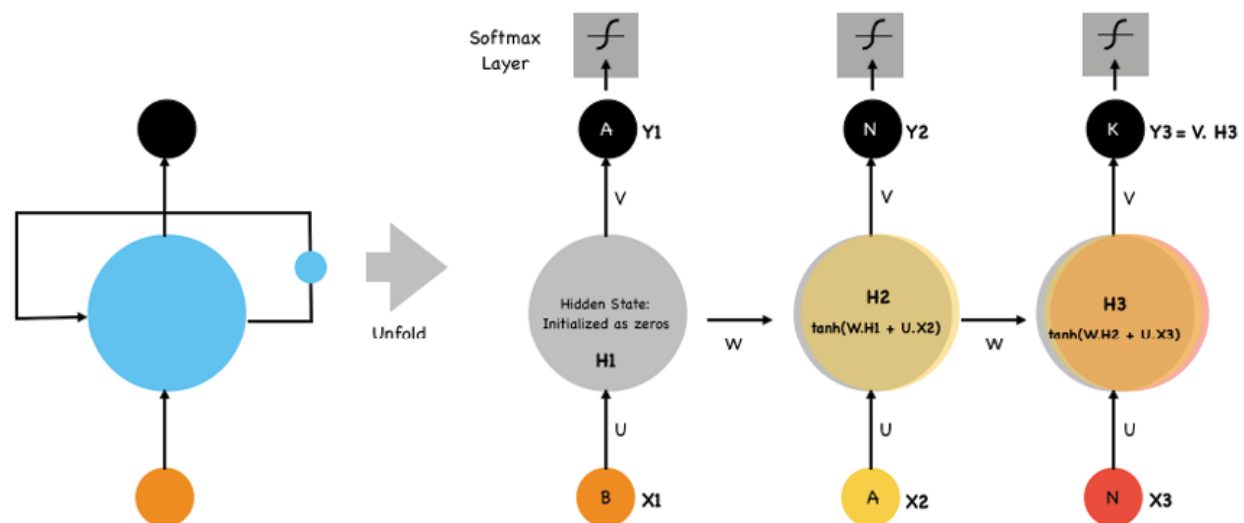


Figure 10.3: Predicting characters of a word using a RNN

The core concept of an RNN is a loop, which, when unfolded, acts as a straight information (hidden vector memory) highway from one step to the next. Additionally, it can be seen in [Figure 10.3](#) that the RNN architecture consists of an input layer (taking encoded words/characters), a hidden layer (which maintains a hidden state), and an output layer (which predicts the next word/character).

To guess the last letter k in the sequence *Bank*, the first step is to feed B into the RNN and initialize the hidden state with zeros. The RNN encodes B and produces an output, which is a prediction for the next letter. For the next step, we feed the letter a and the hidden state from the first step together to update the hidden state again. This way it takes each character, one at a time, and updates its hidden state based on the current input character and the previous hidden state

over and over until all characters are processed. The update of the hidden state is represented by the following equation:

$$H_t = \tanh (W.H_{t-1} + U.X_t)$$

- U represents the weights associated with the current input X_t .
- W represents the weights associated with the previously hidden state H_{t-1} .
- tanh is an activation function that introduces non-linearity in the update equation by squashing the activation range between -1 and 1.

Let us assume that after processing all characters in the sequence, the RNN is in a hidden state H3, shown in [Figure 10.3](#), that represents the context of the input sequence *Ban*. To predict the next character k , it will use the final hidden state H3 as the context and feed it through a softmax layer in the output. The softmax layer produces a probability distribution over all possible characters in the vocabulary. Please note that the softmax layer is a crucial component of the RNN output. It is applied to the output vector at each time step to convert it into a probability distribution.

It's important to note that in practice, RNNs are often more complex, and training involves sequences much longer than an eight-character example discussed here. Additionally, the choice of loss function, learning rate, and other hyperparameters plays a significant role in training an RNN for language modeling. Longer sequences can lead to a problem known as the *vanishing gradient* that hampers the RNN's ability to capture long-range dependencies, causing difficulties in maintaining context over extended sequences.

Similar to [Chapter 9, Structured Data Classification using ANN](#), we talk about vanishing gradient in the context of backpropagation but a modified version called **backpropagation through time (BPTT)**, which allows the

RNN to train its weight based on the output error. As can be seen below in [Figure 10.4](#), going back in every time stamp to update the weights is the reason it is known as Backpropagate through time:

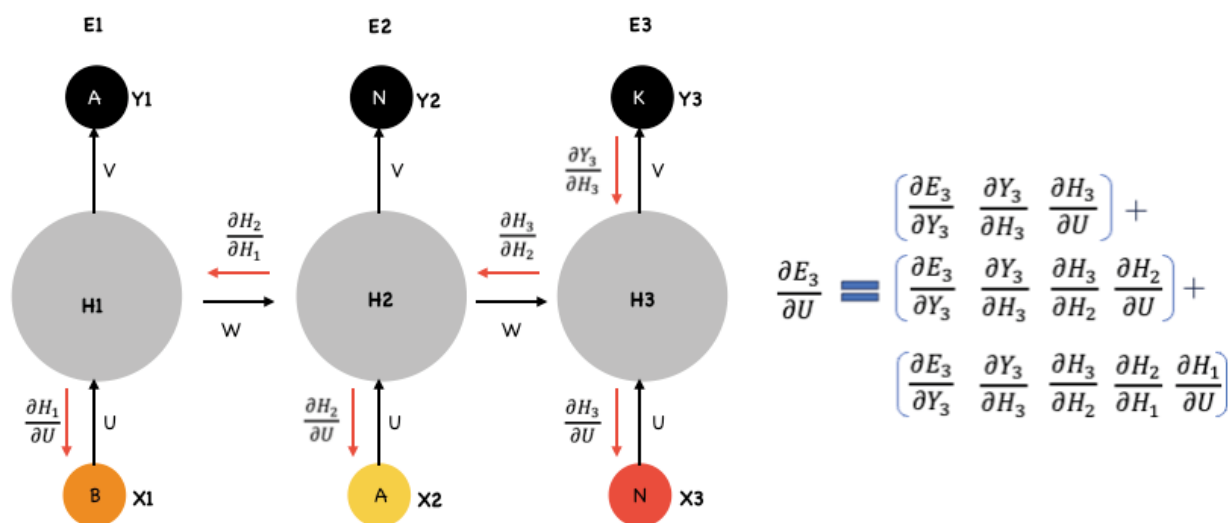


Figure 10.4: Backpropagation through time

It can be observed in [Figure 10.4](#) that a gradient value becomes extremely small (vanishes) as it goes back in time, it does not contribute too much to learning or weight update. This is the problem that will be solved using other sophisticated RNNs, such as LSTMs or **Gated Recurrent Units (GRU)**.

Long short term memory

This variant of RNN has the architectural design to learn long-term dependencies in text data and hence solve for the vanilla RNN's limitation of vanishing gradient. This is achieved by *Memory cells*, which can store data for a long time, and *gates*, which regulate the information flow into and out of the memory cells, making **Long-short Term Memory (LSTM)** networks good at understanding long-term dependencies. LSTMs can choose what to remember and forget making them beneficial for input sequences that can

be very long, and the elements' dependencies can extend over numerous time steps. This is true for language translation, speech recognition, text generation, and image captioning tasks.

Let us see how the LSTM network processes information for prediction:

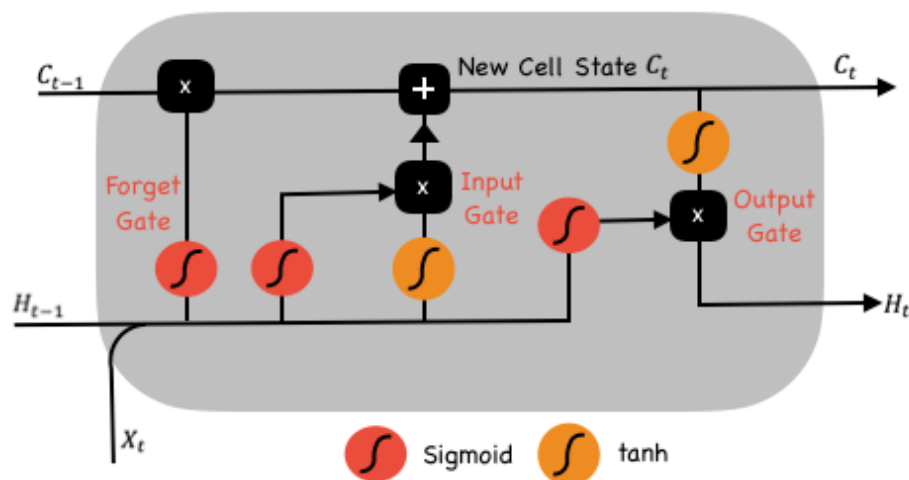


Figure 10.5: Long-short-term memory architecture

As seen in [Figure 10.5](#), the LSTM network has three gates and the cell state at its core. The cell state carries the information throughout the sequence processing and the three gates learn to keep or forget the relevant information during training using the use of sigmoid and tanh activations. Let us understand the role of the three gates first. The three gates are:

- **Forget gate:** Vector information from the current input and the previous hidden state passes through the sigmoid function to result in values ranging between 0 and 1. The closer to 0 value indicates a *forget*, while closer to 1 means to retain the information.
- **Input gate:** The primary role of the input gate is to update the cell state based on the current input. In this regard, the first operation is similar to the forget gate

wherein the previous hidden state (H_{t-1}) and current input (X_t) are passed into a sigmoid function. This updates the input values to be between 0 and 1, indicating 0 as not important, and 1 as important. The second operation involves obtaining the tanh transformation of the hidden state (H_{t-1}) and current input (X_t). The tanh activation function produces values between -1 and 1. It is often referred to as the *memory gate* because it controls how much of the current cell state (memory) should be updated. A value of -1 means *forget* or discard the memory, a value of 0 means *maintain the memory as is*, and a value of 1 means *completely update the memory*.

Lastly, the multiplication of the sigmoid and tanh outputs allows the LSTM to compute an update to the cell state that is a weighted balance between forgetting the old memory and adding the new input information. It essentially controls how much of the memory should be preserved and how much should be replaced. Eventually, the sigmoid output will decide which information is important to keep from the tanh output.

- **Output gate:** The output gate decides the next hidden state, which in turn is also used for predictions. Firstly, the previous hidden state and the current input are passed into a sigmoid function in parallel with the newly modified cell state being passed to the tanh function. The tanh output and the sigmoid output are multiplied to decide the information the hidden state should carry over to the next time step.

With an understanding of the three gates, let us see how gates operate on the cell state.

- **Cell state:** Operationally, the cell state gets multiplied by the forget vector first. This step can potentially eliminate values in the cell state if it gets multiplied by

values closer to 0. This multiplication operation is followed by an addition operation with the input gate output, which updates the cell state to new values. This creates a new cell state.

It is important to note that both the cell state and the output from the output gate that is the hidden state pass over to the next time step. The hidden state from the last LSTM block can be used for prediction using a feed-forward neural network.

To sum it up, LSTMs were created as a method to mitigate short-term memory using mechanisms called gates. Gates are just neural networks that regulate the flow of information flowing through the sequence chain.

Loss: Quest for the best model

Calculating the error in an RNN is slightly different from the regular NN because we have multiple outputs in RNN at each timestamp. Thus, a regular backpropagation algorithm is not suitable here and requires modification. This new algorithm is called **backpropagation through time (BPTT)**, which updates the three weights W - WHH , V - WYH , and U - WXH shown in [Figure 10.4](#). These weights do not change across the timestamps and are updated during the BPTT process.

So effectively the question in BPTT should be, what is the loss we are trying to minimize and how is it minimized by updating the three weights W , U , and V ?

We can answer the first question on the total error by saying the total error is equal to the summation of all the errors across timestamps t . At each timestamp, the derivative of the error with respect to the V is straightforward, but the backward pass goes through the entire network in reverse order, starting from the last time step and working its way backward to the first time step. This makes it slightly complicated since at each time step during the backward

pass, the gradients of the loss function with respect to the model parameters (weights and biases) are computed. These gradients are accumulated as you move back in time through the sequence. Please refer to [Figure 10.4](#) for a BPTT visual of this process. Please note that the entire process of the forward pass, loss calculation, backward pass, and parameter updates is typically repeated for multiple sequences or batches of sequences to improve the model's generalization.

After completing the backward pass through the entire sequence, the accumulated gradients are used to update the model's parameters. This is typically done using an optimization algorithm like **stochastic gradient descent (SGD)** or one of its variants (for example, Adam or RMSprop). In practice, BPTT is applied to multiple sequences in a mini-batch fashion. Gradients are computed and accumulated for each sequence in the mini-batch, and the model parameters are updated after processing all sequences in the batch.

It is important to note that both the vanilla RNN and LSTM use BPTT, but the design of LSTM allows the gradients to flow more freely through the network, preserving important information over many time steps. This mitigates the vanishing gradient problem.

Challenges and assumptions related to text data and model

We have already discussed the challenges of RNN leading to the development of LSTMs that can do a better job of predictions over longer sequences. This RNN issue is driven by the vanishing and exploding gradients problem caused by BPTT. Techniques like gradient clipping, weight initialization strategies, and the use of specialized RNN architectures like

LSTMs and GRUs have been developed to mitigate these issues.

Additionally, the sequence length can significantly impact the training process. Longer sequences may require truncated BPTT, where gradients are computed over a fixed window of time steps rather than the entire sequence. This can make it more memory-efficient but may result in reduced learning capacity. This is done because BPTT can be computationally expensive as the number of timesteps increases. For instance, if the input sequences are comprised of a thousand timesteps, then this will be the number of derivatives required for a single weight update.

Successful training with BPTT requires thoughtful tuning of hyperparameters such as learning rate, batch size, and sequence length.

Case study: Customer complaint classification explained with LIME

As part of the case study, we are going to use the dataset obtained from the CFPB's consumer complaint database shown in the *Data* section earlier. The goal of this case study is to accurately classify consumer complaints into specific product categories, allowing for faster and more efficient resolution of these complaints by respective departments in a Bank. Imagine having to read every comment and route it to the concerned department.

For this use case, we have subsetting the data to only include complaints related to *Debt Collection*, *Credit Card*, *Bank Accounts*, and *Other Financial Services*. These were filtered out to retain just 15,000 records out of the total 4,086,377. This is primarily done for faster training purposes. [Figure 10.6](#) shows the complaint counts by `Product` type found in the filtered dataset:

```
#Product categories in the modeling dataset
data_v2.Product.value_counts(dropna=False)
```

```
Debt collection          9671
Credit card or prepaid card  4660
Bank account or service    654
Other financial service     15
Name: Product, dtype: int64
```

Figure 10.6: Product type distribution

The **Issue** complaint category or the **Product** will be the **Target** variable with the actual **Consumer complaint narrative** text as the text input used for prediction:

```
data_v3.head(2)
```

	Consumer complaint narrative	Product
1963953	I can not access my Ally savings account via o...	Bank account or service
2908451	Account # unknown To Whom It May Concern : Be ...	Debt collection

Figure 10.7: Dataset structure

This raw text data in [Figure 10.7](#) needs some cleaning before getting converted to numerical vectors or embeddings that RNN models can process. This process is called **vectorization** and will require the removal of symbols, punctuation, and so on. The following `clean_text` function will do exactly that on the `Text` column:

```
# Removes all symbols and punctuation from the text
def clean_text(test_string):
    new_string = re.sub(r'[!"#$%&\'()*+,-.\/:;<=>?
@\[\\]^_`{|}~][\d]+', "", test_string)
    return new_string

data_clean['Text'] =
data_clean['Text'].apply(clean_text)
```

Once the clean data is available we start the vectorization process for the `text` column containing the complaints. In this case study, we will explore two vectorization techniques, namely Word2Vec and BERT.

Word2vec is a shallow neural network, developed by Google in 2013, trained with all examples of a target word and its neighbors as inputs. The network weights between the input and hidden layer are saved as the embedding vector for the target word. In contrast, Google released BERT in 2019 with a 12-layer deep neural network model trained to contextually understand language to compute word embeddings.

Essentially, both BERT and Word2vec can be used to represent text data but have very different model architectures. This leads to the most prominent difference between static vs. context context-sensitive word representation of the two techniques. Word2vec saves only one vector for a word in the final trained model, which means the *river bank* and *bank deposit* are represented similarly, based on the one-time training. BERT solves this problem of static context by generating word vectors that are contextual and driven by the current input sentence. This is achieved by the pre-trained bidirectional representations from the unlabeled text that can be fine-tuned with just one additional output layer to create state-of-the-art models for a wide range of tasks.

Word2Vec is known to produce good results in the case of semantic analysis use cases, such as Customer complaints analysis, where the contextual meaning conveyed by the sentence is important. However, since BERT is conceptually superior to Word2Vec given dynamic contextual embeddings, the case study would compare the classification results based on these two word embeddings.

Alright, with this knowledge, let us write some code to develop two vectorized (Word2vec and BERT) text datasets for RNN and LSTM model training. This way we will train four models overall that will help contrast the performance (accuracy) across these models.

Lastly, we will pick the best model to delve into the model explainability using the **Local Interpretable Model-Agnostic Explanations (LIME)** technique. It is a methodology designed to interpret the predictions of machine learning models, providing insights into their decision-making processes on a local (observation or record) level. In the context of NLP, LIME generates interpretable explanations for individual predictions by perturbing (removing or replacing certain words) input instances and observing the corresponding changes in model outputs. This allows practitioners to examine and verify individual predictions, ensuring the model aligns with human intuition and expectations.

Let us start with the below code that leads to a Word2vec-driven vectorized dataset. We will use this dataset to first train an RNN and LSTM model, followed by another set of RNN and LSTM models based on the BERT vectorized dataset.

Import the `gensim` library and methods and download the `Glove-twitter-100`:

```
#Use Pretrained Word Vectors
```

```
import gensim.downloader as api
```

```
w2v = api.load('glove-twitter-100')
```

The preceding code downloads a pretrained Word2Vec model from the Gensim GloVe (Global Vectors for Word Representation) Twitter dataset with 100-dimensional embeddings. This model contains word embeddings trained

on a large corpus of Twitter text. There are other such pre-trained Word2vec models with different dimensions that can affect the model outcome differently. The decision to pick the right one will depend on the modeler's needs and expertise level. The 100-dimensional word embedding for the token *credit* looks like:

```
w2v['credit']
array([-0.29951 ,  0.38661 ,  0.3192 , -0.061972,  0.3758 , -1.1102 ,
        0.62335 , -0.1222 ,  0.68463 ,  0.15073 , -0.10142 , -0.48702 ,
       -3.1825 ,  0.37373 ,  0.38201 ,  0.22224 , -0.12599 ,  0.075487,
       -0.62269 , -1.2717 , -0.1133 , -0.05836 ,  0.63145 ,  0.17987 ,
       -0.41027 ,  0.46678 ,  0.29673 ,  0.6941 ,  0.57335 , -0.03519 ,
        0.030538,  0.29597 , -0.54621 , -0.15322 ,  0.96815 ,  0.36071 ,
       -0.5493 ,  0.2656 ,  0.62643 , -0.27337 , -0.27126 ,  0.58833 ,
       -0.081381,  0.028584,  0.1708 , -0.42654 , -0.065541, -0.15516 ,
       -0.046655,  0.81888 , -0.28734 , -0.17906 , -1.2458 ,  0.86085 ,
        0.24524 ,  0.55634 , -0.098038, -0.68479 , -1.2183 , -0.21803 ,
        0.28268 ,  0.017831,  0.58102 ,  0.86948 ,  0.47769 ,  0.058732,
       -0.43078 ,  0.32896 ,  0.23165 , -0.38578 , -0.38586 ,  0.31696 ,
        0.30498 ,  0.078882,  0.19559 , -0.83164 , -0.29578 , -0.35248 ,
       -0.8883 ,  0.18348 ,  0.71702 , -0.35857 ,  0.73482 , -0.28252 ,
       -0.14956 ,  0.11963 ,  0.34997 ,  0.046162,  0.34402 , -0.066139,
       -0.039914, -0.29485 , -0.2064 , -0.61708 ,  1.0391 ,  0.052161,
       -0.21757 , -0.41162 , -0.5184 ,  0.10626 ], dtype=float32)
```

Figure 10.8: Word2vec word embedding for 'credit'

The processed data with Target encoded looks as shown in [Figure 10.9](#):

	Text	Target_encoded
1963953	I can not access my Ally savings account via o...	0
2908451	Account unknown To Whom It May Concern Be ad...	2

Figure 10.9: Curated Model dataset

The preceding dataset is finally split into Train and Test datasets using the following code:

```
# Split your data into training (80%) and test
(20%) sets
```

```
train_texts, test_texts, train_labels, test_labels  
= train_test_split(data.Text, data.Target_encoded,  
test_size=0.2, random_state=42)
```

Please note that we have `Text` and `Target_encoded` as the two final variables for modeling. With the modeling dataset ready with the raw features, it is time to split the sentences (complaint text) into individual words (tokens) for each record. The following set of codes converts sentences into word tokens and then creates Word2vec embeddings for those tokens:

```
#Import Libraries
```

```
import tensorflow as tf
```

```
from tensorflow.keras.layers import Embedding,  
SimpleRNN, Dropout, Dense
```

```
from keras.preprocessing.text import Tokenizer
```

```
from keras.models import Sequential, load_model
```

```
from keras.callbacks import ModelCheckpoint,  
EarlyStopping, ReduceLROnPlateau
```

```
from keras.utils.data_utils import pad_sequences
```

```
from tensorflow.keras.utils import to_categorical
```

```
from keras.callbacks import History
```

```
from tensorflow.keras.preprocessing.sequence import  
pad_sequences
```

```
from keras.layers import Activation, Dense,  
Embedding, LSTM, SpatialDropout1D, Dropout,  
Flatten, GRU, Conv1D, MaxPooling1D, Bidirectional
```

Next step requires the train and test datasets to be tokenized using the Keras tokenizer.

```
# Tokenize the text data
max_words = 10000 # Maximum number of words to
keep
tokenizer = Tokenizer(num_words=max_words)
tokenizer.fit_on_texts(train_texts)
train_sequences =
tokenizer.texts_to_sequences(train_texts)
test_sequences =
tokenizer.texts_to_sequences(test_texts)
# Pad sequences/sentences to make them the same
length
max_sequence_length = 100 # Choose an appropriate
value
train_data = pad_sequences(train_sequences,
maxlen=max_sequence_length)
test_data = pad_sequences(test_sequences,
maxlen=max_sequence_length)
# Convert labels to a NumPy array
train_labels = np.array(train_labels)
test_labels = np.array(test_labels)
Finally, we create Word2vec embeddings for all the
tokens in every sentence.
# Convert tokens to 100-dimension Word2vec
embeddings
word_index = tokenizer.word_index
embedding_dim = 100 # Embedding dimension for
glove-twitter-100
```

```

embedding_matrix = np.zeros((max_words,
embedding_dim))

for word, i in word_index.items():
    if i < max_words:
        if word in w2v:
            embedding_matrix[i] = w2v[word]

```

Output:

```

array([[ 0.00000000e+00,  0.00000000e+00,  0.00000000e+00, ...,
 0.00000000e+00,  0.00000000e+00,  0.00000000e+00],
 [ 9.51519981e-02,  3.70240003e-01,  5.42909980e-01, ...,
-5.10829985e-01,  4.68769997e-01,  3.48820001e-01],
 [-3.96210002e-04,  4.56699997e-01,  3.38900000e-01, ...,
-4.29100007e-01,  1.07459998e+00, -3.65500003e-01],
 ...,
 [ 5.85269988e-01, -4.23559994e-01,  7.10399985e-01, ...,
 1.85440004e-01,  2.13569999e-02,  5.76839983e-01],
 [-3.08189988e-01,  3.57340008e-01, -1.51960000e-01, ...,
-3.17739993e-01, -1.62560001e-01, -2.31270000e-01],
 [-2.47439995e-01,  1.00150001e+00,  9.50980008e-01, ...,
 4.36239988e-01,  8.86489972e-02, -5.81579983e-01]])

```

This `embedding_matrix` shown above contains pre-trained word embeddings for the words in the tokenizer's vocabulary. This `embedding_matrix` can be used as an initial embedding layer for a neural network.

With raw data converted to embeddings, we are ready to train our first RNN model on the Word2vec dataset. As a first step, we will define the RNN architecture, compile it, and print a summary. The following code performs all three steps:

```
# Create a Sequential model object
model_rnn = Sequential()

# Add an embedding layer
model_rnn.add(Embedding(max_words, embedding_dim,
input_length=max_sequence_length, weights=
[embedding_matrix], trainable=False))

# Add a SimpleRNN layer with 128 units, dropout,
and recurrent dropout
model_rnn.add(SimpleRNN(128, dropout=0.5,
recurrent_dropout=0.5))

# Add a Dense layer with 64 units and ReLU
activation
model_rnn.add(Dense(64, activation='relu'))

# Add a final Dense layer with 4 units (assuming
it's a classification problem) and softmax
activation for multiclass classification
model_rnn.add(Dense(4, activation='softmax'))

# Print a summary of the model's architecture
```

Output:

Model: "sequential"

Layer (type)	Output Shape	Param #
embedding (Embedding)	(None, 100, 100)	1000000
simple_rnn (SimpleRNN)	(None, 128)	29312
dense (Dense)	(None, 64)	8256
dense_1 (Dense)	(None, 4)	260

=====
Total params: 1,037,828
Trainable params: 37,828
Non-trainable params: 1,000,000
=====

None

Figure 10.10: RNN Model summary output

Compile the model using the Adam optimizer and categorical cross-entropy loss

```
model_rnn.compile(optimizer='adam',  
loss='sparse_categorical_crossentropy', metrics=  
['accuracy'])
```

After the model is compiled, it is fitted using the following codes:

#Setting the callbacks

```
callbacks = [  
    ModelCheckpoint(filepath="best_RNN_model.h5",  
save_best_only=True, monitor="val_accuracy",  
mode="max", verbose=1),  
    EarlyStopping(monitor="val_accuracy",  
mode="max", patience=10, verbose=1,  
restore_best_weights=True)
```

```
]
```

```
#Logging the model execution steps in a history object.
```

```
history =model_rnn.fit(train_data, train_labels, epochs=10, batch_size=64, validation_data=(test_data, test_labels),callbacks=callbacks)
```

```
#Prediction on Train data
```

```
loss, accuracy = model_rnn.evaluate(train_data, train_labels)
```

```
print(f'RNN Train Loss: {loss:.4f}')
```

```
print(f'RNN Train Accuracy: {accuracy:.4f}')
```

Output:

```
375/375 [=====] - 4s 11ms/step - loss: 0.7486 - accuracy: 0.6532
```

```
RNN Train Loss: 0.7486
```

```
RNN Train Accuracy: 0.6532
```

```
Prediction on Test data
```

```
loss, accuracy = model_rnn.evaluate(test_data, test_labels)
```

```
print(f'RNN Test Loss: {loss:.4f}')
```

```
print(f'RNN Test Accuracy: {accuracy:.4f}')
```

Output:

```
94/94 [=====] - 1s 11ms/step - loss: 0.7513 - accuracy: 0.6527
```

```
RNN Test Loss: 0.7513
```

```
RNN Test Accuracy: 0.6527
```


With RNN results in hand, let us train an LSTM model on the Word2vec dataset. The following code defines the LSTM architecture, compiles it, and prints the summary:

```
model_lstm = Sequential()
# Add an Embedding layer
model_lstm.add(Embedding(max_words, embedding_dim,
input_length=max_sequence_length, weights=
[embedding_matrix], trainable=False))
# Add an LSTM layer with 128 units, dropout, and
recurrent dropout
model_lstm.add(LSTM(128, dropout=0.5,
recurrent_dropout=0.5))
# Add a Dense layer with 64 units and ReLU
activation
model_lstm.add(Dense(64, activation='relu'))
# Add a final Dense layer with 4 units (assuming
it's a classification problem) and softmax
activation
model_lstm.add(Dense(4, activation='softmax'))
# Compile the model using the Adam optimizer,
categorical cross-entropy loss, and accuracy metric
model_lstm.compile(optimizer='adam',
loss='sparse_categorical_crossentropy', metrics=
['accuracy'])
# Print a summary of the model's architecture
print(model_lstm.summary())
```

Output:

Model: "sequential_1"

Layer (type)	Output Shape	Param #
embedding_1 (Embedding)	(None, 100, 100)	1000000
lstm (LSTM)	(None, 128)	117248
dense_2 (Dense)	(None, 64)	8256
dense_3 (Dense)	(None, 4)	260
Total params: 1,125,764		
Trainable params: 125,764		
Non-trainable params: 1,000,000		
None		

Figure 10.11: LSTM Model summary output

Fitting the model and making predictions is done using the following code:

```
history = model_lstm.fit(train_data, train_labels,
epochs=20, batch_size=64, validation_data=
(test_data, test_labels),callbacks=callbacks)
```

LSTM model loss and accuracy can be plotted using the following code:

```
#Plotting loss by epochs
```

```
plt.plot(history.epoch,
history.history['val_loss'], 'r',
         history.epoch, history.history['loss'],
         'g')
```

```
plt.title('LSTM model loss')
```

```
plt.ylabel('loss')
```

```
plt.xlabel('epoch')
```

```
plt.legend(['test','train'], loc='upper left')
plt.show()
```

The output of the above code is shown as follows:

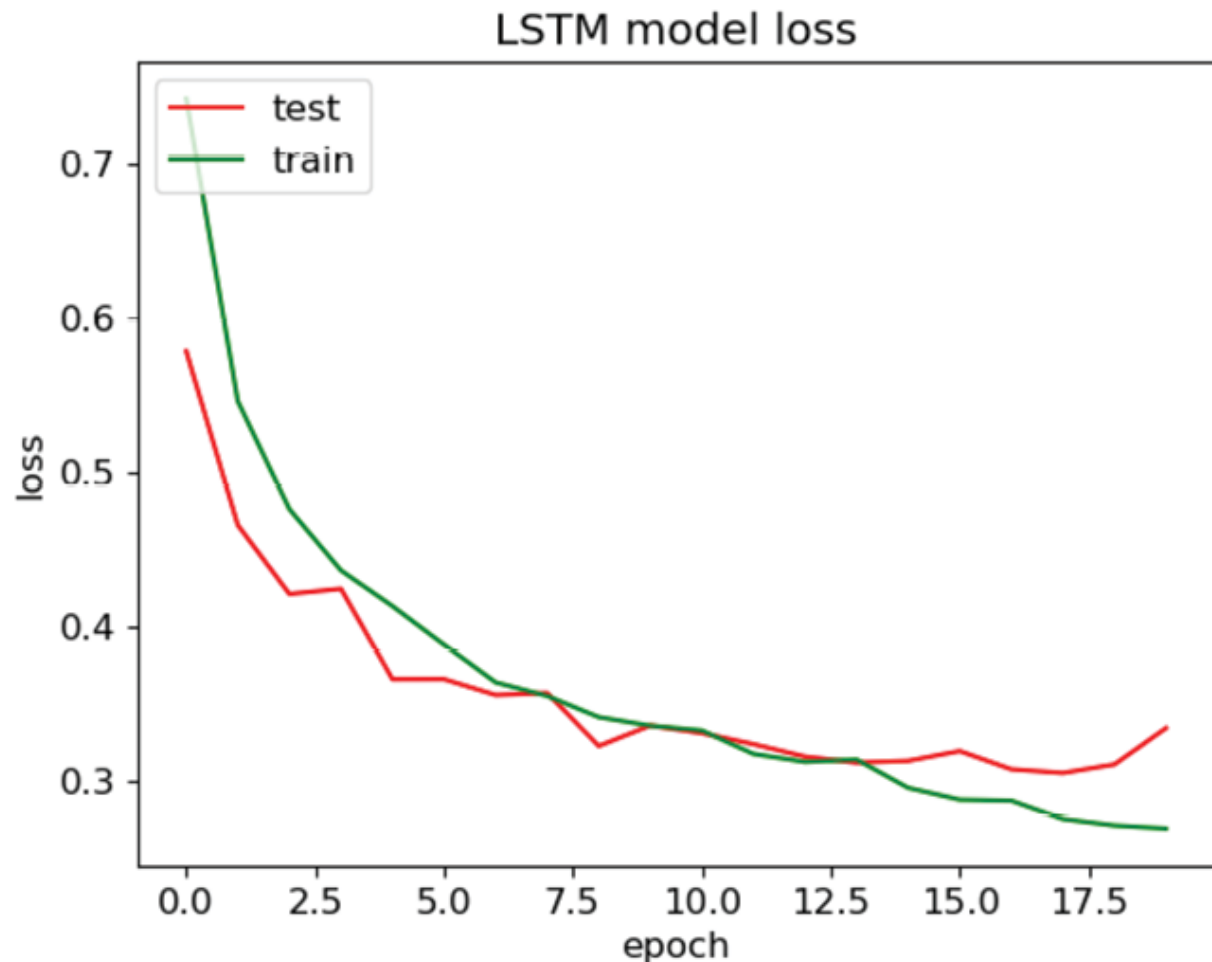


Figure 10.12: Model loss by epoch

Next, we plot the train and test accuracies by epoch using the below code:

```
#Plotting accuracy by epochs
```

```
plt.plot(history.epoch,
history.history['val_accuracy'], 'r',
         history.epoch, history.history['accuracy'],
         'g')
```

```
plt.title('LSTM Model Accuracy')
plt.ylabel('Accuracy')
plt.xlabel('Epoch')
plt.legend(['test', 'train'], loc='upper left')
plt.show()
```

The output of the preceding code is shown as follows:

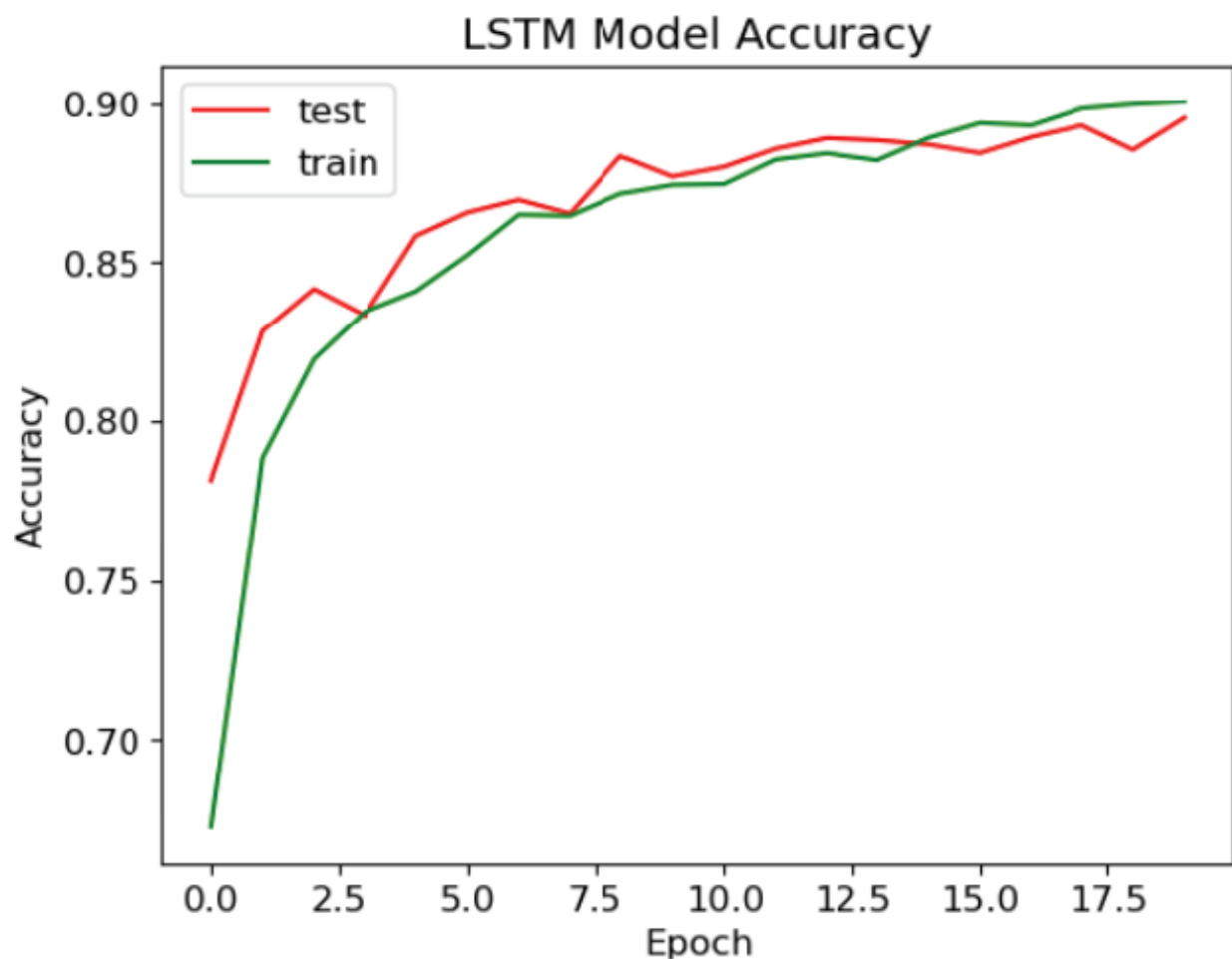


Figure 10.13: Model accuracy by epoch

Final loss and model accuracy can be obtained using the following code. We do this exercise for `train` and `test` both, but the below code runs on `train_data` for demonstration purposes:

```
#Predict on Train data
```

```
loss, accuracy = model_lstm.evaluate(train_data,  
train_labels)
```

```
print(f'LSTM Train Loss: {loss:.4f}')
```

```
print(f'LSTM Train Accuracy: {accuracy:.4f}')
```

Output:

```
375/375 [=====] - 9s 24ms/step - loss:  
0.2070 - accuracy: 0.9288
```

```
LSTM Train Loss: 0.2070
```

```
LSTM Train Accuracy: 0.9288
```

```
#Predict on Test data
```

```
loss, accuracy = model_lstm.evaluate(test_data, test_labels)
```

```
print(f'LSTM Test Loss: {loss:.4f}')
```

```
print(f'LSTM Test Accuracy: {accuracy:.4f}')
```

Output:

```
94/94 [=====] - 2s 24ms/step - loss: 0.3337  
- accuracy: 0.8953
```

```
LSTM Test Loss: 0.3337
```

```
LSTM Test Accuracy: 0.8953
```

Given this particular dataset and chosen model hyperparameters (`epoch=20` and `batch size=64`), we can observe that the LSTM model predicts better than the RNN with a test accuracy of 0.8953 compared to the 0.6527 of RNN. The Training accuracy for the LSTM model is 92.88% which is much higher than that of RNN: 65.32%. The RNN performance can be improved but we will leave that to the readers to experiment with the architecture and

hyperparameters. Even LSTM accuracy can be increased with higher epoch value but readers have to be careful of overfitting.

With Word2vec-based models in place, it is time to build a BERT vectorized dataset to train the same RNN and LSTM architectures. The code steps involved in obtaining a BERT vectorized dataset are discussed as follows:

```
#Import required libraries

import tensorflow_hub as hub
import tensorflow_text as text

from sentence_transformers import
SentenceTransformer, util

from transformers import BertTokenizer,
TFBertModel, TFBertForSequenceClassification

from tensorflow.keras.models import Model

from keras.utils import to_categorical

#Import pre-trained BERT model from
https://tfhub.dev/s?publisher=tensorflow

bert_preprocess =
hub.KerasLayer("https://tfhub.dev/tensorflow/bert\_e
n\_uncased\_preprocess/3")

bert_encoder =
hub.KerasLayer("https://tfhub.dev/tensorflow/bert\_e
n\_uncased\_L-12\_H-768\_A-12/4")

#Define an input layer for text data using
TensorFlow and Keras.

text_input = tf.keras.layers.Input(shape=(),
dtype=tf.string, name='text')
```

In the preceding code, the shape is set to `()` to indicate that the shape of the input data. `shape=()` indicates that the input data is expected to be a scalar, which means a single string value, and `dtype` is set to `tf.string` to indicate that the input data will be individual texts represented as strings.

```
#Preprocess Text Using BERT Tokenizer and Encoder
```

```
preprocessed_text = bert_preprocess(text_input)
```

```
outputs = bert_encoder(preprocessed_text)
```

The above code preprocesses the input text using a BERT tokenizer and encoder. The preprocessing involves tokenization, padding, and formatting the input text to be compatible with BERT's requirements. This is followed by the input text passing through the BERT model to generate contextual embeddings or representations.

As a next step, we will pass the BERT embeddings through the RNN layer for a text classification task. However, before that, we need to reshape our BERT embedding to be compatible with the RNN requirements. The following code performs this action:

```
# Add a Reshape layer to make the output compatible  
with the RNN layer
```

```
reshaped_output =  
tf.keras.layers.Reshape((outputs['pooled_output'].s  
hape[1], 1))(outputs['pooled_output'])
```

We then add a simple RNN layer with 128 units followed by two NN layers (dropout and softmax activation for multiclass classification) using the following code:

```
# Add Recurrent Neural network layer
```

```
rnn_layer = tf.keras.layers.SimpleRNN(128,  
name="rnn")(reshaped_output)
```

```
# Add Neural network layers
```

```
dropout_layer = tf.keras.layers.Dropout(0.1,  
name="dropout")(rnn_layer)
```

```
output_layer = tf.keras.layers.Dense(4,  
activation='softmax', name="output")(dropout_layer)
```

Finally, the `tf.keras.model` creates an RNN model by specifying the inputs (the `text_input` layer) and the outputs (the `output_layer`). This constructs the final neural network model.

```
# Use inputs and outputs to construct the final  
model
```

```
model_bert_rnn = tf.keras.Model(inputs=  
[text_input], outputs = [output_layer])
```

This model, when compiled, fitted, and evaluated on training and test, generates the results as follows. The code for compilation, fitting, and prediction on training and test data using RNN is shown below. Please note that we only execute this for 10 epochs, but readers are encouraged to run it beyond 10 epochs to assess the best accuracy before it starts to overfit. Let us start with model compilation:

```
# Compile the model
```

```
from tensorflow.keras.optimizers import Adam
```

```
model_bert_rnn.compile(optimizer=Adam(learning_rate  
=1e-4), loss='sparse_categorical_crossentropy',  
metrics=['accuracy'])
```

```
# Train the RNN model using the BERT embeddings
```

```
history = model_bert_rnn.fit(X_train, y_train,  
epochs=10, validation_data=(X_test, y_test),  
batch_size=64)
```


After compilation, we make predictions on the training data first:

```
#Predict on Train data

loss, accuracy = model_bert_rnn.evaluate(X_train,
y_train)

print(f'RNN BERT Train Loss: {loss:.4f}')

print(f'RNN BERT Train Accuracy: {accuracy:.4f}')
```

Output:

```
352/352 [=====] - 7427s 21s/step - loss:
0.6732 - accuracy: 0.7162
```

```
RNN BERT Train Loss: 0.6732
```

```
RNN BERT Train Accuracy: 0.7162
```

Next, we make predictions on the test data first:

```
#Predict on Test data

loss, accuracy = model_bert_rnn.evaluate(X_test, y_test)

print(f'RNN BERT Test Loss: {loss:.4f}')

print(f'RNN BERT Test Accuracy: {accuracy:.4f}')
```

Output:

```
118/118 [=====] - 904s 8s/step - loss:
0.6799 - accuracy: 0.7189
```

```
RNN BERT Test Loss: 0.6799
```

```
RNN BERT Test Accuracy: 0.7189
```

Finally, we run an LSTM model on the BERT embedding using the following code:

```

text_input = tf.keras.layers.Input(shape=(),
dtype=tf.string, name='text')

# Preprocess text using BERT tokenizer and encoder
preprocessed_text = bert_preprocess(text_input)
outputs = bert_encoder(preprocessed_text)

# Add an LSTM layer

# Add a Reshape layer to make the output compatible
with the RNN layer

reshaped_output =
tf.keras.layers.Reshape((outputs['pooled_output'].s
hape[1], 1))(outputs['pooled_output'])

lstm_layer = tf.keras.layers.LSTM(128, name="lstm")
(reshaped_output)

# Add Neural network layers

dropout_layer = tf.keras.layers.Dropout(0.1,
name="dropout")(lstm_layer)

output_layer = tf.keras.layers.Dense(4,
activation='softmax', name="output")(dropout_layer)

# Use inputs and outputs to construct the final
model

model_bert_lstm = tf.keras.Model(inputs=
[text_input], outputs = [output_layer])

```

This model when compiled, fitted, and evaluated on training and test datasets, generates the following results. The code for compilation, fitting, and prediction on training and test data using LSTM is shown as follows:

```

# Compile the model

```

```

model_bert_lstm.compile(optimizer='Adam',
loss='sparse_categorical_crossentropy', metrics=
['accuracy'])

# Train the LSTM model using the BERT embeddings

history_lstm =model_bert_lstm.fit(X_train, y_train,
epochs=10, validation_data=(X_test, y_test),
batch_size=64, callbacks=callbacks)

#Predict on Train data

loss, accuracy = model_bert_lstm.evaluate(X_train,
y_train)

print(f'LSTM BERT Train Loss: {loss:.4f}')
print(f'LSTM BERT Train Accuracy: {accuracy:.4f}')

```

Output:

```

352/352 [=====] - 2516s 7s/step - loss:
0.7893 - accuracy: 0.6447

```

```

LSTM BERT Train Loss: 0.7893

```

```

LSTM BERT Train Accuracy: 0.6447

```

```

#Predict on Test data

```

```

loss, accuracy = model_bert_lstm.evaluate(X_test, y_test)

```

```

print(f'LSTM BERT Test Loss: {loss:.4f}')

```

```

print(f'LSTM BERT Test Accuracy: {accuracy:.4f}')

```

Output:

```

118/118 [=====] - 846s 7s/step - loss:
0.7894 - accuracy: 0.6448

```

```

LSTM BERT Test Loss: 0.7894

```

```

LSTM BERT Test Accuracy: 0.6448

```

Based on the results from the four different models, it can be observed that the Word2vec-LSTM model performed the best, with an accuracy of 92.88%. The primary reason for this is that we trained it for 20 epochs which is the highest across all the four models. Probably BERT-LSTM can do a better job with 20 epochs, but we will leave that to the readers to experiment and find out.

Lastly, we now know that no modeling exercise is complete if we cannot explain how the model is making a decision. Model explainability is unavoidable and so far we have talked about SHAP values, but we will explore another popular post-hoc explainable AI technique called **Local Interpretable Model-agnostic Explanations (LIME)** to illuminate a machine learning model. The method explains the classifier for a specific single instance and is therefore suitable for local explanations. The following code performs the steps required to generate explanations on our best model, which is Word2vec - LSTM:

```
#Import libraries

import tensorflow as tf

import lime

from lime import lime_text
from lime import lime_tabular
from lime.lime_text import LimeTextExplainer
from collections import OrderedDict

from tensorflow.keras.preprocessing.sequence import pad_sequences

%matplotlib inline
```

We will write a function to generate LIME explanations that will take the text and row id as input.

```

# Define a prediction function for the LSTM model
def generate_lime_explanation(idx,
text_to_explain):

    explainer = LimeTextExplainer(class_names=
["Bank account", "Credit card", "Debt collection",
"Other service"])

    def lstm_predict(texts):

        # The prediction function is here

        sequences =
tokenizer.texts_to_sequences(texts)

        padded_sequences = pad_sequences(sequences,
maxlen=max_sequence_length)

        predictions =
model_lstm.predict(padded_sequences)

        return predictions

    explanation =
explainer.explain_instance(text_to_explain,
lstm_predict,    num_features=10)

    explanation.show_in_notebook()

```

We will take a couple of records (id= 0 and id= 9) to showcase the LIME output:

```

# Change idx as needed

idx = 0

text_to_explain = data.Text.iloc[idx]
generate_lime_explanation(idx, text_to_explain)

```

Output:

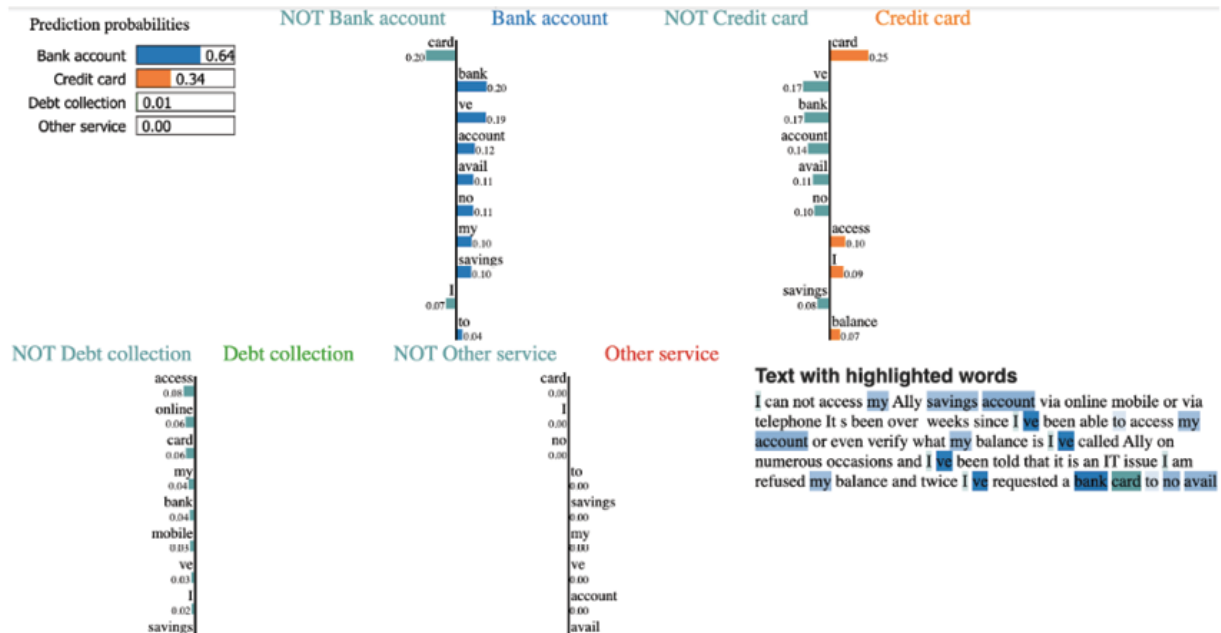


Figure 10.14(a): LIME output for complaint id = 0

Figure 10.14(a) highlights the text tokens on the bottom right that are important in classifying a complaint in a **Bank account** category. The top left chart shows the **Bank account** category with a predicted probability of 0.64 followed by **credit card**. The middle set of vertical bars returns important features that help in predicting a category better. For instance, the first vertical bar on the top shows **bank**, **account**, and **savings** as keywords that help in determining the class of the complaint. **ve** refers to **have** and since it occurs multiple times in the data, it is marked important by the model. However, it has no contribution to class determination and hence should be removed during text preprocessing:

```
# Change idx as needed
```

```
idx = 9
```

```
text_to_explain = data.Text.iloc[idx]
```

```
generate_lime_explanation(idx, text_to_explain)
```

Output:

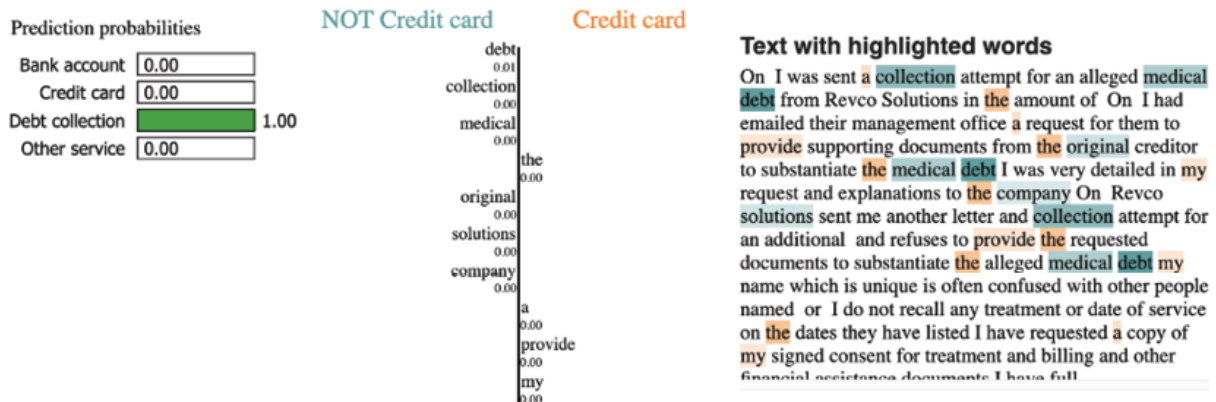


Figure 10.14(b): LIME output for complaint id =9

Figure 10.14(b) highlights the text tokens that are important in classifying a complaint as **Debt collection** related. It is worth noting that none of the tokens in id=9 relate to **credit card** and hence, it gets 0.00 probability, but the debt collection gets a 1.00 probability with the majority of green tokens on the left side of the middle bar. Again, it is clear that the model would deliver better results by removing stopwords such as **the** in the complaint text.

To create trust in our model, we need to make the model outcomes explainable to both ML and domain experts which require a human-understandable explanation. LIME allows practitioners to associate text features with the model output category at the observation level, which in turn drives transparency in model decision-making. This is the foundational principle of model explainability.

This concludes our case study, but please note that the effectiveness of any of the architectures shown in this case study depends on the specific nature of the text classification task and the compatibility of BERT embeddings with a Simple RNN layer. In some cases, you may achieve better results with other RNN variants like LSTMs or by using BERT embeddings in different ways, such as fine-tuning BERT

for your task using a different optimizer or training the model longer. The choice of architecture should be based on the characteristics of your data and task. Again, as always, detailed code can be found in the GitHub location for this case study.

Rise of transformers: A primer on BERT and GPT

RNNs were one of the first neural network architectures applied to sequence modeling that allowed them to model some level of context within language. Thereafter LSTMs were developed as an improvement over RNNs that are much more capable of capturing and utilizing short-term and long-term context. However, this was not enough to truly understand the context within any language because what exactly is a context, and how you can teach the computer to understand the context were two big problems to solve.

Transformers, a class of algorithms, have revolutionized the language modeling space by not just relying on sequential processing, but by employing self-attention mechanisms, that allow them to consider all words in a sequence simultaneously. This mechanism enables Transformers to capture complex contextual relationships, dependencies, and nuances in language. The invention of the attention mechanism solved the problem of how to encode a context into a word, or in other words, how to present a word and its context together in a numerical vector.

Moving from sequence understanding to *attention-grabbing* requires the creation of contextual word embeddings, which capture the meaning of a word in its surrounding context. However, the challenge with Transformers here is that they need large text corpora to train millions of parameters to capture the true context. This is solved by using models pre-trained on large text corpora by institutions and individuals who have the means to train such models. These are then

downloaded and further tuned to work for specific use cases. This approach has led to breakthroughs in democratizing tasks requiring a deep understanding of context, such as question-answering, sentiment analysis, and natural language understanding. With this background, let us dive deeper into the Transformer architecture and understand its inner workings.

The first transformer was presented in the famous paper *Attention is All You Need* by Vaswani et al in 2017. The name of the paper implies that you do not need recurrence or sequence to understand the context of a sentence or text and can simply rely on **Attention**. The transformer architecture processes information by paying **Attention** to a set of words in the sentence that relates to a specific word under processing. This understanding or knowledge of the context is encoded in the form of a vector which is subsequently used by a decoder to complete a language-related task such as text translation or generation. [Figure 10.15](#) showcases this process flow at a high level:

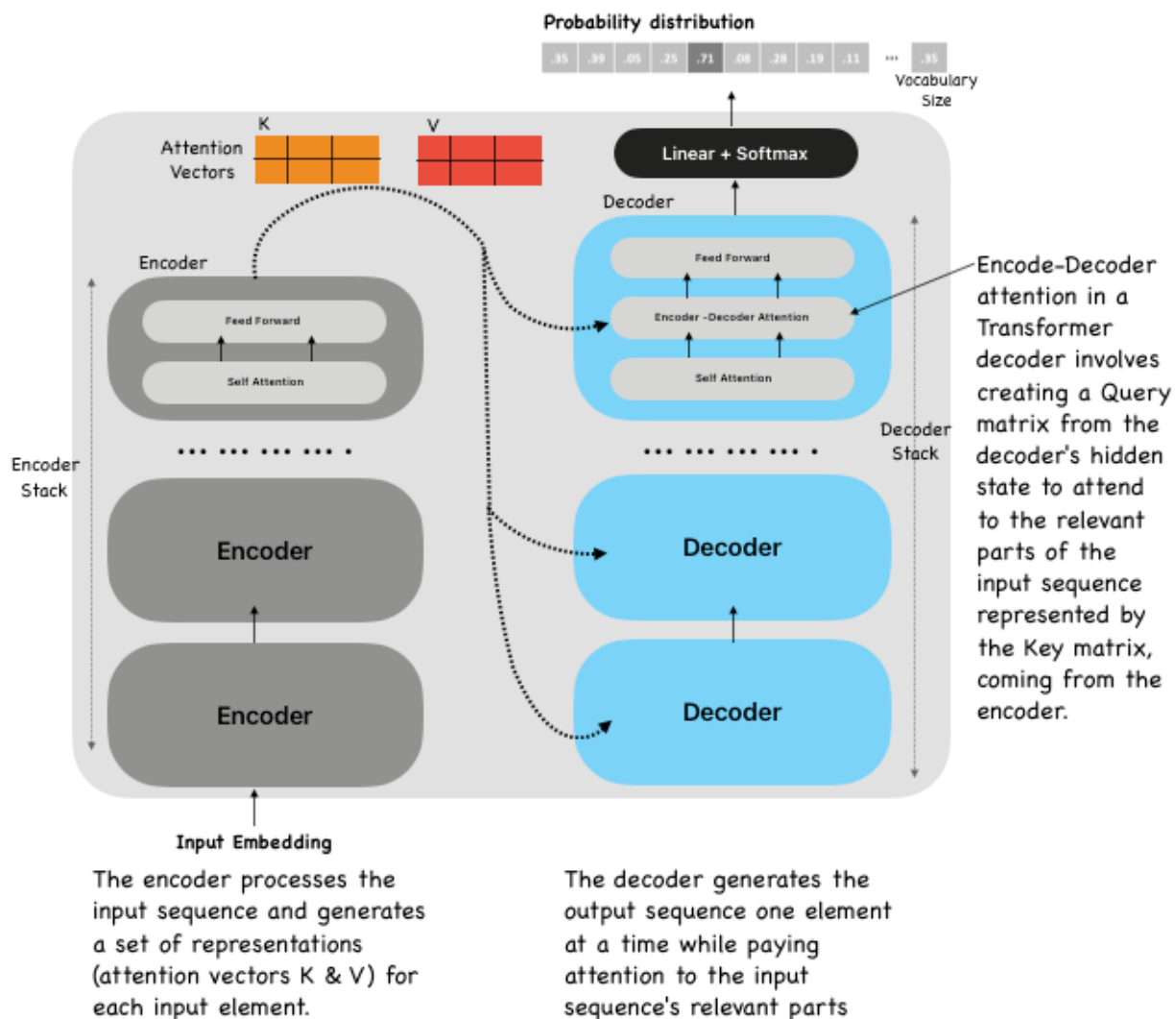


Figure 10.15: Transformer with an encoder-decoder architecture employing "Attention"

For instance, in the sentence *John slept through the day because he was sick* the encoder would identify that **he** refers to **John**. This is done using the self-attention mechanism, which allows it to look at other positions (words) in the input sentence for clues that can help lead to a better encoding for this word **he**. Once the understanding of other relevant words is baked into the one we're currently processing, the next step is to pass these context vectors to a decoder. Each context vector also stores the position information in the

input sequence, and they collectively provide the decoder with information about the entire input sequence.

The decoder employs a self-attention mechanism, similar to the one used in the encoder. However, there is a crucial difference: the self-attention mechanism in the decoder is masked to ensure that each position can only attend to previous positions and not future positions. This is done by masking future positions, before the softmax step in the self-attention calculation, to prevent the model from cheating by looking at future information when generating translations or making predictions. In the self-attention mechanism, the decoder computes attention scores between the current position and all other positions in the input sequence, including itself. Eventually, an attention-weighted representation of the input sequence for every position in the sequence is computed.

After attention computation, the decoder applies layer normalization and a residual connection to the attention-weighted representations. This helps stabilize training and allows for the flow of information. The final step in the decoding process is the linear transformation followed by a softmax activation function. The softmax function assigns probabilities to each possible output token in the vocabulary, allowing the model to make predictions.

Please note that this process is repeated for each position (word) in the output sequence, with the decoder autoregressively generating one token at a time while considering the encoder's output and previously generated tokens as context. The resulting sequence of output tokens constitutes the model's prediction.

With a background in Transformers, it becomes somewhat easy to learn more about the two most popular Transformer models: GPT and BERT. OpenAI developed GPT-1 in 2018, to generate human-like long-sequence texts that are accurate

and coherent. On the other hand, BERT was developed by the Google AI Language team and open-sourced in 2018 with the bidirectional context understanding ability that is beneficial for various tasks requiring deep language understanding.

The primary differences between GPT and BERT are shown in [Table 10.1](#) below:

Dimension	BERT	GPT
Context direction	BERT models are designed for bidirectional context understanding	GPT models are unidirectional, meaning they read text from left to right
Learning method	BERT models follow a “masked language model” pretraining objective, where some words in a sentence are masked, and the model learns to predict those masked words based on both the left and right context	GPT models consist of a stack of transformer decoder layers with self-attention mechanisms that only look at previous tokens during generation. It is trained as a masked language model, predicting the next word in a sentence based on the context provided by previous words.
Purpose	BERT can be fine-tuned for various NLP tasks such as text classification, named entity recognition, question-answering, and more. BERT embeddings can also be used for downstream tasks.	GPT models are autoregressive language models designed for text generation and completion in use cases such as chatbots, content creation, etc.

Table 10.1: BERT vs. GPT

With the rise of transformer-based models, the evolution of NLP metrics has also risen. Accuracy and precision still hold good for classification tasks such as sentiment analysis, complaint classification, and so on. However, precision and recall-based metrics such as BLEU and ROUGE assess machine translation models by comparing n-grams found in

generated sentences to the reference sentence. For image captioning, there are metrics such as CIDEr and SPICE.

We would end the discussion on RNN, LSTM, and Transformers at this point but would urge the readers to follow this knowledge growth trajectory in the NLP space. The below figure demonstrates the exponential rise of language models in the last five years:

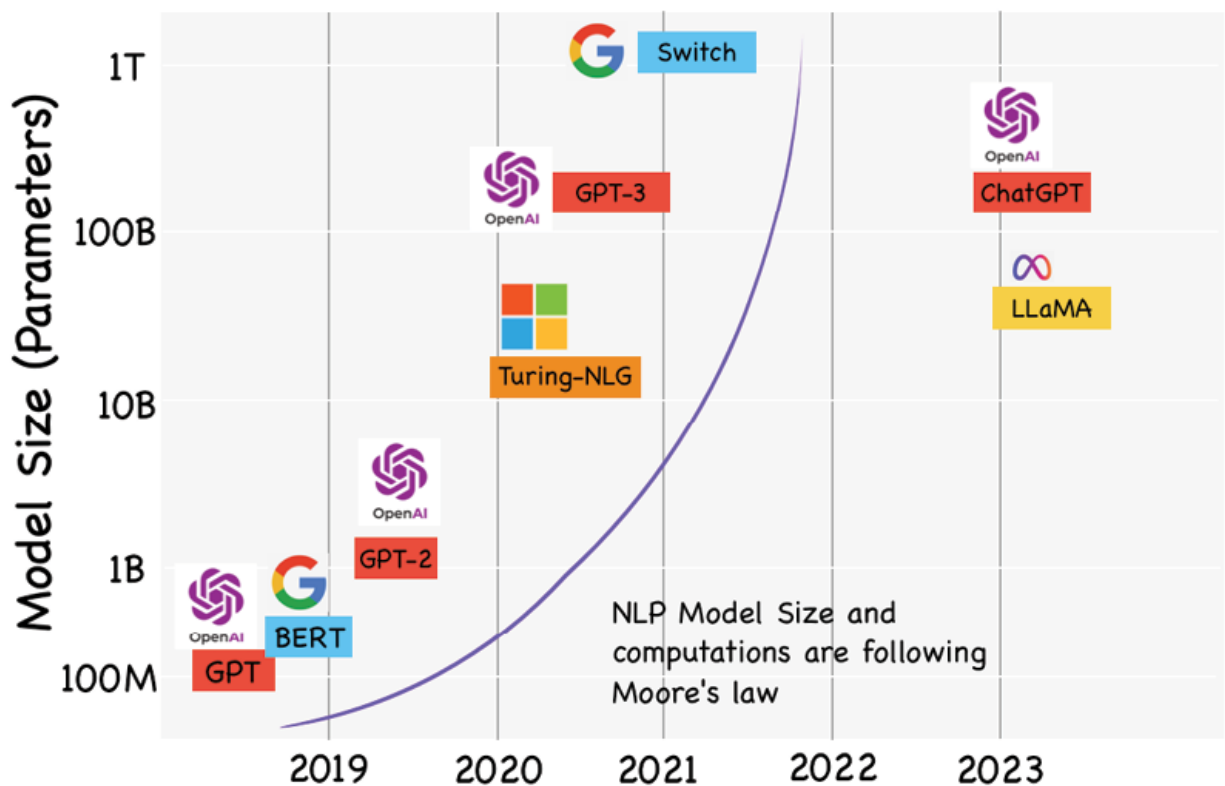


Figure 10.16: BERT, GPT and Other Competitors' journey through Time

Figure 10.16 shows that the industry is booming with brand new models that are launched almost every week now and every model claims to outsmart the existing Transformer model in the market. The case study in this chapter provides you with a great start in the space but will never be enough given this field is changing almost every day.

Conclusion

In this chapter, we explored the evolution of language modeling with a focus on **Recurrent Neural Networks (RNNs)** and their more advanced counterpart, **Long Short-Term Memory (LSTM)** networks, highlighting their pivotal roles in addressing the intricacies of text data, particularly in the context of complaint categorization.

Language modeling serves as the foundation for various natural language understanding and generation tasks, such as text prediction, machine translation, speech recognition, and sentiment analysis. This is a fast-evolving field with numerous applications across NLP, and both RNNs and LSTMs have played pivotal roles in enhancing the capabilities of these applications.

We began by examining traditional RNNs, which possess a recurrent architecture that allows them to maintain a hidden state and process sequences of data step by step. While RNN networks demonstrated promise, with the incorporation of sophisticated gating mechanisms in LSTMs, they became adept at retaining crucial information over extended sequences, making them the go-to choice for many sequence modeling tasks.

However, as our understanding of language and our computational resources continued to grow, so did the demands on language models. Enter the Transformers. The rise of Transformers, particularly models like BERT, GPT, and their variants, revolutionized the field of natural language processing. Transformers replaced sequential processing with self-attention mechanisms, enabling them to consider all words in a sequence simultaneously. This parallelism, combined with massive pretraining on large text corpora, propelled them to new heights of performance. Transformers have redefined state-of-the-art results in various language-related tasks, including machine translation, sentiment analysis, and question-answering. Their ability to capture

contextual information and semantic nuances has set a new standard for language understanding.

As we conclude this chapter on RNNs, LSTMs, and their evolution, it is clear that while these models have played a vital role in language modeling, the torch has been passed to Transformers. The success of Transformers highlights the ever-evolving nature of the field, where innovation continually drives progress.

However, the key is to understand the strengths and weaknesses of each approach as we navigate the ever-expanding landscape of language modeling, with an eye toward solving the right use case efficiently in natural language understanding and generation. With the foundational knowledge gained in this chapter, we urge the readers to delve deeper into the world of Transformers and explore their impact on the growth of NLP. In the next chapter, we will explore the use of neural networks in understanding images better using **convolutional neural networks (CNN)**. This is another exciting area of AI and is known as computer vision.

Further reading

- Efficient Estimation of Word Representations in Vector Space <https://arxiv.org/pdf/1301.3781.pdf>
- BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding:
<https://arxiv.org/abs/1810.04805>

Points to remember

- RNNs are designed for handling sequential data, such as time series or natural language text, where the order of elements matters, but they suffer from the vanishing gradient problem, which makes it

challenging for them to capture long-range dependencies in sequences.

- LSTMs are a type of RNN designed to mitigate the vanishing gradient problem by selectively updating, reading, and writing information in memory using self-gating mechanisms.
- BPTT unfolds the RNN over time, creating a computational graph that extends across multiple time steps. Gradients are propagated backward through this unfolded graph.
- Word embeddings are numerical representations of individual tokens. For instance, BERT uses the encoder part of the Transformer, to encode the semantic and syntactic information in the embedding, which is needed for a wide range of tasks.
- Word2Vec is trained on a lot of data but is not a general language model. One can train Word2Vec from scratch on specific data, or simply use Word2Vec as the first layer on your network to initialize your parameters, and then add on layers to train the model for your specific task.
- RNNs suffer from an exploding gradient problem, which is resolved using gradient clipping. This is often used during BPTT training to limit the magnitude of gradients.
- Transformers use a self-attention mechanism to process input sequences in parallel, allowing them to capture long-range dependencies efficiently.
- Transformers have achieved state-of-the-art results in a wide range of natural language processing tasks and have become the foundation for models like BERT, GPT, and T5.

- GPT is unidirectional and auto-regressive, while BERT is bidirectional, learning from both directions. Both are pre-trained Transformers but serve different NLP purposes.
- Both use the Transformer architecture to learn context from textual-based datasets using attention mechanisms and are unsupervised learning models. They do not require labeled data for training.
- GPT generates text and excels in creative tasks such as summarization or translation. BERT (Bidirectional Encoder Representations) focuses on understanding context and performs well in various NLP tasks such as sentiment analysis, text classification, and so on.
- BERT does not use the decoder part of the vanilla Transformer architecture, so the output of BERT is an embedding, not a textual output. This implies, that using BERT requires something to be done with the embedding, like calculating the cosine similarity of two BERT embeddings.
- GPT does not use the encoder part of the vanilla Transformer architecture. It leverages the stacked decoder layers from the Transformer architecture, focusing on generating text and capturing contextual dependencies within the text by looking at preceding tokens in an autoregressive manner.
- Combining **Convolutional Neural Networks (CNNs)** with **Long Short-Term Memory (LSTM)** networks can be beneficial for tasks that involve sequential data with spatial dependencies. Some examples of text data with spatial dependencies are tables and charts, computer code, GUIs, and so on. However, it is important to note that the impact on performance depends on the specific task and dataset.

Join our book's Discord space

Join the book's Discord Workspace for Latest updates, Offers, Tech happenings around the world, New Release and Sessions with the Authors:

[**https://discord.bpbonline.com**](https://discord.bpbonline.com)



CHAPTER 11

Image Processing with Convolutional Neural Networks

Introduction

Convolutional Neural Networks have given machines the gift of sight, forever altering the course of AI. They are akin to the visual cortex in the human brain.

In a world increasingly flooded with visual data, understanding and interpreting images has become an invaluable skill. From autonomous vehicles navigating complex traffic scenarios to healthcare professionals diagnosing diseases from medical scans, image recognition plays a pivotal role in numerous applications that impact our daily lives. It bridges the gap between raw visual information and actionable insights, making it one of the most transformative technologies of our time. This dynamic field of Image processing has evolved tremendously, thanks to the advent of deep learning, particularly **Convolutional Neural Networks (CNNs)**. They have emerged as groundbreaking technology, revolutionizing the way we tackle image processing challenges.

CNNs excel at automatically learning hierarchical features from images. This hierarchical feature extraction is vital for recognizing patterns, shapes, and objects within images, regardless of their scale, orientation, or position. Since images are high-dimensional data sources, CNN's architectural design works in a bottom-up approach by extracting, abstracting, and combining information until the final decision is made. It is time to peel the onion now.

Structure

In this chapter, we will learn the following topics:

- Deep learning for computer vision tasks
- Background
- Under the hood of CNN models
 - Data
 - Model: How CNN's learn?
 - Loss: How to achieve optimal results
- Challenges and assumptions
- The race for the best model and transfer learning: A primer
- Case study: PDF document parser

Objectives

This chapter is designed as a comprehensive guide to CNNs for image recognition. Just like in previous chapters, we will start by discussing the critical and unique data needs and preprocessing requirements concerning CNN-based image recognition projects. Subsequently, we will demystify the CNN architecture, and explain how they learn to recognize patterns in images. We will further explore the significance of

loss functions and optimization techniques in training CNNs effectively.

Recognizing that image recognition with CNNs is not without its challenges, we will discuss issues such as overfitting, and computational complexity while highlighting the need for transfer learning, and interpretability. Just like with other ML or deep learning techniques, understanding these challenges and solutions is essential to make informed decisions during model development and deployment for CNNs.

Lastly, we will bring theory into practice, by concluding this chapter with a real-world case study, showcasing the application of CNNs in solving a specific image recognition problem that is very relevant for any industry that requires a lot of document processing. This case study will provide practical insights and demonstrate the concepts discussed throughout the chapter.

Deep learning for computer vision tasks

Convolutional Neural Networks remind us of that technology's ultimate purpose is to enhance human experience. They enrich our lives by enhancing our visual comprehension. The importance of CNNs for humans lies in their ability to amplify our innate visual intelligence. They assist us in recognizing patterns, identifying objects, and understanding the world. A silly but relevant example is that of a CNN model identifying a bear in the *Toblerone* logo when most humans never paid attention to the details hidden in the logo.

Background

Back in the 1960s, neuroscientists and psychologists studying the human visual system laid the idea of CNNs. The concept of feature hierarchies, where simple features

combine to form more complex representations, played a role in shaping CNN architecture. *Kunihiko Fukushima*, the man who first developed a machine-learning and pattern-recognition technology called **Neocognitron**, wondered if the electrical components such as switches, transistors, and capacitors working together could mimic the way that human neuroanatomy is set up to process patterns. As an electrical engineer, he pursued his observation and invented the *Neocognitron*, an artificial neural network, similar to those in the visual cortex of the brain.

Later on, in 1998, Yann LeCun introduced LeNet-5, a groundbreaking CNN architecture designed for character recognition. It demonstrated enhanced hierarchical feature extraction capabilities built on top of the Feed Forward Neural Network concept which showcased superior performance in handwritten digit recognition tasks. This enthusiasm faced a slowdown because CNNs needed exponential computation power which was not cheaply available. However, after 2010, CNNs experienced a resurgence, primarily driven by advancements in hardware (GPUs) and the availability of large datasets. Thereafter, some notable developments in this space included AlexNet (2012), GoogLeNet (2014), and VGGNet (2014).

Under the hood of CNN models

In this section, we will further investigate the workings of CNN and the unique data needs of this architecture.

Data

Visual data is complex and has many layers. While the data has a spatial nature where the spread-out pixels together make up an image, there could be an additional element of the color scheme RGB layers related to an image. After all, that's what makes the images colorful. And, naturally, the

world is teeming with an astonishing variety; it encompasses millions of diverse living and non-living beings, each intricately subdivided into specific sub-categories. For instance, the animal kingdom alone boasts an immense array, ranging from majestic mammals like elephants and swift cheetahs to the minute, bustling world of insects, such as industrious ants and colorful butterflies. All of these flaunt numerous features.

To summarize, visual data is complex because its processing requires object recognition, color perception, depth, and motion detection for millions of object types.

In this chapter, we will learn the basics of CNN using a facial emotion detection example and conclude the chapter with a case study on detecting tabular data in unstructured sources such as scanned PDFs. Banks and other financial institutions have a ton of scanned documents that are waiting to be converted into structured data for value realization and reducing manual work.

The PDF might look something like in [Figure 11.1](#):

Financial Information for Textron's Finance and Insurance Subsidiaries

Statement of Income

For each of the three years ended December 31,
(In millions)

	1993	1992	1991
Revenues			
Interest, discount and service charges	\$1,260.2	\$1,273.2	\$1,183.9
Credit life, credit disability and casualty insurance premiums	300.8	298.5	308.7
Non-cancellable disability income, life and group insurance premiums	836.2	795.0	764.6
Investment income (including net realized investment gains)	406.1	360.8	354.8
Total revenues	2,803.3	2,727.5	2,612.0
Costs and expenses			
Selling and administrative	789.7	757.8	702.4
Interest expense	432.3	488.9	510.4
Provision for losses on collection of finance receivables, less recoveries	152.6	160.4	134.8
Credit life, credit disability and casualty insurance losses and adjustment expenses, less recoveries	132.1	137.2	130.1
Death and other insurance benefits	392.9	369.9	362.3
Increase in insurance policy liabilities	324.1	316.7	319.8
Amortization of insurance policy acquisition costs	143.5	131.8	128.5
Total costs and expenses	2,367.2	2,362.7	2,288.3
Income before income taxes	436.1	364.8	323.7
Income taxes	174.6	138.9	118.1
Income before cumulative effect of changes in accounting principles	261.5	225.9	205.6
Cumulative effect of changes in accounting principles, net of income taxes	—	(44.7)	—
Net income	261.5	181.2	205.6
Minority interest in net income	2.6	—	—
Textron's equity in net income	<u>\$ 258.9</u>	<u>\$ 181.2</u>	<u>\$ 205.6</u>

Figure 11.1: PDF snapshot

The facial emotions images we are going to use will look like in [Figure 11.2](#):



Figure 11.2: Multiple facial emotion

The internet can be scraped for images using Python. A sample of around 1000 facial emotion images was scraped from the internet using the following Python code and in four facial emotion categories: Happy, sad, angry, and disgusted.

The Python code to scrape the dataset is as follows. You can execute the script using the following command:

```
python3 bing_scraper.py --search 'happy human face' --limit 10 --  
download --chromedriver </Output_file_folder_path >
```

To be able to successfully run the above script, please download the Chromedriver. Then, place the .py script in the same folder and then CD to the same directory on the terminal. Execute the script by changing the query within the single quotes and the number of images you need to download.

The link to the original dataset can be found at the GitHub location for this chapter.

As you can imagine, sourcing such images from the internet is not easy and a high-accuracy CNN model would require more than 250 images in a category to get trained with sufficient accuracy. To overcome this limitation of the CNN model, we can use two techniques:

Data augmentation. This allows us to create multiple variations of a single image so the models learn to recognize a happy face in many different ways. Let me explain further using the following [Figure 11.3](#):

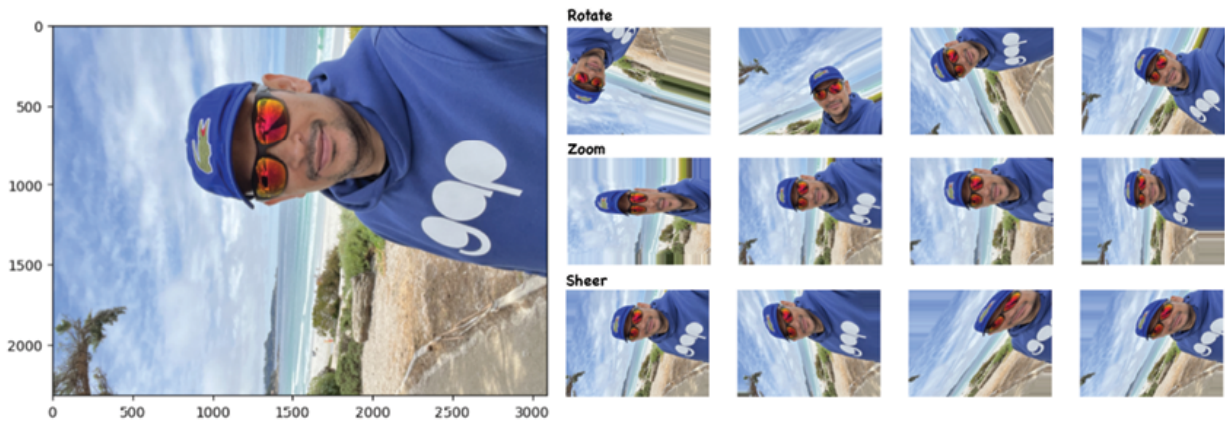


Figure 11.3: *Augmented Happy Face*

It can be observed in the previous figure, that data augmentation is used to increase the amount of image data (a single big image on the left) by using techniques like shearing, zooming, flipping, and so on, to create multiple versions of every image in the data. This makes the model more robust to slight variations and hence prevents the model from overfitting. It is very easy to achieve this in Keras using the `ImageDataGenerator` class, which generates batches of tensor image data with real-time data augmentation with one line of code.

Transfer learning: It reduces the data requirement for training new models by leveraging knowledge learned from one task (for example, ImageNet) to improve performance on a related task (for example, a specific object recognition task). Since it utilizes a pre-trained network for another specific task, new model training is not needed from scratch thereby reducing the data need. We will discuss this later in this chapter in detail.

However, before we conclude the data section, we need to know about another dimension of visual data which is temporal dynamics seen in videos. Capturing and addressing the challenges of high dimensionality, temporal dynamics, data volume, and computation requires substantial resources. We will not be discussing video analysis in this

chapter since it is an advanced topic but would encourage readers to pursue it based on the knowledge gained here.

Model

We know that the human visual cortex processes visual information and the brain stores these impressions of various world objects for later inferencing. Similar to the human brain, the deeper layers of a CNN, recognize and process complex features and patterns. Just as the visual cortex performs intricate visual computations, CNNs use convolutional and pooling layers to learn hierarchical representations of features in images, ultimately enabling object recognition and understanding.

Well, a very pertinent question to ask at this point is why we cannot simply use a feed-forward neural network to process image data?

Here is why, if an image with dimensions $64 \times 64 \times 3$, that is 64 pixels high, 64 pixels wide, and 3 pixels deep for red, green, and blue is fed to an FFNN, the image needs to be stretched out to be a $1,22,88 \times 1$ ($64 * 64 * 3 = 12288$) numpy array.

These pixel inputs are then multiplied by weights, the loss is optimized, and the image is classified based on the score it has been assigned. This process works, but it is not capable of producing outstanding results because stretching the image into a one-dimensional vector does not preserve the spatial structure of the image.

This implies that the features used in the classification tasks are not as informative as they could be if the image had its spatial structure preserved across all three dimensions of height, width, and depth. For this reason, CNNs were invented to process image data in its true form. CNN does this task effectively by extracting state-of-the-art features from this complex spatial structure and then passing it onto a dense layer structured like an FFNN hidden layer. Let us

explore this further one step at a time using the facial emotion dataset example.

CNNs are made up of a series of layers that perform feature extraction, hierarchical representation, and classification. Each of these layers plays a unique role, for instance, CNNs apply a series of filters to the raw pixel data of an image to extract and learn higher-level features, which the model uses for classification. CNNs contain three components. Please refer to [Figure 11.4](#):

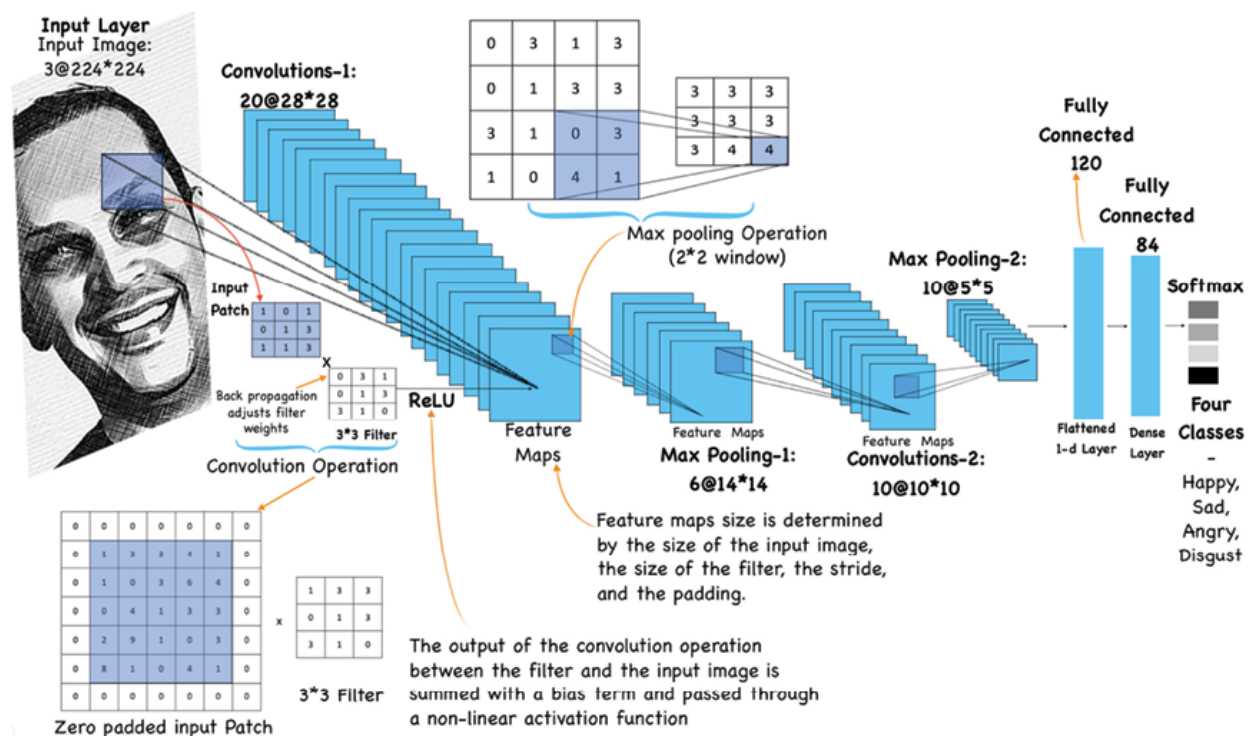


Figure 11.4: Illustrative CNN Architecture processing a 'Happy Face'

As can be seen in [Figure 11.4](#), there are five different layers in the CNN architecture:

Input layer: The input to the CNN is an image containing a face with the target emotion to be recognized.

Convolutional layers: Convolutional layers apply filters (also known as kernels or simply matrix of weights) to the input image. These filters slide over the image to detect

patterns, edges, and features at different scales. In the context of emotion recognition, these layers may detect features like eye shapes, mouth expressions, or wrinkles associated with emotions. For instance, if we were trying to see if an image has happy eyes, we could slide a curved line filter closer to the eyes to see if wrinkles exist near the eyes.

This means that the convolutional layer performs a convolutional operation such as multiplying the filter weights with the input image patch summing the results and calculating a score for each image and each filter. The closer the score is to one, the more the feature is influenced by the patch of input. Please note that convolutional layers apply a specified number of convolution filters to the image and smaller filters collect more local information compared to the bigger filters which represent more global, high-level and representative information. The convolutional layer also applies an activation function (that is, ReLU) that introduces non-linearity to the network.

Another important concept in relation to the convolution layer is the stride and padding that control the spatial dimensions of the output feature maps. Padding involves adding extra border pixels to the input image before convolution to maintain spatial dimensions. It helps preserve information at the image edges. Stride determines how much the convolutional filter shifts during each operation. A larger stride reduces the size of the output feature map.

Let us understand this using a scenario where the size of the filter, the stride, and the size of the image are asymmetric. This asymmetry prevents the spatial structure of the image from being retained. For example, if we have a $32 \times 32 \times 3$ image, a $7 \times 7 \times 3$ filter, and a stride of 1, the filter would not be able to look at all of the pixels in the image. Padding zeros around the perimeter of an image helps the filter to slide over the entire image and produce scores for the entire image. This practically ensures that the edges do not

disappear too quickly as the information passes from one convolution to another and the spatial size and structure of the input data are preserved. Hence, padding ensures that the best data is extracted from the image.

Essentially, these two convolution-layer features influence the network's ability to capture fine details and contextual information in the data while managing computational complexity.

Pooling layers: They down-sample the feature maps produced by convolutional layers thereby reducing computational complexity and extracting only the most salient information. Max-pooling is commonly used, which retains the maximum value in each pooling region. This retains important features while reducing spatial dimensions.

Flatten layer: After several convolutional and pooling layers, a flatten layer reshapes the 2D feature maps into a 1D vector. This prepares the data for the subsequent dense layers.

Lastly, the dense layers also known as the fully connected layers process the flattened feature vector and perform classification. These layers are connected to all neurons in the previous layer. In facial emotion recognition, the final dense layer typically would have as many neurons as there are emotion classes (for example, happy, sad, angry). Softmax activation is often used to produce class probabilities.

This CNN architecture works fine until the normalized input after passing through various adjustments in intermediate layers becomes too big or too small when it reaches far away layers. This causes a problem of internal co-variate which is the change in the distribution of feature values within deep layers during training.

This is tackled by a technique known as **Batch Normalization** or **BatchNorm** to improve the training stability and convergence speed of deep neural networks. It was introduced by *Sergey Ioffe* and *Christian Szegedy* in their 2015 paper titled *Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift* and has been discussed in detail in [Chapter 9, Structured Data Classification using ANN](#). The reduction in co-variate shift by BatchNorm leads to faster convergence and allows the network to start learning useful representations sooner. The BatchNorm layer does this by standardizing (mean centering and variance scaling) the input given to the later layers. This layer must generally be placed in the architecture after passing it through the layer containing the activation function and before the Dropout layer (if any). An exception is for the sigmoid activation function wherein you need to place the batch normalization layer before the activation to ensure that the values lie in the linear region of the sigmoid before the function is applied.

During training, BatchNorm operates on mini-batches of data. For each feature (channel) in the input tensor, BatchNorm normalizes the values within the mini-batch by subtracting the mean and dividing by the standard deviation. This ensures that the features have a mean of approximately zero and a standard deviation of approximately one. A few other things to note with respect to CNN architecture design are:

- Smaller filters (for instance 3x3) closer to the raw input image are advisable to collect as much local information as possible. The filter width should be gradually increased to reduce the generated feature space width to represent more global, high-level, and representative features.
- For moderate or small-sized images, typical filter sizes used are 3x3, 5x5, and 7x7 for the convolutional layer

and 2x2 or 3x3 filter sizes for Max-Pooling parameters. Larger filter sizes and strides may be used to shrink a large image to a moderate size and then go further with the convention stated in the first point.

- If you are confused about creating the right architecture for your first CNN from scratch, it is advisable to use the most popular CV networks like LeNet, AlexNet, VGG-16, VGG-19, and so on, as an inspiration. These architectures have broken records while efficiently learning from millions of images, so there is no reason why they will not work with your use case. Please follow the Convolution-Pool layer sequences used in the architectures such as Conv-Pool-Conv-Pool or Conv-Conv-Pool-Conv-Conv-Pool or trend in the Number of channels 32- 64-128 or 32-32-64-64 or trend in filter sizes, Max-pooling parameters, and so on, refer to [Figure 11.5](#):

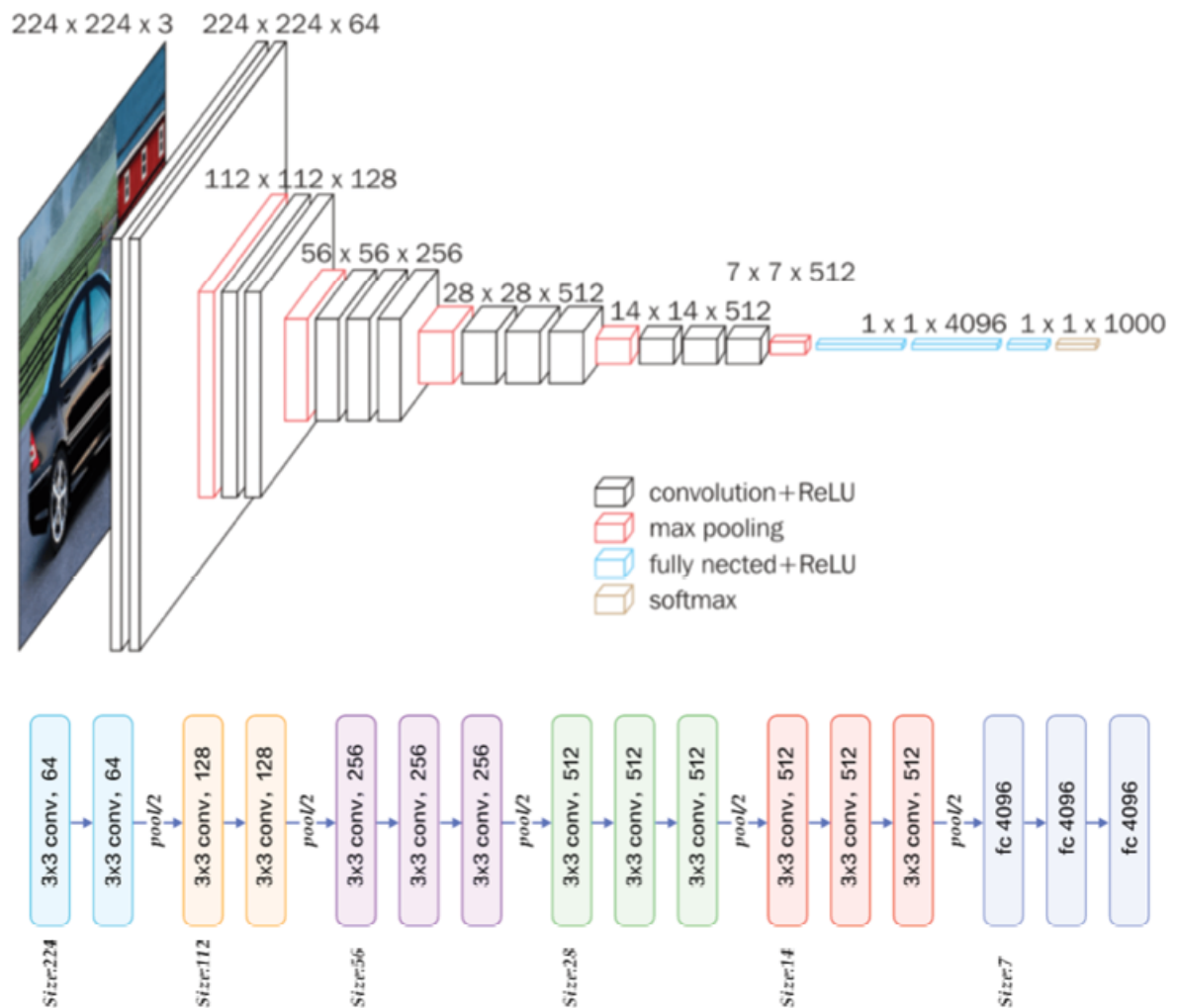


Figure 11.5: VGG-166 Architecture (Source: Kaggle)

Source: <https://www.kaggle.com/code/blurredmachine/vggnet-16-architecture-a-complete-guide>

- Lastly, keep adding layers until the model starts to over-fit. Once considerable accuracy is achieved in the validation set, we can use the various regularization components like data augmentation, dropout, BatchNorm, l1/l2 regularization, etc. to reduce over-fitting.

Since CNNs are hard to train from scratch, and in today's day and age, building a CNN from scratch to differentiate a cat from a dog is common knowledge. We will instead leverage

transfer learning to do something more application-focused, spectacular, and real-world.

Keeping in mind the application focus of this book, it is recommended the ultimate goal of any **computer vision (CV)** exercise should be to have a CV model deployed in an app. This would serve three purposes. Firstly, a model worthy of an app will serve a real-world use case, for example, facial recognition helps unlock phones these days, which is also a perfect example of transfer learning. Secondly, most real-world use cases would need large labeled datasets and enormous computation. Cat vs. Dog model training would limit your learning in today's ever-changing DL space. Thirdly, most of the time you will find yourself imitating a popular architecture like VGG anyways, so why not use it directly to deliver a meaningful use case?

In this spirit, we will present two case studies in this chapter, where model training from scratch is not needed and you will still get real-world exposure. The first case study will be about extracting structured information such as tables from unstructured data such as PDFs. This is because extracting tabular information from PDFs is fundamental in the banking and financial sector for streamlining operations, ensuring compliance, improving decision-making, and enhancing customer service. Automated data extraction from tables contributes to increased efficiency, accuracy, and the ability to derive actionable insights from large volumes of financial documentation. The second case study will discuss building a facial emotion CV model using Apple's no-code developer tool called `createML`. However, for now, let us move on to learning loss minimization for CNN models.

Loss: How to achieve optimal results

Backpropagation is a fundamental training algorithm used in all Neural Networks such as ANNs, CNNs, and RNNs. The goal is to adjust the network's weights and biases during the

learning process. However, there are certain differences in how the backpropagation works in ANNs and CNNs. We already know from [Chapter 9, Structured Data Classification using ANN](#) that Backpropagation in ANNs involves calculating gradients for each weight and bias in the densely connected layers. On the other hand, since CNNs employ convolutional layers that apply filters to extract local spatial patterns, backpropagation in CNNs involves computing gradients for the filter weights in convolutional layers, as well as the weights in fully connected layers, if present.

Note: The elements of the filter (kernel) matrix are equivalent to the unit weights in a standard neural network and will be updated in addition to the regular network weights during the backpropagation phase.

This starts with the gradient calculation for the output layer. At first, the derivative of the loss with respect to the output of the output layer is computed. Subsequently, the gradients are propagated backward through the convolutional and pooling layers. For each filter in a convolutional layer, the gradient is computed concerning its weights. These gradients are computed using the chain rule, as they depend on the gradients from the subsequent layer. With the gradients computed, the network's weights and biases are updated using an optimization algorithm, such as **stochastic gradient descent (SGD)** or one of its variants (for example, Adam or RMSprop). The learning rate determines the size of the weight updates. This process is similar to what was discussed in [Chapter 9, Structured Data Classification using ANN, Figure 9.7](#). However, there are certain CNN-specific nuances with respect to the partial derivatives of the errors. Three excellent papers explaining the math behind the CNN backpropagation are listed in the

further reading section of this chapter. These steps are repeated for each batch of training data for a specified number of epochs until the model converges to a satisfactory solution.

Challenges and assumptions

CNNs have made significant strides in various fields, but they are not without challenges and assumptions. Some of the major challenges are listed below:

- **Data quantity and quality:** CNNs require large amounts of labeled data for effective training. Gathering, annotating, and maintaining high-quality datasets are expensive and time-consuming for the majority of individuals and firms.
- **Computational resources:** Even if you obtain a large volume of data, training CNNs, especially large models, demands substantial computational power and memory, limiting accessibility for many researchers and applications.
- **Transferability:** Transfer learning from pre-trained models is a powerful concept that solves the challenge related to computational resources to a great extent. However, it may not always work seamlessly across domains leading to substantial fine-tuning, which can still require substantial labeled data.
- **Overfitting:** CNNs, especially deep architectures, are prone to overfitting, where they memorize the training data instead of generalizing to unseen examples. Addressing overfitting requires techniques like dropout, regularization, and data augmentation. This problem is akin to all the other traditional ML methods we have seen.

- **Interpretability:** Again CNNs being from a deep learning family, they are often considered *black-box*, making it challenging to interpret their decisions. Understanding how and why a CNN arrives at a specific prediction remains an active research area and hence prevents wide-scale adoption in regulated industries. However, it has seen exemplary success in fields such as medical imaging due to the availability of large-curated data and higher accuracy.

Let us talk about some important CNN assumptions, which are essential to effectively use CNNs and design experiments that account for their limitations:

- **Local relations:** CNNs assume that neighboring pixels or units are more likely to be related. This assumption is valid for grid-like data, like images, but may not apply to all data types.
- **Hierarchical features:** CNNs are built on the assumption that features in data can be represented hierarchically, with lower layers capturing simple features and higher layers learning complex ones.
- **Independence of local patches:** In convolutional layers, CNNs assume that different local patches of an image are treated independently. This assumption implies that CNN is designed to be translation-invariant meaning that the network is expected to recognize the same feature (for example., an edge or a texture) regardless of where it appears in the input. This property is crucial for tasks like object recognition in images because objects can appear in various positions.
- **Scale and rotation invariance:** CNNs are assumed to learn features that are invariant to translation, scale, and rotation. This means they can capture the facial wrinkles regardless of the picture orientation. While

they are robust to such transformations to some extent, achieving perfect invariance can be challenging.

- **Data representativeness and target label:** This one is very important because CNNs assume that the training data is representative of the target domain. We can put a bunch of *happy face* pictures in a folder and train them as happy faces but if the training data does not cover all possible variations, the model may struggle to find happy faces in unseen data. You can also add *happy faces* pictures in a folder named sad faces, and all happy faces will be predicted as sad.

The race for the best model and transfer learning: A primer

The evolution of CNN architectures over time, particularly those that excelled in the ImageNet **Large Scale Visual Recognition Challenge (ILSVRC)** has addressed many real-world issues. In some cases, we talk about the dangers of human surveillance driven by face recognition technologies but also feel amazed by the self-driving Teslas. LeNet, invented by Yann Lecun gave a boost to this technology in the 1990s and used it to scan zip codes, digits, etc., as the first successful CNN application. Let us review a very brief history of CNNs which is both fascinating and puzzling.

LeNet-5 (1998) was an early 7-level convolutional network by LeCun, applied for recognizing hand-written numbers on checks. However, the limitation was its dependence on computing resources due to the need for larger and more convolutional layers. Then 14 years later in 2012 AlexNet outperformed competitors in ILSVRC 2012, reducing the top-5 error from 26% to 15.3%. It featured a deeper architecture, more filters per layer, and various techniques, such as dropout and data augmentation. ZFNet was an improvement

over AlexNet in 2013, achieving a top-5 error rate of 14.8% in ILSVRC 2013 by tweaking hyperparameters while maintaining the same structure.

Next year in 2014, *GoogLeNet* (Inception V1) won ILSVRC 2014 with a top-5 error rate of 6.67% by introducing an inception module, batch normalization, image distortions, and RMSprop. This significantly reduced the number of parameters compared to AlexNet. In the same year, VGGNet popped up which consisted of 16 convolutional layers with a uniform architecture using 3x3 convolutions. Although widely used for feature extraction, its 138 million parameters pose a computational challenge.

These were followed by ResNet in 2015, which introduced skip connections and heavy batch normalization, allowing the training of a deep network with 152 layers. This achieved a top-5 error rate of 3.57%, surpassing human-level performance on the ILSVRC dataset. This progression showcases the refinement of CNN architectures over time, addressing challenges, enhancing performance, and pushing the boundaries of deep learning in image recognition tasks. However, a race for superior architectures with 100+ layers, a million parameters, and a million training images to achieve human-level performance prevents the benefits from getting to society. This posed a pressing question about bringing the benefits of CNNs to practitioners and industry professionals who the means might not have to train such a large network on large samples of data.

Enter transfer learning: It mitigates the challenge of training large architectures by leveraging pre-trained models on vast datasets, reducing the need for extensive training from scratch. It was introduced by researchers *Yann LeCun*, *Léon Bottou*, *Yoshua Bengio*, and *Patrick Haffner* in the late 1990s. The approach involves using knowledge gained from a source task, often pre-training on a large dataset, and applying it to a related target task. This process jump-starts

training, particularly beneficial when dealing with computationally expensive deep architectures, saving time, and enhancing convergence on new tasks with limited data. The new task will be related in some way to the previously trained task, which could be to categorize objects or tables in a specific file type such as PDFs. Please note transfer learning is not a distinct type of machine learning algorithm, instead, it's a technique or method used for training models.

Another key aspect of transfer learning is generalization. This means that only knowledge that can be used by another model in different scenarios or conditions is transferred. Instead of model learning based on a specific training dataset, models used in transfer learning will be more generalized. Models developed in this way can be utilized in changing conditions and with different datasets.

For example, transfer learning can be used for the identification of tabular data in scanned PDFs. A deep learning model can be trained with labeled data to identify and categorize the tables that are the subject of images. The model can then be adapted and reused to identify another specific and similar subject (table) within a set of images in another dataset through transfer learning. Since the deep learning model has learned to differentiate text from tables in PDFs, the general elements of the model will stay the same and will save resources in future model training. For instance, the general elements could be the parts of the model that identify the edge of the table in the scanned PDF image. The transfer of this knowledge saves retraining a new model to achieve the same result.

Transfer learning is particularly valuable in scenarios where labeled data is limited, computational resources are constrained, or where the target task is related to the source task. As one can imagine, examples of machine learning that face such scenarios often and need transfer learning are

neural networks, computer vision, and natural language processing.

In the case study section next, we will leverage the transfer learning concept to identify data in the PDF tables.

Case study: PDF document parser

As part of the case study, we are going to use a **Table Transformer (TATR)** model pre-trained on the PubTables-1M dataset. It is available for download from Microsoft Research Open Data. The modeling approach was proposed in *PubTables-1M: Towards comprehensive table extraction from unstructured documents* by Brandon Smock, Rohith Pesala, and Robin Abraham to benchmark progress in table extraction from unstructured documents, as well as table structure recognition and functional analysis. The authors trained **2 Detection Transformer (DETR)** models, one for table detection and one for table structure (rows, columns, and so on) recognition, and named them table transformers.

The DETR model itself was originally proposed by *Nicolas Carion et al.*, which consists of a convolutional backbone (ResNet-50 or ResNet-101) followed by an encoder-decoder Transformer (discussed in [Chapter 10, Language Modelling with RNN](#)). In their paper, the researcher mentions that *DETR demonstrates accuracy and run-time performance on par with the well-established and highly-optimized Faster RCNN baseline on the challenging COCO object detection dataset. Moreover, DETR can be easily generalized to produce panoptic segmentation in a unified manner. We show that it significantly outperforms competitive baselines.* [Figure 11.6](#) illustrates the innovative DETR architecture:

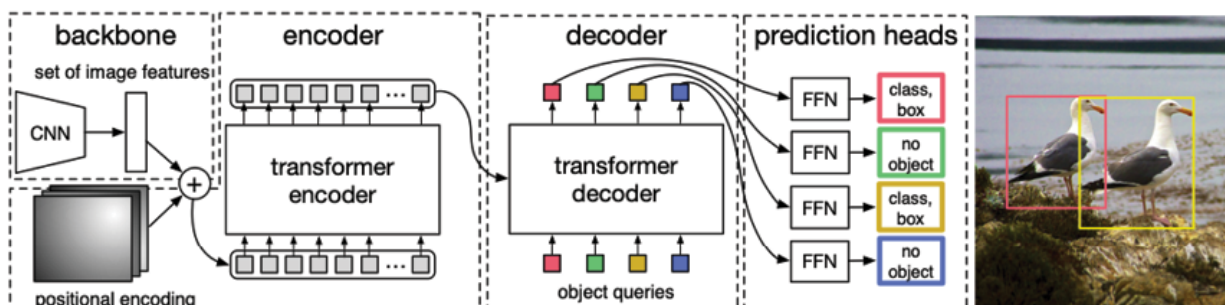


Figure 11.6: DETR architecture (source: <https://arxiv.org/pdf/2005.12872.pdf>)

DETR employs a standard CNN backbone for acquiring a 2D representation of an input image. After flattening and augmenting it with positional encoding, the model feeds it into a transformer encoder. The transformer decoder processes a fixed set of learned positional embeddings, known as object queries, while also attending to the encoder output. Subsequently, each output embedding from the decoder undergoes prediction through a shared **feed-forward network (FFN)**. The FFN predicts either a detection (comprising class and bounding box information) or assigns a label of *no object*.

As discussed in [Chapter 10, Language Modelling with RNN](#), transformers have widely been applied to problems with sequential data, in particular in **natural language processing (NLP)** tasks such as language modeling and machine translation, but surprisingly, computer vision has not yet been swept up by the Transformer revolution. Facebook researchers have corrected this anomaly by successfully integrating transformers as a central building block in the object detection pipeline. It can be fine-tuned just like a BERT model due to the use of a clever loss function called **bipartite matching loss**.

Reference for additional reading on DETR and other related topics can be found at the end in the *Further reading* section of this chapter.

The model and dataset discussed in the DETR example make it obvious that generating state-of-the-art CV results requires millions of images and enormous research and computation time. For perspective, PubTables-1M contains nearly one million tables from scientific articles, supports multiple input modalities, and contains detailed header and location information for table structures, making it useful for a wide variety of modeling approaches. Now, we are simply going to take this learning over to our specific use case of identifying tabular data in financial documents such as balance sheets.

Let us first explore how the DETR model performs on financial documents. We would not have to perform any fine-tuning using domain-specific data unless the results are not as per expectation. The first step is importing the class libraries:

```
#Step 1 - Load all the required Libraries
```

```
from huggingface_hub import hf_hub_download
```

```
from PIL import Image
```

```
#from lime import lime_image
```

```
import matplotlib.pyplot as plt
```

```
from transformers import DetrFeatureExtractor
```

```
from transformers import  
TableTransformerForObjectDetection
```

```
import torch
```

```
#Step 2- Function to visualize the highlighted  
tables in the scanned images
```

```
import matplotlib.pyplot as plt
```

```
# colors for visualization
```

```

COLORS = [[0.000, 0.447, 0.741], [0.850, 0.325,
0.098], [0.929, 0.694, 0.125],
          [0.494, 0.184, 0.556], [0.466, 0.674,
0.188], [0.301, 0.745, 0.933]]

def plot_results(pil_img, scores, labels, boxes):
    plt.figure(figsize=(16,10))
    plt.imshow(pil_img)
    ax = plt.gca()
    colors = COLORS * 100

    for score, label, (xmin, ymin, xmax, ymax), c
in zip(scores.tolist(), labels.tolist(),
boxes.tolist(), colors):
        ax.add_patch(plt.Rectangle((xmin, ymin),
xmax - xmin, ymax - ymin,
                                fill=False,
                                color=c, linewidth=3))
        text = f'{model.config.id2label[label]}:
{score:0.2f}'
        ax.text(xmin, ymin, text, fontsize=15,
                bbox=dict(facecolor='yellow',
alpha=0.5))
    plt.axis('off')
    plt.show()

#Step 3- Source the image location in the file_path
variable

file_path = '/content/Income Statement.png'

```

```
image_IS = Image.open(file_path).convert("RGB")
width, height = image_IS.size
# Resize the image to 90% of its original
dimensions
image.resize((int(width*0.9), int(height*0.9)))
#encode the image using the DetrFeatureExtractor
class
feature_extractor = DetrFeatureExtractor()
# Use the feature extractor to encode the resized
image
encoding = feature_extractor(image_IS,
return_tensors="pt")
# Get the keys of the encoding dictionary
encoding.keys()
#Load the pre-trained Table Transformer model for
object detection and then pass the image encoding
to the model object
model =
TableTransformerForObjectDetection.from_pretrained(
"microsoft/table-transformer-detection")
with torch.no_grad():
    outputs = model(**encoding)
# Post-process object detection results:
# 'image.size[::-1]' swaps the width and height to
match the target size format (height, width)
target_sizes = [image.size[::-1]]
```

```
# Post-process the object detection outputs using
the feature extractor by using a threshold of 0.6
for confidence

results =
feature_extractor.post_process_object_detection(out
puts, threshold=0.6, target_sizes=target_sizes)[0]
#Plot Results using the user-defined plot function
plot_results(image, results['scores'],
results['labels'], results['boxes'])
```

Output:

Financial Information for Textron's Finance and Insurance Subsidiaries

Table: 0.99

(in millions)	1993	1992	1991
Revenues			
Interest, discount and service charges	\$1,260.2	\$1,273.2	\$1,183.9
Credit life, credit disability and casualty insurance premiums	300.8	298.5	308.7
Non-cancellable disability income, life and group insurance premiums	836.2	795.0	764.6
Investment income (including net realized investment gains)	406.1	360.8	354.8
Total revenues	2,803.3	2,727.5	2,612.0
Costs and expenses			
Selling and administrative	789.7	757.8	702.4
Interest expense	432.3	488.9	510.4
Provision for losses on collection of finance receivables, less recoveries	152.6	160.4	134.8
Credit life, credit disability and casualty insurance losses and adjustment expenses, less recoveries	132.1	137.2	130.1
Death and other insurance benefits	392.9	369.9	362.3
Increase in insurance policy liabilities	324.1	316.7	319.8
Amortization of insurance policy acquisition costs	143.5	131.8	128.5
Total costs and expenses	2,367.2	2,362.7	2,288.3
Income before income taxes	436.1	364.8	323.7
Income taxes	174.6	138.9	118.1
Income before cumulative effect of changes in accounting principles	261.5	225.9	205.6
Cumulative effect of changes in accounting principles, net of income taxes	-	(44.7)	-
Net income	261.5	181.2	205.6
Minority interest in net income	2.6	-	-
Textron's equity in net income	<u>\$ 258.9</u>	<u>\$ 181.2</u>	<u>\$ 205.6</u>
Balance Sheet	December 31, 1993	December 31, 1992	
(in millions)			
Assets			
Cash	\$ 14.1	\$ 2.9	
Investments	4,759.9	4,144.8	
Finance receivables - net	7,605.4	7,069.5	
Property, plant and equipment - net	99.0	98.3	
Unamortized insurance policy acquisition costs	783.5	696.3	
Goodwill, less accumulated amortization of \$169.6 and \$151.0	299.1	317.7	
Other assets	660.1	541.1	
Total assets	<u>\$14,221.1</u>	<u>\$12,870.6</u>	
Liabilities and equity			
Accounts payable and accrued liabilities (including income taxes)	\$ 938.7	\$ 817.8	
Insurance reserves and claims	4,091.5	3,614.5	
Debt	6,846.7	6,439.7	
Equity:			
Textron	2,160.8	1,998.6	
Minority interest	183.4	-	
Total liabilities and equity	<u>\$14,221.1</u>	<u>\$12,870.6</u>	

Figure 11.7: Income statement table detection

The DETR model can identify the table with a 99% probability. Let us upload another image of a scanned page with multiple tables.

All the code steps combined in one block below detect multiple tables in one image:

```
# Source the image location in the file_path
variable

file_path = '/content/multiple_table.png'
image_MT = Image.open(file_path).convert("RGB")
width, height = image_MT.size

# Resize the image to 90% of its original
dimensions

image.resize((int(width*0.9), int(height*0.9)))

encoding = feature_extractor(image_MT,
return_tensors="pt")

model =
TableTransformerForObjectDetection.from_pretrained(
"microsoft/table-transformer-structure-
recognition")

with torch.no_grad():

    outputs = model(**encoding)

width, height = image_MT.size

results =
feature_extractor.post_process_object_detection(out
puts, threshold=0.7, target_sizes=[(height,
width)])[0]

plot_results(image1, results['scores'],
results['labels'], results['boxes'])
```


Output:

The increase in purchased patents, trademarks and other intangibles was primarily attributable to the acquisition of Salla S.p.A. discussed in Note 4.

Amortization expense totaled \$12.5, \$11.4 and \$6.9 in 1994, 1993 and 1992, respectively.

NOTE 9 - DEBT:

The components of short-term debt were as follows:

December 31,	1994	1993
Commercial paper	\$641.4	\$507.5
Notes payable - bank and other	250.6	125.5
Current portion of long-term debt	33.1	21.8
	\$925.1	\$654.8

The weighted average interest rate was 6.3 percent and 4.6 percent for commercial paper and notes payable outstanding at December 31, 1994 and 1993, respectively. The company has lines of credit arrangements with numerous banks with interest rates generally equal to the prime rate for domestic banks and the best prevailing rate for foreign banks. At December 31, 1994, worldwide unused short-term lines of credit amounted to \$1.0 billion.

The components of long-term debt were as follows:

December 31,	1994	1993
\$ 5/8% notes due 2002	\$199.6	\$199.6
8% notes due 1998	150.0	150.0
\$ 1/8% notes due 1996	100.0	100.0
Industrial revenue bonds due 2014	24.6	24.6
Other	61.0	71.1
	\$535.2	\$545.3

The industrial revenue bonds due 2014 have a stated interest rate of 7.6 percent and an effective interest rate of 7.2 percent.

The aggregate annual maturities of long-term debt at December 31, 1994, payable in each of the years 1996 through 1999, are \$116.3, \$6.3, \$165.4 and \$2.5, respectively.

The company has entered into interest rate swap agreements to reduce its interest expense on long-term debt, see Note 10.

NOTE 10 - FINANCIAL INSTRUMENTS:

The estimated fair values of financial instruments were

as follows:

	1994		1993	
December 31,	Carrying Amount	Fair Value	Carrying Amount	Fair Value
Investment securities	\$ 482.8	\$ 479.2	\$ 341.5	\$ 345.8
Long-term debt	(535.2)	(513.8)	(546.2)	(575.9)
Interest rate swaps	.3	(17.3)	5.1	8.8
Foreign exchange contracts	.1	(19.2)	—	(16.1)

Investment securities and long-term debt are valued at quoted market prices for similar instruments. The fair values of the remaining financial instruments in the above table are based on dealer quotes and reflect the estimated amounts that the company would pay or receive to terminate the contracts. The carrying values of all other financial instruments in the consolidated balance sheets approximate fair values.

The company adopted the provisions of SFAS No. 115, "Accounting for Certain Investments in Debt and Equity Securities," effective January 1, 1994. Adoption of SFAS No. 115 had no impact on earnings since all nonequity securities are categorized as "held-to-maturity" and, accordingly, continue to be carried at amortized cost. At December 31, 1994 and 1993, respectively, gross unrealized gains were \$.4 and \$4.3. Gross unrealized losses were \$4.0 at December 31, 1994. The investment securities portfolio was comprised of negotiable certificates of deposit, Puerto Rico government bonds, guaranteed collateralized mortgage obligations and Ginnie Mae certificates, repurchase agreements and short-term U.S. dollar-linked Mexican government bonds. Equity securities, categorized as "available-for-sale," were immaterial.

The investment securities (mentioned above) were reported in the following balance sheet categories:

December 31,	1994	1993
Cash and cash equivalents	\$ 74.6	\$173.1
Short-term investments	247.2	61.2
Investments and other assets	161.0	107.2
	\$482.8	\$341.5

As of December 31, 1994, long-term investments of \$161.0 included \$71.2 of interest-bearing, mortgage-backed securities maturing beyond ten years.

Figure 11.8: Multiple table detection in one PDF page

The DETR model identifies all four tables in a sea of text with high probabilities. Lastly, let us use the DETR model to see if it can detect rows and columns within a table.

```
# Load another simpler image to visualize rows and
columns clearly

file_path = '/content/simple.png'
image_sm = Image.open(file_path).convert("RGB")
width, height = image_sm.size
image.resize((int(width*0.5), int(height*0.5)))
#encode image and pass to the model as encoding
encoding = feature_extractor(image_sm,
return_tensors="pt")

model =
TableTransformerForObjectDetection.from_pretrained(
"microsoft/table-transformer-structure-
recognition")

with torch.no_grad():
    outputs = model(**encoding)

width, height = image_sm.size

results =
feature_extractor.post_process_object_detection(out
puts, threshold=0.7, target_sizes=[(height,
width))][0]

plot_results(image1, results['scores'],
results['labels'], results['boxes'])
```

Cell	Format	Formula
B4	Percentage	None
C4	General	None
D4	Accounting	None
F4	Currency	=PMT(B4/12,C4,D4)
F4	Currency	=E4*C4

Figure 11.9: Table structure (rows and columns) identification

[Figure 11.9](#) demonstrates excellent clarity in terms of identifying rows and columns within a table. All our three inferences with the DETR model are fairly accurate and hence, we do not see a need to fine-tune the weights of this DETR model for our financial domain-specific use case.

However, please note that the Microsoft research team already trained and fine-tuned the original DETR model proposed by Carion et al. on the PubTables-1M dataset and obtained superior results. The original DETR model was built on the COCO 2017 detection and panoptic segmentation datasets that contained 118k training images and 5k validation images.

Now that the table has been detected, we should try to extract the text within the table into a CSV file. We will use Python-tesseract, which is an **optical character recognition (OCR)** tool for Python. This library will use the coordinates of the identified table to recognize and “read” the text embedded in images. Let us install the libraries first.

```
#let's extract the actual table contents now
```

```
!pip install pytesseract pillow
```

```
!apt-get install -y tesseract-ocr
!apt-get install -y libtesseract-dev
!pip install pytesseract
import pytesseract
import pandas as pd

# Set the path to the Tesseract executable. This
# should work in Google colab but if it's not working
# try the tesseract_path command
pytesseract.pytesseract.tesseract_path =
'/usr/bin/tesseract'

import subprocess

Next, we need to extract the text using OCR within a set of
bounding boxes. The bounding boxes are created from the
DETR table structure detection model. We will extract the
coordinates of the box using the following code:

# Assuming the 'results' data frame from the
# feature extractor contains coordinate information
# about the detected tables

# let's explore the diagram co-ordinates
boxes = results['boxes']
labels = results['labels']

# Create a DataFrame to store the table information
table_data = {'Label': [], 'Box': []}

# Iterate through the detected tables
for i, (label, box) in enumerate(zip(labels,
boxes)):
```

```

table_data['Label'].append(label.item())
table_data['Box'].append(box.tolist())
# Create a DataFrame from the collected data
table_df = pd.DataFrame(table_data)
# Display the DataFrame
print("Detected Table Information:")
print(table_df)

```

Output:

```

Detected Table Information:
   Label  Box
0      2  [45.98406219482422, 216.60984802246094, 721.85...
1      2  [45.749053955078125, 116.96690368652344, 722.0...
2      2  [46.03727722167969, 65.96841430664062, 721.744...
3      1  [45.696136474609375, 21.0483341217041, 135.567...
4      2  [45.56620788574219, 164.09329223632812, 721.98...
5      2  [46.05487823486328, 21.0458927154541, 722.0369...
6      1  [376.44927978515625, 20.96285057067871, 721.66...
7      2  [46.171478271484375, 266.2109375, 721.35803222...
8      3  [45.73225784301758, 21.1954345703125, 722.3789...
9      1  [134.28567504882812, 20.84220314025879, 377.49...
10     0  [46.000240325927734, 21.167064666748047, 722.0...

```

Figure 11.10: Table structure coordinates

[Figure 11.10](#) shows various (total of 10) boxes, with their labels, that the model has detected. Please look at [Figure 11.11](#), which shows that the label '0' has the coordinates corresponding to the biggest box that is the entire table:

```

Cell Format Formula
B4 Percentage | None
C4 General None
D4 Accounting | None
E4 | Currency =PMT(B4/12,C4,D4)
F4. Currency =E4*C4

tensor([ 46.0002,  21.1671, 722.0531, 312.1535])

```

Figure 11.11: Table structure coordinates and corresponding text

Next, we will use the following code to perform text extraction using the `pytesseract` library within the bounding box of the label 0 or the 10th record:

```

# Flag for start extracting from i=10
start_extraction = False
num_columns = 3
for i, box in enumerate(boxes):
    if i == 10:
        start_extraction = True
    if start_extraction:
        # Crop the image to the bounding box
        table_image = image.crop(box.tolist())
        # Perform OCR to extract text
        table_text =
pytesseract.image_to_string(table_image)
        # Clean up extracted text

```

```

        table_text = table_text.strip() # Remove
leading/trailing whitespace

        table_text = table_text.replace('|',
''.replace('.', '')) # Remove '|' and '.'

        table_text = ' '.join(table_text.split())
# Remove extra whitespaces and newlines

        print(table_text)

        # Append the extracted text to the data
        # table_data['Text'].append(table_text)

        if len(table_text) > 5: # Adjust the
threshold based on your data

            # Split the 'table_text' into words
            words = table_text.split()

            # Calculate the number of rows needed
            num_rows = len(words) // num_columns

            # Reshape the words into rows and
columns

            table_data = [words[i:i + num_columns]
for i in range(0, len(words), num_columns)]

            # Print the result

            for row in table_data:

                print(row)

```

Output:

```

Cell Format Formula B4 Percentage None C4 General None D4 Accounting
None E4 Currency =PMT(B4/12,C4,D4) F4 Currency =E4*C4

```

```

['Cell', 'Format', 'Formula']

```

```
['B4', 'Percentage', 'None']
['C4', 'General', 'None']
['D4', 'Accounting', 'None']
['E4', 'Currency', '=PMT(B4/12,C4,D4)']
['F4', 'Currency', '=E4*C4']
```

Next, we will store the `table_data` in a data frame using the following code:

```
# Create a DataFrame from the reshaped data
# Extract the column names from the first-row
column_names = table_data[0]

table_df = pd.DataFrame(table_data[1:], columns=column_names)

# Save the DataFrame to a CSV file
csv_file_path = '/content/detected_table_text.csv'

table_df.to_csv(csv_file_path, index=False)
```

The exported file looks like in [Figure 11.12](#):

	A	B	C
1	Cell	Format	Formula
2	B4	Percentage	None
3	C4	General	None
4	D4	Accounting	None
5	E4	Currency	=PMT(B4/12,C4,D4)
6	F4	Currency	=E4*C4

Figure 11.12: *Extracted table in CSV*

Detailed code for all the steps is available in the GitHub repository.

Bonus Case Study #2: Building an iOS app that leverages a `createML` image classification model to make inferences.

Create ML is a machine learning framework provided by Apple for building and training machine learning models on macOS and iOS devices. It is part of the Core ML framework, which enables developers to integrate machine learning models into their applications. The best part of Create ML is its user-friendly GUI, and it is accessible for developers, including those who may not have extensive experience with machine learning or coding. The interface provides a drag-and-drop environment for building models. Developers can leverage transfer learning to use pre-trained models and fine-tune them for specific tasks or datasets.

The `createML` model is then deployed on an edge device such as an iPhone using an iOS app built on the Xcode platform.

Conclusion

In conclusion, this chapter has explored the inner workings of CNNs, with a focus on their intricate architecture designed for image recognition. The historical journey, from LeNet to *EfficientNet*, highlighted the evolution of CNNs, emphasizing their increasing depth, complexity, and efficiency. We explored the pivotal components, including convolutional layers, pooling, and fully connected layers, understanding how they contribute to feature extraction and hierarchical learning.

The chapter culminated with two compelling case studies: the PDF document parser highlighted the efficacy of CNNs in extracting information from complex documents, while the facial emotion recognition case study demonstrated the pivotal role of transfer learning in fine-tuning pre-trained

models for nuanced tasks. These applications underscored the pivotal role of transfer learning in CNNs, demonstrating its significance in real-world use cases, where pre-trained models provide a crucial head start in overcoming data constraints and improving model performance. This journey through CNNs illuminates their transformative impact on image processing and the broader landscape of artificial intelligence.

This chapter marks the end of the most common data mining techniques prevalent in the industry. However, modern data mining is not merely about pattern-finding and learning techniques but needs more focus on behalf of institutions to build trustworthy and efficient AI systems. The next two chapters will dive into the principles of trustworthy and risk-free systems that can be built for scale and transparency.

Further reading

More details on the model and dataset discussed in this chapter can be found at below links:

- Detection Transformers (DETR) model:
<https://arxiv.org/abs/2005.12872>
- PubTables-1M: Towards comprehensive table extraction from unstructured documents:
<https://arxiv.org/abs/2110.00061>
- Author's YouTube video on how to build an image classifier using CreateML:
https://www.youtube.com/watch?v=M58_XOyJW04
- Derivation of Backpropagation in Convolutional Neural Network (CNN):
<https://www.semanticscholar.org/paper/Derivation-of-Backpropagation-in-Convolutional-%28-%29-Qi/5d7911c93ddcb34cac088d99bd0cae9124e5dcd1?p2df>

- Backpropagation Mechanics for a Convolutional Neural Network:
<https://cbelwal.blogspot.com/2018/05/part-i-backpropagation-mechanics-for.html>

Points to remember

- Data quality and quantity ensures the quality of your training data, with accurate and well-labeled images. We saw how TATR benefits from the canonical data labeling approach on 1M images. Sufficient data is equally crucial, especially for deep learning which helps the model generalize better.
- Data Augmentation helps to artificially increase the diversity of your training dataset. This includes rotation, flipping, zooming, and changes in lighting. Pre-processing images is another approach that helps the model learn effectively by appropriately resizing, normalization, and centering.
- Choose an appropriate CNN architecture based on the complexity of your task. Popular architectures for inspiration include VGG, ResNet, Inception, and EfficientNet.
- If building a CNN model from scratch is an issue then leverage pre-trained models through transfer learning, especially if you have limited labeled data. Fine-tuning pre-trained models often leads to better results.
- When training CNN models from scratch, then remember to optimize hyperparameters such as learning rate, batch size, and the number of epochs. Experimentation and tuning are crucial for model performance.
- Implement regularization techniques like dropout and L2 regularization to prevent overfitting, especially and

choose appropriate evaluation metrics for your specific task. Take special care in the case of smaller datasets.

- Interpretability is a big concern in computer vision as well. Consider the interpretability of your model. Depending on the application, it might be crucial to understand how the model makes predictions, especially in sensitive domains.
- If deploying models to mobile or edge devices, consider the computational efficiency of your model for on-device inference. **CreateML** models are best because some of their image classification model objects are as small as 50KB but optimize the model size and architecture if you are building from scratch.
- Continuously monitor and evaluate model performance over time. Be prepared to retrain or fine-tune the model as needed, especially when working with dynamic datasets.
- Lastly, computer vision has immense potential to impact human lives. Be mindful of ethical considerations related to biases in the data and the potential impact of model inferences in certain cases such as human surveillance. It is very important to regularly assess and mitigate biases in your model.

Join our book's Discord space

Join the book's Discord Workspace for Latest updates, Offers, Tech happenings around the world, New Release and Sessions with the Authors:

[**https://discord.bpbonline.com**](https://discord.bpbonline.com)



CHAPTER 12

Understanding Model Risk Management for Data Mining Models

Introduction

So far in this book, we have discussed the best practices to build models that are accurate, unbiased to a great extent, and manageable. However, it is equally important to understand, identify, and handle the risks associated with the use of these models when they are being developed or are out in a production environment making inferences on previously unseen data. This chapter will shed light on various guidelines, regulations, and technology-driven approaches to risk mitigation strategies that can start at a developer's desk and benefit all downstream processes. We will conclude with an industry-focused case study to make clear the role of a model developer in risk management efforts within an organization.

Structure

The chapter will cover the following topics:

- Data mining challenges and risks
 - Why do model risks occur
- Introduction to Model Risk Management
 - Key regulatory frameworks
 - Pillars of Model Risk Management
- Introduction to Model Operations
- ModelOps: Product first vs. model first mindset
- How ModelOps facilitates MRM
- Case study: Regulatory requirement fulfillment using MRM and ModelOps

Objectives

As practitioners of modern data mining, it is essential for readers to understand various model risks and the numerous ways they can occur. This chapter aims to build a comprehensive understanding of these risks and discusses different regulations and guidelines that are followed in the industry to mitigate these risks. However, how can the Financial Institutions that deploy many frameworks and tools to reduce model risks, benefit specifically from principles of model operations (ModelOps) to reduce model risk? Can implementation of ModelOps principles using technology fulfill key risk management requirements to ensure the reliability, fairness, and robustness of machine learning models in real-world applications? Readers will get answers to the above questions using an industry-focused case study that discusses a key U.S. regulation in the credit risk space.

Data mining challenges and risks

As discussed in [Chapter 1: Understanding Data Mining in a Nutshell](#), modern data mining faces many challenges that

can be categorized into two broad buckets: learning and usage challenges. In this chapter, we are going to expand upon these challenges and understand the associated risks if the challenges are left unaddressed.

Let us start by understanding some of the challenges associated with building modern data mining solutions that can be categorized as training and usage related:

Model training:

- **Data:** Quality, volume, and complexity of data resources used for building data mining solutions lead to the challenge of large heterogeneous data sources that may potentially drive problems downstream. For instance, inconsistent FICO scores from different score providers can lead the bank to approve loans to less deserving applicants. A worse situation would be high volume of customer data that could not be integrated in time for credit decision-making.
- **Model scalability, explainability, and interpretability:** New learning techniques are becoming more complex driving explainability, interpretability, and scalability concerns. For instance, the inability to scale an ML-based credit scoring system as the loan application data becomes complex. This might demand using cloud-based infrastructure to allow for high-speed model training and deployment based on demand.
- **Code/algorithm:** Privacy and Security could be a huge issue if the algorithms are built to do that. For instance, **Large Language Models (LLMs)** might 'learn' from your prompts and offer that information to others who query for related things.

Model usage:

- **Post-deployment:** Model monitoring and Infrastructure-related issues may drive the breakdown of the decision-making systems built upon the data mining solution. This could be due to a rapid increase in the number of loan applications, needing the model to scale seamlessly to process the growing workload without sacrificing performance.
- **Humans in the loop:** Data mining solutions require human expertise to ensure that the results obtained are meaningful, relevant, and within the guardrails. It is essential to have individuals with a good understanding of the data and the business context to ensure that the results are actionable, validated, and governed.
- **Talent:** Model developers or engineers may not be adequately skilled or may not have the incentives to deliver best-in-class outcomes.

These model training or usage challenges can lead to numerous risks if not addressed appropriately at various stages of the model lifecycle. However, before we list these risks, let us understand what is defined as a risk from the perspective of a data mining solution.

The **International Organization for Standardization (ISO)** defines risk in the context of enterprise risk management as the *effect of uncertainty on objectives*. According to the Open Risk Manual, *Model risk refers to the potential for error in the development, implementation and/or the application or interpretation of results produced of a model. Model risk-related errors are risk factors that can lead to a variety of financial and/or reputational Loss events. Model risk is generally considered to be a type of Operational Risk.*

Another standard way to define the model risk incident is the estimated likelihood of the incident's occurrence multiplied

by its probable monetary impact. For a given data mining system, if the probability of an incident occurring is high, or the monetary or other loss associated with a system incident is large, or if both quantities are large, organizations need to spend extra resources on ensuring the overall safety of that system:

$$\text{Model Risk} = \text{Probability} * \text{Monetary impact (due to the risk incident occurrence)}$$

While Monetary impact is related to the organizational aspirations related to a project, it can be large in many cases, depending on the importance and size of the initiative. Hence given the above definition, it is in the organization's interest to reduce the probability of the occurrence of such model-related risks. A potential list of risks that can be mitigated to reduce the overall probability of failure is listed in [Table 12.1](#) below:

Sr.No.	Risk	Description	Data mining challenge that cause the risk
1	Poor model performance	If the model is developed using low-quality data or flawed models, it may not perform well when deployed. This could lead to incorrect predictions, recommendations, or decisions, leading to financial losses, customer dissatisfaction, or legal issues.	Data, Model
2	Biased results	Data biases can lead to biased model results. If a model is biased towards certain demographics or groups, it may lead to unfair or discriminatory decisions. For example, a hiring model that is biased towards certain educational qualifications could result in discrimination against candidates without those qualifications.	Data, Model

Sr.No.	Risk	Description	Data mining challenge that cause the risk
3	Data breaches	Poor data quality can make it difficult to identify or address the vulnerabilities in a system. Data inconsistencies or inaccuracies can create confusions and make it difficult to detect unauthorized changes or activity. This can lead to data breaches, which can have serious consequences businesses and individuals.	Data, Talent
4	Costly retraining	Costly Retraining: If data quality issues are not addressed during model development, the model may need to be retrained frequently to maintain its accuracy. This can be time-consuming and expensive	Data, Human-in-the-loop, Talent
5	Security breaches	If data privacy and security are not addressed, it can lead to security breaches, which can result in the exposure of sensitive information or unauthorized access to data. This can have severe consequences for individuals and organizations.	Algorithm, Human-in-the-loop, Talent, Post-deployment
6	Lack of trust	If the model is not interpretable, it can lead to a lack of trust in the results obtained. This can lead to resistance from stakeholders and a reluctance to implement the model, which can impact its effectiveness.	Model, Algorithm, Human-in-the-loop, Talent, Post-deployment
7	Reduced adoption	Failure to address data-related challenges during model development can lead to reduced adoption of the model. This can negatively impact the value derived from the model and limit its impact on the organization.	Data, Model, Algorithm, Human-in-the-loop, Talent, Post-deployment

Sr.No.	Risk	Description	Data mining challenge that cause the risk
8	Damage to brand reputation	Poor model performance, biased results, and data breaches can all damage a company's reputation. This can result in lost business and damage to the company's brand image.	Data, Model, Algorithm, Human-in-the-loop, Talent, Post-deployment
9	Regulatory Non-Compliance	Many industries have regulations that govern the use of data and models. If the model does not adhere to these regulations, it could result in legal consequences.	Data, Model, Algorithm, Human-in-the-loop, Talent, Post-deployment

Table 12.1: Model Development Challenge-Potential Risk Mapping

Note: Efforts to realize the return on investment (ROI) of data mining solutions should start at the data scientist level and go all the way up to executives struggling to rationalize data mining-related financial decisions.

Why do model risks occur

A simplistic response to this question is the inability to recognize and address the challenges in the model lifecycle at the right time. The mapping of these challenges to the risks they can potentially cause can be observed in the above [Table 12.1](#).

Let us try to understand this using an example where a lack of appropriate attention to AI model development and

operation led to regulatory non-compliance. In this particular case, the **Consumer Financial Protection Bureau (CFPB)** fined a large bank for using an AI-based model that violated the **Fair Credit Reporting Act (FCRA)**.

The bank had developed a model that used data from credit reports to predict the likelihood of consumers defaulting on their loans. However, the CFPB found that the model unfairly discriminated against certain groups of borrowers, including older borrowers and those living in predominantly minority neighborhoods. The bank was fined \$22 million for violating the FCRA, which requires that credit reports be accurate, fair, and non-discriminatory.

This case illustrates the risks associated with using AI models in regulated industries, particularly when those models rely on data that may contain biases or inaccuracies. Even well-intentioned models can produce discriminatory or unfair results if they are not designed and validated appropriately.

Introduction to Model Risk Management

When the stakes due to automated decision-making and growing digitization are high, risks associated with these models making the decisions are bound to be high. The promise and wider applications of models have brought into the limelight the need for an efficient **Model Risk Management (MRM)** standard within organizations. Driven by regulatory needs, financial institutions have invested millions in building and deploying highly sophisticated MRM frameworks. One such framework is SR 11-7, the supervisory guidance on Model Risk Management, which came into being in April 2011, as learning from the great financial crisis.

This guidance provided an early definition of model risk that subsequently became standard in the industry: *The use of models invariably presents a model risk, which is the potential for adverse consequences from decisions based on*

incorrect or misused model outputs and reports. The key point to note here is that SR 11-7 explicitly addresses incorrect model outputs and wrong uses of the correct model outputs. This can be driven by factors across the model lifecycle and at any point from design through implementation.

Meanwhile, the European Banking Authority's Supervisory Review and Evaluation Process has a slightly different requirement wherein the model risk is identified, mapped, tested, and reviewed for material risk assessment to capital.

A few key points to note with respect to MRM are:

- MRM is not about managing the risks only once the model is deployed:
 - It starts at the conceptualization phase and the onus is passed onto the developer to ensure the concept is implemented as conceptualized. The ball is then passed onto the validators to ensure the model outputs result as expected and is conceptually sound.
 - Upon successful validation, the deployment and post-deployment teams take over to ensure risk-free implementation.
- Model risk management is a cultural manifestation of any organization and starts at its grassroots level. It is not just about having a senior-level committee that oversees risk management efforts.
- MRM functions often involve accountable executives and several teams that are responsible for the safety and performance of models and data mining systems.
- MRM drives risk management takes time, people, money, and other resources.

- An effective MRM helps in cost reduction and loss avoidance. This happens due to increased operational and process efficiency in model development and validation, including the elimination of defective models.

It is interesting to know that many large and mid-size U.S. banks have adopted a standardized development and validation process by aligning templates to use cases and more focused on conceptual soundness standards, which can be customized by use-case and modeling techniques. The right standards and templates will enable automation of tasks in development, documentation, and validation and will also facilitate audits. Furthermore, the financial institutions formalized interactions between the model development and validation, creating requirements for model submission, as well as timelines for model validation. This goes to the extent of entering into service-level agreements with each other.

As stated above, model risk can happen throughout the model lifecycle, from development to deployment and beyond. Following [Figure 12.1](#) outlines various model risks across the model lifecycle:

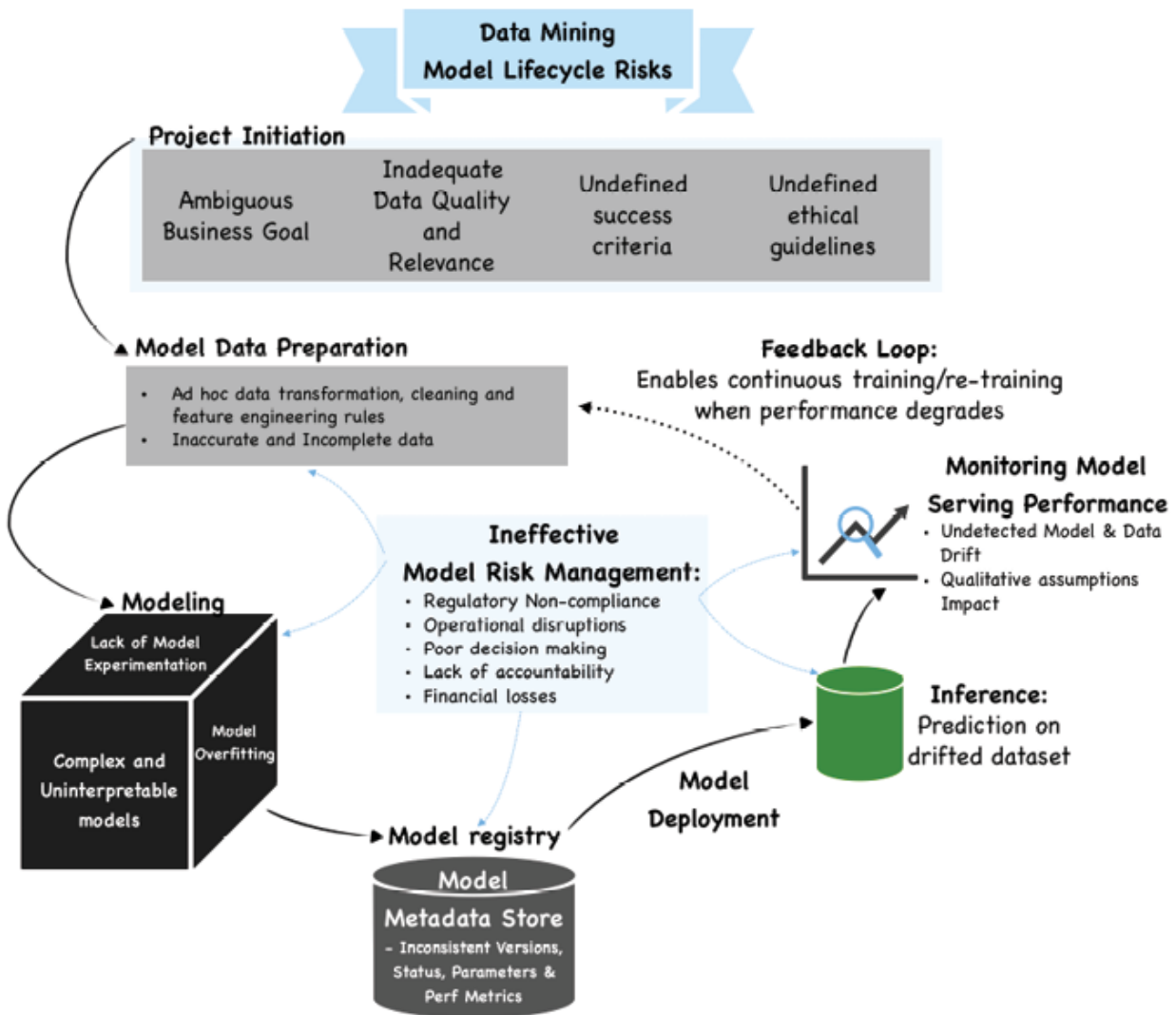


Figure 12.1: Model risks across the model lifecycle

Please note how an ineffective MRM, which is at the center of the model lifecycle processes, impacts all of its stages.

For this reason, almost all leading financial institutions that use or plan to use predictive modeling employ some level of MRM practices. Some key ingredients of a successful MRM function are as follows:

- **Culture:** Risk management is about the organization's culture and flows from the top. It is still everybody's responsibility to incorporate risk identification and mitigation in their daily jobs and strive for continuous

improvement. This would include ongoing employee training and development and regular MRM processes and controls review.

- **Incentives:** It is prudent to tie risk management with institutional metrics such as cost reduction and loss avoidance which are the product of increased operational and process efficiency in model development, validation, and elimination of defective models. This would be a huge incentive for MRM teams specifically.
- **Accountability:** A well-defined and established governance structure is critical for effective MRM. This includes clearly defined roles and responsibilities, escalation protocols, and oversight mechanisms to drive accountability for failures.
- **Rigor:** Perform the unit test, integration tests, and user acceptance tests before passing them on to real users. Use documentation as the most effective risk control measure and design test cases that are in line with the business use case.
- **Layered defense:** This helps with the effective challenge of models by different teams. For instance, developer work is challenged by validators, who are further audited by the internal audit teams operating independently of both teams. Independence is a critical component of any review process.

According to the revered SR 11-7 guidance, *Even with skilled modeling and robust validation, model risk cannot be eliminated*. Hence, the goal should be to proactively identify as many open loopholes and plug them to reduce the risk.

Proactive Model Risk Management should start with Model Management during the conceptualization, design, and development phases. For instance, it is the duty of data

scientists to ensure the conceptual soundness, explainability, and reproducibility of the models by using accurate data and the appropriate modeling techniques. This should be done by following model-appropriate assumptions and limitations and tracking the path to the chosen model version.

Key regulatory frameworks

Federal Trade Commission (FTC) states that in *Aiming for Truth, Fairness, and Equity in Your Company's Use of AI, Hold yourself accountable—or be ready for the FTC to do it for you*. The FTC further mentions *Consider how you hold yourself accountable, and whether it would make sense to use independent standards or independent expertise to step back and take stock of your AI*.

Other regulators are much nicer in their prescriptions wherein they talk about high-level guidance that needs to be followed with respect to the use of data, model, and governance.

Some of the key regulatory briefings are listed as below:

- **General Data Protection Regulation (GDPR):** The GDPR is a European Union regulation that governs the protection of personal data. It requires that organizations implement measures to protect the privacy of personal data, including data used for AI and data mining models. GDPR and other regulations may limit the use of features used to build models.
- **NIST AI Risk Management Framework (AI-RMF):** This relatively new guidance, published on January 26th, 2023, seeks to cultivate trust in AI technologies and promote AI innovation while mitigating risk. The AI RMF follows a direction from US Congress and was produced in close collaboration with the private and public sectors.

- **Consumer Financial Protection Bureau (CFPB):** The CFPB is a U.S. regulatory agency that oversees consumer financial products and services. It has issued guidance on the use of AI and machine learning in consumer financial services and has emphasized the importance of fairness, transparency, and explainability in these models.
- **Federal Reserve SR 11-7:** This supervisory guidance from the Federal Reserve also applies to AI and data mining models. It provides guidance on model development, validation, implementation, and monitoring, as well as governance and controls.
- **Basel Committee on Banking Supervision (BCBS) 239:** This global standard applies to AI and data mining models in the same way as other models. It provides guidance on effective risk data aggregation and risk reporting, including principles for ensuring that banks have accurate and timely risk data.

The regulatory landscape for AI and data mining model risk management is still evolving. Still, financial institutions must be aware of and comply with existing regulations related to data privacy, consumer protection, and risk management. This basically means that data scientists and other experts in the industry need to be aware and fully trained to deliver as per these compliance requirements. These risk management guidelines should not be looked at as guidance for low-level technical advice.

Pillars of Model Risk Management

This section will outline the different actions MRM performs to mitigate risks. While developers are not directly responsible for performing these duties awareness about these topics would help them perform their duties

appropriately. Before we explain this further, first let us go through various pillars of MRM:

- **Risk-tiering:** Risk mitigation is an ongoing and expensive task, especially in large organizations with hundreds of models, and hence should be done as per the criticality of the model. For instance, high-risk models should be allocated more resources compared to low-risk ones.
- **Conceptual soundness:** This involves evaluating the underlying assumptions and theoretical basis of the model. This involves reviewing the model's design and structure to ensure that it is based on sound principles and has a solid conceptual foundation.
- **Input data validation:** This is done to ensure the accuracy, completeness, and consistency of data fed to models
- **Reproducible results:** Model results should not change from one environment to another or from one user to other. Effective packaging and storage of model artifacts in organized inventories with data and model versions need to be ensured.
- **Output analysis and model monitoring:** Model outputs need to be back-tested along with sensitivity and other drift checks. These tests need to be performed at specified intervals to detect any deviation from the intended model behavior
- **Governance and change management:** It refers to the framework of policies, procedures, and controls that are established to manage model risk within a financial institution. Governance provides the structure and oversight necessary to ensure that models are developed, validated, and used in a responsible and effective manner. For instance, change management

and access control are part of an effective governance framework.

- **Documentation:** Last but not least, nothing can be reproduced or maintained for too long without exhaustive documentation. MRM standards require that all systems be thoroughly documented and standardized across systems for the most efficient audit and review processes.

Now that you understand the important areas of focus for MRM, the follow-up question should be: Which of these areas are impacted by the work of a model developer? The response to this question is that except for risk-tiering and governance, the rest of the five areas are directly controlled by a model developer.

For instance, questions such as those below cannot be answered accurately if developers have not taken proper care in building a model that can be logged in a central model repository.

- How many ML systems are currently deployed?
- How many customers or users do these systems affect?
- Who are the accountable stakeholders for each system?

Model inventories can serve as a repository for crucial information for documentation, but should also link to monitoring plans and results, auditing plans and results, important past and upcoming system maintenance and changes, and plans for incident response. The role of model developers is very crucial in the upkeep of these inventories.

Introduction to Model Operations

Model risk comes in various shapes and sizes and stems from immature model lifecycle handling.

As discussed in [Chapter 1, Understanding Data Mining In A Nutshell](#), the **National Institute of Standards and Technology (NIST)** specifies another framework (**NIST AI Risk Management Framework (AI RMF)**) for managing **Artificial Intelligence (AI)** risk to individuals, organizations, or society. These principles are deployed using modern-day technology built around the philosophy of **Model Operations (ModelOps)**. ModelOps is a principle-based approach to model development and usage that focuses primarily on the governance and life cycle management of a wide range of operationalized AI and decision models. For instance, this approach shortens the model development process and improves model stability by automating the repeatable steps, thereby reducing related risks.

Notably, similar to any other innovation endeavor, model development also involves comprehensive experimentation. This requires time commitment, experimentation design skills, and the presence of experimentation guidelines to ensure experiments are interpretable and reproducible. This helps avoid and explain failures to make the model development process efficient. Interestingly, some common model development challenges using the traditional experimentation approaches are:

- Inability to compare different experiments' statistics effectively in MS Excel or worse trying to memorize them.
- Inability to revert to the previous version of the model since the model artifacts and results stayed with the model developer on their laptops.
- Inability or difficulty deploying the champion model since it is hard to track the corresponding code, hyperparameters, transformations, and data. This is all developer-specific knowledge that is beyond the IT/Application deployment team.

- Inability to reproduce the development results in production since the deployment became a patchwork exercise involving data, modeling, and IT teams who speak different production-grade languages.
- Continuous integration/continuous deployment and continuous tracking are impossible to implement with such fragmented model lifecycle management.

ModelOps solves these challenges and more that have now become unavoidable with the rise of AI and ML algorithms. MLOps, as a specific ML-focused instance of ModelOps, is a set of best practices for different components of an ML workflow. This allows organizations to experiment with different experiment settings efficiently and keep what works for them. MLOps helps organizations and business leaders generate long-term value and reduce model lifecycle management risks associated with data mining, machine learning, and AI initiatives.

By setting a clear, consistent methodology for model operation, and more specifically machine learning operations, organizations can:

- Proactively address common business concerns (such as regulatory compliance).
- Acquire, clean, and version large amounts of data.
- Enable reproducible models by tracking data, models, code, and model versioning.
- Package and deliver models in repeatable configurations to support reusability.
- Enable greater collaboration between data scientists and engineers to build production-class services.

In the following section, we will discuss the approaches that enable data scientists to take maximum benefit from the ModelOps philosophy, eventually leading to high-value, low-

risk models that are less complex and hence easy to manage.

ModelOps: Product first vs. model first mindset

From an organizational perspective, ModelOps means risk management, efficiency, and agility. However, what does it mean for a **data scientist (DS)** or a model developer?

As DS matures in their role within the organization, they would realize, more often than not, that their models performed really well in the Jupyter notebooks, but ultimately failed to answer the business questions. The reverse argument is also true when the focus should be on building a robust and transparent model that can deliver accurate results. This balance is what is termed as product vs model balance. What should be prioritized?

The short answer to this dilemma is realizing the goal of the exercise at hand. A data scientist, with a varied skillset, can play a role across the model lifecycle. It is important for a DS to adopt a product mindset when you start building a model and keep the broader impact of the model in mind. When you are neck-deep in model tuning and iterating through experiments, adopt a model-first approach.

This approach will help you deliver useful and high-quality predictions that business users can act on. This would mean collaborating with data engineers, ML engineers, business function owners, and so on, and building an overall alignment with product or service while ensuring seamless integration of machine learning components with existing systems. It is also important to iterate through your models based on user feedback and not just based on the quest for the best model metrics.

Once the overall product development goal is clear, a DS can put on the model developer hat and tune the model within

the constraints of business and other external requirements such as customers and regulators.

This flexibility in approach will have the following advantages:

- It builds credibility with users, and regulators while driving better adoption
- It helps reduce model complexity since the end goal is clear
- It reduces the model development effort since debugging, artifact storage, and experiment tracking is all streamlined due to clear alignment with other internal stakeholders

Product-first vs Model-first approaches will lead to a completely different set of model artifacts and much of this will drive the downstream complexity related to model and model risk management. For instance, A proof of concept model will require a lot more effort and artifacts to be created and logged for showcasing the best model compared to a production data mining pipeline. In the case of the latter, a DS is only required to train a new model based on the metrics that matter for the end product and an extensive set of experimentation may not be required.

Note: Never build a model around a new, fancy technology or tool that can easily cause concept drift.

How ModelOps facilitates MRM

With an understanding of MRM and its need in the industry, we will try to match these needs with what ModelOps features have to offer. As ModelOps is applicable during the model development, deployment, and post-deployment

phases, we will highlight the aspects of ModelOps that will help data scientists develop models that are transparent, explainable, less complex, and hence easy for downstream consumption.

Interestingly, it is easy to confuse ModelOps as a post-model development exercise only, while ignoring the fact that many model usage-related risks sneak in during model training. Hence, the underlying safety principles that need to be followed include tracking model development environments, testing champion-challenger models, and versioning and storing models, to enable easy rollback and deployment.

Other things that the organizations may want to keep tracked and validated using ModelOps are:

- Quantity and quality of input data
- Feature store: Feature schema for the input data that is cleaned and transformed for model development and inferencing
- Model Registry serves as a model store
- Continuous integration and continuous deployment of models
- Model monitoring to detect concept or data drift

Throughout this book, we have used a popular open-source platform MLFlow for managing machine learning workflows. It is used by MLOps teams and data scientists to track experiments and provide pieces of evidence of model development when validators come knocking. It is common practice for model validators to ask for evidence of model development including experiments performed, model outcomes by versions, input data accuracy, and completeness. MLFlow and many similar ModelOps tools in the market help make the process transparent and reproducible for all parties involved.

Before moving on to the next chapter, where we will learn to build a machine learning-based product using different ModelOps tools to overcome the nine risks mentioned in [Table 12.1](#), let us go over an industry-focused case study.

Case study: Regulatory requirement fulfillment using MRM and ModelOps

As part of this case study, let us start by understanding a critical regulation known as **Current expected credit loss (CECL)** in the credit risk industry. The CECL model under **Accounting Standards Update (ASU) 2016-13** aims to simplify U.S. **Generally Accepted Accounting Principles (GAAP)** and provides for a forward-looking recognition of credit losses (that is, expected losses) instead of accounting for incurred losses on financial instruments such as loans and debt securities. In simple terms, the regulations suggest making balance sheet reserves for credit-related losses based on the expected future outlook and not just based on what has happened in the past. This translates to forecasting the losses over the life of the loan based on historical data layered with assumptions for the future.

Any **Financial Institution (FI)** under CECL regulation must explain and justify its entire CECL process, including (but not limited to) model development, validation, and governance. Meeting the complex demands of CECL regulation by manual processes is quite difficult but the broader SR 11-7 guidelines will eventually help both FIs and non-FIs develop effective CECL processes to limit model risk. Similar to any other advanced model in the financial services industry, CECL requires complex modeling inputs, assumptions, analysis, and other necessary compliance processes, such as:

- Alternative benchmark models
- Independent model validation and model governance

- Model back-testing and sensitivity analysis
- Model tuning for balancing bias and variance
- Ongoing model performance monitoring
- Documentation

Regardless of an institution's chosen modeling methodology for estimating expected lifetime losses, a **model risk management (MRM)** framework driven by SR 11-7 guidelines can help minimize potential risks. Such risks include unintended usage, wrong model development, or broken deployment and monitoring.

Now that we understand CECL and some of the associated risks, let us understand how MRM practices supported by ModelOps tools and technology can alleviate model risks in this scenario. Notably, both CECL and MLOps are principle-driven and not prescriptive in nature. An MLOps philosophy includes a process for streamlining model training, packaging, validation, deployment, and monitoring in such a way that suits an organization's size and needs.

While the **Financial Accounting Standards Board's (FASB)** implementation terms for CECL appear daunting, MLOps can help resolve CECL-related issues ranging from business implications, data management, and credit modeling to risk, governance, and technology. The issues presented in [Table 12.2](#) below are typical of a machine learning model life cycle inside an organization where many different professionals with varying skill sets attempt to use entirely different tools. CECL being a model-dependent process faces similar challenges.

Sr.No.	CECL challenge	ModelOps Solution
--------	----------------	-------------------

Sr.No.	CECL challenge	ModelOps Solution
1	Data There are three primary CECL challenges: 1. Data identification and exploration: If internal data is not sufficient for the expected credit loss estimate, external data might be needed. 2. Data collection: Entities will be required to maintain aggregated historical credit loss information for financial assets with similar risk characteristics. 3. Data and feature storage: Data should strive to be of the highest quality and accuracy and structured for flexible applications (for example, to enable multiple model types).	The MLOps data management principle enables organizations to: Quality check all external data sources Track, identify, and account for changes in data sources Write reusable scripts for data cleaning and merging. Create, modify, and store easily accessible features Ensure data labeling is strictly controlled Make data sets available on shared infrastructure (private or public).
2	Model development Management might need to employ several modeling techniques for different financial assets, depending on the company's portfolio. Further, parallel runs and stress testing of several different models might be necessary to determine which model produces the most accurate estimate of the current expected credit losses. Management should weigh multiple factors that might have a material impact on their estimate of the credit losses and infrastructure	MLOps tools have experimentation tracking capabilities, including: Experiment tracking tools that help with benchmarking various models created by different companies, teams, or team members. This MLOps guideline is focused on collecting, organizing and tracking model training information across multiple runs with different configurations (such as hyperparameters, model size, data splits and parameters).

Sr.No.	CECL challenge	ModelOps Solution
3	Technology & Infrastructure The accounting standard for CECL, with an implementation date of January 2023, will require companies to determine whether their existing technology can support the new standard and evaluate potential data, modeling, and deployment solutions. Additionally, higher velocity data may need to be captured and leveraged for model retraining and redeployment at a faster frequency. The modeling team should test multiple models and validation techniques and then run them through the organization's existing technology to understand potential limitations.	MLOps principles focus on model lifecycle tasks that lead to key architectural choices about the storage, processing and retrieval of data. This is in addition to model development, optimization, serving, orchestration and scaling strategies. The MLOps tool market is replete with open-source and vendor tools that offer various benefits depending on the immediate needs and implemented strategies of an organization.
4	Model deployment and maintenance Having a post-implementation plan is a component to success that often gets overlooked, but meeting the CECL implementation requirement is not the end of the project. A sustainable and repeatable process is needed for model refinement, ongoing data collection and other production requirements.	A post-implementation support plan includes: • Model deployment and maintenance • Technology and infrastructure Maintaining a central model registry to track trained, staged and deployed ML models by storing model lineage, versioning, metadata and configuration Ensuring models are validated efficiently and predicting accurately Ensuring visibility into the data quality, drift and lineage Maintaining a compliance audit trail.

Sr.No.	CECL challenge	ModelOps Solution
5	Model Documentation CECL requires disclosure of the allowance estimation methods for credit losses and key assumptions used to develop estimates. In addition, the effectiveness of these methods and assumptions needs to be monitored post-implementation. Implementation itself requires a certain level of coordination between finance, credit, risk, IT and other functions, and appropriate processes and controls need to be set up.	The goal of MLOps is to integrate model development with other systems across an organization. This requires communication, alignment and collaboration with others outside the team. MLOps tools offer communication, visualization, dashboards, and other democratization tools. These resources make it easier for teams with different professional backgrounds to collaborate on model development by establishing a framework that provides complete control over the access, review and approval workflows. This is achieved by building a RACI (responsible, accountable, consulted and informed) structure, developing automated processes, activating controls and writing documentation for effective disclosure and regulatory needs.

Table 12.2: *CECL challenges vs ModelOps solutions*

Please note that prior to CECL, the incurred loss methodology did not require any future loss forecasting but mandated reporting just the historical losses. This only required FIs to collect historical data and most quarterly reporting could be done in Excel. However, with the rising complexity in credit risk modeling, CECL regulation presents a great opportunity to streamline the execution approach using ModelOps.

In the next chapter, we are going to get hands-on and dig deeper into these ModelOps capabilities to build a data mining/machine learning-based application. This product will demonstrate compliance with the principles of model risk management as per the industry standards. Additionally, this will also allow readers to bring together various learnings from the book to build a Python-based web application with a

focus on mitigating the model risk while learning the product development process.

Conclusion

Among the many model types that are proliferating are those that potentially deliver high business value. Naturally high value cannot be realized without high-risk stakes. However, that does not mean risk cannot be mitigated if not completely eliminated. There are many widely respected risk mitigation frameworks that are designed to meet regulatory requirements and protect overall business goals.

Financial Institutions particularly have adopted these frameworks and have established sophisticated MRM functions that review and validate the models, thereby managing the associated risks. However, risk management responsibility also lies with developers who are putting together the build blocks, the need to follow a principled approach to model development by leveraging MRM and ModelOps to deliver high-quality, low-risk business products.

In the next chapter we will learn about adopting ModelOps.

Points to remember

- Challenges in building data mining solutions pose numerous risks if the challenges are not addressed appropriately
- The risk management related to building and operating a data mining solution is handled by large organizations such as FIs by leveraging MRM frameworks
- There are no standard certifications or courses available for MRM in the market but different public and private institutions have laid out guidelines for consideration

- SR 11-7 and NIST's AI-RMF are some of the prominent frameworks in the financial services industry alongside CFPB, GDPR, etc.
- For efficient implementation of these frameworks, technology frameworks such as ModelOps are helpful
- ModelOps tools such as MLFlow help DS with experiment tracking, registering models in a single repository, code reproducibility, standard packaging, and deployment, etc. There are many other tools that help with data and post-deployment-related risk management.
- Data scientists play a key role in risk mitigation since model risk management starts with model management
- Risks are prevalent across the model lifecycle from conceptualization to maintenance to end-of-life, hence it is important to have a product mindset and think holistically for deriving maximum value from data mining or machine learning initiatives.

Join our book's Discord space

Join the book's Discord Workspace for Latest updates, Offers, Tech happenings around the world, New Release and Sessions with the Authors:

<https://discord.bpbonline.com>



CHAPTER 13

Adopting ModelOps to Manage Model Risk

Introduction

The art of AI lies not only in the algorithms but in the conscious choices of its creators.

In the ever-evolving landscape of data-driven decision-making, fairness, transparency, and efficiency are both a conscious choice and a commitment. The world of finance, in particular, has recognized the profound importance of fairness in lending practices. As we stand at the crossroads of financial innovation and ethical responsibility, it is essential to explore how **Model Risk Management (MRM)** can pave the way for fairer and faster lending decisions.

In the previous [Chapter 12, Understanding Model Risk Management for Data Mining Models](#), we talked about MRM, and ModelOps while understanding their relationship using a **current expected credit loss (CECL)** model example. Fast forward to this chapter, we dig deep into the realms of MRM, Fair Lending, and the innovative power of a ModelOps solution. At its core, this chapter unravels the intricacies of ensuring fairness in lending, one of the most critical aspects of responsible financial services. Our journey will not only explore the significance of MRM but will also walk you through a compelling case study: the implementation of a Fair Lending solution leveraging ModelOps.

As model developers, let us champion this transformative journey where we bridge the gap between financial innovation and ethical

responsibility, all while championing fairness and agility in lending decisions.

Structure

In this chapter, we will learn the following topics:

- Model risk management for fair banking
- Background
- Case study: Fair lending model lifecycle implementation - concept to inference
 - Fair lending model lifecycle
 - Data
 - Model operations tool primer
 - Architecting **the model lifecycle using ModelOps**
- Challenges and assumptions
- Future of AI and its practitioners

Objectives

The objective of this chapter is to navigate the multifaceted landscape of Fair Lending and unearth how industry professionals and organizations may take proactive steps to ensure that lending decisions are not just profitable but also unbiased and quick. The case study within this chapter will illuminate how a forward-thinking financial institution should harness ModelOps to deploy a Fair Lending solution that not only adheres to regulatory requirements but also champions fairness in regard to financial services.

Specifically, readers will be able to build, deploy, explain, and monitor credit decisions for a credit risk model built on a public dataset. This solution is closely aligned to serve the various regulatory requirements on fair lending in the USA. We will learn how MRM contributes to fair lending practices by ensuring data quality assessment, bias detection, regular audits, reporting, transparency, explainability, and overall model lifecycle management from development to retirement.

Model risk management for fair banking

Organizations with matured MRM processes are committed to addressing fairness issues as they arise and continuously refining their lending models to enhance fairness. A matured MRM process can be defined as one with faster and more comprehensive model validation, deployment, streamlined compliance processes, and, most importantly, the promotion of equitable model-based decision-making.

We discussed the enablers of these elements of a matured MRM process in [Chapter 12, Understanding Model Risk Management for Data Mining Models](#), but in an introductory manner. Let us study these in the context of fair lending laws that require the active presence of these elements within a financial institution for their fair lending program to be termed responsible.

Background

Understanding fair lending and its regulatory requirements:

Fair lending is one of the highest-ranking risk areas for financial institutions such as banks, credit unions, mortgage companies, mortgage servicers, indirect auto lenders, and other third parties involved in lending credit to a diverse set of customers. Interestingly, fair lending compliance risk exists at every stage of the lending process, including marketing, application, underwriting, valuation, pricing, servicing, and collection. To navigate this complex compliance landscape and avoid the devastating consequences to financial institutions, a comprehensive framework for fair lending compliance is needed. While there are many qualitative methods such as setting up of policy and procedures, training, and executive oversight to address the inherent risks in the process, this field is not left untouched by data and algorithmic interventions.

To that point, the **Consumer Financial Protection Bureau (CFPB)** confirmed on May 26, 2022, that US federal anti-discrimination law requires companies to explain to applicants the specific reasons for denying an application for credit or taking other adverse actions, even if the creditor is relying on credit models using complex algorithms. The reasoning behind such notices is that these models are inherently considered **black-box** and that makes the rationale behind the models' outputs unknown to the model's business users and sometimes even the model's developers.

Let us look at a few regulations centered around fair lending in the United States, such as **Regulation B (ECOA)**, **Regulation C (HMDA)**, and the **Community Reinvestment Act (CRA)**, which are highly driven by data and algorithms.

Compliance Monitoring: Regulation B relates to the **Equal Credit Opportunity Act (ECOA)** and prohibits lenders from discriminating against race, color, religion, national origin, sex, marital status, and age of the credit applicants. This regulation additionally ensures the collection and maintenance of information related to credit applications, loan approvals, and denials for internal compliance monitoring while providing equal access to any kind of credit for all applicants. This drives lenders to maintain early warning systems that describe the health of internal compliance systems related to various kinds of lending activities:

- **Regulatory reporting:** Regulation C specifically relates to the **Home Mortgage Disclosure Act (HMDA)**, which requires lenders to report data on mortgage lending activity to the US government. This data is used to identify potential fair lending violations and to monitor lenders for compliance with fair lending laws. This data is available free of charge for lenders to study and benchmark their lending activities with peers. We have used this data in [Chapter 2, Basic Statistics And Exploratory Data Analysis](#), for EDA and will be using this again in this chapter for training a credit risk model. HMDA requirements drive lenders to collect and maintain accurate data on mortgage lending activities.
- **Community focus:** The **CRA** requires banks and other financial institutions to meet the credit needs of their local communities, including low and moderate-income neighborhoods. Banks are evaluated by regulators on their performance in meeting these needs, which is done to prevent exclusionary tactics in real estate. Ranging from racial steering by real estate agents from certain neighborhoods to systematic denial of mortgages, insurance, loans, and other financial services based on location, this regulation prevents discriminatory tactics. Community commitment ensures alignment with fair practices envisioned by laws.

Together, these three regulations enable a comprehensive approach to fair lending by driving internal compliance monitoring, and

regulatory reporting, while ensuring community focus. These three pillars of fair lending need data and analytics for successful implementation. Please note we are not focusing on any qualitative controls here but will list out some common analytical exercises that are conducted as part of the fair lending risk assessment. These can be approached in two ways:

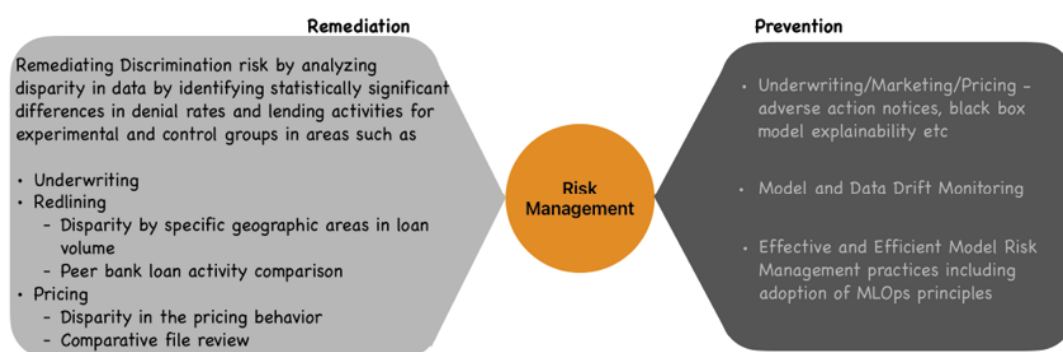


Figure 13.1: Fair lending risk mitigation

Case study: Fair lending model lifecycle implementation - concept to inference

With a background in specific regulations and analytical approaches to fair lending, we will now spend time understanding and implementing a fair lending solution that is efficient, agile, transparent, explainable, and, most importantly can be looked at as both a fair lending risk prevention and remediation tool. This is a prototype solution and does not claim to solve all the needs of a fair lending risk assessment but it will demonstrate the regulatory requirement fulfillment using cutting-edge technology and analytics. These are some of the elements of the growing **regulatory technology (RegTech)** space.

Fair lending model lifecycle

An advanced **Fair Lending Platform (FLP)** illustrates a significant shift from conventional approaches to fair lending risk assessment. Its various features can holistically help address the demands of a diverse set of stakeholders within a financial institution. These stakeholders with different roles and responsibilities across the model lifecycle have several common needs with respect to fair lending

programs that need to be addressed by the infrastructure and process support.

The typical model lifecycle, as shown in [Figure 13.2](#), includes several sub-processes that are highly interdependent and involve activities across multiple teams. There are four broad areas for management to enhance collaboration and hand-offs:

- **Model development:** This typically involves the statistical analysis of historical loss data and subsequent credit risk model development as per business specifications. The model specification includes an algorithm that leverages specific customer attributes to predict the corresponding credit risk parameters such as **probability of default (PD)**, **Loss Given Default (LGD)**, and **Exposure at Default (EAD)**. This is a significant effort and hence requires tracking of all the experimental models for reproducibility and eventual logging of the champion model into a model registry.
- **Independent model risk management:** This team evaluates the process of model development including conceptual soundness, input data, output sensitivity, benchmarking analysis, and logged experiments for developmental evidence before providing approval for the model to be used by underwriters.
- **Model inference:** The MRM-approved model package is converted into a technology solution, which makes it easier for the underwriters/business users to perform model inferencing and assess model outcomes at the individual or batch level. This step of the model lifecycle involves customer-facing decision-making and hence is an important step of the lifecycle. The customer and laws demand an explanation for credit application approval or denial and hence, the model needs to behave like a glass-box explaining its decisions precisely. The MRM is also involved in the model inferencing process by performing ongoing monitoring. This comprises input data drift analysis and model performance degradation. Based on the ongoing monitoring results, The model development team may recalibrate or redevelop the model as appropriate.
- **Early warning or monitoring system:** Despite best efforts, model biases can creep into lending decisions. These could be driven by changes in the nature of input data, policy decisions,

infrequent model overrides, inherent model weaknesses, and so on. These could lead to *disparate impact* on consumers and need to be tracked alongside model performance degradation.

The following figure illustrates an end-to-end fair lending risk assessment model lifecycle:

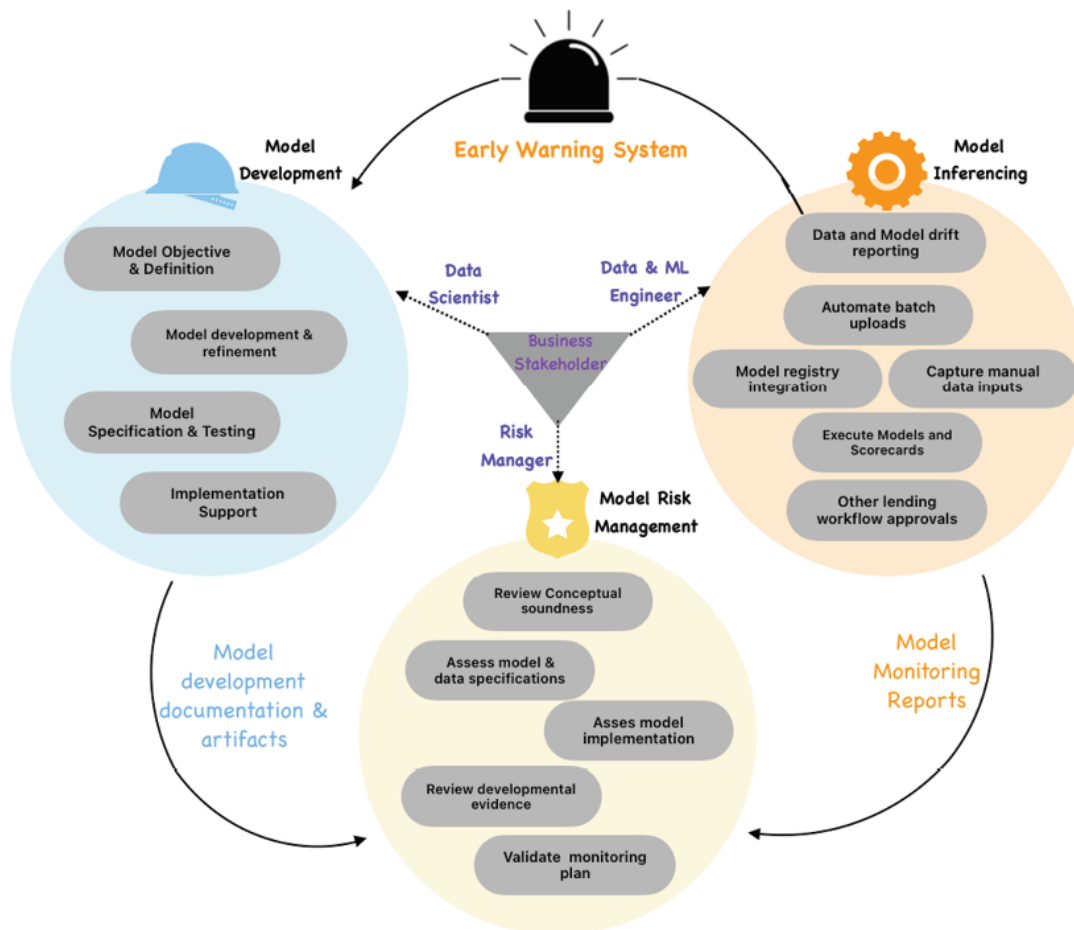


Figure 13.2: Fair lending risk assessment - model lifecycle

The end-to-end process of model development, validation, inferencing, and monitoring of a complex fair lending assessment system takes a long time to stand. On the other hand, customer needs and the macroeconomic environment typically change at a faster rate. This presses the need to significantly compress the credit risk model lifecycle, which is at the heart of the fair lending system. In the current environment, banks are enhancing existing models while also developing new models at a rapid pace by employing a variety of technology solutions ranging from Excel spreadsheets to custom applications and tools.

While there are many technology solutions out there in the market, this chapter will focus on the popular open-source solutions. This will teach the readers to build a comprehensive web-based fair lending assessment solution that covers all aspects of the model lifecycle and enables compliance with the previously discussed three regulations. However, it is important to note the key limitations of any such automated compliance monitoring platform. These limitations such as data privacy and security, data bias, legal expertise, ethical considerations, and resource considerations are discussed in detail in the challenges and assumption section later in the chapter.

While organizations should plan to augment modern technology-driven platforms by addressing the above limitations at a higher level, it is important in today's fast-paced environment for model developers (data scientists) to understand the needs of business and technology operations teams. They should build data mining solutions that are easy to explain, deploy, and monitor. This chapter will enable readers to appreciate the end-to-end **model operations (ModelOps)** through the development of this fair lending risk assessment solution.

Data

Beginning with an understanding of HMDA data used for training a credit risk model, we will evaluate the right features to be used for modeling. The available features are shown in [Figure 13.3](#). However, since age, sex, and race are prohibited factors for making lending decisions, we will drop these features from the raw dataset:

	Age	Sex	Job	Housing	Saving accounts	Checking account	Credit amount	Duration	Purpose
0	67	male	2	own	NaN	little	1169	6	radio/TV
1	22	female	2	own	little	moderate	5951	48	radio/TV
2	49	male	1	own	little	NaN	2096	12	education
3	45	male	2	free	little	little	7882	42	furniture/equipment
4	53	male	2	free	little	little	4870	24	car

Figure 13.3: *HMDA modeling dataset snapshot*

The link to the original dataset can be found as follows:

<https://www.consumerfinance.gov/data-research/hmda/historic-data/>

Please note that we are not spending significant time training any new models in this chapter and will build a basic logistic and random forest classification model trained on HMDA data. The best among these two models will be deployed in the *Credit Risk Assessment* section of the **Fair Lending Risk Assessment (FLRA)** platform. Details on data preparation and model training can be found in the Python notebook.

Model Operations tools primer

This section talks about all the open-source tools that we will leverage to build a self-serve FLRA platform:

- **MLflow:** We have discussed MLflow in [Chapter 3, Digging Into Linear Regression](#), in detail and we know that this popular tool helps track experiments during the model development phase. This has a GUI that provides granular details of all the models built in pursuit of the champion model. We will also use MLflow as a model registry to log our best model version. [Figure 13.4](#) shows the MLflow GUI setup before kickstarting the model training in the Google Colab environment. Colab requires a web URL to access MLflow UI instead of a local host server. This can be achieved by having a ngrok account and key, downloading and setting up ngrok client software, and establishing a secure ngrok connection tunnel to transmit data to the open localhost port. This is done in two steps:
 1. Assign a local system port to MLFlow. In this case, 5000 is assigned.
 2. Create a remote tunnel using ngrok.com to access the port 5000 on the local machine.

This detailed code to execute the above steps is available in the Jupyter Notebook for [Chapter 13, Adopting ModelOps to Manage Model Risk](#).

The following figure shows the Python steps in the colab notebook and the MLFlow screen invoked as a result of these steps:

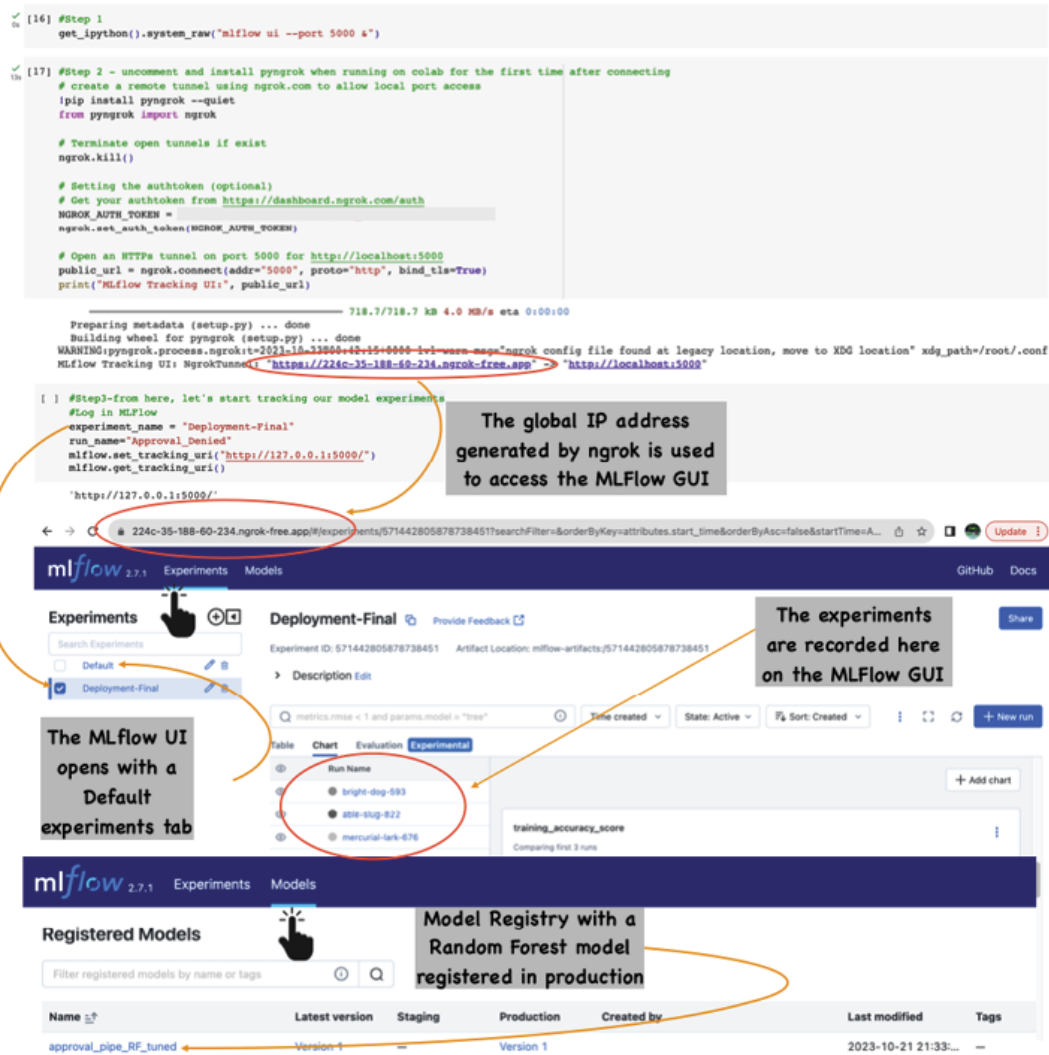


Figure 13.4: MLflow URL, Experiment Tracking, and Model Registry Tab UI

More details on ngrok can be found below link:

<https://www.sitepoint.com/use-ngrok-test-local-site/>

Please note that, ngrok only temporarily grants access by issuing a randomly generated URL. It is best to assume that the app access is only available while the tunnel is open. In our case, it would mean only while the Colab notebook is connected.

- **Evidently AI:** As part of the ongoing monitoring effort, Evidently helps evaluate, test, and monitor data and ML models from model validation to production. It is an open-source Python library for data scientists and ML engineers that works well with both tabular and text data.

It can be used to generate individual test outcomes in the inferencing pipeline when a new batch of production data is received. This would require building a conditional workflow based on the results, for example, to trigger an alert, retrain, or get a report when production data distribution has drifted from the training dataset distribution. Alternatively, dataset distribution comparison from two subsequent periods can also be performed before the latest dataset is sent for inferencing. Additionally, developers can be innovative and use Evidently reports to visually evaluate the data or model performance in instances such as exploratory data analysis, model assessment on the training dataset, diagnosing a model quality decline, or multiple model benchmarking.

Evidently also allows seamless incorporation of an evaluation phase into the data and ML pipeline, captures the results in a JSON and HTML format, and subsequently logs them for in-depth analysis or visualization using **business intelligence (BI)** tools. However, in our case study, we will display the visuals inline in the Python notebook and also leverage the MLflow environment to log our HTML and JSON outputs for model performance and data drift tracking.

Please see the following [Figure 13.5](#) for Evidently UI in a Python notebook:



Figure 13.5: Evidently UI in a Colab Notebook – Sample Model & Data drift reports

The libraries needed for the preceding output are:

#evidently-0.4.4

#install model monitoring package

try:

import evidently

except:

!pip install git+https://github.com/evidentlyai/evidently.git

from evidently import ColumnMapping

from evidently.report import Report

from evidently.metric_preset import DataDriftPreset, TargetDriftPreset, ClassificationPreset

The code needed for generating the model and data drift report is as shown below:

```

#Comparing classification performance between the current and reference
datasets

Classification_perfomance = Report(metrics=[ClassificationPreset()])

Classification_perfomance.run(current_data=current,
reference_data=reference,
column_mapping=column_mapping)

Classification_perfomance.show()

#Comparing Data Drift

column_mapping = ColumnMapping()

column_mapping.numerical_features = numerical_features

data_drift = Report(metrics = [DataDriftPreset()])

data_drift.run(current_data = current,
reference_data = reference,
column_mapping=column_mapping)

data_drift.show()

```

Detailed code is available as part of the Python notebook for [Chapter 13: Adopting ModelOps to Manage Model Risk](#).

- **GitHub:** This is required to store the Streamlit front-end application code, Dockerfile, and other required files in the GitHub repository. We will also use GitHub Actions, which is an automation and **continuous integration/continuous deployment (CI/CD)** service provided by GitHub. This will create a workflow that is a series of steps defined in a YAML file (usually stored in `.github/workflows` in the repository). These steps outline the tasks that need to be automated, such as building the application, running tests, or deploying to a server.

The YAML file will automatically trigger the workflow when changes are pushed to the GitHub main branch. It will log in to the Heroku container registry, build the Docker image using the Dockerfile, and deploy it to the web.

Once the workflow completes successfully, we can access the deployed Streamlit app on the Heroku platform. [Figure 13.6](#) shows the GitHub repository and GitHub Action set up for the project:

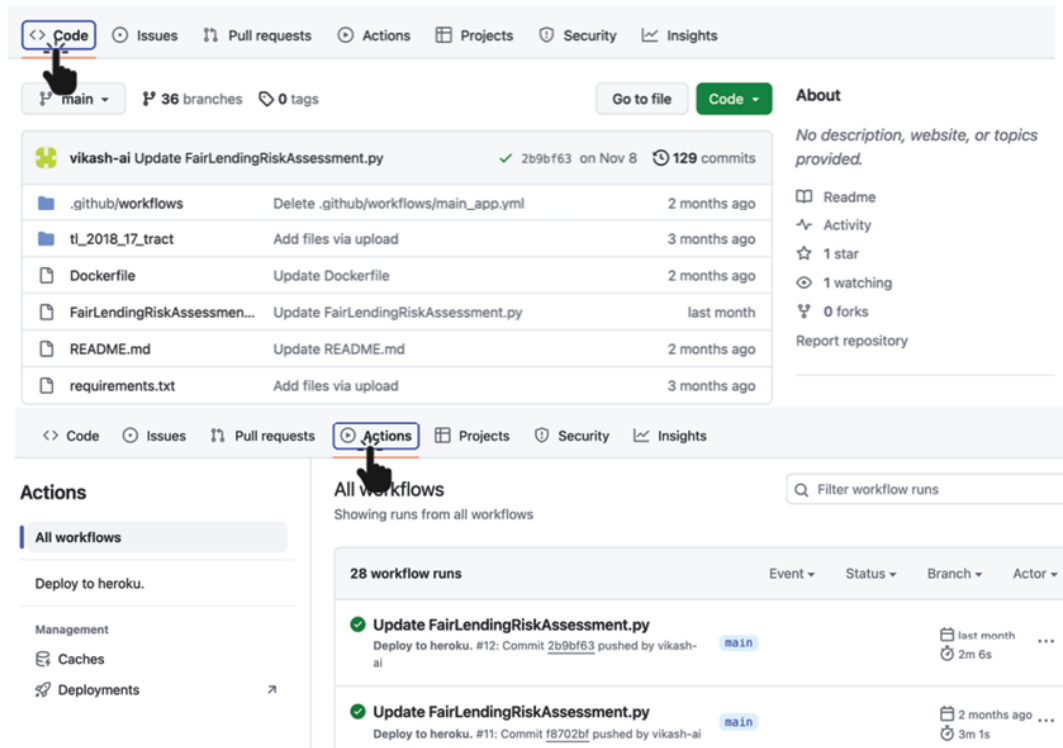
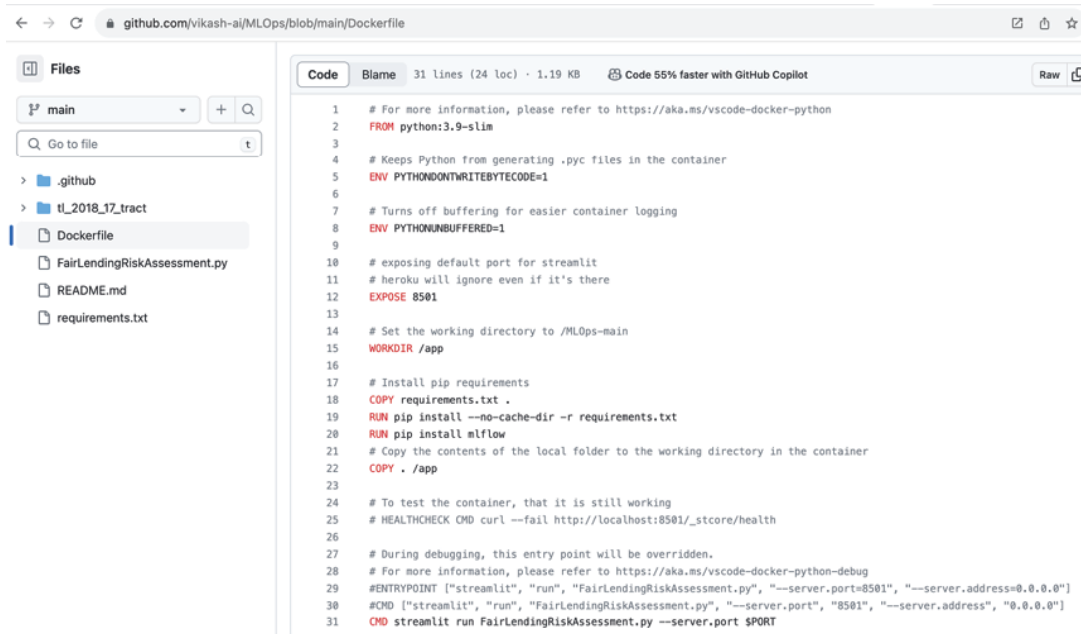


Figure 13.6: GitHub and GitHub Action

- **Docker:** It is a platform for developing, shipping and running applications in container form, enabling consistent and reproducible deployments across different environments. Containers, created using read-only templates called images, run on the Docker platform. They are lightweight, standalone, and executable packages that include everything needed to run a piece of software. This includes the code, runtime, system tools, and libraries, and hence, are an excellent choice for deploying machine learning models, including those stored as *MLflow Models* registry or `.py` files in a GitHub repository. The Dockerfile used to create the image is shown in [Figure 13.7](#):

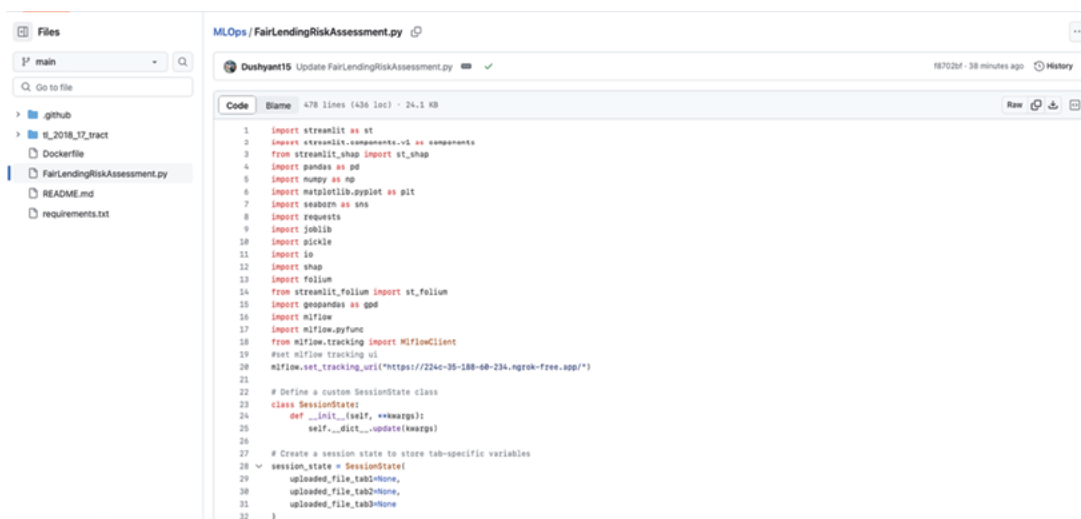


The screenshot shows a GitHub repository for 'vikash-ai/MLOps'. The 'Dockerfile' is selected in the file explorer on the left. The code in the main pane is a Dockerfile for a Python application. It starts with 'FROM python:3.9-slim' and includes instructions to set environment variables like 'PYTHONONWRITEBYTECODE=1' and 'PYTHONUNBUFFERED=1'. It also sets the working directory to '/MLOps-main' and installs pip requirements from 'requirements.txt'. The entrypoint is set to 'streamlit run FairLendingRiskAssessment.py --server.port \$PORT'.

```
1 # For more information, please refer to https://aka.ms/vscode-docker-python
2 FROM python:3.9-slim
3
4 # Keeps Python from generating .pyc files in the container
5 ENV PYTHONONWRITEBYTECODE=1
6
7 # Turns off buffering for easier container logging
8 ENV PYTHONUNBUFFERED=1
9
10 # exposing default port for streamlit
11 # heroku will ignore even if it's there
12 EXPOSE 8501
13
14 # Set the working directory to /MLOps-main
15 WORKDIR /app
16
17 # Install pip requirements
18 COPY requirements.txt .
19 RUN pip install --no-cache-dir -r requirements.txt
20 RUN pip install mlflow
21 # Copy the contents of the local folder to the working directory in the container
22 COPY . /app
23
24 # To test the container, that it is still working
25 # HEALTHCHECK CMD curl --fail http://localhost:8501/_stcore/health
26
27 # During debugging, this entry point will be overridden.
28 # For more information, please refer to https://aka.ms/vscode-docker-python-debug
29 ENTRYPOINT ["streamlit", "run", "FairLendingRiskAssessment.py", "--server.port=8501", "--server.address=0.0.0.0"]
30 CMD ["streamlit", "run", "FairLendingRiskAssessment.py", "--server.port", "8501", "--server.address", "0.0.0.0"]
31 CMD streamlit run FairLendingRiskAssessment.py --server.port $PORT
```

Figure 13.7: Dockerfile used to create the Docker image

- **Streamlit:** It is an open-source Python library for developing and sharing custom web applications for serving machine learning models. We will use Streamlit to develop a GUI that will enable the end-users to interact with the fair lending model and perform other fair lending risk assessment tasks such as batch and individual credit decision-making and disparate impact assessment. The GUI code snippet from the GitHub is shown as follows:



The screenshot shows a GitHub repository for 'vikash-ai/MLOps'. The 'FairLendingRiskAssessment.py' file is selected in the file explorer on the left. The code in the main pane is a Python script that uses Streamlit to create a web application. It imports various libraries including streamlit, pandas, numpy, matplotlib, seaborn, requests, joblib, pickle, shap, folium, and mlflow. It defines a custom SessionState class and creates a session state to store tab-specific variables. The script also includes a function to track MLflow runs.

```
1 import streamlit as st
2 import streamlit.components.v1 as components
3 from streamlit_shap import st_shap
4 import pandas as pd
5 import numpy as np
6 import matplotlib.pyplot as plt
7 import seaborn as sns
8 import requests
9 import joblib
10 import pickle
11 import shap
12 import folium
13 import streamlit_folium as st_folium
14 from streamlit_folium import st_folium
15 import geopandas as gpd
16 import mlflow
17 import mlflow.pyfunc
18 from mlflow.tracking import MlflowClient
19 # Set mlflow tracking uri
20 mlflow.set_tracking_uri("https://224c-35-188-48-234.ngrok-free.app/")
21
22 # Define a custom SessionState class
23 class SessionState:
24     def __init__(self, **kwargs):
25         self.__dict__.update(kwargs)
26
27 # Create a session state to store tab-specific variables
28 session_state = SessionState(
29     uploaded_file_tab1=None,
30     uploaded_file_tab2=None,
31     uploaded_file_tab3=None
32 )
```

Figure 13.8: Application development using Python Streamlit library

- **Heroku:** It is a platform as a service that helps businesses and individuals create and manage high-performance applications. Heroku relies on a Dyno, which is a container encapsulating resources such as memory, CPU, application code, and related dependencies. While this **platform-as-a-service (PaaS)** service has its many limitations, we will use its limited and free features to demonstrate application web hosting. Deploying applications in Docker containers ensures consistent runtime environments, while Heroku provides a PaaS solution that abstracts infrastructure management, allowing enterprises to focus on application development and deployment.

These six open-source or free developer version tools, along with the Python environment are used to build an end-to-end model operations pipeline from development to deployment going into monitoring. These tools are easy to implement and provide cost-effective enterprise access for democratizing data insights and deploying applications across diverse business units.

Architecting the model lifecycle using ModelOps

Let us start this section by understanding how bad coffee and bad models have a lot in common. The bad coffee dispensed from the coffee machine can taste bad for the reasons shown in [Figure 13.9](#):

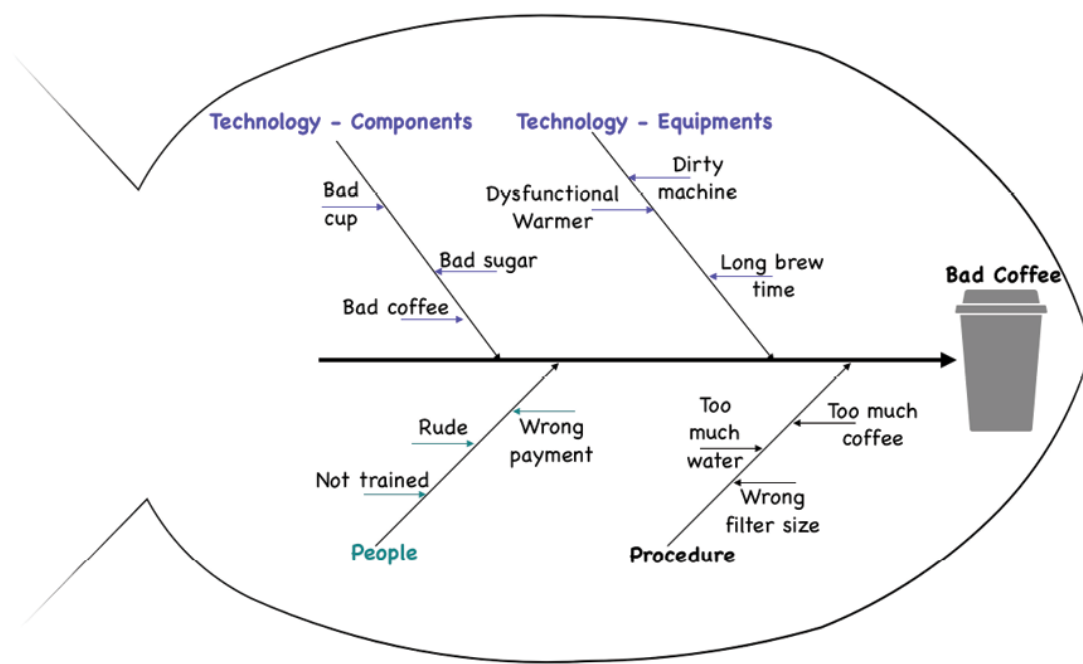


Figure 13.9: Fishbone diagram for bad coffee

The factors that impact a bad outcome in the preceding coffee scenario are:

- **People:** This can be described as having little objective clarity, little training, and bad motives on the part of the humans involved in the process. These are common reasons for having a bad model development objective as well.
- **Procedure:** Secondly, if there is little or no knowledge of risks involved and the presence of a bad mix of various stakeholders while forming the procedures and policies, then this could lead to a bad model outcome, just like a watery coffee.
- **Technology (Components and equipment):** When model techniques don't suit the use case, the framework design is not technically sound, model components don't work seamlessly, no monitoring and missing modelOps can lead to model-related risks. Coffee can suffer from similar risks when materials and pieces of equipment are not of high quality and fail often.

Suppose we use the fishbone diagram to figure out why our models do not work well, we will realize the similarities between coffee brewing and model inferencing. The problem factors of people, process, and technology impact the operational issues for an FLRA solution as well. Of course, these can be remediated by using sound ModelOps principles.

As listed in [Chapter 12, Understanding Model Risk Management for Data Mining Models](#), [Table 12.2](#), ModelOps serves five key requirements of the MRM, which should drive the deployment of a fair and unbiased lending solution as well. These MRM requirements are:

- Data should be accurate and complete
- Documentation should be comprehensive
- Model development should be reproducible and tested
- Model deployment and maintenance need automation and monitoring
- Technology and infrastructure implementation should enable automation and efficiency

[Figure 13.10](#) outlines an FLRA solution architecture built for the aforementioned five MRM requirements using ModelOps principles:

Model development (GUI-1): This is supported by MLflow wherein the model developers can store and version their models for collaborative development and track their experiments in MLflow before registering the champion model in the same environment. GitHub is also used for storing the Streamlit app file. [Figures 13.11](#) and [Figure 13.12](#) show the two GUIs:

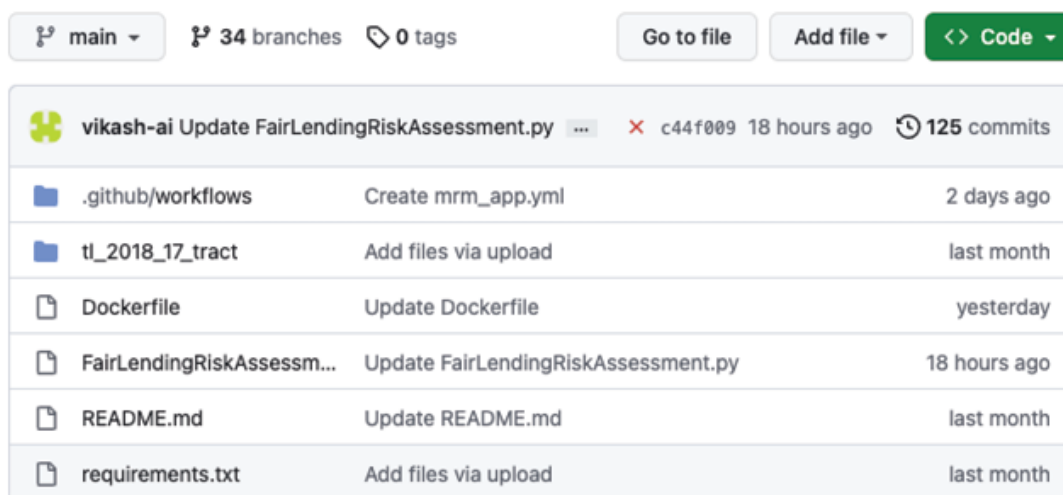


Figure 13.11: Fair Lending Solution– GitHub code storage

The following figure shows the MLFlow **GUI-1**:

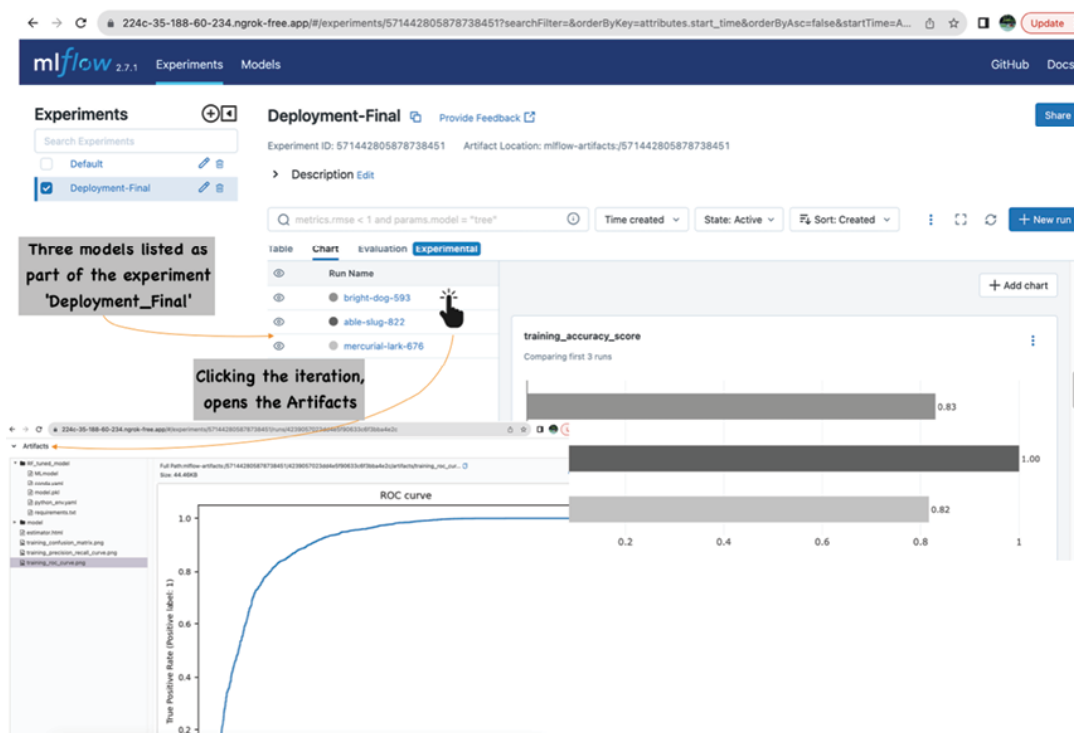


Figure 13.12: Fair Lending Solution- MLflow experiment tracking

Model Deployment (GUI-2): Our use case is to deploy the FLRA Streamlit application on Heroku using Docker and Github Action. This is done in three steps:

1. Assemble a Docker image:
 - a. Create a Dockerfile that defines the Docker image to run the Streamlit web app .py file. This would include defining all the necessary dependencies, such as Python, Streamlit, MLflow, and copying the local git repository content to the app folder in the container before making it the working directory.
 - b. This will also include a CMD step in the container to run the FairLendingRiskAssessment.py Streamlit application on a specified PORT. This is specified in the form of an environmental variable because Heroku uses its own port, which can change from run to run.
2. Build and publish the Docker image:
 - a. Test the Docker image build locally and publish the Docker image to the Docker Hub container registry.
3. Create GitHub Actions Workflow (.yaml file) to connect and publish to Heroku:
 - a. Set up a GitHub Actions workflow that automates the deployment process. This requires configuring the workflow to authenticate with the Heroku container registry and push the existing Docker image to Heroku:

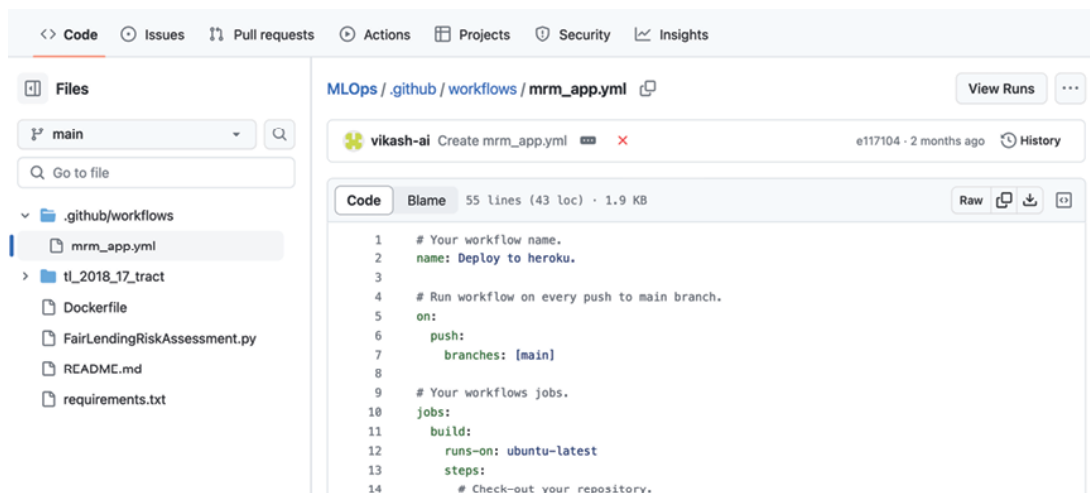


Figure 13.13: Fair Lending Solution- Github Action Workflow

- b. This would require signing up and downloading the Heroku **command line interface (CLI)** before you can push the docker image to Heroku's container registry.
- c. It is important to note that this solution requires the **MLflow** web interface service to be up when the FLRA web app is in service. This is because the Streamlit FLRA app directly references the **approval_pipe_RF_tuned** model registered in the **MLflow** registry for inferencing.

These tasks are typically set up by an ML engineer or a DevOps engineer, but as you can see, the GitHub Action workflow-driven automation makes the job of deploying a new model version consistent and effortless for even data scientists. The workflow to deploy a new model is triggered whenever there is a new **FairLendingRiskAssessment.py** pushed to the main branch. This could happen when the model version that this streamlit .py file is referring to changes in the MLflow model registry.

Model Usage (GUI-3): There are other non-technical stakeholders within a Bank that are responsible for making customer-facing decisions such as loan approvals and denials. They need various tools to make their job successful and avoid trouble while dealing with customer queries. An application dashboard with three tabs as shown in [Figures 13.14, 13.15, and 13.16](#) would be very helpful:

First tab: The following are the features of an individual loan assessment screen

- **Explainability:** Offers model explainability features that help users understand how the model makes lending decisions. For instance, the importance of `applicant_credit_score_type` and `loan_amount` in a loan decision should be clearly explained as per ECOA rules. This can be achieved using feature importance rankings and visualizations like **SHapley Additive exPlanations (SHAP)** plots. The following figure shows the credit risk assessment screen for individual loans:

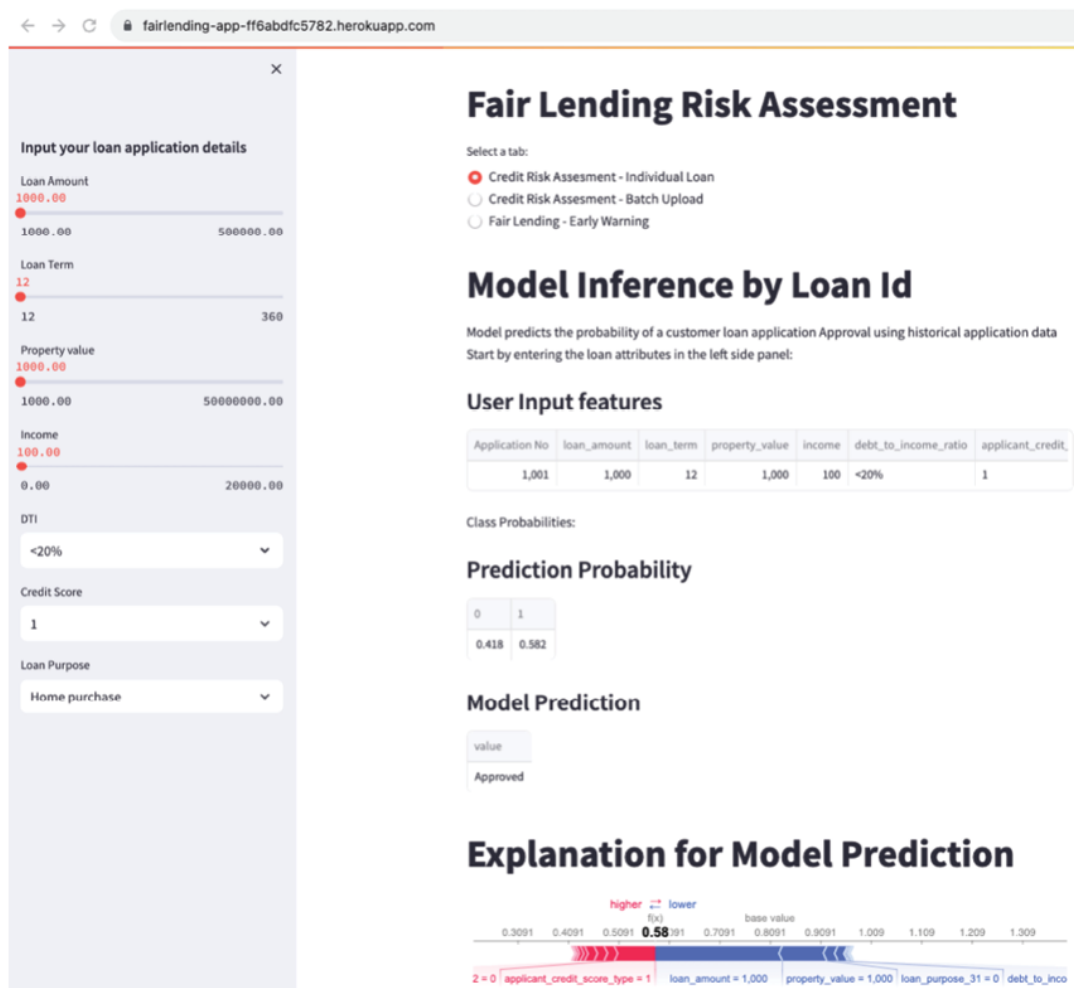


Figure 13.14: Fair Lending Solution– Model Inferencing and Explainability

Force plots, like the one shown in [Figure 13.14](#) (bottom end), show how various features contributed to the model's prediction for a specific loan application that was approved with a probability of default of 0.582. Red-colored features pushed the model score higher, and blue features pushed the score lower. This is perfect for

being able to explain to a non-technical audience exactly how the model arrived at the prediction it did for a specific observation.

Second tab: The following are the features of a batch loan assessment screen

- **Model performance metrics:** Display comprehensive model performance metrics, including accuracy, precision, recall, F1-score, and ROC curves. Users should be able to assess the trade-offs between fairness and model performance.
- **Customization:** Allow users to customize fairness thresholds and parameters according to their specific business and regulatory needs. Offer the flexibility to tailor the risk assessment process.
- **Alerts and notifications:** Implement a notification system that alerts users to potential fairness or compliance violations. Users should be promptly informed of any issues that require attention.

The following figure shows the credit risk assessment screen for a loan batch upload:

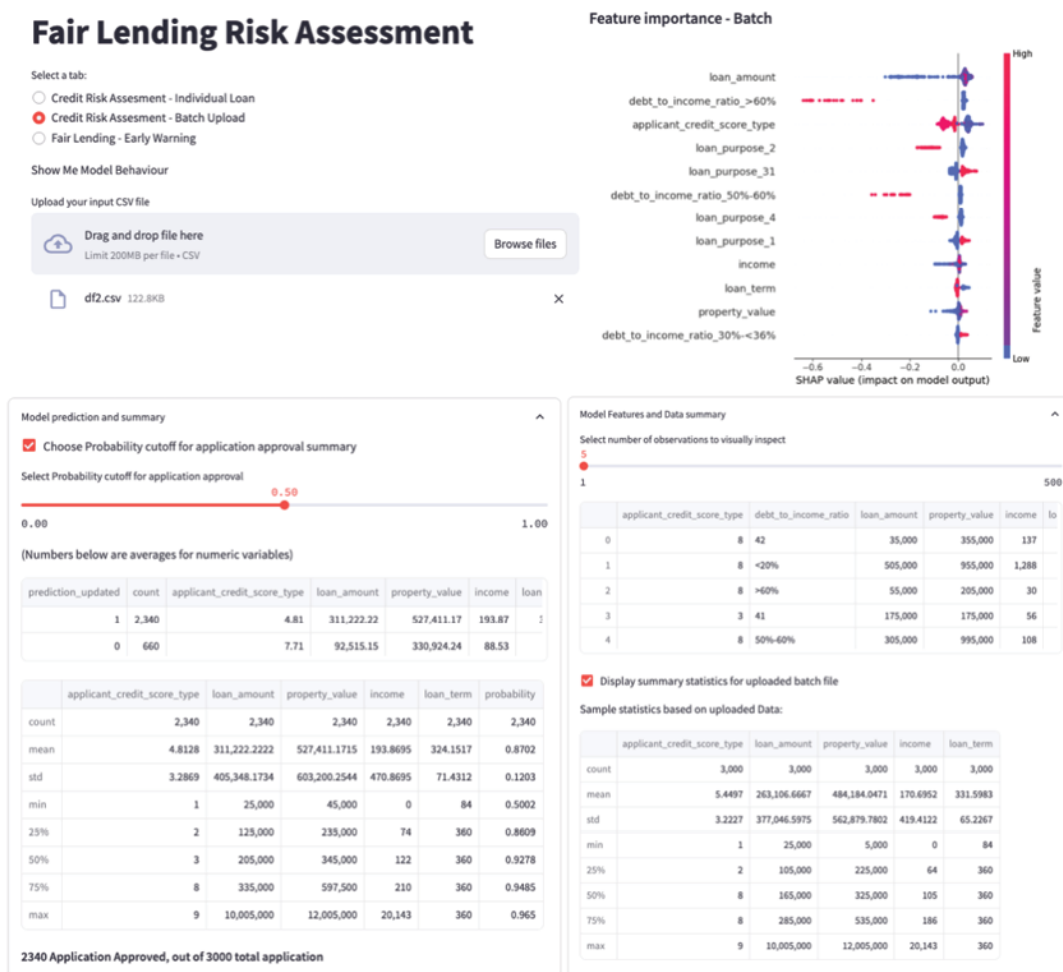


Figure 13.15: Fair Lending Solution – Model Inferencing, Prediction Summary, and Explainability

In [Figure 13.15](#), the top right beeswarm chart shows that the most important feature `loan_amount` is on the top and its lower values (blue dots) have negative SHAP values (the points extending towards the left are increasingly blue), and higher values of `loan_amount` have smaller positive SHAP values (the small cluster extending towards the right are a mix of blue and red). This indicates that higher loan amounts (records indicated by red dots) can sometimes lead to increased log odds of loan approval implying better chances of approval. However, a beeswarm plot is a global feature importance plot that does not always tell the actual relationship between the independent feature and the target variable. For this purpose, it is better to use partial dependency plots.

Third tab: The following are the features of an early warning system screen

- **Dashboard overview:** Provide an initial dashboard that offers a high-level summary of key metrics and insights related to fair lending risk assessment. This could include an overview of lending portfolios, fairness indicators, and compliance status.
- **Data upload and integration:** This allows users to upload and integrate their lending data easily. Support multiple data formats, such as CSV, Excel, or direct database connections. Ensure data security and privacy compliance.
- **Fairness assessment:** Include fairness assessment tools to evaluate and measure potential biases in lending decisions. Provide visualizations and reports showing fairness metrics, such as disparate impact or equal opportunity.
- **Compliance monitoring:** Implement features for monitoring and ensuring compliance with regulatory requirements, such as the **Equal Credit Opportunity Act (ECOA)** or Fair Housing Act. Automatically flag potential compliance issues such as Redlining.
- **Reporting and documentation:** Generate detailed reports and documentation of the fair lending risk assessment process. These reports should be suitable for regulatory compliance and auditing.
- **Data visualization:** Incorporate interactive data visualizations, such as charts, graphs, and heatmaps, to help users gain insights into lending data and fairness assessments.

The following figure shows the screen grab for a fair lending early warning system:

Fair Lending Risk Assessment

Select a tab:

- ☐ Credit Risk Assessment - Individual Loan
- ☐ Credit Risk Assessment - Batch Upload
- ☒ Fair Lending - Early Warning

Upload your input CSV file



Drag and drop file here
Limit 200MB per file • CSV

Browse files

state_IL_lei_B4TYDEB6GKMZO031MB27.csv 4.2MB

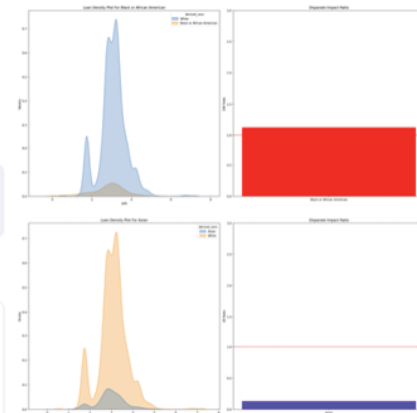
derived_msa-md	derived_race	derived_sex	applicant_age_above_62	loan_amount	interest_ra
16,984	White	Male	No	225,000	3.2
16,984	White	Male	No	365,000	2.8
29,404	White	Male	Yes	105,000	4
16,984	White	Male	Yes	75,000	4.2
16,984	White	Male	No	235,000	2

☒ Show Loan Population

Loan application Population By Prohibited Basis Factor



Loan Density By Prohibited Basis Factor



Redlining By Census tract

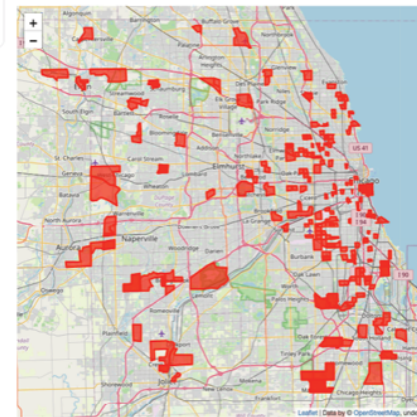


Figure 13.16: Fair Lending Solution- early warning

The application solution in this chapter does not include MRM guidelines related to governance, change control, and seamless integration with upstream and downstream systems in a bank's environment. These measures contribute to the overall stability and reliability of the banking infrastructure in the face of evolving challenges and regulatory requirements.

The following wishlist with regards to integration technology, control and governance can be used to enhance the application:

Technology:

- **Integration with other systems:** Allow integration with external systems, such as databases, reporting tools, or **customer relationship management (CRM)** systems.
- **Scalability:** Ensure the GUI is scalable to handle large datasets and can accommodate future growth in data volume and complexity.

Governance and control:

- **User feedback:** Provide a mechanism for users to provide feedback, report issues, and request enhancements to improve the application continuously.
- **Reporting and documentation:** Generate detailed reports and documentation of the fair lending risk assessment process. These reports should be suitable for regulatory compliance and auditing.
- **User management:** Provide role-based access control to restrict access to sensitive features and data. Support user authentication and authorization to maintain data security.
- **Audit trail:** Maintain an audit trail of all actions and changes made within the application. This helps with transparency and accountability.

As we discuss the three GUIs and their importance for various audiences, one area that interests a lot of stakeholders is Model Maintenance and Monitoring. These stakeholders are senior management, model risk managers, internal auditors, regulators, and IT, who would care to know the ongoing health of the platform. This requires assessing model and data drift to understand how much the model performance has degraded in days after moving to production. Achieving improved transparency through automated reporting and analytics and greater business involvement will drive risk-focused supervision and heightened support for ongoing model monitoring/validation of the systems. [Figure 13.7](#) shows an Evidently AI-based monitoring system that tracks potential model performance degradation driven by changes in the production data compared to the training data:

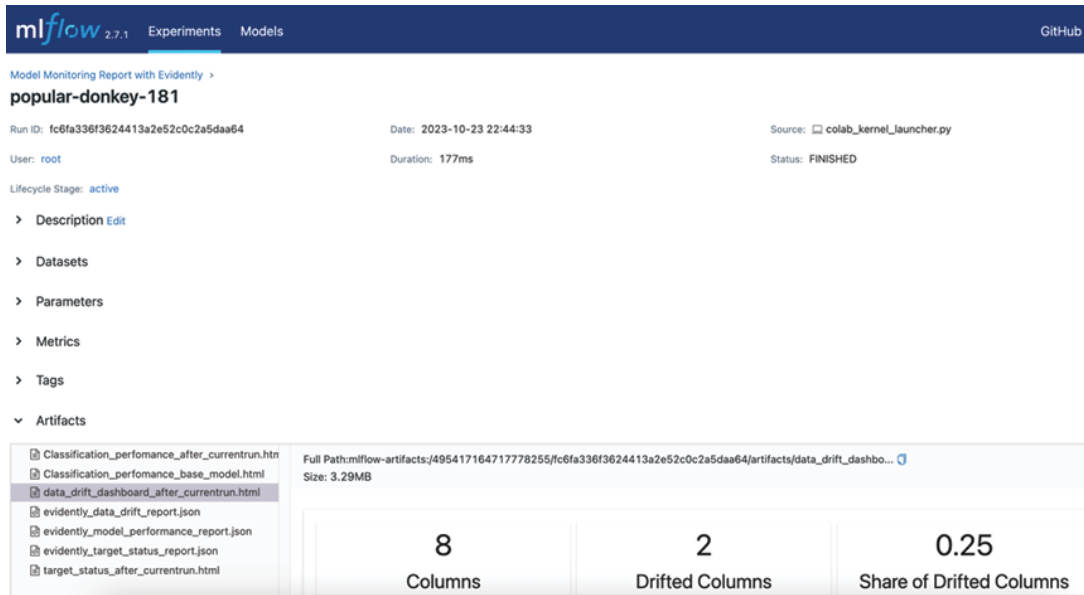


Figure 13.17: Fair Lending Solution – Model Performance Monitoring using Evidently+MLflow

JSON outputs from Evidently can be logged to the FLRA application for non-technical stakeholders as well in GUI-3. However, for application simplicity, we have chosen to log these to MLflow for monitoring by model developers in GUI-1. Detailed code for generating the reports and logging in MLflow GUI is available in the Python notebook.

Additionally, please note that keeping in mind the scope of this book we have limited the application development to model and related components. Data and feature engineering-related modules, such as feature stores, have been assumed to be available for ready use during the inferencing and model development phase. The other reason for this omission is also the small size of our dataset in the current use case. This is a limited-scope use case with a single model, hence demonstrating data governance, maintenance, and engineering concepts in full bloom is excluded.

Given all the architecture components shown in [Figure 13.10](#) that have been detailed so far, it is important to summarize the end-to-end Model lifecycle architecture at this point. There are eight key steps in the process:

1. Build models in Python notebook
2. Log model iterations in MLflow under Experiments and move the best model to the MLflow model registry.

- a. Set up Evidently AI model and data drift reports for monitoring purposes in MLflow
3. Build a Streamlit application that references the model registered in the registry
4. Create a Dockerfile to build a Docker container
5. Store 3 and 4 in a GitHub repository along with a requirement.txt file
6. Create a Heroku account to spin up small applications and download the Heroku CLI
7. Create a GitHub Actions workflow (.yml) to automate the Docker image creation and deployment on the Docker registry in Heroku. This is complete automation of the deployment pipeline every time a change is committed to the main branch of the GitHub source control.
8. Access the application URL when the workflow run is complete in GitHub Actions. This would need the MLflow local server to be up and running for the underlying models to be accessible by the Streamlit app.

Challenges and assumptions

As can be observed from the eight steps outlined in the previous section, addressing challenges related to building a fair lending risk assessment platform requires a multidisciplinary approach, involving data scientists, data and ML engineers, ethicists, compliance experts, legal teams, and domain specialists. It also involves ongoing efforts to ensure that the lending risk assessment solution remains fair and compliant as data and regulations evolve. This is driven by the need to ensure fairness, transparency, and compliance with regulations.

Below are some of the common considerations or challenges as they relate to building the aforementioned platform. These can be broken down into two categories:

Technology challenges:

- **Data privacy and security:** In the case of cloud-based solutions, storing sensitive financial and personal data in the cloud requires robust security measures to protect against

breaches and unauthorized access. Compliance with data privacy regulations such as GDPR is essential.

- **Data quality:** Ensuring the accuracy and completeness of the data used for risk assessment is challenging. Data errors or biases in historical data can lead to unfair lending practices. For instance, historical data used to train machine learning models can contain biases reflecting past discriminatory practices. This bias inherent data bias can perpetuate unfair lending decisions when models trained on biased data are used. Careful feature engineering goes a long way in solving such issues.
- **ModelOps- Build vs. buy:** The market is flooded with tools that solve various challenges related to feature engineering, model experimentation, continuous integration/continuous deployment, monitoring, etc. No one tool serves across the lifecycle except the major cloud player offerings. However, be cautious, these may be more than your use case needs and you do not want to overspend.
- **Scalability and cost management:** The cloud allows for scalability, but ensuring that the solution can handle varying loads efficiently and cost-effectively can be challenging. Cloud costs can escalate rapidly if not properly monitored and controlled.
- **Interoperability:** Integrating with existing systems and databases, especially legacy systems, can be complicated. Ensuring smooth data flows and system compatibility is essential. ModelOps helps in cracking the data flow code.
- **Monitoring and maintenance:** Once the solution is deployed, continuous monitoring and maintenance are necessary to ensure that it remains fair, accurate, and up-to-date.

Regulatory challenges:

- **Fairness and bias:** Addressing bias in algorithms and ensuring fair lending practices is a complex challenge. It's critical to validate models regularly and independently of model developers for bias and fairness checks to take corrective actions on time.

- **Regulatory compliance:** Financial institutions are subject to numerous regulations and need to comply with various regulations, such as the **Equal Credit Opportunity Act (ECOA)** and the Fair Housing Act. Meeting these requirements while deploying advanced AI models can be hard. Additional challenges related to cloud operations can make it harder.
- **Ethical considerations:** Ensuring that lending decisions align with ethical standards and societal values is a critical challenge because fair lending goes beyond legal compliance. Decisions about who gets a loan and on what terms should be transparent and unbiased.
- **Model explainability:** Understanding why a model makes a particular lending decision is essential for transparency and fairness. These issues can be solved to a large extent by leveraging innovative solutions such as SHAP and LIME.
- **Interpretable features:** Interpretability should start with the business context and the data used. This drives informed hypotheses around the chosen features leading to the usage of relevant and interpretable features for risk assessment. Complex features may obscure the reasons behind lending decisions.
- **Customer education:** Ensuring that clients understand the lending process and the role of AI and machine learning can be challenging but is crucial for trust and transparency.

The big news is that these challenges do not deter us from the path of AI adoption as long as we are diligent in adopting a transparent AI implementation roadmap. Let us see why solving current challenges prepares us for the future.

Future of AI and its practitioners

The future of AI has to be scalable, ethical, and responsible. As AI continues to advance, there is a growing need to manage, monitor, and maintain various AI solution components effectively to stay compliant with regulations. This implies that it is not sufficient to have a great bias-variance trade-off alone. Instead, as a data science practitioner, one should know the route from analysis to application to derive the best value from the AI effort.

Until 2019-2020, the focus of AI development was primarily on building and training models leading to the demise of many Python Notebook-based models. However, as AI applications became more complex and widespread and the drive for regulatory compliance surged, it has become evident that building a transparent machine learning model lifecycle is crucial. This is where MLOps steps in, providing a framework for organizations to manage their AI systems effectively.

One of the key reasons why MLOps is so important in the future of AI is its ability to ensure that machine learning models are developed and deployed in a consistent and reproducible manner. This prevents any ensuing model risks by building trust in the AI systems. Additionally, this is particularly important because AI systems now require frequent updates and improvements as they become more complex. This complexity arises primarily from the veracity, volume, velocity, variety, and variability of the data. This makes the models obsolete very quickly.

In this age of **Large language models (LLMs)**, MLOps has evolved to be known as LLMOps enabling organizations to automate various aspects of the model lifecycle, including text data preprocessing, model training, and deployment. This not only saves time and resources for production but also reduces the risk of human error. As larger multi-modal models become prevalent in the future, the need for full-scale automation will become indispensable. Data scientists should be the first to take responsibility for making AI fast and transparent.

Lastly, please note that this chapter touched upon all but one aspect of the model lifecycle, which is the **Feature Store (FS)**. Given the size of our use case with just seven loan approval prediction variables for a single model, we did not create a purpose-built FS that could fully store, share, and manage features for **machine learning (ML)** models. However, as we can see, such a feature repository would provide a huge consistency advantage in organizations that use the same features across multiple models during training, and in production.

The future of AI is here, but unlocking its full potential and subsequent adoption will depend on effective management and deployment of the underlying models. This will lay the foundation for modern data mining.

Conclusion

Our journey through the pages of this book has led us to a remarkable destination, the creation of an efficient and transparent Fair Lending Risk Assessment platform. It is a culmination of knowledge gathered across twelve chapters, spanning data mining techniques, the importance of model explainability and interpretability, and the overarching principles of MRM and ModelOps. With so much focus on AI regulations in today's world, it is critical for data science practitioners and enthusiasts to equip and employ these concepts in daily deliverables for a successful end product.

A fusion of cutting-edge technology with principled model development practices enables this solution for an equitable future in lending. In effect, this platform embodies the idea of efficiency, fairness, accountability, and transparency. Please note that this is not a perfect solution and definitely demands more features, but it is a step in the right direction for reaping benefits from modern-day data mining solutions.

As we pen down the concluding lines of this book, we stand on the precipice of a new era in modern data mining. We have the knowledge, tools, and wisdom to build not just efficient models but models that are equitable, models that empower, and models that serve the greater good.

Further reading

You can find the full Heroku documentation at:

<https://devcenter.heroku.com/articles/container-registry-and-runtime>

Points to remember

- Model transparency is paramount in high-risk applications such as fair lending or fair banking. This means that explainable AI is essential to meet regulatory and ethical requirements.
- Data preprocessing matters since it sets the stage for robust model development.
- Machine learning models have varying strengths and weaknesses in the context of a problem statement. For a use

case such as fair lending risk assessment, evaluate all models, including regression, decision trees, random forests, and neural networks, but focus on models with high explainability.

- Model explainability techniques such as SHAP and LIME demystify model predictions. However, they may be misleading if the goal is to also assess feature importance.
- Another good indicator of feature importance is the feature's ability to reduce prediction error rather than the ability to improve model accuracy. SHAP can be used to assess both.
- MLflow for model lifecycle management is effective for tasks ranging from experimentation to production. It has tracking, packaging, and deployment capabilities.
- Docker for containerization creates consistent, reproducible, and isolated environments for deploying machine learning models.
- Streamlit for interactive apps is used to build interactive web applications to visualize model results and predictions. It bridges the gap between data scientists and business users, given Streamlit is a Python library.
- GitHub Actions for automation automates the deployment process by enabling CI/CD of machine learning models.
- Heroku is a **Platform as a Service (PaaS)** that enables developers to carry out hassle-free application deployment, scaling, and management. It runs applications through virtual containers known as **Dynos**.
- Fair Lending Compliance addresses the importance of adhering to fair lending regulations and how a platform built on ModelOps principles can assist in meeting these compliance requirements, fostering trust with regulatory bodies.

Join our book's Discord space

Join the book's Discord Workspace for Latest updates, Offers, Tech happenings around the world, New Release and Sessions with the Authors:

<https://discord.bpbonline.com>



Index

A

- Accounting Standards Update (ASU) 2016-13 [373](#)
- adaboost [157](#)
- ADAM (Adaptive Moment Estimation) [276](#)
- advanced unsupervised learning [259-262](#)
- adverse impact ratio (AIR) [289](#)
- agglomerative clustering [248](#)
- AI Risk Management Framework (AI RMF) [17](#)
- Analysis of Variance (ANOVA) test [57](#), [58](#)
 - limitation [58](#)
- area under the curve (AUC) [230](#)
- artificial intelligence (AI) [3](#)
- Artificial Neural Networks (ANNs) [265](#), [266](#)
 - anatomy [268](#)
 - background [267](#), [269](#)
 - backpropagation [275](#), [276](#)
 - model [269](#), [270](#)
 - optimal results, achieving [273-275](#)
 - regularization [275](#), [276](#)
- Augmented Dickey-Fuller (ADF) test [64](#)
- autoencoders [239](#), [260](#)
 - Denoising Autoencoders [261](#)
 - temporal autoencoders [260](#)
 - Variational Autoencoders (VAEs) [260](#)

B

- backpropagation through time (BPTT) [303-305](#)
- Bag of Words (BoW) [298](#)
- bank customer portfolio segmentation case study [254-259](#)
- Batch Normalization [275](#), [339](#)
- Bayes theorem [15](#)
- Bernoulli distribution [43](#)
- BERT [298](#)
 - versus GPT [324-327](#)
- bipartite matching loss [345](#)

C

- call-to-action (CTA) buttons [16](#)
- CatBoost [157](#)
- central limit theorem (CLT) [28](#)
- Chat Generative Pre-Trained Transformer (ChatGPT) [301](#)
- Chi-squared (χ^2) test [60-63](#)
- classification algorithms, with twist [180](#)
 - background [181](#)
 - data [181](#), [182](#)
 - loss function [188-190](#)
 - model [183-188](#)
- Classification and Regression Tree (CART) [138](#), [141](#)
- cognitive analytics [16](#)
- Consumer Financial Protection Bureau (CFPB) [300](#), [364](#), [381](#)
- Continuous Bag of Words (CBOW) [299](#)
- continuous probability distribution [43](#)
- continuous variable [25](#)
- convolutional neural networks (CNN) [268](#), [331](#)
 - assumptions [342](#), [343](#)
 - background [333](#)
 - challenges [342](#)
 - convolutional layers [337](#), [338](#)
 - data [333-341](#)
 - flatten layer [338](#)
 - input layer [337](#)
 - model [336](#), [337](#)
 - optimal results, achieving [341](#)
 - pooling layers [338](#)
- covariance [33](#)
- covariance matrix [34](#)
- Cross-Industry Standard Process for Data Mining (CRISP-DM) [4](#), [5](#)
- cross-validation (CV) [122](#)
- current expected credit loss (CECL) [373](#)
 - challenges [374-376](#)
- customer complaint classification with LIME, case study [306-323](#)
- customer propensity prediction case study
 - for term deposit subscription [192-204](#)
- customer relationship management (CRM) systems [3](#)

D

- data augmentation [335](#)
- data mining
 - challenges [360](#), [361](#)
 - evolution [2](#), [3](#)
 - history [3](#)
 - modern data mining [3](#)

- risks [361-363](#)
- statistics [23-25](#)
- data sources
 - cloud-based data storage [9](#)
 - columnar databases [9](#)
 - graph databases [9](#)
 - key-value databases [9](#)
 - NoSQL databases [9](#)
 - object-relational databases [9](#), [10](#)
- data to insights consumption [3](#), [4](#)
- DBSCAN [237](#), [246](#), [247](#)
- decision trees [136](#)
 - assumptions [144](#)
 - background [137](#)
 - challenges [144](#)
 - data [137](#), [138](#)
 - loss function [143](#), [144](#)
 - model [138-142](#)
 - result and interpretation [145-151](#)
- decision trees (DT) [137](#)
- Deep Belief Networks (DBNs) [261](#)
- deep learning
 - for computer vision tasks [332](#)
- Deep Neural Networks (DNNs)
 - assumptions [278](#)
 - challenges [277](#), [278](#)
- degrees of freedom (df) [52](#)
- Denoising Autoencoders [261](#)
- descriptive statistics [25](#), [26](#)
 - graphical and non-graphical exploratory data analysis [26](#)
 - graphical representation of multivariate data [35](#), [36](#)
 - non-graphical and graphical representation of univariate data [27-32](#)
 - non-graphical representation of multivariate data [32-35](#)
- Detection Transformer (DETR) models [345](#)
- dimensionality reduction [209](#), [210](#)
 - assumptions [220](#), [221](#)
 - background [209](#), [210](#)
 - challenges [220](#)
 - data [210](#), [211](#)
 - linear discriminant analysis (LDA) [216-219](#)
 - model [212](#)
 - principal component analysis (PCA) [213-215](#)
 - variance reduction [219](#), [220](#)
- discrete variable [25](#)
- DML (data, model, and loss function) framework [73](#)
- Docker [390](#)

E

- Elastic-Net regression [93](#), [94](#)
- ensemble learning [123](#), [124](#)
- Ensemble learning techniques [151](#)
 - bagging [151](#), [152](#)
 - boosting [152](#)
 - stacking [153](#)
 - stacking method, using [173-175](#)
- Epsilon (ϵ) [247](#)
- Equal Credit Opportunity Act (ECOA) [405](#)
- Evidently AI [387](#)
- Explainable and Interpretable ANN model case study [278-285](#)
 - SHAP and PiML, using [285-291](#)
- Exploratory Data Analysis (EDA) [25](#), [65](#)

F

- Fair Credit Reporting Act (FCRA) [364](#)
- fair lending model lifecycle
 - architecting, with ModelOps [392-394](#)
 - data [385](#)
 - implementation [382-384](#)
 - Model Operations tools [385-388](#)
- Fair Lending Platform (FLP) [383](#)
- Fair Lending Risk Assessment application [394-403](#)
- feature selection [83](#)
- Federal Trade Commission (FTC) [367](#)
- feed-forward network (FFN) [345](#)
 - activation functions [271](#), [272](#)
 - hidden layer [270](#), [271](#)
 - output layer [272](#), [273](#)
- feedforward neural networks (FFNN) [269](#)
- Financial Accounting Standards Board's (FASB) [374](#)
- Financial Institution (FI) [373](#)
- financial technology (FinTech) revolution [1](#)

G

- Gated Recurrent Units (GRU) [303](#)
- Gaussian distribution [44](#)
- Gaussian Mixture Model (GMM) [245](#)
- Generally Accepted Accounting Principles (GAAP) [373](#)
- Generative Adversarial Networks (GANs) [239](#), [261](#)
- Gini gain [142](#)
- Gini impurity [142](#)
- GitHub [389](#)

- gradient boosting (Xgboost) [156-170](#)
 - adaboost [157](#)
 - CatBoost [157](#)
 - comparison of different boosting methods [158](#)
 - Light Gradient Boosting Machine (LightGBM) [157](#)
- gradient descent (GD) [89](#), [90](#), [276](#)
 - Mini Batch Gradient Descent [90](#)
 - Stochastic Gradient Descent (SGD) [90](#)

H

- Heroku [392](#)
- HMDA case study [65](#)
- Home Mortgage Disclosure Act (HMDA) [29](#)
- homoscedasticity [74](#)

I

- ImageNet Large Scale Visual Recognition Challenge (ILSVRC) [343](#)
- inferential statistics [49-53](#)
 - hypothesis testing, with statistical tests [53-55](#)
- Information Gain (IG) [140](#)
- International Organization for Standardization (ISO) [361](#)
- Interquartile Range (IQR) [30](#)

K

- key regulatory frameworks [367](#)
 - AI Risk Management Framework (AI-RMF) [367](#)
 - Basel Committee on Banking Supervision (BCBS) 239 [368](#)
 - Consumer Financial Protection Bureau (CFPB) [367](#)
 - Federal Reserve SR 11-7 [368](#)
 - General Data Protection Regulation (GDPR) [367](#)
- K-fold cross-validation [122](#), [123](#)
- k-nearest Neighbors (KNN) [179-188](#)
 - assumptions [190](#), [191](#)
 - challenges [190](#), [191](#)
- Knowledge Discovery in Databases (KDD) [4](#)

L

- language modeling [296](#), [297](#)
 - background [297](#)
 - model [301](#)
 - spoken languages to modeling datasets [298](#), [299](#)
- Large Language Models (LLMs) [361](#)
- Lasso regression [91](#)
- Leave-one-out CV (LOOCV) [123](#)

- Light Gradient Boosting Machine (LightGBM) [157](#)
- Linear Discriminant Analysis (LDA) [207](#)
- linear regression [67](#), [68](#)
 - assumptions, checking [81](#), [82](#)
 - background [69](#)
 - challenges and assumptions [73](#), [74](#)
 - correlation [80](#), [81](#)
 - dataset description [74-76](#)
 - detailed EDA [74](#)
 - execution and results [83-85](#)
 - feature selection [83](#)
 - outlier analysis [77-79](#)
 - result interpretation [85-88](#)
 - simple linear regression [69-73](#)
 - value treatment, missing [76](#), [77](#)
- loan repayment likelihood prediction
 - case study [126-131](#)
- loan repayment prediction case study
 - with logistic regression, PCA, and LDA [221-233](#)
- Local Interpretable Model-Agnostic Explanations (LIME) [308](#), [321](#)
- logistic regression [105-107](#)
 - assumptions [114](#)
 - background [107](#), [108](#)
 - challenges [114](#)
 - data [109](#), [110](#)
 - loss function [112](#), [113](#)
 - probabilities, estimating [110-112](#)
 - result and interpretation [114-116](#)
- Long-short Term Memory (LSTM) networks [295](#), [303](#), [304](#)
 - cell state [305](#)
 - forget gate [304](#)
 - input gate [304](#)
 - output gate [305](#)
- loss function [112](#)

M

- machine learning (ML) [3](#)
- machine learning process [11-13](#)
 - biases and learning shortfalls [13](#), [14](#)
 - classification [13](#)
 - data size and sample, impacting learning [15](#)
 - learning accuracy and balancing trade-offs, measuring [14](#)
 - regression [13](#)
- machines
 - data, leveraging to build models [10](#), [11](#)

- Maximum Likelihood Estimation (MLE) [114](#)
- Mean Absolute Percentage Error (MAPE) [87](#)
- Mean Squared Error (MSE) [71](#)
- Mini Batch Gradient Descent [90](#)
- MLflow [94](#), [385](#), [386](#)
 - case study [95-101](#)
 - experiment tracking [95](#)
- MLFlow GUI-1 [395](#)
- model development process [125](#), [126](#)
- model explainability [117](#)
- model generalization [122](#)
 - ensemble learning [123](#), [124](#)
 - K-fold cross-validation [122](#), [123](#)
- model interpretability [116](#)
- model lifecycle processes [124](#), [125](#)
- Model Operations (ModelOps) [17](#), [369](#), [370](#)
 - facilitating MRM [372](#)
 - product first, versus model first mindset [371](#)
- Model Operations tools
 - Docker [390](#)
 - Evidently AI [387](#)
 - GitHub [389](#)
 - Heroku [392](#)
 - Streamlit [391](#)
- Model Risk Management (MRM) [364-367](#)
 - challenges [404](#)
 - considerations [404](#), [405](#)
 - for fair banking [380-382](#)
 - key regulatory frameworks [367](#)
 - pillars [368](#), [369](#)
 - principles [17](#)
- model risks
 - occurring [364](#)
- modern data mining [3](#)
 - challenges [17](#), [18](#)
- multicollinearity (MC) [74](#)
- multi-layer perceptrons (MLP) [270](#)
- Mutual Information (MI) [127](#)

N

- National Institute of Standards and Technology (NIST) [17](#)
- natural language processing (NLP) [295](#), [345](#)
- Neocognitron [333](#)
- non-parametric [12](#)
- normal distribution [44-48](#)

O

- Online Transactions Processing (OLTP) databases [9](#)
- optical character recognition (OCR) [351](#)
- optimization algorithm [88](#), [89](#)
 - gradient descent (GD) [89](#), [90](#)
- Ordinary Least Square (OLS) [72](#)
- Ordinary Linear Regression (OLR) [108](#)
- out-of-bag (OOB) evaluation [156](#)

P

- parameter [11](#)
- Partial Dependence Plot (PDP) [172](#), [288](#)
- pattern recognition [5](#), [6](#)
 - data [8](#), [9](#)
 - human learning process [6](#), [7](#)
 - human learning process, and mental models [7](#), [8](#)
- PDF document parser case study [344-355](#)
- performance metrics [117-121](#)
- platform-as-a-service (PaaS) service [392](#)
- primer [259](#)
- Principal Component Analysis (PCA) [207](#)
- probability distribution [38-43](#)
 - continuous probability distribution [43](#)
 - normal distribution [44](#)
- Probability Mass Function (PMF) [39](#)
- probability theory [36-38](#)
- ProfileReport function [192](#)
- Python 3.x
 - setting up [22](#)

Q

- qualitative variable [25](#)
- quantitative variable [25](#)

R

- Radial Basis Function (RBF) Kernel [185](#)
- random forest (RF) [154-156](#)
- Rectified Linear Unit (ReLU) activation function [272](#)
- recurrent neural networks (RNNs) [268-303](#)
 - assumptions [306](#)
 - challenges [306](#)
 - quest, for best model [305](#), [306](#)
- regression [67](#)
- regression analysis [69](#)

- regularization [90](#), [91](#)
 - Elastic-Net regression [93](#), [94](#)
 - Lasso regression [91](#)
 - ridge regression [92](#), [93](#)
- regulatory requirement fulfillment
 - MRM and ModelOps, using [373](#)
- regulatory technology (RegTech) [382](#)
- residuals squared (RSS) [86](#)
- return on investment (ROI) [18](#)
- ridge regression [92](#), [93](#)
- risk [361](#)
- RMSprop [277](#)

S

- Self-Organizing Maps (SOMs) [239](#), [261](#)
- Shapely Additive Explanations (SHAP) [170-173](#)
 - versus Scikit-learn [170](#)
- Shapiro-Wilk Test [55](#)
- squared (TSS) [86](#)
- standard distribution (SD) [29](#)
- Stochastic Gradient Descent (SGD) [90](#), [277](#)
- Streamlit [391](#)
- Support Vector Machines (SVM) [179-186](#)
 - assumptions [190](#), [191](#)
 - challenges [190](#), [191](#)
- SweetViz [109](#)
- Synthetic Minority Over-sampling Technique (SMOTE) [131](#)

T

- Table Transformer (TATR) model [344](#)
- t-distributed Stochastic Neighbor Embedding (t-SNE) [207](#)
- temporal autoencoders [260](#)
- time series [63](#)
- time series data [63-65](#)
- transfer learning [336](#)
- t-test [55](#)
 - independent t-test [56](#)
 - one sample t-test [57](#)
 - paired t-test [55](#)

U

- Universal Function Approximator (UFA) [271](#)
- unsupervised learning [239-241](#)
 - assumptions [253](#)

- challenges [253](#)
- cluster profiling [243](#)
- clusters, building [243](#)
- data [241-243](#)
- DBSCAN [246](#), [247](#)
- hierarchical clustering [247-249](#)
- K-means clustering [244-246](#)
- optimal number of clusters, achieving [249-252](#)

V

- Variational Autoencoders (VAEs) [260](#)

W

- Word2Vec [299](#), [307](#)

Z

- z-test [57](#)